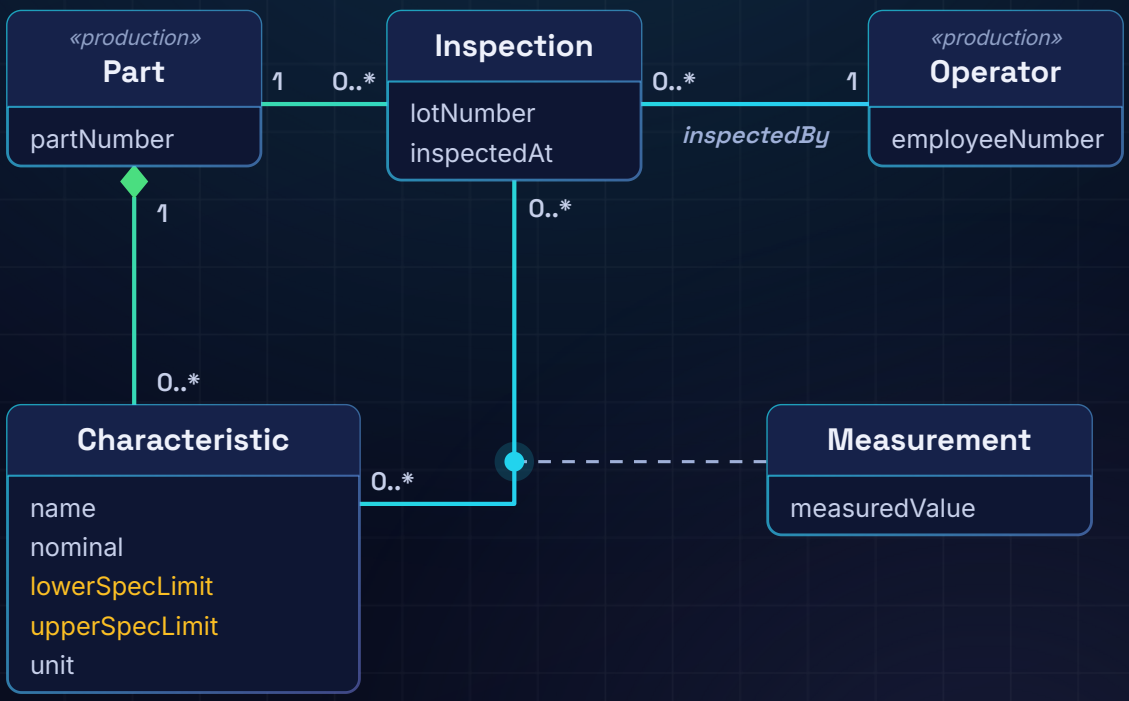
OPEN-TEXT EDITION

Model it, build it, query it, use it

Database Concepts

for **Industrial Engineers**, with **SQL** and **PostgreSQL**



Manuel D. Rossetti

University of Arkansas

Open Educational Resource

Free & open source

Database Concepts for Industrial Engineers

Manuel D. Rossetti

2026-10-30

Table of contents

Preface **1**
Approach 1
How this book is organized 2
Software 3
How this book was written 4
Acknowledgments 6
License 6

About the Author **7**

1. Databases in Industrial Engineering **8**
1.1. From files and spreadsheets to databases 9
1.2. Data in industrial systems 12
1.3. Database systems and their users 14
1.4. From a model to a database 16
1.5. The databases in this book 17
1.6. Summary 18
1.7. Exercises 19

I. Reading and Querying a Database **23**

2. The Relational Model **24**
2.1. Tables, rows, and domains 26
2.1.1. The properties of a relation 28
2.2. Keys 35

2.3. Integrity rules 40
2.4. Writing down a schema 44
2.5. A database describes itself 51
2.6. Exploring a database: DataGrip, DBeaver, psql, and pgAdmin 55
2.7. Other ways to organize data 61
2.8. Summary 64
2.9. Exercises 66

3. Querying Tables **72**
3.1. The SELECT statement 73
3.1.1. The shape of a query 73
3.1.2. Choosing columns from one table 76
3.2. Filtering rows 80
3.3. NULL and three-valued logic 87
3.4. Sorting and limiting results 93
3.5. Expressions and built-in functions 98
3.6. Inner joins 103
3.7. Outer joins 111
3.8. Subqueries 118
3.9. Set operations 129
3.10. Common table expressions 134
3.11. Summary 137
3.12. Exercises 139

4. Aggregation and Analytical Queries **148**
4.1. The shape of an analytical query 149
4.2. Aggregate functions and grouping 152
4.3. Filtering groups 159
4.4. CASE expressions 163
4.5. Window functions 172
4.5.1. How a window function sees the rows 174
4.5.2. Partitions 176
4.5.3. Order and the default frame 179
4.5.4. Explicit frames 184

4.5.5. Functions that ignore the frame 190
4.5.6. When a window query takes two steps 193
4.6. Recursive queries 201
4.7. Analytical queries on operational data 210
4.8. Summary 218
4.9. Exercises 220

II. Designing and Building a Database 228

5. Class Modeling with UML 229
5.1. What a class model adds 230
5.2. Where a class model starts 232
5.3. Classes and attributes 236
5.3.1. Attributes with parts 239
5.4. Associations and multiplicity 241
5.5. Association classes and qualified associations 245
5.5.1. Ternary associations 257
5.6. Aggregation and composition 259
5.7. Generalization 264
5.8. Building a class model 267
5.9. Reviewing a class model 274
5.10. Reading other notations 278
5.11. Summary 281
5.12. Exercises 283

6. From Class Models to Tables 306
6.1. The mapping process 307
6.2. Mapping classes 310
6.3. Mapping associations 313
6.3.1. One-to-many associations 313
6.3.2. One-to-one associations 315
6.3.3. Choosing the side of a one-to-one association 317
6.3.4. Many-to-many associations and association classes . 323

6.3.5. Recursive associations . . . 325
6.3.6. Compositions and aggregations . . . 334
6.3.7. Ternary associations . . . 336
6.4. Mapping qualified associations . . . 341
6.5. Mapping generalization . . . 344
6.5.1. A table for each class . . . 344
6.5.2. A table for each subclass only . . . 348
6.5.3. One table for the whole hierarchy . . . 349
6.6. Choosing and naming keys . . . 351
6.7. Summary . . . 357
6.8. Exercises . . . 359

7. Normalization **379**
7.1. The normalization procedure . . . 380
7.2. Anomalies in a poorly designed table . . . 381
7.3. Functional dependencies . . . 386
7.4. Normal forms . . . 389
7.5. Decomposition . . . 393
7.5.1. Lossless decomposition . . . 397
7.5.2. Preserving dependencies . . . 401
7.6. Normalization and class modeling . . . 408
7.7. Multivalued dependencies and fourth normal form . . . 410
7.8. Summary . . . 413
7.9. Exercises . . . 415

8. Defining and Populating Tables **419**
8.1. From a design to tables . . . 420
8.2. Data types . . . 422
8.3. Creating tables and constraints . . . 427
8.4. Inserting, updating, and deleting rows . . . 434
8.5. Loading data from files . . . 446
8.6. Changing a table's definition . . . 451
8.7. Summary . . . 455
8.8. Exercises . . . 456

9. Views, Transactions, and Concurrency **462**
9.1. Views . . . 463
9.2. Transactions and the ACID properties . . . 468
9.3. Concurrency and isolation levels . . . 474
9.4. Security and privileges . . . 482
9.5. Summary . . . 486
9.6. Exercises . . . 487

III. Databases in Practice **490**

10. Programming with Databases **491**
10.1. A program's view of a database . . . 492
10.2. Parameterized queries and SQL injection . . . 495
10.3. CRUD and the data access object . . . 498
10.3.1. The data access object pattern . . . 499
10.3.2. The data access object for inspections . . . 501
10.4. The KSL database utilities . . . 512
10.5. Importing and exporting data . . . 519
10.6. Libraries that write the SQL . . . 522
10.7. Summary . . . 528
10.8. Exercises . . . 529

11. Data Warehousing and Decision Support **533**
11.1. Operational and analytical processing . . . 534
11.2. Dimensional modeling . . . 538
11.3. Extract, transform, and load . . . 546
11.4. Slowly changing dimensions . . . 553
11.5. Multidimensional queries . . . 559
11.6. Summary . . . 568
11.7. Exercises . . . 569

12. Databases for Simulation and Analytics **574**
12.1. Input data for models . . . 575

12.2. Simulation output databases 581
12.3. Analyzing experiments with SQL 590
12.3.1. A confidence interval computed in SQL 592
12.3.2. Comparing scenarios by pairing replications 594
12.4. Designing a database for experimental work 600
12.4.1. The common shape 600
12.4.2. Long and wide tables 601
12.4.3. The design decisions 611
12.4.4. Quick tables from data classes 612
12.5. Summary 614
12.6. Exercises 615

Appendices **624**

A. Installing PostgreSQL and Its Tools **624**
A.1. Installing the PostgreSQL server 624
A.2. Connecting with DataGrip 626
A.3. Connecting with DBeaver 627
A.4. psql and pgAdmin 628
A.5. Creating the book's databases 629
A.6. Loading public sample databases 630

B. Kotlin and KSL Setup **633**
B.1. Installing a JDK and IntelliJ IDEA 633
B.2. Opening the companion code project 634
B.3. Configuring the database connection 634
B.4. Running your first program 635

C. The Book's Databases **638**
C.1. HG Industries 638
C.2. Loading the database 639
C.3. Naming and keys 642
C.4. The distribution schema 643

C.5. The production schema 646
C.6. The quality schema 650
C.7. The year of history 652
C.8. The sales star schema 653
C.9. Rows in each table 655
C.10. Millbrook Transit 656
C.11. Public sample databases for further practice 661

D. Translating Between Crow's Foot and UML 664
D.1. Why two notations 664
D.2. Entities and classes 665
D.3. Attributes and keys 665
D.4. Relationships and multiplicity 667
D.5. Associative entities and association classes 668
D.6. Identifying relationships and composition 670
D.7. Subtypes and generalization 672
D.8. What does not translate 674
D.9. Translating a model, step by step 675
D.10. Exercises 678

E. Tools for Drawing UML and ER Diagrams 686
E.1. Two jobs, two kinds of tool 686
E.2. Diagrams as text: PlantUML and Graphviz 687
E.3. Diagrams from a database: DataGrip, DBeaver, and pgAdmin 694
E.4. Drawing tools with a canvas 695
E.5. Choosing a tool 697
E.6. Exercises 697

F. SQL Quick Reference 699
F.1. Defining tables 699
F.2. Changing data 702
F.3. Queries 704
F.4. PostgreSQL data types and functions 709

G. Embedded Databases: SQLite and DuckDB **713**
G.1. What an embedded database is . . . 713
G.2. When to use one . . . 714
G.3. SQLite . . . 714
G.4. DuckDB . . . 719
G.4.1. The stock tables in DuckDB . . . 719
G.4.2. Analysis straight from files . . . 722
G.5. Embedded databases from Kotlin and the KSL . . . 725
G.6. Choosing a database . . . 727
G.7. Exercises . . . 728

H. NoSQL and Other Data Stores **731**
H.1. Why NoSQL databases exist . . . 732
H.2. The four families . . . 732
H.2.1. Time-series databases and plant historians . . . 734
H.3. The trade-offs . . . 735
H.3.1. Schema on write and schema on read . . . 735
H.3.2. Nesting is denormalization . . . 736
H.3.3. Consistency across copies . . . 736
H.3.4. Questions nobody planned for . . . 737
H.4. Documents in PostgreSQL . . . 737
H.5. Choosing . . . 745
H.6. Exercises . . . 746

References **749**

Preface

This book is intended as a textbook for an undergraduate course on database concepts for industrial engineering students. It is written for engineers who will design, query, and depend on databases in their work on production, inventory, logistics, quality, and service systems. It is not a computer science text: it does not cover the theory and internals of database management systems, such as relational algebra, storage structures, or query optimization.

The book assumes no earlier course on databases. Its first eight chapters need only comfort with a computer and the patience to check an answer by hand. Two later chapters use more: Chapter 10 assumes a first course in programming, in any language, and introduces the Kotlin it uses as it goes; and Chapter 12 assumes a first course in probability and statistics, and is most useful to a reader who has met, or is meeting, discrete-event simulation. Everything else, from SQL to data modeling, is taught from the beginning.

Approach

I have taught databases to industrial engineering students at the University of Arkansas for thirty years, in the course now numbered INEG 38303 and in its predecessors. Over that time the question the course answers has not changed, although the tools have:

> *How should an engineer keep, find, and trust the data a system runs on?*

Industrial engineers are rarely asked to build a database management system, and seldom asked to administer one. They are asked, constantly, to find out what a system's records say, to design the records a new process or study will need, and to decide whether a number in a report can be believed. This book is organized around those three tasks. It teaches SQL early and thoroughly, because querying is what an engineer does first and most. It teaches design through UML class models, because a model states what a business records in terms its people can check, and then maps the model to tables by explicit rules. It asks, throughout, that every result be checked, by working it out by hand on a few rows before trusting the computer with thousands.

Every example uses the same company, HG Industries, a small maker of hand trucks and garden carts, whose database grows a few tables at a time as the chapters need them. A student who has queried its orders in Chapter 3 meets the same orders again when they are modeled in Chapter 5, normalized in Chapter 7, programmed against in Chapter 10, and summarized in a warehouse in Chapter 11. The setting stays familiar so that the new idea is the only new thing.

How this book is organized

The chapters move from reading and querying a database, to designing and building one, to using databases in engineering work:

- **Reading and querying a database**: why industrial systems need databases (Chapter 1); the relational model, enough to read an existing database and find your way around it (Chapter 2); and SQL, querying one table and then many (Chapter 3), then aggregation and analytical queries (Chapter 4).

- **Designing and building a database**: describing the data a system needs with UML class models (Chapter 5); mapping a class model to tables in the tradition of Blaha and Premerlani (1998) (Chapter 6); normalization (Chapter 7); defining and populating tables (Chapter 8); and views, transactions, and security (Chapter 9).
- **Databases in practice**: programming with databases in Kotlin (Chapter 10), data warehousing for decision support (Chapter 11), and the databases that feed and record simulation and analytics studies (Chapter 12).

Each chapter closes with exercises at several levels: terms and short conceptual questions, true-or-false questions aimed at common misunderstandings, design problems, query problems, and programming problems. Most use HG Industries' database. Some use small settings of their own, a transit agency or a tool crib, and some use public sample databases, so that you practice on schemas that other people designed.

The appendices hold what a reader needs beside the chapters: installing PostgreSQL and its tools (Appendix A) and the Kotlin tools (Appendix B); the book's databases and how to load them (Appendix C); translating between the book's UML and the crow's foot diagrams of other books and tools (Appendix D); software for drawing both kinds of diagram (Appendix E); a quick reference to the SQL the book teaches (Appendix F); and embedded databases, SQLite and DuckDB, which a program or a study can keep in a single file (Appendix G).

Software

PostgreSQL is the database management system used throughout the book. It is free, it runs on every major platform, and its dialect of SQL follows the standard closely, so what you learn here carries over to other systems. Where PostgreSQL departs from the standard, the text says so. You install and run your own server. **DataGrip** and **DBeaver** are the

tools used to connect to it, write queries, and inspect results. PostgreSQL's own tools, **psql** and **pgAdmin**, are introduced briefly so that you recognize them. Appendix A covers installing all of them.

Data models are drawn as UML class diagrams. Much of the database literature, and many tools, use crow's foot entity-relationship diagrams instead; Appendix D shows how to translate between the two notations.

Kotlin and the Kotlin Simulation Library (KSL) carry the programming work, through the KSL's database utilities. Appendix B covers installing the tools and running your first program. The KSL is developed and documented separately; see *Simulation Modeling using the KSL* for a full treatment of the library.

The book's examples use small databases created for it, each designed to be understood in a sitting. The SQL scripts that create and populate them, and the Kotlin examples, are distributed with the book as a companion code project, https://github.com/rossetti/DbConceptsBookCode. For practice at a larger scale, Appendix C describes four public sample databases and a loader that fetches each from its original source; they are not distributed with the book.

How this book was written

This book began as course notes. Over thirty years the course accumulated lecture slides, breakout-session handouts, homework sets, exams, SQL scripts for teaching databases, and a Kotlin starter project, and I drew on it to decide what the book covers and in what order. The raw material was plentiful. What it lacked was a single running example, a single notation, and a consistent standard of proof that a result printed on the page is the result the database actually returns. It also mixed my own questions with questions from the textbooks I taught from over the years. Every archived question was traced to its origin before any idea from it was used, and none

from another book appears here; where an old question tested a concept worth testing, the book tests it with a new question on its own data.

The book is written in Quarto. Each chapter is a Markdown file, and one source renders to both the website and the PDF. The whole book, including its databases, its code, and its build scripts, lives in a version-controlled repository, so every change to a chapter is recorded alongside the data and code that support it.

Every result is computed, not typed. The book's databases are built by SQL scripts, one stage at a time, and each stage is loaded into a PostgreSQL database of its own; every query shown with its result in a chapter runs against its stage's database when the book is rendered, and the result on the page is what PostgreSQL returned. The relational schemas printed in the chapters are read from those databases; the product structure trees and instance diagrams are drawn from them; the output of every Kotlin program is pasted from a real run; and the result of every query in the solutions is produced by running it. The diagrams are written as text and rendered by a script, so a model and its figure cannot drift apart. In addition, a set of scripts checks the cross-references, the citations, the spelling, the consistency of the prose with a written style guide, and the separation of the instructor's solutions from the student's book, on every build. Thus, when the book quotes a number, a query or a program produced it, and a check will fail if it changes.

I wrote this book with the help of an AI assistant, Claude, developed by Anthropic. That is, I used it the way I would use a fast and patient research assistant. It turned thirty years of course materials into draft sections under the style guide derived from my simulation textbook; it designed, with me, and then built the book's databases, and wrote the scripts that load them, render their results, and check the book; it wrote and tested the companion code; it worked the exercise solutions and produced their results from the databases; and it traced the origin of every question in the course archive, so that no other author's work entered the book. For example, the transit exercises of Chapter 5 were drawn from a study I led

for the Transportation Research Board (Rossetti 1999), and each model was redrawn from the report before any exercise was written against it. The decisions were mine: what the book covers and in what order, the design of HG Industries and its database, the conventions for naming and keys, the notations, and what a student should be able to do at the end of each chapter. I reviewed every section, and the assistant's work was held to the same standard as my own, which is that a result is accepted when the database confirms it. Any errors that remain are my responsibility.

Acknowledgments

My first thanks go to the industrial engineering students who have taken this course. Their questions, in class and in the breakout sessions, decided what the notes explained and in what order, long before the notes became a book. The transit exercises draw on work sponsored by the Transportation Research Board's IDEA program and carried out with the cooperation of the Charlottesville Transit System, and I am grateful to both.

License

The online version of this book is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License. The companion code, including the SQL scripts, is licensed under the GNU General Public License, version 3, the same license as the KSL itself. The public sample databases that the exercises use are not part of the book and keep their own licenses, which Section C.11 lists. To cite this book:

> Rossetti, M.D. (2026). *Database Concepts for Industrial Engineers*, On-line and Open Text Edition. Licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

About the Author

Dr. Manuel Rossetti, P.E. Dr. Rossetti is a University Professor of Industrial Engineering at the University of Arkansas. He served as the inaugural Director for the University of Arkansas Data Science program from 2019-2024. In addition, Dr. Rossetti served as the Director for the NSF I/UCRC Center for Excellence in Logistics and Distribution (CELDi) from 2013-2020. Dr. Rossetti has published over 125 journal and conference articles in the areas of simulation, logistics/inventory, and healthcare and has been the PI or Co-PI on funded research projects totaling over 6.4 million dollars. He was selected as a Lilly Teaching Fellow in 1997/98 and was voted Best IE Teacher by IE students in 2007, 2009, and 2017. He won the IE Department Outstanding Teacher Award in 2001-02, 2007-08, and 2010-11. He received the College of Engineering Imhoff Teaching Award in 2012 and was elected an IIE Fellow. In 2013, the UA Alumni Association awarded Dr. Rossetti the Charles and Nadine Baum Faculty Teaching Award, the highest award for teaching at the university. In 2015, Dr. Rossetti served as Program Chair for the Winter Simulation Conference. In 2024, Dr. Rossetti served as the General Chair for the Winter Simulation Conference. He is also the author of the book, Simulation Modeling and Arena, published by John Wiley & Sons.

Dr. Rossetti grew up in Canton, Ohio and is an ardent Cleveland sports fan. He received his Ph.D. and MSIE degrees in Industrial and Systems Engineering from The Ohio State University and his BSIE degree from the University of Cincinnati. He is a registered professional engineer in the State of Arkansas.

1. Databases in Industrial Engineering

Learning objectives

After reading this chapter you should be able to:

- explain why industrial engineers store operational data in a database rather than in files or spreadsheets
- identify the data an industrial system generates: orders, inventory, production, quality, and logistics records
- describe the components of a database system and the role of a database management system
- distinguish modeling a system's data from implementing it in a database
- recognize the databases used throughout this book

Industrial engineers design and improve systems that make, move, and serve: production lines, warehouses, supply chains, hospitals, transit networks. Every such system runs on records. An order is taken, a part is made, a lot is inspected, a shipment leaves, and each event is written down somewhere, by someone, so that the next person can act on it. The quality of those records limits the quality of every decision made from them, from tomorrow's schedule to next year's capacity plan.

This chapter asks why those records belong in a database and what that means for an engineer. It shows what goes wrong when a system's data

lives in spreadsheets, surveys the data that industrial systems generate, describes the parts of a database system and the people who use it, and lays out the path the book follows, from a model of a system's data to a working database. It ends by introducing the small company whose database the rest of the book builds.

1.1. From files and spreadsheets to databases

This section shows the problems a spreadsheet of operational records runs into, and what a database does about them.

HG Industries, the manufacturer of hand trucks and garden carts that this book follows, took its first orders the way many small companies do: in a spreadsheet kept by the sales office. Each row records one line of an order, with everything the office needed to know about it. Table 1.1 shows the log's first seven rows: every line of the company's first three orders of January 2026, and the first line of its fourth.

Table 1.1.: The Sales Office's Order Log (Hypothetical)

Order	Date	Customer	City	State	Product	Description	Qty	Price
SO-5001	2026-01-05	Riverside Hardware	Fayetteville	AR	HT-100	Two-wheel hand truck	10	89.00
SO-5001	2026-01-05	Riverside Hardware	Fayetteville	AR	GC-300	Steel garden cart	4	119.00

Order	Date	Customer	City	State	Product	Description	Qty	Price
SO-5002	2026-01-07	Cedar Creek Home Center	Tulsa	OK	HT-200	Convertible hand truck	6	139.00
SO-5003	2026-01-12	Summit Farm Supply	Springdale	AR	HT-100	Two-wheel hand truck	12	85.00
SO-5003	2026-01-12	Summit Farm Supply	Springdale	AR	HT-200	Convertible hand truck	4	135.00
SO-5003	2026-01-12	Summit Farm Supply	Springdale	AR	GC-300	Steel garden cart	6	115.00
SO-5004	2026-01-15	Riverside Hardware	Fayetteville	AR	HT-100	Two-wheel hand truck	20	89.00

The log is easy to start and easy to read, and for a few dozen orders it works. Look closer at what it stores, though. Riverside Hardware's city and state appear on three of the log's seven rows, and the description of the HT-100 on three. Each row mixes facts about four different things: an

order, a customer, a product, and the quantity and price of one product on one order. Thus, most facts in the log are stored many times, and nothing ties the copies together. Four problems follow, each a request the log handles badly.

- **Changing a fact.** Riverside Hardware moves to Rogers. Every one of its rows must be changed, and a row missed leaves the company in two cities at once, with no way to tell which is right.
- **Recording a fact on its own.** HG Industries designs a new poly garden cart, the GC-400, and wants to record its description and list price before anyone orders it. The log has no place for a product without an order.
- **Removing a fact.** Cedar Creek Home Center cancels order SO-5002, and its row is deleted. The log no longer knows that Cedar Creek is a customer, or where it is, because that row was the only place it said so.
- **Keeping the rules.** Nothing stops a clerk from typing `HT-10O` for `HT-100`, a quantity of `ten`, or an order date in 2062. Each mistake is found, if at all, when a report comes out wrong.

The first three are the **anomalies** of a table that stores several kinds of fact together, and Section 7.2 studies them precisely as update, insertion, and deletion anomalies. The fourth is not an anomaly. The log holds no rules, so nothing can enforce them. Two more problems appear as soon as more than one person needs the log. Two clerks editing their copies of the file on the same afternoon produce two versions, and one clerk's work is lost when the other's is saved. And a question that crosses the log and the warehouse's own spreadsheet of stock (which products are ordered but not in stock?) means copying and matching rows by hand.

A **database** is an organized collection of related data, stored so that many people and programs can share it, with the rules the data must obey stored alongside it. In HG Industries' database, each customer is recorded once, in a table of customers; each product once, in a table of products; each order once; and each line of an order once, referring to its order and its product

rather than copying them. Moving Riverside Hardware changes one row. The GC-400 is a row in the products table with no orders. Canceling SO-5002 removes the order, not the customer. The database refuses a product number that does not exist, a quantity that is not a number, and, if told to, an order date outside a sensible range. The next chapter describes these tables and how they refer to one another.

None of this makes spreadsheets the wrong tool. A spreadsheet is the right place to explore a set of numbers, chart them, or try a what-if calculation, and engineers use them every day. The distinction is between records that a business keeps and shares, which belong in a database, and an analysis of some of those records, which may well belong in a spreadsheet filled by a query. The next section surveys the records an industrial system keeps.

1.2. Data in industrial systems

This section surveys the data that industrial systems generate and the decisions that use it.

Every function an industrial engineer works on produces records, and each record serves a decision. Table 1.2 lists the common ones, with an example from HG Industries and the kind of decision each supports.

Table 1.2.: Data in an Industrial System

Function	Records	Example at HG Industries	Decisions that use them
Order management	customers, orders, order lines, prices	Summit Farm Supply orders 12 HT-100 hand trucks	demand forecasts, customer service, pricing

Function	Records	Example at HG Industries	Decisions that use them
Inventory	items, locations, stock levels, reorder points	18 steel garden carts on hand in Fayetteville	replenishment, safety stock, warehouse layout
Production	parts, bills of materials, routings, work centers, work orders	the HT-100 is a frame, two wheel assemblies, two grips, and a kit	scheduling, capacity planning, costing
Quality	characteristics, specification limits, inspections, measurements, defects	the weld strength of lot L7004-03 is below its minimum	process control, capability studies, supplier quality
Logistics	shipments, carriers, routes, delivery dates	order SO-5001 ships with Boston Mountain Freight three days early	carrier selection, on-time performance
People and equipment	operators, certifications, machines, maintenance	three operators are certified to run the welders	staffing, training, maintenance planning
Analysis and simulation	historical summaries, model inputs, experiment results	a simulation of the frame line with a third welder	improvement projects, investment decisions

Two kinds of use run through Table 1.2. The first is **operational**: recording and looking up the individual events of the business, one order or one inspection at a time, as they happen. The second is **analytical**: summarizing many events to see a pattern, such as on-time performance by carrier over a year or the capability of a process over a quarter. The same records serve both, but the two uses ask different things of a database, and Chapter 11 explains why a company often keeps its data twice, once for each.

Notice also that the functions of Table 1.2 share their data. The product that sales takes an order for is the part that production makes and quality inspects, and the operator who welds a frame is the one whose certification the plant records. Thus, the records of an industrial system form one connected body of data, and a database that holds them should connect them too, rather than leave each department with its own copy. That connected body of data is what a database system manages.

1.3. Database systems and their users

This section describes the parts of a database system and the people who work with it.

A **database management system** (DBMS) is the software that stores a database and does everything with it on behalf of the people and programs that use it. This book uses PostgreSQL, a DBMS that is free, widely used in industry, and close to the SQL standard; others include Oracle, Microsoft SQL Server, MySQL, and SQLite. Figure 1.1 shows how the parts fit together.

Read Figure 1.1 from the left. People do not open a database directly. They use database tools, such as DataGrip and DBeaver, the tools this book teaches, or psql, which comes with PostgreSQL; or they use programs written for a purpose, a report, a scheduling application, a simulation that reads its inputs from the database. Tools and programs alike communicate

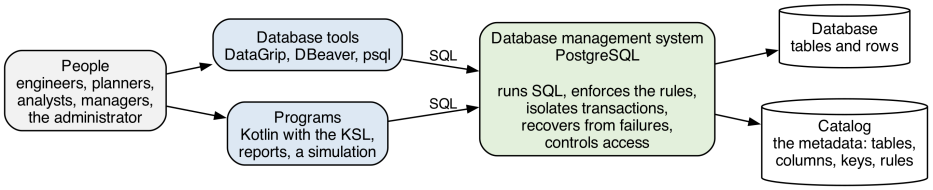


Figure 1.1.: The Parts of a Database System

with the DBMS in **SQL**, the standard language for defining, changing, and querying relational databases, which Chapter 3 begins to teach. The DBMS does the work: it stores the rows, finds the ones a query asks for, and keeps a **catalog** of **metadata**, the data about the data, recording what tables exist, their columns and types, and the rules each must obey.

The DBMS also provides services that no spreadsheet does. Integrity, concurrency, and querying answer three problems of Section 1.1: the rules nothing kept, the clerk's lost work, and the question that crossed two files. Recovery and security answer two problems that section did not raise, a failure in the middle of a change and a user who should not see everything.

- **Integrity.** The rules stored with the data (a product number must be one that exists, a quantity must be a positive whole number) are enforced on every change, by every program. Chapter 8 shows how they are declared.
- **Concurrency.** Many people can read and change the database at once, and the DBMS keeps their work from interfering. Chapter 9 shows how.
- **Recovery.** A change is either made completely or not at all, and a change the DBMS has confirmed survives a power failure.
- **Security.** Each user can be allowed to see and change only what their work requires.
- **Querying.** Any question the data can answer can be asked in SQL, without writing a program for it and without copying the data first.

The people around a database system have different roles, and you may hold several of them. **End users** record and look up data, usually through programs built for them, without seeing SQL. **Analysts** query the database to answer questions and build reports, which is where most industrial engineers begin. **Application programmers** write the programs that others use. **Database designers** decide what the database records and how, the work of Chapter 5 through Chapter 7. A **database administrator** installs and runs the DBMS, grants access, and keeps backups. An industrial engineer is often the analyst of a system and, on an improvement project, its designer too; the next section describes how design proceeds.

1.4. From a model to a database

This section lays out the path from a business's needs to a working database, which is the path the book follows.

Building a database begins well before any table is created. It begins with what people need to record and to know, and with a model of the business that says what the database must hold. Figure 1.2 shows the steps.

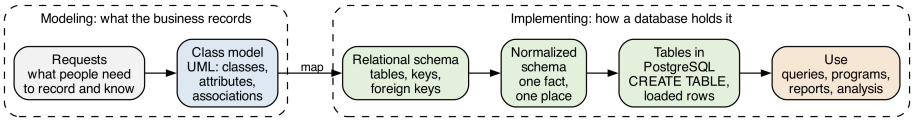


Figure 1.2.: From a Business's Requests to a Working Database

The steps of Figure 1.2 fall into two parts, and the difference between them is one of the most useful ideas in this book. **Modeling** describes the business: what things it records, what it records about each, and how they are related, independently of any software. The book's models are UML class diagrams (Chapter 5), the notation used throughout software and systems engineering. **Implementing** turns the model into a database a

DBMS can hold: tables with keys that link them (Chapter 6), checked so that each fact is stored once (Chapter 7), created and loaded with SQL (Chapter 8), and then used by queries (Chapter 3 and Chapter 4), programs (Chapter 10), and analyses (Chapter 11 and Chapter 12).

The two parts answer different questions, and mixing them causes most bad designs. That is, "Can a customer have more than one address?" is a modeling question, answered by the business; "Should the address be a separate table?" is an implementation question, answered by the model and the rules of Chapter 6. Thus, you should settle what the business records before you decide how to store it. The book, however, starts in the middle, with tables and queries (Chapter 2 through Chapter 4), because a reader who has queried a well-designed database knows what good design produces before learning how to produce it.

1.5. The databases in this book

This section introduces the databases the book's examples and exercises use.

Almost every example in the book uses one database, built for it: the records of **HG Industries**, the small manufacturer of Table 1.1. HG makes four products, two hand trucks and two garden carts, in a plant in Fayetteville, Arkansas, and sells them to hardware stores, farm stores, and distributors from warehouses in Fayetteville and Tulsa, Oklahoma. Its database has a part for each department that keeps records: distribution (customers, orders, products, stock, shipments), production (parts, bills of materials, work centers, machines, operators, routings, work orders), and quality (characteristics, inspections, measurements, defects). Later chapters add a year of history, a star schema for analyzing sales, and the output of a simulation of the plant.

The database is small on purpose. Its base data holds eight customers and twelve orders, few enough that you can check any query's answer by hand,

and the book asks you to, before you trust the computer to do it. The year of history that later chapters add brings the database to 30 customers and 614 orders, for the questions that need more data than a reader checks by hand. The database is also introduced in stages, a few tables at a time, as each chapter needs them, so that a new idea arrives against tables you already know. Appendix C describes the whole database and how to load it into your own PostgreSQL server (Appendix A).

Exercises also use a second database of the book's own, **Millbrook Transit**, a week of service of a small town's bus agency (Section C.10), and four public sample databases, among them a music store and a food importer (Section C.11), so that you practice on schemas designed by other people as well. The next chapter begins with HG Industries' first tables.

1.6. Summary

Operational records belong in a database, not in spreadsheets. A spreadsheet that mixes facts about several things stores each fact many times, so changing, adding, and removing facts goes wrong, nothing enforces the rules, and sharing it loses work. A database stores each fact once, enforces its rules, and lets many people use it at once.

Industrial systems generate connected data. Orders, inventory, production, quality, logistics, and people and equipment each produce records that serve decisions, and the records of one function refer to those of another. The same data serves operational work, one event at a time, and analysis, many events at once.

A database management system does the work. It stores the data and a catalog of metadata, runs SQL for tools and programs, and provides integrity, concurrency, recovery, security, and querying. Its users range from end users through analysts and programmers to designers and administrators.

Modeling comes before implementing. A class model describes what the business records; mapping, normalization, and SQL turn it into tables that a DBMS holds and that queries, programs, and analyses use.

The book's examples use one small database, introduced in stages. HG Industries' base data is small enough to check by hand; Millbrook Transit and four public databases give the exercises other settings.

1.7. Exercises

Exercise 1.1. Match each term on the left with the phrase on the right that defines it.

Term	Definition
a. database	1. the language tools and programs use to define, change, and query a relational database
b. database management system	2. data about the data: the tables, columns, types, and rules
c. metadata	3. an organized collection of related data, shared, with its rules stored alongside it
d. SQL	4. describing what a business records, independently of any software
e. modeling	5. the software that stores a database and does everything with it

Exercise 1.2. Before its database, HG Industries' quality department logged inspections in a spreadsheet. Table 1.3 shows its first rows, for three lots of the hand truck frame.

Table 1.3.: The Quality Department's Inspection Log (Hypothetical)

Lot	Date	Part	Description	Characteristic	Lower limit	Upper limit	Measured	Inspector
L7004-01	2026-01-28	FR-110	Hand truck frame	Coating thickness	2.0	4.0	3.1	Robert Hayes
L7004-01	2026-01-28	FR-110	Hand truck frame	Weld strength	1500		1840	Robert Hayes
L7004-02	2026-01-29	FR-110	Hand truck frame	Coating thickness	2.0	4.0	2.4	Linda Nguyen
L7004-02	2026-01-29	FR-110	Hand truck frame	Weld strength	1500		1795	Linda Nguyen
L7004-03	2026-01-30	FR-110	Hand truck frame	Coating thickness	2.0	4.0	3.3	Robert Hayes
L7004-03	2026-01-30	FR-110	Hand truck frame	Weld strength	1500		1460	Robert Hayes

a. Name four different things the rows describe facts about.
b. Give one example from these rows of each problem of Section 1.1: changing a fact, recording a fact on its own, removing a fact, and keeping the rules.
c. Weld strength has no upper limit. What does the empty cell mean, and how might a reader of the log misread it?

d. Which lot fails an inspection, and what would it take to find all such lots in a log of ten thousand rows?

Exercise 1.3. Choose an industrial system you know, such as a distribution center, a hospital pharmacy, a machine shop, or a campus dining service.

a. List five kinds of records it generates, in the manner of Table 1.2.
b. For each, name one decision that uses it, and say whether the use is operational or analytical.
c. Name two records of different functions that refer to each other.

Exercise 1.4. For each problem, name the service of the database management system (Section 1.3) that addresses it.

a. A power failure strikes while a shipment is being recorded, after its first line is saved and before its second.
b. A clerk records an order for product `HT-100`.
c. Two planners change the same work order's quantity at the same moment.
d. A temporary employee can read every customer's credit terms.
e. A manager wants to know which customers ordered garden carts in March and has no program for it.

Exercise 1.5. On a project to reduce late shipments at HG Industries, name the role (Section 1.3) of each person, and say which roles an industrial engineer on the project might hold.

a. The shipping clerk who records each shipment in the order system.
b. The engineer who queries a year of shipments by carrier and week.
c. The person who adds a table of carrier contracts and decides its columns.
d. The person who restores last night's backup after a disk fails.
e. The programmer who adds a late-shipment warning to the order system.

Exercise 1.6. Say whether each statement is a modeling decision or an implementation decision (Section 1.4), and why.

a. A work order is for exactly one manufactured part.
b. The `work_order` table has a foreign key column `manufactured_part_fk`.
c. An operator may be certified on many machines, with a date and a level for each.
d. Quantities are stored as `INTEGER`.
e. A shipment may carry the lines of only one order.

Exercise 1.7. For each use, say whether a spreadsheet or a database is the better tool, and why.

a. The record of every customer order HG Industries takes.
b. A chart of last quarter's on-time shipments, prepared for one meeting.
c. The inspection results that quality engineers, production supervisors, and an auditor all consult.
d. An engineer's what-if calculation of the capacity added by a third welder.

Exercise 1.8. Using Table 1.2 and Section 1.5, say which of HG Industries' departments' records each question needs, and which questions need records of more than one department.

a. How many HT-100 hand trucks are on hand in Tulsa?
b. Which operators inspected lots whose weld strength was below its minimum?
c. Which customers ordered products made on the saw?
d. How many work orders for the wheel assembly were completed late in March?

Part I.

Reading and Querying a Database

2. The Relational Model

Learning objectives

After reading this chapter you should be able to:

- define a table, a row, a column, and a domain, and state the properties of a relation and why each matters
- identify candidate keys, primary keys, and foreign keys
- state the entity integrity and referential integrity rules, and explain which changes to a table can break referential integrity
- read and write a relational schema, and trace how its tables connect through their keys
- explain what it means for a database to be self-describing, and query its catalog for the tables and columns it holds
- connect to a PostgreSQL database with DataGrip or DBeaver and browse its schemas and tables
- run a script with psql and browse a database with pgAdmin, the tools distributed with PostgreSQL
- contrast the relational model with hierarchical, network, object, and document databases

Most engineers meet a database that someone else built. An intern is handed a connection to the plant's order system and asked how many orders shipped late last quarter; a quality engineer needs the inspection results for one lot. In both cases the first task is the same: find out what

the database holds, and how its parts fit together. This chapter presents the ideas that make that possible.

Chapter 1 described why industrial data belongs in a database and introduced HG Industries, the manufacturer of hand trucks and garden carts whose databases this book uses. This chapter presents the *relational model*, the way nearly every database in industrial use organizes its data. We begin with tables, rows, columns, and the values they may hold, and with the properties that set a table apart from a spreadsheet. We then take up keys, which identify rows and connect tables, and the integrity rules a database enforces to keep those connections sound. With those ideas in place, we write a database's schema down in a compact notation that shows every table's keys and how they connect, and we find that the database stores the same description of itself. We then explore a real database with the tools you will use throughout the book. The chapter closes with a brief look at the other ways databases organize data.

The chapter does not ask you to design anything. Every table in it already exists, and our job is to read it. Designing tables of your own begins in Chapter 5, after the SQL chapters have shown what a well-built database lets you do.

Following along

The examples run in your `dbconcepts` database, created and loaded with `through-q.sql` as Section A.5 describes. The chapter meets the distribution schema a table at a time, in the order of its stages D1 to D5 (Section C.2), and every row it shows is a row of your database. If you are not sure what your database holds, run `code/sql/setup/which-stage.sql`; it says whether a reset is needed.

> **Reading the SQL in this chapter**
>
> The chapter shows SQL statements so that the database itself supplies every result and every refusal. You are not asked to write SQL yet. A `SELECT` reads rows, an `INSERT` adds a row, an `UPDATE` changes the rows its `WHERE` names, and a `DELETE` removes them. Read each statement in three parts: the sentence before it says what it attempts, the statement does it, and the table or message after it is PostgreSQL's reply. You need not decode the syntax; Chapter 3 teaches queries, and Chapter 8 teaches defining tables and changing their rows (Section 8.4).

2.1. Tables, rows, and domains

This section presents the building blocks of the relational model: tables, their rows and columns, and the values a column may hold.

Codd (1970) proposed the relational model in 1970, and it has been the basis of commercial databases ever since. In the model, all data is held in **relations**. A relation has a fixed set of named **attributes**, and each attribute draws its values from a **domain**, the set of values that are allowed for it. Formally, a relation is a set of **tuples**, and each tuple holds one value for each attribute. That is, if a relation has the attributes of a customer, each tuple describes one customer.

In practice a relation is drawn and stored as a **table**. Each attribute is a **column**, and each tuple is a **row**. This book uses the words table, row, and column throughout, and it treats *tuple* and *row* as meaning the same thing. A table looks like a spreadsheet, but it obeys rules that a spreadsheet does not, and Section 2.1.1 states them.

Let's look at a real table. HG Industries' distribution department keeps a table of its customers. The following query asks for every column (the

`*`) of every row of the table named after `FROM`. Queries are the subject of Chapter 3; for now, notice only what comes back.

Table 2.1.: The Customer Table

```
SELECT *
FROM distribution.customer;
```

customer_pk	customer_number	name	city	state
1	C-1001	Riverside Hardware	Fayetteville	AR
2	C-1002	Summit Farm Supply	Springdale	AR
3	C-1003	Cedar Creek Home Center	Tulsa	OK
4	C-1004	Midtown Tool Supply	Joplin	MO
5	C-1005	Prairie Garden Center	Fort Smith	AR
6	C-1006	Lakeside Distributors	Springfield	MO
7	C-1007	Tri-County Farm and Home	Bentonville	AR
8	C-1008	Northgate Hardware	Little Rock	AR

Table 2.1 shows the eight rows of the `customer` table in stage D1. Each row describes one customer, and each column records one fact about every customer: an internal key (`customer_pk`), the customer number the business uses, the customer's name, and the city and state where it is located.

The structure of a table (its name, its columns, and their domains) is called its **schema**, and the rows it holds at a given moment are called its **state**. The schema changes rarely, when the database is redesigned; the state changes every time a row is added, changed, or removed. A **database schema** is the collection of all its tables' schemas. In other words, the schema is what the database *can* hold, and the state is what it holds *now*. Section 2.2 shows why the difference matters when we look for keys.

2.1.1. The properties of a relation

This subsection presents the six properties that every relation has and that a well-built table keeps. A row holds one value in each column, and the place where a row and a column meet is called a **cell**, as in a spreadsheet. The six properties are:

1. Each cell holds one value that cannot be broken into parts, or none at all.
2. The values of each column come from one domain.
3. Each column has a name that no other column of its table shares.
4. No two rows are identical.
5. The order of the rows carries no meaning.
6. The order of the columns carries no meaning.

For each property, we state what it says, see what it means for the `customer` table of Table 2.1, and see what goes wrong without it.

The first property concerns what one cell may hold. Notice in Table 2.1 that each value is a single item, one name or one state. A value that is one member of its column's domain, rather than several values joined in one, is called **atomic**, and the relational model requires every value to be atomic. A column holding "Fayetteville, AR" would break that rule. A query for the customers in Arkansas would have to pick the state out of the text, and an entry typed as "Fayetteville AR" or "Fayetteville, Ark." would slip past it. That is why the city and the state are separate columns.

Whether a value is atomic depends on its domain and on how it is used; how it looks does not settle it. Prairie Garden Center's city in Table 2.1 is "Fort Smith", two words with a space between them, and it is atomic. "Fort Smith" is one member of the domain of cities, and no query, rule, or sort ever needs "Fort" alone. Nor does atomic mean a single character: AR is two characters, one value, and no one ever needs the A alone. A date has a year, a month, and a day, yet it is one value of the date domain. Its data type defines those parts and keeps them well formed, so a query can

ask for a date's month (Section 3.5) and no date can have a 13th month. "Fayetteville, AR" joins members of two domains, a city and a state, in a text value whose parts the database does not know. Thus, the database cannot pick out the state reliably, and it cannot check that the state is a real one. The test is a question about use: does any query, rule, or sort need part of the value on its own, in a way the value's domain does not supply? If so, the parts belong in separate columns.

The same test settles values that have a structure of their own. A telephone number that is always stored and searched whole is atomic, since nothing needs its area code alone. A location's latitude and longitude are two values, because a question such as which warehouse lies north of Joplin, or which is nearest a customer, needs each one alone. An address is atomic only if nothing ever asks for its city or its ZIP code. Section 5.3.1 returns to such values when it models attributes that have parts.

The first property allows one thing besides an atomic value: a cell with no value at all. A **NULL** marks a value that is unknown or that does not apply. For example, an order placed without a requested ship date has NULL in that column. NULL is neither zero nor an empty string, and Section 3.3 explains how queries treat it. Every row of Table 2.1 has a value in every column, because each column of the `customer` table is declared to require one.

The first property has a companion that spreadsheets break just as often. Suppose that HG Industries recorded up to two telephone numbers per customer, in columns named `phone1` and `phone2`. Each cell then holds one value, yet the two columns hold the same attribute, a telephone number, told apart only by position. Several columns that hold the same attribute are called a **repeating group**, and the relational model forbids them. A customer with a third number would need a new column, and a search for one number would have to look in every column of the group. The model records such facts as rows of a table of their own, one row per telephone number, and Section 7.4 returns to repeating groups when it defines first normal form.

The second property says that a column's values all come from its domain. A domain is more than a data type, although in SQL the data type is its main expression. For example, the `state` column is declared `CHAR(2)`, a string of exactly two characters. The domain of US state codes is smaller than that: it has about fifty members, and `ZZ` is not one of them. A database can narrow a column's domain further with a rule that every value must pass, a `CHECK` constraint, which Chapter 8 presents. Thus, the data type is the first line of defense for a domain, and it is usually not the last.

The data type is enforced on every value. The `customer_pk` column is declared `INTEGER`. An `UPDATE` changes values in the rows its `WHERE` names; the following one tries to set Riverside Hardware's `customer_pk` to the text `'ten'`. PostgreSQL refuses it:

```
UPDATE distribution.customer
SET customer_pk = 'ten'
WHERE customer_number = 'C-1001';
```

```
ERROR:  invalid input syntax for type integer: "ten"
LINE 2: SET customer_pk = 'ten'
                          ^
```

Notice that the message names the type and the value, and points to the value in the statement. PostgreSQL refuses the statement for the type alone, before it considers anything else about the change (Section 2.3 shows that it refuses a new number in `customer_pk` too, for another reason). A refused statement changes nothing, so the table is as it was before. Without the second property, a column of quantities could hold `10`, `ten`, and `a dozen`, and no sum or comparison over it would mean anything.

The third property says that each column of a table has its own name. A query finds a column by its name: `city` means the same column of the

`customer` table whatever other columns the table holds. Two columns named `city` would make every mention of `city` ambiguous, and PostgreSQL refuses to create a table that has them. Suppose that a designer wanted both a billing city and a shipping city for each customer. The designer names each column for its role, `billing_city` and `shipping_city`, and every mention of either is then clear. A query result is more lenient than a table: a query may ask for the same column twice, and its result then has two columns with one name. Thus, the third property is enforced for the tables a database stores, and a query that repeats a name is best avoided.

The fourth property says that no two rows of a relation are identical. Each row states a fact (here, that a customer with this number, name, city, and state exists), and a fact stated twice is still one fact. Were Riverside Hardware's row stored twice, a count of the customers in Arkansas would report six where there are five, and every sum over the rows would count its figures twice. In addition, the two copies could not be told apart: no condition in a query could pick out one of them and leave the other. Here SQL departs from the model. SQL does not require a table to be a true set, so a table with no key can hold two identical rows, which a relation never can. The keys of Section 2.2 are what prevent that, and every table in the book's database has one. A query result has no key and may also repeat a row, and a query can ask for each different row only once (Section 3.1).

The fifth property says that the rows of a table have no order. A spreadsheet has a row 1 and a row 2; a table does not, and a database may return the same rows in a different order the next time you ask. When order matters, a query must ask for its rows in an order (Section 3.4). Suppose that someone treated the last row of the `customer` table as the newest customer. That meaning would vanish the first time the rows came back in another order. A fact that matters, such as the date a customer was added, belongs in a column of its own.

The sixth property says that the columns of a relation have no order either. Each column is known by its name (the third property), so `name` holds the

customer's name whether it is the third column or the fifth. Here SQL departs from the model again. A table keeps the order in which its columns were declared, and a query that asks for every column returns them in that order, as Table 2.1 shows. A statement that adds a row without naming its columns also depends on that order, matching its values to the columns by position. A new column changes what a query for every column returns. A table rebuilt with its columns in another order would send such a statement's values into the wrong columns, or have it refused. Thus, a statement that must keep working as the table changes should name its columns, and a statement that adds a row should always name them.

Table 2.2 collects the six properties, what goes wrong without each, and where SQL departs from the model. Notice that the second and fifth properties hold in SQL as the model states them, and that each departure has a remedy in how you write a table or a query.

Table 2.2.: The Properties of a Relation

Property	What goes wrong without it	Where SQL departs
1. One atomic value or NULL per cell	A query must pick values out of text, and variants slip past it	PostgreSQL also offers array and document types, which the book uses only in its appendix on NoSQL (Section H.4)
2. One domain per column	Sums and comparisons over the column mean nothing	None: the data type enforces the domain, and `CHECK` narrows it
3. Unique column names	A mention of a column is ambiguous	A query result may repeat a name, as in `SELECT city, city`

Property	What goes wrong without it	Where SQL departs
4. No identical rows	Counts and sums are inflated, and the copies cannot be told apart	A table with no key, and a query result, may hold identical rows; keys and `DISTINCT` restore the rule
5. No order of rows	Meaning carried by position is lost when the order changes	None: order must be asked for with `ORDER BY`
6. No order of columns	Meaning carried by position is lost when a column is added	A table keeps its declared column order, so `SELECT *` and an `INSERT` with no column list depend on it; name the columns

The properties are easiest to see where they are broken. Table 2.3 shows how the same customers might be kept in a spreadsheet. Its rows are customers of Table 2.1, and its telephone numbers are invented, since the book's database holds none.

Table 2.3.: A Spreadsheet Version of the Customer List (Hypothetical)

Customer	Name	Location	Phone 1	Phone 2
C-1001	Riverside Hardware	Fayetteville, AR	479-555-0101	
C-1002	Summit Farm Supply	Springdale, AR	479-555-0144	479-555-0145
C-1001	Riverside Hardware	Fayetteville, AR	479-555-0101	

Customer	Name	Location	Phone 1	Phone 2
Missouri customers				
C-1004	Midtown Tool Supply	Joplin	417-555-0120	
C-1006	Lakeside Distributors	Springfield	417-555-0188	

Let's walk through Table 2.3 one break at a time.

- The Location cells hold a city and a state together, which breaks the first property: "Fayetteville, AR" is not atomic.
- Phone 1 and Phone 2 hold the same attribute, which makes them a repeating group. Summit Farm Supply's third number would need a third column.
- Riverside Hardware appears twice, which breaks the fourth property. A count of the sheet's customers gives five where there are four.
- The row "Missouri customers" describes no customer, so its Customer cell holds a value outside the column's domain, which breaks the second property. Joplin and Springfield are in Missouri only because their rows sit under that heading, which breaks the fifth property. Sort the sheet by name, and the two customers lose their state.

Undoing each break gives back the `customer` table. Splitting Location into a City column and a State column makes every value atomic. Moving the telephone numbers to a table of their own removes the repeating group, and the `customer` table of stage D1 holds no telephone numbers at all. Deleting the second Riverside Hardware row leaves each customer once. Finally, deleting the heading row and writing MO in the State cell of each Missouri customer puts the meaning of the heading into values. What

remains has the columns of Table 2.1 except `customer_pk`, the internal key that Section 2.2 explains.

Taken together, the six properties say one thing: a table's meaning lives in its values and its names, never in the position of a row or a column or in the layout of a sheet. That is what lets a query say what it wants (the customers whose `state` is MO) without saying how to walk through the rows to find them. It is also why a new column leaves a query that names its columns unaffected, and why the database is free to return rows in whatever order is quickest without changing an answer. One property takes more than care to keep: a table holds no identical rows only if it has a way to tell its rows apart, and that is the subject of keys.

2.2. Keys

This section presents keys: the columns that identify a row and, in another role, connect one table to another.

A table must give us a way to tell its rows apart. A **superkey** is a set of one or more columns whose values identify a row: no two rows of the table may ever have the same values in all of them. The set of all of a table's columns is always a superkey, since two rows that agree in every column would be the same row. Superkeys usually carry more columns than they need, so the model asks for the smallest ones. A **candidate key** is a superkey from which no column can be removed without losing the property. That is, a candidate key satisfies two conditions:

1. The table will never contain two rows that agree on all the columns of the candidate key.
2. No smaller set of those columns satisfies the first condition.

The word *never* in the first condition is the important one. A candidate key is a rule about every state the table could ever be in, which makes it a fact about the business the table describes. The rows the table holds

today can show that a set of columns is *not* a key, by containing two rows that agree on it; however, they can never prove that it is one.

Example 2.1 (Finding the candidate keys of the customer table)**.** *The system.* HG Industries records each customer it sells to in the `customer` table of Table 2.1, which has the columns `customer_pk`, `customer_number`, `name`, `city`, and `state`.

The request. Which columns, alone or together, identify a customer?

Let's examine each column against the eight rows of Table 2.1, and then against what we know about the business. The following table records the check.

Table 2.4.: Checking the Columns of the Customer Table

Column	Two rows share a value?	A rule of the business makes it unique?	Candidate key?
`customer_pk`	No	The database generates a different value for every row	Yes
`customer_number`	No	The company assigns each customer one number, never reused	Yes
`name`	No	No: two stores may share a name	No
`city`	No	No: a second customer may open in the same city	No
`state`	Yes (AR appears five times)	No	No

Notice that the data rules out only `state`. The names and cities of the eight customers happen to be different, yet neither column is a candidate

key, because nothing about the business stops a second hardware store from opening in Fayetteville. Indeed, the year of history in stage G adds a second customer in Springfield, Missouri. In contrast, `customer_number` is a key because the company promises it is: it is the identifier on every invoice.

The check. Each candidate key found has a single column, so the second condition holds automatically: there is no smaller set to try. The same condition settles the columns taken together. Any larger set that contains `customer_pk` or `customer_number` identifies a customer, but it is not minimal, so it is not a candidate key. The only sets left are combinations of `name`, `city`, and `state`, and no rule of the business makes any of them unique: two stores of the same name may open in the same city.

Interpretation. The `customer` table has two candidate keys, `customer_pk` and `customer_number`. A query that must find one customer should look it up by one of them. A report that groups customers by city is doing something else: it gathers customers together, and it never needs a city to single one out.

Where it breaks. If the company ever reused the number of a customer who had closed, `customer_number` would stop being a key, and every report that assumed it was one would silently merge two customers' histories.

A table often has more than one candidate key, and the designer chooses one to be the **primary key**, the key by which the table's rows are identified and referred to. The candidate keys not chosen are called **alternate keys**. In Example 2.1, the primary key is `customer_pk` and `customer_number` is an alternate key. The database enforces both: it rejects any row that would repeat a value of either. For example, when we tried to add a second customer numbered C-1001, PostgreSQL refused with this message:

```
ERROR:  duplicate key value violates unique constraint
 ↪  "customer_customer_number_key"
DETAIL:  Key (customer_number)=(C-1001) already exists.
```

The two candidate keys of the customer table are of different kinds. A **natural key** is made of data that means something to the business, such as a customer number or a part number. A **surrogate key** is a value the database generates only to identify rows, such as `customer_pk`, and it means nothing outside the database. Almost every table in the book's database has a surrogate primary key, named for its table with the suffix `_pk`, and keeps its natural key as an alternate key. Section 6.6 discusses when each kind is the better choice. For reading a database, it is enough to recognize both.

A key may also need more than one column. A key of several columns is called a **composite key**. For example, HG Industries numbers the lines of each order 1, 2, 3, and so on. A line number alone identifies nothing, since every order has a line 1; the order and the line number together identify one line. Thus, the pair (`customer_order_fk`, `line_number`) is a candidate key of the `order_line` table, which Section 2.4 shows.

The last kind of key connects tables. A **foreign key** is a set of columns in one table whose values must match the values of a candidate key in another table (or, occasionally, in the same table). The foreign key refers to the other table's rows. Recall that each customer places orders. The following query lists the orders of stage D2, the stage in which the `customer_order` table first appears.

Table 2.5.: The Customer Order Table

```
SELECT *
FROM distribution.customer_order;
```

customer_order_pk	order_number	order_date	requested_ship_date	customer_fk
1	SO-5001	2026-01-05	2026-01-12	1
2	SO-5002	2026-01-07	2026-01-14	3
3	SO-5003	2026-01-12	NULL	2
4	SO-5004	2026-01-15	2026-01-22	1
5	SO-5005	2026-01-21	2026-02-02	4
6	SO-5006	2026-01-28	2026-02-04	6
7	SO-5007	2026-02-03	2026-02-10	3
8	SO-5008	2026-02-11	NULL	5
9	SO-5009	2026-02-18	2026-02-25	7
10	SO-5010	2026-02-24	2026-03-03	1
11	SO-5011	2026-03-04	2026-03-11	6
12	SO-5012	2026-03-10	2026-03-17	2

Table 2.5 shows the twelve orders placed in January through March 2026. Notice the last column, `customer_fk`. Its value in each row is the `customer_pk` of the customer who placed the order, so it is a foreign key that refers to the `customer` table. For example, order SO-5001 has `customer_fk` 1, and the customer whose `customer_pk` is 1 in Table 2.1 is Riverside Hardware. Notice also that orders SO-5003 and SO-5008 have NULL for their requested ship dates: those customers did not ask for a date.

Customer 1 appears three times in the `customer_fk` column, and customer 8 (Northgate Hardware) does not appear at all. That pattern, one customer with many orders and some customers with none, is what a foreign key records. The next section presents the rules that keep such references sound.

2.3. Integrity rules

This section presents the two rules that every relational database enforces on its keys.

The first rule protects primary keys. The **entity integrity** rule states that no column of a primary key may be NULL. The reason is simple: a primary key is how a row is identified, and a row whose identifier is unknown cannot be found or referred to. PostgreSQL applies the rule automatically; declaring a column part of a primary key also declares that it may not be NULL.

The second rule protects foreign keys. The **referential integrity** rule states that every value of a foreign key must either match a value of the key it refers to or be NULL. In other words, a reference must point to a row that exists. An order that refers to customer 9 when there is no customer 9 would be an *orphan*: an order that no one placed. The database enforces the rule in both directions, as the following attempts on stage D2 illustrate.

First, suppose that we try to record an order for customer 9, who does not exist. An `INSERT` adds a row to the table it names, with the values it lists for the columns it lists:

```
INSERT INTO distribution.customer_order
    (order_number, order_date, customer_fk)
VALUES ('SO-5013', '2026-03-12', 9);
```

PostgreSQL rejects the new row and names the rule it would break:

```
ERROR:  insert or update on table "customer_order" violates
 ↪  foreign key constraint "customer_order_customer_fk_fkey"
DETAIL:  Key (customer_fk)=(9) is not present in table
 ↪  "customer".
```

Second, suppose that we try to remove Riverside Hardware (C-1001), who has placed three orders. A `DELETE` removes the rows its `WHERE` names; this one names Riverside Hardware's row:

```
DELETE FROM distribution.customer
WHERE customer_number = 'C-1001';
```

Deleting the customer would leave its three orders pointing at nothing, so PostgreSQL refuses again:

```
ERROR:  update or delete on table "customer" violates foreign
 ↪  key constraint "customer_order_customer_fk_fkey" on table
 ↪  "customer_order"
DETAIL:  Key (customer_pk)=(1) is still referenced from table
 ↪  "customer_order".
```

Notice what the second message says: the key is *still referenced*. The same statement for Northgate Hardware (C-1008), which has no orders, succeeds, because no row refers to it. Thus, referential integrity depends on the state of the database as well as on its schema.

Notice that the two attempts changed different tables. The first added a row to the table that holds the foreign key; the second removed a row from the table the foreign key refers to. A change to either table can break referential integrity, but not every change can. Table 2.6 considers each kind of change to the two tables of stage D2.

Table 2.6.: Changes That Can Break Referential Integrity

Table	Change	Can it break the rule?	What the database can do
`customer`	add a row	No	Nothing is needed
`customer`	change `customer_pk`	Yes	Refuse, cascade, set NULL, or set a default
`customer`	remove a row	Yes	Refuse, cascade, set NULL, or set a default
`customer_order`	add a row	Yes	Refuse
`customer_order`	change `customer_fk`	Yes	Refuse
`customer_order`	remove a row	No	Nothing is needed

Let's read Table 2.6 one table at a time. A new customer is referred to by no order, so adding one is always safe; removing an order leaves no reference behind, so it is always safe too. An order that is added, or whose `customer_fk` is changed, may refer to a customer who does not exist. The only sound response is to refuse the change, as PostgreSQL did with order SO-5013.

The remaining rows change the table that is referred to. When a customer who has orders is removed, or its `customer_pk` is changed, the database has four possible responses. The designer chooses one for each foreign key when the table is defined:

- refuse the change, as PostgreSQL did with Riverside Hardware (`NO ACTION`, the default, or `RESTRICT`);
- cascade the change: remove the customer's orders along with it, or change their `customer_fk` to the new value (`CASCADE`);
- set NULL: keep the orders but set their `customer_fk` to NULL, so that they refer to no customer (`SET NULL`);

- set a default: set their `customer_fk` to the column's default value, which must itself match a customer that exists (`SET DEFAULT`).

In PostgreSQL, `NO ACTION` and `RESTRICT` both refuse; they differ only in when the check is made, which matters only inside the transactions of Chapter 9.

Almost every foreign key in the book's database refuses: its definition names no response, so it takes the default. The exceptions cascade deletions from a whole to its parts, such as an order's lines, which cannot exist without their order; Chapter 6 explains why. The foreign key of `customer_order` refuses, as the Riverside Hardware attempt showed. Setting NULL could never succeed in the book's database, since every foreign key column in it is declared `NOT NULL`. In addition, the surrogate key `customer_pk` refuses any value we supply. PostgreSQL generates its values, and it rejects an attempt to set one:

```
UPDATE distribution.customer
SET customer_pk = 20
WHERE customer_number = 'C-1001';
```

```
ERROR:  column "customer_pk" can only be updated to DEFAULT
DETAIL:  Column "customer_pk" is an identity column defined
 ↪  as GENERATED ALWAYS.
```

Notice that the message allows one change: the column may be set to `DEFAULT`, which draws a new generated value. Nothing about a customer ever requires that, because the value means nothing outside the database. Thus, a surrogate key is not meant to change, and a reference to one never needs to follow a change. That is one reason for surrogate keys, which Section 6.6 takes up, and Chapter 8 shows how to declare each response. For reading a database, the lesson is that a foreign key is a promise the database keeps. When you query a table with a foreign key, every value

in it that is not NULL has a matching row in the table it refers to, and a query can rely on that.

2.4. Writing down a schema

This section presents a compact way to write down a database's schema: its tables, their keys, and the references that connect them.

Recall from Section 2.1 that a database schema is the collection of all its tables' schemas. To read a database, we need that collection in front of us, with the keys of Section 2.2 marked. This book writes a database schema down in a compact text notation, which we call a **relational schema**. At a minimum, a relational schema gives:

- each table's name, followed by its columns in parentheses;
- each table's primary key, and its alternate keys;
- each foreign key, with the key it refers to, the table that holds that key, and whether the foreign key may be NULL.

A key of several columns is written in parentheses. Listing 2.1 gives the relational schema of stage D2.

Listing 2.1 The Relational Schema of Stage D2

```
customer (customer_pk, customer_number, name, city, state)
    primary key: customer_pk
    alternate key: customer_number

customer_order (customer_order_pk, order_number, order_date,
                requested_ship_date, customer_fk)
    primary key: customer_order_pk
    alternate key: order_number
    customer_fk references customer_pk in customer,
        must not be NULL
```

The first entry of Listing 2.1 says that the `customer` table has five columns, that `customer_pk` is its primary key, and that `customer_number` is its alternate key: the keys found in Example 2.1. The last line of the `customer_order` entry is its foreign key. It says that `customer_fk` refers to `customer_pk` in the `customer` table and must not be NULL, so every order has a customer. Read in the other direction, the same line says that a customer may be referred to by any number of orders, including none.

Notice that the book's key names do some of the work. A column ending in `_pk` is its table's primary key, and a column ending in `_fk` is a foreign key named for the table it refers to, or sometimes for the role the referenced row plays (a bill of materials line names an assembly and one of its components, as `assembly_fk` and `component_fk`). The relational schema states the keys anyway, because a key of several columns, an alternate key, and a database named by someone else's rules all need saying.

The minimal form leaves out what is needed to build the tables rather than read them. A **complete relational schema** adds each column's data type and whether it may be NULL, the `CHECK` constraints that narrow each domain, and each foreign key's response to deleting or updating the row it

refers to (Table 2.6). Listing 2.2 gives the complete relational schema of the `customer_order` table.

Listing 2.2 The Complete Relational Schema of the Customer Order Table

```
customer_order
    customer_order_pk    INTEGER      not NULL, generated
    order_number         VARCHAR(10)  not NULL
    order_date           DATE         not NULL
    requested_ship_date  DATE         may be NULL
    customer_fk          INTEGER      not NULL
    primary key: customer_order_pk
    alternate key: order_number
    customer_fk references customer_pk in customer,
        must not be NULL
    customer_fk: on delete no action, on update no action
    check: requested_ship_date >= order_date
```

The column lines of Listing 2.2 confirm what Table 2.5 showed: `requested_ship_date` is the one column that may be NULL. The word *generated* on `customer_order_pk` says that the database generates its values, which is what makes it a surrogate key. The line that begins `customer_fk: on delete` records the response chosen for the foreign key, no action, which means refuse. The `check` line narrows the domain of `requested_ship_date`: a customer cannot ask for an order to ship before the order was placed. In other words, a complete relational schema holds everything needed to write the statements that create the tables, which Chapter 8 presents.

The relational schema of the whole distribution schema, as your database holds it, is too long for one listing, so it is given in two. Listing 2.3 gives

the tables of orders and products, and Listing 2.4 the tables of warehouses and shipments.

Listing 2.3 The Relational Schema of the Distribution Schema: Orders and Products

```
customer (customer_pk, customer_number, name, city, state)
    primary key: customer_pk
    alternate key: customer_number

customer_order (customer_order_pk, order_number, order_date,
                requested_ship_date, customer_fk)
    primary key: customer_order_pk
    alternate key: order_number
    customer_fk references customer_pk in customer,
        must not be NULL

product (product_pk, product_number, description,
         list_price, manufactured_part_fk)
    primary key: product_pk
    alternate key: product_number
    manufactured_part_fk references part_fk in
        production.manufactured_part, must not be NULL

order_line (order_line_pk, customer_order_fk, line_number,
            product_fk, quantity, unit_price)
    primary key: order_line_pk
    alternate key: (customer_order_fk, line_number)
    customer_order_fk references customer_order_pk in
        customer_order, must not be NULL
    product_fk references product_pk in product,
        must not be NULL
```

Listing 2.4 The Relational Schema of the Distribution Schema: Warehouses and Shipments

```
warehouse (warehouse_pk, warehouse_code, city, state)
    primary key: warehouse_pk
    alternate key: warehouse_code

stock_level (stock_level_pk, warehouse_fk, product_fk,
             quantity_on_hand, reorder_point)
    primary key: stock_level_pk
    alternate key: (warehouse_fk, product_fk)
    warehouse_fk references warehouse_pk in warehouse,
        must not be NULL
    product_fk references product_pk in product,
        must not be NULL

shipment (shipment_pk, shipment_number, shipped_on, carrier,
          customer_order_fk, warehouse_fk)
    primary key: shipment_pk
    alternate key: shipment_number
    customer_order_fk references customer_order_pk in
        customer_order, must not be NULL
    warehouse_fk references warehouse_pk in warehouse,
        must not be NULL

shipment_line (shipment_line_pk, shipment_fk, order_line_fk,
               quantity)
    primary key: shipment_line_pk
    alternate key: (shipment_fk, order_line_fk)
    shipment_fk references shipment_pk in shipment,
        must not be NULL
    order_line_fk references order_line_pk in order_line,
        must not be NULL
```

The foreign key lines of Listing 2.3 and Listing 2.4 sort the eight tables into three groups. Three tables refer to no other table of the distribution schema: `customer`, `product`, and `warehouse`. (A product does refer to a table of the `production` schema, `production.manufactured_part`, the part the plant makes and sells as that product; Chapter 5 shows why.) Four refer only to those three or to each other: an order refers to a customer, an order line to an order and a product, a stock level to a warehouse and a product, and a shipment to an order and a warehouse. The last, `shipment_line`, refers to a shipment and to the order line it ships. Notice also the three alternate keys written in parentheses. The first, in `order_line`, is the composite key of Section 2.2, an order and a line number. The second, in `stock_level`, says that a warehouse holds at most one stock level for each product. The third, in `shipment_line`, says that a shipment ships a given order line at most once.

The tables of the book's database also carry a schema name, the name of a group of tables within the database. The distribution tables belong to the schema `distribution`, so a query names the customer table `distribution.customer`, as the queries of this chapter have. The production and quality departments' tables belong to schemas of their own, and Appendix C shows all of them.

The main use of a relational schema is to find where the answer to a question lives, before any query is written. The following example illustrates the process.

Example 2.2 (Tracing a request through the schema)**.** *The system.* The distribution schema of Listing 2.3 and Listing 2.4.

The request. The sales manager asks: which customers have ordered the GC-300 steel garden cart?

The request names two things, customers and a product, so we begin by finding the tables that describe them: `customer` and `product`. Neither refers to the other, so we look for a path of foreign keys between them. The

foreign keys of `order_line` refer to `product` and to `customer_order`, and the foreign key of `customer_order` refers to `customer`. Thus, the path runs from `product` to `order_line`, then to `customer_order`, and finally to `customer`. The columns along the path are the keys that connect each pair:

Table 2.7.: The Path from a Product to Its Customers

Foreign key	Refers to
`order_line.product_fk`	`product.product_pk`
`order_line.customer_order_fk`	`customer_order.customer_order_pk`
`customer_order.customer_fk`	`customer.customer_pk`

The check. Every step of Table 2.7 is a foreign key line of Listing 2.3, and every step matches a foreign key to the primary key it refers to, so each connection is guaranteed by referential integrity.

Interpretation. Four tables answer the question. A query needs the product's number from `product`, the customers' names from `customer`, and the two tables between them only to make the connection.

Where it breaks. The path finds customers who *ordered* the cart. A manager who asks which customers *received* one needs a different path, through `shipment_line` and `shipment`, because an order line that has not shipped is still an order.

Tracing a path of keys through a schema is the reasoning behind a join, the operation Section 3.6 presents. So far the schema has been given to us as a listing. The next section shows that the database holds the same description itself, where anyone can ask for it.

2.5. A database describes itself

This section shows that a relational database stores the description of its own tables, and that the description is queried like any other data.

Recall from Section 1.3 that a DBMS keeps a *catalog* of *metadata*, the data about the data. The relational schemas of Section 2.4 are metadata: they say what tables exist, what columns each has, and which keys and rules apply, without listing a single customer. A database that stores this description inside itself, where people and programs can read it, is called **self-describing**. Every relational database is self-describing, and it holds the description the relational way, in tables.

Compare the sales office's order log of Table 1.1. Its only description is its heading row. The heading "Qty" names a column, but nothing in the spreadsheet records that a quantity must be a positive whole number or that an order number identifies one order. Those rules live in the heads of the people who keep the log, and they hold only as long as those people are careful. In a database, the same rules are part of the catalog, and the DBMS reads them on every change.

The SQL standard defines a set of views of the catalog, called the `information_schema`, which every system that follows the standard provides. Its views are queried with the same SQL as the data. The following query asks the catalog which tables the distribution schema holds. Unlike the queries before it, it names the columns it wants after `SELECT`, keeps only the rows its `WHERE` allows, and sorts them with `ORDER BY`.

Table 2.8.: The Tables of the Distribution Schema, from the Catalog

```
SELECT table_name, table_type
FROM information_schema.tables
WHERE table_schema = 'distribution'
ORDER BY table_name;
```

table_name	table_type
customer	BASE TABLE
customer_order	BASE TABLE
order_line	BASE TABLE
product	BASE TABLE
shipment	BASE TABLE
shipment_line	BASE TABLE
stock_level	BASE TABLE
warehouse	BASE TABLE

Table 2.8 lists the eight tables of Listing 2.3 and Listing 2.4. Each is a `BASE TABLE`, a table that stores rows; the catalog also lists views, which Section 9.1 presents. The next query asks the catalog about the columns of one table. Its `WHERE` keeps the rows that meet both conditions joined by `AND`, and `AS` gives the column `character_maximum_length` the shorter name `length` in the result.

Table 2.9.: The Columns of the Customer Order Table, from the Catalog

```
SELECT column_name, data_type,
       character_maximum_length AS length,
       is_nullable
FROM information_schema.columns
WHERE table_schema = 'distribution'
  AND table_name = 'customer_order'
ORDER BY ordinal_position;
```

column_name	data_type	length	is_nullable
customer_order_pk	integer	NULL	NO
order_number	character varying	10	NO
order_date	date	NULL	NO
requested_ship_date	date	NULL	YES
customer_fk	integer	NULL	NO

Notice that Table 2.9 holds the column lines of the complete relational schema of Listing 2.2, in the catalog's words. The type `character varying` with a length of 10 is the `VARCHAR(10)` of `order_number`, spelled as the standard spells it, and `is_nullable` is `YES` only for `requested_ship_date`. Other views of the `information_schema` record the keys, the foreign keys, and the `CHECK` constraints, and Chapter 8 queries them. Thus, a relational schema can be produced from the catalog instead of being written by hand. The listings in this book were produced that way, by a program that reads the catalog of each stage.

Recall from Section 2.1 the difference between a schema and a state. The rows of the catalog are the database's schema. Creating a table adds rows to the catalog; adding a customer does not. Thus, the catalog changes only when the database is redesigned.

The DBMS itself reads the catalog more than anyone. Before it runs a query, it looks up every table and column the query names. A query that names a column the table does not have is refused before a single row is read. The following query asks for `order_no`, but the column is named `order_number`:

```
SELECT order_no
FROM distribution.customer_order;
```

```
ERROR:  column "order_no" does not exist
LINE 1: SELECT order_no
               ^
```

The tools read it too. The schema tree of DataGrip and DBeaver, the schema diagram they draw, and psql's `\d` command, all presented in the next section, show what the catalog holds; so does a program that asks the database for a table's columns, as Section 10.4 shows. That is what makes a database you have never seen readable: you can always ask it what it holds.

The catalog records structure, not meaning. It says that `unit_price` is a number with two decimal places that may not be NULL. It does not say whether the price is in dollars, whether it includes a discount, or how it relates to the product's list price. That knowledge belongs in the design's data dictionary, which Section 5.3 presents. PostgreSQL can store a sentence of description with each table and column (`COMMENT ON COLUMN`), and DataGrip and DBeaver show it beside the column; several other systems can as well, but the SQL standard does not define it. With comments, the catalog holds a little of the meaning too. Before we write queries, you need a way to open a database and look around in its catalog, which the next section provides.

2.6. Exploring a database: DataGrip, DBeaver, psql, and pgAdmin

This section presents the tools you will use to connect to a PostgreSQL database and explore it. Appendix A explains how to install each of them and how to load the book's database into your server.

DataGrip and DBeaver are the tools this book uses. Both connect to many kinds of database, and both show a database the same way: a tree of the database's schemas, each schema's tables, and each table's columns, keys, and constraints, beside an editor in which you write and run SQL. To connect to the book's database, you create a data source (DataGrip) or a database connection (DBeaver) for PostgreSQL, giving the host `localhost`, the port `5432`, the database name `dbconcepts`, and your user name. Once connected, expand the `distribution` schema and then its tables, and you will find the columns and keys of Listing 2.3 and Listing 2.4. The tree is drawn from the catalog of Section 2.5, which is why it is the same for every database you connect to.

Figure 2.1 shows the tree as DataGrip draws it, here inside IntelliJ IDEA, whose database tools are DataGrip's. The connection to the server is at the top, the `dbconcepts` database below it, and its schemas below that. The `distribution` schema is open to its eight tables, and `customer_order` is open to its five columns, in the order of Listing 2.3. The icons mark `customer_order_pk` as the primary key and `customer_fk` as a foreign key. The folders below the columns count the table's constraints: two keys (the primary key and the alternate key on `order_number`), one foreign key, and one check, which keeps the requested ship date from preceding the order date. PostgreSQL builds an index to enforce each key, which is why the tree also lists two indexes; this book does not cover indexes further.

Notice that `information_schema` and `pg_catalog` appear in Figure 2.1 beside `distribution`, as schemas like any other. They are the catalog of Section 2.5, and the tool reads them to draw the tree. The `production`

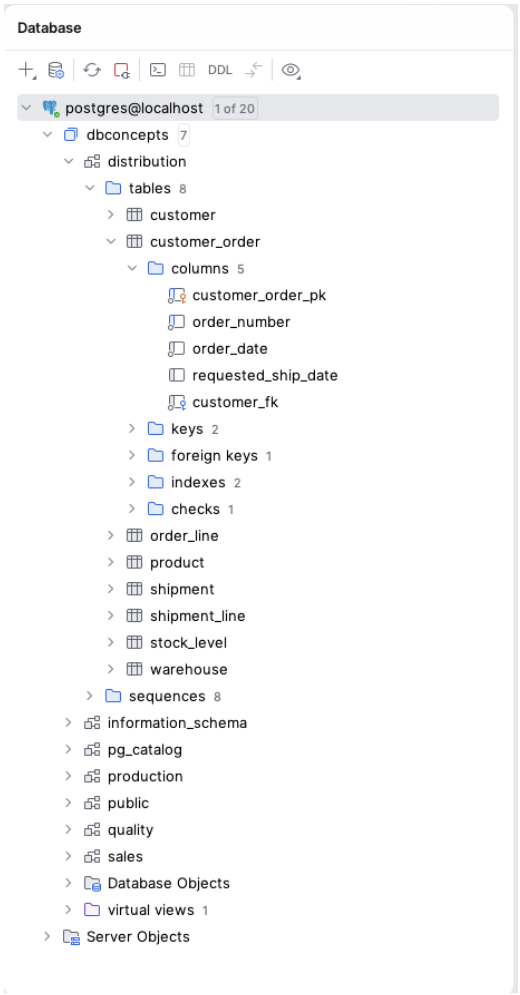


Figure 2.1.: The Database Tree of DataGrip, Open to the `customer_order` Table

and `quality` schemas hold the other departments' tables, which later chapters use. The `sales` schema is the warehouse of Chapter 11, which your `dbconcepts_history` database holds; your `dbconcepts` has none. If your tree shows only some schemas, Section A.2 explains how to show them all. DBeaver's tree, and the tree of a later version of DataGrip, may look a little different, but the outline is the same: a database, its schemas, their tables, and each table's columns and constraints.

Both tools will also draw a **schema diagram**, a picture of the tables you select and the foreign keys that connect them. Figure 2.2 draws the tables of the distribution schema the way these tools do. Each box is a table, with its key columns first, marked `PK` and `FK`, and its other columns below a line. Each column shows its SQL data type; `UNIQUE` marks an alternate key, `(optional)` marks a column that may be NULL, and a key of several columns is listed at the bottom as a `UNIQUE (...)` line. Each arrow is one foreign key, running from the table that holds it to the table it refers to and labeled with the foreign key column. The foreign key of `product` has no arrow, since the table it refers to is outside the schema drawn.

A schema diagram shows at a glance what the relational schema says line by line. For example, the arrow from `customer_order` to `customer` in Figure 2.2 is the line `customer_fk references customer_pk in customer` of Listing 2.3. For an unfamiliar database, you should ask a tool for its diagram before writing a query: reading it takes a few minutes and saves hours of guessing. Tools differ in how they draw a diagram, however, while a relational schema says the same things whichever tool produced it.

PostgreSQL itself comes with two tools. **psql** is a command-line program that connects to a database and runs SQL typed at its prompt or read from a file. It is the quickest way to run a script, and it is available wherever PostgreSQL is installed. For example, this command loads the book's database, from the `sql` folder of the companion code:

```
psql -d dbconcepts -f sql/load/through-q.sql
```

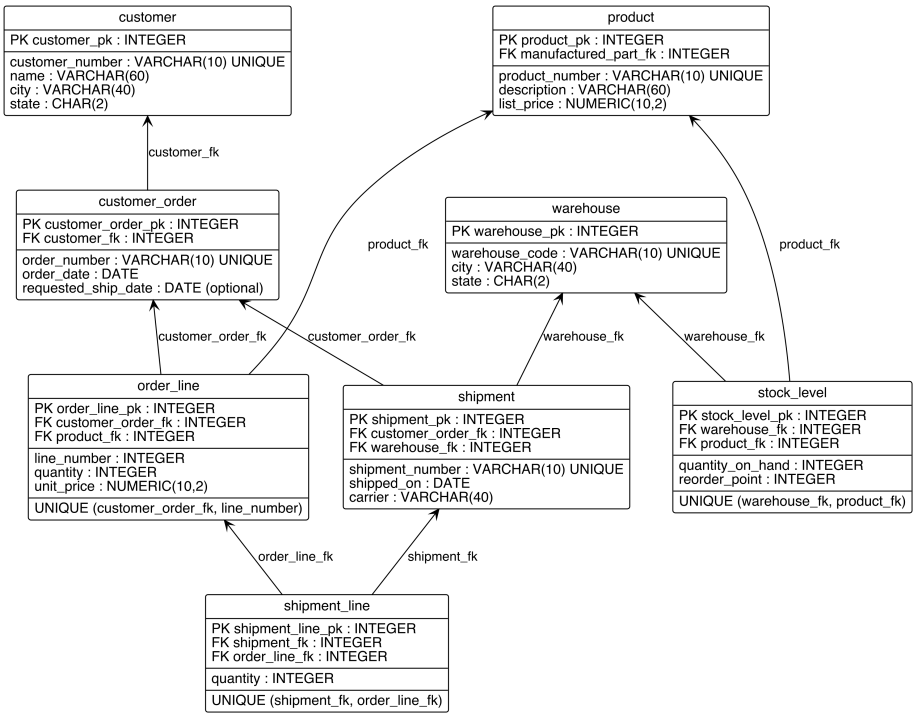


Figure 2.2.: The Distribution Schema, Drawn as a Database Tool Draws It

On Windows, add `-U postgres` and give the superuser's password when psql asks; Section A.4 gives the form for each system.

psql also has commands of its own, which begin with a backslash. The command `\dt distribution.*` lists the tables of the `distribution` schema:

```
                List of tables
    Schema    |      Name      | Type  |  Owner
--------------+----------------+-------+----------
 distribution | customer       | table | rossetti
 distribution | customer_order | table | rossetti
 distribution | order_line     | table | rossetti
 distribution | product        | table | rossetti
 distribution | shipment       | table | rossetti
 distribution | shipment_line  | table | rossetti
 distribution | stock_level    | table | rossetti
 distribution | warehouse      | table | rossetti
(8 rows)
```

The command `\d distribution.customer_order` describes one table: its columns, their types, and its constraints:

```
                              Table
 ↪  "distribution.customer_order"
      Column        |         Type          | Collation |
 ↪  Nullable |           Default
⌋
 ↪  --------------------+----------------------+-----------+----------+-----------
 customer_order_pk   | integer               |           |
 ↪  not null | generated always as identity
 order_number        | character varying(10) |           |
 ↪  not null |
 order_date          | date                  |           |
 ↪  not null |
```

```
 requested_ship_date | date                    |           |
 ↪  |
 customer_fk         | integer                 |           |
 ↪  not null |
Indexes:
    "customer_order_pkey" PRIMARY KEY, btree
 ↪  (customer_order_pk)
    "customer_order_order_number_key" UNIQUE CONSTRAINT,
 ↪  btree (order_number)
Check constraints:
    "customer_order_check" CHECK (requested_ship_date >=
 ↪  order_date)
Foreign-key constraints:
    "customer_order_customer_fk_fkey" FOREIGN KEY
 ↪  (customer_fk) REFERENCES
 ↪  distribution.customer(customer_pk)
Referenced by:
    TABLE "distribution.order_line" CONSTRAINT
 ↪  "order_line_customer_order_fk_fkey" FOREIGN KEY
 ↪  (customer_order_fk) REFERENCES
 ↪  distribution.customer_order(customer_order_pk) ON DELETE
 ↪  CASCADE
    TABLE "distribution.shipment" CONSTRAINT
 ↪  "shipment_customer_order_fk_fkey" FOREIGN KEY
 ↪  (customer_order_fk) REFERENCES
 ↪  distribution.customer_order(customer_order_pk)
```

Notice that the output holds what the complete relational schema of Listing 2.2 says, in PostgreSQL's words. psql reads it from PostgreSQL's own catalog, the tables of the `pg_catalog` schema, over which the standard's `information_schema` views are defined. The `Nullable` column marks the columns that may not be NULL; only `requested_ship_date` is blank, since it is optional. The primary key and the alternate key appear under

`Indexes`, and the foreign key appears under `Foreign-key constraints`, with the table and column it refers to. Its response to a deletion is not printed, because it is the default, no action. Under `Referenced by` are the foreign keys of other tables that refer to this one: an order line's, whose deletion response is cascade (Section 2.3), and a shipment's. The `Owner` column of the first listing, and the constraint names, will differ on your machine: the owner is whoever loaded the database, and PostgreSQL names a constraint that its creator did not name.

pgAdmin is PostgreSQL's graphical tool, often installed with the server. It shows the same tree of schemas, tables, and columns as DataGrip and DBeaver, and it can run SQL. You may meet it on a machine where nothing else is installed; this book does not use it further.

2.7. Other ways to organize data

This section places the relational model among the other ways databases organize data, so that you recognize them when you meet them.

The relational model is not the only way to organize a database, and it was not the first. Two questions separate the alternatives from it and from one another. First, where does the description of the data live: in a catalog, in each record, or only in the programs that use the data? Second, how is one record connected to another: by matching values, as a foreign key matches a primary key, or by a stored link that a program follows?

The databases that came before the relational model connected records by stored links. A **hierarchical database** arranges records in trees, each record under one parent. IBM's IMS, the best known, was built in the 1960s to keep track of the parts of the Apollo program (Kleppmann 2017), and a bill of materials is a tree, so the fit was natural. The fit breaks when a part goes into two assemblies, since a record has only one parent: the part is stored twice, or reached through a special link. A **network database**, standardized by the CODASYL committee around 1970, lets

a record have many parents, and a program finds data by following links from record to record. In both, a question nobody had planned for meant a new program to walk the links. The relational model replaced them because it connects tables by values, so any question can be asked in a query; Section 4.6 shows how SQL walks a bill of materials. IMS still runs in some banks and manufacturers.

An **object database** stores the objects of a program directly, each with an identity of its own, connected by references as the objects of a running program are. Its appeal is that the program's classes and the database's structure are the same thing, so nothing is translated between them. Object databases found uses in engineering design and telecommunications, but they never displaced relational ones. Instead, relational systems took on some of their features, and libraries now do the translation, as Section 10.6 shows. The class models of Chapter 5 describe objects in just this way, and Chapter 6 maps them to tables.

The most common alternatives today go by the name **NoSQL**, read "not only SQL": document, key-value, wide-column, and graph databases, built for data spread over many machines and for data whose structure changes often. The **document database** is the one you are most likely to meet. It stores each record as a **document**, usually written in JSON, a text format of named fields in which a field may hold a list or another document. Listing 2.5 is order SO-5001 of stage D5 written as one JSON document, built from the database by a query.

Listing 2.5 Order SO-5001 as a JSON Document

```
{
  "order_number": "SO-5001",
  "order_date": "2026-01-05",
  "customer": {
    "customer_number": "C-1001",
    "name": "Riverside Hardware",
    "city": "Fayetteville",
    "state": "AR"
  },
  "lines": [
    {
      "line_number": 1,
      "product_number": "HT-100",
      "description": "Two-wheel hand truck",
      "quantity": 10,
      "unit_price": 89.00
    },
    {
      "line_number": 2,
      "product_number": "GC-300",
      "description": "Steel garden cart",
      "quantity": 4,
      "unit_price": 119.00
    }
  ]
}
```

Compare Listing 2.5 with Listing 2.3. The relational database holds this order in four tables connected by keys: one row of `customer_order`, one of `customer`, two of `order_line`, and two of `product`. The document holds

it in one place, with the order's lines nested inside it and copies of the customer's and the products' facts written in. A program that needs the whole order reads one document, without a join.

Notice two things about the document. First, every field carries its own name, so each document describes itself and no catalog is needed to read it. The cost is that nothing requires the next document to use the same names or the same structure; a misspelled field is simply a new field. Second, the description of the HT-100 is now stored in every order that includes one, which is the duplication of the order log of Section 1.1. Changing a product's description means finding and changing every document that copies it. Thus, a document database suits data that is written and read whole, such as an order or a test record, better than facts shared by many records. Appendix H describes the four families of NoSQL databases, when each suits, and how PostgreSQL itself stores and queries JSON documents.

2.8. Summary

A relational database holds all of its data in tables. Each table has a fixed set of columns, each with a name of its own and each drawing its values from a domain. Every value is atomic: no query, rule, or sort needs a part of it that its domain does not supply. A value may be NULL when it is unknown or does not apply. A relation's rows are never identical, and neither its rows nor its columns have an order. A SQL table needs a key to keep the first rule, and it keeps the order in which its columns were declared, which a statement should not depend on. Thus, a table's meaning lives in its values and names, never in position or layout.

A table's schema is its structure, and its state is its current rows. The schema changes when the database is redesigned; the state changes with every change to its rows. Keys and integrity rules are part of the schema, which is why they hold for every state.

Keys identify rows, and the business decides what the keys are. A candidate key is a smallest set of columns that no two rows may ever share. The data can show that a set of columns is not a key, but only the rules of the business can make one a key. The designer chooses one candidate key as the primary key, and the others become alternate keys; in the book's database, almost every primary key is a surrogate key, and each natural identifier is an alternate key.

Foreign keys connect tables, and referential integrity keeps the connections sound. A foreign key's values must match a row of the table it refers to, or be NULL. A change can break the rule by adding a reference to a missing row, or by removing or changing a row that is still referred to. The database refuses the first kind; for the second, the designer chooses whether it refuses, cascades, sets NULL, or sets a default.

A relational schema is the map to read before writing a query. It lists every table's columns and keys, and a line for every foreign key that names the table it refers to. Following those lines from one table to another finds the tables that connect them, which is the reasoning behind the joins of Chapter 3. A complete relational schema adds what is needed to build the tables, and a schema diagram draws the same map as a picture.

A relational database is self-describing. Its catalog of metadata (the tables, columns, keys, and rules) is stored in the database and queried with SQL through the standard `information_schema`. The DBMS reads it to check every query, and every tool and program that explores a database reads it too. The catalog records structure; the meaning of the data belongs in the design's data dictionary.

Other databases organize data in other ways. Hierarchical and network databases connected records by stored links that programs followed. Object databases store a program's objects as they are. Document databases store each record whole, with its own field names, nesting what a relational database spreads over several tables and copying what it stores once.

With a database connected and its schema in hand, you are ready to ask it questions. Chapter 3 presents the SQL `SELECT` statement, beginning with a single table and building to queries that follow the paths of this chapter across many.

2.9. Exercises

Exercise 2.1. Match each term on the left with the phrase on the right that defines it.

Term	Definition
a. domain	1. a candidate key the designer has chosen to identify rows
b. superkey	2. the rows a table holds at one moment
c. candidate key	3. the set of values a column may hold
d. primary key	4. a set of columns that no two rows may share
e. alternate key	5. a smallest set of columns that no two rows may share
f. state	6. a candidate key that was not chosen as the primary key

Exercise 2.2. Mark each statement true or false, and correct each false one.

a. The rows of a table are stored in the order in which they were added, and a query returns them in that order.
b. A column of a primary key may be NULL if the value is not yet known.
c. A foreign key column may be NULL.
d. If no two rows of a table currently share a value of a column, that column is a candidate key.

e. A table may have more than one candidate key.

Exercise 2.3. A clerk at HG Industries keeps the stock of its two warehouses in the spreadsheet of Table 2.10. Its quantities come from the `stock_level` table of stage D4 (Listing 2.4); its bin locations are invented. The clerk knows that the first Quantity column is the quantity on hand and the second is the reorder point.

Table 2.10.: A Clerk's Stock Sheet (Hypothetical)

Product	Quantity	Quantity	Bin 1	Bin 2
Fayetteville warehouse				
HT-100 Two-wheel hand truck	120	40	A-12	
GC-300 Steel garden cart	18	25	B-03	B-04
Tulsa warehouse				
HT-100 Two-wheel hand truck	60	30	C-01	
GC-400 Poly garden cart	none	5	C-07	
GC-400 Poly garden cart	none	5	C-07	

a. Name each property of a relation (Table 2.2) that the sheet breaks, and point to the cells, rows, or columns that break it. Count a repeating group as a break of the first property.
b. For each break, give one question about the stock that the sheet would answer wrongly, or could not answer, because of it.
c. Describe the changes that would turn the sheet into tables that keep all six properties. You need not choose keys.

Exercise 2.4. The `warehouse` table of stage D4 has the columns `warehouse_pk`, `warehouse_code`, `city`, and `state`, and holds two rows: W-FAY in Fayetteville, Arkansas, and W-TUL in Tulsa, Oklahoma. HG Industries assigns each warehouse a code that is never reused, and it could open a second warehouse in a city where it already has one. Identify every candidate key of the table, its primary key, and its alternate keys, and

explain why `city` is not a candidate key even though the two rows have different cities.

Exercise 2.5. Using Listing 2.3, answer these questions about the `order_line` table.

a. Why is `line_number` alone not a candidate key?
b. Name every candidate key of the table.
c. The table has two foreign keys. For each, name the table it refers to and state in words what the reference means for the business.

Exercise 2.6. The database is at stage D2 (Listing 2.1 and Table 2.5). For each change below, state whether PostgreSQL accepts or rejects it, and which integrity rule or key decides.

a. Adding an order numbered SO-5013 for customer 8 (Northgate Hardware).
b. Adding an order numbered SO-5001 for customer 2.
c. Removing customer 8.
d. Removing customer 3 (Cedar Creek Home Center).
e. Changing the `customer_fk` of order SO-5002 to 12.

Exercise 2.7. At stage D4, the `stock_level` table refers to the `warehouse` table through `warehouse_fk` (Listing 2.4), and each of the two warehouses holds four stock levels.

a. Build a table like Table 2.6 for the `warehouse` and `stock_level` tables.
b. HG Industries closes its Tulsa warehouse (W-TUL) and asks for it to be removed from the database. What does PostgreSQL do, and why?
c. Suppose `warehouse_fk` had been declared to cascade deletions. What would removing W-TUL do at stage D4? Stage D5 adds the `shipment` table, whose `warehouse_fk` also refers to `warehouse`

(Listing 2.4). Why might the company regret the same choice on `shipment.warehouse_fk`?

d. Suppose instead that `warehouse_fk` had been declared to set NULL. What would happen when W-TUL is removed, and why?

Exercise 2.8. Using Listing 2.3 and Listing 2.4, list the tables, and the key columns that connect them, needed to answer each request. Do not write any SQL.

a. Which warehouse shipped order SO-5004?
b. How many units of the HT-100 hand truck have been ordered but not yet shipped?
c. Which customers have received a shipment from the Tulsa warehouse?

Exercise 2.9. Connect to your `dbconcepts` database with DataGrip or DBeaver, and find the `stock_level` table in the `distribution` schema. List its columns with their data types, its primary key, its foreign keys and the tables they refer to, and its other constraints. Then write the complete relational schema of `stock_level` in the form of Listing 2.2, and compare it with the `stock_level` box of Figure 2.2. What does the relational schema say that the box does not?

Exercise 2.10. Open the `orders`, `order_details`, `products`, and `shippers` tables of the Northwind database of Section C.11, loaded with the book's loader (Section A.6), in DataGrip or DBeaver, and read their definitions.

a. Write the four tables in the relational schema notation of Section 2.4: columns, primary keys, alternate keys, and each foreign key with the table it refers to and whether it may be NULL.
b. Name two ways Northwind's conventions differ from this book's (Section C.3).

c. `orders.customer_id` may be NULL. What would an order with no customer mean, and what would you ask the business before allowing it?

Exercise 2.11. The query of Table 2.9 lists the columns of `customer_order`. Change it to list the columns of `order_line` instead, run it in your `dbconcepts` database, and compare the result with the `order_line` entry of Listing 2.3.

a. Which columns of `order_line` may be NULL?
b. Name one thing the result tells you that the relational schema of Listing 2.3 does not, and one thing the relational schema tells you that the result does not.
c. The result gives no length for `unit_price`, which is declared `NUMERIC(10,2)`. Look through the columns of `information_schema.columns` in DataGrip or DBeaver, and name the two that record the 10 and the 2.

Exercise 2.12. A process engineer keeps a log of gauge calibrations in a spreadsheet whose headings are Gauge, Date, Standard, Reading, and Technician. Each row records one calibration of one gauge against a reference standard.

a. Name three facts about this data that the catalog of a database holding the log would record and the spreadsheet does not.
b. Name one fact about the readings that neither the spreadsheet nor the catalog records, and say where a database designer would record it.
c. A colleague renames the heading Reading to Reading (mm) and adds a column of readings in inches beside it. Explain what a database's catalog would make the colleague do differently, and why that matters to the people who read the log.

Exercise 2.13. Suppose that HG Industries stored every order of stage D5 as one JSON document in the form of Listing 2.5, and nothing else.

a. Riverside Hardware (C-1001) has placed three orders. How many times is its city stored in the relational database of stage D5, and how many times in the documents?
b. HG Industries renames the GC-300 from “Steel garden cart” to “Steel utility cart”. The GC-300 appears on eight orders. How many rows change in the relational database, and how many documents change? What goes wrong if one document is missed?
c. The GC-400 poly garden cart is in the `product` table of stage D5 but has never been ordered. Where would its description and list price be stored if the orders were the only documents? Which problem of Section 1.1 is this?
d. A hierarchical database stores HG’s bill of materials as trees, one for each product, with every part under the assembly it goes into. At stage P2, the wheel assembly WA-10 goes into all four products, and it has four components of its own. Describe two ways the wheel assembly could be stored, and what each costs when HG Industries changes the wheel assembly’s tire.

3. Querying Tables

Learning objectives

After reading this chapter you should be able to:

- write SELECT statements that choose columns, filter rows, and sort results
- evaluate a query's conditions by hand, row by row, before running it
- handle NULL values correctly in conditions and expressions
- compute derived columns with arithmetic, string, and date functions
- write inner and outer joins across two or more tables
- choose between a join and a subquery for a given request, and write correlated subqueries with EXISTS and IN
- combine query results with UNION, INTERSECT, and EXCEPT
- structure a long query with common table expressions

Most of what an engineer asks of a database is a question. Which orders are waiting to ship? Which work orders finished late? Which stock levels have fallen below their reorder points? Each question is answered by a query, a statement that describes the rows wanted and leaves the database to find them. This chapter presents the query language that nearly every relational database shares, SQL, and its central statement, `SELECT`.

Chapter 2 showed how to read a database: its tables, their keys, and the

foreign keys that connect them. This chapter puts that reading to work. We begin with queries on a single table: choosing its columns, keeping the rows that satisfy a condition, handling the NULLs that some rows hold, sorting the result, and computing new values from old ones. We then combine tables, first with joins and then with subqueries, set operations, and common table expressions. Chapter 4 takes up summarizing many rows into one.

A habit runs through the whole chapter. Before the database evaluates a query, we evaluate it ourselves, by hand, on a few rows, and write down what each step produces. The query is then run, and its result must match ours. The habit catches most mistakes in a query before they reach a report, and it builds the understanding that makes a long query readable.

Following along

The examples run in your `dbconcepts` database, loaded with `through-q.sql` (Section A.5), which holds every table the chapter mentions. If you are not sure what it holds, run `code/sql/setup/which-stage.sql`; it says whether a reset is needed (Section C.2).

3.1. The SELECT statement

This section presents the `SELECT` statement: first the shape that every query shares, and then its simplest form, which reads columns from one table.

3.1.1. The shape of a query

Every query in this chapter is built from `SELECT` statements, and each `SELECT` statement has the same shape. Listing 3.1 gives that shape as a

template, with each clause in the order in which it is written.

Listing 3.1 The Shape of a SELECT Statement

```
SELECT [DISTINCT] column, expression AS alias, ...
FROM schema.table
[WHERE condition]
[GROUP BY column, ...]
[HAVING condition]
[ORDER BY column [DESC], ...]
[LIMIT count];
```

A template is read as follows. Words in capitals, such as `SELECT` and `FROM`, are keywords, written as shown. Words in lower case, such as `column` and `condition`, stand for what you supply: a column's name, or a condition on a row. Square brackets mark an optional part, and they are not typed; the three dots mean that the item before them may be repeated, separated by commas. Thus, the `SELECT` and `FROM` lines are the only ones that every query of this chapter has, and the first queries of this section use nothing else.

A query answers a question, and each clause answers part of it. Table 3.1 lists the clauses, the part of the question that each answers, and where the book presents it.

Table 3.1.: The Clauses of a SELECT Statement

Clause	Answers	Presented in
`SELECT`	Which columns, or which computed values?	This section; Section 3.5

Clause	Answers	Presented in
`FROM`	From which table, or which tables joined?	This section; Section 3.6
`WHERE`	Which rows?	Section 3.2
`GROUP BY`	Into which groups?	Chapter 4
`HAVING`	Which groups to keep?	Chapter 4
`ORDER BY`	In what order?	Section 3.4
`LIMIT`	How many rows?	Section 3.4

For example, the request "the number, description, and list price of each product under $125" has one part beyond its columns and its table. "Under $125" says which rows, so it becomes a `WHERE` clause, and Section 3.2 writes the query. Had the request added "cheapest first", that part would say in what order, and it would become an `ORDER BY` clause. Reading a request against Table 3.1 in this way tells you which clauses its query needs before you write any of them. `LIMIT` is PostgreSQL's keyword rather than the standard's, as Section 3.4 explains.

The clauses are written in the order of Listing 3.1, but the database, at least in principle, evaluates them in another:

1. `FROM` names the table the rows come from.
2. `WHERE` keeps the rows that satisfy its condition (Section 3.2).
3. `GROUP BY` divides the rows that were kept into groups, and `HAVING` keeps the groups that satisfy its condition (Chapter 4).
4. `SELECT` computes the columns of the result from each row, or each group, that was kept.
5. `DISTINCT`, if present, removes repeated rows.
6. `ORDER BY` sorts the result, and `LIMIT` keeps its first rows (Section 3.4).

This order explains several rules that otherwise seem arbitrary, and the following sections refer back to it. Notice that `SELECT`, written first, is

evaluated nearly last. Thus, a new name that the `SELECT` line gives a column does not yet exist while `FROM` and `WHERE` are evaluated.

A few rules of writing apply to every statement, whatever its clauses:

- Keywords ignore case: `select`, `Select`, and `SELECT` are the same keyword. The book writes keywords in capitals, so that they stand apart from names.
- PostgreSQL folds names to lower case unless they are written in double quotes, so `LIST_PRICE` and `list_price` name the same column. (The standard folds them to upper case; the effect is the same while names go unquoted.) The book's names are in lower case, with underscores between words, so they need no quotes.
- A text value is written in single quotes, as in `'C-1001'`; a number is not quoted, as in `125`. Double quotes mark a name, so `"C-1001"` asks for a column of that name, and PostgreSQL refuses it.
- A statement ends with a semicolon, which tells the tool where one statement stops and the next begins.
- Outside quoted text and comments, line breaks and indentation do not change a statement's meaning. The book puts each clause on its own line, so that each clause can be read against its row of Table 3.1.
- Two hyphens (`--`) start a comment, which runs to the end of the line and which the database ignores.
- A table is named with its schema, as in `distribution.product`. Recall from Section 2.4 that the book's tables belong to schemas, one for each department.

3.1.2. Choosing columns from one table

The simplest query fills in only the first two lines of Listing 3.1. It names the columns it wants after the keyword `SELECT` and the table that holds them after the keyword `FROM`. The following query lists every product's number, description, and list price.

Table 3.2.: The Products and Their List Prices

```
SELECT product_number, description, list_price
FROM distribution.product;
```

product_number	description	list_price
HT-100	Two-wheel hand truck	89.00
HT-200	Convertible hand truck	139.00
GC-300	Steel garden cart	119.00
GC-400	Poly garden cart	149.00

Table 3.2 shows the four products HG Industries sells. The query named three of the table's four columns, so the result has three columns, in the order the query named them; the surrogate key `product_pk` does not appear because the query did not ask for it. Writing `SELECT *` instead asks for every column, in the order the table defines them, as the queries of Chapter 2 did. The star is convenient for a first look at a table. In a query that you keep, however, you should name the columns, so that the query still returns what its reader expects after someone adds a column to the table.

Notice that the result of a query is itself a table: it has named columns and a set of rows. That is what lets queries be combined, as the later sections of this chapter show. The result differs from a stored table in one respect. Recall from Section 2.1 that a table with a key cannot hold two identical rows. A query result has no key, so it can.

For example, each shipment records the carrier that took it. The following query lists the carrier of every shipment.

Table 3.3.: The Carrier of Every Shipment

```
SELECT carrier
FROM distribution.shipment;
```

carrier
Boston Mountain Freight
Tri-State Logistics
Customer pickup
Boston Mountain Freight
Boston Mountain Freight
Tri-State Logistics
Tri-State Logistics
Tri-State Logistics
Customer pickup
Boston Mountain Freight
Boston Mountain Freight

Table 3.3 has eleven rows, one for each shipment, and many of them repeat. Adding the keyword `DISTINCT` after `SELECT`, the optional word in the first line of Listing 3.1, removes the repeats, so that each different row appears once.

Table 3.4.: The Different Carriers

```
SELECT DISTINCT carrier
FROM distribution.shipment;
```

carrier
Customer pickup
Tri-State Logistics
Boston Mountain Freight

Table 3.4 shows that the eleven shipments went by three carriers, counting a customer who picks up an order as a carrier. `DISTINCT` compares whole rows: with two columns named, it removes a row only when both of its values repeat another row's. Thus, `DISTINCT` answers "which different values occur", and it is the first tool to reach for when a result repeats rows that the request does not want repeated.

The three rows come back in no particular order. A result has an order only when the query asks for one with `ORDER BY` (Section 3.4), so your own run, on another release of PostgreSQL, may list the carriers in another order.

A column of the result can be given a new name with `AS`, the form `expression AS alias` of Listing 3.1. For example, `SELECT list_price AS price` names the column `price`. A new name is called a **column alias**. It matters little for a stored column, but it is essential for a computed one, as Section 3.5 shows.

Recall the order of evaluation from Section 3.1.1. A column alias is created in step 4, when `SELECT` is evaluated, so the `WHERE` clause of step 2 cannot use it, while the `ORDER BY` clause of step 6 can. The next section presents step 2.

3.2. Filtering rows

This section presents the `WHERE` clause, which keeps the rows of a table that satisfy a condition and discards the rest.

A `WHERE` clause holds a **condition**, an expression that is true or false for each row. The database evaluates the condition once for every row that the `FROM` clause supplies, and it keeps a row only when the condition is true. The simplest conditions compare a column with a value using the comparison operators `=`, `<>` (not equal), `<`, `<=`, `>`, and `>=`.

Let's evaluate a condition by hand before the database does. Suppose that we want the products whose list price is under $125. The condition is `list_price < 125`, and the table has four rows. We evaluate the condition on each row in turn and record the result, with a column for the condition's value and a column for whether the row is kept:

Table 3.5.: Evaluating a Condition on Each Product

Product	List price	Under $125?	Kept?
HT-100	89.00	true	yes
HT-200	139.00	false	no
GC-300	119.00	true	yes
GC-400	149.00	false	no

Table 3.5 predicts two rows, HT-100 and GC-300. The query produces the same two:

Table 3.6.: Products with a List Price Under $125

```
SELECT product_number, description, list_price
FROM distribution.product
WHERE list_price < 125;
```

product_number	description	list_price
HT-100	Two-wheel hand truck	89.00
GC-300	Steel garden cart	119.00

Notice that the condition in Table 3.5 is evaluated on the whole row, although the query returns only three of its columns. The `WHERE` clause of step 2 sees every column of the table, whether or not the `SELECT` list of step 4 names it.

A condition may also compare two columns of the same row. That is often what an engineering question needs, as the following example illustrates.

Example 3.1 (Finding the stock levels that need reordering)**.** *The system.* HG Industries keeps a stock level for each product in each of its two warehouses, in the `stock_level` table. Each row records the quantity on hand and a reorder point, the quantity at or below which the warehouse should order more.

The request. Which stock levels are at or below their reorder points?

The request compares two columns of the same row, so the condition is `quantity_on_hand <= reorder_point`. Let's evaluate it by hand on the eight rows of the table:

Table 3.7.: Evaluating the Reorder Condition by Hand

Warehouse	Product	On hand	Reorder point	Condition
1	1	120	40	false
1	2	35	20	false
1	3	18	25	true
1	4	30	10	false
2	1	60	30	false
2	2	15	10	false
2	3	40	20	false
2	4	0	5	true

Table 3.7 predicts two rows: product 3 in warehouse 1 and product 4 in warehouse 2. The query is:

Table 3.8.: Stock Levels at or Below Their Reorder Points

```
SELECT warehouse_fk, product_fk,
       quantity_on_hand, reorder_point
FROM distribution.stock_level
WHERE quantity_on_hand <= reorder_point;
```

warehouse_fk	product_fk	quantity_on_hand	reorder_point
1	3	18	25
2	4	0	5

The check. Table 3.8 holds the two rows that Table 3.7 predicted, and no others.

Interpretation. Recall from Table 3.2 that product 3 is the GC-300 steel garden cart and product 4 the GC-400 poly garden cart. The Fayetteville

warehouse (warehouse 1) is short of steel carts, and the Tulsa warehouse (warehouse 2) has no poly carts at all. Both should be replenished, and the Tulsa shortage is the more urgent, since an order for a poly cart there cannot be filled.

Where it breaks. The query sees only the quantity on hand. If a replenishment order has already been placed for the steel carts, the stock level is not really a problem; the database does not record replenishment orders, so no query of it can tell.

Conditions combine with `AND`, `OR`, and `NOT`. A condition joined by `AND` is true when both of its parts are true; one joined by `OR` is true when either part is true; and `NOT` reverses a condition. When a condition mixes `AND` and `OR`, the database evaluates the `AND` first, just as arithmetic evaluates multiplication before addition. That rule is a frequent source of wrong answers.

For example, suppose that the Tulsa warehouse manager asks for the stock levels in warehouse 2 that are at or below their reorder points or have fewer than 20 units on hand. Written as it is spoken, the condition is:

Table 3.9.: The Condition Without Parentheses

```
SELECT warehouse_fk, product_fk,
       quantity_on_hand, reorder_point
FROM distribution.stock_level
WHERE warehouse_fk = 2 AND quantity_on_hand <= reorder_point
   OR quantity_on_hand < 20;
```

warehouse_fk	product_fk	quantity_on_hand	reorder_point
1	3	18	25
2	2	15	10
2	4	0	5

Table 3.9 includes a row from warehouse 1, which the manager did not ask for. The database read the condition as `(warehouse_fk = 2 AND quantity_on_hand <= reorder_point) OR quantity_on_hand < 20`, so any row with fewer than 20 units qualifies, in either warehouse. Parentheses say what the manager meant:

Table 3.10.: The Condition with Parentheses

```
SELECT warehouse_fk, product_fk,
       quantity_on_hand, reorder_point
FROM distribution.stock_level
WHERE warehouse_fk = 2
  AND (quantity_on_hand <= reorder_point
       OR quantity_on_hand < 20);
```

warehouse_fk	product_fk	quantity_on_hand	reorder_point
2	2	15	10
2	4	0	5

Thus, whenever a condition mixes `AND` and `OR`, you should write the parentheses, even where the database would group the parts the same way without them. They cost nothing, and they tell the next reader which grouping you intended.

SQL has shorter ways to write three common conditions. `BETWEEN` tests a range, and it includes both ends: `order_date BETWEEN DATE '2026-02-01' AND DATE '2026-02-28'` is the same as `order_date >= DATE '2026-02-01' AND order_date <= DATE '2026-02-28'`. `IN` tests membership in a list: `state IN ('MO', 'OK')` is the same as `state = 'MO' OR state = 'OK'`. And `LIKE` matches a pattern, in which `%` stands for any run of characters (including none) and `_` for exactly one character. The following query uses `BETWEEN` to find the orders placed in February 2026.

Table 3.11.: Orders Placed in February 2026

```
SELECT order_number, order_date, requested_ship_date
FROM distribution.customer_order
WHERE order_date BETWEEN DATE '2026-02-01'
                     AND DATE '2026-02-28';
```

order_number	order_date	requested_ship_date
SO-5007	2026-02-03	2026-02-10
SO-5008	2026-02-11	NULL
SO-5009	2026-02-18	2026-02-25
SO-5010	2026-02-24	2026-03-03

Notice how the query writes a date: the keyword `DATE` followed by the date in quotes, year first. That form is standard SQL. PostgreSQL also accepts the quoted date alone, `'2026-02-01'`, when it can tell from the column that a date is meant, and you will see that shorter form in other people's queries.

`BETWEEN` has a trap for columns that hold a time as well as a date. The `quality.inspection` table records when each inspection happened in its `inspected_at` column, which is a `TIMESTAMP`. The condition `inspected_at BETWEEN DATE '2026-02-01' AND DATE '2026-02-28'` compares each timestamp with midnight at the start of February 28, so it misses every inspection made later that day. The safe way to write a range of days is a closed start and an open end: `inspected_at >= DATE '2026-02-01' AND inspected_at < DATE '2026-03-01'`.

`LIKE` has a trap of its own: in PostgreSQL it distinguishes upper case from lower case. The following query looks for the parts whose descriptions mention a cart.

Table 3.12.: Parts Whose Descriptions Contain 'cart'

```
SELECT part_number, description
FROM production.part
WHERE description LIKE '%cart%';
```

part_number	description
GC-300	Steel garden cart
GC-400	Poly garden cart

Table 3.12 finds the two garden carts but not part CF-310, whose description is "Cart frame": the pattern asks for a lower-case `c`. Converting the description to lower case before matching finds all three:

Table 3.13.: The Same Search, Ignoring Case

```
SELECT part_number, description
FROM production.part
WHERE LOWER(description) LIKE '%cart%';
```

part_number	description
GC-300	Steel garden cart
GC-400	Poly garden cart
CF-310	Cart frame

`LOWER` is a function, one of those Section 3.5 presents. PostgreSQL also offers `ILIKE`, a `LIKE` that ignores case, but `ILIKE` is not standard SQL, and `LOWER` works in every database. The next section takes up the one kind of value for which none of these conditions behaves as expected: NULL.

3.3. NULL and three-valued logic

This section presents how SQL evaluates a condition when a value in it is NULL, and how to write conditions that handle NULL correctly.

Recall from Section 2.1 that NULL marks a value that is unknown or does not apply. In the `production.work_order` table, a work order that is still in process has no completion date, so its `completed_on` column is NULL. Consider the condition `completed_on > due_on`, which asks whether a work order was finished after it was due. For an open work order, the database cannot say whether the condition is true or false, because it does not know the completion date. SQL therefore gives the condition a third value, **unknown**. That is, a comparison in which either side is NULL is neither true nor false; it is unknown.

A system of logic with three values is called **three-valued logic**. The rules for combining unknown with `AND`, `OR`, and `NOT` follow from one idea: unknown stands for a value that is really true or false, we just do not know which. For example, false `AND` anything is false, whatever the anything turns out to be, so false `AND` unknown is false. True `AND` unknown could be either, so it is unknown. The complete rules are:

Table 3.14.: The Truth Table of AND

`AND`	true	false	unknown
true	true	false	unknown
false	false	false	false
unknown	unknown	false	unknown

Table 3.15.: The Truth Table of OR

`OR`	true	false	unknown
true	true	true	true
false	true	false	unknown
unknown	true	unknown	unknown

Table 3.16.: The Truth Table of NOT

	true	false	unknown
`NOT`	false	true	unknown

Read Table 3.14 by finding the left operand in a row and the right operand in a column. The rule that matters most for queries is this: **a `WHERE` clause keeps a row only when its condition is true.** A row whose condition is false is discarded, and so is a row whose condition is unknown.

Let's evaluate the late-completion condition by hand on the five work orders released on or after February 9, 2026:

Table 3.17.: Work Orders Released on or After February 9

```
SELECT work_order_number, due_on, completed_on
FROM production.work_order
WHERE released_on >= DATE '2026-02-09';
```

work_order_number	due_on	completed_on
WO-7006	2026-02-20	2026-02-24
WO-7007	2026-02-27	2026-02-26
WO-7008	2026-03-13	NULL
WO-7009	2026-03-16	NULL
WO-7010	2026-03-27	NULL

For each row of Table 3.17, we evaluate the condition `completed_on > due_on`, and then its negation, `NOT (completed_on > due_on)`:

Table 3.18.: Evaluating the Late-Completion Condition by Hand

Work order	Due	Completed	Condition	Negation
WO-7006	2026-02-20	2026-02-24	true	false
WO-7007	2026-02-27	2026-02-26	false	true
WO-7008	2026-03-13	NULL	unknown	unknown
WO-7009	2026-03-16	NULL	unknown	unknown
WO-7010	2026-03-27	NULL	unknown	unknown

Table 3.18 predicts that a query for late work orders among these five returns WO-7006 alone, and that a query for the work orders that are *not* late returns WO-7007 alone. The two queries together return two of the five rows. The three open work orders are in neither result, because both conditions are unknown for them. The queries confirm it:

Table 3.19.: Late Work Orders Among the Five

```
SELECT work_order_number, due_on, completed_on
FROM production.work_order
WHERE released_on >= DATE '2026-02-09'
  AND completed_on > due_on;
```

work_order_number	due_on	completed_on
WO-7006	2026-02-20	2026-02-24

Table 3.20.: Work Orders Among the Five That Are Not Late

```
SELECT work_order_number, due_on, completed_on
FROM production.work_order
WHERE released_on >= DATE '2026-02-09'
  AND NOT (completed_on > due_on);
```

work_order_number	due_on	completed_on
WO-7007	2026-02-27	2026-02-26

Notice what Table 3.19 and Table 3.20 together say. In ordinary logic, a condition and its negation divide the rows between them, and every row lands in exactly one result. In SQL, a row can land in neither. Thus, whenever a column in a condition may be NULL, you should ask what the query should do with the rows where it is, and write the condition to say so.

To test for NULL, SQL provides `IS NULL` and `IS NOT NULL`, which are always true or false, never unknown. The open work orders are the rows where `completed_on IS NULL`. The comparison `completed_on = NULL`

does not work: it compares a value with an unknown value, so it is unknown for every row, and the query returns no rows at all. We ran it to confirm:

```
SELECT work_order_number
FROM production.work_order
WHERE completed_on = NULL;
```

That query returns no rows, although three work orders have no completion date. It is one of the most common mistakes in SQL, and the database gives no warning.

Example 3.2 (Listing the late work orders)**.** *The system.* The production department releases work orders to make parts, in the `production.work_order` table of stage P3. Each work order has a due date and, once it is finished, a completion date.

The request. The plant manager asks, on March 20, 2026, which work orders are late: those finished after their due dates, and those still open although their due dates have passed.

The request has two parts, and the first is the condition we have already evaluated: `completed_on > due_on`. The second part concerns the open work orders, whose `completed_on` is NULL, so it must test for NULL explicitly: `completed_on IS NULL AND due_on < DATE '2026-03-20'`. A work order is late when either part is true, so the parts are joined by `OR`, with parentheses to show the grouping:

Table 3.21.: Work Orders Late as of March 20, 2026

```
SELECT work_order_number, due_on, completed_on
FROM production.work_order
WHERE completed_on > due_on
   OR (completed_on IS NULL AND due_on < DATE '2026-03-20');
```

work_order_number	due_on	completed_on
WO-7003	2026-01-30	2026-02-02
WO-7006	2026-02-20	2026-02-24
WO-7008	2026-03-13	NULL
WO-7009	2026-03-16	NULL

The check. We can count the rows before running the query. Of the ten work orders, seven are complete, and two of those finished after their due dates: WO-7003 (due January 30, finished February 2) and WO-7006 (due February 20, finished February 24). Three are open, and two of them were due before March 20: WO-7008 and WO-7009. Thus, we expect four rows, and Table 3.21 has those four.

Interpretation. Four of the ten work orders released this quarter are late, and two of them are still on the floor. The two open ones, WO-7008 and WO-7009, need the planner's attention now; the two completed ones are history, useful for measuring the plant's on-time performance.

Where it breaks. The date March 20 is written into the query, so the query answers the question as of that day only. A query run every day would use `CURRENT_DATE`, the date on which it runs. In addition, the query trusts `completed_on`: a work order that was finished but never recorded as finished looks open, and the query would call it late.

NULL affects arithmetic as well as conditions: any arithmetic with a NULL

operand produces NULL. Section 3.5 shows the consequence, and a function that replaces a NULL with a value of your choosing.

3.4. Sorting and limiting results

This section presents `ORDER BY`, which sorts the rows of a result, and `LIMIT`, which keeps only the first rows of a sorted result.

Recall from Section 2.1 that the rows of a table have no order. A query result has no order either, unless the query asks for one. The rows of every result in this chapter so far came back in an order that happened to look sensible, but nothing guaranteed it, and the same query may return its rows in a different order tomorrow. When the order matters, the query must end with an `ORDER BY` clause naming the columns to sort by.

`ORDER BY` sorts in ascending order unless a column is followed by `DESC`. Naming several columns sorts by the first, then breaks ties with the second, and so on. The following query lists the stock levels by warehouse, and within each warehouse from the largest quantity on hand to the smallest.

Table 3.22.: Stock Levels Sorted by Warehouse and Quantity

```
SELECT warehouse_fk, product_fk, quantity_on_hand
FROM distribution.stock_level
ORDER BY warehouse_fk, quantity_on_hand DESC;
```

warehouse_fk	product_fk	quantity_on_hand
1	1	120
1	2	35
1	4	30
1	3	18
2	1	60
2	3	40
2	2	15
2	4	0

NULLs need a rule of their own, since a NULL is neither larger nor smaller than any value. The SQL standard leaves the choice to each database. PostgreSQL sorts NULLs as if they were larger than every value: last in ascending order, first in descending order. The following query sorts the orders by their requested ship dates, and the two orders with no requested date come last.

Table 3.23.: Orders Sorted by Requested Ship Date

```
SELECT order_number, order_date, requested_ship_date
FROM distribution.customer_order
ORDER BY requested_ship_date, order_number;
```

order_number	order_date	requested_ship_date
SO-5001	2026-01-05	2026-01-12
SO-5002	2026-01-07	2026-01-14
SO-5004	2026-01-15	2026-01-22
SO-5005	2026-01-21	2026-02-02
SO-5006	2026-01-28	2026-02-04
SO-5007	2026-02-03	2026-02-10
SO-5009	2026-02-18	2026-02-25
SO-5010	2026-02-24	2026-03-03
SO-5011	2026-03-04	2026-03-11
SO-5012	2026-03-10	2026-03-17
SO-5003	2026-01-12	NULL
SO-5008	2026-02-11	NULL

Writing `NULLS FIRST` or `NULLS LAST` after a column overrides the default, and it makes the query's meaning plain in any database. For example, `ORDER BY requested_ship_date NULLS FIRST` would put the two orders without a requested date at the top of Table 3.23.

A sorted result is often wanted only in part: the three largest orders, the five slowest machines. `LIMIT n`, written after `ORDER BY`, keeps the first `n` rows. The following query finds the three order lines with the largest quantities.

Table 3.24.: The Three Largest Order Lines by Quantity

```
SELECT customer_order_fk, line_number, product_fk, quantity
FROM distribution.order_line
ORDER BY quantity DESC
LIMIT 3;
```

customer_order_fk	line_number	product_fk	quantity
11	1	1	30
6	1	1	25
4	1	1	20

`LIMIT` is PostgreSQL's keyword, shared by several other databases. The SQL standard writes the same thing `FETCH FIRST 3 ROWS ONLY`, which PostgreSQL also accepts.

Two traps come with `LIMIT`. The first is a `LIMIT` without an `ORDER BY`: it keeps some rows, but which ones is undefined, so the query has no meaning. The second is ties. The following query asks for the highest unit price on any order line.

Table 3.25.: The Highest Unit Price, with LIMIT 1

```
SELECT order_line_pk, customer_order_fk, line_number,
       unit_price
FROM distribution.order_line
ORDER BY unit_price DESC
LIMIT 1;
```

order_line_pk	customer_order_fk	line_number	unit_price
3	2	1	139.00

Table 3.25 shows one line, but four order lines share the highest price, $139.00, and the query chose one of them without saying so. Which one it chose is not defined. The standard's form has an answer: `WITH TIES` keeps every row that ties with the last one kept.

Table 3.26.: The Highest Unit Price, with Ties

```
SELECT order_line_pk, customer_order_fk, line_number,
       unit_price
FROM distribution.order_line
ORDER BY unit_price DESC
FETCH FIRST 1 ROW WITH TIES;
```

order_line_pk	customer_order_fk	line_number	unit_price
15	8	1	139.00
22	12	2	139.00
3	2	1	139.00
18	10	1	139.00

Thus, when you ask for the "top" rows of anything, you should decide what a tie means for the request, and write the query to match. Table 3.26 answers "which lines have the highest price"; Table 3.25 answers a question nobody asked.

Recall from Section 3.1 that `ORDER BY` is evaluated after `SELECT`, so it may sort by a column alias defined in the `SELECT` list. The next section creates columns that need one.

3.5. Expressions and built-in functions

This section presents expressions that compute new values from the columns of a row, and the built-in functions that SQL provides for numbers, text, and dates.

The `SELECT` list may hold any expression, not only a column name. The arithmetic operators are `+`, `-`, `*`, and `/`, and the database evaluates an expression once for each row that the `WHERE` clause kept. For example, the value of an order line is its quantity times its unit price, which the following query computes for the lines of order SO-5004 (order 4).

Table 3.27.: The Value of Each Line of Order SO-5004

```
SELECT line_number, product_fk, quantity, unit_price,
       quantity * unit_price AS extended_price
FROM distribution.order_line
WHERE customer_order_fk = 4
ORDER BY line_number;
```

line_number	product_fk	quantity	unit_price	extended_price
1	1	20	89.00	1780.00
2	3	5	119.00	595.00
3	1	10	79.00	790.00

The computed column of Table 3.27 has no name of its own, so the query gives it one with `AS`. Its first row is $20 \times 89.00 = 1780.00$, and the others follow the same way.

Recall from Section 3.1 that a column alias does not exist when the `WHERE` clause is evaluated. A query that tries to keep only the valuable lines by writing `WHERE extended_price > 1000` fails:

```
SELECT customer_order_fk, line_number,
       quantity * unit_price AS extended_price
FROM distribution.order_line
WHERE extended_price > 1000;
```

```
ERROR:  column "extended_price" does not exist
LINE 4: WHERE extended_price > 1000;
              ^
```

The `WHERE` clause must repeat the expression instead: `WHERE quantity *`

`unit_price > 1000`. A common table expression (Section 3.10) offers a way to name an expression once and use the name everywhere.

Division needs care, because the type of the result follows the types of the operands. When both operands are integers, PostgreSQL divides as integers and discards the remainder: `7 / 2` is `3`. For example, a planner might measure each stock level's cover, its quantity on hand divided by its reorder point. A cover of 1.5 means half again as much stock as the reorder point.

Table 3.28.: Stock Cover, with Integer and Decimal Division

```
SELECT warehouse_fk, product_fk,
       quantity_on_hand, reorder_point,
       quantity_on_hand / reorder_point AS integer_cover,
       ROUND(CAST(quantity_on_hand AS NUMERIC)
             / reorder_point, 2) AS cover
FROM distribution.stock_level
ORDER BY cover, warehouse_fk, product_fk;
```

warehouse_fk	product_fk	quantity_on_hand	reorder_point	integer_cover	cover
2	4	0	5	0	0.00
1	3	18	25	0	0.72
2	2	15	10	1	1.50
1	2	35	20	1	1.75
2	1	60	30	2	2.00
2	3	40	20	2	2.00
1	1	120	40	3	3.00
1	4	30	10	3	3.00

The `integer_cover` column of Table 3.28 is wrong in every row whose true cover is not a whole number. For example, 35 units against a reorder point of 20 is a cover of 1.75, but integer division reports 1. The `cover` column converts the quantity to the `NUMERIC` type with `CAST` before dividing, so

the division keeps its fraction, and then `ROUND` rounds the result to two places. Notice also that the query sorts by the alias `cover`, which `ORDER BY` may use, and breaks ties in cover by warehouse and product, so the order of its rows is fixed.

NULL, as Section 3.3 warned, propagates through arithmetic. PostgreSQL subtracts one date from another to give the number of days between them, so `completed_on - released_on` is the number of days a work order took. For an open work order, `completed_on` is NULL, and so is the difference. The function `COALESCE` returns the first of its arguments that is not NULL, which lets a query substitute a value for a missing one. The following query computes both the days each work order took and the days it has been in process as of March 20, 2026.

Table 3.29.: Days in Process for Each Work Order

```
SELECT work_order_number, released_on, completed_on,
       completed_on - released_on AS days_taken,
       COALESCE(completed_on, DATE '2026-03-20')
         - released_on AS days_in_process
FROM production.work_order
ORDER BY released_on;
```

work_order_number	released_on	completed_on	days_taken	days_in_process
WO-7001	2026-01-05	2026-01-15	10	10
WO-7002	2026-01-06	2026-01-12	6	6
WO-7003	2026-01-19	2026-02-02	14	14
WO-7004	2026-01-26	2026-02-05	10	10
WO-7005	2026-02-02	2026-02-09	7	7
WO-7006	2026-02-09	2026-02-24	15	15
WO-7007	2026-02-16	2026-02-26	10	10
WO-7008	2026-03-02	NULL	NULL	18
WO-7009	2026-03-09	NULL	NULL	11
WO-7010	2026-03-16	NULL	NULL	4

In the `days_taken` column of Table 3.29, the three open work orders have NULL, while `days_in_process` counts their days up to March 20. For example, WO-7008 was released on March 2, so it has been in process for 18 days. Subtracting dates this way, and adding a whole number of days to a date with `+`, are PostgreSQL features; the SQL standard does the same with interval types that this book does not use.

Text has functions of its own. `||` joins two strings end to end, `UPPER` and `LOWER` change their case, and `SUBSTRING(text FROM start FOR length)` takes part of one. For date values, `EXTRACT(field FROM date)` takes out a part such as the `YEAR`, `MONTH`, or `DAY`. The following query uses all three kinds on the product table: it takes the family of each product from the first two characters of its number and builds a label for a price list.

Table 3.30.: Product Families and Price-List Labels

```
SELECT product_number,
       SUBSTRING(product_number FROM 1 FOR 2) AS family,
       product_number || ' ' || UPPER(description) AS label
FROM distribution.product;
```

product_number	family	label
HT-100	HT	HT-100 TWO-WHEEL HAND TRUCK
HT-200	HT	HT-200 CONVERTIBLE HAND TRUCK
GC-300	GC	GC-300 STEEL GARDEN CART
GC-400	GC	GC-400 POLY GARDEN CART

The `family` column of Table 3.30 separates the hand trucks (HT) from the garden carts (GC), a grouping that Chapter 4 will summarize. Notice also what the `label` column does: it builds, for display, a value that the table rightly stores as two columns. Recall from Section 2.1 that a value such as "Fayetteville, AR" would break the rule that every stored value is atomic.

A query may combine values freely, because its result is for reading, not for storing.

These functions are a small sample, chosen because industrial data needs them often. PostgreSQL provides hundreds more, described in its documentation, and Appendix F lists the ones this book uses. One more kind of expression, the `CASE` expression, chooses a value by conditions; it appears in Section 4.4, where it is most useful.

Every query in this chapter so far has read one table. The questions of Section 2.4, however, cross several: which customers ordered a product, which warehouse shipped an order. The next section combines tables.

3.6. Inner joins

This section presents the inner join, which combines the rows of two or more tables by matching their keys.

Recall from Section 2.2 that each order's `customer_fk` holds the `customer_pk` of the customer who placed it. A report that lists orders with their customers' names needs both tables, since the name is in `customer` and the order number in `customer_order`. SQL combines them in two steps. First, it pairs every row of one table with every row of the other; second, it keeps the pairs whose keys match. The first step alone is called the **cross product** of the tables, and the two steps together are called a **join**.

Let's carry out both steps by hand on a few rows. We take three customers, Summit Farm Supply, Cedar Creek Home Center, and Northgate Hardware, and the orders placed by the first two:

Table 3.31.: Three Customers

```
SELECT customer_pk, name
FROM distribution.customer
WHERE customer_pk IN (2, 3, 8);
```

customer_pk	name
2	Summit Farm Supply
3	Cedar Creek Home Center
8	Northgate Hardware

Table 3.32.: The Orders of Customers 2 and 3

```
SELECT order_number, customer_fk
FROM distribution.customer_order
WHERE customer_fk IN (2, 3);
```

order_number	customer_fk
SO-5002	3
SO-5003	2
SO-5007	3
SO-5012	2

The cross product pairs each of the three customers of Table 3.31 with each of the four orders of Table 3.32, so it has $3 \times 4 = 12$ rows. For each pair, we evaluate the join condition, that the order's `customer_fk` equals the customer's `customer_pk`:

Table 3.33.: The Cross Product and the Join Condition, by Hand

Customer	Name	Order	Order's customer	Condition
2	Summit Farm Supply	SO-5002	3	false
2	Summit Farm Supply	SO-5003	2	true
2	Summit Farm Supply	SO-5007	3	false
2	Summit Farm Supply	SO-5012	2	true
3	Cedar Creek Home Center	SO-5002	3	true
3	Cedar Creek Home Center	SO-5003	2	false
3	Cedar Creek Home Center	SO-5007	3	true
3	Cedar Creek Home Center	SO-5012	2	false
8	Northgate Hardware	SO-5002	3	false
8	Northgate Hardware	SO-5003	2	false
8	Northgate Hardware	SO-5007	3	false
8	Northgate Hardware	SO-5012	2	false

Table 3.33 keeps four pairs, one for each order, and each pair puts an order beside the customer who placed it. Northgate Hardware appears in no kept pair, since it has placed no order. The cross product itself is written in SQL by listing both tables in the `FROM` clause, separated by a comma, and the join condition goes in the `WHERE` clause:

Table 3.34.: The Join, Written with a Comma

```
SELECT c.customer_pk, c.name, o.order_number
FROM distribution.customer AS c,
     distribution.customer_order AS o
WHERE o.customer_fk = c.customer_pk
  AND c.customer_pk IN (2, 3, 8)
ORDER BY c.customer_pk, o.order_number;
```

customer_pk	name	order_number
2	Summit Farm Supply	SO-5003
2	Summit Farm Supply	SO-5012
3	Cedar Creek Home Center	SO-5002
3	Cedar Creek Home Center	SO-5007

Table 3.34 holds the four pairs that Table 3.33 kept. Notice the letters `c` and `o` after the table names. Each is a **table alias**, a short name for the table within the query, and a column is named with its table's alias, as in `c.name`, whenever the two tables could both have a column of that name. The join condition matches a foreign key with the primary key it refers to, just as the path of Table 2.7 did. In other words, a join follows a foreign key line of the relational schema.

The database does not actually build the cross product; for tables of thousands of rows it would be far too large, and the database finds the matching pairs by quicker means. The cross product is how to *think* about a join, and the result is the same either way. Thus, evaluating a join by hand on a few rows, as Table 3.33 does, is a reliable check on what any join will return.

SQL has a second way to write a join, which separates the join condition from the conditions that filter rows. The keyword `JOIN` names the second

table, and `ON` gives the join condition:

Table 3.35.: The Same Join, Written with JOIN and ON

```
SELECT c.customer_pk, c.name, o.order_number
FROM distribution.customer AS c
JOIN distribution.customer_order AS o
  ON o.customer_fk = c.customer_pk
WHERE c.customer_pk IN (2, 3, 8)
ORDER BY c.customer_pk, o.order_number;
```

customer_pk	name	order_number
2	Summit Farm Supply	SO-5003
2	Summit Farm Supply	SO-5012
3	Cedar Creek Home Center	SO-5002
3	Cedar Creek Home Center	SO-5007

Table 3.35 is the same as Table 3.34. The two forms mean the same thing, and the `JOIN` form is also called an **inner join**, written `INNER JOIN` in full. This book uses the `JOIN` form from here on, for two reasons. It keeps each join condition beside the table it joins, which makes a query of several tables easier to read, and the outer joins of Section 3.7 can be written no other way.

A query may join any number of tables, one `JOIN` at a time. Each join adds a table and a condition that connects it to a table already joined, so a join of four tables needs three conditions. The following example illustrates a join of four tables, and a trap that comes with it.

Example 3.3 (Listing the customers who ordered the steel garden cart)**.** *The system.* The distribution schema of stage D5 (Listing 2.3): customers place orders, each order has lines, and each line is for a product.

The request. The sales manager asks which customers have ordered the GC-300 steel garden cart. This is the request of Example 2.2, where we traced its path through the schema without writing any SQL.

The path of Table 2.7 runs from `product` through `order_line` and `customer_order` to `customer`, so the query joins those four tables, one foreign key at a time:

Table 3.36.: Every Order Line for the GC-300, with Its Customer

```
SELECT c.customer_number, c.name, o.order_number
FROM distribution.product AS p
JOIN distribution.order_line AS l
  ON l.product_fk = p.product_pk
JOIN distribution.customer_order AS o
  ON o.customer_order_pk = l.customer_order_fk
JOIN distribution.customer AS c
  ON c.customer_pk = o.customer_fk
WHERE p.product_number = 'GC-300'
ORDER BY c.customer_number, o.order_number;
```

customer_number	name	order_number
C-1001	Riverside Hardware	SO-5001
C-1001	Riverside Hardware	SO-5004
C-1002	Summit Farm Supply	SO-5003
C-1002	Summit Farm Supply	SO-5012
C-1003	Cedar Creek Home Center	SO-5007
C-1004	Midtown Tool Supply	SO-5005
C-1006	Lakeside Distributors	SO-5011
C-1007	Tri-County Farm and Home	SO-5009

Table 3.36 has eight rows, but the manager asked for customers, and

Riverside Hardware and Summit Farm Supply each appear twice. A join returns one row for each combination of rows that matches, and here that is one row for each order line for the cart. Riverside Hardware ordered the cart on two orders, so it appears twice. To list each customer once, the query drops the order number and asks for distinct rows:

Table 3.37.: The Customers Who Ordered the GC-300

```
SELECT DISTINCT c.customer_number, c.name
FROM distribution.product AS p
JOIN distribution.order_line AS l
  ON l.product_fk = p.product_pk
JOIN distribution.customer_order AS o
  ON o.customer_order_pk = l.customer_order_fk
JOIN distribution.customer AS c
  ON c.customer_pk = o.customer_fk
WHERE p.product_number = 'GC-300'
ORDER BY c.customer_number;
```

customer_number	name
C-1001	Riverside Hardware
C-1002	Summit Farm Supply
C-1003	Cedar Creek Home Center
C-1004	Midtown Tool Supply
C-1006	Lakeside Distributors
C-1007	Tri-County Farm and Home

The check. We can count both results in advance from the order lines of stage D5: eight lines are for product 3, the GC-300, so the first query returns eight rows. Those lines belong to eight orders placed by six different customers, since Riverside Hardware placed two of them (SO-5001 and SO-5004) and Summit Farm Supply two (SO-5003 and SO-5012). Thus,

the second query returns six rows.

Interpretation. Six of the eight customers have bought the steel cart; only Prairie Garden Center and Northgate Hardware have not. Those two are the obvious prospects for a sales call about it.

Where it breaks. `DISTINCT` compares the columns selected, so it must select the key. Had the query selected only `c.name`, two different customers who happened to share a name would have been merged into one row.

The repeated rows of Table 3.36 are called **fan-out**: when a join passes from a table to one that holds many rows for each of its rows (from a customer to its orders, from an order to its lines), each row of the first table is repeated once for each matching row of the second. Fan-out is correct, and often what the request needs, but a query that should count or list the first table's rows must account for it. Thus, before you run a join, you should reason out how many rows each table contributes to each row of the result.

A table can also be joined to itself, which is called a **self-join**. The two copies need different aliases, so that the query can tell them apart. For example, a planner studying which products are ordered together wants the pairs of products that appear on the same order:

Table 3.38.: Pairs of Products Ordered Together

```
SELECT DISTINCT a.product_fk AS first_product,
       b.product_fk AS second_product
FROM distribution.order_line AS a
JOIN distribution.order_line AS b
  ON b.customer_order_fk = a.customer_order_fk
 AND b.product_fk > a.product_fk
ORDER BY first_product, second_product;
```

first_product	second_product
1	2
1	3
2	3

The join condition of Table 3.38 has two parts. The first pairs two lines of the same order; the second keeps each pair once, in one order, and drops the pairing of a line with itself. Without `b.product_fk > a.product_fk`, the result would hold every pair twice, as (1, 3) and (3, 1), and every product paired with itself. The result says that every pair of the three products that have been ordered has appeared together on at least one order. The next section takes up the customers, and the products, that a join leaves out.

3.7. Outer joins

This section presents the outer join, which keeps the rows of a table that have no match in the other.

Recall from Table 3.33 that Northgate Hardware appeared in no pair that the join condition kept, because it has placed no order. An inner join

returns only matched rows, so a report of customers and their orders built from one leaves Northgate Hardware out entirely. For a report of orders that is right. For a report of customers, it is wrong: Northgate Hardware is a customer, and the fact that it has no orders is itself worth reporting.

A **left outer join**, written `LEFT JOIN`, returns every row that the inner join returns, and in addition every row of the table on its left that matched nothing, with NULL in each column of the table on its right. In other words, a left join is an inner join followed by adding back the unmatched rows of the left table. On the three customers of Table 3.31, the inner join kept four rows, and one customer, Northgate Hardware, matched nothing, so the left join returns five:

Table 3.39.: Three Customers and Their Orders, with a Left Join

```
SELECT c.customer_pk, c.name, o.order_number
FROM distribution.customer AS c
LEFT JOIN distribution.customer_order AS o
  ON o.customer_fk = c.customer_pk
WHERE c.customer_pk IN (2, 3, 8)
ORDER BY c.customer_pk, o.order_number;
```

customer_pk	name	order_number
2	Summit Farm Supply	SO-5003
2	Summit Farm Supply	SO-5012
3	Cedar Creek Home Center	SO-5002
3	Cedar Creek Home Center	SO-5007
8	Northgate Hardware	NULL

Figure 3.1 draws the three customers and their orders three ways: the two tables as they are stored, the inner join, and the left join. Each table's columns keep their own shading, and the two keys meet in the middle.

customer

name	customer_pk
Summit Farm Supply	2
Cedar Creek Home Center	3
Northgate Hardware	8

customer_order

customer_fk	order_number
2	SO-5003
2	SO-5012
3	SO-5002
3	SO-5007

JOIN: only the pairs whose keys match

name	customer_pk	customer_fk	order_number
Summit Farm Supply	2	2	SO-5003
Summit Farm Supply	2	2	SO-5012
Cedar Creek Home Center	3	3	SO-5002
Cedar Creek Home Center	3	3	SO-5007

LEFT JOIN: every customer; NULL where no order matches

name	customer_pk	customer_fk	order_number
Summit Farm Supply	2	2	SO-5003
Summit Farm Supply	2	2	SO-5012
Cedar Creek Home Center	3	3	SO-5002
Cedar Creek Home Center	3	3	SO-5007
Northgate Hardware	8	*NULL*	*NULL*

Figure 3.1.: How the Rows of a Join Line Up: the Tables, the Inner Join, and the Left Join

In the inner join of Figure 3.1, every row is a customer row lined up beside one of its orders, and the customer's `customer_pk` stands beside the order's equal `customer_fk`: that equality is the join condition, seen in the result. Summit Farm Supply appears twice because it lines up with two orders. The left join holds the same four rows and one more. Northgate Hardware lined up with no order, so the left join keeps its row and supplies NULL in every column that would have come from `customer_order`, shaded differently in the figure. In other words, the outer join supplies the NULLs; the customer's own data comes from its table as before.

The last row of Table 3.39 is Northgate Hardware, with NULL for its order number. The order of the tables matters: the table named before `LEFT JOIN` is the one whose rows are all kept. A `RIGHT JOIN` keeps every row of the table named after it instead, so `a RIGHT JOIN b` is the same as `b LEFT JOIN a`; this book writes every outer join as a left join, which reads in the order of the request. A `FULL JOIN` keeps the unmatched rows of both tables. It is used to compare two lists, such as a count taken in the warehouse and the stock levels on record, where either list may hold rows the other lacks.

An outer join has a trap in its conditions. Suppose that the sales manager wants every customer with the orders it placed in February 2026, including the customers that placed none. The February condition concerns the orders, the right-hand table. Placed in the `WHERE` clause, it undoes the outer join:

Table 3.40.: The February Condition in WHERE

```
SELECT c.name, o.order_number, o.order_date
FROM distribution.customer AS c
LEFT JOIN distribution.customer_order AS o
  ON o.customer_fk = c.customer_pk
WHERE o.order_date >= DATE '2026-02-01'
  AND o.order_date < DATE '2026-03-01'
ORDER BY c.name;
```

name	order_number	order_date
Cedar Creek Home Center	SO-5007	2026-02-03
Prairie Garden Center	SO-5008	2026-02-11
Riverside Hardware	SO-5010	2026-02-24
Tri-County Farm and Home	SO-5009	2026-02-18

Table 3.40 holds only the four customers that ordered in February. Recall the order of evaluation from Section 3.1: the join, including its added rows, is formed first, and then `WHERE` filters it. The added rows have NULL for `order_date`, so the `WHERE` condition is unknown for them (Section 3.3), and they are discarded, along with every order outside February. Placed in the `ON` clause instead, the condition becomes part of the matching: a customer whose only orders fall outside February has no match, so it is kept with NULLs.

Table 3.41.: The February Condition in ON

```
SELECT c.name, o.order_number, o.order_date
FROM distribution.customer AS c
LEFT JOIN distribution.customer_order AS o
  ON o.customer_fk = c.customer_pk
 AND o.order_date >= DATE '2026-02-01'
 AND o.order_date < DATE '2026-03-01'
ORDER BY c.name;
```

name	order_number	order_date
Cedar Creek Home Center	SO-5007	2026-02-03
Lakeside Distributors	NULL	NULL
Midtown Tool Supply	NULL	NULL
Northgate Hardware	NULL	NULL
Prairie Garden Center	SO-5008	2026-02-11
Riverside Hardware	SO-5010	2026-02-24
Summit Farm Supply	NULL	NULL
Tri-County Farm and Home	SO-5009	2026-02-18

Table 3.41 lists all eight customers, four with a February order and four with NULLs. Thus, in a left join, a condition on the right-hand table belongs in `ON`, and a condition on the left-hand table belongs in `WHERE`.

The NULLs that a left join adds are also a way to find the rows that have no match, as the following example illustrates.

Example 3.4 (Finding the products that have never been ordered)**.** *The system.* The products and order lines of stage D5.

The request. The product manager asks whether any product has never been ordered.

A product that has never been ordered is one with no order line, so it has no match when products are joined to order lines. A left join from `product` keeps it, with NULL in every column of `order_line`. The query keeps exactly those rows by testing a column of `order_line` that is never NULL in a real order line, its primary key:

Table 3.42.: Products That Have Never Been Ordered

```
SELECT p.product_number, p.description
FROM distribution.product AS p
LEFT JOIN distribution.order_line AS l
  ON l.product_fk = p.product_pk
WHERE l.order_line_pk IS NULL;
```

product_number	description
GC-400	Poly garden cart

The check. The 22 order lines of stage D5 are for products 1, 2, and 3 only, so one product, product 4, has no order line, and one row is expected.

Interpretation. The GC-400 poly garden cart has not sold at all in the first quarter. Recall from Example 3.1 that the Tulsa warehouse has none in stock, while Fayetteville holds 30. Nobody has been turned away for want of stock in Tulsa, then; the cart has simply not been ordered.

Where it breaks. The test must use a column that cannot be NULL in a matched row. A column that may be NULL in a real order line would make an ordered product look unordered whenever that column happened to be NULL.

A query that keeps the rows of one table that have no match in another is called an **anti-join**. The left join with `IS NULL` is one way to write it; the next section shows another, and Section 3.9 a third.

3.8. Subqueries

This section presents subqueries, queries written inside another query, and the conditions that use them: comparisons, `IN`, and `EXISTS`.

A **subquery** is a `SELECT` statement in parentheses, placed where a value or a set of values is expected. The simplest returns a single value. For example, the product manager asks which products are priced above the HT-200 convertible hand truck. The outer query needs the HT-200's price, which a subquery looks up:

Table 3.43.: Products Priced Above the HT-200

```
SELECT product_number, list_price
FROM distribution.product
WHERE list_price > (SELECT list_price
                    FROM distribution.product
                    WHERE product_number = 'HT-200');
```

product_number	list_price
GC-400	149.00

The subquery returns one value, 139.00, and the outer query compares each product's price with it. A subquery used this way must return at most one row. If it returned several, the comparison would have no meaning, and PostgreSQL would stop with an error; if it returned none, its value would be NULL. Here the subquery looks a product up by its alternate key, which guarantees one row at most.

A subquery that returns a column of values is used with `IN`. For example, the customers who have ordered in March 2026 are those whose keys appear among the `customer_fk` values of the March orders:

Table 3.44.: Customers Who Ordered in March 2026

```
SELECT customer_number, name
FROM distribution.customer
WHERE customer_pk IN
      (SELECT customer_fk
       FROM distribution.customer_order
       WHERE order_date >= DATE '2026-03-01');
```

customer_number	name
C-1006	Lakeside Distributors
C-1002	Summit Farm Supply

Table 3.44 lists each customer once, however many March orders it placed, because the outer query reads the `customer` table and the subquery only tests membership. A join would return one row per order instead, as Table 3.36 did. Thus, when a request asks for the rows of one table that have some match in another, a subquery with `IN` avoids fan-out without needing `DISTINCT`.

`NOT IN` reverses the test, and it has the worst trap in this chapter. Suppose that the shipping supervisor asks which orders were placed on a day when no order was due to ship, meaning the order date equals no order's requested ship date. The request reads naturally as `NOT IN`:

Table 3.45.: The Request Written with NOT IN

```
SELECT order_number, order_date
FROM distribution.customer_order
WHERE order_date NOT IN (SELECT requested_ship_date
                         FROM distribution.customer_order);
```

order_number	order_date

Table 3.45 is empty, although only one order date, January 12, is also a requested ship date. The cause is NULL. Two orders have no requested ship date, so the subquery's list contains NULL, and `x NOT IN (a, b, NULL)` means `x <> a AND x <> b AND x <> NULL`. The last comparison is unknown for every row, and by Table 3.14 a chain of `AND`s containing an unknown can be false or unknown, but never true. Thus, a `NOT IN` whose subquery can return a NULL returns no rows at all. You should use `NOT IN` only with a subquery whose column is declared `NOT NULL`, or use `NOT EXISTS`, which follows.

`EXISTS` tests whether a subquery returns any row at all. It is usually used with a **correlated subquery**, one that refers to the row of the outer query it is testing. For example, the products with a stock level at or below its reorder point in some warehouse are:

Table 3.46.: Products Short in Some Warehouse

```
SELECT p.product_number, p.description
FROM distribution.product AS p
WHERE EXISTS (SELECT *
              FROM distribution.stock_level AS s
              WHERE s.product_fk = p.product_pk
                AND s.quantity_on_hand <= s.reorder_point);
```

product_number	description
GC-300	Steel garden cart
GC-400	Poly garden cart

Read the subquery of Table 3.46 as being run once for each product, with `p.product_pk` standing for that product's key. For the GC-300, it finds the Fayetteville stock level of Example 3.1, so `EXISTS` is true; for the HT-100, it finds nothing, so `EXISTS` is false. `EXISTS` is never unknown, and the select list of its subquery does not matter, since only whether a row exists is tested; `SELECT *` is the convention.

Notice the table alias in `s.product_fk = p.product_pk`. Every column in a subquery should be named with its table's alias. A column named without one belongs to the subquery's own table when that table has a column of that name, and otherwise to the outer query's table, with no message. Thus, a slip in a column name can make a subquery correlated by accident. For example, suppose the query of Table 3.44 had been written with `customer_pk` in the subquery's select list instead of `customer_fk`:

Table 3.47.: The March Customers, with an Unqualified Column

```
SELECT customer_number, name
FROM distribution.customer
WHERE customer_pk IN
      (SELECT customer_pk
       FROM distribution.customer_order
       WHERE order_date >= DATE '2026-03-01');
```

customer_number	name
C-1001	Riverside Hardware
C-1002	Summit Farm Supply
C-1003	Cedar Creek Home Center
C-1004	Midtown Tool Supply
C-1005	Prairie Garden Center
C-1006	Lakeside Distributors
C-1007	Tri-County Farm and Home
C-1008	Northgate Hardware

The `customer_order` table has no column `customer_pk`, so PostgreSQL takes the name from the outer query's `customer` table. For each customer, the subquery returns that customer's own key once per March order, and the key is always in its own list. Thus, Table 3.47 lists all eight customers instead of the two of Table 3.44, a wrong answer rather than a message. Written with an alias, `o.customer_pk`, the subquery would have been refused.

`NOT EXISTS` is the anti-join of Section 3.7 written as a subquery, and it is not troubled by NULL. The supervisor's request, rewritten with it, returns the orders it should:

Table 3.48.: The Request Written with NOT EXISTS

```
SELECT o.order_number, o.order_date
FROM distribution.customer_order AS o
WHERE NOT EXISTS
      (SELECT *
       FROM distribution.customer_order AS r
       WHERE r.requested_ship_date = o.order_date);
```

order_number	order_date
SO-5001	2026-01-05
SO-5002	2026-01-07
SO-5004	2026-01-15
SO-5005	2026-01-21
SO-5006	2026-01-28
SO-5007	2026-02-03
SO-5008	2026-02-11
SO-5009	2026-02-18
SO-5010	2026-02-24
SO-5011	2026-03-04
SO-5012	2026-03-10

Table 3.48 holds eleven of the twelve orders, all but SO-5003, placed on January 12. For each order, the subquery looks for an order due to ship that day; where the requested ship date is NULL, the comparison is unknown, that row is simply not found, and the test goes on.

One kind of request needs `NOT EXISTS` twice. A request that says *every* ("operators certified on every machine", "customers who have ordered every hand truck") has no keyword in SQL, which can only test whether a row exists. Such a request is written in three steps:

1. **Restate the request as a double negative.** An operator is certified on every machine exactly when there is *no* machine on which the operator is *not* certified.
2. **Write the inner query for one particular object.** For one operator, list the machines on which that operator is not certified. The request holds for that operator exactly when the query returns no rows.
3. **Generalize.** Put the query of step 2 inside `NOT EXISTS`, and replace the particular operator's key with the outer query's operator.

The following example takes the three steps.

Example 3.5 (Finding the operators certified on every machine in a work center)**.** *The system.* The production schema of stage P3. The Machining work center (work center 2) has two machines, the CNC lathe M-02 and the drill press M-03. The `certification` table records which operators are certified on which machines.

The request. The production supervisor, planning a weekend shift that needs one operator to run the whole Machining work center, asks which operators are certified on every machine in it.

Step 1 restates the request: an operator qualifies when there is no machine of Machining on which the operator is not certified. Step 2 begins with one operator. For James Carter (operator 2), the machines of Machining on which he is *not* certified are those for which no certification row pairs him with the machine:

```
SELECT m.machine_number
FROM production.machine AS m
WHERE m.work_center_fk = 2
  AND NOT EXISTS (SELECT *
                  FROM production.certification AS c
                  WHERE c.operator_fk = 2
                    AND c.machine_fk = m.machine_pk);
```

If that query returns no machine, James Carter is certified on every machine in Machining. Step 3 asks the question of every operator at once: we put the query inside `NOT EXISTS` and replace the operator's key, 2, with the outer query's operator:

Table 3.49.: Operators Certified on Every Machine in Machining

```
SELECT o.employee_number, o.name
FROM production.operator AS o
WHERE NOT EXISTS
      (SELECT *
       FROM production.machine AS m
       WHERE m.work_center_fk = 2
         AND NOT EXISTS
             (SELECT *
              FROM production.certification AS c
              WHERE c.operator_fk = o.operator_pk
                AND c.machine_fk = m.machine_pk));
```

employee_number	name
E-202	James Carter

The check. We evaluate the request by hand from the certifications of stage P3. For each operator we ask whether each Machining machine has a certification, and whether some machine lacks one:

Table 3.50.: Checking Each Operator Against the Machining Machines

Operator	M-02	M-03	A machine lacks certification?	Qualifies?
Maria Lopez	no	no	yes	no
James Carter	yes	yes	no	yes

Operator	M-02	M-03	A machine lacks certification?	Qualifies?
Linda Nguyen	no	no	yes	no
Robert Hayes	no	no	yes	no
Aisha Patel	no	no	yes	no
Tom Becker	no	no	yes	no

Table 3.50 predicts one row, James Carter, and Table 3.49 has that one row.

Interpretation. Only James Carter can run the Machining work center alone. The weekend shift depends on one person, and cross-training a second operator on the lathe and the drill press would remove that dependence.

Where it breaks. The double negative behaves strangely when the work center has no machines. The inner query then finds no machine lacking a certification for anyone, so every operator qualifies. The Assembly work center (work center 5) has no machines in stage P3, and the same query for it returns all six operators. Whether that is the right answer depends on the request, and a query for "every" should be checked on an empty set before it is trusted.

A request that says "every" can also be answered by counting: an operator qualifies when the number of Machining machines on which the operator is certified equals the number of Machining machines. Counting needs the aggregate functions of Chapter 4, which takes up that method and shows that it disagrees with the double negative exactly when the set is empty.

The double negative of step 1 is not a trick; it follows from a rule of logic about the two phrases such requests use. "For every" is called the **universal quantifier**, and "there is", meaning there is at least one, the **existential quantifier**. SQL's `EXISTS` is the existential quantifier, and SQL has no universal one. The rule is that the word *not* can move past a quantifier if the quantifier changes to the other one. "It is not true that every machine has a certification" says the same as "there is a machine

that does not have a certification", and "there is no machine without a certification" says the same as "every machine has a certification". Written with the symbols of logic, $\forall$ for "for every", $\exists$ for "there is", and $\neg$ for "not", the request of Example 3.5 is

$$\forall m \in M : \ C(m) \quad \text{is the same as} \quad \neg\, \exists m \in M : \ \neg\, C(m),$$

where M is the set of Machining machines and $C(m)$ says that the operator is certified on machine m. The right-hand side is the query: the outer `NOT EXISTS` is $\neg\exists m$, and the inner `NOT EXISTS` is $\neg C(m)$.

The rule applies one quantifier at a time, so it restates a request that mixes the two quantifiers as well. To put *not* in front of any such statement:

1. Write *not* in front of the whole statement.
2. While a quantifier stands to the right of the *not*, move the *not* past it and change the quantifier: "for every" becomes "there is", and "there is" becomes "for every".
3. When no quantifier remains to its right, join the *not* to the condition that follows it.

For example, the production supervisor asks which work centers have, for every machine, some operator certified on it. Step 1 of the recipe needs the request as a double negative. Negating "some operator is certified on the machine" by the rule gives "every operator is not certified on the machine", or more plainly, "no certification exists for the machine". Thus, a work center qualifies when there is no machine in it for which no certification exists, and the two `NOT EXISTS` follow the words:

Table 3.51.: Work Centers in Which Every Machine Has a Certified Operator

```
SELECT w.name
FROM production.work_center AS w
WHERE NOT EXISTS
      (SELECT *
       FROM production.machine AS m
       WHERE m.work_center_fk = w.work_center_pk
         AND NOT EXISTS
             (SELECT *
              FROM production.certification AS c
              WHERE c.machine_fk = m.machine_pk))
ORDER BY w.name;
```

name
Assembly
Machining
Paint
Saw

The Saw, Machining, and Paint work centers each have at least one certification on every machine. Welding does not appear, so some machine of Welding has no certified operator; Exercise 3.16 asks which. Assembly qualifies too, since it has no machines at all; as in Example 3.5, the double negative is true of an empty set, and the supervisor should read that row as “nothing to staff”, not as “fully staffed”.

3.9. Set operations

This section presents the set operations `UNION`, `INTERSECT`, and `EXCEPT`, which combine the results of two queries.

Recall from Section 3.1 that the result of a query is a table. Two results with the same number of columns, of matching types, can be combined as sets of rows. `UNION` returns the rows that appear in either result, `INTERSECT` the rows that appear in both, and `EXCEPT` the rows of the first that do not appear in the second. Each removes duplicate rows from its result, as `DISTINCT` does.

For example, the distribution department wants a list of the cities where HG Industries has a customer or a warehouse, for planning delivery routes. The cities are in two tables, so the list is the union of two queries:

Table 3.52.: Cities with a Customer or a Warehouse

```
SELECT city, state
FROM distribution.customer
UNION
SELECT city, state
FROM distribution.warehouse
ORDER BY state, city;
```

city	state
Bentonville	AR
Fayetteville	AR
Fort Smith	AR
Little Rock	AR
Springdale	AR
Joplin	MO
Springfield	MO
Tulsa	OK

The two queries return eight rows and two rows, and Table 3.52 has eight, because both warehouse cities, Fayetteville and Tulsa, are also customer cities and `UNION` lists each once. `UNION ALL` keeps the duplicates, and it would return all ten rows. Notice that the `ORDER BY` at the end sorts the combined result, not the second query.

`INTERSECT` answers requests that say “both”. The customers who ordered in January and again in March are the keys that appear in both months’ orders:

Table 3.53.: Customers Who Ordered in Both January and March

```
SELECT customer_fk
FROM distribution.customer_order
WHERE order_date < DATE '2026-02-01'
INTERSECT
SELECT customer_fk
FROM distribution.customer_order
WHERE order_date >= DATE '2026-03-01';
```

customer_fk
6
2

The January orders of Table 2.5 come from customers 1, 2, 3, 4, and 6, and the March orders from customers 2 and 6, so Table 3.53 holds customers 2 and 6, Summit Farm Supply and Lakeside Distributors.

`EXCEPT` is the third way to write an anti-join. The customers who have never ordered are all the customers except those that appear in an order:

Table 3.54.: Customers Who Have Never Ordered

```
SELECT customer_pk
FROM distribution.customer
EXCEPT
SELECT customer_fk
FROM distribution.customer_order;
```

customer_pk
8

Table 3.54 holds customer 8, Northgate Hardware, the customer that Table 3.39 padded with NULL. The set operations compare whole rows, so they fit requests about keys, as here; to report the customer's name as well, the result must be joined back to the `customer` table, or the request written with `NOT EXISTS` instead. `EXCEPT` is the standard's name, and PostgreSQL's. Oracle calls it `MINUS`, a name you may meet in older material.

The set operations also show what an outer join is. Recall from Section 3.7 that a left join is an inner join followed by adding back the unmatched rows of the left table. With `UNION ALL`, the sentence can be written as a query: the inner join, then the customers that match nothing, each with NULL where an order number would be.

Table 3.55.: The Left Join of the Three Customers, Written as a Union

```
SELECT c.customer_pk, c.name, o.order_number
FROM distribution.customer AS c
JOIN distribution.customer_order AS o
  ON o.customer_fk = c.customer_pk
WHERE c.customer_pk IN (2, 3, 8)
UNION ALL
SELECT c.customer_pk, c.name, NULL
FROM distribution.customer AS c
WHERE c.customer_pk IN (2, 3, 8)
  AND NOT EXISTS (SELECT *
                  FROM distribution.customer_order AS o
                  WHERE o.customer_fk = c.customer_pk)
ORDER BY customer_pk, order_number;
```

customer_pk	name	order_number
2	Summit Farm Supply	SO-5003
2	Summit Farm Supply	SO-5012
3	Cedar Creek Home Center	SO-5002
3	Cedar Creek Home Center	SO-5007
8	Northgate Hardware	NULL

Table 3.55 holds the five rows of Table 3.39: the four pairs the inner join kept, from the first query, and Northgate Hardware with NULL, from the second. `UNION ALL` is right here, since the two queries cannot return the same row, one matching and one not. Thus, an outer join adds nothing that an inner join, an anti-join, and a union could not say; it says it in one clause, and that is why it is the form to use.

3.10. Common table expressions

This section presents common table expressions, which name the result of a query so that a later query can use it like a table.

A **common table expression** is a query given a name at the start of a statement, with `WITH`. The statement that follows may refer to the name as it would to a table. Common table expressions are often called CTEs. The simplest use solves a problem of Section 3.5: a column alias cannot be used in `WHERE`. Recall that listing the order lines worth more than $1,000 required repeating the expression `quantity * unit_price`. With a CTE, the expression is computed and named once:

Table 3.56.: Order Lines Worth $1,000 or More, with a CTE

```
WITH line AS (
  SELECT customer_order_fk, line_number,
         quantity * unit_price AS line_value
  FROM distribution.order_line
)
SELECT customer_order_fk, line_number, line_value
FROM line
WHERE line_value >= 1000
ORDER BY line_value DESC;
```

customer_order_fk	line_number	line_value
11	1	2460.00
6	1	2100.00
4	1	1780.00
11	2	1344.00
9	1	1335.00
6	2	1320.00
9	2	1150.00
3	1	1020.00

Read Table 3.56 from the top down: the `WITH` clause defines a result named `line`, in which every order line has a `line_value` column, and the final `SELECT` reads `line` as if it were a stored table. By the time that `SELECT` is evaluated, `line_value` is an ordinary column, so `WHERE` may use it.

The larger use of CTEs is to build a long query in steps, each named for what it means. For example, the sales manager asks which customers have ordered a product that is currently short in some warehouse, so that they can be warned of delays. The request has two parts, which become two CTEs:

Table 3.57.: Customers Who Have Ordered a Product That Is Now Short

```
WITH short_product AS (
  SELECT DISTINCT product_fk
  FROM distribution.stock_level
  WHERE quantity_on_hand <= reorder_point
),
buyer AS (
  SELECT DISTINCT o.customer_fk
  FROM distribution.order_line AS l
  JOIN distribution.customer_order AS o
    ON o.customer_order_pk = l.customer_order_fk
  WHERE l.product_fk IN (SELECT product_fk
                         FROM short_product)
)
SELECT c.customer_number, c.name
FROM distribution.customer AS c
JOIN buyer AS b
  ON b.customer_fk = c.customer_pk
ORDER BY c.customer_number;
```

customer_number	name
C-1001	Riverside Hardware
C-1002	Summit Farm Supply
C-1003	Cedar Creek Home Center
C-1004	Midtown Tool Supply
C-1006	Lakeside Distributors
C-1007	Tri-County Farm and Home

The first CTE of Table 3.57 finds the short products, the GC-300 and GC-400 of Example 3.1. The second finds the customers who have ordered one of them, and it may refer to the first, since a CTE may use any CTE

defined before it. The final query adds the customers' names. Each step can be run and checked on its own, by replacing the final query with `SELECT * FROM` the step, which is the same habit of checking each step that this chapter began with. The result is the six customers of Table 3.37, since nobody has ordered the GC-400.

A CTE does not store anything; it exists only while its statement runs. A CTE that does not refer to itself also adds no power that subqueries lack, since it could be written as a subquery in the `FROM` clause. What it adds is readability, which in a long query is what decides whether the next person, or you in six months, can check it. A CTE can also refer to itself, which lets a query follow a chain of rows of unknown length, such as the components of an assembly; Section 4.6 presents those recursive queries.

3.11. Summary

A query describes the rows wanted, and the database finds them. Every `SELECT` statement has one shape: a `SELECT` list and a `FROM` clause, which every query of this chapter has, then optional clauses in a fixed order, each answering part of the question. The `SELECT` list names the columns of the result, `FROM` the table they come from, and `WHERE` the condition a row must satisfy. The clauses are evaluated in the order `FROM`, `WHERE`, `GROUP BY` and `HAVING`, `SELECT`, `DISTINCT`, `ORDER BY`, and `LIMIT`, which is why a column alias may be used in `ORDER BY` but not in `WHERE`.

A `WHERE` clause keeps a row only when its condition is true. Conditions compare columns with values or with each other, and combine with `AND`, `OR`, and `NOT`, in which `AND` binds first. Evaluating a condition by hand on each row, before running the query, predicts the result and catches a wrong condition.

NULL makes a condition unknown, and unknown is not true. A comparison with NULL is unknown, a `WHERE` clause discards rows whose condition is unknown, and so a condition and its negation may together

miss some rows. `IS NULL` and `IS NOT NULL` test for NULL; `= NULL` never does.

A result has an order only when the query asks for one. `ORDER BY` sorts, and `LIMIT` or `FETCH FIRST` keeps the first rows of a sorted result. A limit without an order is meaningless, and a limit that ignores ties answers a question nobody asked.

Expressions compute new values from each row. Arithmetic, text, and date functions build derived columns, which a column alias names. Integer division discards fractions, and NULL propagates through arithmetic unless `COALESCE` replaces it.

A join is a cross product restricted by a join condition. Pairing every row of one table with every row of another, then keeping the pairs whose foreign key matches the key it refers to, is how to think about any join and how to check one by hand. A join returns one row for each matching combination, so it fans out when it passes to a table with many rows for each of its own.

An outer join keeps the rows that have no match. A left join adds back the unmatched rows of its left-hand table, padded with NULL. A condition on the right-hand table belongs in `ON`; placed in `WHERE`, it discards the added rows and turns the outer join back into an inner one.

A subquery answers a question inside a question. A subquery may supply one value to a comparison, a set of values to `IN`, or a test to `EXISTS`. `NOT IN` returns nothing when its subquery returns a NULL, while `NOT EXISTS` is safe. A request that says "every" is restated as a double negative, since *not* moves past a quantifier by changing it to the other one, and is written with two `NOT EXISTS` tests, first for one object and then for all of them. It must be checked on an empty set.

Set operations combine whole results. `UNION`, `INTERSECT`, and `EXCEPT` combine the rows of two queries with matching columns, removing duplicates unless `ALL` is written. Together with the left join and `NOT EXISTS`, `EXCEPT` is one of three ways to find the rows with no match.

A common table expression names a step. `WITH` gives a query a name that the rest of the statement uses as a table, so that a long query is built, read, and checked one named step at a time.

With these tools, you can answer most questions that a single database can answer one row at a time. Chapter 4 takes up the questions that summarize many rows into one: totals, averages, and counts, by group and over time.

3.12. Exercises

The exercises use the stages named in each, loaded as Section C.2 explains.

Exercise 3.1. The clauses of the following query are written in one order and evaluated in another.

```
SELECT DISTINCT carrier AS shipped_by
FROM distribution.shipment
WHERE shipped_on >= DATE '2026-02-01'
ORDER BY shipped_by
LIMIT 2;
```

a. List the clauses in the order in which they are evaluated.
b. Explain why `ORDER BY shipped_by` is allowed, but `WHERE shipped_by <> 'Customer pickup'` would not be.
c. Explain why the query is meaningful with its `ORDER BY` clause and would not be without it.

Exercise 3.2. Mark each statement true or false, and correct each false one.

a. `WHERE requested_ship_date = NULL` finds the orders with no requested ship date.
b. For any condition `c`, every row of a table is returned by either `WHERE c` or `WHERE NOT (c)`.
c. `BETWEEN 10 AND 20` includes 10 and 20.
d. In PostgreSQL, `7 / 2` is `3.5`.
e. `LIKE 'gc-%'` matches the product number GC-300 in PostgreSQL.
f. A query without `ORDER BY` returns rows in the order they were added to the table.

Exercise 3.3. The `quality.characteristic` table of stage Q records, for each characteristic of a part that the quality department measures, its nominal value and its lower and upper specification limits. Weld strength has only a lower limit, so its `upper_spec_limit` is NULL. Consider the condition `upper_spec_limit - lower_spec_limit < 1`.

a. List the table's five rows and evaluate the condition on each by hand, giving true, false, or unknown.
b. Which rows does `WHERE upper_spec_limit - lower_spec_limit < 1` return? Which does `WHERE NOT (upper_spec_limit - lower_spec_limit < 1)` return?
c. Explain why the two results together do not hold all five rows.
d. Write a query that lists the characteristics with a one-sided specification, one with only one limit.

Exercise 3.4. At stage D5, a sales manager asks for the customers in Arkansas or Missouri whose cities begin with the letter S. An analyst writes:

```
SELECT name, city, state
FROM distribution.customer
WHERE state = 'AR' OR state = 'MO' AND city LIKE 'S%';
```

a. List the eight customers with `SELECT name, city, state FROM distribution.customer`. Evaluate the condition by hand on each, and give the rows the query returns.
b. Correct the query, and give the rows it returns.

Exercise 3.5. At stage D5, list the number and date of every shipment that Tri-State Logistics carried in February 2026, earliest first. Reason out the number of rows before running the query.

Exercise 3.6. At stage D5, list the order lines whose value (quantity times unit price) is at least $1,000, showing the order key, the line number, the product key, and the value, from the largest value to the smallest. How many rows do you expect, and why?

Exercise 3.7. At stage P3, list the completed work orders with the number of days each took from release to completion and the number of days it finished after its due date (negative when it finished early). Sort the result from the most days late to the fewest, breaking ties by work order number. Which work orders finished late, and by how much?

Exercise 3.8. At stage P3, the planner asks for the two largest work orders by quantity.

a. Write the query with `LIMIT 2`, run it, and explain why its result is not a good answer.
b. Write the query so that it answers the request correctly, and give its result.

Exercise 3.9. At stage P3, a buyer searches the part list for everything to do with carts.

a. Write a query with `LIKE` that finds every part whose description contains the word “cart” in any mix of upper and lower case, using only standard SQL.

b. Write the same query with PostgreSQL's `ILIKE`.
c. How many rows does each return, and which part would a case-sensitive search miss?

Exercise 3.10. At stage D5, consider the two warehouses and the three shipments of orders SO-5002 and SO-5004 (SH-9002, SH-9004, and SH-9005).

a. Write out the cross product of these two warehouses and three shipments, and evaluate the join condition `s.warehouse_fk = w.warehouse_pk` on each pair by hand.
b. Which pairs does the join keep, and what does each kept pair say?
c. Write the join with `JOIN` and `ON`, restricted to these three shipments, and confirm that it returns the pairs you kept.

Exercise 3.11. At stage D5, list every shipment made in February 2026 with its shipment number, the date it shipped, the code of the warehouse it left from, and the name of the customer who placed the order, in the order shipped. Name the tables the query joins and the foreign key each join follows, and reason out the number of rows before running it.

Exercise 3.12. At stage D5, an analyst joins `customer` to `customer_order` and `customer_order` to `order_line`, and selects only `c.name`.

a. How many rows does the query return, and why?
b. How many times does Riverside Hardware appear, and why?
c. The analyst wanted each customer who has placed an order, listed once. Give two ways to write that query.

Exercise 3.13. At stage D5, the distribution manager asks for each warehouse's code with the numbers of the shipments that customers picked up there (carrier `'Customer pickup'`), including any warehouse where no customer has picked up an order.

a. Write the query with a left join and the carrier condition in `WHERE`, and give its result.
b. Write it with the carrier condition in `ON`, and give its result.
c. Which query answers the request, and why does the other not?

Exercise 3.14. At stage D5, list the orders for which no shipment has been made, in three ways:

a. with a left join;
b. with `NOT EXISTS`;
c. with `EXCEPT`.

Which of the three returns order numbers most directly, and why?

Exercise 3.15. At stage P3, a buyer wants the parts that are never used as a component of another part, and writes:

```
SELECT part_number, description
FROM production.part
WHERE part_pk NOT IN (SELECT component_fk
                      FROM production.bom_line);
```

a. Can this query fall into the `NOT IN` trap of Section 3.8? Use the relational schema of the `bom_line` table to decide.
b. What does the query return, and what do the parts it returns have in common?
c. Why would the same form be unsafe for a subquery that selects `requested_ship_date` from `distribution.customer_order`?

Exercise 3.16. At stage P3, the Welding work center (work center 3) has three machines: two MIG welders, M-04 and M-05, and the robotic welder M-07.

a. List the operators certified on every machine in Welding: those for whom there is no Welding machine without a certification pairing the operator with it, written as two nested `NOT EXISTS`. Run the query, and explain its result from the certifications.
b. List the operators certified on every MIG welder, the machines whose model is `'MIG welder MW-250'`.
c. What should the production supervisor do about the result of part (a)?

Exercise 3.17. At stage D5, the sales manager asks which customers have ordered every product that customer C-1007 has ordered.

a. Restate the request as a double negative, a sentence of the form "there is no … that …".
b. Write the inner query for one particular customer, C-1001 (customer 1): the products that C-1007 has ordered and C-1001 has not.
c. Generalize it to every customer, run it, and give its result.
d. Why does C-1007 appear in its own result? Change the query so that it does not.

Exercise 3.18. Using the three steps for putting *not* in front of a statement, write the negation of each statement in plain English, with no "it is not true that".

a. Every order has at least one order line.
b. There is a machine on which no operator is certified.
c. For every work center, there is an operator certified on every machine in it.
d. Write a query at stage P3 that returns the machines that make statement (b) true, and say whether (b) or its negation holds at stage P3.

Exercise 3.19. At stage D5, let the *short* products be those with a stock level at or below its reorder point in some warehouse, and the *unsold* products those with no order line.

a. Write a query for the product keys that are short or unsold, and give its result.
b. Write a query for the product keys that are both short and unsold, and give its result.
c. How many rows would part (a) return with `UNION ALL`, and why?

Exercise 3.20. At stage D5, the sales manager wants every product beside the key of each order line for it, including the products never ordered.

a. Write the request as a left join, sorted by product number and order line key.
b. Write the same request as an inner join, `UNION ALL`, and an anti-join, as Table 3.55 does. Show that it returns the same rows.
c. Before running either query, predict its number of rows from the number of order lines and the number of products never ordered.
d. Why is `UNION ALL` correct here, and what would `UNION` change?

Exercise 3.21. At stage P3, write a query with a common table expression that computes each completed work order's days late (its completion date minus its due date), and then lists the work orders that finished more than two days late: each one's work order number and days late, the most days late first. Explain why the common table expression lets the `WHERE` clause use the name `days_late`.

The remaining exercises use a second database, Millbrook Transit, which records a week of a small town's bus service (Section C.10). Its tables are in the `transit` schema; load it as the appendix describes before you start. Each exercise names the columns its result should have.

Exercise 3.22. List the access points that have no shelter: each one's point code, name, and number of benches, in order of point code. Which two tables does the query need, and what does the join between them rest on?

Exercise 3.23. List the points of route 2 in the order the bus reaches them, from the segments alone: each segment's sequence number, and the code and name of the point it starts from. Why does this list the Transit Center only once, although the route both starts and ends there?

Exercise 3.24. Which vehicles were never assigned to a route during the week? Give each one's vehicle number and model year, using an outer join (Section 3.7).

Exercise 3.25. List the smart cards that made no trip during the week: the card number, its kind, and the name of the person it is registered to, if any. Use `NOT EXISTS` (Section 3.8). Why must the join to `person` be an outer join?

Exercise 3.26. List the names of the people who hold no smart card, in alphabetical order, using `NOT IN` with a subquery on `smart_card.person_fk`.

a. Write the query, and explain why the subquery needs `WHERE c.person_fk IS NOT NULL`.
b. What does the query return without that condition, and why (Section 3.3)?

Exercise 3.27. Which days of the week have no schedule on any route? Give the day's name.

Exercise 3.28. Some people share a mailing address. List every pair of people at the same address, each pair once: the street, the first person's name, and the second person's name, where the first person is the one with the smaller person key, ordered by street and then by the names. What condition keeps a person from being paired with themselves, and each pair from appearing twice?

Exercise 3.29. Using the Northwind database of Section C.11, loaded with the book's loader (Section A.6), count the orders shipped after their required date, by shipper, with the columns `shipper` and `late_orders`, the most first.

a. Northwind lists six shippers. Why do only three appear?
b. The 21 orders not yet shipped have NULL in `shipped_date`. Are any of them counted as late? Explain with Section 3.3.

Exercise 3.30. Using the Chinook database of Section C.11, loaded with the book's loader (Section A.6), how many of the store's tracks have never been sold? Return the count beside the total number of tracks, with the columns `tracks_never_sold` and `tracks`, using `NOT EXISTS` for the first and a scalar subquery for the second.

4. Aggregation and Analytical Queries

Learning objectives

After reading this chapter you should be able to:

- summarize data with aggregate functions, GROUP BY, and HAVING
- form the groups of a query by hand and compute each group's aggregate before running it
- classify and recode values with CASE expressions, and build a cross tab with them
- compute running totals, rankings, and moving averages with window functions
- write recursive queries over hierarchical data such as a bill of materials
- answer operational questions about production, inventory, and quality data with SQL

The queries of Chapter 3 return rows: the orders that are late, the customers who ordered a cart. Many of the questions an engineer is asked are about many rows at once. How many units of each product did we ship last quarter? What is the average time a work order spends on the floor? Is the axle diameter drifting? Each is answered by a single number, or a single number for each group of rows, computed from rows the reader never sees one by one.

This chapter presents the parts of SQL that summarize. We begin with the aggregate functions, which reduce a set of rows to one value, and with `GROUP BY`, which forms the groups they are applied to. `HAVING` then filters the groups, and the `CASE` expression classifies rows so that they can be counted by class. Window functions compute a summary for each row without collapsing the rows, which is what running totals and moving averages need. Recursive queries follow a chain of rows, such as the components of an assembly, to any depth. The chapter ends with the analytical queries that industrial data calls for: on-time performance, inventory position, and control limits.

The habit of Chapter 3 continues. Before the database groups rows, we group a few of them by hand, write down each group and its aggregate, and then check that the query agrees.

Following along

The hand-evaluated examples run in your `dbconcepts` database, as in Chapter 3. The examples that need volume use the year of history, stage G, in your `dbconcepts_history` database: every table of the book's database with twelve more months of orders, shipments, and inspections (Section C.2). Each example says which database it uses.

4.1. The shape of an analytical query

This section places the tools of this chapter in the shape of a `SELECT` statement, and extends the order in which the database evaluates its clauses.

Recall from Section 3.1.1 that every query has the shape of Listing 3.1. This chapter presents the two clauses of that template that Chapter 3 did not use, `GROUP BY` and `HAVING`. Its other tools are expressions, which are

written inside clauses the reader already knows, and a recursive form of the `WITH` of Section 3.10. Listing 4.1 shows where each is written. Since a group or an order may be formed by any expression, not only a column, the template's `GROUP BY` and `ORDER BY` now take an `expression`.

Listing 4.1 The Shape of a SELECT Statement, with This Chapter's Additions

```
[WITH [RECURSIVE] name AS (query), ...]
SELECT [DISTINCT] expression AS alias,
       aggregate(...) [FILTER (WHERE condition)],
       window_function(...) OVER (...), ...
FROM schema.table
[WHERE condition]
[GROUP BY expression, ...]
[HAVING condition]
[ORDER BY expression [DESC], ...]
[LIMIT count];
```

Table 4.1 lists the tools, what kind of thing each is, and where it may be written. An expression computes a value, so it may appear wherever a value may: the `CASE` expression, for example, may stand in the `SELECT` list, in `WHERE`, in `GROUP BY`, in `ORDER BY`, and inside an aggregate.

Table 4.1.: The Tools of This Chapter and Where They Are Written

Tool	What it is	Where it may be written	Presented in
Aggregate function	An expression, one value for each group	`SELECT`, `HAVING`, `ORDER BY`	Section 4.2

Tool	What it is	Where it may be written	Presented in
`GROUP BY`	A clause	After `WHERE`	Section 4.2
`HAVING`	A clause	After `GROUP BY`	Section 4.3
`CASE`	An expression, one value for each row	Wherever a value may go	Section 4.4
`FILTER`	A condition attached to an aggregate	After the aggregate	Section 4.4
Window function	An expression, one value for each row	`SELECT`, `ORDER BY`	Section 4.5
`WITH RECURSIVE`	A common table expression that refers to itself	Before `SELECT`	Section 4.6

The order of evaluation of Section 3.1.1 gains one step. Window functions are computed after the `SELECT` list and before `DISTINCT`, so the database, at least in principle, evaluates a query in seven steps:

1. `FROM` names the table the rows come from, or joins several.
2. `WHERE` keeps the rows that satisfy its condition.
3. `GROUP BY` divides the rows that were kept into groups and computes each group's aggregates, and `HAVING` keeps the groups that satisfy its condition.
4. `SELECT` computes the columns of the result, one row for each row or group that was kept.
5. Window functions are computed over the rows of step 4.
6. `DISTINCT`, if present, removes repeated rows.
7. `ORDER BY` sorts the result, and `LIMIT` keeps its first rows.

A common table expression comes before all seven: in principle, the query that follows `WITH` is evaluated first, and the main query reads its result in `FROM` as a table.

The order explains where each tool may be written. `WHERE` cannot test an aggregate, since no group exists at step 2; that test belongs in `HAVING`. Neither `WHERE` nor `HAVING` can test a window function's result, since both are evaluated before step 5, and PostgreSQL refuses a window function in either. A window function may, however, rank the groups of step 3, since their aggregates already exist when it is computed.

For example, a sales manager asks for each customer's number of orders in the year of history, with the customer's rank. "In the year" says which rows, so it becomes a `WHERE` clause. "Each customer's" says into which groups, a `GROUP BY`, and "number of orders" is the aggregate `COUNT(*)`. "Rank" is a window function computed over the groups, at step 5. Section 4.5 writes the query, and then shows what changes when the manager wants only the top five ranks, a condition on a window function's result. Reading a request against Table 4.1 in this way tells you which tools its query needs, and the order of evaluation tells you where each is computed.

The next section presents the aggregate functions and `GROUP BY`, steps 3 and 4 of the order.

4.2. Aggregate functions and grouping

This section presents the aggregate functions, which compute one value from many rows, and the `GROUP BY` clause, which divides the rows into groups and computes the aggregates once for each group.

An **aggregate function** takes a column of values from a set of rows and returns one value. SQL provides five that every database shares: `COUNT`, `SUM`, `AVG`, `MIN`, and `MAX`. Used in the `SELECT` list of a query without `GROUP BY`, an aggregate is computed over every row that the `WHERE` clause kept,

and the result has exactly one row. The following query summarizes the 22 order lines of stage D5.

Table 4.2.: A Summary of the Order Lines

```
SELECT COUNT(*) AS lines,
       SUM(quantity) AS units,
       MIN(unit_price) AS lowest_price,
       MAX(unit_price) AS highest_price,
       AVG(unit_price) AS average_price
FROM distribution.order_line;
```

lines	units	lowest_price	highest_price	average_price
22	215	79.00	139.00	111.1818181818181818

Table 4.2 says that 22 lines ordered 215 units at prices from $79.00 to $139.00. `COUNT(*)` counts rows. The other four take a column, or any expression, and summarize its values. Notice the average: PostgreSQL keeps every digit it computes, and `ROUND` (Section 3.5) shortens it when a report needs fewer.

Aggregates treat NULL in one consistent way: they ignore it. `COUNT(*)` counts rows, whatever they hold, but `COUNT(column)` counts only the rows where the column is not NULL, and `SUM`, `AVG`, `MIN`, and `MAX` summarize only the values that are present. For example, recall that two of the twelve orders of stage D5 have no requested ship date:

Table 4.3.: Counting Rows and Counting Values

```
SELECT COUNT(*) AS orders,
       COUNT(requested_ship_date) AS with_requested_date
FROM distribution.customer_order;
```

orders	with_requested_date
12	10

The difference in Table 4.3 is usually what a request means. The number of orders is 12, and the number with a requested ship date is 10. `AVG` behaves the same way, which is sometimes a trap. The average of `completed_on - released_on` over the ten work orders of stage P3 is the average over the seven completed ones, since the difference is NULL for the three that are open (Table 3.29). That average says how long finished work took, not how long work takes, because the three orders still on the floor, some of them already late, are left out without a word.

When no rows reach an aggregate, `COUNT` returns 0, and the other four return NULL: there is no sum, average, or extreme of nothing. For example, at stage D5 no order line is for product 4, the GC-400, so `SUM(quantity)` over its lines is NULL, not zero. A report that should show zero writes `COALESCE(SUM(quantity), 0)`.

An aggregate over the whole table answers one question. Usually the request is for the same summary for each of several groups: units by product, orders by customer, defects by type. `GROUP BY` names the columns that define the groups. The database divides the rows that the `WHERE` clause kept into groups, one for each different value of those columns, and computes every aggregate once per group. The result has one row for each group.

Let's form the groups by hand. We take the order lines of the first four orders of stage D5 and ask for the units ordered of each product. The rows,

sorted by product so that the groups are easy to see, are:

Table 4.4.: The Lines of the First Four Orders

```
SELECT customer_order_fk, product_fk, quantity
FROM distribution.order_line
WHERE customer_order_fk <= 4
ORDER BY product_fk, customer_order_fk;
```

customer_order_fk	product_fk	quantity
1	1	10
3	1	12
4	1	10
4	1	20
2	2	6
3	2	4
1	3	4
3	3	6
4	3	5

Grouping by `product_fk` puts the nine rows of Table 4.4 into three groups. We then compute the aggregates of each group from its rows:

Table 4.5.: Forming the Groups and Their Aggregates by Hand

Product	Quantities in the group	Lines, `COUNT(*)`	Units, `SUM(quantity)`
1	10, 12, 10, 20	4	52
2	6, 4	2	10
3	4, 6, 5	3	15

Table 4.5 predicts three rows, and the query produces them:

Table 4.6.: Lines and Units by Product for the First Four Orders

```
SELECT product_fk,
       COUNT(*) AS lines,
       SUM(quantity) AS units
FROM distribution.order_line
WHERE customer_order_fk <= 4
GROUP BY product_fk
ORDER BY product_fk;
```

product_fk	lines	units
1	4	52
2	2	10
3	3	15

Notice what disappears when rows are grouped. The result of Table 4.6 has one row per product, so a column that differs from row to row within a group, such as `quantity` or `unit_price`, has no single value to show. SQL therefore allows only two kinds of expression in the `SELECT` list of a grouped query: the columns named in `GROUP BY`, and aggregates. Asking for another column is an error:

```
SELECT product_fk, unit_price, SUM(quantity) AS units
FROM distribution.order_line
GROUP BY product_fk;
```

```
ERROR:  column "order_line.unit_price" must appear in the
 ↪  GROUP BY clause or be used in an aggregate function
LINE 1: SELECT product_fk, unit_price, SUM(quantity) AS units
```

```
                ^
```

The message offers the two repairs. Grouping by `unit_price` as well makes a group for each product and price, which is a different question. Aggregating it, as `AVG(unit_price)` or `MAX(unit_price)`, answers a question about the prices of each product. Which repair is right depends on the request, and the database cannot choose for you.

A grouped query often joins tables first, so that the groups can be labeled with something a reader recognizes. The following query totals the units ordered of each product over the year of history in stage G, from April 2026 through March 2027, and labels each group with the product's number:

Table 4.7.: Units Ordered of Each Product, April 2026 to March 2027

```
SELECT p.product_number,
       COUNT(*) AS lines,
       SUM(l.quantity) AS units
FROM distribution.order_line AS l
JOIN distribution.customer_order AS o
  ON o.customer_order_pk = l.customer_order_fk
JOIN distribution.product AS p
  ON p.product_pk = l.product_fk
WHERE o.order_date >= DATE '2026-04-01'
  AND o.order_date < DATE '2027-04-01'
GROUP BY p.product_number
ORDER BY p.product_number;
```

product_number	lines	units
GC-300	317	4030
GC-400	132	1639
HT-100	295	3584
HT-200	299	3636

The query of Table 4.7 groups by `product_number`, the alternate key, so each group is one product. Grouping by a column that is not a key is different: two products with the same description would fall into one group. When you group by a descriptive column such as a name, you should group by the key as well, `GROUP BY p.product_pk, p.description`, even when the key is not displayed.

PostgreSQL relaxes the rule of the `SELECT` list in one case. When a query groups by a table's primary key, it may select any other column of that table, since the key determines those columns' values. The SQL standard allows this, but many databases do not, and this book groups by every

column it selects.

Recall from Section 4.1 that `GROUP BY` is step 3 of the order of evaluation, and that the `SELECT` list is computed at step 4, one row for each group. That order is the reason for the rule of the `SELECT` list: by step 4 each group is one row, and a column whose value differs within a group has no one value to show. The next section presents `HAVING`, the rest of step 3.

4.3. Filtering groups

This section presents the `HAVING` clause, which keeps the groups whose aggregates satisfy a condition.

`WHERE` filters rows before they are grouped, so it cannot test an aggregate: the aggregate does not exist until the groups are formed. `HAVING` holds a condition that is tested once for each group, after grouping, and it may use aggregates. For example, the sales manager asks which customers placed more than 40 orders in the year of history:

Table 4.8.: Customers with More Than 40 Orders in the Year

```
SELECT c.customer_number, c.name,
       COUNT(*) AS orders
FROM distribution.customer AS c
JOIN distribution.customer_order AS o
  ON o.customer_fk = c.customer_pk
WHERE o.order_date >= DATE '2026-04-01'
  AND o.order_date < DATE '2027-04-01'
GROUP BY c.customer_number, c.name
HAVING COUNT(*) > 40
ORDER BY orders DESC, c.customer_number;
```

customer_number	name	orders
C-1014	Green Country Hardware	45
C-1023	Sunflower Hardware	45
C-1003	Cedar Creek Home Center	44
C-1006	Lakeside Distributors	43
C-1001	Riverside Hardware	42

The query of Table 4.8 uses both clauses, and each does the job only it can do. `WHERE` keeps the orders of the year, row by row, before any group exists; `HAVING` keeps the customers whose count of those orders exceeds 40. Moving the date condition into `HAVING` would be an error, since `order_date` is neither grouped nor aggregated. Moving the count condition into `WHERE` would be an error too, since no count exists when `WHERE` is evaluated. Thus, a condition on a row belongs in `WHERE`, and a condition on a group belongs in `HAVING`.

`COUNT(DISTINCT column)` counts the different values in a group rather than the rows. It answers requests such as "how many different products has each customer ordered", where `COUNT(*)` would count order lines. The

distinction matters most after a join, because the fan-out of Section 3.6 repeats rows, and `COUNT(*)` counts every repeat.

Recall from Example 3.5 that a request with the word *every* can be answered by a double `NOT EXISTS`. Counting gives a second method. An operator is certified on every machine in Machining when the number of Machining machines on which the operator holds a certification equals the number of Machining machines:

Table 4.9.: Operators Certified on Every Machining Machine, by Counting

```
SELECT o.employee_number, o.name
FROM production.operator AS o
JOIN production.certification AS c
  ON c.operator_fk = o.operator_pk
JOIN production.machine AS m
  ON m.machine_pk = c.machine_fk
WHERE m.work_center_fk = 2
GROUP BY o.employee_number, o.name
HAVING COUNT(*) = (SELECT COUNT(*)
                   FROM production.machine
                   WHERE work_center_fk = 2);
```

employee_number	name
E-202	James Carter

Table 4.9 agrees with Table 3.49: James Carter. The joins keep one row for each certification an operator holds on a Machining machine, the groups count them per operator, and `HAVING` compares each count with the subquery's count of Machining machines, which is 2. Counting certifications counts machines only because the alternate key (`operator_fk`, `machine_fk`) of the `certification` table allows at most one row for each

operator and machine; without it, a repeated certification would be counted twice.

The two methods disagree in one case. For the Assembly work center, which has no machines, the double `NOT EXISTS` returned every operator. The counting query returns none: no operator holds a certification on an Assembly machine, so the joins keep no rows, no groups are formed, and there is nothing for `HAVING` to compare with zero. Neither answer is wrong in itself. They answer the question "who is certified on every machine of a work center that has none" in two different ways, and you should decide which the request means before you choose a method.

An aggregate cannot be applied to another aggregate in the same query. A request such as "the busiest month's number of orders" needs the count for each month first, and then the largest of those counts. Recall from Section 3.10 that a common table expression names a step. The first step groups; the second summarizes the groups:

Table 4.10.: The Busiest Month of the Year

```
WITH monthly AS (
  SELECT EXTRACT(YEAR FROM order_date) AS year,
         EXTRACT(MONTH FROM order_date) AS month,
         COUNT(*) AS orders
  FROM distribution.customer_order
  WHERE order_date >= DATE '2026-04-01'
    AND order_date < DATE '2027-04-01'
  GROUP BY EXTRACT(YEAR FROM order_date),
           EXTRACT(MONTH FROM order_date)
)
SELECT year, month, orders
FROM monthly
WHERE orders = (SELECT MAX(orders) FROM monthly);
```

year	month	orders
2027	3	65

The final query of Table 4.10 keeps every month whose count equals the largest, so a tie for the busiest month would show both months, which `ORDER BY orders DESC LIMIT 1` would not (Section 3.4). The next section presents the expression that lets a query group rows by a classification that no column holds.

4.4. CASE expressions

This section presents the `CASE` expression, which chooses a value by testing conditions in turn, and its main uses: classifying rows, counting them by class, and turning rows into columns.

A **`CASE` expression** lists conditions and the value to return for each. The database tests the conditions in order, returns the value of the first that is true, and returns the value after `ELSE` if none is true, or NULL if there is no `ELSE`. For example, the production manager classifies each work order of stage P3 as early, on time, late, or open:

Table 4.11.: Work Orders Classified by Completion

```
SELECT work_order_number, due_on, completed_on,
       CASE
         WHEN completed_on IS NULL THEN 'open'
         WHEN completed_on < due_on THEN 'early'
         WHEN completed_on = due_on THEN 'on time'
         ELSE 'late'
       END AS status
FROM production.work_order
ORDER BY work_order_number;
```

work_order_number	due_on	completed_on	status
WO-7001	2026-01-16	2026-01-15	early
WO-7002	2026-01-13	2026-01-12	early
WO-7003	2026-01-30	2026-02-02	late
WO-7004	2026-02-06	2026-02-05	early
WO-7005	2026-02-09	2026-02-09	on time
WO-7006	2026-02-20	2026-02-24	late
WO-7007	2026-02-27	2026-02-26	early
WO-7008	2026-03-13	NULL	open
WO-7009	2026-03-16	NULL	open
WO-7010	2026-03-27	NULL	open

The first `WHEN` in Table 4.11 catches the open work orders, so the later comparisons are made only for completed ones, whose `completed_on` is

not NULL. Had there been no test for NULL, an open work order would have reached `ELSE`, since every comparison with its NULL completion date is unknown (Section 3.3), and it would have been called late. An unknown condition is not true, so the row passes on to the next `WHEN`; a test for NULL placed last would still catch it. Thus, a `CASE` that compares a column that may be NULL needs its own test for NULL, and placing that test first lets every later condition assume a value.

When every condition compares one expression with a value, `CASE` has a shorter form. It names the expression once, after `CASE`, and each `WHEN` gives a value to match. For example, the part list spells out each unit of measure:

Table 4.12.: Units of Measure Spelled Out

```
SELECT part_number, unit_of_measure,
       CASE unit_of_measure
         WHEN 'EA' THEN 'each'
         WHEN 'FT' THEN 'feet'
         ELSE 'unknown'
       END AS unit
FROM production.part
WHERE part_number LIKE 'T%';
```

part_number	unit_of_measure	unit
TU-12	FT	feet
TI-10	EA	each
TR-300	EA	each
TR-400	EA	each

A `CASE` expression can appear anywhere an expression can, including inside an aggregate and in `GROUP BY`. Together these count rows by a class that

no column holds. The quality department classifies each measurement of stage G as below, within, or above its characteristic's specification limits, and counts each class for each characteristic:

Table 4.13.: Measurements by Specification Class

```
SELECT ch.characteristic_pk, ch.name,
       CASE
         WHEN m.measured_value < ch.lower_spec_limit
           THEN 'below'
         WHEN m.measured_value > ch.upper_spec_limit
           THEN 'above'
         ELSE 'within'
       END AS class,
       COUNT(*) AS measurements
FROM quality.measurement AS m
JOIN quality.characteristic AS ch
  ON ch.characteristic_pk = m.characteristic_fk
GROUP BY ch.characteristic_pk, ch.name, class
ORDER BY ch.characteristic_pk, class;
```

characteristic_pk	name	class	measurements
1	Diameter	above	1
1	Diameter	within	138
2	Length	below	1
2	Length	within	138
3	Weld strength	below	5
3	Weld strength	within	53
4	Coating thickness	below	1
4	Coating thickness	within	57
5	Coating thickness	above	1
5	Coating thickness	below	1
5	Coating thickness	within	57

Table 4.13 groups by the alias `class`, which PostgreSQL allows in `GROUP BY` as a convenience; the SQL standard requires the `CASE` expression to

be repeated there. Notice how the weld strength is classified. Its upper limit is NULL, so the second condition is unknown for every weld and no weld is ever called "above". That is right for a one-sided specification, but it is right because "within" is the value after `ELSE`, not by design. A classification that must treat a missing limit in some other way should say so with its own `WHEN ... IS NULL` test.

A `CASE` expression may stand anywhere a value may, and in `ORDER BY` it sorts by an order that no column holds. For example, the production manager's morning list puts the open work orders first, the late ones next, and the rest after, each group by its due date:

Table 4.14.: Work Orders, Open First, Then Late, Then the Rest

```
SELECT work_order_number, due_on, completed_on
FROM production.work_order
ORDER BY CASE
           WHEN completed_on IS NULL THEN 1
           WHEN completed_on > due_on THEN 2
           ELSE 3
         END,
         due_on, work_order_number;
```

work_order_number	due_on	completed_on
WO-7008	2026-03-13	NULL
WO-7009	2026-03-16	NULL
WO-7010	2026-03-27	NULL
WO-7003	2026-01-30	2026-02-02
WO-7006	2026-02-20	2026-02-24
WO-7002	2026-01-13	2026-01-12
WO-7001	2026-01-16	2026-01-15
WO-7004	2026-02-06	2026-02-05
WO-7005	2026-02-09	2026-02-09
WO-7007	2026-02-27	2026-02-26

The `CASE` of Table 4.14 gives each work order a rank, 1, 2, or 3, that appears in no column of the result; the rows are sorted by the rank first and by the due date within it. The three open work orders lead, then WO-7003 and WO-7006, the two completed after their due dates, as Table 4.11 classified them. A sort like this one replaces a report's hand-arranged sections.

A `CASE` inside an aggregate also turns rows into columns. A table with one row for each category and period, turned so that each category has a column of its own, is called a **cross tab**, or pivot table. It is how a

spreadsheet user expects to see a summary, and it is the form the following example needs.

Example 4.1 (Seeing the seasons in garden cart sales)**.** *The system.* The orders of stage G, the year of history. HG Industries sells two garden carts: the GC-300 steel cart, sold all year, and the GC-400 poly cart, sold from September 1, 2026.

The request. The planning manager asks whether garden cart demand is seasonal, so that production of carts can be leveled. The request is for units ordered by quarter, with a column for each cart and one for the two together.

Each quarter is a row and each cart a column, so each column adds up the quantities of the lines for one cart, and a `CASE` returns the quantity for the lines that belong to the column and zero for the others:

Table 4.15.: Garden Cart Units Ordered by Quarter

```
SELECT EXTRACT(YEAR FROM o.order_date) AS year,
       EXTRACT(QUARTER FROM o.order_date) AS quarter,
       SUM(CASE WHEN p.product_number = 'GC-300'
                THEN l.quantity ELSE 0 END) AS gc_300,
       SUM(CASE WHEN p.product_number = 'GC-400'
                THEN l.quantity ELSE 0 END) AS gc_400,
       SUM(l.quantity) AS all_carts
FROM distribution.order_line AS l
JOIN distribution.customer_order AS o
  ON o.customer_order_pk = l.customer_order_fk
JOIN distribution.product AS p
  ON p.product_pk = l.product_fk
WHERE p.product_number LIKE 'GC-%'
  AND o.order_date >= DATE '2026-04-01'
  AND o.order_date < DATE '2027-04-01'
GROUP BY EXTRACT(YEAR FROM o.order_date),
         EXTRACT(QUARTER FROM o.order_date)
ORDER BY year, quarter;
```

year	quarter	gc_300	gc_400	all_carts
2026	2	1645	0	1645
2026	3	728	200	928
2026	4	941	723	1664
2027	1	716	716	1432

The check. Each row of Table 4.15 must satisfy `gc_300 + gc_400 = all_carts`, since every line in the `WHERE` clause is for one of the two carts. For example, the fourth quarter of 2026 has $941 + 723 = 1664$. The GC-400 column must also be zero in the second quarter of 2026, before the poly cart was launched, and it is.

Interpretation. The steel cart is strongly seasonal: 1,645 units in the spring quarter against 941 in the fall, about 1.75 times as many. The total for both carts hides it. The poly cart's first full quarter fell in the fall, so the total for the fourth quarter, 1,664, is as large as the spring's, and a planner reading only the total would conclude that cart demand is level. It is not; one product's launch offset the other's season.

Where it breaks. The quarters come from the order date, not the ship date, so they measure demand, which is what leveling production needs. A question about the load on the warehouse would group shipments by `shipped_on` instead. In addition, the first quarter of 2026 is left out, because stage G's history begins in April and the three months before it hold only the small starting data of stage D5.

The SQL standard has a shorter way to write an aggregate over some of a group's rows: `SUM(l.quantity) FILTER (WHERE p.product_number = 'GC-300')`. PostgreSQL supports it, but many databases do not, and the `CASE` form works everywhere. PostgreSQL also offers a `crosstab` function, in an extension named `tablefunc` that must be installed separately, which builds a cross tab from a query of rows. This book uses the `CASE` form, which shows in the query itself where each column comes from.

4.5. Window functions

This section presents window functions, which compute an aggregate or a ranking for each row from a set of rows related to it, without collapsing the rows into groups.

A grouped query answers "what is the average price of each product" with one row per product. Often the request wants each row *beside* its group's summary: each order line with its product's average price, so that a sales analyst can see which lines were discounted. `GROUP BY` cannot do that, since it keeps only one row per group, and adding the line's own columns

to the SELECT list is the error of Section 4.2. One repair joins each line to a subquery of the group summaries. We take the nine lines of the first four orders again (Table 4.4):

Table 4.16.: Each Line Beside Its Product's Average, by a Join

```
SELECT l.customer_order_fk, l.product_fk, l.unit_price,
       ROUND(g.average_price, 2) AS product_average
FROM distribution.order_line AS l
JOIN (SELECT product_fk, AVG(unit_price) AS average_price
      FROM distribution.order_line
      WHERE customer_order_fk <= 4
      GROUP BY product_fk) AS g
  ON g.product_fk = l.product_fk
WHERE l.customer_order_fk <= 4
ORDER BY l.product_fk, l.customer_order_fk, l.unit_price;
```

customer_order_fk	product_fk	unit_price	product_average
1	1	89.00	85.50
3	1	85.00	85.50
4	1	79.00	85.50
4	1	89.00	85.50
2	2	139.00	137.00
3	2	135.00	137.00
1	3	119.00	117.67
3	3	115.00	117.67
4	3	119.00	117.67

The query of Table 4.16 works, but it says the same thing twice: the subquery must repeat the outer query's condition on the orders, and a change to one that is not made to the other silently changes the answer. A

window function computes the group's aggregate for each row directly. Before we write one, we look at how it sees the rows.

4.5.1. How a window function sees the rows

A window function is computed once for each row of the result, which is called the **current row**. For each current row, it answers three questions about the other rows of the result:

1. *Whose rows?* The rows that belong with the current row, called its **partition**. The rows are sliced into partitions by the values of one or more columns, such as the product.
2. *In what order?* The order of the rows within the partition, such as the order of their dates.
3. *Which of them, for this row?* The rows of the partition that the function reads for the current row, called its **frame**, such as the rows from the first up to the current row.

Slice, sort, and slide: each answer is one part of the `OVER` clause that follows the function, written in that order, as Listing 4.2 shows.

Listing 4.2 The Shape of a Window Function

```
function(arguments) OVER (
  [PARTITION BY expression, ...]
  [ORDER BY expression [DESC], ...]
  [ROWS BETWEEN start AND end]
)
```

Here `start` and `end` each name a row relative to the current one: `UNBOUNDED PRECEDING` (the partition's first row), `n PRECEDING`, `CURRENT`

`ROW`, `n FOLLOWING`, or `UNBOUNDED FOLLOWING` (its last row). Every part is optional, and Table 4.17 gives what each means when it is left out.

Table 4.17.: The Parts of OVER, and What Each Means When It Is Left Out

Part of `OVER`	Question it answers	When it is left out
`PARTITION BY`	Whose rows?	Every row of the result is one partition
`ORDER BY`	In what order?	The rows have no order: every row is a peer of every other
`ROWS BETWEEN`	Which of them, for this row?	With an `ORDER BY`, the rows from the partition's first to the current row and the rows that tie with it; without one, the whole partition

Not every window function reads the frame. An aggregate such as `SUM` or `AVG`, and `FIRST_VALUE` and `LAST_VALUE`, compute over the frame. The ranking functions (`ROW_NUMBER`, `RANK`, `DENSE_RANK`) and the functions that reach into a neighboring row (`LAG`, `LEAD`) use only the partition and its order, and ignore the frame: the third question does not apply to them.

The default frame of Table 4.17 is written in the SQL standard with `RANGE` in place of `ROWS`. `RANGE` measures the frame by the values of the `ORDER BY` columns rather than by counting rows, which is why rows that tie are in each other's frames. This book writes its explicit frames with `ROWS`, which counts rows, and the subsections that follow take the three parts of `OVER` in turn.

4.5.2. Partitions

`PARTITION BY` inside `OVER` names the columns that define each row's partition. The following query is Table 4.16 written with a window function:

Table 4.18.: The Same Result, by a Window Function

```
SELECT customer_order_fk, product_fk, unit_price,
       ROUND(AVG(unit_price)
               OVER (PARTITION BY product_fk), 2)
         AS product_average
FROM distribution.order_line
WHERE customer_order_fk <= 4
ORDER BY product_fk, customer_order_fk, unit_price;
```

customer_order_fk	product_fk	unit_price	product_average
1	1	89.00	85.50
3	1	85.00	85.50
4	1	79.00	85.50
4	1	89.00	85.50
2	2	139.00	137.00
3	2	135.00	137.00
1	3	119.00	117.67
3	3	115.00	117.67
4	3	119.00	117.67

Table 4.18 is the same as Table 4.16. The `OVER` clause has no `ORDER BY`, so by Table 4.17 each row's frame is its whole partition: the rows of the result that share its `product_fk`. Figure 4.1 follows the database as it computes the result, one current row at a time, in two of its nine steps; each step is numbered by its current row's place in the result. In each step, a bracket marks the frame of the current row, the frame's cells are shaded, and the

current row itself is darker, with its values in bold. The box under the table lists the unit prices in the frame and the arithmetic on them, and the arrow carries the average into the current row's `product_average`. The rows already processed show their averages in gray, and the rows still to come are blank.

Read in order, the steps of Figure 4.1 show what a frame is: the set of rows whose values the function reads for one current row. The frame of row 1 is rows 1 to 4, the four lines of product 1, and their average is 85.50. Rows 2, 3, and 4, the steps between, have the same frame, since here the frame is the whole partition, and so the same average. At row 5 the current row enters the partition of product 2, and the frame moves with it, to rows 5 and 6. At row 7 it moves again, to rows 7 to 9, the three lines of product 3, whose average is 117.67.

Figure 4.2 shows all nine steps at once. Each column is one current row, and its shaded cells are that row's frame, each showing the price the average reads; the darker cell, with its value in bold, is the current row itself. The bottom row is the average of each column's shaded cells. Thus, column 1 is step 1 of Figure 4.1, and its average is the same 85.50.

The shading of Figure 4.2 forms three blocks, one for each product. Each of the four rows of product 1 sees the same four prices, so each receives the same average, $85.50; each row of product 2 sees two prices, and each row of product 3 sees three. The rows keep their identities: nine rows go in and nine come out. Thus, `GROUP BY` answers a question about groups, and a window function answers a question about each row in the light of its group.

A window function is computed at step 5 of the order of evaluation (Section 4.1), after `WHERE`, `GROUP BY`, and `HAVING`, and before `ORDER BY`. The partition therefore holds only the rows that the `WHERE` clause kept, which is why the averages of Table 4.18 are those of the four orders and not of every order.

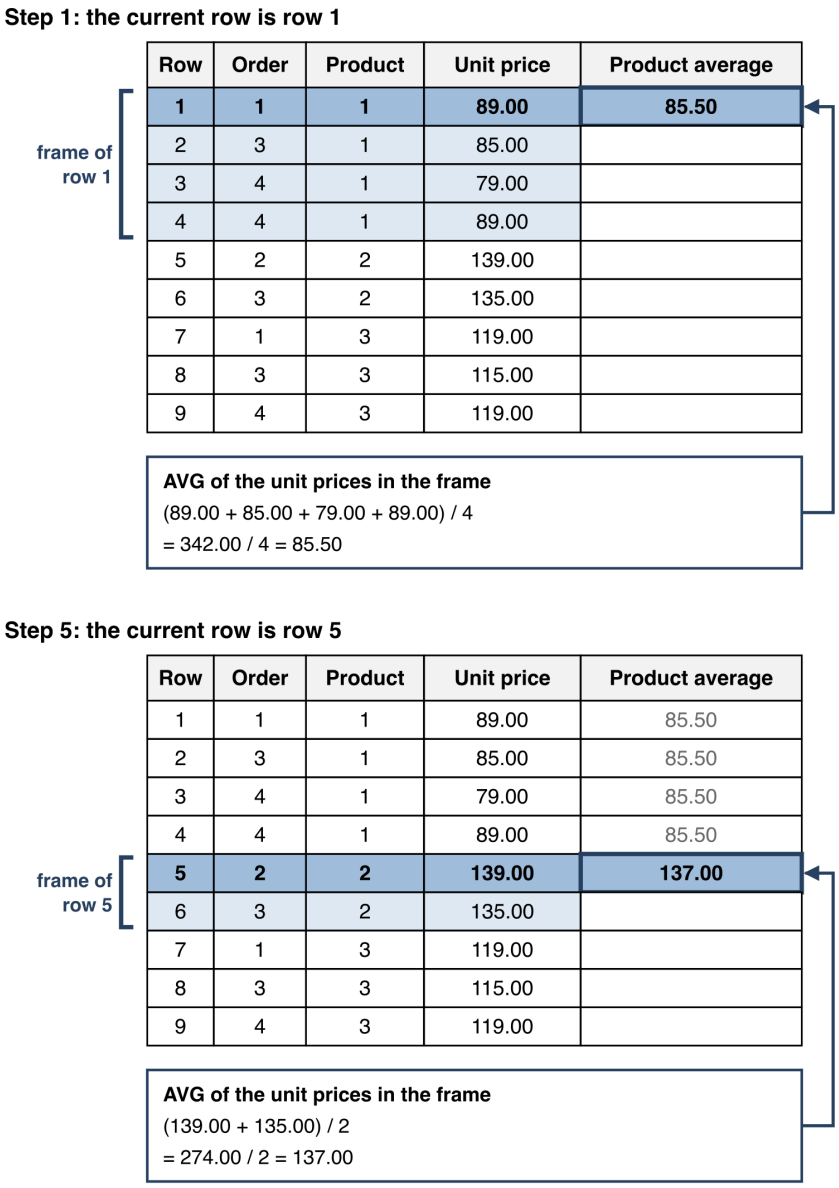


Figure 4.1.: The Frame Moving Down the Rows, for AVG(unit_price) OVER (PARTITION BY product_fk)

				The frame, when the current row is row								
Row	Order	Product	Unit price	1	2	3	4	5	6	7	8	9
1	1	1	89.00	**89.00**	89.00	89.00	89.00					
2	3	1	85.00	85.00	**85.00**	85.00	85.00					
3	4	1	79.00	79.00	79.00	**79.00**	79.00					
4	4	1	89.00	89.00	89.00	89.00	**89.00**					
5	2	2	139.00					**139.00**	139.00			
6	3	2	135.00					135.00	**135.00**			
7	1	3	119.00							**119.00**	119.00	119.00
8	3	3	115.00							115.00	**115.00**	115.00
9	4	3	119.00							119.00	119.00	**119.00**
AVG of the shaded values above				**85.50**	**85.50**	**85.50**	**85.50**	**137.00**	**137.00**	**117.67**	**117.67**	**117.67**

Figure 4.2.: The Frame of Each Row's Average, with PARTITION BY and No ORDER BY

4.5.3. Order and the default frame

An `ORDER BY` inside `OVER` gives the rows of each partition an order, and it changes which rows an aggregate sees. With an order and no `ROWS BETWEEN`, the frame of each row is, by Table 4.17, the rows of the partition from the first up to the current row, and its peers, which are explained below. As the current row moves down the partition, the frame grows by one row at a time, and `SUM` becomes a running total.

Let's compute a running total by hand. The eight order lines for the HT-100 (product 1) at stage D5, in the order of their orders' dates, are shown below with the frame of each row and the sum over the frame. Two lines were ordered on the same day, January 15, and the default frame treats rows that tie on the `ORDER BY` columns, called **peers**, as one: each tied row's frame includes all of its peers.

Table 4.19.: A Running Total by Hand, with a Tie

Line	Order date	Quantity	Frame (lines)	Running total
1	2026-01-05	10	1	10
4	2026-01-12	12	1, 4	22
7	2026-01-15	20	1, 4, 7, 9	52
9	2026-01-15	10	1, 4, 7, 9	52
11	2026-01-28	25	1, 4, 7, 9, 11	77
14	2026-02-03	8	1, 4, 7, 9, 11, 14	85
16	2026-02-18	15	1, 4, 7, 9, 11, 14, 16	100
19	2026-03-04	30	all eight	130

Figure 4.3 follows two of the eight steps of Table 4.19, in the manner of Figure 4.1, each numbered by its current row's place in the table.

Step 3 of Figure 4.3 shows the effect of the tie. The frame of line 7 reaches below the current row, to line 9, its peer, so the sum at line 7 is already 52, before line 9 is processed. At step 4, which the figure skips, the frame of line 9 holds the same four rows, and the sum is again 52. At step 5 the bottom of the frame has moved down to line 11 while its top stays at line 1, so the frame has grown to five rows and the sum to 77. Figure 4.4 shows all eight steps at once, in the manner of Figure 4.2: each column is one current row, and its shaded cells are the quantities its `SUM` adds.

The shading of Figure 4.4 is a staircase: each column holds one more row than the column before it, and the sums along the bottom are the running totals. The tie breaks the staircase in one place. The column for line 7 shades line 9 below its current row, since line 9 is its peer, and the columns for lines 7 and 9 hold the same four rows and the same sum, 52. Adding a column that breaks the tie, such as the line's key, gives each row a frame of its own. The query computes both:

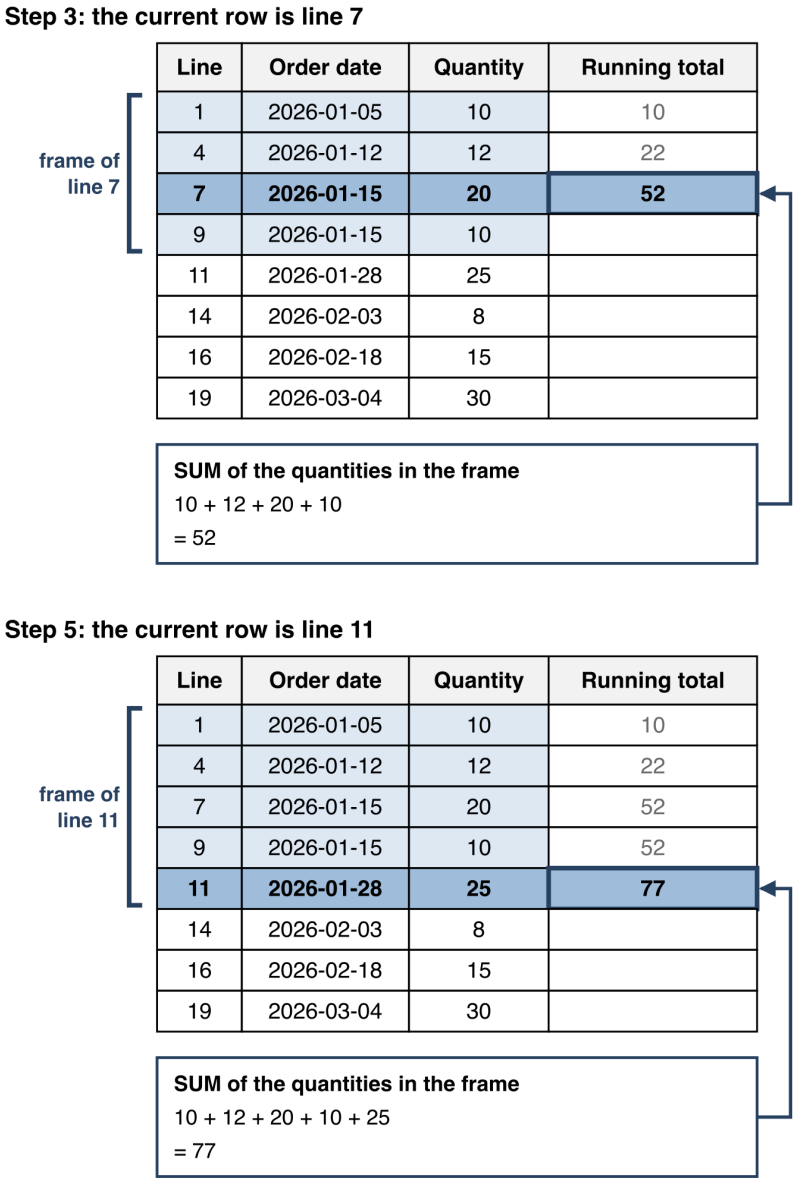


Figure 4.3.: The Frame Moving Down the Rows, for SUM(quantity) OVER (ORDER BY order_date)

			The frame, when the current row is line							
Line	**Order date**	**Quantity**	**1**	**4**	**7**	**9**	**11**	**14**	**16**	**19**
1	2026-01-05	10	**10**	10	10	10	10	10	10	10
4	2026-01-12	12		**12**	12	12	12	12	12	12
7	2026-01-15	20			**20**	20	20	20	20	20
9	2026-01-15	10			10	**10**	10	10	10	10
11	2026-01-28	25					**25**	25	25	25
14	2026-02-03	8						**8**	8	8
16	2026-02-18	15							**15**	15
19	2026-03-04	30								**30**
SUM of the shaded values above			**10**	**22**	**52**	**52**	**77**	**85**	**100**	**130**

Figure 4.4.: The Frame of a Running Total, SUM(quantity) OVER (ORDER BY order_date)

Table 4.20.: Running Totals of HT-100 Units Ordered

```
SELECT l.order_line_pk, o.order_date, l.quantity,
       SUM(l.quantity) OVER (ORDER BY o.order_date)
         AS by_date,
       SUM(l.quantity) OVER (ORDER BY o.order_date,
                                      l.order_line_pk)
         AS by_line
FROM distribution.order_line AS l
JOIN distribution.customer_order AS o
  ON o.customer_order_pk = l.customer_order_fk
WHERE l.product_fk = 1
ORDER BY o.order_date, l.order_line_pk;
```

order_line_pk	order_date	quantity	by_date	by_line
1	2026-01-05	10	10	10
4	2026-01-12	12	22	22
7	2026-01-15	20	52	42
9	2026-01-15	10	52	52
11	2026-01-28	25	77	77
14	2026-02-03	8	85	85
16	2026-02-18	15	100	100
19	2026-03-04	30	130	130

The `by_date` column of Table 4.20 matches Table 4.19, and the `by_line` column counts the two January 15 lines one at a time, 42 and then 52. Which is right depends on the request: "units ordered through each day" wants `by_date`, and a line-by-line tally wants `by_line`. The query has no `PARTITION BY`, so its one partition is every row that the `WHERE` clause kept, the HT-100's eight lines.

4.5.4. Explicit frames

A frame can also be given explicitly, with `ROWS BETWEEN`. `ROWS BETWEEN 4 PRECEDING AND CURRENT ROW` holds the current row and the four before it, counted as rows, whatever their values. A frame of the last five rows, moving down the partition one row at a time, is the frame of a five-period moving average, the forecast that every industrial engineer knows. We take the seven inspections of the axle diameter from June 8 to 16, 2026, at stage G (part AX-10, characteristic 1), and average each with the four before it:

Table 4.21.: A Five-Inspection Moving Average of the Axle Diameter

```
SELECT CAST(i.inspected_at AS DATE) AS inspected_on,
       m.measured_value,
       ROUND(AVG(m.measured_value) OVER (
               ORDER BY i.inspected_at
               ROWS BETWEEN 4 PRECEDING AND CURRENT ROW
             ), 4) AS moving_average
FROM quality.measurement AS m
JOIN quality.inspection AS i
  ON i.inspection_pk = m.inspection_fk
WHERE m.characteristic_fk = 1
  AND i.inspected_at >= DATE '2026-06-08'
  AND i.inspected_at < DATE '2026-06-17'
ORDER BY i.inspected_at;
```

inspected_on	measured_value	moving_average
2026-06-08	0.6267	0.6267
2026-06-09	0.6268	0.6268
2026-06-10	0.6273	0.6269
2026-06-11	0.6264	0.6268
2026-06-12	0.6240	0.6262
2026-06-15	0.6240	0.6257
2026-06-16	0.6252	0.6254

Figure 4.5 follows two of the seven steps of Table 4.21, each numbered by its current row's place in the result.

At step 3 of Figure 4.5 the frame is short: a frame cannot reach before the partition's first row, so the average of June 10 is that of three inspections, $(0.6267 + 0.6268 + 0.6273)/3 = 0.626933\ldots$, which rounds to the 0.6269 of Table 4.21. From step 5 on, it holds five rows. The frame of June 12

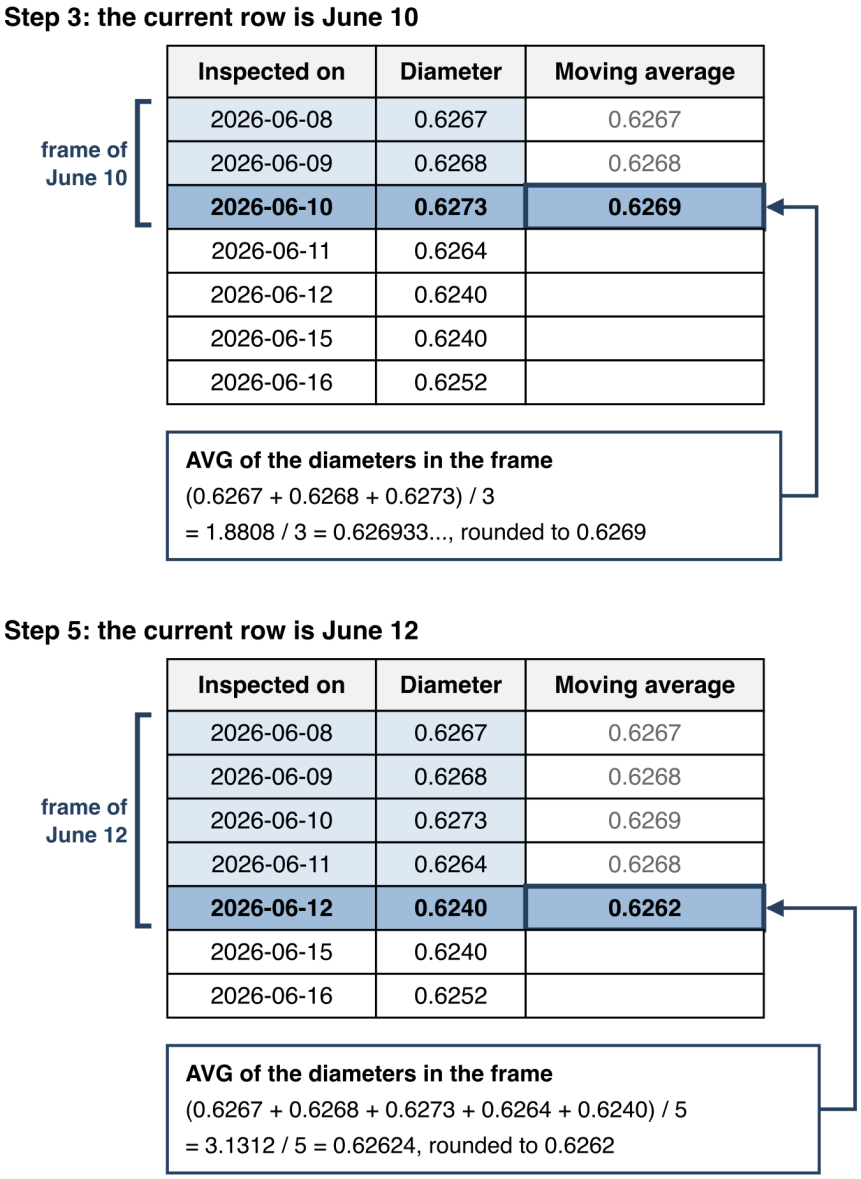


Figure 4.5.: The Frame Moving Down the Rows, for a Five-Inspection Moving Average

holds June 8 to 12, and its average is $(0.6267 + 0.6268 + 0.6273 + 0.6264 + 0.6240)/5 = 0.62624$, which rounds to 0.6262.

Figure 4.6 shows all seven steps at once. Unlike the frame of a running total, this frame moves at both ends: at each step after June 12 it gains the current row and drops the oldest, so its shading is a band of five rows that slides down one row at a time, shorter in the first four columns, where the average of June 8 is the measurement itself. The frame of June 16 holds June 10 to 16, since June 13 and 14 were a weekend with no inspection: `ROWS` counts rows, not days.

		The frame, when the current row is the inspection of						
Inspected on	**Diameter**	**June 8**	**June 9**	**June 10**	**June 11**	**June 12**	**June 15**	**June 16**
2026-06-08	0.6267	**0.6267**	0.6267	0.6267	0.6267	0.6267		
2026-06-09	0.6268		**0.6268**	0.6268	0.6268	0.6268	0.6268	
2026-06-10	0.6273			**0.6273**	0.6273	0.6273	0.6273	0.6273
2026-06-11	0.6264				**0.6264**	0.6264	0.6264	0.6264
2026-06-12	0.6240					**0.6240**	0.6240	0.6240
2026-06-15	0.6240						**0.6240**	0.6240
2026-06-16	0.6252							**0.6252**
AVG of the shaded values above		**0.6267**	**0.6268**	**0.6269**	**0.6268**	**0.6262**	**0.6257**	**0.6254**

Figure 4.6.: The Frame of a Moving Average, ROWS BETWEEN 4 PRECEDING AND CURRENT ROW

The short frames are where a moving average breaks. The `WHERE` clause began the partition at June 8, so the first four averages of Table 4.21 are averages of fewer than five inspections, although an axle was inspected on June 5 and earlier. The worked example below shows how to give the first rows full frames.

The frame also explains a trap. `FIRST_VALUE(x)` and `LAST_VALUE(x)` return the value of `x` in the first and last rows of the frame, in the order that `OVER` gives. For example, a planner reviewing the HT-100's demand

asks for each of its order lines beside the quantity of the first line and of the latest. The natural query orders both functions by the order date, with the line's key to break the tie of January 15, and a third column gives `LAST_VALUE` a frame extended to the partition's end:

Table 4.22.: The First and Last Quantities of the HT-100's Lines

```
SELECT l.order_line_pk, o.order_date, l.quantity,
       FIRST_VALUE(l.quantity) OVER (
         ORDER BY o.order_date, l.order_line_pk
       ) AS first_qty,
       LAST_VALUE(l.quantity) OVER (
         ORDER BY o.order_date, l.order_line_pk
       ) AS last_default,
       LAST_VALUE(l.quantity) OVER (
         ORDER BY o.order_date, l.order_line_pk
         ROWS BETWEEN UNBOUNDED PRECEDING
                  AND UNBOUNDED FOLLOWING
       ) AS last_whole
FROM distribution.order_line AS l
JOIN distribution.customer_order AS o
  ON o.customer_order_pk = l.customer_order_fk
WHERE l.product_fk = 1
ORDER BY o.order_date, l.order_line_pk;
```

order_line_pk	order_date	quantity	first_qty	last_default	last_whole
1	2026-01-05	10	10	10	30
4	2026-01-12	12	10	12	30
7	2026-01-15	20	10	20	30
9	2026-01-15	10	10	10	30
11	2026-01-28	25	10	25	30
14	2026-02-03	8	10	8	30
16	2026-02-18	15	10	15	30
19	2026-03-04	30	10	30	30

The `first_qty` column of Table 4.22 is 10 in every row, as the planner asked, since every frame begins at line 1. The `last_default` column is not the latest quantity, 30; it repeats each row's own quantity. Figure 4.7

shows why, for line 4. With an `ORDER BY` and the default frame, the frame ends at the current row, so the last row of the frame of line 4 is line 4 itself, and `LAST_VALUE` returns its 12. Extended with `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`, the frame of every row is the whole partition, its last row is line 19, and `last_whole` is 30 in every row.

The default frame can be wrong in a second way. Had `OVER` ordered by the date alone, the default frame of line 7 would end at its last peer, line 9, and `LAST_VALUE` would return line 9's quantity, 10: neither line 7's own 20 nor the latest 30. `RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` extends the frame as well; either form reaches the partition's last row. `FIRST_VALUE` needs no such repair, since the default frame already begins at the partition's first row.

4.5.5. Functions that ignore the frame

Ranking functions number the rows of each partition in the order that `OVER` gives. `ROW_NUMBER` numbers them 1, 2, 3, and so on, breaking ties arbitrarily; `RANK` gives tied rows the same rank and skips the ranks they use up; `DENSE_RANK` gives tied rows the same rank and skips nothing. A ranking function needs the order and ignores the frame. The following query ranks every customer that ordered in the year of history by its number of orders. It is a single query: the ranks are computed for the groups that `GROUP BY` forms, and `ORDER BY` lists the result by rank.

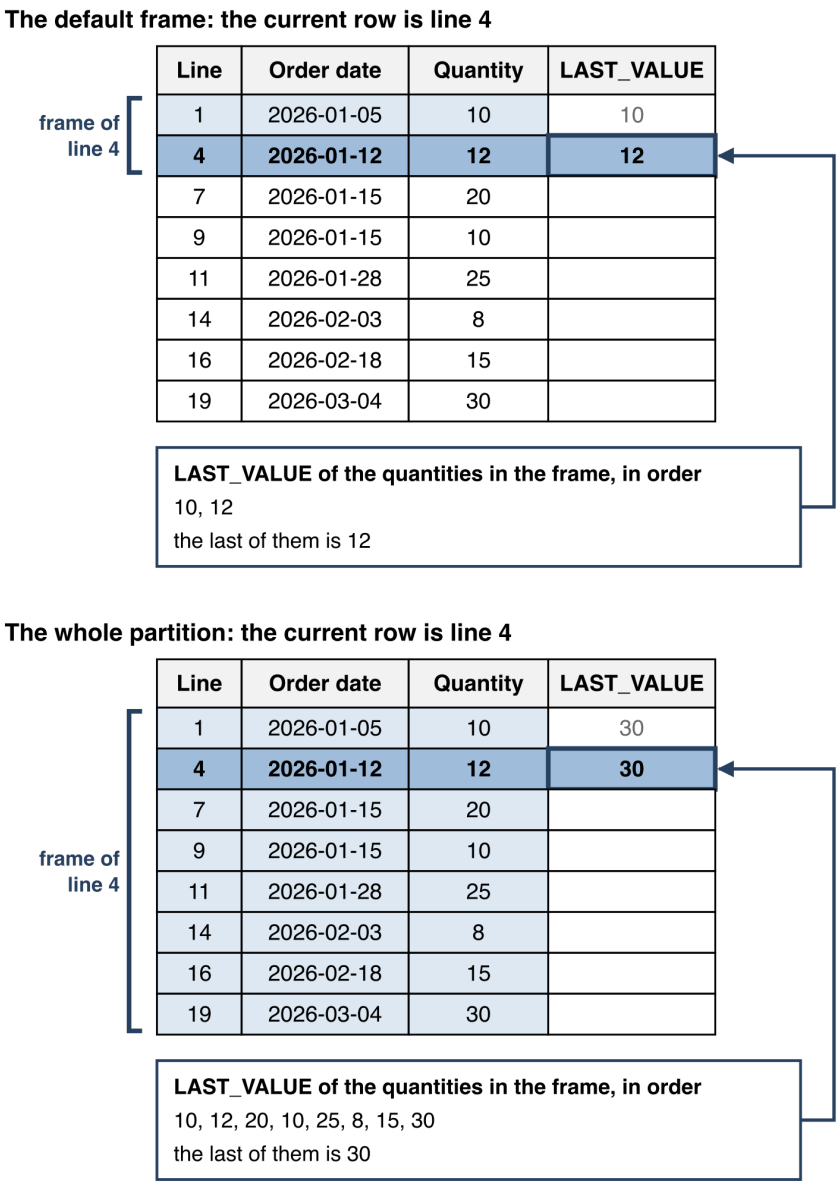


Figure 4.7.: LAST_VALUE for Line 4, under the Default Frame and under the Whole Partition

Table 4.23.: Customers Ranked by Orders in the Year

```
SELECT c.customer_number,
       COUNT(*) AS orders,
       RANK() OVER (ORDER BY COUNT(*) DESC)
         AS rank_skipping,
       DENSE_RANK() OVER (ORDER BY COUNT(*) DESC)
         AS rank_dense
FROM distribution.customer AS c
JOIN distribution.customer_order AS o
  ON o.customer_fk = c.customer_pk
WHERE o.order_date >= DATE '2026-04-01'
  AND o.order_date < DATE '2027-04-01'
GROUP BY c.customer_number
ORDER BY rank_skipping, c.customer_number;
```

customer_number	orders	rank_skipping	rank_dense
C-1014	45	1	1
C-1023	45	1	1
C-1003	44	3	2
C-1006	43	4	3
C-1001	42	5	4
C-1011	28	6	5
C-1002	27	7	6
C-1013	27	7	6
C-1026	26	9	7
C-1017	25	10	8
C-1020	25	10	8
C-1029	24	12	9
C-1005	23	13	10
C-1009	22	14	11
C-1007	21	15	12
C-1022	18	16	13
C-1025	16	17	14
C-1004	15	18	15
C-1028	12	19	16
C-1010	11	20	17
C-1015	10	21	18
C-1030	9	22	19
C-1016	8	23	20
C-1019	8	23	20
C-1027	8	23	20
C-1012	6	26	21
C-1018	6	26	21
C-1024	6	26	21
C-1021	2	29	22

192

Table 4.23 has a row for each of the 29 customers that ordered in the year, so it scrolls within its box. Two customers tie for first place with 45 orders each. `RANK` gives both 1 and gives the next customer 3, since two customers finished ahead of it; `DENSE_RANK` gives the next customer 2. The column aliases avoid the names `rank` and `dense_rank`, since `RANK` is a reserved word of the SQL standard. Notice that the window function here ranks groups: `COUNT(*)` is computed by `GROUP BY` first, and the window function then orders the groups by it, as step 5 of the order of evaluation promises.

Two more window functions reach into other rows of the partition. `LAG(x)` returns the value of `x` in the row before the current one, and `LEAD(x)` the value in the row after; each returns NULL where there is no such row. Like the ranking functions, they need the order and ignore the frame. They compare each row with its neighbor, which is how a drift or a sudden change in a series shows itself, as the worked example below illustrates.

4.5.6. When a window query takes two steps

A window function needs nothing more than the `OVER` clause: every query of this section so far is an ordinary query. The order of evaluation, however, creates two situations in which a window query takes two steps. First, a window function cannot appear in `WHERE` or `HAVING`, since both are evaluated before it. A query that keeps rows by a window function's result must therefore compute the window function in one step and filter in the next. Second, a query may show only some rows while its frames must reach more rows than it shows. It must then compute the window functions before it restricts the rows; the short frames of Figure 4.6 are the cost of restricting first. Recall from Section 3.10 that a common table expression names a step, and a later query in the same statement reads its result as a table. That is how the two-step queries of this section are written. A subquery in the `FROM` clause would do the same; the common table expression reads from the top down, in the order of the steps.

A sales manager who wants only the top five ranks of Table 4.23 must keep the rows whose rank is at most 5, a condition on a window function's result, which is the first of the two situations. `LIMIT 5` would not do, since it cuts the list at five rows whether or not a tie straddles the cut (Section 3.4). `FETCH FIRST 5 ROWS WITH TIES`, from the same section, would keep the right rows here, since a rank of 5 or less means the first five rows after sorting, with their ties. A condition of any other kind, such as a dense rank of 5 or less (the customers with the five highest order counts, six customers here), needs the general form. The ranking query becomes the first step, a common table expression, and the filter the second:

Table 4.24.: The Top Five Ranks of Customers by Orders

```
WITH ranked AS (
  SELECT c.customer_number,
         COUNT(*) AS orders,
         RANK() OVER (ORDER BY COUNT(*) DESC)
           AS rank_skipping,
         DENSE_RANK() OVER (ORDER BY COUNT(*) DESC)
           AS rank_dense
  FROM distribution.customer AS c
  JOIN distribution.customer_order AS o
    ON o.customer_fk = c.customer_pk
  WHERE o.order_date >= DATE '2026-04-01'
    AND o.order_date < DATE '2027-04-01'
  GROUP BY c.customer_number
)
SELECT customer_number, orders, rank_skipping, rank_dense
FROM ranked
WHERE rank_skipping <= 5
ORDER BY rank_skipping, customer_number;
```

customer_number	orders	rank_skipping	rank_dense
C-1014	45	1	1
C-1023	45	1	1
C-1003	44	3	2
C-1006	43	4	3
C-1001	42	5	4

The common table expression `ranked` is the query of Table 4.23 without its `ORDER BY`, and the final query reads it as a table, so its `WHERE` clause may test `rank_skipping` like any column. The five rows of Table 4.24 are the customers ranked 1 to 5.

Example 4.2 (Tracking tool wear on the axle diameter)**.** *The system.* The quality schema of stage G. Each machined axle (part AX-10) is inspected once a day, and its diameter is recorded in the `quality.measurement` table, with the time of the inspection in `quality.inspection`. The diameter's specification is 0.625 inches, within limits of 0.620 and 0.630. The cutting tool wears as it cuts, so the diameter it leaves grows until the tool is changed.

The request. The machining supervisor asks when the tool has been changed, and how the diameter moved during June 2026, smoothed so that the trend is visible through the day-to-day scatter.

A tool change shows as a sudden drop in the diameter from one inspection to the next. `LAG` supplies the previous measurement, and the change is the difference. Let's first look at one week of June directly, in a single query:

Table 4.25.: The Axle Diameter from June 8 to 16, with Its Changes

```
SELECT CAST(i.inspected_at AS DATE) AS inspected_on,
       m.measured_value,
       LAG(m.measured_value)
         OVER (ORDER BY i.inspected_at) AS previous_value,
       m.measured_value
         - LAG(m.measured_value)
             OVER (ORDER BY i.inspected_at) AS change
FROM quality.measurement AS m
JOIN quality.inspection AS i
  ON i.inspection_pk = m.inspection_fk
WHERE m.characteristic_fk = 1
  AND i.inspected_at >= DATE '2026-06-08'
  AND i.inspected_at < DATE '2026-06-17'
ORDER BY i.inspected_at;
```

inspected_on	measured_value	previous_value	change
2026-06-08	0.6267	NULL	NULL
2026-06-09	0.6268	0.6267	0.0001
2026-06-10	0.6273	0.6268	0.0005
2026-06-11	0.6264	0.6273	-0.0009
2026-06-12	0.6240	0.6264	-0.0024
2026-06-15	0.6240	0.6240	0.0000
2026-06-16	0.6252	0.6240	0.0012

Table 4.25 shows the tool change of June 12: the diameter falls by 0.0024 inches from one inspection to the next, several times the ordinary day-to-day change. It also shows the limit of the single query. The first row has no previous value, although an axle was inspected on June 5, because the `WHERE` clause removed that inspection before the window was computed. And to list only the drops, the query would have to test `change` in `WHERE`,

where no window exists yet.

Both limits are the two situations of Section 4.5.6, and both call for two steps. The first step, a common table expression, computes the change for every diameter measurement; the second keeps the rows whose change is below −0.002 inches, a larger step than the day-to-day variation:

Table 4.26.: Drops in the Axle Diameter

```
WITH diameter AS (
  SELECT i.inspected_at, m.measured_value,
         m.measured_value
           - LAG(m.measured_value)
               OVER (ORDER BY i.inspected_at) AS change
  FROM quality.measurement AS m
  JOIN quality.inspection AS i
    ON i.inspection_pk = m.inspection_fk
  WHERE m.characteristic_fk = 1
)
SELECT inspected_at, measured_value, change
FROM diameter
WHERE change < -0.002
ORDER BY inspected_at;
```

inspected_at	measured_value	change
2026-04-01 08:00:00	0.6239	-0.0065
2026-05-07 08:00:00	0.6248	-0.0026
2026-06-12 08:00:00	0.6240	-0.0024
2026-07-20 08:00:00	0.6232	-0.0031
2026-08-25 08:00:00	0.6241	-0.0034
2026-09-30 08:00:00	0.6234	-0.0040

Table 4.26 finds six drops. The first, on April 1, 2026, lies between the last inspection of stage Q's February data and the first of the year of history; the other five fall roughly five weeks apart, the tool's working life.

The smoothed trend for June is a five-inspection moving average. It is the second situation again: the average must be computed over every inspection and only then restricted to June, since a `WHERE` clause on June in the same query would remove the May inspections from the frames of the first June rows, just as it shortened the first frames of Figure 4.6:

Table 4.27.: The Axle Diameter in June 2026, with a Moving Average

```
WITH diameter AS (
  SELECT i.inspected_at, m.measured_value,
         AVG(m.measured_value) OVER (
           ORDER BY i.inspected_at
           ROWS BETWEEN 4 PRECEDING AND CURRENT ROW
         ) AS moving_average
  FROM quality.measurement AS m
  JOIN quality.inspection AS i
    ON i.inspection_pk = m.inspection_fk
  WHERE m.characteristic_fk = 1
)
SELECT CAST(inspected_at AS DATE) AS inspected_on,
       measured_value,
       ROUND(moving_average, 4) AS moving_average
FROM diameter
WHERE inspected_at >= DATE '2026-06-01'
  AND inspected_at < DATE '2026-07-01'
ORDER BY inspected_at;
```

inspected_on	measured_value	moving_average
2026-06-01	0.6254	0.6255
2026-06-02	0.6267	0.6258
2026-06-03	0.6262	0.6260
2026-06-04	0.6264	0.6260
2026-06-05	0.6267	0.6263
2026-06-08	0.6267	0.6265
2026-06-09	0.6268	0.6266
2026-06-10	0.6273	0.6268
2026-06-11	0.6264	0.6268
2026-06-12	0.6240	0.6262
2026-06-15	0.6240	0.6257
2026-06-16	0.6252	0.6254
2026-06-17	0.6245	0.6248
2026-06-18	0.6243	0.6244
2026-06-19	0.6251	0.6246
2026-06-22	0.6249	0.6248
2026-06-23	0.6249	0.6247
2026-06-24	0.6255	0.6249
2026-06-25	0.6252	0.6251
2026-06-26	0.6253	0.6252
2026-06-29	0.6258	0.6253
2026-06-30	0.6250	0.6254

The check. Each moving average is the mean of its row and the four before it. The row count can be reasoned too: the axle is inspected once each working day, and June 2026 has 22 working days, so 22 rows are expected, which Table 4.27 has.

Interpretation. The moving average climbs through the first part of June and falls after the tool change of June 12, then climbs again: the sawtooth of tool wear. Because the diameter grows steadily, the supervisor can change the tool on a schedule, before the average approaches the upper limit, rather than waiting for an axle to fail inspection.

Where it breaks. The moving average lags the process: after a tool change it mixes old-tool and new-tool measurements for four inspections, so it shows the drop gradually although the change was sudden. The threshold of -0.002 that detects a change was chosen by looking at the data; a larger day-to-day scatter would call for a larger threshold, or for recording tool changes in the database instead of inferring them.

Whenever a window function's result looks wrong, you should write out the frame of a few rows by hand, as Table 4.19 does, or shade them in a grid like Figure 4.4. The next section turns from rows related by a partition to rows related by a chain.

4.6. Recursive queries

This section presents recursive queries, which follow a chain of rows to any depth, and applies them to the bill of materials.

Recall from Section 2.4 that the `production.bom_line` table of stage P2 records, for each assembly, the components it is made of and how many of each: `bom_line (bom_line_pk, assembly_fk, component_fk, quantity_per)`, where both keys refer to `part`. A component may itself be an assembly. The HT-100 hand truck is made of a frame, two wheel assemblies, two grips, and a hardware kit; each wheel assembly is made of

an axle, a tire, a rim, and two bearings; and each axle is machined from bar stock. A request for everything that goes into an HT-100 must follow those links down as far as they go, and no fixed number of joins does that, since the depth differs from product to product.

A **recursive query** is a common table expression that refers to itself. It has two parts joined by `UNION ALL`. The first, the **anchor**, is an ordinary query that returns the starting rows. The second, the **recursive part**, joins the rows of the previous step to a table to find the next rows. The database runs the anchor once, then runs the recursive part on the rows the previous step produced, again and again, and stops when a step produces no rows. Figure 4.8 draws the process.

Follow Figure 4.8 from the top. The anchor runs once, and its rows go into the result and also become the **working table**, the rows of the most recent step. The recursive part joins the working table, and only the working table, to `bom_line`, the dashed link, to find the next rows. If it finds any, they are added to the result and replace the working table, and the loop goes round again. When a step finds none, the query takes the "no" branch and stops, and the result holds the rows of every step. Notice that each pass through the loop sees only the previous step's rows, not everything found so far; that is why a part found in step 1 is not exploded again in step 2.

Let's run the steps by hand for the HT-100. Each row records a part, its depth below the product (the number of links from the product down to it), and how many of it one HT-100 needs, which is the quantity of its parent times the component's `quantity_per`:

Table 4.28.: Exploding the HT-100 by Hand, One Step at a Time

Step	Rows produced (part, depth, quantity per HT-100)
Anchor	HT-100, 0, 1

Step	Rows produced (part, depth, quantity per HT-100)
1	FR-110, 1, 1; WA-10, 1, 2; GR-01, 1, 2; HW-01, 1, 1
2	TU-12, 2, 14 (from FR-110); AX-10, 2, 2; TI-10, 2, 2; RM-10, 2, 2; BR-10, 2, 4 (from WA-10)
3	BS-075, 3, 2.5 (from AX-10, 2×1.25)
4	none: bar stock has no components, so the query stops

Each step of Table 4.28 uses only the rows of the step before it, the working table of Figure 4.8. Step 2, for example, finds components of FR-110 and WA-10, the step 1 rows that are assemblies; GR-01 and HW-01 are purchased and have no components, so they contribute nothing.

Figure 4.9 draws the same steps as the product structure tree familiar from material requirements planning. It is drawn from the `bom_line` table of stage P3 by a recursive query like the one below, so it shows what the data holds.

Each box of Figure 4.9 is a part, with its number, its description, and the quantity needed for one HT-100. Each line joins an assembly to one of its components and carries the component's `quantity_per`, the number of the component in one of the assembly. The labels at the left, and the shading, mark the step that finds each part, so each row of Table 4.28 is one level of the tree: the anchor is the top box, step 1 the second level, and so on. Read down any branch and the quantities multiply. The branch to the bar stock reads two wheel assemblies per truck, one axle per wheel assembly, and 1.25 feet of bar stock per axle, so 2.5 feet per truck, the last row of Table 4.28. The lowest label, step 4, has no boxes: its working table held only BS-075, which has no components, so the loop of Figure 4.8 took

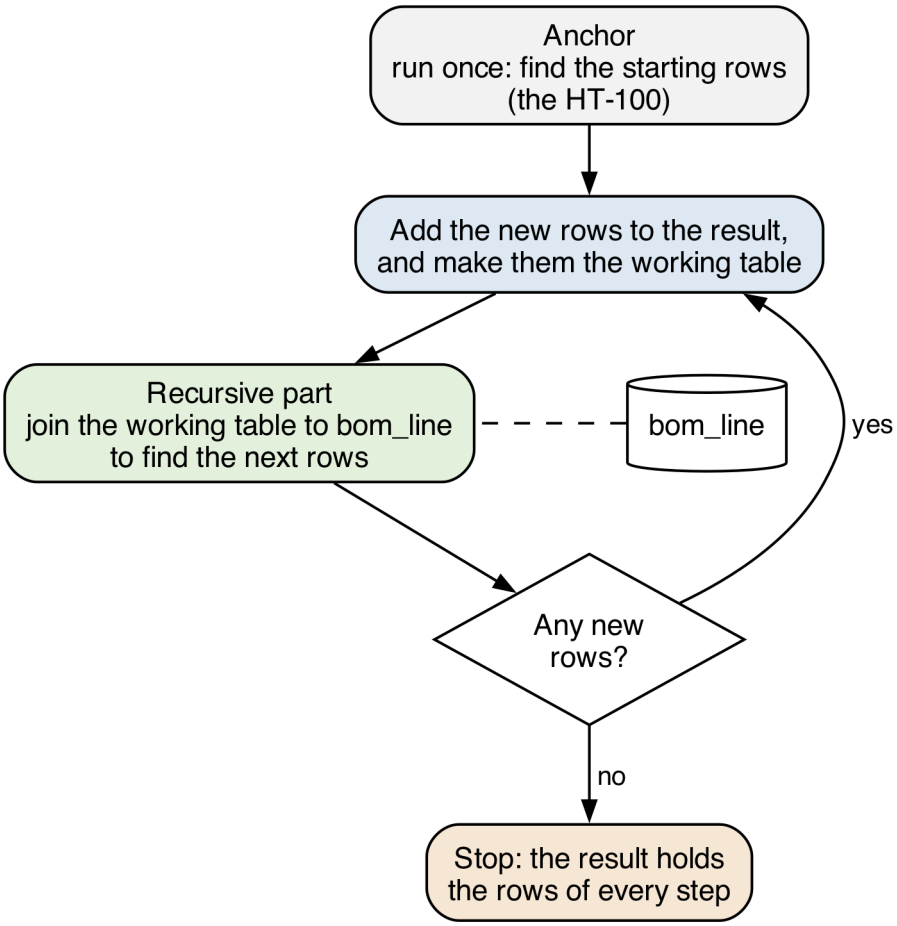


Figure 4.8.: How a Recursive Query Runs

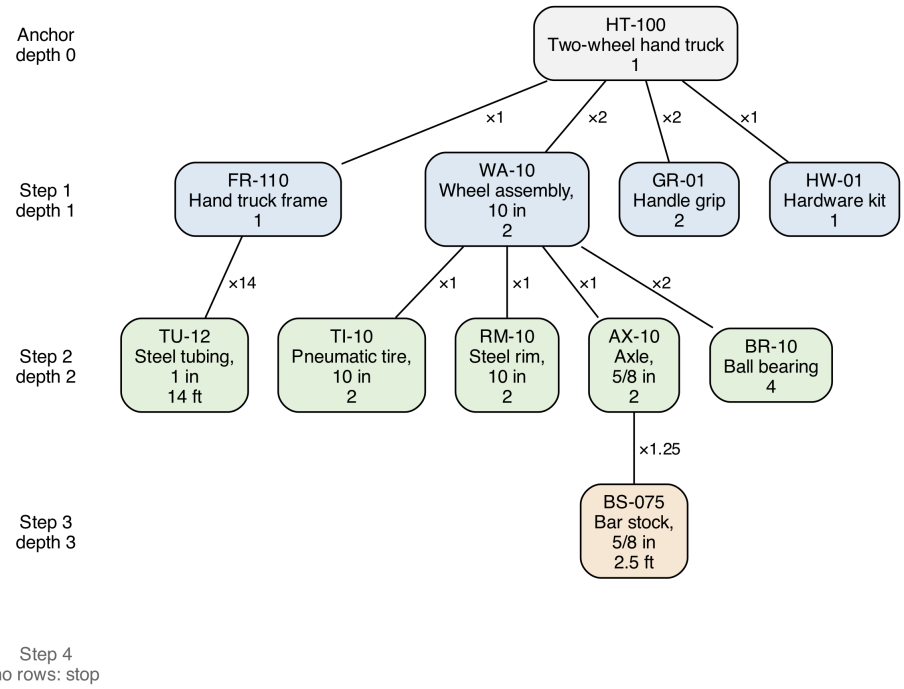


Figure 4.9.: The HT-100 Product Structure, Shaded by the Step That Finds Each Part

its “no” branch. The figure shows the shape of the explosion, and the table records the rows that each step adds; together they predict the query’s result. The query writes the anchor and the step:

Table 4.29.: Everything That Goes into One HT-100

```
WITH RECURSIVE explosion AS (
  SELECT part_pk AS part_fk, 0 AS depth,
         CAST(1 AS NUMERIC) AS quantity
  FROM production.part
  WHERE part_number = 'HT-100'
  UNION ALL
  SELECT b.component_fk, e.depth + 1,
         e.quantity * b.quantity_per
  FROM explosion AS e
  JOIN production.bom_line AS b
    ON b.assembly_fk = e.part_fk
)
SELECT e.depth, p.part_number, p.description,
       ROUND(e.quantity, 3) AS quantity
FROM explosion AS e
JOIN production.part AS p
  ON p.part_pk = e.part_fk
ORDER BY e.depth, p.part_number;
```

depth	part_number	description	quantity
0	HT-100	Two-wheel hand truck	1.000
1	FR-110	Hand truck frame	1.000
1	GR-01	Handle grip	2.000
1	HW-01	Hardware kit	1.000
1	WA-10	Wheel assembly, 10 in	2.000
2	AX-10	Axle, 5/8 in	2.000
2	BR-10	Ball bearing	4.000
2	RM-10	Steel rim, 10 in	2.000
2	TI-10	Pneumatic tire, 10 in	2.000
2	TU-12	Steel tubing, 1 in	14.000
3	BS-075	Bar stock, 5/8 in	2.500

Table 4.29 has the eleven rows of Table 4.28, the eleven boxes of Figure 4.9. In the recursive part, the name `explosion` stands for the working table of Figure 4.8, and the part joins it to `bom_line` through `assembly_fk`, following the recursive association from an assembly to its components. The quantities multiply down the chain, so the 2.5 feet of bar stock is two axles' worth of 1.25 feet each. A planner who multiplies the last column by a work order's quantity has the materials the work order needs.

A recursive query needs care in two ways. First, it stops only when a step produces no rows. If the data held a cycle, a part that is directly or indirectly its own component, the steps would never end; a condition such as `WHERE e.depth < 20` in the recursive part bounds the depth, and PostgreSQL 14 and later also offer a `CYCLE` clause that detects a repeat. Second, `UNION ALL` keeps every row, which is what an explosion needs: a part used in two places, such as the hardware kit in every product, must appear once for each place. `UNION` would remove rows that repeat exactly, which is right for a list of distinct parts but wrong for quantities.

Table 4.29 lists the parts by depth, so the axle appears three rows away from the wheel assembly it belongs to. A planner often wants the tree instead, each part under its assembly. The recursive part can carry, for each row, the **path** of part numbers from the top of the tree down to the row's part; sorting by the path then lists each part right after its assembly, and indenting by the depth draws the levels:

Table 4.30.: The HT-100 Explosion as an Indented Tree

```
WITH RECURSIVE explosion AS (
  SELECT part_pk AS part_fk, 0 AS depth,
         CAST(part_number AS VARCHAR(200)) AS path
  FROM production.part
  WHERE part_number = 'HT-100'
  UNION ALL
  SELECT b.component_fk, e.depth + 1,
         CAST(e.path || '/' || p.part_number
              AS VARCHAR(200))
  FROM explosion AS e
  JOIN production.bom_line AS b
    ON b.assembly_fk = e.part_fk
  JOIN production.part AS p
    ON p.part_pk = b.component_fk
)
SELECT REPEAT('. ', e.depth) || p.part_number AS part,
       p.description, e.path
FROM explosion AS e
JOIN production.part AS p
  ON p.part_pk = e.part_fk
ORDER BY e.path;
```

part	description	path
HT-100	Two-wheel hand truck	HT-100
. FR-110	Hand truck frame	HT-100/FR-110
. . TU-12	Steel tubing, 1 in	HT-100/FR-110/TU-12
. GR-01	Handle grip	HT-100/GR-01
. HW-01	Hardware kit	HT-100/HW-01
. WA-10	Wheel assembly, 10 in	HT-100/WA-10
. . AX-10	Axle, 5/8 in	HT-100/WA-10/AX-10
. . . BS-075	Bar stock, 5/8 in	HT-100/WA-10/AX-10/BS-075
. . BR-10	Ball bearing	HT-100/WA-10/BR-10
. . RM-10	Steel rim, 10 in	HT-100/WA-10/RM-10
. . TI-10	Pneumatic tire, 10 in	HT-100/WA-10/TI-10

Table 4.30 holds the same eleven parts as Table 4.29, now in the order of Figure 4.9 read branch by branch: the frame and its tubing, the grip, the hardware kit, and the wheel assembly with its axle, bar stock, bearing, rim, and tire. The anchor starts each path with the top part's number, and the recursive part appends the component's number to its assembly's path, so the path of the bar stock, `HT-100/WA-10/AX-10/BS-075`, spells its branch. A path sorts after its assembly's path because it begins with it. The dots come from `REPEAT`, which writes its text as many times as the depth; `REPEAT` is a PostgreSQL function that most databases share, though it is not in the standard. Notice the `CAST` in both parts: the type of a recursive query's column is set by the anchor, so the path is given a length long enough for the deepest branch.

`WITH RECURSIVE` is standard SQL, and PostgreSQL requires the keyword `RECURSIVE`; some databases, such as SQL Server, omit it. Recursive queries serve any data that refers to itself: an organization chart, a network of routes, a chain of work orders each waiting on another.

4.7. Analytical queries on operational data

This section applies the chapter's tools to three questions that industrial engineers ask of operational data: how reliably orders ship on time, how long stock will last, and whether a process is in statistical control.

The first is on-time shipment. A shipment is on time when it leaves on or before its order's requested ship date. Recall that some orders have no requested date, and for those the question has no answer. A `CASE` expression can return 1 for on time, 0 for late, and NULL for no requested date, and `AVG` of the result is then the fraction on time among the shipments that have a date, because `AVG` ignores NULLs (Section 4.2):

Table 4.31.: The Fraction of Shipments on Time, by Month

```
SELECT EXTRACT(YEAR FROM s.shipped_on) AS year,
       EXTRACT(MONTH FROM s.shipped_on) AS month,
       COUNT(*) AS shipments,
       ROUND(AVG(CASE
                   WHEN o.requested_ship_date IS NULL
                     THEN NULL
                   WHEN s.shipped_on <= o.requested_ship_date
                     THEN 1.0
                   ELSE 0.0
                 END), 3) AS on_time
FROM distribution.shipment AS s
JOIN distribution.customer_order AS o
  ON o.customer_order_pk = s.customer_order_fk
WHERE s.shipped_on >= DATE '2026-04-01'
  AND s.shipped_on < DATE '2027-04-01'
GROUP BY EXTRACT(YEAR FROM s.shipped_on),
         EXTRACT(MONTH FROM s.shipped_on)
ORDER BY year, month;
```

year	month	shipments	on_time
2026	4	56	0.980
2026	5	59	0.982
2026	6	66	0.937
2026	7	60	0.900
2026	8	45	0.886
2026	9	47	0.936
2026	10	50	0.932
2026	11	66	0.969
2026	12	54	0.887
2027	1	30	0.929
2027	2	39	0.946
2027	3	69	0.971

Table 4.31 shows on-time performance between about 89 and 98 percent, highest in the spring and lowest in July, August, and December. The query says when performance slipped, not why; the next question would compare the late shipments' warehouses, carriers, and order sizes. A measure built this way states its denominator: the shipments whose orders requested a date. A report should say so, since a reader will otherwise assume it is all shipments.

The second question is how long the stock on hand will last at the rate it is being shipped, its **days of supply**. The rate is each product's units shipped from each warehouse over the year, divided by 365 days, and the days of supply is the quantity on hand divided by that daily rate:

Table 4.32.: Days of Supply for Each Stock Level

```
WITH demand AS (
  SELECT s.warehouse_fk, l.product_fk,
         SUM(sl.quantity) AS units
  FROM distribution.shipment_line AS sl
  JOIN distribution.shipment AS s
    ON s.shipment_pk = sl.shipment_fk
  JOIN distribution.order_line AS l
    ON l.order_line_pk = sl.order_line_fk
  WHERE s.shipped_on >= DATE '2026-04-01'
    AND s.shipped_on < DATE '2027-04-01'
  GROUP BY s.warehouse_fk, l.product_fk
)
SELECT st.warehouse_fk, st.product_fk,
       st.quantity_on_hand, d.units AS units_shipped,
       ROUND(st.quantity_on_hand / (d.units / 365.0), 1)
         AS days_of_supply
FROM distribution.stock_level AS st
JOIN demand AS d
  ON d.warehouse_fk = st.warehouse_fk
 AND d.product_fk = st.product_fk
ORDER BY days_of_supply, st.warehouse_fk, st.product_fk;
```

warehouse_fk	product_fk	quantity_on_hand	units_shipped	days_of_supply
2	4	0	851	0.0
2	2	15	1832	3.0
1	3	18	1976	3.3
1	2	35	1760	7.3
2	3	40	2013	7.3
2	1	60	1919	11.4
1	4	30	698	15.7
1	1	120	1635	26.8

Table 4.32 orders the stock levels from the most urgent. The GC-400 at Tulsa has none; the HT-200 at Tulsa and the GC-300 at Fayetteville have about three days of stock. Two cautions apply. The rate for the GC-400 averages its sales over 365 days although it was sold only from September, so its true daily rate, and its urgency, are higher than the table shows. And the stock levels are those recorded in the `stock_level` table, which the history of stage G does not update, so the table measures the recorded stock against a year's demand rather than tracking a real warehouse day by day.

The third question comes from statistical process control. A **control chart** plots a measured characteristic over time between a lower and an upper **control limit**, computed from the process's own variation. A point outside the limits signals a cause worth investigating. When a characteristic is measured once per sample, as the axle length is at each inspection, the chart is an individuals chart. Its center line is the average measurement, and its limits are the center line plus and minus $2.66\,\overline{MR}$, where $\overline{MR}$ is the average **moving range**, the absolute difference between consecutive measurements. The constant 2.66 converts the average moving range into three standard deviations of the process. The moving range is a `LAG`, and the rest is aggregation:

Table 4.33.: Individuals Chart Limits for the Axle Length

```
WITH axle_length AS (
  SELECT m.measured_value,
         ABS(m.measured_value
             - LAG(m.measured_value)
                 OVER (ORDER BY i.inspected_at))
           AS moving_range
  FROM quality.measurement AS m
  JOIN quality.inspection AS i
    ON i.inspection_pk = m.inspection_fk
  WHERE m.characteristic_fk = 2
)
SELECT COUNT(*) AS measurements,
       ROUND(AVG(measured_value), 4) AS center_line,
       ROUND(AVG(moving_range), 4) AS average_range,
       ROUND(AVG(measured_value)
             - 2.66 * AVG(moving_range), 4) AS lower_limit,
       ROUND(AVG(measured_value)
             + 2.66 * AVG(moving_range), 4) AS upper_limit,
       ROUND(STDDEV_SAMP(measured_value), 4) AS std_dev
FROM axle_length;
```

measurements	center_line	average_range	lower_limit	upper_limit	std_dev
139	11.9992	0.0140	11.9618	12.0365	0.0134

Table 4.33 gives limits of about 11.962 and 12.037 inches around a center line of 11.999. The first moving range is NULL, since the first measurement has no predecessor, and `AVG` ignores it. The last column, `STDDEV_SAMP`, is the sample standard deviation of all the measurements, shown for comparison; standard practice computes an individuals chart's limits from the moving range, because a slow drift in the process inflates the standard deviation

but hardly changes the range from one measurement to the next. The measurements outside the limits are then found by comparing each with them; an exercise asks for that query.

The last question of this section asks whether a measurement depends on who takes it. It uses one more kind of aggregate, a **percentile**, the value below which a given fraction of the group falls. `PERCENTILE_CONT(0.5)` is the median. Its syntax differs from the other aggregates: the column to be ordered is named after `WITHIN GROUP`, because a percentile depends on the order of the values.

Example 4.3 (Checking whether an inspector reads coating thickness differently)**.** *The system.* The quality schema of stage G. The powder coat on the hand truck frame (FR-110) and the cart frame (CF-310) is measured for thickness at inspection, by one of two inspectors, Linda Nguyen or Robert Hayes, who alternate. The specification is 3.0 mils, within limits of 2.0 and 4.0.

The request. The quality manager has noticed that frames inspected on some days read thinner, and asks whether the thickness measured depends on the inspector.

The request compares groups formed by characteristic and inspector, so the query groups by both and computes the count, the average, and the median of each group:

Table 4.34.: Coating Thickness by Characteristic and Inspector

```
SELECT ch.characteristic_pk, o.name AS inspector,
       COUNT(*) AS measurements,
       ROUND(AVG(m.measured_value), 2) AS average,
       PERCENTILE_CONT(0.5)
         WITHIN GROUP (ORDER BY m.measured_value)
         AS median
FROM quality.measurement AS m
JOIN quality.characteristic AS ch
  ON ch.characteristic_pk = m.characteristic_fk
JOIN quality.inspection AS i
  ON i.inspection_pk = m.inspection_fk
JOIN production.operator AS o
  ON o.operator_pk = i.operator_fk
WHERE ch.name = 'Coating thickness'
GROUP BY ch.characteristic_pk, o.name
ORDER BY ch.characteristic_pk, o.name;
```

characteristic_pk	inspector	measurements	average	median
4	Linda Nguyen	29	2.44	2.5
4	Robert Hayes	29	3.13	3.2
5	Linda Nguyen	29	2.61	2.6
5	Robert Hayes	30	3.11	3

The check. The groups should split the measurements of each characteristic roughly in half, since the inspectors alternate, and Table 4.34 has 29 and 29 for the hand truck frame (characteristic 4), and 29 and 30 for the cart frame (characteristic 5).

Interpretation. On both frames, Linda Nguyen's readings are lower than Robert Hayes's, by half a mil to seven tenths of a mil in the average and by

four tenths to seven tenths in the median. The frames come from the same paint line, and the inspectors alternate, so the frames each one measures should be alike; the difference is in the measuring, not in the coating. The next step is a measurement system study, of the gauge and of how each inspector uses it, before any frame is rejected or the paint line adjusted.

Where it breaks. The comparison assumes that the two inspectors see similar frames. If one inspected mostly on days after a change to the paint line, the difference could be real coating, not measurement, and the query cannot tell the two apart. Grouping by inspector and month as well would show whether the difference holds throughout the year.

These questions share a shape: a measure defined with care (on time, days of supply, a control limit), computed with aggregates over groups or windows, and reported with the denominator and the cautions that its definition implies. That care, more than any function, is what makes a query an analysis.

4.8. Summary

This chapter's tools are two clauses, a set of expressions, and a recursive `WITH`. `GROUP BY` and `HAVING` are clauses. Aggregates, `CASE`, and window functions are expressions written inside the clauses of Chapter 3, and `WITH RECURSIVE` extends a common table expression. The order of evaluation gains a step: window functions are computed after the `SELECT` list and before `DISTINCT`.

An aggregate reduces many rows to one value. `COUNT`, `SUM`, `AVG`, `MIN`, and `MAX` summarize the rows a query keeps. They ignore NULL, except that `COUNT(*)` counts rows whatever they hold, and over no rows at all `COUNT` returns zero while the others return NULL.

`GROUP BY` forms groups, and the aggregates are computed once for each. The result has one row per group, so its `SELECT` list may hold

only the grouping columns and aggregates. Forming a few groups by hand, and computing each group's aggregate, predicts the result of any grouped query.

`WHERE` filters rows before grouping, and `HAVING` filters groups after. A condition on a row belongs in `WHERE`, a condition on an aggregate in `HAVING`. An aggregate of an aggregate takes two steps, which a common table expression names. Counting answers requests with the word *every*, but it disagrees with the double `NOT EXISTS` when the set counted is empty.

`CASE` classifies rows by conditions tested in order. It returns the value of the first true condition, so a column that may be NULL needs its own test, best placed first. Inside an aggregate, `CASE` counts or sums rows by class, and one aggregate per class turns rows into the columns of a cross tab.

A window function summarizes a row's group without collapsing the rows. Its `OVER` clause answers three questions for each current row: whose rows (`PARTITION BY`), in what order (`ORDER BY`), and which of them (the frame). `OVER (PARTITION BY ...)` computes an aggregate for each row from the rows of its partition. An `ORDER BY` inside `OVER` changes each row's frame, which by default then runs from the first row to the current row and its peers, so `SUM` becomes a running total; an explicit frame such as `ROWS BETWEEN 4 PRECEDING AND CURRENT ROW` gives a moving average. Ranking functions number rows and treat ties in different ways, and `LAG` and `LEAD` compare a row with its neighbors; both kinds use the order and ignore the frame.

Window functions are computed late, so filtering them takes a step. A window function needs only its `OVER` clause, but it follows `WHERE`, `GROUP BY`, and `HAVING`. A `WHERE` therefore restricts its partitions, and a query that filters on a window function's result, or whose frames must reach rows it does not show, computes the window functions in a common table expression first. Writing out the frame of a few rows by hand explains a window result that looks wrong.

A recursive query follows a chain of rows to any depth. An anchor finds the starting rows, and a recursive part joins the rows of the previous step, the working table, to find the next, until a step finds none. It explodes a bill of materials, multiplying quantities down the chain, and it needs a bound on its depth if the data might hold a cycle.

An analysis is a measure defined with care, then aggregated. On-time shipment, days of supply, control limits, and percentiles are each an aggregate over carefully chosen rows. The definition decides the denominator, and the report must state it, with the cautions that the data implies.

The queries of this chapter and Chapter 3 read a database built by someone else. Chapter 5 turns to the question behind every one of them: why the tables are shaped as they are, and how to design tables of your own.

4.9. Exercises

The exercises use the stages named in each, loaded as Section C.2 explains.

Exercise 4.1. Mark each statement true or false, and correct each false one.

a. `COUNT(requested_ship_date)` and `COUNT(*)` always return the same number.
b. `AVG` of a column that holds NULLs treats each NULL as zero.
c. `SUM` over a set of no rows returns zero.
d. A condition on an aggregate may appear in `WHERE`.
e. `MAX(AVG(unit_price))` finds the largest average price among groups.
f. In a grouped query that does not group by a table's primary key, every column in the `SELECT` list must be named in `GROUP BY` or appear inside an aggregate.

Exercise 4.2. At stage D5, consider the order lines of orders SO-5005 through SO-5008 (`customer_order_fk` 5 through 8).

a. List the lines and form the groups of `GROUP BY product_fk` by hand.
b. For each group, compute `COUNT(*)`, `SUM(quantity)`, and `MAX(quantity)`.
c. Write the query and confirm that it returns your groups.

Exercise 4.3. At stage D5, count the shipments that left each warehouse by each carrier, labeling each row with the warehouse code and the carrier, and show only the warehouse and carrier pairs that occur. Reason out the number of rows and each count before running the query.

Exercise 4.4. An analyst writes the following query at stage G to count each customer's orders.

```
SELECT c.name, c.city, COUNT(*) AS orders
FROM distribution.customer AS c
JOIN distribution.customer_order AS o
  ON o.customer_fk = c.customer_pk
GROUP BY c.name;
```

a. Explain why PostgreSQL rejects the query.
b. The analyst repairs it by removing `c.city`. Explain what could go wrong with the repaired query in a database where two customers share a name.
c. Write the query correctly, with each customer's number, name, and city and the number of orders, counting every order in the database and listing only the customers who have placed one.

Exercise 4.5. At stage G, list the orders placed from April 2026 through March 2027 whose total value (the sum of quantity times unit price over their lines) exceeds $8,000, with the order number, the number of lines, and the total value, largest first.

Exercise 4.6. At stage G, the quality manager asks for the number of defects recorded of every defect type, including any type that has never been recorded, with zero for such a type.

a. Write the query with an inner join and `GROUP BY`. Which defect type is missing from the result, and why?
b. Write the query so that every defect type appears, with zero where no defect has been recorded.
c. What does the result say about the plant's quality problems?

Exercise 4.7. At stage P3, run both methods for "operators certified on every machine" of Section 4.3 against the Assembly work center (work center 5), which has no machines. The double `NOT EXISTS` keeps an operator for whom no machine of the work center lacks a certification; counting groups each operator's certifications on the work center's machines and keeps the operators whose group counts as many certifications as the work center has machines.

a. What does each method return?
b. Explain each result from the way its query is evaluated.
c. The production supervisor's rule is that an operator may staff a work center alone only when certified on every one of its machines, and on at least one. Which answer follows that rule, and what does the result tell the supervisor about Assembly?

Exercise 4.8. At stage P3, a planner writes this classification of work orders:

```
CASE
  WHEN completed_on <= due_on THEN 'on time'
  WHEN completed_on > due_on THEN 'late'
  ELSE 'open'
END
```

a. Evaluate it by hand for WO-7005, WO-7006, and WO-7008.
b. Is this `CASE` correct for every work order? Explain why it needs no test for NULL here, while the `CASE` of Table 4.11 has one.

Exercise 4.9. At stage G, build a cross tab of the units of the HT-100 and HT-200 hand trucks ordered in each quarter from April 2026 through March 2027, with a column for each hand truck and one for both. Do hand trucks show a season, and does it run with or against the garden carts' season of Example 4.1?

Exercise 4.10. At stage D5, the HT-200 (product 2) appears on six order lines.

a. List the lines in the order of their orders' dates, and compute by hand the running total of their quantities, writing out the frame of each row.
b. Compute by hand a three-line moving average of the quantities, with the frame `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`. What is the frame of the first row, and of the second?
c. Write the query that computes both, and confirm your results.

Exercise 4.11. At stage G, rank the customers by the number of orders they placed from April 2026 through March 2027, with `ROW_NUMBER`, `RANK`, and `DENSE_RANK`, and list ranks 5 through 9 of `RANK`.

a. Which customers share a rank, and what rank does each function give them?
b. Why does `ROW_NUMBER` need a second column in its `ORDER BY` to give a reproducible result?

Exercise 4.12. At stage D5, an analyst wants each order line with the highest unit price charged for its product, and writes `LAST_VALUE(unit_price) OVER (PARTITION BY product_fk ORDER BY unit_price)`.

a. Evaluate the expression by hand for the three lines of product 3 in orders SO-5001, SO-5003, and SO-5004, as if the query kept only those three lines (`WHERE product_fk = 3 AND customer_order_fk <= 4`), writing out each row's frame.
b. Explain why the result is not the highest price for every row.
c. Give two ways to write the expression correctly.

Exercise 4.13. At stage G, the sales manager asks how regularly Riverside Hardware (C-1001) orders. List its orders from April 2026 through March 2027 with the number of days since its previous order in that year, and then compute the average and the largest gap. Why does the first order have no gap, and how does that affect the average?

Exercise 4.14. At stage P3, engineering plans to change the AX-10 axle. Write a recursive query that lists every part that uses the AX-10, directly or through another assembly, with its depth above the axle. Which finished products are affected?

Exercise 4.15. At stage P3, a work order calls for 50 HT-100 hand trucks. Using the explosion of Table 4.29, list the purchased parts and raw materials, those with no components of their own, and the total quantity of each that the work order needs, with its unit of measure.

Exercise 4.16. At stage P3, list everything that goes into one GC-300 steel garden cart as an indented tree, in the manner of Table 4.30, with each part's quantity per cart rounded to three places and without the path column. Indent each part number as that table does, with `REPEAT('. ', depth)`, a dot and a space for each level below the cart.

a. Write and run the query.
b. Which parts appear in the explosions of both the HT-100 and the GC-300, and where in the GC-300's tree does each appear?
c. Why must the path be sorted, rather than the part number within each depth, to keep each part under its assembly?

Exercise 4.17. At stage G, using the individuals chart limits of Table 4.33 for the axle length (characteristic 2), the average measurement plus and minus 2.66 times the average moving range, about 11.962 and 12.037 inches:

a. Write a query that lists the axle length measurements outside the control limits, with the date of each.
b. Compute the individuals chart limits for the axle *diameter* (characteristic 1) in the same way. Why are limits computed from all the diameter measurements a poor basis for charting the diameter? Use the tool wear of Example 4.2.

Exercise 4.18. At stage G, compute the fraction of shipments on time for each carrier over the year of history, in the way of Table 4.31: the average of a `CASE` that gives 1 for a shipment on or before its order's requested ship date, 0 for one after it, and NULL when the order requested no date. Which carrier is most reliable? What does the denominator of each fraction count, and why does a customer pickup have a requested ship date at all?

The remaining exercises use the Millbrook Transit database (Section C.10), the week of bus service that chapter 3's last exercises introduced. A route traversal's route is found through its assignment; `EXTRACT(ISODOW FROM ...)` gives a timestamp's day of the week, 1 for Monday to 7 for Sunday.

Exercise 4.19. For each route, give the total boardings on Monday, on Saturday, and over the whole week, one row per route with the columns `route_number`, `monday`, `saturday`, and `week`. Use `FILTER` (Section 4.4). Which route carries the most passengers?

Exercise 4.20. For each driver, give the number of shifts assigned during the week and the hours they cover, rounded to two decimal places, with the columns `name`, `shifts`, and `hours`, the most hours first and ties by name. Subtracting two timestamps gives an interval; `EXTRACT(EPOCH FROM ...)` turns an interval into seconds. Check one driver's hours by hand from the shift times.

Exercise 4.21. For each vehicle, give the number of route traversals it made during the week and the miles it covered, taking each traversal as its route's full distance. Every vehicle must appear, with 0 miles if it made no traversal. Which join keeps the idle vehicle in the result, and which function turns its NULL into 0?

Exercise 4.22. For each rider category, give the number of people with a registered smart card, the number of trips their cards made, and the trips per rider, rounded to two decimal places. Why must the count of riders use `COUNT(DISTINCT ...)`?

Exercise 4.23. The agency bills each credit account for its cards' fares. List the accounts whose charges for the week exceed $10.00: the account number, the holder's name, the number of trips, and the charges, the largest first. Why does the condition belong in `HAVING` and not in `WHERE` (Section 4.3)?

Exercise 4.24. For each route, which point has the most boardings over the week? Give the route number, the point code, and the boardings, using a window function to rank the points within each route (Section 4.5). What would your query return if two points tied for the most?

Exercise 4.25. Route 1 is scheduled to leave the Transit Center every 30 minutes. Using `LAG`, compute for Monday, 2 March 2026, the time between each run's start and the previous run's start, and list the five longest gaps with the start of the run that ended each. What does a gap longer than 30 minutes mean for a rider waiting at the Transit Center?

Exercise 4.26. The passengers on board after each point are the running sum of boardings less alightings along a run. For each route, give the largest number on board at any point of any run, and the average over every point of every run, rounded to two decimal places. Compare the largest with the seating capacity of the agency's smallest bus.

Exercise 4.27. Using the Northwind database of Section C.11, loaded with the book's loader (Section A.6), compute for each shipper the orders shipped, the orders shipped on or before their required date, and the fraction on time, rounded to three places, in the way of Table 4.31. Leave out the orders not yet shipped, and say why they must be left out of the denominator.

Exercise 4.28. The `employee` table of the Chinook database of Section C.11, loaded with the book's loader (Section A.6), records each employee's manager in `reports_to`. Write a recursive query (Section 4.6) that lists every employee with their depth in the organization, 1 for the employee who reports to no one, with the columns `depth`, `name` (first and last name), and `title`, ordered by depth and name.

Exercise 4.29. Using the `us_exports` table of the Practical SQL database of Section C.11, list the twelve months of 2019 with the value of citrus exports and its twelve-month moving average, the average of that month and the eleven before it, rounded to the dollar.

a. Write the query with a window function (Section 4.5).
b. Why must the window be computed before the rows are restricted to 2019? What would a `WHERE year = 2019` in the same query as the window do to January's average?

Exercise 4.30. The `titles` table of the Employees database of Section C.11, loaded with the book's loader (Section A.6), records each title an employee has held, from `from_date` to `to_date`. A title still held has the `to_date` `9999-01-01`. Count the employees holding each title now, the most first. Why might a designer record a current title with that date rather than with NULL, and what does the choice cost a query writer?

Part II.

Designing and Building a Database

5. Class Modeling with UML

Learning objectives

After reading this chapter you should be able to:

- explain what a class model says about a system's data that a relational schema does not
- write a problem statement, use cases, and requirements that bound a system and say what it must do
- identify the classes, attributes, and associations in a narrative description of an industrial system, eliminating and testing the candidates
- specify an attribute's domain by enumeration, citation, acceptance rule, or range
- draw a cloud instance diagram of a few objects and links, and read multiplicity from it
- specify the multiplicity of each end of an association
- model association classes, qualified associations, aggregation, and composition
- decide when an association of three classes cannot be replaced by associations of two
- model generalization and state when it is appropriate
- review a class model against the requirements it is meant to serve
- read a model drawn in crow's foot notation by translating it to a class model

Every query of Chapter 3 and Chapter 4 read tables that someone had already designed. The tables had keys, the keys connected them, and the connections answered the questions. None of that was an accident. Behind every well-built database is a decision about what the business needs to remember, made before any table was defined: which things matter, what is recorded about each, and how they are related.

This chapter presents the language in which those decisions are written, the class diagram of the Unified Modeling Language (UML). A class diagram describes the things of a business and their relationships without saying how they will be stored, so it can be discussed with the people who know the business, most of whom have never seen a table. Chapter 6 then turns a class diagram into tables, and the tables of the book's database are what that turning produced.

Throughout the chapter, we also draw the *instances* that a class diagram describes: the particular customers, orders, and parts. A class diagram makes a general statement, such as "each order is placed by one customer", and a picture of a few particular orders and customers is often the quickest way to see whether the statement is true, and what it forbids.

Following along

This chapter builds the class models behind the stages of the book's database, one stage at a time, as Appendix C describes them. The few queries it shows run in your `dbconcepts` database.

5.1. What a class model adds

This section presents the difference between a class model and the relational schema of Section 2.4, and why a design begins with the first.

Recall from Listing 2.1 the relational schema of stage D2. It says that `customer_order` has a foreign key `customer_fk` that refers to `customer_pk` in `customer` and must not be NULL. From that line, a reader can work out that each order belongs to exactly one customer, and, with some thought, that a customer may have any number of orders. The schema says it in the language of storage: columns, keys, and NULL.

Figure 5.1 says the same thing in the language of the business.

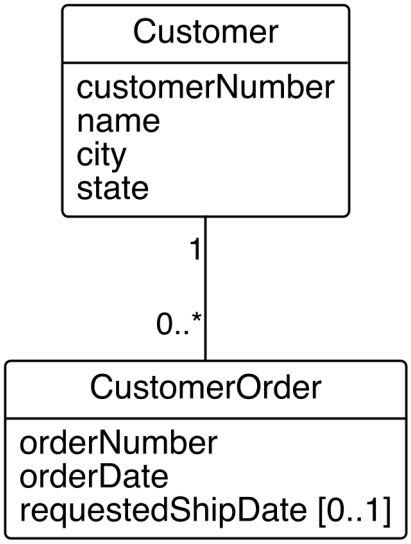


Figure 5.1.: A Class Model of Stage D2

Each box of Figure 5.1 is a class, a kind of thing the business records: customers and customer orders. The line between them is an association, a relationship between the two kinds of thing, and the numbers at its ends say how many of each may be related: each order belongs to exactly `1` customer, and each customer has `0..*` orders, zero or more. Notice what the diagram does not show. There is no `customer_pk`, no `customer_fk`, and no mention

of NULL. The keys are how tables implement the relationship, and a class model leaves that choice to Chapter 6.

A class model adds three things that a relational schema does not state. First, it states every relationship in both directions, with a number at each end; the schema states one direction and leaves the other to be inferred. Second, it names what the relationships mean, in the words the business uses. Third, it can express relationships that tables can only implement indirectly, such as the association classes and qualified associations of Section 5.5. Thus, a class model is the design, and the tables are one implementation of it. That is the view of Blaha and Premerlani (Blaha and Premerlani 1998), whose method for turning class models into tables this book follows.

5.2. Where a class model starts

This section presents the documents from which a class model is built: the problem statement, the use cases, and the requirements.

A class model records what a business needs to remember, and the business has to say what that is before anyone can draw it. Three short documents, written with the people who will use the database, say it. Each is a work product of **analysis**, the stage of a project that establishes what a system must do before anyone decides how it will do it.

The first is the **problem statement**, a short narrative of why the system is needed, the setting it works in, and what it must do. For a database of the size of the book's, half a page is enough. A problem statement is easiest to write by answering six questions: *who* the system is for, *what* problem it solves, *where* it will be used, *when* it is needed, *why* it is needed, and *how*, in general terms, it will work. It helps to ask a seventh: what problems the system will *not* solve. For HG Industries' order and shipment records, the distribution department might write:

> *HG Industries sells hand trucks and garden carts to hardware stores, farm suppliers, and garden centers, which order by phone and by e-mail. The sales office records each customer order, with a line for each product ordered, its quantity, and the price agreed. The company ships from two warehouses, in Fayetteville and Tulsa, each of which keeps a stock of every product. An order may leave in several shipments, from either warehouse, each by a carrier, and a shipment may carry only part of an order line. The sales office needs to know which orders are still open and what remains to be shipped on each; the warehouses need to know when a product's stock falls to its reorder point. The system does not handle invoicing or payment, which the accounting department keeps separately.*

The last sentence is as important as the others. It draws the boundary of the system, and a boundary left undrawn grows during design.

The second document lists the system's **use cases**. A use case is a function of the system as one of its users sees it: a sequence of steps that a user starts in order to get something done (Jacobson et al. 1992). The users are named by role, not by person; a role that uses the system is called an **actor**. A **use case diagram** draws the actors as figures, the use cases as ovals inside a box that marks the system's boundary, and a line from each actor to the use cases it starts. Figure 5.2 draws the use cases of the order and shipment records.

The sales clerk of Figure 5.2 enters orders; the shipping clerk ships them and checks stock for reordering; the sales manager reports the open orders. Each oval stands for a short written description. For example:

Table 5.1.: The Use Case Ship an Order

Use case	Ship an order
Actor	Shipping clerk

Use case	Ship an order
Description	The clerk selects an open order and a warehouse, and records a shipment: its date, its carrier, and the quantity of each order line it carries. The quantities reduce the warehouse's stock of each product.
Notes	A shipment may carry part of a line; the rest stays open for a later shipment.

Notice that Table 5.1 names things the database must remember: an order, its lines, a warehouse, a shipment, a carrier, a quantity, and stock. The use cases are where most of a class model's nouns first appear.

The third document turns the use cases into **requirements**, statements precise enough to test, written in one form: *the system shall* do something, under stated conditions. For example: *The system shall record, for each shipment, the warehouse it leaves from, its date, its carrier, and the quantity of each order line it carries.* And: *The system shall list, for any open order, the quantity of each line not yet shipped.* Requirements of the second kind are the requests that Section 5.9 traces through a model. Requirements about qualities rather than functions, such as how fast a report must appear or who may change a price, are called **nonfunctional requirements**; Section 9.4 takes up one kind that a database enforces, its privileges.

Thus, the problem statement bounds the system, the use cases name its functions and most of its nouns, and the requirements say what it must answer. Section 5.8 returns to these documents to build a class model from them, after the next sections present the parts of a class model.

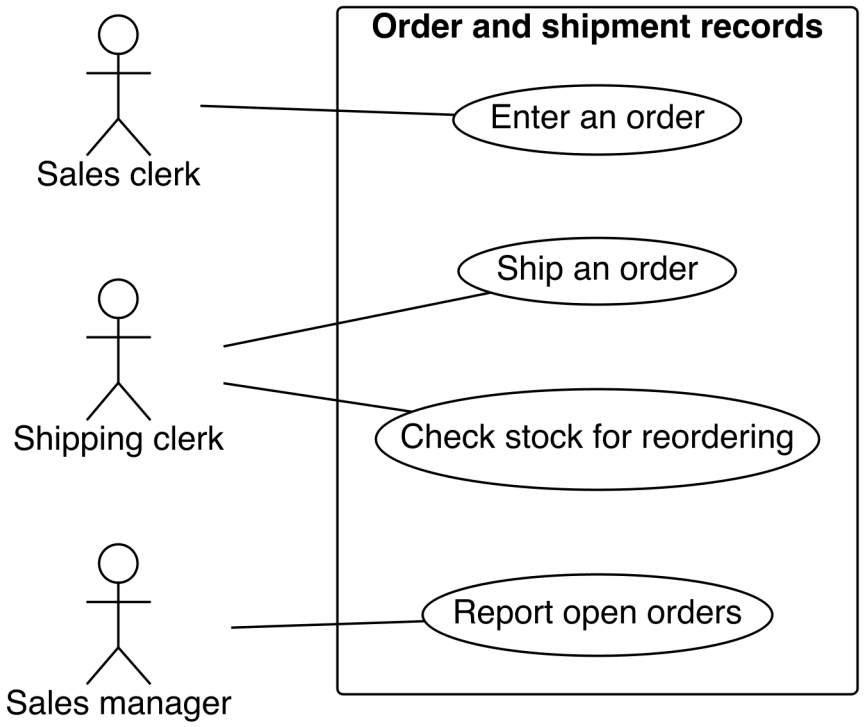


Figure 5.2.: The Use Cases of HG Industries' Order and Shipment Records

5.3. Classes and attributes

This section presents classes, the objects they describe, and the attributes and domains that describe the objects.

An **object** is a particular thing the business needs to remember: the customer Riverside Hardware, the order SO-5001, the axle AX-10. An object has an identity of its own, so two objects remain two even when every recorded fact about them is the same. A **class** describes a group of objects with the same kinds of facts, the same relationships, and the same meaning. Riverside Hardware and Summit Farm Supply are objects of the class `Customer`.

A class can be understood in two ways. Its definition, the test that decides whether a thing belongs to it, is called its **intension**. The set of objects that pass the test, at any moment, is called its **extension**. The extension suggests a picture. Figure 5.3 draws the class `Customer` of stage D1 as a cloud, the set of its objects, with each object a dot labeled by its customer number.

A picture of a class's objects, and of the links between them, is called a **cloud instance diagram**. The objects of Figure 5.3 are the eight rows of Table 2.1, now seen as members of a set rather than as rows of a table. UML has its own picture of instances, the object diagram, which draws each object as a box. The cloud instance diagram says the same things, and its clouds make one idea hard to miss: a class is a set, and every statement a class diagram makes is a statement about the members of sets.

An **attribute** is a named property that every object of a class has a value for, such as a customer's name or an order's date. Each attribute draws its values from a domain, the set of values allowed for it, just as a column does (Section 2.1). In Figure 5.1, `Customer` has the attributes `customerNumber`, `name`, `city`, and `state`. Class and attribute names follow a UML convention: a class name is a singular noun with each word capitalized (`CustomerOrder`), and an attribute name begins in lower case

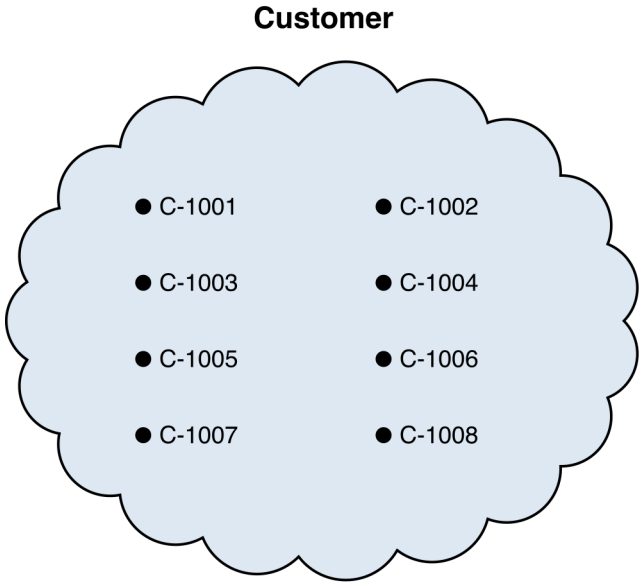


Figure 5.3.: The Customer Class of Stage D1 as a Cloud of Objects

(`requestedShipDate`). Class names are singular because a class describes one object: `Customer`, not `Customers`.

An attribute that an object may lack is marked with a multiplicity in brackets. `requestedShipDate [0..1]` in Figure 5.1 says that an order has zero or one requested ship date, the fact that the column's NULLs record in Table 2.5. Attributes without a mark have exactly one value.

Two kinds of attribute do not belong in a class model. The first is an identifier invented for the database, such as `customer_pk`: objects are distinguishable in themselves, and an identifier is a way of implementing that, chosen in Section 6.6. A business identifier that people use, such as the customer number printed on every invoice, does belong, since it is a fact the business records. The second is a reference to another object, such as `customer_fk`: a reference is a relationship, and a class model draws relationships as associations, not as attributes.

Deciding what should be a class and what an attribute is the first judgment of modeling. A thing deserves a class when it has facts of its own, or relationships of its own, or when the business needs to know about it even while nothing refers to it. Otherwise it is an attribute of whatever it describes. For example, a shipment's carrier is recorded in stage D5 as a name, an attribute of `Shipment`. If the business began to record each carrier's contract rate and contact, or wanted a list of approved carriers before any shipment used one, the carrier would become a class of its own.

A good way to test a class is to write its description, a few sentences that let any reader decide whether a given thing is one of its objects. For example: *A customer is a business that buys HG Industries' products for resale or its own use, and to which the company has assigned a customer number. A business becomes a customer when it is assigned a number, before it places an order.* The last sentence is a decision about the business, and it has a consequence in the data: Northgate Hardware, with no orders, is still a customer. Descriptions like this one, collected for every class, attribute,

and association, form the model's **data dictionary**, the reference that says what every name in the model means.

5.3.1. Attributes with parts

Many values that a business records have parts. A telephone number has an area code, a location has a latitude and a longitude, and an address has a street, a city, a state, and a ZIP code. Recall from Section 2.1.1 that a value is *atomic* when it is one member of its column's domain, rather than several values joined in one. Thus, whether a value is *atomic* depends on its domain and on how the business uses it, not on how it looks. While modeling, the test is a question: does any query, rule, or sort need part of the value on its own, in a way the value's domain does not supply? The answer leads to one of three choices.

The first choice keeps the value as one attribute, because the business uses it whole. A customer's name is printed on invoices and searched as a whole, and no request asks for its second word. A telephone number is dialed whole, and the sales office never asks which customers share an area code. Each is one attribute, with one domain.

The second choice splits the value into attributes, because its parts are used separately. The `Customer` class of Figure 5.1 records `city` and `state` as two attributes, since the business asks for the customers in a state and sorts them by city. The points of a transit network record a `latitude` and a `longitude` rather than one location. A request for the stops north of a given latitude compares latitudes alone, and a request for the stop nearest a rider computes a distance from both. Each part is an attribute with a domain of its own (e.g. a latitude between -90 and 90 degrees).

The third choice promotes the value to a class of its own, because it repeats or is shared. A customer may have several telephone numbers, one for orders and one for accounts payable. Recall that an attribute has one value for each object, or none when it is marked `[0..1]`. Thus, several telephone

numbers of one customer are objects of a class, linked to the customer by an association. (UML also allows an attribute marked `[0..*]`; this book draws a class instead, since the numbers need a table of their own in any case.) In the same way, a delivery site that several customers share, such as the receiving dock of a shopping center, is one thing, recorded once. When the dock's address or its receiving hours change, one change updates every customer it serves. This is the judgment between a class and an attribute, made earlier for a shipment's carrier, applied again: a thing with facts or relationships of its own deserves a class.

Figure 5.4 draws the third choice for a customer's telephone numbers. HG Industries' database records no telephone numbers, so the model is hypothetical.

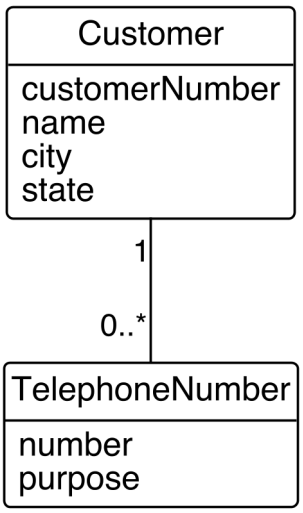


Figure 5.4.: A Customer's Telephone Numbers as a Class (Hypothetical)

The class `TelephoneNumber` of Figure 5.4 has two attributes: `number`, the number as it is dialed, and `purpose`, what the number is for. Read both

ways, the association says that *each customer has zero or more telephone numbers*, and *each telephone number belongs to exactly one customer*; 1 presents these numbers in full. Notice that `TelephoneNumber` has no attribute naming its customer. The line records which customer a number belongs to, as every association does. A fourth number for a customer is one more object, and the model does not change.

Each choice becomes something different when Chapter 6 turns the model into tables. An attribute kept whole becomes one column, and an attribute split into parts becomes one column per part. A class becomes a table of its own. That last table, with one row per telephone number, is where Section 2.1.1 moves a customer's telephone numbers to remove a repeating group. Section 7.4 makes the ban on repeating groups part of first normal form.

The wrong choice shows itself in the requests. Suppose that a customer's address were kept as one attribute, "1200 Main St., Fayetteville, AR 72701", and the shipping department then asked for the customers in each ZIP code. A query would have to pick the ZIP code out of the text, and an address typed without it, or with the state spelled out, would slip past it. Suppose instead that the telephone numbers were attributes `phone1` and `phone2` of `Customer`. They form the repeating group of Section 2.1.1: a third number needs a change to the model, and a search for one number must look in both attributes. Thus, you should ask the test question of every value with parts, and let the requests decide.

5.4. Associations and multiplicity

This section presents links and associations, and the multiplicity that says how many objects each object may be linked to.

A **link** is a connection between two particular objects: Riverside Hardware placed order SO-5001. An **association** describes a group of links with the same meaning between objects of two classes: customers place orders. In

other words, a link is to an association as an object is to a class. A cloud instance diagram draws each link as a line from a dot in one cloud to a dot in another, and Figure 5.5 draws the links of the association between four customers and their orders at stage D2.

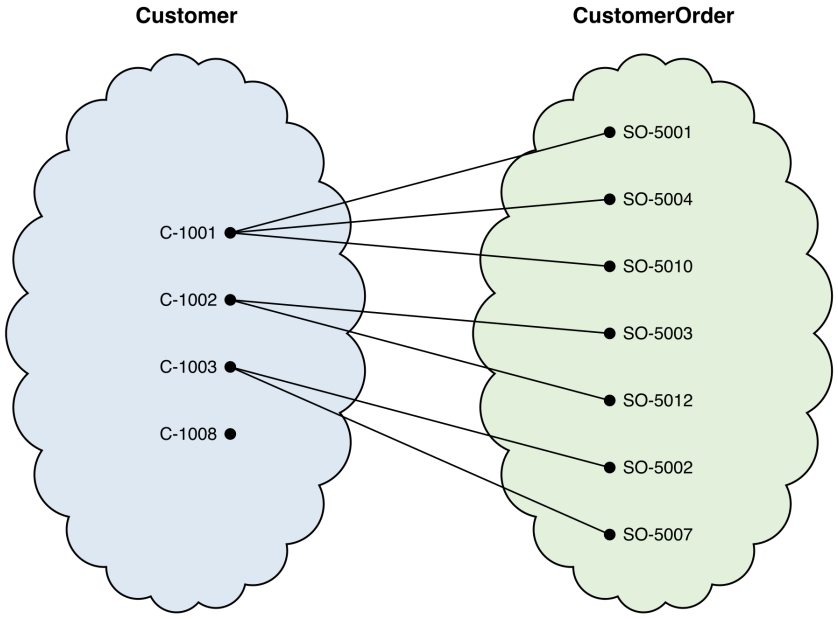


Figure 5.5.: Four Customers, Their Orders, and the Links Between Them

Each line of Figure 5.5 is one link. Riverside Hardware (C-1001) has three lines, one to each of its orders; Summit Farm Supply (C-1002) and Cedar Creek Home Center (C-1003) have two each; and Northgate Hardware (C-1008) has none. On the other side, every order has exactly one line.

The **multiplicity** at an end of an association states how many objects of the class at that end may be linked to one object at the other end. In a cloud instance diagram, multiplicity is counted: the number of lines that

may leave each dot. From Figure 5.5, every order dot has exactly one line, so each order is linked to exactly one customer, the `1` at the customer end of Figure 5.1. A customer dot has three lines, two, or none, so a customer may be linked to any number of orders, including zero, the `0..*` at the order end. The multiplicities UML writes most often are:

Table 5.2.: Common Multiplicities and How to Count Them

Multiplicity	Read as	Counted in a cloud instance diagram
`1`	exactly one	every dot has exactly one line
`0..1`	at most one	a dot has one line or none
`0..*` or `*`	any number	a dot has any number of lines, including none
`1..*`	at least one	every dot has at least one line

Notice where each number is written: at the far end from the object being counted. The `0..*` at the order end of Figure 5.1 counts the orders of one customer. The rule is easy to reverse by mistake, and reading an association as two sentences guards against it: *each customer places zero or more orders*, and *each order is placed by exactly one customer*. You should read every association you draw both ways before you trust it.

A diagram of a few objects can show that a multiplicity is violated, but it can never prove that one holds. The four customers of Figure 5.5 all fit `0..*`, and so would a thousand. Whether a customer may have no orders is a rule of the business, decided in the class description of Section 5.3, just as a candidate key is a rule of the business and not a property of the rows that happen to exist (Section 2.2). A cloud instance diagram is still the

quickest way to test a multiplicity: draw a few objects, draw the links the business allows, and count.

The minimum of a multiplicity, the number before the two dots, decides whether an object may exist without a link. A minimum of `1`, as at the customer end, means that an order cannot exist without its customer. A minimum of `0`, as at the order end, means that a customer can. When Chapter 6 turns associations into tables, a minimum of 1 becomes a foreign key declared `NOT NULL`, which is where the `must not be NULL` of Listing 2.1 comes from.

An end of an association can be given a **role name**, which says what the objects at that end are to the objects at the other. Role names are optional when the class names make the meaning plain, as in Figure 5.1, and essential when they do not. The most common case is a **recursive association**, in which both ends attach to the same class, or one end to a class and the other to one of its subclasses (Section 5.7), so that an object of the class can be linked to another object of the same class. The bill of materials of Section 4.6 is one, with the roles *assembly* and *component*: its `component` end attaches to `Part` and its `assembly` end to `ManufacturedPart`, a kind of part. Section 5.5 draws its links.

The objects at a many end of an association are, by default, a set: the model says which orders a customer placed, not in what sequence. When the sequence is itself a fact of the business, such as the stops of a bus route in the order the bus reaches them, the end is marked `{ordered}`. The constraint says that the sequence matters, not how it is recorded; mapped to a table, it becomes a column, a sequence number, by which the rows are sorted.

One more rule governs links, and it matters more than it seems: **one association holds at most one link between a given pair of objects.** A customer either has placed a given order or has not; it cannot have placed the same order twice. A second link between the same two objects would need a second association, with a different meaning, such as a customer who *placed* an order and also *pays* for it, or an association of more than two

classes, which Section 5.5.1 presents. The rule is what the next section's worked example turns on.

5.5. Association classes and qualified associations

This section presents associations whose links carry facts of their own, and associations whose links are told apart by a qualifier.

Some facts belong to a link rather than to either object it connects. At stage D4, each warehouse holds a quantity of each product. The quantity is not a fact about the warehouse, which holds different quantities of different products, nor about the product, which is held in different quantities in different warehouses. It is a fact about the pair. Figure 5.6 draws the links between the two warehouses and two of the products, with each link's quantity on hand written on its line.

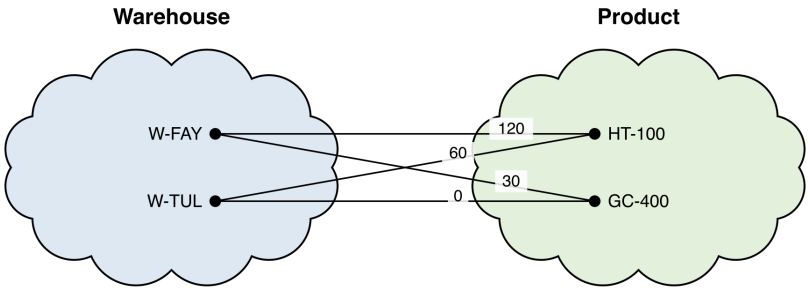


Figure 5.6.: Warehouses, Products, and the Quantity on Each Link

Each line of Figure 5.6 carries one quantity: 120 HT-100 hand trucks at Fayetteville, 0 GC-400 poly carts at Tulsa, and so on. A fact that belongs to a link in this way is modeled with an **association class**, a class whose objects are the links of an association. UML draws it as a class box attached to the association's line by a dashed line, as in Figure 5.7.

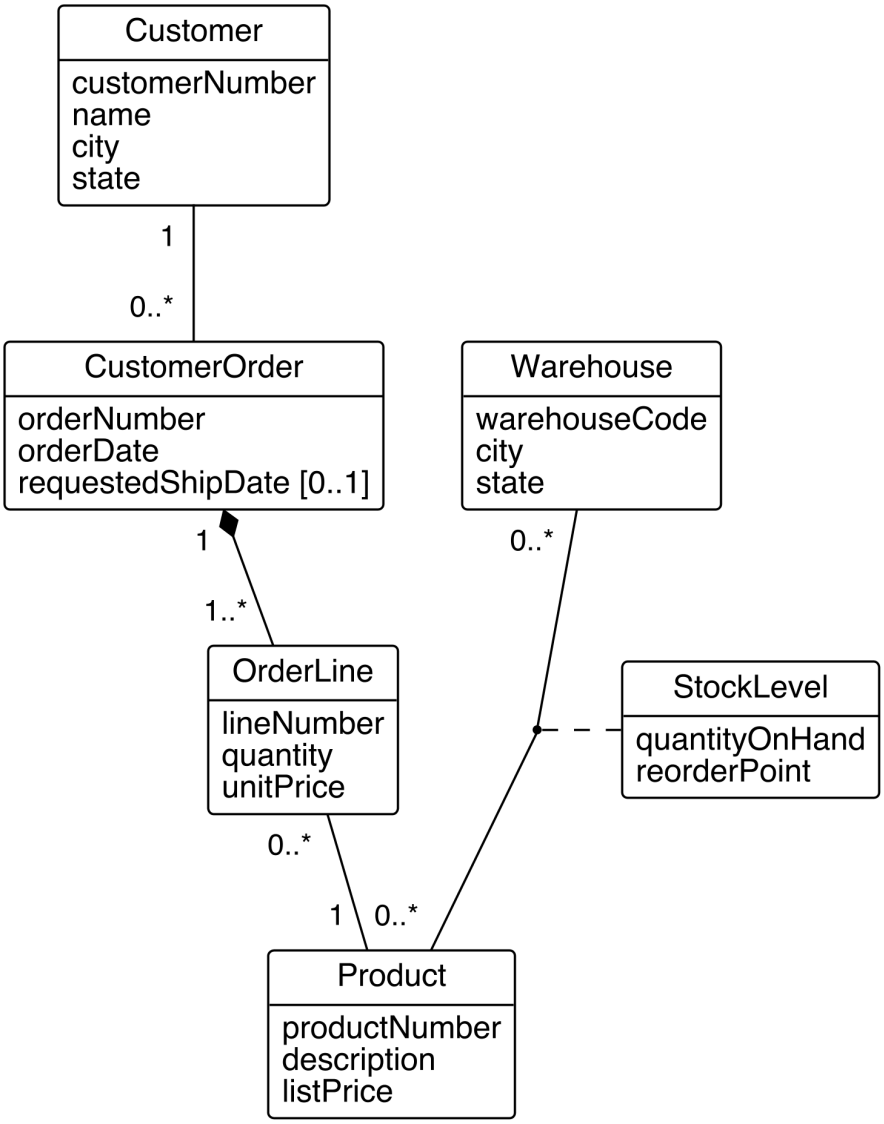


Figure 5.7.: A Class Model of Stage D4, with the Association Class StockLevel

In Figure 5.7, the association between `Warehouse` and `Product` has the multiplicity `0..*` at both ends: a warehouse may stock any number of products, and a product may be stocked in any number of warehouses. `StockLevel`, the box on the dashed line, holds the attributes of each link, `quantityOnHand` and `reorderPoint`. Notice that `StockLevel` has no multiplicity of its own and no line to either class: it *is* the association, seen as a class. An object of `StockLevel` cannot exist without its link, and each link has exactly one `StockLevel`.

An association class is therefore more than a place to put a link's attributes; it states a rule of the business. Compare Figure 5.8, which makes `StockLevel` an ordinary class with an association to each of `Warehouse` and `Product`.

Each stock level of Figure 5.8 still belongs to one warehouse and one product. But the model no longer says that a warehouse and a product are related at all; each is related only to stock levels. And nothing in it prevents two stock levels for the same warehouse and product, each with its own quantity on hand, so that "how many HT-100 hand trucks are in Fayetteville?" would have two answers. The association class of Figure 5.7 rules that out, for the reason the next paragraph gives. Thus, an association class says that the business has at most one of its objects for each pair, and an ordinary class says that it may have several. Section 6.3.4 carries the difference into the tables: the association class's pair of foreign keys is `UNIQUE`.

The rule of 1 applies to association classes with full force. Since an association holds at most one link between a pair of objects, an association class holds at most one object for each pair. That is right for stock levels, since a warehouse has one stock level for each product. It is wrong for the order line, as the following example shows.

Example 5.1 (Deciding what an order line is)**.** *The system.* The distribution schema of stage D3. Customers place orders, and each order lists products, each with a quantity and a unit price.

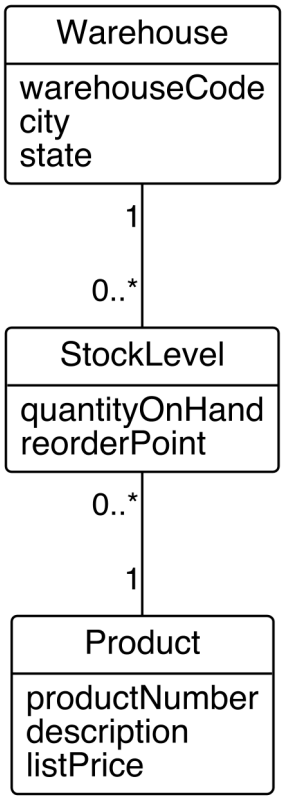


Figure 5.8.: StockLevel as an Ordinary Class

The request. Model the order line: the fact that an order includes a quantity of a product at a price.

The quantity and the unit price are facts about a pair, an order and a product, just as a quantity on hand is. The first model therefore makes the order line an association class, as Figure 5.9 does.

Before accepting Figure 5.9, let's test it against the data. The model says that an order line is a link between an order and a product, so each line of an order must link that order to a different product. The lines of the first four orders of stage D3 are:

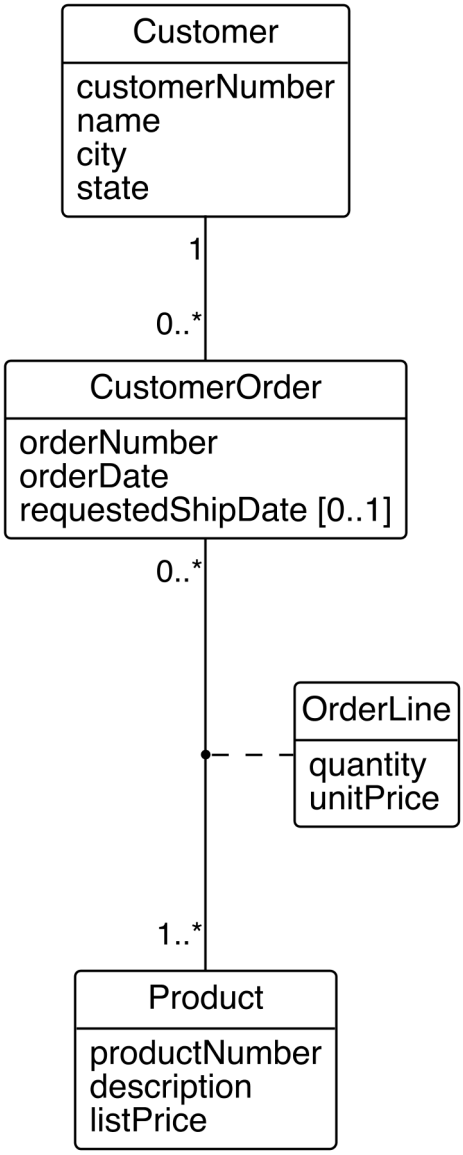


Figure 5.9.: A First Model of Stage D3: the Order Line as an Association Class

Table 5.3.: The Lines of the First Four Orders

```
SELECT o.order_number, l.line_number,
       p.product_number, l.quantity, l.unit_price
FROM distribution.order_line AS l
JOIN distribution.customer_order AS o
  ON o.customer_order_pk = l.customer_order_fk
JOIN distribution.product AS p
  ON p.product_pk = l.product_fk
WHERE o.order_number <= 'SO-5004'
ORDER BY o.order_number, l.line_number;
```

order_number	line_number	product_number	quantity	unit_price
SO-5001	1	HT-100	10	89.00
SO-5001	2	GC-300	4	119.00
SO-5002	1	HT-200	6	139.00
SO-5003	1	HT-100	12	85.00
SO-5003	2	HT-200	4	135.00
SO-5003	3	GC-300	6	115.00
SO-5004	1	HT-100	20	89.00
SO-5004	2	GC-300	5	119.00
SO-5004	3	HT-100	10	79.00

Figure 5.10 draws the lines of Table 5.3 as links, with each line's quantity on its link.

Most of the links of Figure 5.10 are fine. Order SO-5004, however, lists the HT-100 twice: 20 units on line 1 at $89.00, and 10 more on line 3 at $79.00, a discounted second lot. As links, the two lines are two connections between the same order and the same product, drawn dashed in the figure. One association cannot hold them, so an association class cannot hold the two order lines. The model of Figure 5.9 is wrong.

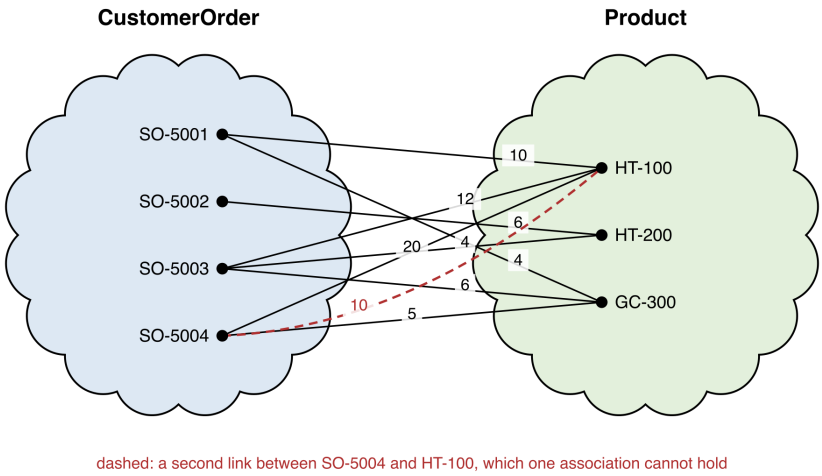


Figure 5.10.: The Order Lines of Table 5.3 as Links, One Too Many

The repair makes the order line a class of its own, whose objects each belong to one order and refer to one product, as Figure 5.11 shows.

In Figure 5.11, an order has `1..*` order lines, and each order line refers to exactly `1` product; a product may appear on `0..*` lines. Two lines of SO-5004 are now two objects of `OrderLine`, each linked to the HT-100, and no pair of objects is linked twice in any association. The filled diamond at the order end says that an order line is part of its order and cannot outlive it, a relationship Section 5.6 presents.

The check. The refined model must hold every row of Table 5.3 as objects and links without breaking the rule of one link per pair. Each of the nine rows becomes one `OrderLine` object, linked to its order once and to its product once, so every pair of linked objects is linked once. The first model fails on exactly one pair, SO-5004 and the HT-100, and a single counterexample is enough to reject a model.

Interpretation. The business does sell one product twice on an order, at

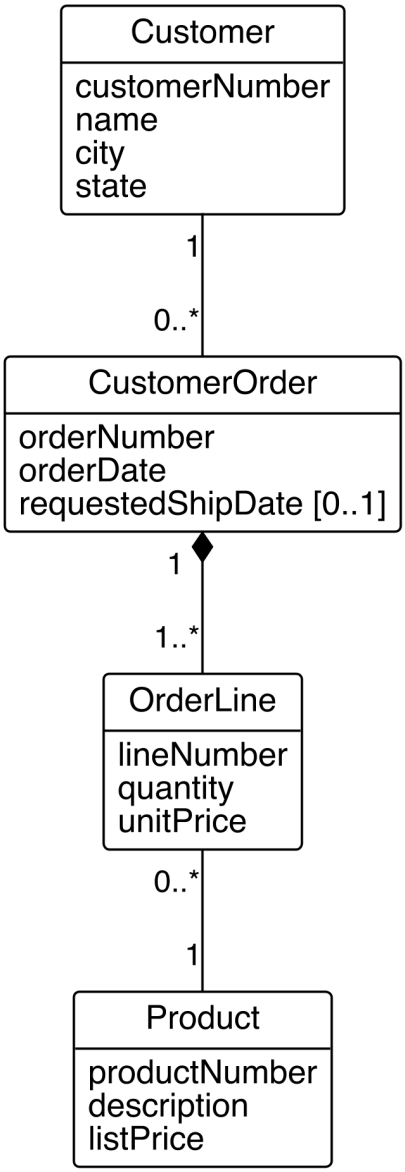


Figure 5.11.: A Refined Model of Stage D3: the Order Line as a Class of Its Own

two prices, and the model must allow it. A model that forbade it would force the order clerk to combine the two lots into one line at an averaged price, losing the record of the discount.

Where it breaks. The refined model allows any number of lines for the same product on an order. If the business wanted at most one line per product, the association class of Figure 5.9 would have been the right model after all; the data decides between the models only because it holds a counterexample. Without SO-5004, the question would have to be asked of the business.

An order line has one more property the refined model does not yet capture. Its line number identifies it only within its order: every order has a line 1. A **qualifier** is an attribute of an association that selects among the objects linked to one object, and a **qualified association** is an association with a qualifier. UML draws the qualifier as a small box on the association's end, against the class it qualifies, as in Figure 5.12.

Read Figure 5.12 as follows: given an order *and* a line number, there is at most one order line. That is the `0..1` at the `OrderLine` end, where Figure 5.11 had `1..*`. The qualifier reduces the multiplicity by splitting an order's lines into groups of one, one group for each line number. That is what a composite key does in a table, and Section 6.4 maps the qualifier to the composite key (`customer_order_fk`, `line_number`) of Listing 2.3. A qualifier is often a name or a number that is unique only within something: a line number within an order, a machine number within a work center, an account number within a bank.

Association classes also serve a recursive association. The bill of materials of stage P2 records, for each assembly, how many of each component it uses. The quantity belongs to the pair, so `BomLine` is an association class of the recursive association between an assembly and its components. Figure 5.13 draws the links of two assemblies, the HT-100 and its wheel assembly WA-10, with a cloud for each role: on the left the manufactured parts in the role of assembly, and on the right the parts in the role of component.

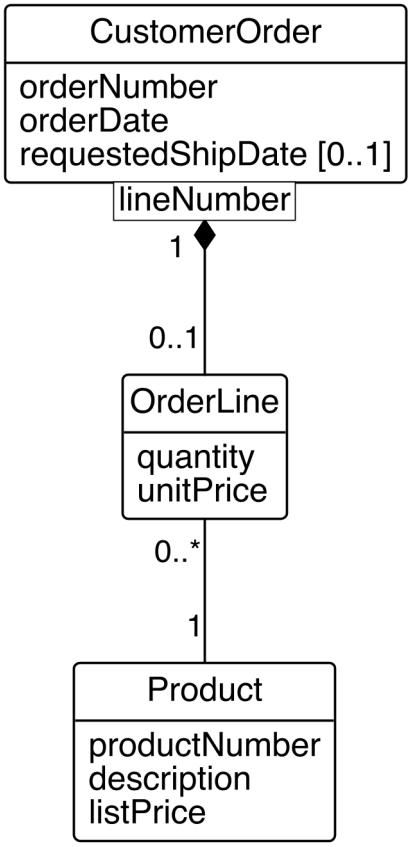


Figure 5.12.: The Order Line Qualified by Its Line Number

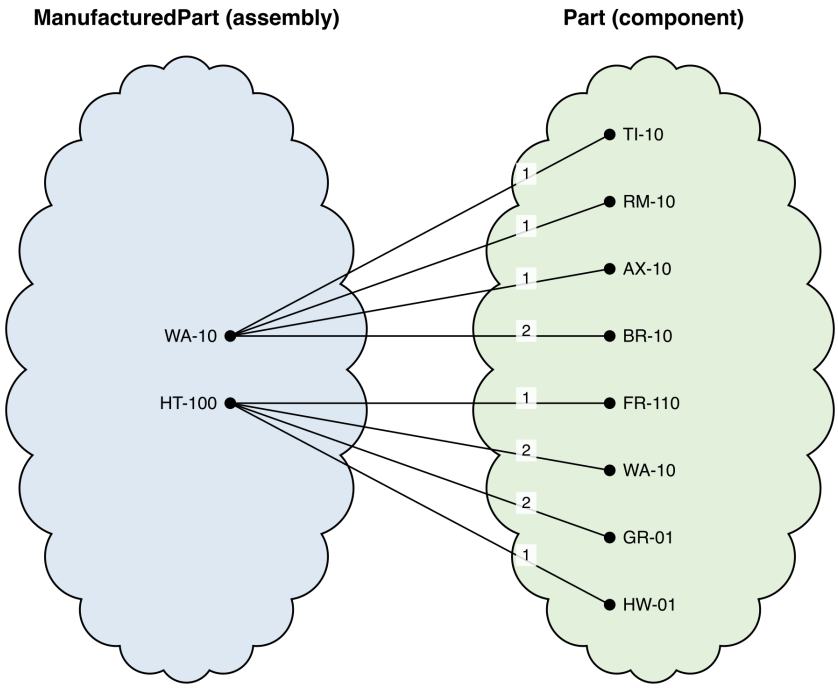


Figure 5.13.: Two Assemblies and Their Components, with the Parts Drawn Once per Role

The WA-10 appears in both clouds of Figure 5.13: on the left as an assembly of four components, on the right as a component of the HT-100. It is one object playing two roles. Drawing a cloud for each role keeps the lines readable, and the role names say which end of each link is which. Each pair of assembly and component is linked once, so an association class suits the bill of materials, as it did the stock levels and did not the order lines.

5.5.1. Ternary associations

Most associations relate two classes. An association of three classes, a **ternary association**, has links that each connect three objects at once. The book's database has none, but a plant that tracked labor would need one: which operator ran which machine on which work order, and for how many hours. Figure 5.14 draws such a record for three operators, three machines, and two work orders of stage P3. The objects are real, and each operator runs only machines on which the operator is certified; the runs and their hours are hypothetical.

Each ring of Figure 5.14 is one link, joined to one operator, one machine, and one work order, and the hours belong to the link. Maria Lopez (E-201) ran the band saw M-01 on both work orders and the welder M-04 on WO-7001; James Carter (E-202) ran the lathe M-02 on both work orders. The question to ask of any ternary association is whether three binary associations would do: operator with machine, machine with work order, and operator with work order. Here they would not, for two reasons.

The first is the hours, which belong to the combination of all three, not to any pair. The second is that the pairs lose which combinations happened. In Figure 5.14, Maria Lopez ran M-04 (on WO-7001), M-04 was run on WO-7003 (by Linda Nguyen), and Maria Lopez worked on WO-7003 (on M-01). The three pairs, Maria Lopez with M-04, M-04 with WO-7003, and Maria Lopez with WO-7003, are all true, so joined together they imply that Maria Lopez ran M-04 on WO-7003. That run never happened. A

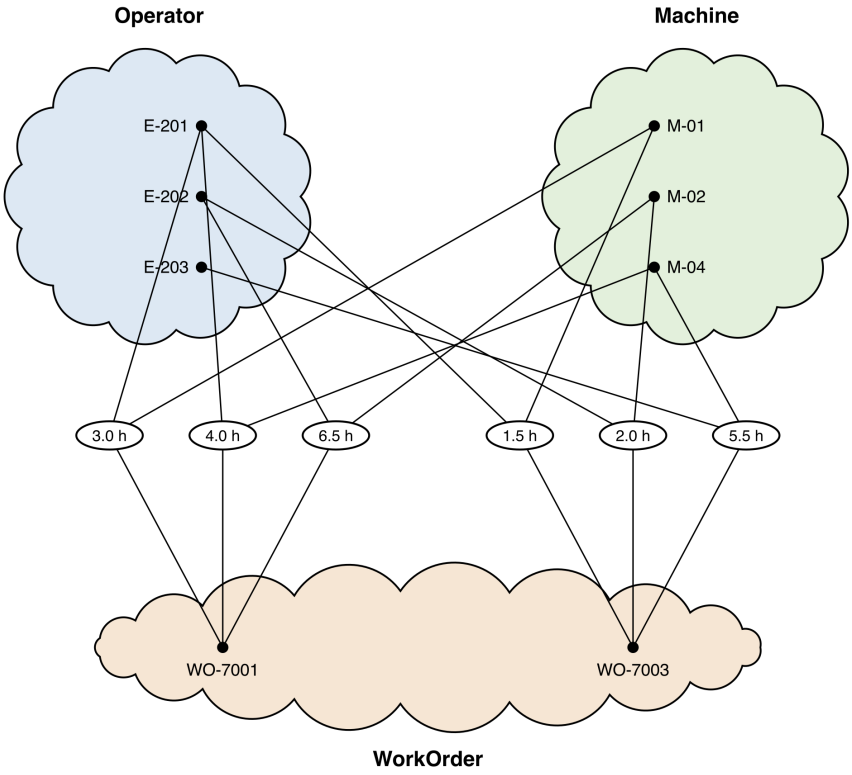


Figure 5.14.: A Hypothetical Ternary Association: Operators Run Machines on Work Orders

ternary association is rare in practice, and a model that seems to need one deserves that question every time. Exercise 5.8 explores it.

Nor would an association class do. It is tempting to keep a binary association between `Operator` and `Machine` and record the work order and the hours in an association class, `Run`, on its links. But James Carter ran the lathe M-02 on both work orders, which is two runs for one pair of operator and machine. Recall from 1 that an association holds at most one link between a given pair of objects, so its association class holds at most one run for the pair, and the model could not record the second. The ternary association's links are triples, and the two runs are two different triples.

UML draws a ternary association as a hollow diamond with a line to each of the three classes, as in Figure 5.15. This diamond marks the association itself and is unrelated to the diamonds of Section 5.6, which sit at the end of a line.

In Figure 5.15, the lines from the diamond go to `Operator`, `Machine`, and `WorkOrder`, with a few of each class's attributes. Each link of the association is one ring of Figure 5.14. The multiplicity at an end of a ternary association says how many objects at that end may be linked to a given pair of objects at the other two ends. The `0..*` at `WorkOrder`, for example, says that an operator may run a machine on any number of work orders, including none. The hours are an attribute of the association class `Run`, attached to the diamond by a dashed line, just as `StockLevel` is attached to its association's line in Figure 5.7. One association holds at most one link among a given operator, machine, and work order, so `Run` holds one object, with its hours, for each triple. Section 6.3.7 maps the association to a table with three foreign keys.

5.6. Aggregation and composition

This section presents aggregation and composition, the associations that say one object is part of another.

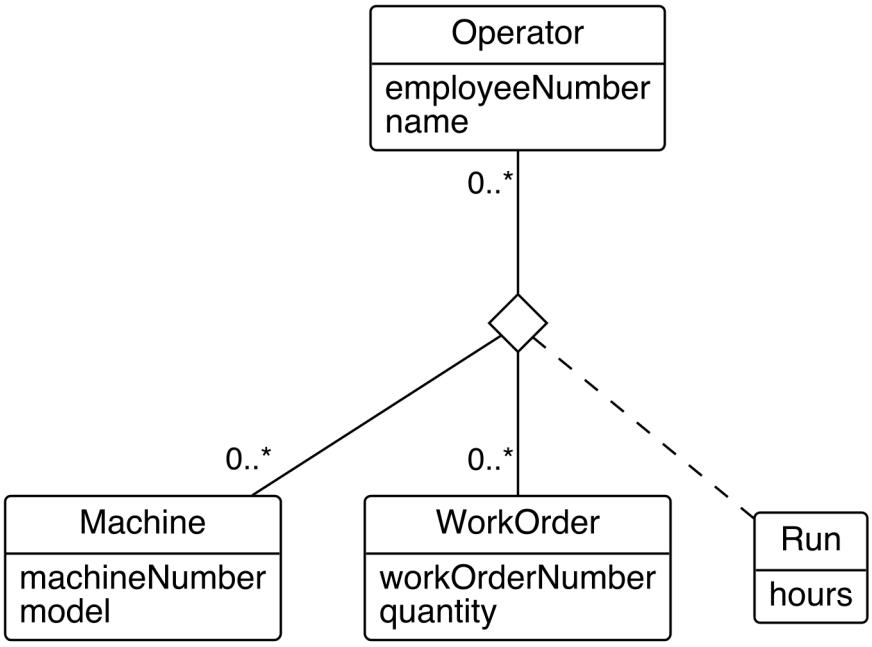


Figure 5.15.: A Ternary Association with an Association Class (Hypothetical)

Some associations have a special meaning: one object is part of another. An order line is part of its order; a machine is part of the work center it stands in. UML marks such an association with a diamond at the end of the whole. The diamond comes in two strengths.

Composition, drawn with a filled diamond, says that each part belongs to exactly one whole and cannot exist without it: when the whole is removed, its parts go with it. (UML allows a part to be moved from one whole to another; this book uses composition only for a part that never moves.) The order line of Figure 5.11 is composed in its order. An order line of SO-5004 is meaningless without SO-5004, it never moves to another order, and canceling the order removes its lines. The multiplicity at the whole's end of a composition is therefore `1`, which is why Figure 5.11 writes it there. The one exception is a recursive composition, a hierarchy in which a whole is itself a part of a larger whole of the same class: the object at the root of the hierarchy is part of nothing, so the whole's end is `0..1`. A part's quality characteristics in stage Q are composed in their part in the same way as order lines.

Aggregation, drawn with a hollow diamond, is the weaker claim. UML leaves its exact meaning to the modeler; this book uses it for a part that is part of the whole but has a life of its own. Figure 5.16 shows the routing of stage P3, where a work center aggregates its machines.

In Figure 5.16, a `Machine` is part of a `WorkCenter`, the hollow diamond, but it has a life of its own: a machine can be moved from one work center to another, so it is not created with its work center and is not destroyed with it. The `1` at the work center's end says that a machine always stands in some work center; if the Welding work center were removed from the plan, its welders would first be moved to another work center, not scrapped. The `Operation`, by contrast, is composed in its `ManufacturedPart`, the filled diamond, and qualified by its operation number, just as an order line is qualified by its line number. An operation "Weld frame" means nothing apart from the part whose routing it belongs to.

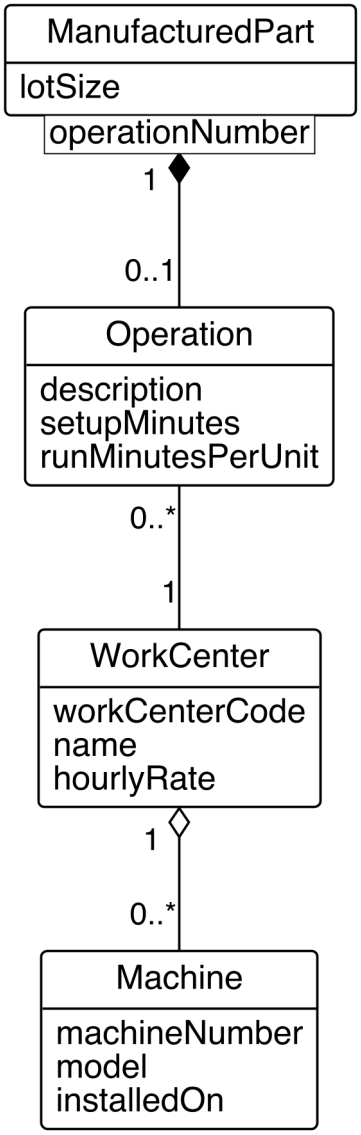


Figure 5.16.: A Class Model of Stage P3's Routing: Composition and Aggregation

Three questions separate the two, and they decide most cases in industrial data:

1. Is the relationship "is part of"? If the phrase sounds wrong, as it does for a customer and its orders, the association is an ordinary one, whatever its multiplicity.
2. Can the part exist without the whole, or move to another whole? If not, the association is a composition; if so, an aggregation.
3. Does something done to the whole happen to its parts? Copying an order copies its lines; moving a work center's floor space does not scrap its machines.

Notice what the bill of materials is *not*. An HT-100 "is made of" two wheel assemblies, which sounds like composition, yet `BomLine` in Figure 5.13 is an ordinary association with an association class. The bill of materials describes kinds of parts, a catalog: the part number WA-10 is a component of the part number HT-100. It says nothing about a particular hand truck and the particular wheels bolted to it, and the WA-10 is a component of four products at once. A composition among particular objects, this serialized truck and those two wheels, would describe an assembled product, which HG Industries does not track. Thus, before drawing a diamond, you should ask whether the objects are particular things or descriptions of kinds of things. A composition among descriptions is still right when each description belongs to one whole, as an operation belongs to the routing of one part number and a quality characteristic to one part; the bill of materials fails the test because one component is shared by many assemblies.

Chapter 6 implements composition and aggregation with the same foreign keys as ordinary associations; the difference appears in how deletions are handled (Table 2.6). A composition is where cascading a deletion is right.

5.7. Generalization

This section presents generalization, which organizes classes into more general and more specific kinds.

HG Industries makes some parts and buys the others. Every part has a part number, a description, a unit of measure, and a standard cost. A part the company makes also has a lot size, the quantity it makes at once, and a routing of operations; a part it buys has a lead time, the days a supplier needs to deliver, and no routing. **Generalization** is the relationship between a general class and the more specific classes that are kinds of it. The general class is the **superclass**, `Part`, and the specific classes are its **subclasses**, `ManufacturedPart` and `PurchasedPart`. UML draws it as a line with a hollow triangle at the superclass, as in Figure 5.17.

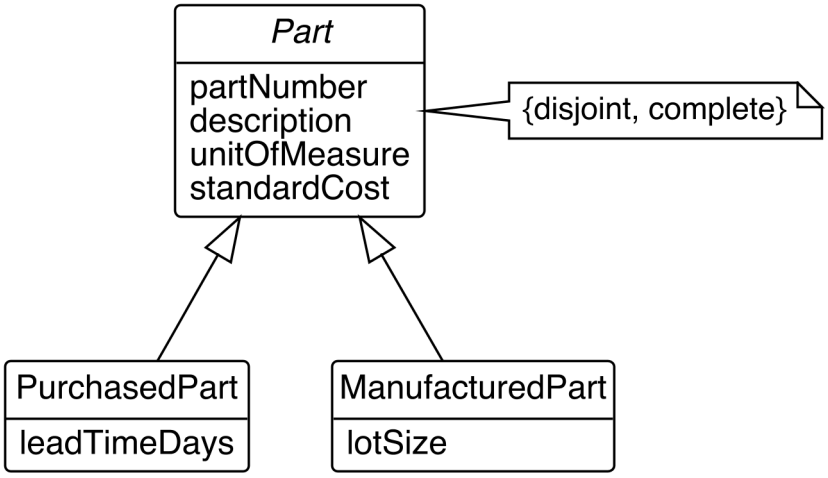


Figure 5.17.: Generalization in Stage P1: Every Part Is Manufactured or Purchased

A subclass **inherits** everything the superclass has: its attributes, its

associations, and its meaning. A manufactured part *is* a part, so it has a part number, a description, a unit of measure, and a standard cost, and it adds a lot size. In terms of clouds, a subclass's objects are a subset of its superclass's objects. Figure 5.18 draws the eighteen parts of stage P1 inside the cloud of `Part`, divided between the two subclass clouds.

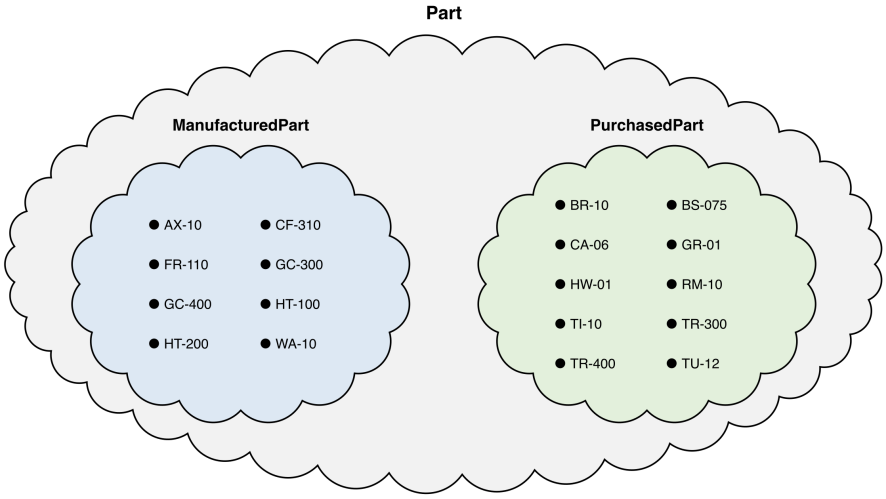


Figure 5.18.: The Parts of Stage P1: Two Subclass Clouds Inside the Part Cloud

Every dot of Figure 5.18 lies in one of the inner clouds, and no dot lies in both. Those two facts are what the note `{disjoint, complete}` in Figure 5.17 says. **Disjoint** means that no object belongs to two subclasses: no part is both made and bought. **Complete** means that every object of the superclass belongs to some subclass: there is no part that is neither. Both are rules of the business, and both could be otherwise. A part bought from a supplier in one season and made in the plant in another would break *disjoint*; a part not yet classified would break *complete*. UML names the opposite of each: `{overlapping}` allows an object to belong to more than one subclass, and `{incomplete}` allows an object of the superclass

to belong to none.

Generalization earns its place when the subclasses differ in attributes or in associations. Here they differ in both. Only a manufactured part has a routing, so the composition of Figure 5.16 attaches to `ManufacturedPart`, not to `Part`. In the bill of materials, only a manufactured part can be an assembly, while any part can be a component, so the `assembly` end attaches to `ManufacturedPart` and the `component` end to `Part` (Figure 5.13). Without the subclasses, the model could not say either rule.

A superclass whose every object belongs to some subclass, as `{complete}` says of `Part`, has no objects of its own. Such a class is called an **abstract class**, and UML writes its name in italics. An abstract class still earns its place: it holds what the subclasses share, and an association drawn to it, such as the `component` end of the bill of materials, accepts an object of any subclass. A class may also have more than one superclass, which is called **multiple inheritance**: a part that is both bought and made, in a model that dropped `{disjoint}`, could belong to a class that is a subclass of both. The book's databases have no case of it, and a model is usually clearer without it, since each generalization should divide its superclass by one question, such as "is it made or bought?".

Generalization is often misused for roles. The quality department's inspections are carried out by operators: Linda Nguyen and Robert Hayes inspect coating thickness (Example 4.3). It is tempting to add a subclass `Inspector` of `Operator`. But inspecting is something an operator *does* in an association with inspections, not a kind of operator. An operator may inspect one day and not the next, and nothing about the operator changes when the operator does. The quality model therefore draws an association between `Inspection` and `Operator`, named `inspectedBy`, in which an operator plays the part of inspector. Thus, you should use a subclass for a kind of thing that stays that kind, with facts of its own, and a role name for a part that an object plays in an association.

5.8. Building a class model

This section presents a method for building a class model from the documents of Section 5.2: finding the classes, testing them, and then finding their attributes and associations.

The method is old and plain, and it works. Its steps come from the object-oriented methods on which UML was built (Rumbaugh et al. 1991; Starr 1996; Larman 2004):

1. **Find candidate classes.** List every noun and noun phrase in the problem statement and the use case descriptions. Add the things that domain knowledge says are missing, and look through the kinds of thing that industrial systems usually record: physical things (parts, machines), descriptions or specifications of things (a product, a part's characteristics), places (warehouses, work centers), transactions and their line items (orders and order lines), organizations, and records of events (inspections, shipments).
2. **Eliminate the candidates that are not classes.** A candidate goes if it is *redundant*, another name for a class already listed; *irrelevant*, outside the problem statement's boundary; *vague*; an *attribute*, a fact about something else; an *operation*, something done rather than something remembered; a *role* that some class plays in an association; *derived*, computable from what is recorded; or an *implementation detail*, such as a file or a screen.
3. **Test the classes that remain.** Each should pass four tests (Starr 1996). The *uniformity* test: every object of the class has the same kinds of facts and relationships. The *more than a name* test: the class has facts of its own beyond a name; otherwise it is an attribute. The *or* test: a description full of "or" ("a customer is a store or a supplier or a farm") may hide two classes with nothing in common. The *more than a list* test: a class defined only by listing its members ("the HT-100 and the HT-200") is not yet understood.

4. **Find the attributes.** Look for nouns that follow a possessive ("the order's date", "the quantity of each line") and for adjectives, which often name the values of an enumeration ("open", "rush"). Ask what every object of the class has, and what the business must know to tell one object from another. An attribute that can be computed from others is a **derived attribute**; leave it out, or mark it as derived, since storing it records a fact twice.
5. **Find the associations.** Look for verbs and prepositions that connect two classes: a customer *places* an order, a shipment leaves *from* a warehouse. The verb should describe structure that lasts, not an action of a moment. Table 5.4 lists the kinds of association that recur (Larman 2004), with examples from the book's database. Each association then gets its multiplicities, and, where its links carry facts, an association class (Section 5.5).
6. **Write the data dictionary**, the descriptions of Section 5.3, for every class, attribute, and association.

Table 5.4.: Kinds of Association That Recur in Industrial Data

Kind of association	Example
A is a line item of a transaction B	an order line of a customer order
A is a transaction related to another transaction B	a shipment of a customer order
A takes part in transaction B	a customer in its orders
A is a physical part of B	a machine of a work center
A is stored in, or located at, B	a product's stock in a warehouse
A describes B	a characteristic of a manufactured part
A is recorded in B	a measurement in an inspection
A is qualified to use B	an operator certified on a machine

The steps are not done once in order. Eliminating a candidate often turns it into an attribute of another class, and finding an association often shows

that its links carry facts, which adds an association class. The following example takes the steps on the problem statement of Section 5.2.

Example 5.2 (Building the shipping model of stage D5)**.** *The system.* HG Industries' order and shipment records, described by the problem statement of Section 5.2 and the use cases of Figure 5.2.

The request. The distribution department asks for a class model of the records it described: what must be stored to enter orders, ship them from either warehouse, report what remains open, and know when stock is low.

Let's begin with step 1. The nouns of the problem statement and the use case *Ship an order* (Table 5.1) give the candidates. Step 2 then decides each one, and Table 5.5 records the decisions and their reasons.

Table 5.5.: Candidate Classes for the Order and Shipment Records

Candidate	Decision	Reason
HG Industries	eliminate	irrelevant: the owner of the records, not something they record
hand truck, garden cart	eliminate	a list of instances of *product*
hardware store, farm supplier, garden center	eliminate	kinds of *customer* with the same facts
customer	**class** `Customer`	
customer order	**class** `CustomerOrder`	
line (of an order)	**class** `OrderLine`	a line item of a transaction
product	**class** `Product`	
quantity, price	attribute	of an order line
warehouse	**class** `Warehouse`	
Fayetteville, Tulsa	attribute value	the city of a warehouse

Candidate	Decision	Reason
stock	**class** `StockLevel`	a fact about a warehouse and a product together
reorder point	attribute	of a stock level
shipment	**class** `Shipment`	
carrier	attribute	of a shipment; no facts of its own here
date (of a shipment)	attribute	of a shipment
quantity (of each line it carries)	**class** `ShipmentLine`	a fact about a shipment and an order line together
open order, quantity remaining	eliminate	derived from the order lines and the shipments
sales clerk, shipping clerk, sales manager	eliminate	actors, who use the records but are not recorded
sales office	eliminate	irrelevant: an organization the records do not describe
phone, e-mail	eliminate	irrelevant: how an order arrives, not a fact about it
invoicing, payment	eliminate	outside the boundary the problem statement drew

Step 3 tests the eight classes that remain. Each passes the *uniformity* test: every shipment, for example, has a date, a carrier, and a warehouse. The carrier fails the *more than a name* test, since the problem statement records nothing about a carrier but its name, so it stays an attribute, as Section 5.3

argued it should. A description of a customer as "a hardware store or a farm supplier or a garden center" raises the *or* test, but the statement records the same facts for all three, so one class `Customer` suffices.

Steps 4 and 5 add the attributes and associations. The verbs of the problem statement give the associations: a customer order is *placed by* a customer; it *has* lines, each *for* a product; a warehouse *keeps* a stock *of* each product; a shipment *is for* one order, *leaves from* one warehouse, and *carries* quantities of order lines. Most match a kind in Table 5.4. A stock level is a fact about a warehouse and a product together, so it is the association class of Figure 5.7. The quantity a shipment carries of an order line is likewise a fact about a shipment and an order line together, and a shipment carries a given order line at most once, with one quantity. Thus, `ShipmentLine` is the association class of an association between `Shipment` and `OrderLine`, not an ordinary class composed in the shipment. Compare the order line of Section 5.5, which had to be an ordinary class because an order may list one product twice. A shipment has at least one order line, and an order line is carried by any number of shipments, none while it waits and several when it ships in parts.

Step 4 also adds what every object needs although no document names it as a noun. Each class gets the business identifier by which the business tells its objects apart: the shipment, warehouse, product, customer, and order numbers and codes printed on the company's documents. And the order's dates, the product's list price, and the city and state of customers and warehouses are already kept in the order records of stages D2 to D4, which the distribution department brings to the model. Figure 5.19 draws the result.

Step 6 writes the data dictionary. An association's entry gives its name, its classes, its multiplicities, and the reason for each:

> *Shipment leaves from Warehouse.* Each shipment leaves from exactly one warehouse, since it is loaded at one dock; a warehouse has any number of shipments, including none, as a newly

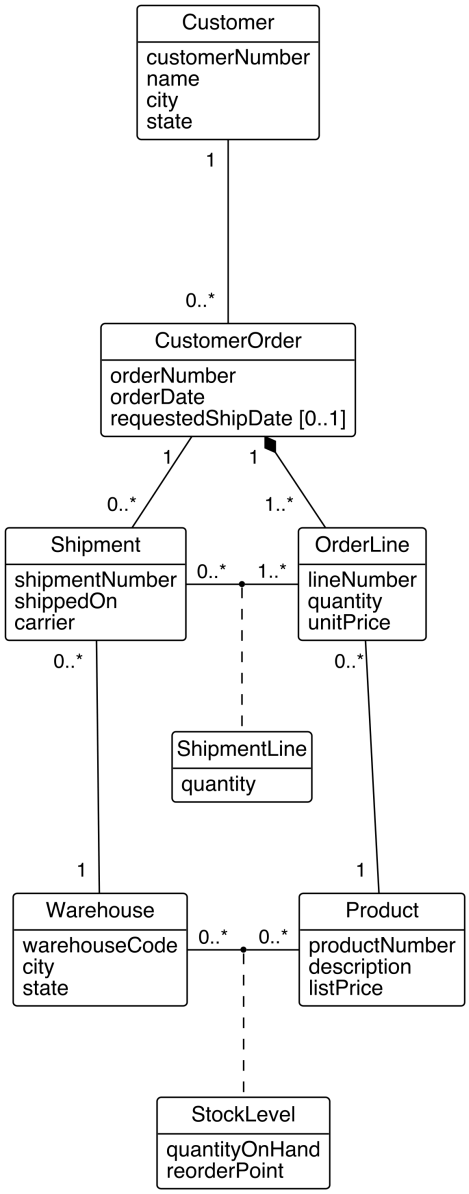


Figure 5.19.: The Class Model of Stage D5, Built from the Problem Statement

opened warehouse would.

An attribute's entry gives its meaning and its **domain specification**, the rule that says which values are allowed. A domain is specified in one of four ways: by an *enumeration* of its values, by a *citation* of an authority that defines them, by an *acceptance rule*, or by a *range* with its units. For example:

- *StockLevel.quantityOnHand.* The number of units of the product on the warehouse's shelves, as last counted or updated by a shipment. Domain: a whole number of units, from 0 up (a range).
- *Customer.state.* Domain: a two-letter state abbreviation of the U.S. Postal Service (a citation).
- *Shipment.carrier.* Domain: the carrier's name as printed on the shipping papers, or *Customer pickup* when the customer collects the goods (an acceptance rule).

A domain written as "an integer" would say how the value is stored, not what it may be. The range "a whole number of units, from 0 up" says both what a quantity means and the constraint `CHECK (quantity_on_hand >= 0)` that implements it in stage D4.

The check. Every requirement of Section 5.2 should trace through Figure 5.19, as Section 5.9 will do for the production model. The quantity of each line not yet shipped is an order line's `quantity` less the `quantity` of its shipment lines, a path of one association; the stock to reorder compares a stock level's two attributes.

Interpretation. The model the method produced is the one the book's database has implemented since stage D5. Its eight classes came from about twenty nouns, and each eliminated noun has a recorded reason, which is what lets the department check the model: a manager who expected carriers to be tracked can see that they were judged a name only, and ask for more.

Where it breaks. The method finds only what the documents mention. A fact the problem statement left out, such as a shipment's weight or the dock it left from, will be missing from the model, and no step of the method will notice. The documents, like the requests of Section 5.9, are part of what the business must review.

5.9. Reviewing a class model

This section presents how to review a class model against the business it is meant to serve, before any table is built from it.

A class model is reviewed in two ways. The first is a check of the model against itself, a list of questions every model should survive:

Table 5.6.: Questions for Reviewing a Class Model

Question	What a failure looks like
Does each class have a description that decides membership?	Two classes for one thing; a class that is only a name
Is each attribute a single fact about its own class?	A list in one attribute; a fact about a pair placed in one of the pair
Is any attribute an identifier or a reference?	`customer_pk` or `customer_fk` in a class box
Does each association read correctly in both directions?	A multiplicity written at the wrong end
Is each multiplicity a rule of the business?	A `1` that the business sometimes breaks
Does a cloud instance diagram of real data fit the model?	Two links between one pair of objects
Is each subclass a kind, not a role?	A subclass an object moves into and out of

The second way is a check against the requests the database must answer, the queries of Chapter 3 and Chapter 4 turned into questions about the model. For each request, trace a path of associations from the classes the request names to the facts it needs, as Example 2.2 traced a path of foreign keys. A request with no path means the model is missing something. The following example reviews the production model that way.

Example 5.3 (Reviewing the production model against planning requests)**.** *The system.* The production class model of stage P3, in Figure 5.20.

The request. The production planner will ask two things of the database. First, which machines can perform each operation of a part's routing, and which operators are certified to run them? Second, for a finished work order, which machine and operator performed each of its operations, and for how long?

Let's trace the first request through Figure 5.20. A `ManufacturedPart` has its `Operation`s through the qualified composition; each operation is performed in one `WorkCenter`; the work center aggregates its `Machine`s; and each machine has its certified `Operator`s through the association class `Certification`. The path exists, so the model can answer the request, and the tables built from it do. For operation 20 of the hand truck frame, the query follows the same path as a chain of joins:

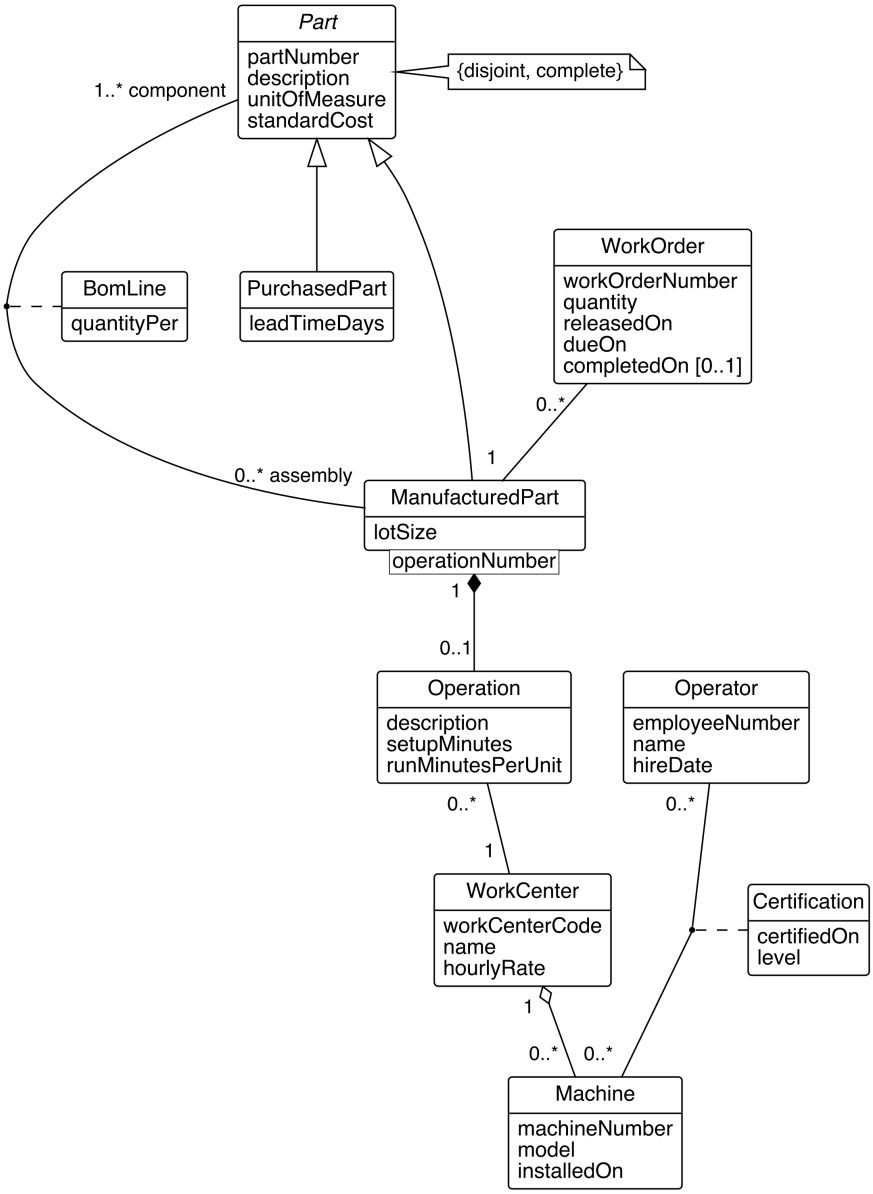


Figure 5.20.: The Production Class Model of Stage P3

Table 5.7.: Machines and Certified Operators for Welding the FR-110

```
SELECT m.machine_number, m.model, o.name, c.level
FROM production.operation AS op
JOIN production.part AS p
  ON p.part_pk = op.manufactured_part_fk
JOIN production.machine AS m
  ON m.work_center_fk = op.work_center_fk
LEFT JOIN production.certification AS c
  ON c.machine_fk = m.machine_pk
LEFT JOIN production.operator AS o
  ON o.operator_pk = c.operator_fk
WHERE p.part_number = 'FR-110'
  AND op.operation_number = 20
ORDER BY m.machine_number, o.name;
```

machine_number	model	name	level
M-04	MIG welder MW-250	Linda Nguyen	3
M-04	MIG welder MW-250	Maria Lopez	2
M-05	MIG welder MW-250	Aisha Patel	1
M-05	MIG welder MW-250	Linda Nguyen	3
M-07	Robotic welder RW-6	NULL	NULL

The second request fails the trace. A `WorkOrder` is linked to the `ManufacturedPart` it makes, and through it to the part's operations, but nothing links a work order to a machine or an operator. The model records what *can* be done, the routing and the certifications, and what was *ordered*, the work orders, but not what was *done*.

The check. The joins follow the associations of Figure 5.20 in the order the trace found them. The join from the operation to its machines passes through the work center without naming it, by matching the two foreign

keys that refer to it, and the left joins keep a machine that has no certified operator. The operation is performed in the Welding work center, which has three machines, so the result must show at least three machines; M-04 and M-05 each have two certified operators, and M-07 has none, which gives five rows.

Interpretation. For the first request, the review confirms the model and repeats a finding of Exercise 3.16: the robotic welder M-07 has no certified operator. For the second, the review finds a missing piece: a record of each run, which operator ran which machine for which operation of which work order, and for how long. That is the ternary association of Figure 5.14, extended with the operation, and promoted to a class `Run` with an association to each of the classes it connects. Adding it is a decision for the business: recording runs costs a clerk's time on the floor.

Where it breaks. A review against requests can only find what the requests ask for. A request nobody thought to list, such as the scrap produced by each run, will not be traced, and the model will lack it. The list of requests is therefore part of what the business must review.

A model that survives both checks is ready to be turned into tables, which Chapter 6 does. A model that fails either is far cheaper to repair now than after the tables are loaded with data.

5.10. Reading other notations

This section presents how to read a data model drawn in the crow's foot notation, which you will meet in other books and in many database tools.

UML is not the only notation for data models. The **entity-relationship diagram** (ERD) is older and widespread, and its most common form is the **crow's foot notation**, named for the three-pronged mark that stands for "many". Figure 5.21 draws the refined model of stage D3 (Figure 5.11) in crow's foot notation.

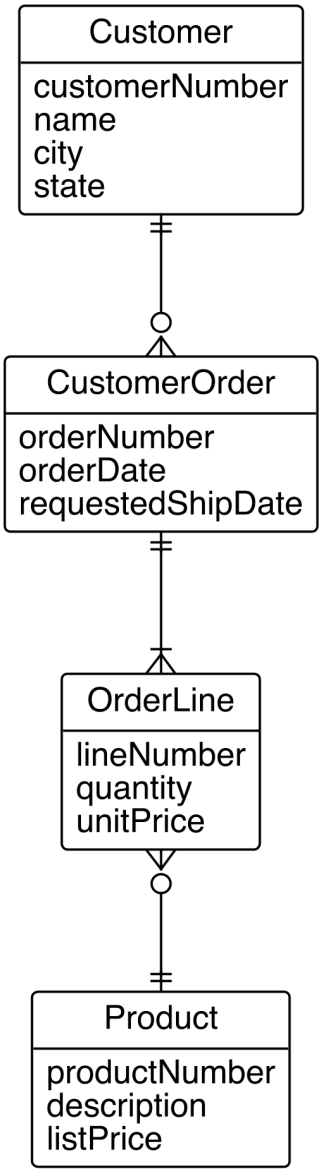


Figure 5.21.: Stage D3's Refined Model in Crow's Foot Notation

Most of Figure 5.21 maps one to one onto the class model. A box is an **entity type**, the ERD's name for what UML calls a class, with its attributes listed below its name. A line is a relationship, the ERD's association. The difference is in how the multiplicity is written: a pair of marks at each end instead of numbers. The mark nearer the box is the maximum, a bar for one and the crow's foot for many; the mark farther from the box is the minimum, a bar for one and a circle for zero. The four combinations are:

Table 5.8.: Crow's Foot Marks and Their UML Multiplicities

Marks (farther from the box, nearer the box)	Read as	UML multiplicity
bar, bar	exactly one	`1`
circle, bar	zero or one	`0..1`
circle, crow's foot	zero or more	`0..*`
bar, crow's foot	one or more	`1..*`

Read Figure 5.21 with Table 5.8, and it says nearly all that Figure 5.11 says. The line from `Customer` to `CustomerOrder` has two bars at the customer and a circle with a crow's foot at the order: each order belongs to exactly one customer, and each customer has zero or more orders. The marks sit at the same ends as UML's numbers do, so the reading rule of 1 applies unchanged.

Some UML constructs have no direct crow's foot equivalent. An association class is drawn as an ordinary entity type, called an associative entity, between the two it connects; a qualifier has no mark at all and appears only as a key attribute; aggregation and composition have no diamonds; and an optional attribute has no mark, so `requestedShipDate [0..1]` in Figure 5.11 appears in Figure 5.21 as plain `requestedShipDate`. Crow's foot tools also often show foreign keys as attributes, which makes their

diagrams closer to the schema diagram of Figure 2.2 than to a class model. Appendix D translates each construct in both directions, with a procedure for converting a whole model. Other notations exist, including the original entity-relationship notation of Peter Chen, with diamonds for relationships; this book does not use them, and a reader who meets one can translate it by the same method.

5.11. Summary

A class model is the design, and tables are one implementation of it. It describes the things a business records and how they are related, in the business's words, without keys or NULLs. Chapter 6 turns it into tables.

A class model starts from what the business says it needs. A problem statement bounds the system, use cases name its functions and the actors who start them, and requirements, each written as what the system *shall* do, say what it must answer. Most of the model's nouns appear first in these documents.

A class is a set of objects, and a cloud instance diagram draws the set. An object has an identity of its own; a class describes objects with the same kinds of facts and relationships. Attributes record the facts, drawing values from domains, and leave identifiers and references to the implementation.

How a value is used decides how a value with parts is modeled. A value used whole is one attribute, a value whose parts are used separately is split into attributes, and a value that repeats or is shared becomes a class of its own. Each becomes one column, several columns, or a table.

An association is a set of links, and multiplicity is counted. A link connects two particular objects, and the multiplicity at an end of an association says how many lines may leave each object at the other

end. Read every association as two sentences, and remember that one association holds at most one link between a given pair of objects.

Facts about a pair belong to the link. An association class holds them, one object per link, which suits stock levels and shipment lines and fails the order lines of an order that lists a product twice. A qualifier selects among the objects linked to one object, as a line number does within an order, and becomes a composite key.

A diamond says that one object is part of another. Composition, the filled diamond, makes a part belong to one whole for life, as an order line belongs to its order; aggregation, the hollow diamond, is left to the modeler by UML, and this book uses it for a part with a life of its own, as a machine is part of a work center. A bill of materials relates kinds of parts, not particular objects, and is an ordinary association.

A subclass is a kind of its superclass, and inherits all of it. Its objects are a subset of the superclass's objects, a cloud inside a cloud, and `{disjoint, complete}` states how the subsets divide the whole. Use a subclass for a kind of thing with facts or associations of its own, and a role name for a part an object plays.

A model is built by finding, eliminating, and testing candidates. The nouns of the documents are the candidate classes; each is kept, or eliminated as redundant, irrelevant, vague, an attribute, an operation, a role, derived, or an implementation detail, with the reason recorded. The classes that remain pass the uniformity, more than a name, *or*, and more than a list tests; verbs give the associations, and the data dictionary gives every attribute a domain specification.

A model is reviewed against itself and against its requests. The questions of Table 5.6 check the model's internal soundness, and tracing each request through the associations finds what the model cannot answer, before any table is built.

Other notations say the same things with different marks. Crow's foot notation writes multiplicity as a pair of marks at each end of a line,

at the same ends as UML's numbers; association classes, qualifiers, and diamonds need translation, which Appendix D provides.

Chapter 6 turns the class models of this chapter into the tables of the book's database, following Blaha and Premerlani's rules for each construct.

5.12. Exercises

Exercise 5.1. Match each term on the left with the phrase on the right that defines it.

Term	Definition
a. object	1. a connection between two particular objects
b. class	2. the set of a class's objects at a moment
c. extension	3. how many objects may be linked to one object at the other end
d. link	4. a particular thing the business needs to remember
e. multiplicity	5. an attribute of an association that selects among linked objects
f. qualifier	6. a description of a group of objects with the same kinds of facts and relationships

Exercise 5.2. Using Figure 5.7, write each association as two sentences, one in each direction, of the form "each … is linked to …". Then say, for each end, whether an object may exist without a link at that end.

Exercise 5.3. At stage D5, each shipment leaves from one warehouse, and the two warehouses have made eleven shipments between them.

a. Draw a cloud instance diagram of the warehouses, the shipments SH-9001 to SH-9005, and the links between them. The query of Exercise 3.11 shows how to find each shipment's warehouse.
b. Read the multiplicity at each end from your diagram.
c. In the sample, every warehouse has at least one shipment. Which minimum is a rule of the business that the diagram cannot show? Explain.

Exercise 5.4. The `Shipment` class of stage D5 records its carrier as the attribute `carrier`. The distribution manager now wants to record each carrier's contract rate per pound and its dispatcher's phone number, and to keep a list of approved carriers that includes carriers not yet used. Treat a customer pickup as one of the carriers on the list.

a. Should the carrier remain an attribute? Give the test of Section 5.3 that decides it.
b. Sketch the change to the class model, with multiplicities, and read the new association both ways.

Exercise 5.5. The production schema of stage P3 records, for each operator certified on a machine, the date of the certification and its level.

a. Explain why neither the date nor the level is an attribute of `Operator` or of `Machine`.
b. Model the certification with an association class, and give the multiplicities.
c. Could an operator be certified on the same machine twice, say at level 1 in 2020 and at level 3 in 2024? What would the association class model say, and what would the business need to decide?

Exercise 5.6. The plant extends the certifications of Exercise 5.5 with two more facts. Each certification is signed off by an operator who holds a level 3 certification on the same machine. And each machine has one lead operator, responsible for its upkeep; an operator may lead any number of machines, or none.

a. Draw a class model in which `Certification` is an association class of `Operator` and `Machine`, and add an association for the operator who signed each certification off and one for each machine's lead. Name the roles and give every multiplicity.
b. Draw a second model in which `Certification` is an ordinary class, associated with `Operator` twice and with `Machine` once.
c. Four certifications of stage P3 are to be recorded with their sign-offs: Maria Lopez (E-201) on M-04 and Aisha Patel (E-205) on M-05, both signed off by Linda Nguyen (E-203); Robert Hayes (E-204) on M-01, signed off by Maria Lopez; and Linda Nguyen on M-06, signed off by Robert Hayes. Linda Nguyen leads M-04 and M-05, Maria Lopez leads M-01, and Robert Hayes leads M-06. Draw a cloud instance diagram of these facts, in the manner of Figure 5.6, for each model.
d. Name a set of objects and links that the model of part (b) allows and the model of part (a) forbids. Would the plant want it?
e. Neither model enforces the rule that the signer holds a level 3 certification on the same machine. State that rule in words beside the diagram, and say what else a signer must not be.

Exercise 5.7. HG Industries decides to record where each product is stored within a warehouse. Each warehouse is divided into bins, identified by a bin code such as `A-03` that is unique within its warehouse; two warehouses may both have a bin `A-03`.

a. Model warehouses and bins without a qualifier, and give the multiplicities.
b. Model them with a qualifier, and explain how the multiplicity at the bin end changes.
c. What composite key would the qualifier become in a table?

Exercise 5.8. Consider the six hypothetical runs of Figure 5.14.

a. List the links as triples (operator, machine, work order), with their hours.

b. Replace the ternary association with three binary associations (operator with machine, machine with work order, operator with work order), and list the links each would hold.
c. List every triple consistent with the three binary associations, that is, every operator, machine, and work order whose three pairs are all links of part (b). Mark the triples that are not runs.
d. Remove Maria Lopez's run of M-01 on WO-7003 from the data. Show that every triple consistent with the binary associations is now a real run. Does that make the binary model right for these runs? Explain.
e. Where would the hours be recorded in the binary model? Explain why that is a problem.

Exercise 5.9. For each association, say whether it is a composition, an aggregation, or an ordinary association, and give the question of Section 5.6 that decides it.

a. `ManufacturedPart` and its `Operation`s (stage P3).
b. `WorkCenter` and `Machine` (stage P3).
c. `Customer` and `CustomerOrder` (stage D2).
d. `ManufacturedPart` and `Part` in the bill of materials (stage P2).
e. `Part` and its quality `Characteristic`s (stage Q).

Exercise 5.10. Using Figure 5.17 and Figure 5.18:

a. List the attributes an object of `PurchasedPart` has, and say where each comes from.
b. Why does the `assembly` end of the bill of materials attach to `ManufacturedPart` rather than `Part`? (Figure 5.20 shows the bill of materials.)
c. HG Industries considers buying wheel assemblies from a supplier when the plant is at capacity, while still making them at other times. Which of `{disjoint, complete}` would that break, and what are two ways to change the model?

Exercise 5.11. For each proposal, say whether it should be a subclass or a role name, and why.

a. `Inspector`, a subclass of `Operator`, for the operators who inspect parts.
b. `RetailCustomer` and `WholesaleCustomer`, subclasses of `Customer`, where wholesale customers have a credit limit and a sales representative and retail customers pay on order.
c. `ReferringCustomer`, a subclass of `Customer`, for customers who have referred another customer to HG Industries.

Exercise 5.12. A new analyst proposes the following model of part of stage D5, written as text:

- `Product` (productNumber, description, listPrice, fayetteville_on_hand, tulsa_on_hand, reorder_point)
- `Warehouse` (warehouse_pk, warehouseCode, city, state)
- `Shipment` (shipmentNumber, shippedOn, carrier, warehouse_fk, customerName)

Review it with the questions of Table 5.6. List every problem you find, say which question finds it, and describe the repair.

Exercise 5.13. HG Industries' quality department inspects lots of manufactured parts. An inspector measures a sample of parts from a lot against the part's characteristics, each with a lower and an upper specification limit, and counts defects by type. The department wants to know which lots failed, which characteristics drift, and whether inspectors agree.

a. Write a problem statement for the department's inspection records, answering the six questions of Section 5.2, and say what the system will not do.
b. Name the actors and at least three use cases, and draw the use case diagram.
c. Write the description of one use case, in the form of Table 5.1.

d. Write three requirements in the form "The system shall …", at least one of which is a request the model must answer.

Exercise 5.14. The receiving dock writes:

> *Parts that HG Industries buys arrive from suppliers by truck. Each delivery brings the parts of one purchase order, though an order may arrive in several deliveries. A purchase order lists, for each part, the quantity ordered and the price agreed. At the dock, a receiving clerk counts each part in the delivery and records the quantity received and any quantity rejected as damaged. Accounts payable pays suppliers from the receipts, which this system does not handle. The buyer needs to know which purchase orders are still open and how often each supplier delivers late.*

a. List every candidate class in the statement.
b. Decide each candidate as Table 5.5 does: a class, an attribute, or eliminated, with the reason.
c. Apply the four tests of step 3 to the classes you kept. Which candidate is closest to failing the *more than a name* test, and what would the buyer have to want recorded for it to pass?
d. Name each association, its kind from Table 5.4, and its multiplicities, and draw the class model.
e. Which requirement in the statement does your model answer with a derived value rather than a stored one?

Exercise 5.15. Write a domain specification for each attribute, and say which of the four ways (enumeration, citation, acceptance rule, range) you used.

a. `PurchasedPart.leadTimeDays`, the days a supplier needs to deliver.
b. `Customer.state`.

c. `Inspection.result`, an attribute the department proposes to add, which records whether a lot was accepted or rejected.
d. `Measurement.measuredValue` of a diameter, measured in inches.
e. An analyst wrote, for `OrderLine.unitPrice`, "Domain: a decimal number." Say what is wrong with it, and write a better specification.

Exercise 5.16. A supplier's catalog system is drawn in crow's foot notation with three entity types, `Supplier`, `CatalogItem`, and `PurchasedPart`. The line from `Supplier` to `CatalogItem` has two bars at `Supplier` and a bar with a crow's foot at `CatalogItem`. The line from `CatalogItem` to `PurchasedPart` has a circle with a crow's foot at `CatalogItem` and two bars at `PurchasedPart`. `CatalogItem` has the attributes `supplierPartNumber` and `unitPrice`.

a. Translate each line into a UML association with multiplicities, and read each both ways.
b. `CatalogItem` connects a supplier and a purchased part and carries a price. Should it be an association class of an association between `Supplier` and `PurchasedPart`? Consider whether a supplier might list the same part twice, say in two package sizes.

Exercise 5.17. HG Industries wants to record machine maintenance. Model the following in a class diagram, with multiplicities, and write a class description for each new class.

Each machine receives preventive maintenance on a schedule: a maintenance plan for a machine lists tasks, each with an interval in days, and a task is numbered within its plan. When a technician performs a task, the plant records the date, the technician, and the minutes it took. Technicians are employees but not operators, although an operator may be trained as a technician. A machine may also break down; each breakdown is recorded with its start and end times and the technician who repaired it.

Say which parts of your model are compositions, which multiplicities are decisions the business must confirm, and how you modeled "an operator may be trained as a technician".

The exercises from here on model settings outside HG Industries, each a short statement of business rules in plain English. Model each with the constructs of this chapter, give every multiplicity, and read each association both ways before you trust it. Where a statement leaves a rule open, say what you assumed. Chapter 6 maps several of these models to tables, starting from the instructor's model of each.

Exercise 5.18. A plant's tool crib lends tools to its mechanics. Each tool has a tool number, unique in the crib, a description, and the date it was purchased. Each mechanic has an employee number and a name. When a mechanic checks out a tool, the crib records the time; when the tool comes back, it records the return time. Over its life a tool is checked out many times, by many mechanics, and a mechanic often checks out the same tool again. A tool is out with at most one mechanic at a time, and a mechanic may have several tools out at once. New tools have never been checked out.

a. Model the tools, the mechanics, and the checkouts, with multiplicities.
b. Explain why a checkout cannot be an association class of an association between `Tool` and `Mechanic`.
c. Which rule of the statement can no multiplicity in the diagram state? How would you record it?

Exercise 5.19. A quality department keeps its gauges calibrated. Each gauge has a serial number, a type (such as micrometer or caliper), a measuring range, and a calibration interval in days. Calibrations are done by outside laboratories, each with a name and an accreditation number. Each calibration of a gauge records the date, the result (passed, adjusted, or failed), and the certificate the laboratory issued. A laboratory numbers its certificates itself, so two laboratories may both issue a certificate numbered

1047. A gauge may be calibrated by different laboratories over the years, and a newly bought gauge has not yet been calibrated. A laboratory is recorded only once it has calibrated a gauge.

a. Model the gauges, laboratories, and calibrations, with multiplicities.
b. Model the certificate number with a qualifier, and say what the qualifier states that an attribute of `Calibration` would not.
c. The department also wants each gauge's next due date. Is it an attribute to record? Explain.

Exercise 5.20. A machining cell pulls parts with kanban containers. Give the multiplicity at each end of each association, and read each association both ways.

a. Each container is dedicated to one part number. Each part number circulates in at least two containers, since the cell runs a two-bin system.
b. Each container belongs to the loop of one cell; a cell's loop may hold containers of many part numbers, and a new cell has none yet.
c. A kanban card is printed for one container and is never moved to another. A container carries at most one card; a new container has none until its card is printed.
d. Draw the class model of parts (a) to (c), with `PartNumber`, `Container`, `Cell`, and `Card` and attributes of your choosing.
e. Should the card be a class of its own, or attributes of `Container`? Give one argument each way.

Exercise 5.21. A plant's occupational health clinic schedules appointments. Each appointment is for one employee, with one provider (a nurse or a physician), in one exam room, at a date and time, for a number of minutes, with a reason and a status (scheduled, completed, or missed). An employee has many appointments over the years, often with the same provider in the same room.

a. Model the employees, providers, exam rooms, and appointments.
b. A classmate models the appointment as a ternary association of `Employee`, `Provider`, and `ExamRoom`, with an association class. Explain, with the rule of 1 about links, why that model cannot record the clinic's appointments.
c. Each provider has a home exam room, and a room is the home of at most one provider. Add this to your model.

Exercise 5.22. A project scheduler records projects and their activities. Each project has a name and a start date. Each activity belongs to one project, has an identifier unique within its project (such as `A`, `B`, `C`), a description, and a duration in days. An activity may have predecessors, activities that must finish before it can start, and it may be a predecessor of several other activities; the first activities of a project have no predecessors. Each precedence may carry a lag, a number of days that must pass between the end of the predecessor and the start of its successor.

a. Model projects, activities, and precedences, with role names on the recursive association.
b. Where does the lag belong, and why?
c. A precedence must join two activities of the same project, and no chain of precedences may lead from an activity back to itself. Can the diagram state either rule?

Exercise 5.23. A company records the places in its plants: each site has buildings, each building has areas, and each area may have production lines. Every building is in one site, every area in one building, and every line in one area. A site has at least one building, and a building at least one area.

a. Model the four levels as four classes.
b. Model the same places with one class, `Location`, and a recursive association.

c. The company acquires a plant whose areas are divided into zones, and zones into lines. What must change in each model?
d. Which model states a request for everything inside a building as one idea, whatever the depth, and which lets SQL answer it without a recursive query? Recall Section 4.6.

Exercise 5.24. A city's bike share has stations, each with a name and a location. Each station has docks, numbered 1, 2, 3, and so on within the station. A bike, identified by its bike number, is locked in at most one dock, and a dock holds at most one bike. Members, with a member number and a name, take rides: a ride uses one bike, starts at one station at a time, and ends at a station at a later time, often a different station; a ride in progress has not yet ended.

a. Model the stations, docks, bikes, members, and rides.
b. Explain the role names your model needs, and why.
c. What does the qualifier on the dock number state, and what would change if docks were numbered across the whole city instead?

Exercise 5.25. A port's container yard is divided into blocks, each identified by a block code within the yard. A place for a container in a block is a slot, identified by its bay, row, and tier numbers together; a slot may be refrigerated. Each shipping container has a container number, a size in feet, and an owner, and occupies at most one slot while in the yard; a slot holds at most one container.

a. Model the yard, its blocks, slots, and containers, using qualifiers.
b. Which associations are compositions? Apply the questions of Section 5.6.
c. A qualifier of three attributes, bay, row, and tier: what composite key would it become in a table?

Exercise 5.26. A hospital's emergency department records each patient visit: the arrival time and, when the patient leaves, the departure time.

On arrival a nurse assigns the visit a triage level from 1 (resuscitation) to 5 (non-urgent); each level has a description and a target number of minutes within which the patient should be seen. During a visit a patient may be moved between beds, and may return to a bed used earlier in the same visit; each move records when the patient took the bed and when the patient left it. Each bed has a code and a type (such as trauma or general).

a. Model patients, visits, triage levels, beds, and bed assignments.
b. Is the triage level an attribute of `Visit` or a class? Give the argument for your choice.
c. Why is the bed assignment a class and not an association class between `Visit` and `Bed`?

Exercise 5.27. An equipment rental yard rents units of equipment, each unit identified by its serial number and of one equipment type (such as a 20-foot scissor lift). Rates depend on the type and the rental period: a scissor lift rents for one amount per day, another per week, and another per month, and every unit of the type rents at the same rates. Customers rent units; each rental is of one unit and records the date out, the date due, the date returned, and the rental period it is charged at.

a. Model equipment types, units, rate periods, rates, customers, and rentals.
b. Where does the rate's amount belong, and why is it not an attribute of `EquipmentUnit`?
c. The distinction between a type and its units recurs in many settings. Name one other pair from this chapter's exercises or from HG Industries.

Exercise 5.28. A fleet manager tracks the plant's vehicles. Every vehicle has a unit number, a make, a model year, and an hour meter. Trucks and vans are road vehicles, with a license plate, a registration expiry date, and an odometer; trucks also have a payload and a number of axles, and vans

a cargo volume. Forklifts are not road vehicles; they have a lift capacity and a fuel type. Every vehicle is a truck, a van, or a forklift, and never more than one.

a. Model the vehicles with generalization, placing each attribute on the highest class it applies to, and state the constraints.
b. Some vehicles are leased and some are owned. Should `LeasedVehicle` be a subclass? Recall Exercise 5.11.

Exercise 5.29. For each pair, say whether the association is a composition, an aggregation, or an ordinary association, and give the question of Section 5.6 that decides it.

a. A pallet and the cartons stacked on it.
b. An aircraft and its engines, which are removed for overhaul and installed on other aircraft.
c. An invoice and its lines.
d. A building and its rooms.
e. A maintenance crew and its technicians.
f. A kit and the parts packed in it for an assembly station.

The next six exercises model one system, a transit agency's monitoring system, adapted from a Transportation Research Board study (Rossetti 1999). Each models one part of it; together they cover its routes, stops, schedules, operations, fares, and the record of each trip a vehicle makes.

Exercise 5.30. A transit agency runs bus routes. Each route has a number, a name, a color on the system map, and the distance the agency publishes for it. The places on the network are points, each with a name and a latitude and longitude. A route is a sequence of segments, in the order the bus travels them; each segment starts at one point and ends at another and has a length. Each segment belongs to exactly one route; two routes that run along the same street have segments of their own. A point may lie on many routes, and a route has at least one segment. Points of interest,

such as a shopping center or a hospital, each with a name and an address, are recorded with the points near them; a point of interest is near at least one point.

a. Model routes, segments, points, and points of interest.
b. A route's points, in order, are also wanted. Should the model have an association between `Route` and `Point`? Explain what it would add and what it would cost.
c. Read the association between `Segment` and `Point` both ways, for each role.

Exercise 5.31. Some points of the network of Exercise 5.30 are access points, where passengers may board or leave the bus; the others are places such as timing points and turnarounds. An access point records where it is on its block (midblock, near side, or far side of an intersection), the capacity of its waiting area, its number of benches, and whether it has a shelter. Some access points are scheduled points, which appear on the printed timetables and record whether a timetable is posted there.

a. Model the three kinds of point with generalization, and state each generalization's constraints.
b. A rider asks for a stop where the bus will drop off only on request. Is a `RequestStop` a new subclass, or an attribute? Explain.

Exercise 5.32. Each route of Exercise 5.30 has one or more schedules, each with a name (such as "Weekday" or "Saturday"), the date it was last updated, and the headway, the minutes between buses. A schedule applies to one or more days of the week, and a day of the week is covered by schedules of many routes. A schedule lists, in order, the times a bus is due at the scheduled points of Exercise 5.31: each line gives one scheduled point, an arrival time, and a departure time. A scheduled point appears on many lines of many schedules.

a. Model routes, schedules, days of the week, schedule lines, and scheduled points.

b. Why is the schedule line a class of its own rather than an association class between `Schedule` and `ScheduledPoint`?

Exercise 5.33. A transit manager assigns a driver to a vehicle on a route for a period of time, with a start time, an end time, and comments. Over a month the same driver drives the same bus on the same route on many days, and a driver drives many vehicles, a vehicle is driven by many drivers, and a route is served by many vehicles and drivers. Each driver has a name and a license number; each vehicle has a vehicle number and a seating capacity.

a. Model the assignment first as a ternary association with an association class, then as a class with three binary associations.
b. Which model can record the month described above? Use the rule of 1.
c. The study this exercise is adapted from made the assignment a class. What else does the class allow that the ternary association does not?

Exercise 5.34. The agency sells smart cards, each with a card number and the date issued. Every card is either a debit card, which holds a balance that each trip draws down, or a credit card, whose trips are billed to an account; an account, with an account number and a balance, has one or more credit cards. Each account has one holder, a person, who may authorize other people to use the account's cards; a person may hold several accounts and be authorized on others. A person has a name and, optionally, a rider category (student, senior, or standard), and may have a mailing address, which several people in one household may share. Each trip is made with one card, from an origin point to a destination point of Exercise 5.30, at a start and end time, for a fare. Payments are made to an account, each with a date, an amount, and a payment type (cash, check, or charge).

a. Model the cards, accounts, people, addresses, trips, and payments, recording the address by one of the choices of Section 5.3.1.

b. State the generalization's constraints, and say what they rule out.
c. Read the two associations between `Person` and `Account` both ways.
d. Where would you record that a person may use an account's cards only until a given date?

Exercise 5.35. Each time a vehicle runs a route of Exercise 5.30, the monitoring system records a route traversal, with a start and end time. A route traversal consists of point traversals, one for each point the bus reached, in order, each with the arrival and departure times and the numbers of passengers boarding and alighting; and link traversals, one for each segment the bus traveled between two point traversals, in order, each with a start time, an end time, and an average speed. Each point traversal is at one point, and each link traversal travels one segment.

a. Model route traversals, point traversals, and link traversals, with their associations to vehicles, routes, points, and segments.
b. The structure of a traversal mirrors the structure of a route. Explain the correspondence, class by class.
c. Which rules connecting the two structures can the diagram not state? Give two.

The last six exercises reverse the direction: each gives a class diagram and asks for the English description a business reader could check. For each diagram, write every association as two sentences, one in each direction, with the multiplicities in words; state each constraint; describe each association class and qualifier; and name one rule of the business that the diagram does not state.

Exercise 5.36. Figure 5.22 is part of the monitoring system of the transit exercises, the summary statistics computed from trips and traversals. Describe it in English.

Exercise 5.37. Figure 5.23 shows the event logs a vehicle's data terminal records and the office imports. Describe it in English, and explain what the generalization's `{disjoint, incomplete}` allows.

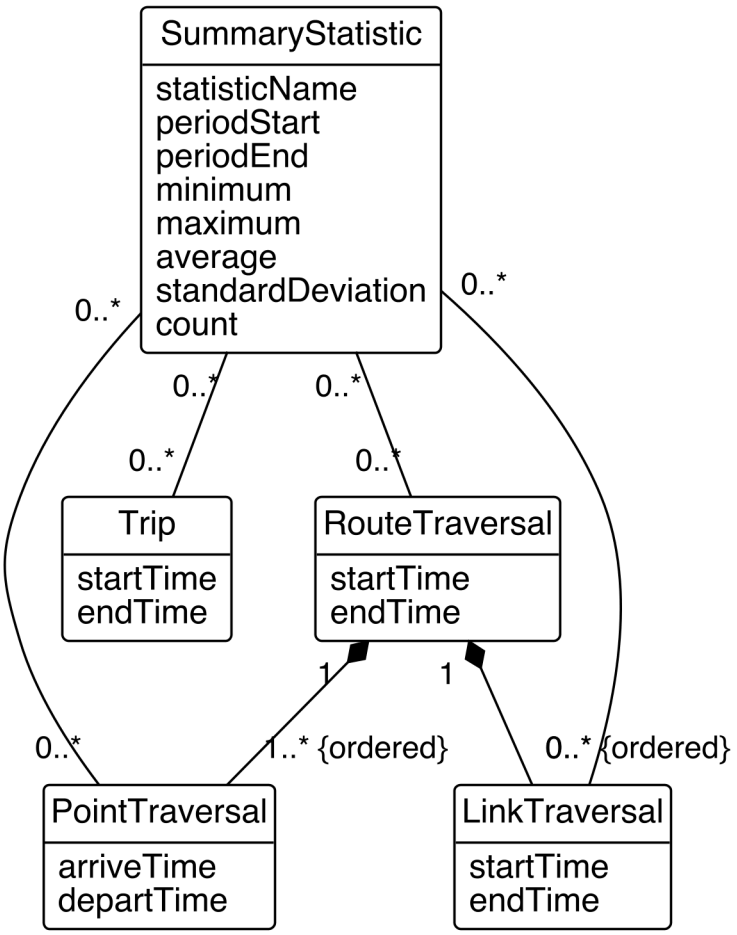


Figure 5.22.: Summary Statistics of the Monitoring System

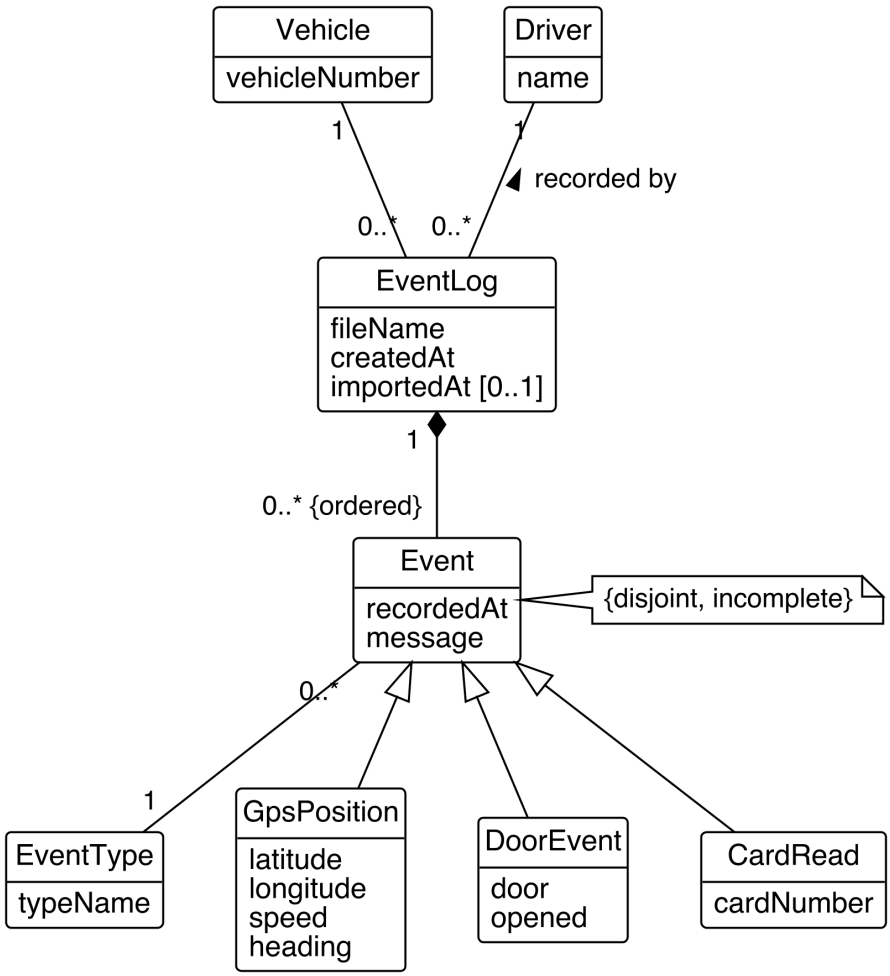


Figure 5.23.: Event Logs of the Monitoring System

Exercise 5.38. Figure 5.24 shows transfers between access points. Describe it in English, and give an example of a transfer that is not symmetric.

Exercise 5.39. Figure 5.25 shows a testing laboratory's samples and their chain of custody. Describe it in English. What does the qualifier on the sample number say about how samples are numbered?

Exercise 5.40. Figure 5.26 shows cycle counting in a distribution center. Describe it in English. Can a count exist whose item is not stocked at its location? What does the diagram say, and what should the business decide?

Exercise 5.41. Figure 5.27 shows engineering drawings and their revisions. Describe it in English, including the two associations between `Revision` and `Engineer` and what the generalization's constraints mean for a drawing.

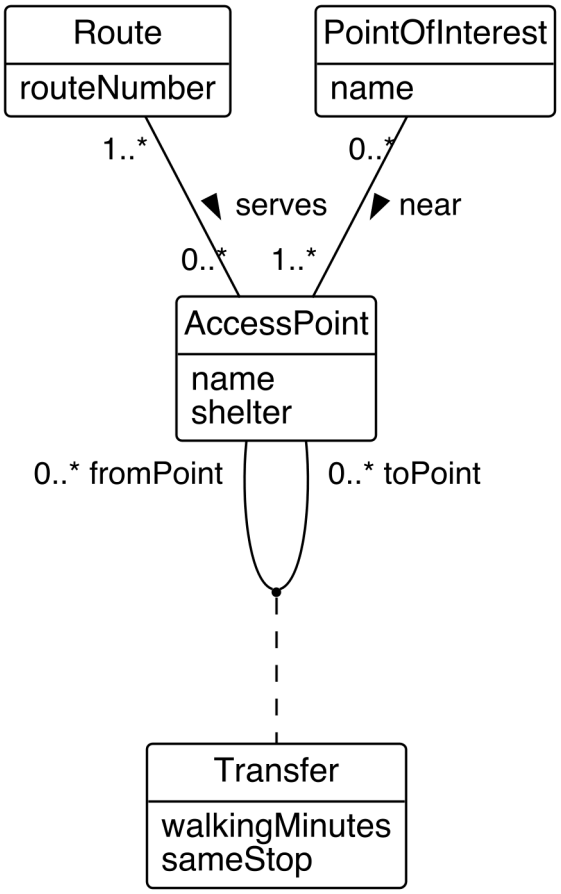


Figure 5.24.: Transfers Between Access Points

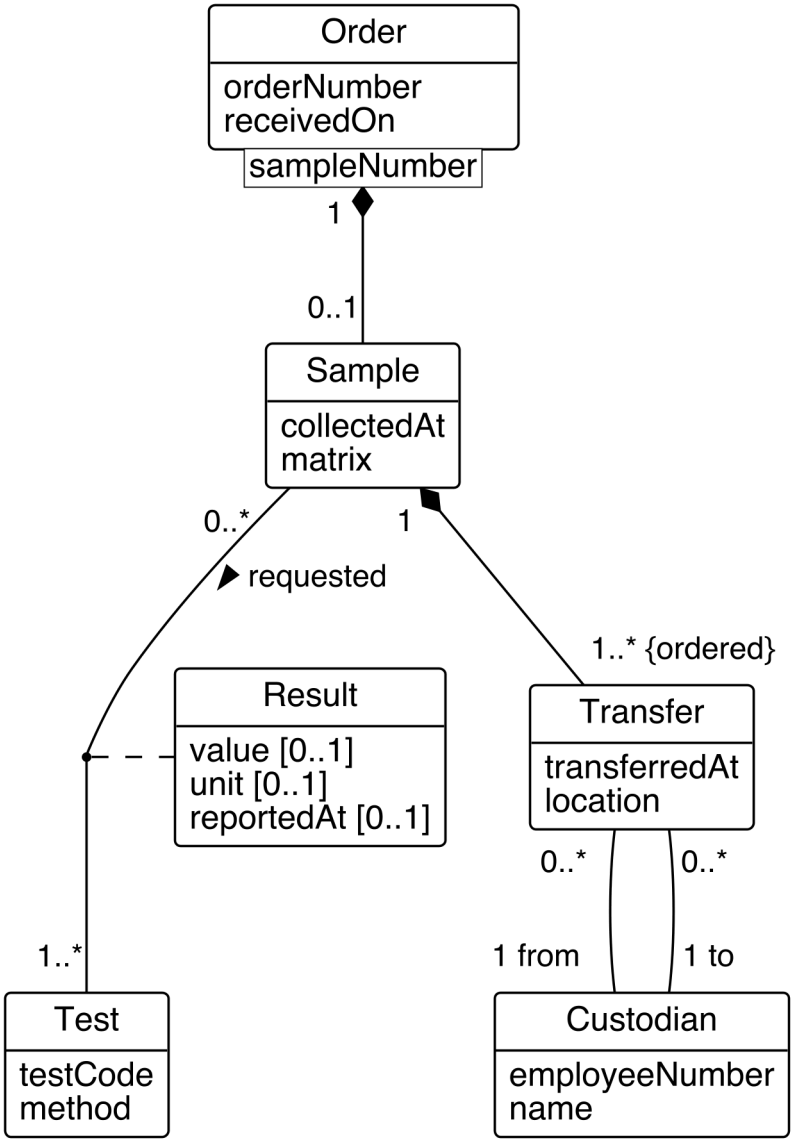


Figure 5.25.: A Laboratory's Samples and Their Chain of Custody

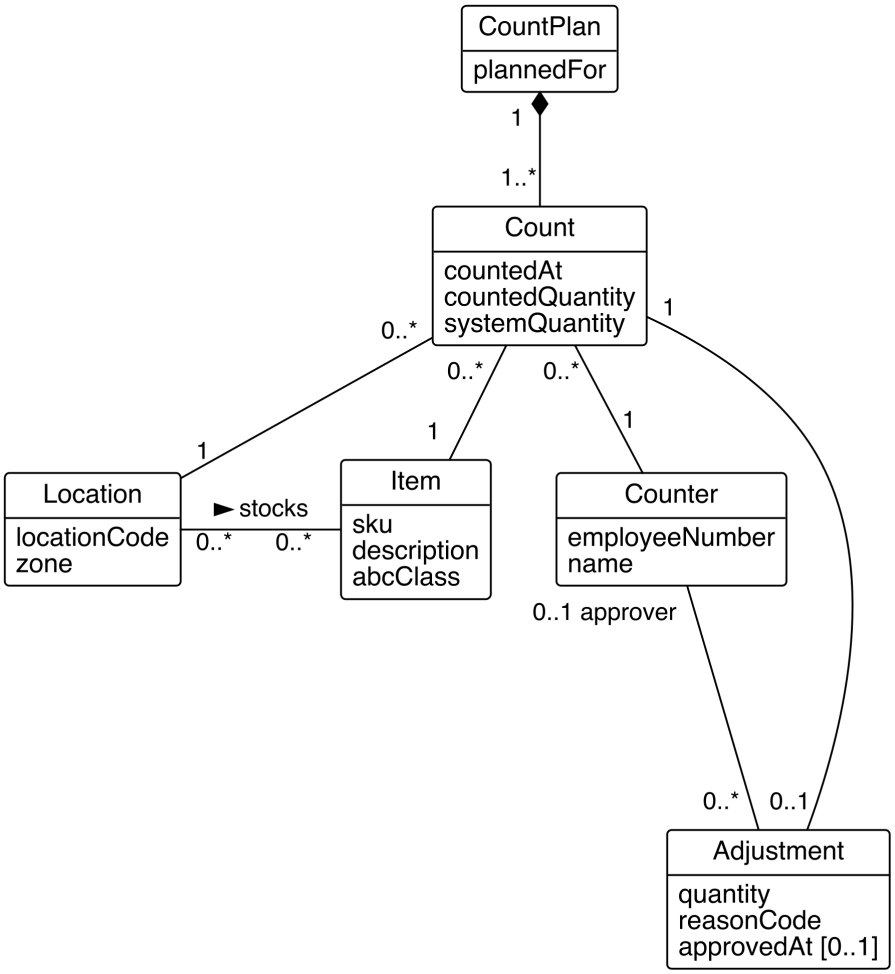


Figure 5.26.: Cycle Counting in a Distribution Center

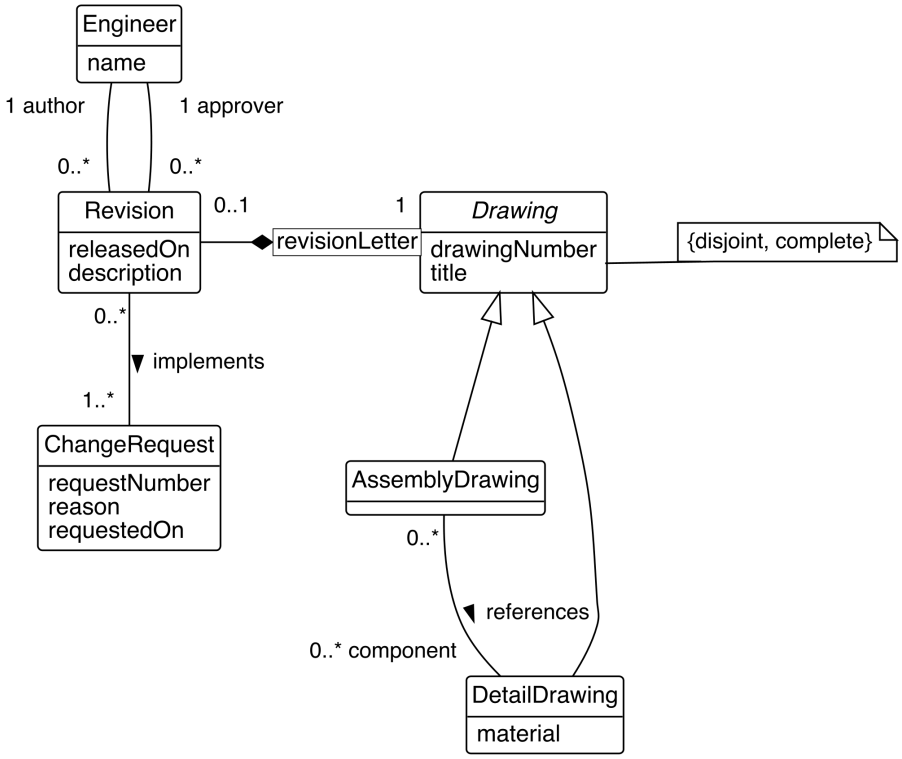


Figure 5.27.: Engineering Drawings and Their Revisions

6. From Class Models to Tables

Learning objectives

After reading this chapter you should be able to:

- list the steps of mapping a class model to tables, and say what each step produces
- map a class to a table, with columns whose types and constraints implement the attributes' domains
- map one-to-many, one-to-one, and many-to-many associations to foreign keys and tables
- decide which table holds the foreign key of a one-to-one association, from the business context
- map association classes, recursive associations, and compositions
- map a qualified association to a composite key
- map generalization, and state what each mapping enforces and what it does not
- choose between surrogate and natural keys, and name keys consistently
- write the complete relational schema that a mapping produces

Chapter 5 built class models of HG Industries: what the business records and how the things it records are related. Chapter 2 read the tables that implement those models. This chapter connects the two. It presents the rules that turn each construct of a class diagram into tables, columns, keys,

and constraints, following Blaha and Premerlani (Blaha and Premerlani 1998), and it applies them to the models of Chapter 5 to produce, step by step, the tables of the book's database.

The chapter begins with the whole process, seven steps from a reviewed class model to a complete relational schema, and then illustrates each step on the models of Chapter 5, showing each time the part of the class diagram being mapped and where each of its parts goes in the tables. The rules are few, and most of them are mechanical once the class model is right. That is the point of designing in a class model first: the judgment goes into the model, where the business can review it, and the mapping carries the model's meaning into tables without adding any. Every mapping in this chapter ends in a complete relational schema, in the notation of Section 2.4, that holds everything needed to create the tables, which Chapter 8 does.

Following along

The chapter maps the class models of Chapter 5, stage by stage, and shows the resulting schemas as your `dbconcepts` database holds them.

6.1. The mapping process

This section presents the whole mapping, from a class model to tables, as a sequence of steps, before the sections that illustrate each step.

Designing a database has two phases. The first turns a class model into a relational schema: the tables, their columns, their keys, and the rules that connect them. The second turns the relational schema into statements that a database system runs to create the tables. This chapter presents the first phase, and Chapter 8 the second. Figure 6.1 shows the first phase as seven steps.

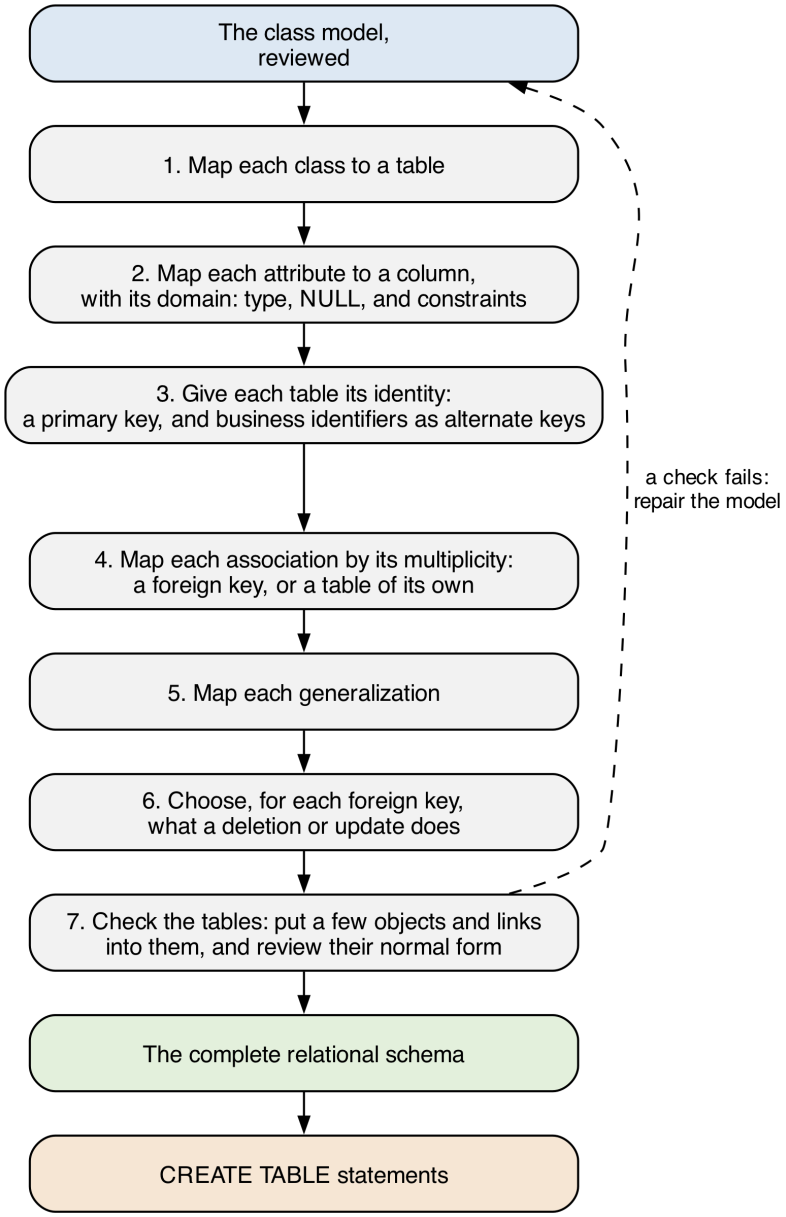


Figure 6.1.: The Steps of Mapping a Class Model to Tables

The steps of Figure 6.1 are applied to a whole model, one step at a time: all of its classes, then all of their attributes, and so on, since a later step needs the tables and keys of the earlier ones. In words, they are:

1. **Map each class to a table.** Every class becomes a table of its own, named for the class.
2. **Map each attribute to a column, with its domain.** Each column gets a type that holds the attribute's values, `NOT NULL` unless the attribute may be missing, and a `CHECK` for a domain narrower than its type.
3. **Give each table its identity.** Each table gets a primary key, which the class model does not name, and each business identifier becomes an alternate key.
4. **Map each association by its multiplicity.** An association becomes a foreign key in one of its classes' tables when a link can be recorded there as a single value, and a table of its own when it cannot.
5. **Map each generalization.** A superclass and its subclasses become tables that share a key, or are combined, in one of three ways.
6. **Choose what a deletion does to each foreign key.** A foreign key refuses the deletion of the row it refers to, unless its association is a composition, whose parts go with their whole.
7. **Check the tables.** Put a few objects and links into the tables, as rows, and see whether they hold them without loss or contradiction; then review the tables' normal form, which Chapter 7 presents. A check that fails sends the designer back to the class model, along the dashed arrow of the figure.

The result is a complete relational schema, in the notation of Section 2.4: every table with its columns, types, NULL rules, keys, checks, and the deletion rule of each foreign key. It holds everything Chapter 8 needs to create the tables. The sections that follow illustrate the steps in order:

Table 6.1.: The Steps of the Mapping and Where They Are Illustrated

Step	Illustrated in
1 to 3: classes, attributes, identity	Section 6.2, and in depth Section 6.6
4 and 6: associations and deletions	Section 6.3 and Section 6.4
5: generalization	Section 6.5
7: checking the tables	the check and the interpretation of Example 6.3

Step 7 deserves a word before the details. When a mapping seems hard, the trouble is usually in the class model, not the rules. The remedy is the one of 1: draw a cloud instance diagram of a few objects and links, and put the same objects and links into the tables as rows. If the rows cannot hold what the diagram shows, or hold something it forbids, the model or the mapping is wrong, and the diagram says where.

6.2. Mapping classes

This section illustrates steps 1 to 3 of Figure 6.1: how a class becomes a table, its attributes become columns, and the table gets its keys. Figure 6.2 shows the `Customer` class of Figure 5.1, with a note that says where each of its parts goes.

Each class maps to a table, as the note of Figure 6.2 begins. The table is named for the class, in the snake_case of Section 2.4 (`CustomerOrder` becomes `customer_order`), and in the singular, since the table implements the class and the class describes one object. Each attribute maps to a column, named the same way (`customerNumber` becomes `customer_number`). Recall from Section 5.3.1 that a value split into parts, such as a point's location, is several attributes (`latitude` and `longitude`), and so becomes several columns.

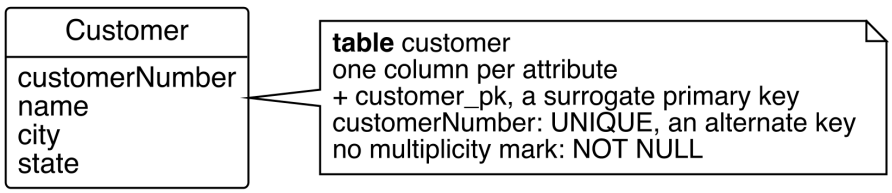


Figure 6.2.: Mapping a Class

Each column needs a data type, and the type comes from the attribute's domain (Section 5.3). Choosing it is the first place where a design meets the implementation, and it is best done in two steps. The first specifies each attribute's domain in words, with whether the attribute may be missing; the second chooses the SQL type and the constraints that implement each domain. For the `Customer` class of Figure 5.1, the two steps give:

Table 6.2.: Specifying the Domains of the Customer Class

Attribute	Domain	May be missing?	SQL type and constraints
`customerNumber`	a number assigned by the company, `C-` and four digits	no	`VARCHAR(10) NOT NULL UNIQUE`
`name`	the business's name	no	`VARCHAR(60) NOT NULL`
`city`	a city name	no	`VARCHAR(40) NOT NULL`
`state`	a two-letter US state code	no	`CHAR(2) NOT NULL`

Three things in Table 6.2 come straight from the class model. An attribute without a multiplicity has exactly one value, so its column is `NOT NULL`; an

attribute marked `[0..1]`, such as `requestedShipDate`, becomes a column that allows NULL. A business identifier becomes a `UNIQUE` column, since the business promises that no two objects share it; `customerNumber` is the alternate key found in Example 2.1. And a domain narrower than any SQL type, such as a price that must be positive, becomes a `CHECK` constraint, as `list_price > 0` does for products.

Each table needs a primary key, and here the class model gives no help, deliberately: objects are distinguishable in themselves, and a class model names no identifier (Section 5.3). The mapping must supply one. The book's database supplies a **surrogate key** for every table: a column named for its table with the suffix `_pk`, whose values PostgreSQL generates. The business identifier stays a `UNIQUE` column beside it. Section 6.6 weighs this choice against using the business identifier itself as the primary key.

With a key chosen, the class is fully mapped. Listing 6.1 gives the complete relational schema of the `customer` table of stage D1, the result of mapping Table 6.2.

Listing 6.1 The Complete Relational Schema of the Customer Table

```
customer
    customer_pk      INTEGER      not NULL, generated
    customer_number  VARCHAR(10)  not NULL
    name             VARCHAR(60)  not NULL
    city             VARCHAR(40)  not NULL
    state            CHAR(2)      not NULL
    primary key: customer_pk
    alternate key: customer_number
```

Every line of Listing 6.1 traces back to the model or to a decision of this section: the columns to the attributes, `not NULL` to the attributes' multiplicities, the alternate key to the business identifier, and the primary

key, generated by the database, to the surrogate key. The table has no foreign key, since `Customer` in stage D1 has no associations. The next section adds them.

6.3. Mapping associations

This section illustrates steps 4 and 6 of Figure 6.1: how each kind of association becomes a foreign key or a table, according to its multiplicities, and what a deletion does to it.

Recall the cloud instance diagram of Figure 5.5: each link of the association between customers and their orders is a line from one customer dot to one order dot. A table has no lines. It records a link as a value: the row of each order holds the key of the customer it is linked to, in a foreign key column. That works because each order has exactly one link. Every rule of this section follows from one question: where can a link be recorded as a single value? Each case below shows the fragment of a class diagram being mapped, with a note on the class whose table receives what the association becomes.

6.3.1. One-to-many associations

Figure 6.3 shows the association between customers and their orders of Figure 5.1.

Read the multiplicities of Figure 6.3 from the `CustomerOrder` end. Each order is linked to exactly `1` customer, so each order's row can hold its customer's key; the note on the link says so. A one-to-many association therefore maps to a foreign key in the table of the many end: `customer_fk` in `customer_order`, referring to `customer_pk`. The minimum at the *one* end decides NULL. The `1` at the `Customer` end has a minimum of one, so every order must have its customer and `customer_fk` is `NOT NULL`, as the note on `CustomerOrder` says; a minimum of `0` there would allow NULL for

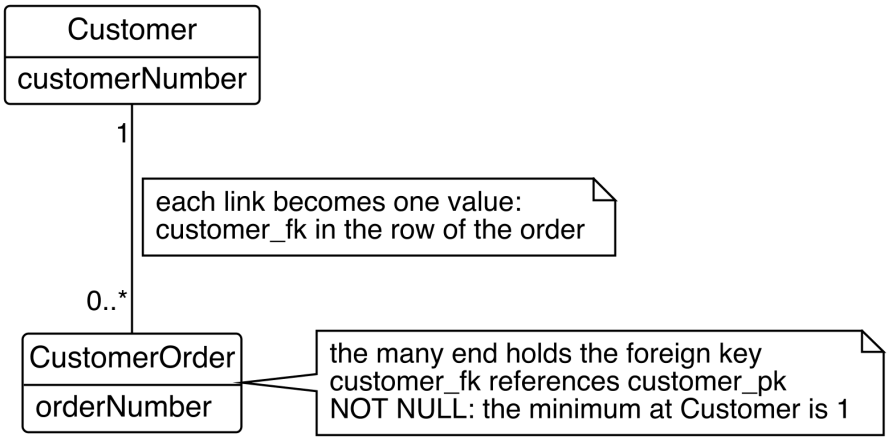


Figure 6.3.: Mapping a One-to-Many Association

an order with no customer. The `0..*` at the `CustomerOrder` end needs nothing in the tables: a customer with no orders is simply a customer whose key appears in no order's `customer_fk`. Listing 6.2 gives the resulting relational schema.

Listing 6.2 The Relational Schema of the One-to-Many Association

```
customer (customer_pk, customer_number, name, city, state)
    primary key: customer_pk
    alternate key: customer_number

customer_order (customer_order_pk, order_number, order_date,
                requested_ship_date, customer_fk)
    primary key: customer_order_pk
    alternate key: order_number
    customer_fk references customer_pk in customer,
        must not be NULL
```

Each element of Figure 6.3 has a line in Listing 6.2. The two classes are the two tables; the association is the last line, the foreign key that references `customer_pk`; and the minimum of `1` at the `Customer` end is its `must not be NULL`. This is the schema of stage D2 that Chapter 2 read in Listing 2.1, now seen as the product of a mapping.

6.3.2. One-to-one associations

The book's database has no one-to-one association of its own, but the maintenance model of Exercise 5.17 has one: each machine has at most one maintenance plan, and each plan belongs to exactly one machine, as Figure 6.4 shows.

Each object at each end of Figure 6.4 has at most one link, so either table could hold the foreign key. The note puts it in the table of the end whose objects always have a link, the plan's, so that it can be `NOT NULL`: every plan has its machine. A second constraint keeps the association one-to-one. Without it, two plans could name the same machine, and the `0..1` at the plan end would become `0..*`. A one-to-one association therefore maps to

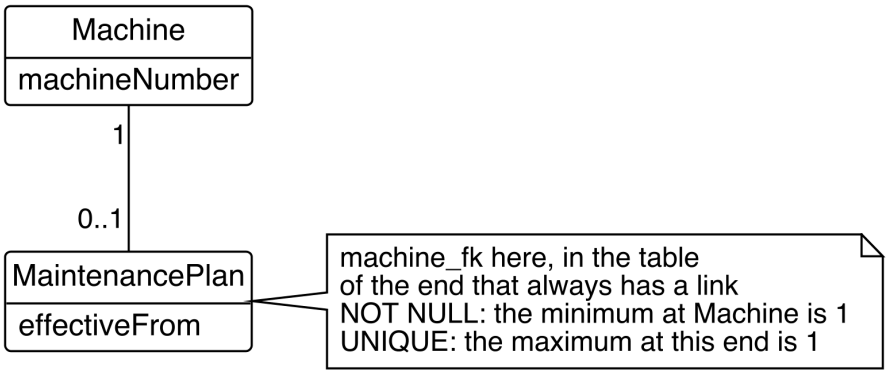


Figure 6.4.: Mapping a One-to-One Association

a foreign key made `UNIQUE`: `machine_fk` in `maintenance_plan`, `NOT NULL` and `UNIQUE`. The `machine` table is stage P3's, as Listing 6.3 gives it.

Listing 6.3 The Relational Schema of the Machine Table

```
machine (machine_pk, machine_number, model, installed_on,
         work_center_fk)
    primary key: machine_pk
    alternate key: machine_number
    work_center_fk references work_center_pk in work_center,
        must not be NULL
```

The `maintenance_plan` table is hypothetical, since the book's database holds no maintenance plans, so Listing 6.4 is written out by hand, in the notation of the book's generated listings.

Listing 6.4 The Relational Schema of the Maintenance Plan (Hypothetical)

```
maintenance_plan (maintenance_plan_pk, effective_from,
                  machine_fk)
    primary key: maintenance_plan_pk
    alternate key: machine_fk
    machine_fk references machine_pk in machine,
        must not be NULL
```

The `UNIQUE` constraint on `machine_fk` appears in Listing 6.4 as an alternate key, which is what it is: no two plans share a machine, so a machine identifies at most one plan. The `machine` table of Listing 6.3 is unchanged by the association, as the `0..1` at the plan end requires nothing of it.

6.3.3. Choosing the side of a one-to-one association

In Figure 6.4 the minimums decided the side: one end is mandatory and the other optional, so the foreign key went in the table of the objects that always have a link, where it could be `NOT NULL`. When both ends are `1`, or both are `0..1`, the multiplicities treat the two tables alike, and either side gives a correct schema. The choice is then a design decision, and the business context decides it. You should weigh three questions, in order:

1. **Does one object depend on the other?** An object that exists only for the other, or is always created after it, holds the foreign key: a maintenance plan exists for its machine. If the association is a composition, step 6 then lets the dependent object's row be deleted with its whole.
2. **Which end could become many?** A rule that holds today may loosen. Put the foreign key in the table of the end that could become many; if the rule loosens, the association becomes one-to-many, and the change is only to drop the `UNIQUE` constraint.

3. **Which table do users start from?** The question asked most often names one object and wants that object's own facts with its partner's identifier. A foreign key in that object's table answers it from one table, with no join, provided the column holds an identifier that people read.

The order matters. A schema that must be changed when the business changes costs more than a join, so the second question outranks the third.

Two other mappings complete the choices. When both ends are `1` and the two objects are created and removed together, one table can hold the attributes of both classes, and no foreign key is needed. When both ends are `0..1`, a foreign key on either side would often be NULL; a table of its own for the association, holding the two foreign keys, each `UNIQUE`, records only the links that exist.

Whichever side holds it, one foreign key enforces one end fully and the other end only in part. A foreign key that is `NOT NULL` and `UNIQUE` gives each row of its table exactly one partner, the `1` at the partner's end, and lets no two rows share a partner, the maximum of the `1` at its own end. Nothing makes every row of the other table a partner, so the minimum of that `1` is left to the application, or to a query that looks for the rows it misses.

Example 6.1 (Assigning lockers to operators)**.** *The system.* The production schema of stage P3 holds the plant's six operators. Suppose the plant also tracks the locker room: each locker is numbered on its door (L-101 to L-106), every operator has exactly one locker, and, apart from the day between an operator leaving and the next hire, every locker has exactly one operator. Figure 6.5 models the rule: the `1` at the `Locker` end says that each operator has exactly one locker, and the `1` at the `Operator` end that each locker has exactly one operator.

The request. Human resources asks almost every day for an operator's details with the number of the operator's locker, to issue a key or to

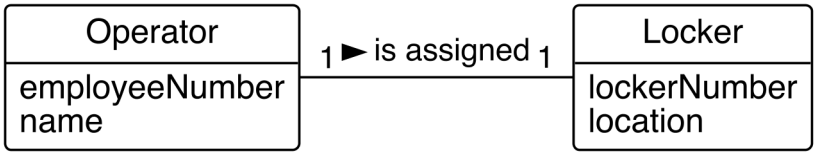


Figure 6.5.: Assigning Lockers to Operators, a One-to-One Association

welcome a new hire. “Who uses locker L-104?” is asked only when a locker is found open.

The design. Both ends of Figure 6.5 are `1`, so the minimums do not decide, and we take the three questions in turn. Neither object depends on the other: the lockers are fixtures of the room, and operators are hired and leave. The second question asks the business about the future, and the plant’s answer is that operators on different shifts may one day share a locker. The operator end could then become many, so the foreign key belongs in the `operator` table. The third question agrees: human resources starts from the operator and wants the operator’s own details with the locker’s number.

Under the book’s convention of surrogate keys (Section 6.6), the operator’s row would hold `locker_fk`, the locker’s `locker_pk`, as Listing 6.5 gives it.

Listing 6.5 Lockers and Operators, with a Surrogate Foreign Key (Hypothetical)

```
locker (locker_pk, locker_number, location)
    primary key: locker_pk
    alternate key: locker_number

operator (operator_pk, employee_number, name, hire_date,
          locker_fk)
    primary key: operator_pk
    alternate keys: employee_number, locker_fk
    locker_fk references locker_pk in locker,
        must not be NULL
```

A `locker_fk` of 4 means nothing to a clerk, so the everyday question still needs a join to `locker` to print L-104, although the foreign key sits in the table the question starts from. Recall from Section 2.2 that a foreign key refers to a candidate key, not only to the primary key, so any column declared `UNIQUE` will do. Referring to the locker's number instead, as Listing 6.6 does, puts the identifier people read in the operator's row. The column keeps the name of the key it holds, `locker_number`, rather than `locker_fk`, which the book keeps for a column that holds another table's surrogate key.

Listing 6.6 Lockers and Operators, with a Foreign Key to the Locker's Number (Hypothetical)

```
locker (locker_pk, locker_number, location)
    primary key: locker_pk
    alternate key: locker_number

operator (operator_pk, employee_number, name, hire_date,
          locker_number)
    primary key: operator_pk
    alternate keys: employee_number, locker_number
    locker_number references locker_number in locker,
        must not be NULL
```

The result. With the tables of Listing 6.6, the everyday question reads one table:

Table 6.3.: Each Operator with a Locker, from One Table (Hypothetical Lockers)

```
SELECT employee_number, name, locker_number
FROM production.operator
ORDER BY employee_number;
```

employee_number	name	locker_number
E-201	Maria Lopez	L-104
E-202	James Carter	L-101
E-203	Linda Nguyen	L-106
E-204	Robert Hayes	L-102
E-205	Aisha Patel	L-105
E-206	Tom Becker	L-103

The check. Each operator's row holds one locker number, so the query returns one row per operator, six in all, which Table 6.3 has. The six locker numbers are all different, as the alternate key on `locker_number` requires.

Interpretation. The design follows the plant's own answers: the foreign key sits where a shared locker would put it, and the question human resources asks every day needs no join. "Who uses locker L-104?" still has an answer, through the operator table, which Table 6.3 shows is Maria Lopez.

Where it breaks. Two conditions break the design. First, the locker numbers must stay as they are: if the plant renumbered its lockers, every operator's `locker_number` would have to change with them, which the response of cascading an update (Section 2.3), declared as `ON UPDATE CASCADE`, can do. Second, the foreign key enforces the `1` at the `Locker` end, but only the maximum of the `1` at the `Operator` end. A locker that no operator is assigned to breaks the minimum of that `1`, and nothing refuses it:

```
INSERT INTO production.locker (locker_number, location)
VALUES ('L-107', 'Locker room, row 2');
```

Table 6.4.: A Locker That No Operator Uses (Hypothetical Lockers)

```
SELECT l.locker_number, l.location
FROM production.locker AS l
WHERE NOT EXISTS (SELECT *
                  FROM production.operator AS o
                  WHERE o.locker_number = l.locker_number);
```

locker_number	location
L-107	Locker room, row 2

The insert of L-107 is accepted, and only a query like that of Table 6.4 finds

it. A business that changed its mind about the second question, deciding instead that an operator might be given a second locker for tools, would reverse the design: the locker end could become many, and the foreign key would belong in `locker`, at the price of a join for the everyday question.

6.3.4. Many-to-many associations and association classes

Figure 6.6 shows the association of warehouses and products from Figure 5.7, with its association class `StockLevel`.

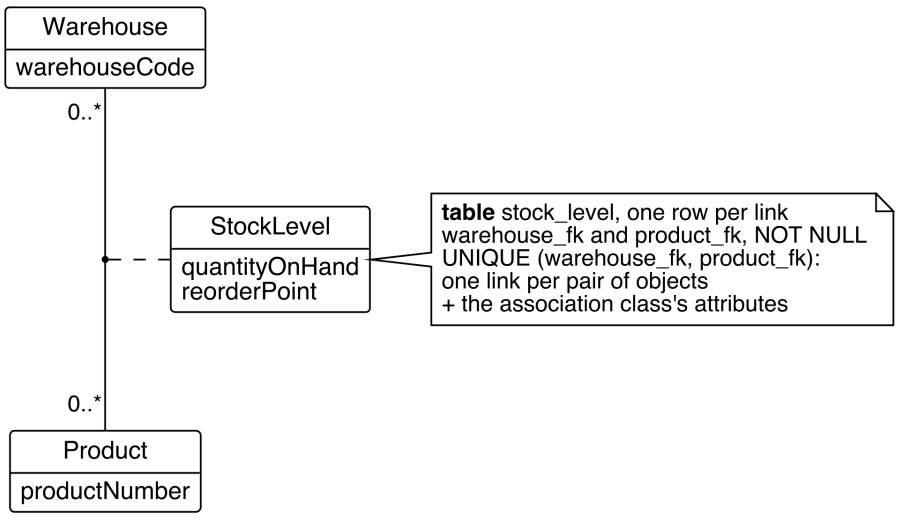


Figure 6.6.: Mapping an Association Class

Both ends of Figure 6.6 are `0..*`, so an object at either end may have many partners, and neither end's table can hold its links as a single value. A many-to-many association therefore maps to a table of its own, whose rows are the links: each row holds two foreign keys, one to each end. The association class supplies the rest of the row, its attributes, which is why the

note sits on `StockLevel`: the association and its class map together to the `stock_level` table, with `warehouse_fk`, `product_fk`, `quantity_on_hand`, and `reorder_point`. Recall Figure 5.6: each line there, with its quantity, is now one row.

The rule of 1, one link per pair of objects, also maps, as the note's `UNIQUE` line says. Two rows of `stock_level` with the same warehouse and product would be two links between one pair, so the pair of foreign keys is a key of the table: `UNIQUE (warehouse_fk, product_fk)`. The database then enforces the rule. An attempt to add a second stock level for the HT-100 at Fayetteville, at stage D4, fails:

```
INSERT INTO distribution.stock_level
    (warehouse_fk, product_fk,
     quantity_on_hand, reorder_point)
VALUES (1, 1, 5, 10);
```

```
ERROR:  duplicate key value violates unique constraint
 ↪  "stock_level_warehouse_fk_product_fk_key"
DETAIL:  Key (warehouse_fk, product_fk)=(1, 1) already
 ↪  exists.
```

Thus, an association class maps to a table whose foreign keys, taken together, are a key. In the book's database that table also has a surrogate key, `stock_level_pk`, and the pair of foreign keys is an alternate key; Section 6.6 explains why. Listing 6.7 gives the resulting relational schema of the three tables.

Listing 6.7 The Relational Schema of the Association Class

```
product (product_pk, product_number, description,
         list_price)
    primary key: product_pk
    alternate key: product_number

warehouse (warehouse_pk, warehouse_code, city, state)
    primary key: warehouse_pk
    alternate key: warehouse_code

stock_level (stock_level_pk, warehouse_fk, product_fk,
             quantity_on_hand, reorder_point)
    primary key: stock_level_pk
    alternate key: (warehouse_fk, product_fk)
    warehouse_fk references warehouse_pk in warehouse,
        must not be NULL
    product_fk references product_pk in product,
        must not be NULL
```

The two classes of Figure 6.6 map to `product` and `warehouse`, which the association leaves unchanged, and the association with its class maps to `stock_level`: its two foreign keys are the two ends of each line of Figure 5.6, its alternate key is the rule of one link per pair, and its other columns are the class's attributes.

6.3.5. Recursive associations

A recursive association maps by the same rules as any other; its ends happen to attach to the same class, or to a class and one of its subclasses. Only where the rules end up is new. In a one-to-many recursive association,

both ends are the same table, so the table's foreign key refers to the table itself. This subsection maps one of each kind, a one-to-many association first.

Example 6.2 (Mapping each operator's lead)**.** *The system.* The production schema of stage P3 holds the plant's six operators. Suppose the plant organizes them into crews: each operator reports to at most one lead operator, and a lead may lead any number of operators. No operator is the operator's own lead. Robert Hayes leads the plant and reports to no one. Figure 6.7 models the rule as an association of `Operator` with itself, with a role name at each end: the `0..1` at the `lead` end says that each operator has at most one lead, and the `0..*` at the `crewMember` end that a lead has any number of crew members.

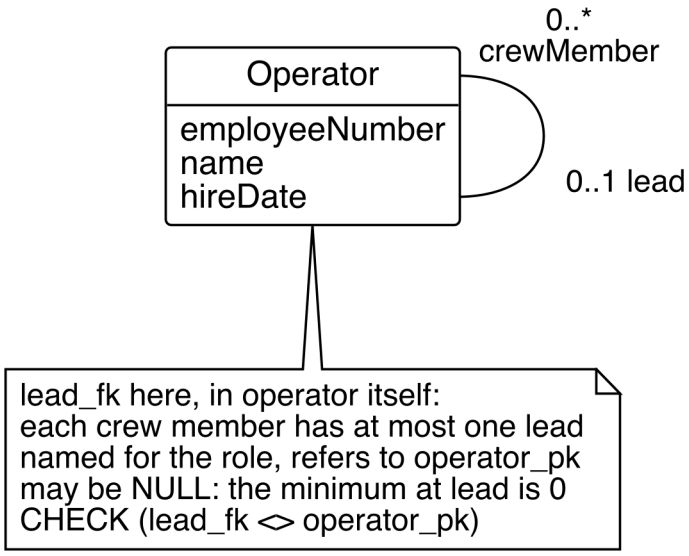


Figure 6.7.: Mapping a Recursive One-to-Many Association (Hypothetical Leads)

The request. Map the association so that the database records who leads whom.

The design. Both ends of Figure 6.7 attach to one class, so both ends are the `operator` table, and the rule that the foreign key goes in the table of the many end no longer says enough. The question becomes which rows hold the foreign key, and whose key they hold. A cloud instance diagram answers it, as Figure 5.13 did for the bill of materials, by drawing the class once for each role. Figure 6.8 draws the six operators twice: on the left as crew members, on the right as leads. Each line is one link, from a crew member to the crew member's lead.

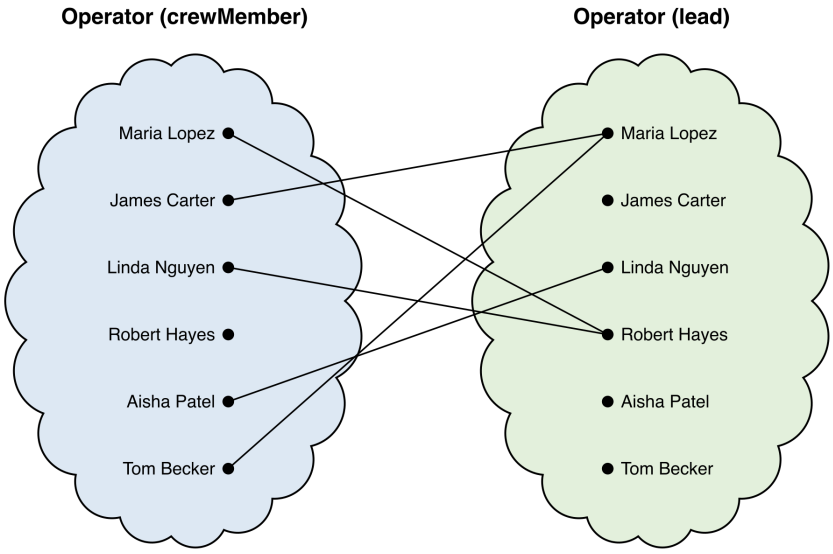


Figure 6.8.: The Six Operators, Drawn Once as Crew Members and Once as Leads (Hypothetical Leads)

Read Figure 6.8 from the crew-member cloud. Every dot has at most one line: James Carter's goes to Maria Lopez, Maria Lopez's to Robert Hayes,

and Robert Hayes's dot has none. Each operator's link can therefore be recorded as one value in that operator's own row, the row of the crew member. The row is the crew member's, and the value it holds is the lead's key: the rule of one-to-many, read once per role. Read the other cloud, and the dots have any number of lines: Robert Hayes has two, and James Carter none. A lead's row cannot hold its crew, just as a customer's row cannot hold its orders.

Four details complete the mapping. The foreign key is named for the role of the operator it refers to, `lead_fk`, since a column named `operator_fk` would not say which of the two operators it means, and `crew_fk` would name the wrong one. It refers to `operator_pk` in its own table. The `0..1` at the `lead` end lets it be NULL, for the operator with no lead. A `CHECK` that `lead_fk` differs from `operator_pk` implements the rule that no operator is the operator's own lead, which the class model states only in words. And by step 6 the foreign key refuses a deletion: a lead's row cannot be deleted while any row's `lead_fk` holds its key. Listing 6.8 gives the relational schema; the column is hypothetical, since the book's database records no leads.

Listing 6.8 The Operator Table with Each Operator's Lead (Hypothetical)

```
operator (operator_pk, employee_number, name, hire_date,
          lead_fk)
    primary key: operator_pk
    alternate key: employee_number
    lead_fk references operator_pk in operator,
        may be NULL
```

The result. With the column of Listing 6.8 added, each operator's row holds the key of its lead:

Table 6.5.: Each Operator's Row with Its Lead's Key (Hypothetical Leads)

```
SELECT operator_pk, employee_number, name, lead_fk
FROM production.operator
ORDER BY operator_pk;
```

operator_pk	employee_number	name	lead_fk
1	E-201	Maria Lopez	4
2	E-202	James Carter	1
3	E-203	Linda Nguyen	4
4	E-204	Robert Hayes	NULL
5	E-205	Aisha Patel	3
6	E-206	Tom Becker	1

A key means nothing to a reader, so a query that shows each operator beside the lead joins `operator` to itself, a self-join (Section 3.6), once as the crew member and once as the lead. A left join (Section 3.7) keeps the operator who has no lead:

Table 6.6.: Each Operator and the Operator's Lead (Hypothetical Leads)

```
SELECT o.name AS crew_member, l.name AS lead_name
FROM production.operator AS o
LEFT JOIN production.operator AS l
  ON l.operator_pk = o.lead_fk
ORDER BY o.employee_number;
```

crew_member	lead_name
Maria Lopez	Robert Hayes
James Carter	Maria Lopez
Linda Nguyen	Robert Hayes
Robert Hayes	NULL
Aisha Patel	Linda Nguyen
Tom Becker	Maria Lopez

The check. The table has six operators, so Table 6.5 has six rows, and Table 6.6 one for each of them. Figure 6.8 has five lines, and Table 6.5 has five values of `lead_fk`, one per line; the sixth row, Robert Hayes's, is NULL, his dot with no line.

Interpretation. Robert Hayes leads Maria Lopez and Linda Nguyen, who lead the rest of the plant's operators. A lead's crew is found from the other side, as the rows whose `lead_fk` is the lead's key, just as a customer's orders are found as the rows whose `customer_fk` is the customer's key.

Where it breaks. The `CHECK` refuses an operator as the operator's own lead:

```
UPDATE production.operator
SET lead_fk = operator_pk
WHERE employee_number = 'E-204';
```

```
ERROR:  new row for relation "operator" violates check
 ↪  constraint "operator_check"
DETAIL:  Failing row contains (4, E-204, Robert Hayes,
 ↪  2019-10-07, 4).
```

A longer loop is accepted, however. Were Robert Hayes's lead set to James Carter, every row would still satisfy the foreign key and the `CHECK`, yet the chain of leads would never end, and a recursive query that follows it, like the one of Section 4.6, would never end either, unless a bound on its depth or a `CYCLE` clause stopped it. That the leads form a hierarchy is a rule the tables leave to a checking query.

The bill of materials is the other kind. Figure 6.9 shows it, a recursive association between an assembly and its components, with its association class `BomLine`.

Its ends attach to a class and its subclass. A part may be a component of many assemblies, so, unlike an operator's lead, a part's links cannot be recorded as one value in its own row. The association in Figure 6.9 is many-to-many with an association class, so it maps to a table of its own, `bom_line`, with two foreign keys and the attribute `quantity_per`. Both foreign keys refer to parts, so they cannot both be named `part_fk`; the note names them for the roles at the ends of the line, `assembly_fk` and `component_fk`, which are the two role clouds of Figure 5.13. The pair is `UNIQUE`, as for any association class, and a `CHECK` that the two differ stops a part from being its own component, a rule the class model states only in words. The complete relational schema of Listing 6.9 shows both constraints.

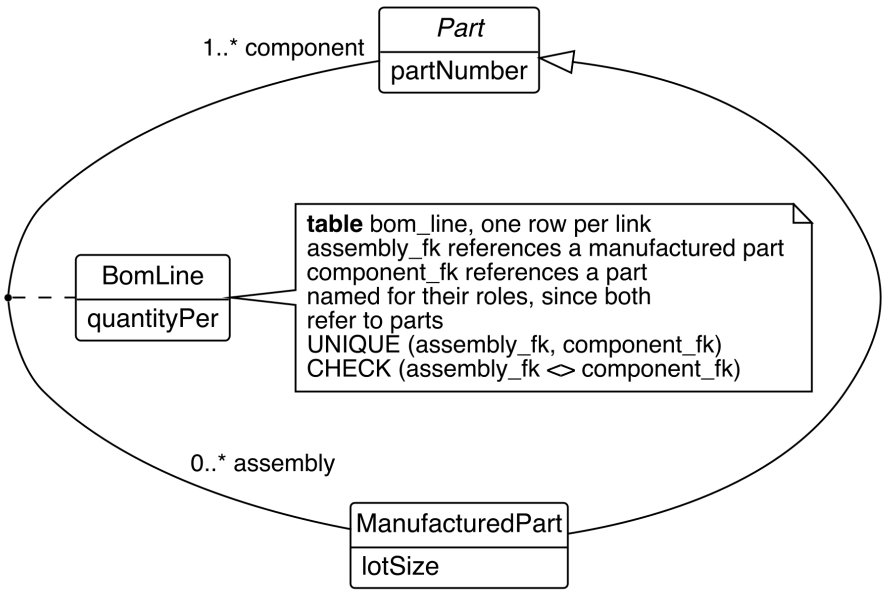


Figure 6.9.: Mapping a Recursive Association with an Association Class

Listing 6.9 The Complete Relational Schema of the Bill of Materials

```
bom_line
    bom_line_pk   INTEGER       not NULL, generated
    assembly_fk   INTEGER       not NULL
    component_fk  INTEGER       not NULL
    quantity_per  NUMERIC(8,3)  not NULL
    primary key: bom_line_pk
    alternate key: (assembly_fk, component_fk)
    assembly_fk references part_fk in manufactured_part,
        must not be NULL
    component_fk references part_pk in part,
        must not be NULL
    assembly_fk: on delete no action, on update no action
    component_fk: on delete no action, on update no action
    check: quantity_per > 0
    check: assembly_fk <> component_fk
```

Notice that the two foreign keys of Listing 6.9 refer to different tables, although both lead to parts. `assembly_fk` refers to `manufactured_part`, since the `assembly` end of Figure 6.9 attaches to `ManufacturedPart`, and `component_fk` refers to `part`. The generalization between the two, mapped in Section 6.5, is what makes both refer to the same parts.

The two recursive associations of this section thus map by the rule of Section 6.3, asked of a class drawn once per role: where can a link be recorded as one value? An operator has at most one lead, so the link fits in the operator's own row, and `operator` refers to itself. A part has many assemblies, so its links need a table of their own, with two foreign keys to parts.

6.3.6. Compositions and aggregations

Figure 6.10 shows the composition of an order's lines in the order, from Figure 5.11.

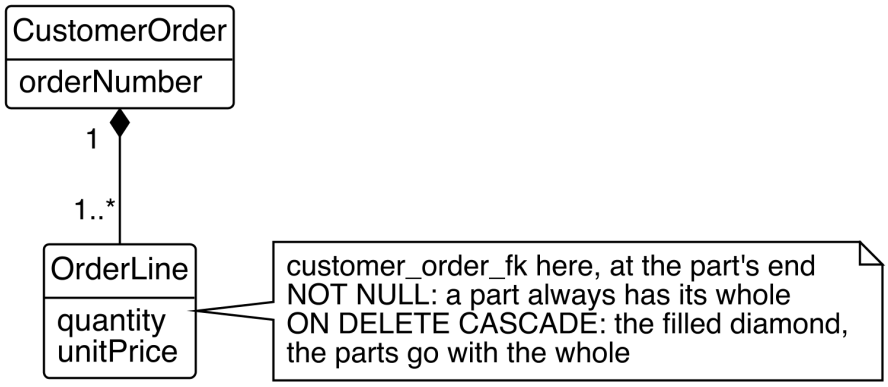


Figure 6.10.: Mapping a Composition

A composition is an association, so it maps by its multiplicities first: the `1` at the `CustomerOrder` end and the `1..*` at the `OrderLine` end make it one-to-many, with the foreign key `customer_order_fk` in `order_line`, `NOT NULL`. The filled diamond adds step 6 of Figure 6.1: since a part cannot outlive its whole, deleting an order deletes its lines, and the foreign key is declared `ON DELETE CASCADE`, as the note says. Table 2.6 listed cascading as one of the responses a designer may choose; composition is the case that calls for it. Listing 6.10 gives the complete relational schema of the `order_line` table as the book's database holds it at stage D3.

Listing 6.10 The Complete Relational Schema of the Order Line Table

```
order_line
    order_line_pk      INTEGER        not NULL, generated
    customer_order_fk  INTEGER        not NULL
    line_number        INTEGER        not NULL
    product_fk         INTEGER        not NULL
    quantity           INTEGER        not NULL
    unit_price         NUMERIC(10,2)  not NULL
    primary key: order_line_pk
    alternate key: (customer_order_fk, line_number)
    customer_order_fk references customer_order_pk in
        customer_order, must not be NULL
    product_fk references product_pk in product,
        must not be NULL
    customer_order_fk: on delete cascade, on update
        no action
    product_fk: on delete no action, on update no action
    check: line_number > 0
    check: quantity > 0
    check: unit_price > 0
```

The composition of Figure 6.10 is the line `customer_order_fk: on delete cascade`. The other foreign key, to `product`, maps an ordinary association and refuses, which Example 6.3 interprets. An aggregation, the hollow diamond, maps like an ordinary association, and its foreign key refuses, since its parts outlive the whole: `machine.work_center_fk` refuses the deletion of a work center that still has machines.

6.3.7. Ternary associations

A ternary association maps to a table with three foreign keys. Each link of Figure 5.14 connects an operator, a machine, and a work order, so each row holds three keys, and the attribute of the association class `Run` of Figure 5.15, the hours, joins them. Which columns form a key depends on the multiplicities. With `0..*` at every end, as in Figure 5.15, a link can be recorded only in a table of its own, as for a many-to-many association, and the three foreign keys together are its key. The runs are hypothetical, so Listing 6.11 is written out by hand.

Listing 6.11 The Relational Schema of the Ternary Association (Hypothetical)

```
run (run_pk, operator_fk, machine_fk, work_order_fk, hours)
    primary key: run_pk
    alternate key: (operator_fk, machine_fk, work_order_fk)
    operator_fk references operator_pk in operator,
        must not be NULL
    machine_fk references machine_pk in machine,
        must not be NULL
    work_order_fk references work_order_pk in work_order,
        must not be NULL
```

Each ring of Figure 5.14 is one row of `run`, and its three lines are the row's three foreign keys. The alternate key of the three says that one association holds at most one link among a given operator, machine, and work order. A plant that records several runs of the same three on different days would add the date to the key, or promote the run to a class of its own, as the review of Example 5.3 suggested. The class `Run` then maps by step 1, and its association with each of the three classes by the one-to-many rule. That

gives the same columns with no key over the three foreign keys, so the table holds any number of runs of the same three.

The following example maps a whole model and checks the result against it.

Example 6.3 (Mapping the refined order model of stage D3)**.** *The system.* The class model of Figure 5.11: customers, their orders, the order lines composed in each order, and the products the lines are for.

The request. Produce the tables that implement the model, with their keys, foreign keys, and deletion rules.

Let's apply the rules one construct at a time, and record where each element of the model goes:

Table 6.7.: Mapping the Refined Order Model, Construct by Construct

Model construct	Rule	Result in the tables
Classes `Customer`, `CustomerOrder`, `OrderLine`, `Product`	a table each, with a surrogate key	`customer`, `customer_order`, `order_line`, `product`, each with a `_pk`
Business identifiers `customerNumber`, `orderNumber`, `productNumber`	a `UNIQUE` column	alternate keys
`requestedShipDate` `[0..1]`	a column that allows NULL	`requested_ship_date` may be NULL
The other attributes: `name`, `city`, `state`, `orderDate`, `description`, `listPrice`, `quantity`, `unitPrice`	a column each, `NOT NULL`	`name`, `city`, `state`, `order_date`, `description`, `list_price`, `quantity`, `unit_price`

Model construct	Rule	Result in the tables
`Customer` `1` to `0..*` `CustomerOrder`	a foreign key at the many end, `NOT NULL`	`customer_order.customer_fk`
`CustomerOrder` `1` to `1..*` `OrderLine`, a composition	a foreign key at the many end, `NOT NULL`, cascading	`order_line.customer_order_fk`, `ON DELETE CASCADE`
`OrderLine` `0..*` to `1` `Product`	a foreign key at the many end, `NOT NULL`	`order_line.product_fk`

The order line's `lineNumber` is left for Section 6.4, where it becomes part of a key. The relational schema of the four tables, as the book's database holds them at stage D3, is:

Listing 6.12 The Relational Schema of the Refined Order Model

```
customer (customer_pk, customer_number, name, city, state)
    primary key: customer_pk
    alternate key: customer_number

customer_order (customer_order_pk, order_number, order_date,
                requested_ship_date, customer_fk)
    primary key: customer_order_pk
    alternate key: order_number
    customer_fk references customer_pk in customer,
        must not be NULL

product (product_pk, product_number, description,
         list_price)
    primary key: product_pk
    alternate key: product_number

order_line (order_line_pk, customer_order_fk, line_number,
            product_fk, quantity, unit_price)
    primary key: order_line_pk
    alternate key: (customer_order_fk, line_number)
    customer_order_fk references customer_order_pk in
        customer_order, must not be NULL
    product_fk references product_pk in product,
        must not be NULL
```

The check. Every row of Table 6.7 appears in a line of Listing 6.12, and every column of the listing comes from a row of the ledger: the surrogate keys from its first row, the attributes from the next three, and the foreign keys from the last three. The one exception is `line_number`, which Section 6.4 accounts for. The deletion rules are in the complete

schema of Listing 6.10: the `on delete cascade` of `customer_order_fk` is the composition's diamond, and the `no action` of `product_fk` says that deleting a product that has been ordered is refused.

Interpretation. The two deletion rules say what the business can and cannot undo. An order can be canceled with its lines in one statement; a product that customers have ordered cannot be removed, since the order history refers to it. For example, order SO-5012 of stage D5 has two lines and no shipments:

Table 6.8.: The Lines of Order SO-5012

```
SELECT o.order_number, l.line_number,
       l.product_fk, l.quantity
FROM distribution.customer_order AS o
JOIN distribution.order_line AS l
  ON l.customer_order_fk = o.customer_order_pk
WHERE o.order_number = 'SO-5012';
```

order_number	line_number	product_fk	quantity
SO-5012	1	3	4
SO-5012	2	2	2

Both rows of Table 6.8 refer to the order through `customer_order_fk`. If the order is deleted, the cascade deletes these two order lines with it, and nothing else: the order's customer and the lines' products are untouched, since their foreign keys point the other way. Chapter 8 presents the `DELETE` statement and shows the cascade at work.

Where it breaks. The cascade stops at the order lines. An order that has been shipped has shipments that refer to it, and their foreign key refuses, so deleting a shipped order fails, as it should: a shipment is not part of an order, and its record must survive.

6.4. Mapping qualified associations

This section continues step 4 of Figure 6.1 with the qualified association, which becomes a composite key.

Figure 6.11 shows the order line qualified by its line number, from Figure 5.12.

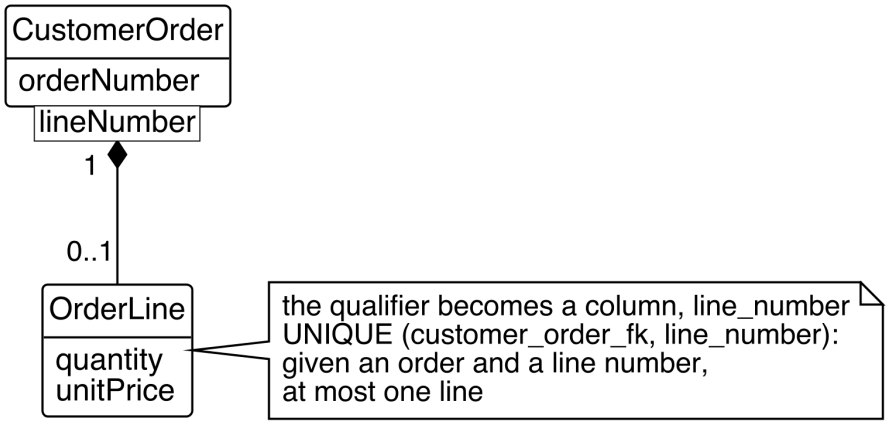


Figure 6.11.: Mapping a Qualified Association

Recall that a qualifier reduces the multiplicity at the far end: in Figure 6.11, given an order and a line number, there is at most one order line, the `0..1` at the `OrderLine` end. The qualifier's meaning is a uniqueness rule, and a table states uniqueness with a key. A qualified association therefore maps like the association without its qualifier, and the qualifier becomes a column that, with the foreign key to the qualifying class, forms a key. For the order line, the qualifier becomes the column `line_number` of `order_line`, and the key is `UNIQUE (customer_order_fk, line_number)`, as the note of Figure 6.11 says: the alternate key of Listing 6.10. The foreign key `customer_order_fk` is the one the composition already put in `order_line`;

the qualifier adds only its column and the key. Listing 6.13 gives the resulting relational schema of the order line.

Listing 6.13 The Relational Schema of the Qualified Order Line

```
order_line (order_line_pk, customer_order_fk, line_number,
            product_fk, quantity, unit_price)
    primary key: order_line_pk
    alternate key: (customer_order_fk, line_number)
    customer_order_fk references customer_order_pk in
        customer_order, must not be NULL
    product_fk references product_pk in product,
        must not be NULL
```

Read Listing 6.13 against Figure 6.11. The qualifier box `lineNumber` is the column `line_number`, and the `0..1` it produces at the `OrderLine` end is the alternate key `(customer_order_fk, line_number)`: with a given order and line number, at most one row. The rest of the listing comes from the other two associations of the order line, with its order and its product.

The key is what keeps the qualifier's meaning. Mapping the association without the qualifier's key, as a plain column, would let two lines of one order share a line number, which the model forbids. It is the order line's version of the rule of one link per pair: here, one line per pair of order and line number.

The same rule maps the routing of Figure 5.16. An operation is qualified by its operation number within its manufactured part, so `operation` has the column `operation_number` and the key `UNIQUE (manufactured_part_fk, operation_number)`; operation 10 of the axle and operation 10 of the frame are different operations. The composition of the routing makes the same foreign key cascade, so an operation goes with

its part's routing. Listing 6.14 gives the complete relational schema of the `operation` table of stage P3.

Listing 6.14 The Complete Relational Schema of the Operation Table

```
operation
    operation_pk           INTEGER       not NULL, generated
    manufactured_part_fk   INTEGER       not NULL
    operation_number       INTEGER       not NULL
    description            VARCHAR(40)   not NULL
    setup_minutes          NUMERIC(6,1)  not NULL
    run_minutes_per_unit   NUMERIC(6,2)  not NULL
    work_center_fk         INTEGER       not NULL
    primary key: operation_pk
    alternate key: (manufactured_part_fk, operation_number)
    manufactured_part_fk references part_fk in
        manufactured_part, must not be NULL
    work_center_fk references work_center_pk in work_center,
        must not be NULL
    manufactured_part_fk: on delete cascade, on update
        no action
    work_center_fk: on delete no action, on update no action
    check: operation_number > 0
    check: setup_minutes >= 0
    check: run_minutes_per_unit > 0
```

The qualifier of Figure 5.16 is again an alternate key, `(manufactured_part_fk, operation_number)`, and the filled diamond is again `on delete cascade`. The foreign key `work_center_fk` maps the ordinary association of an operation with the work center that performs it, and it refuses: a work center that some routing uses cannot be deleted.

A qualifier often looks like an identifier, and it is one, but only within its

qualifying object. A line number, an operation number, a bin code within a warehouse: none is unique alone, and none should be made `UNIQUE` alone. The test is the one Section 2.2 applied to candidate keys: ask whether the business promises uniqueness for the qualifier alone, or only within something.

An `{ordered}` end (1) maps the same way. The order becomes a column, a sequence number, in the table at the ordered end, and the foreign key with the sequence number is an alternate key, as if the association were qualified by the number. A query that wants the objects in order sorts by it; the rows themselves have no order (Section 2.1).

6.5. Mapping generalization

This section illustrates step 5 of Figure 6.1: how a superclass and its subclasses become tables.

Figure 6.12 shows the generalization of stage P1, in which every part is either manufactured or purchased (Figure 5.17), with notes on the mapping the book's database uses.

A generalization has no single obvious mapping, because a table has no notion of one kind of row being a kind of another. There are three ways to map it, and each keeps some of what the model says and gives up some. We map the parts of stage P1 all three ways.

6.5.1. A table for each class

The first mapping, the one of Figure 6.12 and of the book's database, gives each class a table of its own. The superclass table `part` holds the superclass's attributes, and each subclass table holds the subclass's own attributes and a key. The key is the link: a subclass table's primary key is also a foreign key to the superclass table, so a manufactured part and the

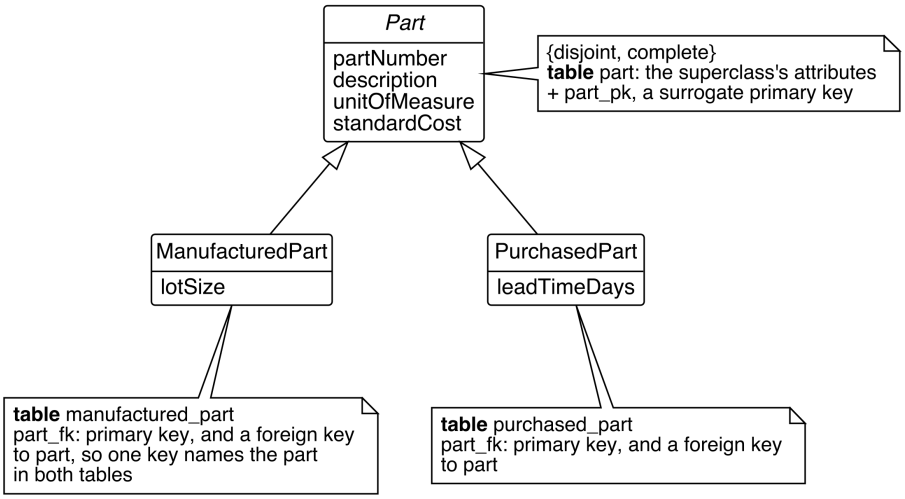


Figure 6.12.: Mapping a Generalization, Table per Class

part it is have the same key value in both tables. Listing 6.15 gives the resulting relational schema.

Listing 6.15 The Relational Schema of the Generalization, Table per Class

```
part (part_pk, part_number, description, unit_of_measure,
      standard_cost)
    primary key: part_pk
    alternate key: part_number

manufactured_part (part_fk, lot_size)
    primary key: part_fk
    part_fk references part_pk in part, must not be NULL

purchased_part (part_fk, lead_time_days)
    primary key: part_fk
    part_fk references part_pk in part, must not be NULL
```

Notice the name of the subclass tables' key, `part_fk`. It is the primary key of `manufactured_part` and a foreign key to `part` at once, and the book names it for what it refers to, so the link to the superclass is visible in the listing. The generalization's line of Figure 6.12 is that foreign key: every manufactured part is a part.

An object of a subclass is split across two tables, so a query that wants a whole manufactured part, its inherited attributes and its own, joins them on the shared key:

Table 6.9.: The Manufactured Parts, Whole

```
SELECT p.part_number, p.description, m.lot_size
FROM production.part AS p
JOIN production.manufactured_part AS m
  ON m.part_fk = p.part_pk
ORDER BY p.part_number;
```

part_number	description	lot_size
AX-10	Axle, 5/8 in	400
CF-310	Cart frame	80
FR-110	Hand truck frame	100
GC-300	Steel garden cart	40
GC-400	Poly garden cart	20
HT-100	Two-wheel hand truck	50
HT-200	Convertible hand truck	25
WA-10	Wheel assembly, 10 in	200

The table-per-class mapping keeps what the model says about associations. An association attached to a subclass maps to a foreign key that refers to the subclass's table, and one attached to the superclass to the superclass's table. Recall from Listing 6.9 that `assembly_fk` refers to `manufactured_part` and `component_fk` to `part`: the database itself enforces that only a manufactured part can be an assembly, while any part can be a component.

What the mapping does not keep is `{disjoint, complete}`. Nothing in Listing 6.15 stops a part from having a row in both subclass tables, which would break *disjoint*, or in neither, which would break *complete*. A query can find such parts, using the `EXISTS` of Section 3.8:

Table 6.10.: Parts That Break Disjoint or Complete

```
SELECT p.part_number
FROM production.part AS p
WHERE (EXISTS (SELECT *
               FROM production.manufactured_part AS m
               WHERE m.part_fk = p.part_pk)
       AND EXISTS (SELECT *
                   FROM production.purchased_part AS u
                   WHERE u.part_fk = p.part_pk))
   OR (NOT EXISTS (SELECT *
                   FROM production.manufactured_part AS m
                   WHERE m.part_fk = p.part_pk)
       AND NOT EXISTS (SELECT *
                       FROM production.purchased_part AS u
                       WHERE u.part_fk = p.part_pk));
```

part_number

Table 6.10 is empty: every part of stage P1 is in exactly one subclass table. The query does not prevent a violation; it finds one, and it is the kind of check that step 7 of Figure 6.1 runs, and that the book's own tests run on the database.

6.5.2. A table for each subclass only

The second mapping drops the superclass table and repeats its attributes in each subclass table. Since the book's database does not use it, Listing 6.16 is written out by hand.

Listing 6.16 The Generalization Mapped to Subclass Tables Only (Hypothetical)

```
manufactured_part (manufactured_part_pk, part_number,
                   description, unit_of_measure,
                   standard_cost, lot_size)
    primary key: manufactured_part_pk
    alternate key: part_number

purchased_part (purchased_part_pk, part_number, description,
                unit_of_measure, standard_cost,
                lead_time_days)
    primary key: purchased_part_pk
    alternate key: part_number
```

Each whole object is now one row, with no join, and a part cannot be in two subclasses unless it is entered twice. But the mapping loses the superclass. `part_number` is unique within each table, not across both, so a manufactured part and a purchased part could share a number. Worse, an association attached to the superclass has nowhere to go: `component_fk` must refer to any part, and a foreign key refers to one table. The bill of materials cannot be mapped at all. This mapping suits a superclass that no association and no query treats as a whole, which is rare in industrial data.

6.5.3. One table for the whole hierarchy

The third mapping puts every class's attributes in one table, with a column that says which subclass each row belongs to. Such a column is called a **discriminator**. Listing 6.17 writes it out by hand.

Listing 6.17 The Generalization Mapped to One Table (Hypothetical)

```
part (part_pk, part_number, description, unit_of_measure,
      standard_cost, part_kind, lot_size, lead_time_days)
    primary key: part_pk
    alternate key: part_number
    may be NULL: lot_size, lead_time_days
    check: part_kind IN ('manufactured', 'purchased')
    check: (part_kind = 'manufactured'
            AND lot_size IS NOT NULL
            AND lead_time_days IS NULL)
        OR (part_kind = 'purchased'
            AND lead_time_days IS NOT NULL
            AND lot_size IS NULL)
```

Here *disjoint* and *complete* are enforced, and by a single column: each row has exactly one `part_kind`, and the first check allows only the two subclasses. The second check ties each kind to its own columns. The cost is in the NULLs, since every row leaves the other kind's columns empty, and in the associations: `assembly_fk` can refer only to `part`, so the database can no longer stop a purchased part from being an assembly. You may read in older material that this mapping violates a normal form. It does not; every column still depends on the key. What it gives up is the ability of foreign keys to tell the subclasses apart.

Table 6.11 compares the three mappings.

Table 6.11.: The Three Mappings of Generalization Compared

	Table per class	Subclass tables only	One table
Whole object in one row	no, a join	yes	yes
Enforces *disjoint* and *complete*	no, a query checks	partly	yes, with checks
Foreign keys to the superclass	yes	no	yes
Foreign keys to one subclass	yes	yes	no
NULLs for columns that do not apply	none	none	many

The book maps every generalization a table per class, because the rows of Table 6.11 about foreign keys decide it for HG Industries: the bill of materials needs a foreign key to the superclass and one to a subclass at once. You should decide each generalization by the same test. Ask which associations attach to the superclass and which to the subclasses, and choose the mapping that lets the database enforce them.

6.6. Choosing and naming keys

This section returns to step 3 of Figure 6.1, a table's identity, to weigh surrogate keys against natural keys and to state the book's naming conventions.

A table's primary key can come from the data or be invented for the table. A **natural key**, one made of the class's own attributes, uses what the business already has: a customer's number, a part's number. A surrogate key, as Section 2.2 defined it, is invented: PostgreSQL generates it and it means nothing outside the database. Blaha and Premerlani call the two *value-based* and *existence-based* identity (Blaha and Premerlani 1998), and both work. Choosing between them is a design decision, made table by table, and one schema may use both. A surrogate key is a sound default, for four reasons:

1. A surrogate key never changes. A natural key can: a customer renumbered after a merger would need every order's foreign key changed with it. Recall from Section 2.3 that PostgreSQL refuses even to update a generated key.
2. A surrogate key is one column. A natural key of a dependent class grows as the dependence deepens: an order line's natural key is its order's key and its line number, and a shipment line's natural key would repeat all of that.
3. Every foreign key is then one column of the same type, named in the same way, which makes joins uniform to write and to read.
4. A surrogate key does not depend on the business's identifiers being right. A customer entered with a mistyped number can be corrected without touching any reference to it.

A surrogate key does not replace the natural key; it adds to it. The business's identifier is still a candidate key, and without its `UNIQUE` constraint the table would accept two rows for customer C-1001, each with its own surrogate key, and nothing would notice. Thus, whenever you give a table a surrogate key, you should declare its natural keys as alternate keys, as every table of the book's database does.

A natural key earns its place, as a primary key or as the column a foreign key refers to, when the reasons above are weak for it: the business assigns it, it never changes and is never reused, it is short and one column, and people read it. The two-letter state codes of the `customer` table, the units

of measure of parts, and the locker numbers of Example 6.1 qualify; a customer number that a merger might change does not. A foreign key may refer to any column declared `UNIQUE`, not only to a primary key, so a table can keep a surrogate primary key and still let other tables refer to its natural key. The referring row then carries an identifier that people read, and a question that wants only that identifier needs no join, as Table 6.3 showed. Its price is the first reason above: if the identifier ever changes, every referring row must change with it, which cascading the update (Section 2.3) can do but which a design should not have to rely on.

Many databases therefore mix the two kinds: surrogate keys for the business's main things, such as customers, orders, and parts, and natural keys for codes and for stable identifiers that people read. The book's database uses surrogate keys throughout, and stores codes such as states in the referring table itself rather than in a table of their own; Exercise 6.11 asks when a table of codes is worth having. That is one consistent policy, chosen so that every example follows the same rules. It is not the only sound one, and you should choose the keys of your own designs by the reasons of this section.

The names of keys follow five rules, which make the book's schema readable without its diagram. Like the choice of surrogate keys, they are a convention: teams adopt their own, and every convention has exceptions, which a design should state when it makes one.

- A table's surrogate key is named for the table with the suffix `_pk`: `customer_pk`, `order_line_pk`.
- A foreign key is named for the table it refers to, with the suffix `_fk`: `customer_fk` refers to `customer_pk`.
- When a table refers to another twice, or refers to its own class, each foreign key is named for its role instead: `assembly_fk` and `component_fk` in `bom_line`.
- A subclass table's key, both its primary key and its reference to the superclass, is named for the superclass, `part_fk`, and a table

that refers to the subclass names its foreign key for the subclass, `manufactured_part_fk`.
- A foreign key that refers to a natural key keeps that key's name, since it holds the key's value: `locker_number` in Listing 6.6.

Surrogate keys have a further cost, which the following example makes concrete.

Example 6.4 (Seeing what single-column foreign keys cannot enforce)**.** *The system.* The shipments of stage D5. A shipment is made for one order, and each shipment line ships a quantity of one order line.

The request. Every shipment line must ship a line of its own shipment's order: shipment SH-9004, made for order SO-5004, must not ship a line of order SO-5001. Can the tables enforce that rule?

The rule involves two paths from a shipment line to an order: through its shipment (`shipment_line.shipment_fk` to `shipment.customer_order_fk`) and through its order line (`shipment_line.order_line_fk` to `order_line.customer_order_fk`). The rule says the two paths must reach the same order. With surrogate keys, each foreign key of `shipment_line` is one number, and nothing in the tables compares where the two paths lead. A query can check the rule by joining along both paths:

Table 6.12.: Shipment Lines That Ship Another Order's Line

```
SELECT sl.shipment_line_pk, s.shipment_number,
       l.customer_order_fk AS line_order,
       s.customer_order_fk AS shipment_order
FROM distribution.shipment_line AS sl
JOIN distribution.shipment AS s
  ON s.shipment_pk = sl.shipment_fk
JOIN distribution.order_line AS l
  ON l.order_line_pk = sl.order_line_fk
WHERE l.customer_order_fk <> s.customer_order_fk;
```

shipment_line_pk	shipment_number	line_order	shipment_order

Table 6.12 is empty, so the data of stage D5 obeys the rule, but only because it was loaded correctly: the database would have accepted a line that broke it.

Composite foreign keys can enforce the rule. If a shipment line holds its order's key itself, one foreign key can pair it with the shipment and another with the order line. Natural keys supply such keys directly. With value-based identity, an order line's key is its order and line number, so a shipment line that refers to an order line must hold the order's key, and a shipment can declare its number unique together with its order's key. Listing 6.18 writes the two tables out by hand.

Listing 6.18 Shipment Lines with Composite Foreign Keys (Hypothetical)

```
shipment (shipment_number, customer_order_fk, shipped_on,
          carrier, warehouse_fk)
    primary key: shipment_number
    unique: (shipment_number, customer_order_fk)
    customer_order_fk references customer_order_pk in
        customer_order, must not be NULL
    warehouse_fk references warehouse_pk in warehouse,
        must not be NULL

shipment_line (shipment_number, customer_order_fk,
               line_number, quantity)
    primary key: (shipment_number, line_number)
    (shipment_number, customer_order_fk) references
        (shipment_number, customer_order_fk) in shipment
    (customer_order_fk, line_number) references
        (customer_order_fk, line_number) in order_line
```

The `unique:` line of `shipment` is not a key, since `shipment_number` alone identifies a shipment. PostgreSQL lets a foreign key refer only to columns declared `UNIQUE` or a primary key, so the pair is declared unique for the shipment line's foreign key to refer to.

In Listing 6.18, the shipment line holds one `customer_order_fk`, and both of its composite foreign keys use it. The first says the shipment was made for that order; the second says the order line belongs to that order. A shipment line that shipped another order's line would need two different values in one column, which is impossible. The rule is enforced by the structure of the keys.

The natural keys are a convenience here, not a requirement. The same two composite foreign keys work beside surrogate primary keys, if

`shipment_line` gains a `customer_order_fk` column that must not be NULL and `shipment` and `order_line` each declare `UNIQUE` the pair of columns that a foreign key refers to. The column must not be NULL because PostgreSQL does not check a composite foreign key when one of its columns is NULL; a line with no order would pass both foreign keys. That is the real cost of surrogate keys: they do not carry the order's key along, so a designer who wants the rule enforced must add a `NOT NULL` column and two constraints that the tables would not otherwise need.

The check. Table 6.12 joins along the two paths that the rule compares, one join per foreign key, and keeps only the rows where they disagree, so an empty result means that every shipment line passes.

Interpretation. The book keeps its surrogate keys, without the added column, and checks the rule with a query, a choice made for the reasons listed above. The example shows what the choice gives up, and a designer who judges this rule important enough can have the database enforce it with composite foreign keys. The tests that load the book's database run the same check on stage D5.

Where it breaks. A check query finds a violation after it has happened; it does not stop one. A program that records shipments must therefore apply the rule itself, as the programs of Chapter 10 can.

With the keys chosen and named, a mapping is complete: every table has its columns, its keys, its foreign keys, and their deletion rules. Chapter 7 presents the review of step 7 in full, and Chapter 8 writes the result as `CREATE TABLE` statements.

6.7. Summary

Mapping follows seven steps, from a reviewed class model to a complete relational schema. Classes, attributes, identity, associations,

generalization, deletion rules, and a check, in that order; a check that fails sends the designer back to the model.

Each class maps to a table, and each attribute to a column. The column's type and constraints implement the attribute's domain: `NOT NULL` from a multiplicity of one, `UNIQUE` from a business identifier, and `CHECK` for a domain narrower than its type. Each table gets a primary key the class model does not name.

An association maps according to where a link can be recorded as one value. A one-to-many association maps to a foreign key at the many end, `NOT NULL` when the one end's minimum is one; a one-to-one association to a `UNIQUE` foreign key; a many-to-many association, with its association class, to a table of its own whose foreign keys together are a key. A recursive association maps the same way, its foreign keys named for their roles, so a recursive one-to-many association gives a table a foreign key to itself. A composition's foreign key cascades a deletion.

The side of a one-to-one association is a design decision. When the minimums do not settle it, the dependent object, then the end that could become many, then the table users start from decide where the foreign key goes. One foreign key enforces one end fully and only the maximum of the other, and one that refers to an identifier people read saves the everyday question a join.

A qualifier maps to a composite key. The qualifier becomes a column that, with the foreign key to the qualifying class, is `UNIQUE`, which keeps the qualifier's meaning in the tables.

A generalization maps in one of three ways, each keeping part of the model. A table per class keeps every association but leaves `{disjoint, complete}` to a checking query; subclass tables alone lose the superclass; one table with a discriminator enforces the subclasses but not associations to one of them. The associations decide which to use.

A table's key is chosen, and a schema may mix surrogate and natural keys. A surrogate key never changes and is always one column,

which makes it a sound default; a natural key earns its place when it is stable, short, and read by people, and a foreign key may refer to it. Either way, the natural key's `UNIQUE` constraint is what stops duplicates. Keys are named for their tables, `_pk` and `_fk`, or for their roles. A rule comparing two paths to the same row needs composite foreign keys, which natural keys supply and surrogate keys need an added column for; without them, a query must check the rule.

Chapter 7 reviews the tables that a mapping produces, and Chapter 8 creates them.

6.8. Exercises

Exercise 6.1. Specify the domains of the `CustomerOrder` class of Figure 5.1 in the form of Table 6.2, and give the SQL type and constraints of each column. Then compare your result with the complete relational schema of Listing 2.2, and explain each difference.

Exercise 6.2. Map the `Shipment` class of the stage D5 class model of Figure 5.19, with its associations to `CustomerOrder` and `Warehouse` and its association with `OrderLine` (`Shipment` `0..*` to `1..*` `OrderLine`), whose association class is `ShipmentLine`, in the form of Table 6.7. Give the keys of the `shipment_line` table. Say which foreign keys cascade and which refuse by step 6 of Figure 6.1, and why.

Exercise 6.3. Map the association class `Certification` between `Operator` and `Machine` (Exercise 5.5).

a. Give the table, its columns, and its keys.
b. Explain which constraint enforces the rule of one certification per operator and machine.
c. If the business decided to keep a history of certifications, how would the class model change, and how would the table's keys change?

Exercise 6.4. The class model of Figure 5.16 has multiplicity `1` at the `WorkCenter` end of its aggregation with `Machine`, and the table `machine` has `work_center_fk` declared `NOT NULL`.

a. What does that say about a machine that has been bought but not yet installed, or taken out of service and stored?
b. Change the model so that such a machine can be recorded, and give the change to the table.
c. Should the foreign key cascade? Explain from the difference between aggregation and composition.

Exercise 6.5. Each machine may have at most one maintenance plan, and each plan belongs to exactly one machine (Exercise 5.17).

a. In which table does the foreign key go, and why?
b. Which two constraints does it carry, and what would go wrong without each?
c. Could the foreign key be placed in the other table instead? What would it cost?

Exercise 6.6. A trucking fleet tracks its toll transponders. The toll authority issues each transponder with a serial number printed on its case, and the fleet keeps no spares: each truck carries exactly one transponder, and each transponder is mounted in exactly one truck. When a truck is retired, its transponder is moved to the truck that replaces it. Trucks are identified by a fleet number painted on the cab, such as T-12.

a. Draw the association with its multiplicities. Do the minimums, or the first question of Section 6.3.3, decide where the foreign key goes? Why or why not?
b. The fleet manager says that a truck that runs in a second toll region may soon need a second transponder. Where does the foreign key go, and why?

c. Dispatch's most frequent question starts from a truck and wants the truck's own details (its model and driver) with the serial number of its transponder. Which table would the third question of Section 6.3.3 then point to, and does that change your choice in (b)?
d. Write the relational schema of the two tables for your choice, with the foreign key referring to an identifier people read. What does that choice cost, and which rule of the association can the schema not enforce?

Exercise 6.7. Map the bins of Exercise 5.7, qualified by bin code within a warehouse. Give the table, and explain why `bin_code` must not be declared `UNIQUE` by itself.

Exercise 6.8. Map the generalization of Figure 6.12 to one table, as in Listing 6.17.

a. Explain how the two `CHECK` constraints enforce *disjoint* and *complete*.
b. Which rule of the bill of materials can the database no longer enforce, and why?
c. A manufactured part's lot size is required. How does the single table enforce that, and how does the table-per-class mapping of Listing 6.15?

Exercise 6.9. The query of Table 6.10 finds parts that break *disjoint* or *complete*.

a. Explain each half of its condition, and say which rule each half checks.
b. Rewrite it as two queries, one for each rule, using `INTERSECT` for one and `EXCEPT` for the other (Section 3.9).

Exercise 6.10. HG Industries decides to record, for each machine, whether it is CNC or manual. A CNC machine has a controller model and a program memory in megabytes; a manual machine has neither. Certifications on CNC machines, and only on them, must record the controller version the

operator was trained on. Record that version in a second association class, `CncCertification`, between `Operator` and the subclass for CNC machines; the association class `Certification` between `Operator` and `Machine` stays as it is.

Model the change as a generalization of `Machine`, choose a mapping, and justify the choice with Table 6.11. Give the resulting relational schema.

Exercise 6.11. The `customer` table stores each customer's state as a two-letter code, and nothing stops a code such as `ZZ`.

a. Design a table of states with the code as its natural primary key, and change `customer` to refer to it. Give the relational schema.
b. Why is a natural key a sound choice here, when the book's database uses surrogate keys everywhere else?
c. What does the new table give the business besides a check on the code?

Exercise 6.12. A plant's access system records who opens which door. Each employee has an employee number, assigned at hiring and never reused, and carries a badge whose number is printed on it and recorded with the employee; a lost badge is replaced with a badge of a new number. Each employee belongs to a department, identified by a three-letter code such as `MNT` that the plant has used for twenty years. Each door has a number stenciled on it, such as D-14, and doors are never renumbered. Each entry in the access log records a badge reading at a door at a moment.

a. For each of the tables `employee`, `department`, `door`, and `access_entry`, choose a surrogate or a natural primary key, and justify each choice by the reasons of Section 6.6.
b. For each foreign key, say whether it should refer to a surrogate or to a natural key, and why.
c. Should `access_entry` record the badge number or the employee? What goes wrong with the other choice when a badge is replaced?

Exercise 6.13. HG Industries starts moving stock between its warehouses. A transfer moves a quantity of one product from one warehouse to another on a date.

a. Model the transfer, with its associations and multiplicities.
b. Map it to a table, naming its foreign keys by the rules of Section 6.6, and give the relational schema.
c. Add a constraint that stops a transfer from a warehouse to itself.

Exercise 6.14. Using Listing 6.18:

a. Suppose shipment SH-9004, made for order SO-5004, were given a line for line 1 of order SO-5001. Write the values the shipment line would need, and say which foreign key rejects it.
b. What does the natural-key design cost, compared with the surrogate keys of Listing 2.4? Name two costs.

The remaining exercises map the models of chapter 5's exercises outside HG Industries. Each figure is the instructor's model of that exercise, so that your mapping does not depend on your own answer to it. For each, give the relational schema of every table in the notation of Section 2.4, with its primary key, its alternate keys, and each foreign key with the table it refers to and whether it may be NULL; follow the book's naming rules (Section 6.6); and say, for each foreign key, what happens when the row it refers to is deleted.

Exercise 6.15. Map the tool crib of Exercise 5.18, drawn in Figure 6.13.

a. Give the relational schema of the three tables.
b. The note says a tool has at most one checkout without a return time. Can a `UNIQUE` constraint on `tool_fk` enforce it? What would it forbid?
c. PostgreSQL can enforce the rule with a unique index on `tool_fk` restricted to the rows `WHERE returned_at IS NULL`. Explain what that index allows and what it refuses.

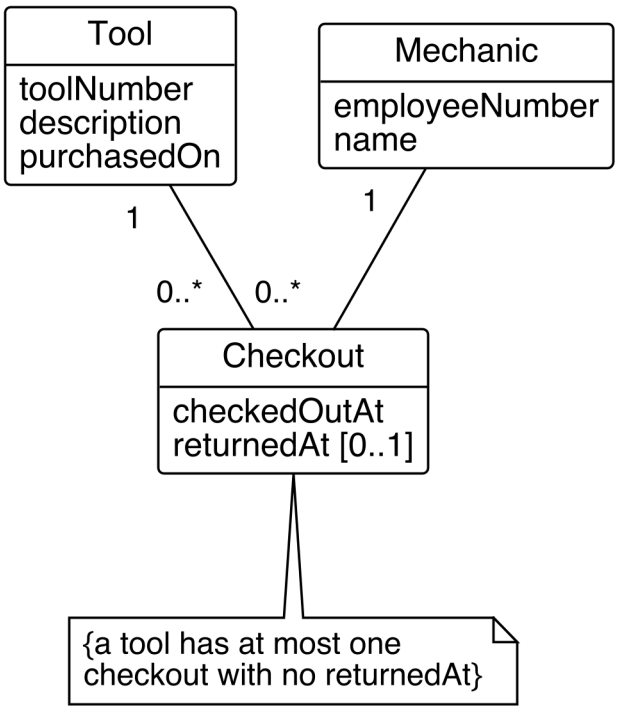


Figure 6.13.: The Tool Crib (Model for Exercise 5.18)

Exercise 6.16. Map the gauges and calibrations of Exercise 5.19, drawn in Figure 6.14.

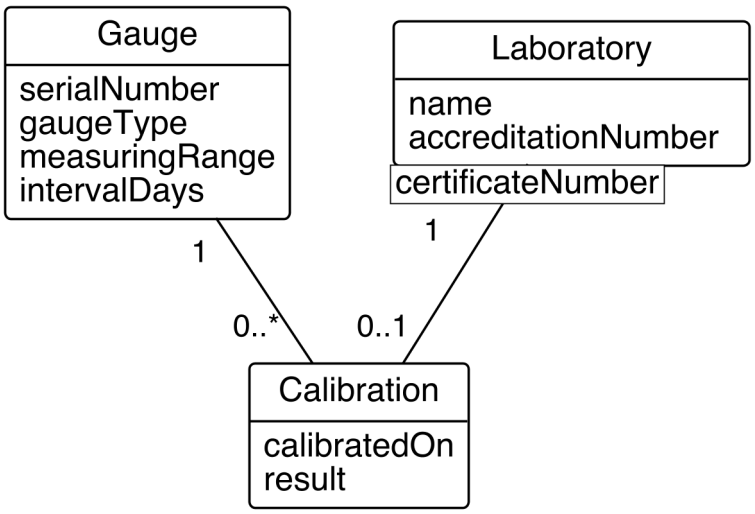


Figure 6.14.: Gauge Calibration (Model for Exercise 5.19)

a. Give the relational schema.
b. What does the qualifier on `certificateNumber` become? Why must `certificate_number` not be `UNIQUE` by itself?
c. A laboratory loses its accreditation and is to be removed from the database. What should happen to its calibrations, and which delete response gives that?

Exercise 6.17. Map the kanban loop of Exercise 5.20, drawn in Figure 6.15.

a. Give the relational schema. For the association of `Container` and `Card`, say in which table the foreign key goes and why (Section 6.3.2).

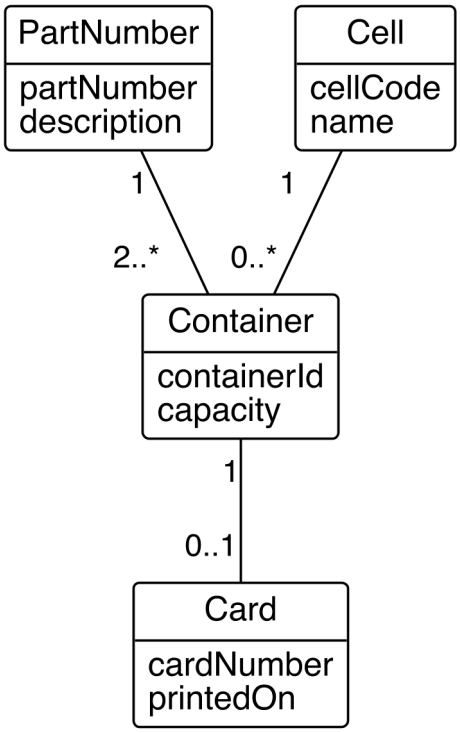


Figure 6.15.: The Kanban Loop (Model for Exercise 5.20)

b. Which multiplicity of the model can no table definition enforce? How would you check it with a query?

Exercise 6.18. Map the clinic of Exercise 5.21, drawn in Figure 6.16.

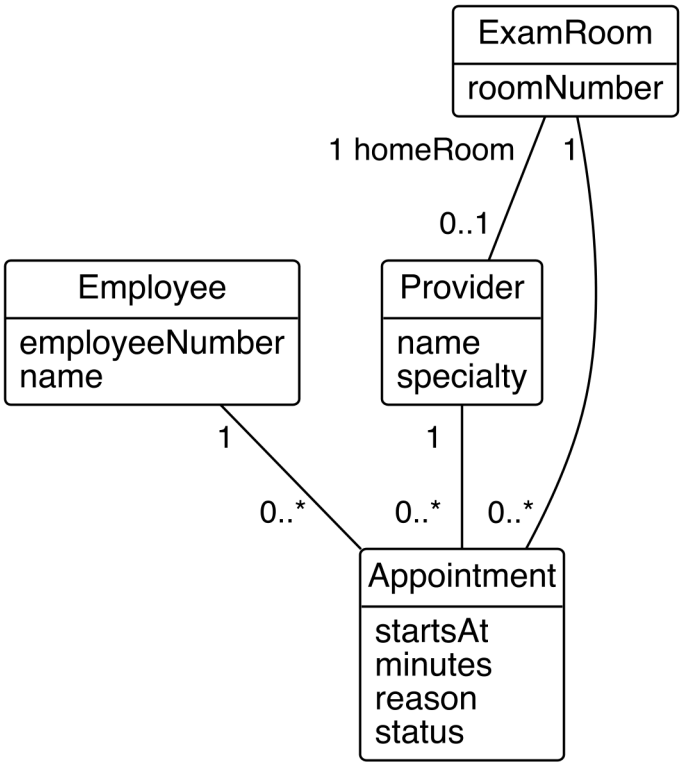


Figure 6.16.: Clinic Appointments (Model for Exercise 5.21)

a. Give the relational schema.
b. Two associations join `Provider` and `ExamRoom`, one directly and one through `Appointment`. Name the foreign keys each produces, and

explain why the home room's needs a `UNIQUE` constraint and the appointment's does not.

c. Write a `CHECK` constraint or explain why none can stop two appointments in the same room at overlapping times.

Exercise 6.19. Map the project network of Exercise 5.22, drawn in Figure 6.17.

a. Give the relational schema with surrogate keys, as the book does.
b. The model requires a precedence to join two activities of the same project. Show that your schema does not enforce it, with two rows that break it.
c. Give a second schema in which `activity` has the natural key `(project_fk, activity_id)` and `precedence` refers to both activities through it, sharing one `project_fk` column. Explain why this schema enforces the rule, and what it costs (recall Exercise 6.14).

Exercise 6.20. Map both models of the plant locations of Exercise 5.23, drawn in Figure 6.18.

a. Give the relational schema of the fixed levels, with the delete responses the compositions call for.
b. Give the relational schema of the recursive class. Which foreign key may be NULL, and which rows have NULL in it?
c. In the recursive table, what stops a line from being made the parent of a site? Propose a check the database can make, or say why it cannot.

Exercise 6.21. Map the bike share of Exercise 5.24, drawn in Figure 6.19.

a. Give the relational schema.
b. The association of `Dock` and `Bike` is `0..1` at both ends. Where does the foreign key go, what constraint makes it one-to-one, and why may it be NULL?

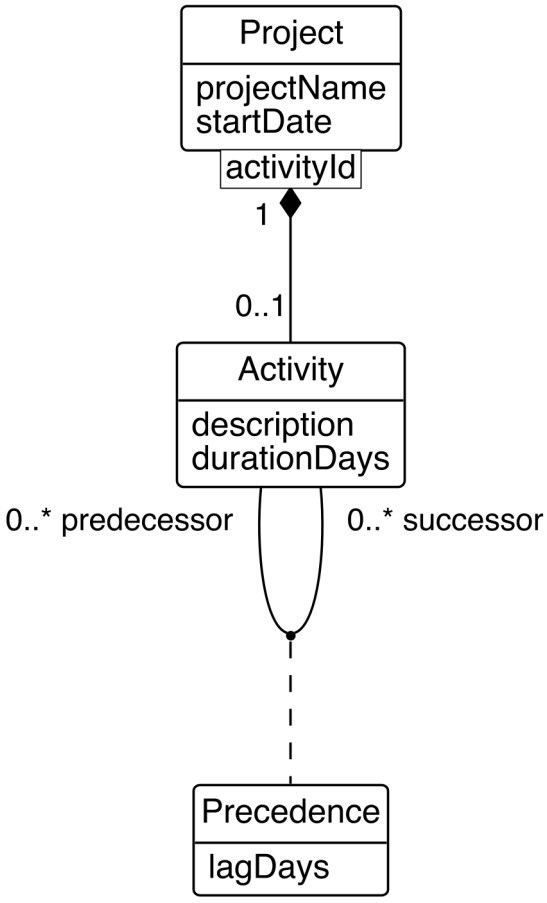


Figure 6.17.: The Project Network (Model for Exercise 5.22)

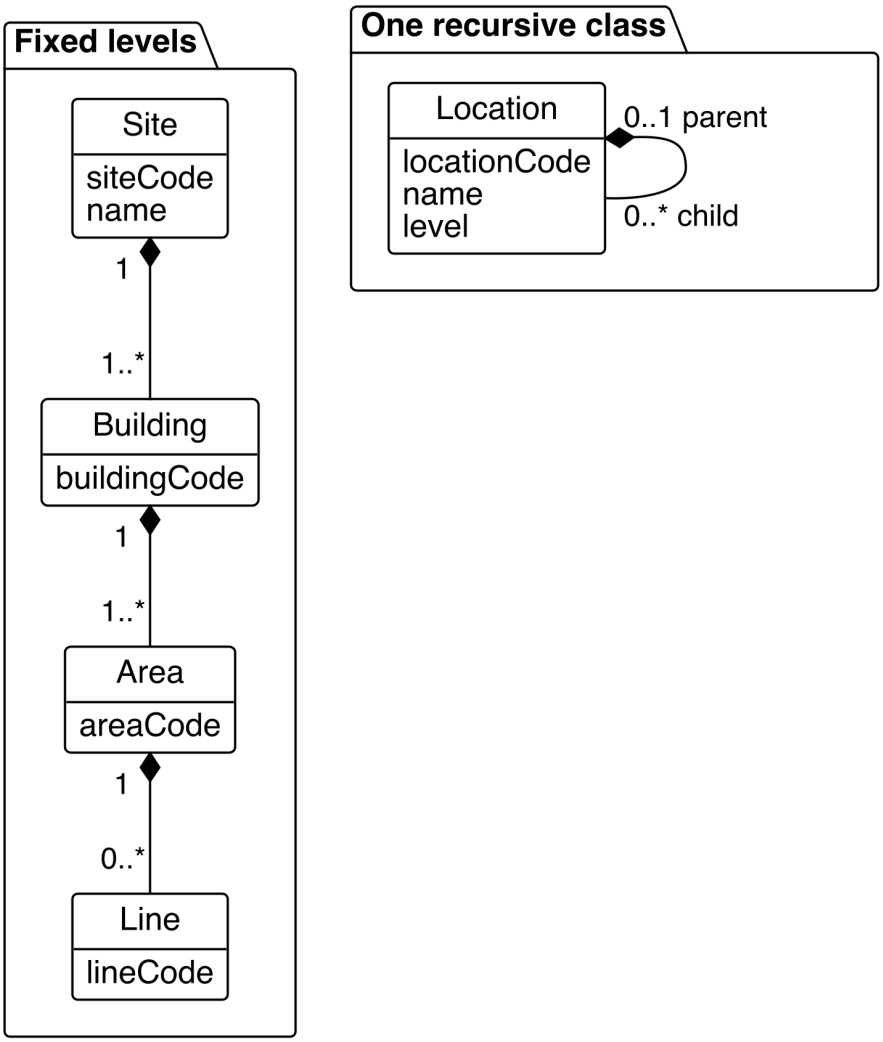


Figure 6.18.: Plant Locations, Two Ways (Model for Exercise 5.23)

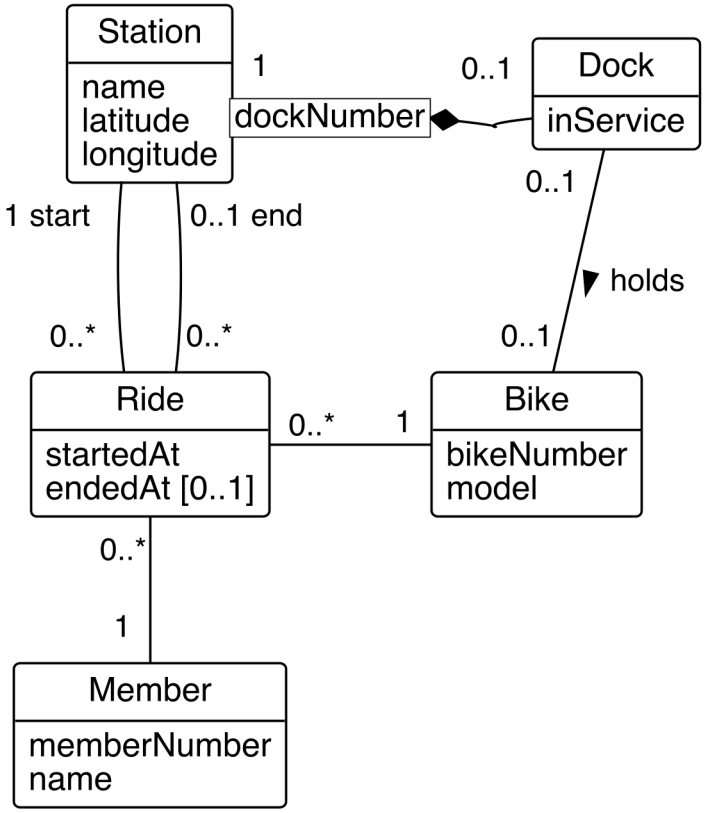


Figure 6.19.: The Bike Share (Model for Exercise 5.24)

c. Name the two foreign keys from `ride` to `station`, and say which may be NULL.

Exercise 6.22. Map the container yard of Exercise 5.25, drawn in Figure 6.20.

a. Give the relational schema. Watch the names of the slot's columns: which of the model's attribute names is a reserved word of the SQL standard?
b. Give the alternate key of `slot`, and explain why a key of three numbers alone would be wrong.

Exercise 6.23. Map the equipment rental of Exercise 5.27, drawn in Figure 6.21.

a. Give the relational schema.
b. A rental refers to its unit and its rate period, not to a rate. Explain how a query finds the amount a rental is charged at, naming each table the joins pass through.
c. The yard raises its weekly rates. What happens to the charges of rentals already returned, and what would you change if past charges must not move?

Exercise 6.24. Map the fleet of Exercise 5.28, drawn in Figure 6.22.

a. Give the relational schema for a table for each class (Section 6.5.1).
b. Give it for one table for the whole hierarchy (Section 6.5.3), with the `CHECK` constraints that keep each kind's columns filled or empty.
c. Choose between them with Table 6.11, and say why.

Exercise 6.25. Map the route network of Exercise 5.30, drawn in Figure 6.23.

a. Give the relational schema. Map the `{ordered}` end as this chapter describes.

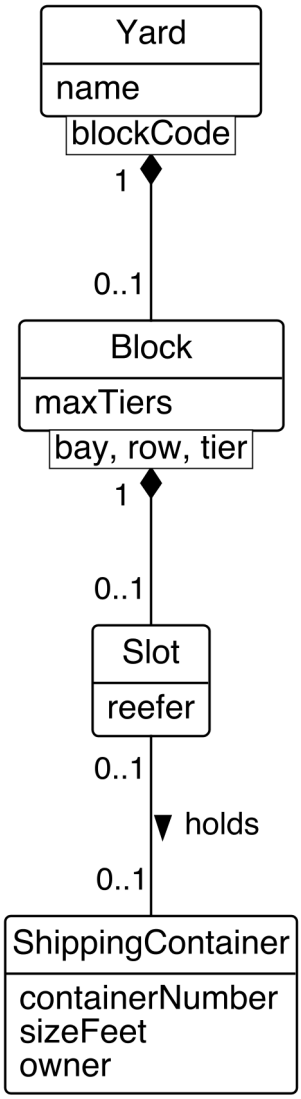


Figure 6.20.: The Container Yard (Model for Exercise 5.25)

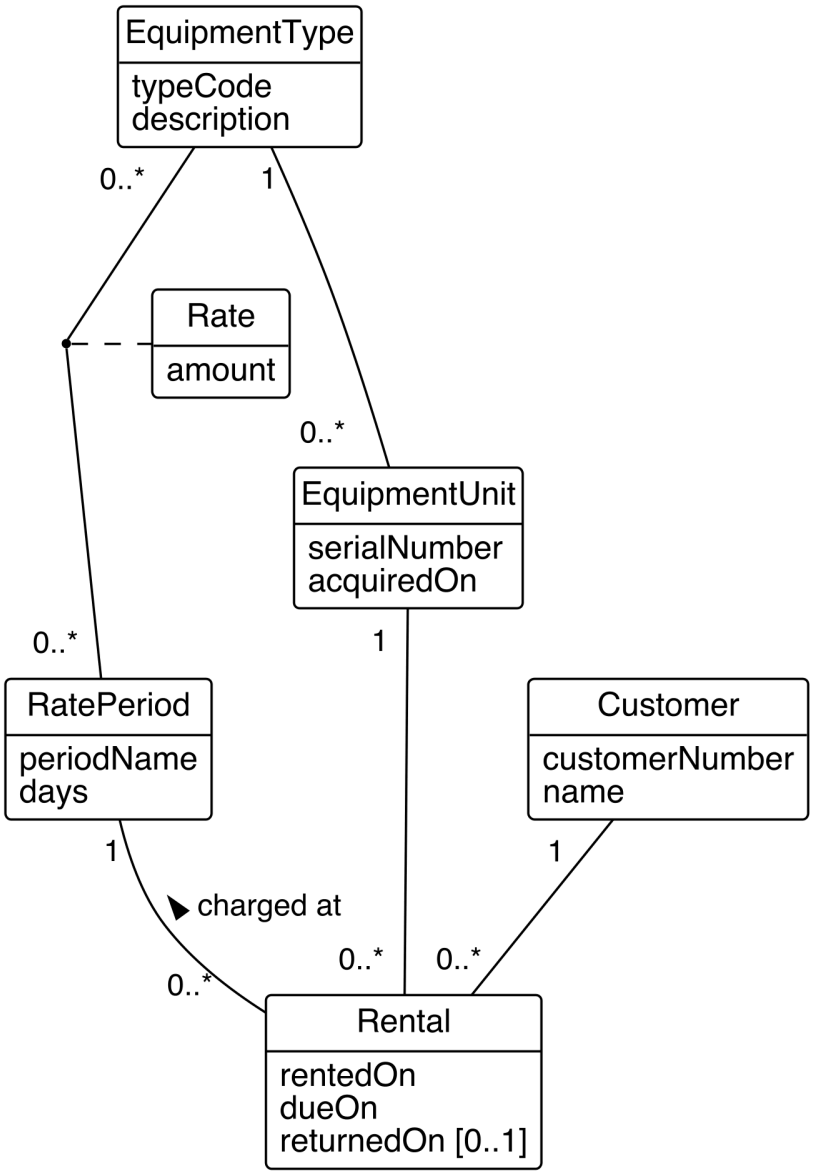


Figure 6.21.: Equipment Rental (Model for Exercise 5.27)

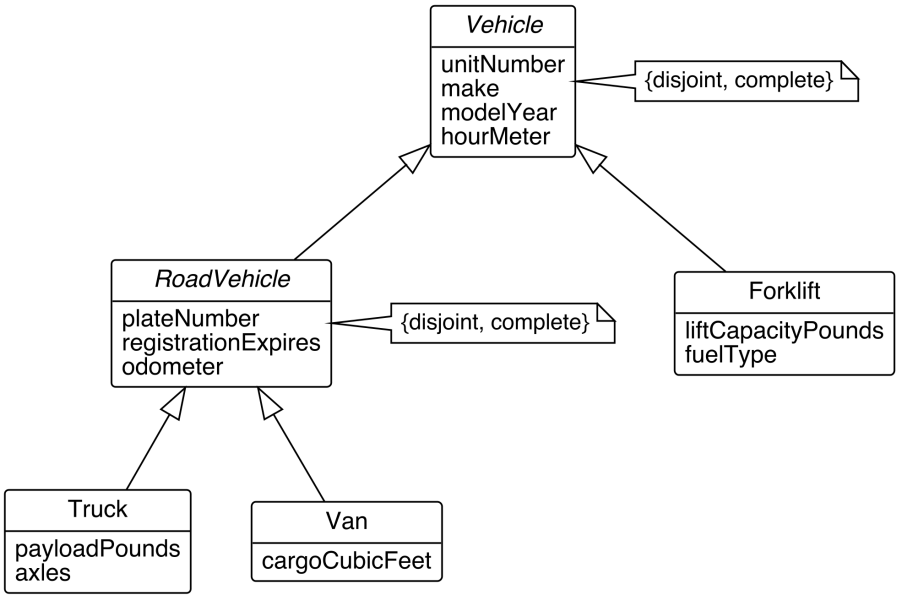


Figure 6.22.: The Fleet (Model for Exercise 5.28)

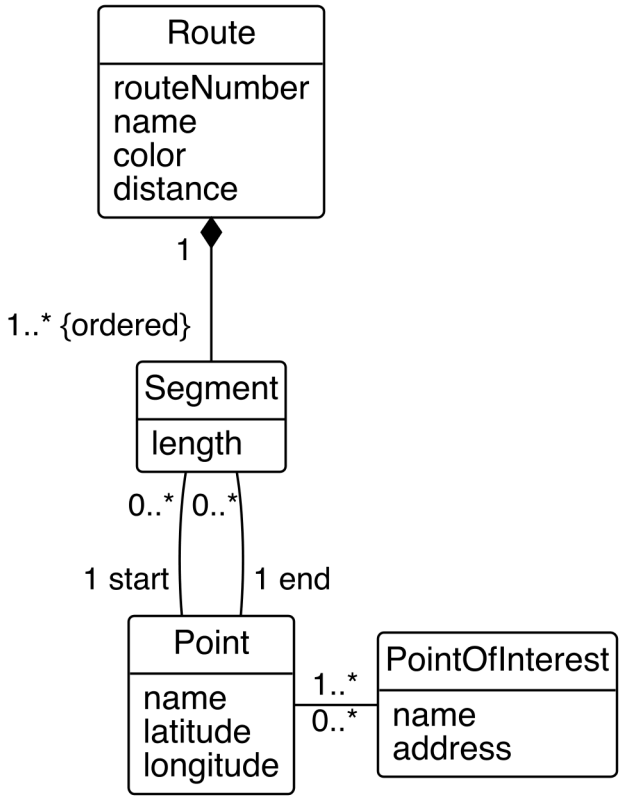


Figure 6.23.: The Route Network (Model for Exercise 5.30)

b. Write a query that lists the points of route 7, in order, using only the segments.
c. The study this model comes from stored a sequence number on each point of a route as well as on each segment. What does that add, and what can go wrong?

Exercise 6.26. Map the cards, accounts, and trips of Exercise 5.34, drawn in Figure 6.24.

a. Choose a mapping for the generalization of `SmartCard`, and justify it. Where does the foreign key of the association between `Account` and `CreditCard` go?
b. Give the relational schema of the rest: people, addresses, accounts, the authorizations, trips, and payments.
c. The study this exercise comes from mapped a generalization of transactions (fares and payments) to a table for each subclass only, repeating the common columns, so that a trip could refer to its fare directly. Explain what that mapping gives up.

Exercise 6.27. Reverse the mapping of this chapter on the Chinook database of Section C.11, loaded with the book's loader (Section A.6): from its tables, recover the class model.

a. Draw the class diagram, with multiplicities, and no foreign keys or identifiers.
b. Which table implements a many-to-many association with no attributes? Which implements a recursive association, and what are its role names?
c. Is `invoice_line` an association class of `Invoice` and `Track`, or a class of its own? Use the test of Section 5.5.

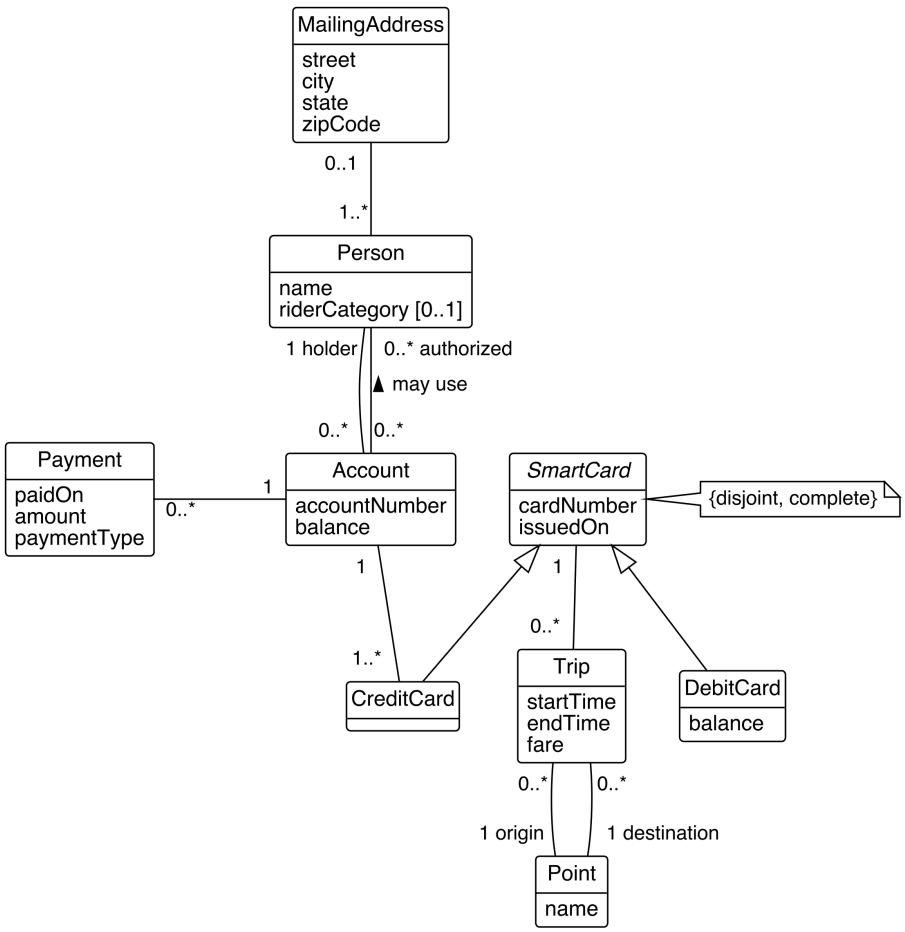


Figure 6.24.: Cards, Accounts, and Trips (Model for Exercise 5.34)

7. Normalization

Learning objectives

After reading this chapter you should be able to:

- list the steps of normalizing a table, and say what each step produces
- recognize update, insertion, and deletion anomalies in a poorly designed table, and find their symptoms with queries
- state the functional dependencies that the rules of a business impose on a table, and draw them
- test a table for first, second, third, and Boyce-Codd normal form
- decompose a table into normal form without losing information
- explain why a sound class model tends to produce normalized tables
- recognize a multivalued dependency and put a table into fourth normal form

Not every table an engineer meets was designed. Many began as a spreadsheet: an order log kept by a sales clerk, an inspection sheet filled in on the floor, a report exported from another system. Such a table puts everything about an event on one row, which is convenient to read and troublesome to keep. The same fact is written many times, and a change to it must be made everywhere at once. When it is not, the table contradicts itself.

This chapter presents **normalization**, a method for testing a table's design and for repairing it by splitting the table into smaller ones. The test uses the table's functional dependencies, the rules that say which columns decide the values of which others, and the repair produces tables in which each fact is recorded once. Chapter 6 arrived at well-designed tables from a class model; this chapter arrives at them from the other direction, from a table that already exists. The two routes meet: normalizing a flat report of HG Industries' orders produces the tables that the mapping produced.

The chapter begins, as Chapter 6 did, with the whole procedure, and then illustrates each step on one running example.

> **Following along**
>
> The running example is built from the orders of the book's database, in your `dbconcepts` database, and the counts of repeated facts from the year of history, in your `dbconcepts_history` database.

7.1. The normalization procedure

This section presents the steps of normalizing a table, before the sections that illustrate each step.

A table is normalized in five steps:

1. **Look at the data.** Take a few rows of the table, or make up a few that obey the business's rules, and find the facts that repeat.
2. **State the functional dependencies.** For each rule of the business that says one fact decides another, write the dependency, and draw them all in a dependency diagram.
3. **Find the key.** From the dependencies, find the smallest set of columns that decides every other column.

4. **Name the normal form.** Test the table against the normal forms, from the first up, and find the first one it fails.
5. **Decompose.** Split the table into tables that each pass, without losing any row and, where the normal form allows it, without losing any dependency, and repeat the test on each.

Table 7.1 says where each step is illustrated.

Table 7.1.: The Steps of Normalization and Where They Are Illustrated

Step	Illustrated in
1: look at the data	Section 7.2
2 and 3: dependencies and the key	Section 7.3
4: the normal form	Section 7.4
5: decompose	Section 7.5

The first step deserves a word of its own. Normalization is often taught as symbols, a list of dependencies manipulated by rules, and it goes wrong when the dependencies are guessed. Looking at rows first, real ones or made-up ones that obey the business's rules, grounds each dependency in something concrete and shows the repeated facts that the method exists to remove. It is the same habit as the hand evaluations of Chapter 3: see a few rows before trusting a rule about all of them.

7.2. Anomalies in a poorly designed table

This section illustrates step 1: it looks at the rows of a flat table and at what goes wrong when the table is kept up to date.

Suppose that the sales department of HG Industries kept its orders in one table, a flat order report with a row for each order line: the order's number, its customer's number and name, the line's number, its product's number

and description, and the quantity ordered. The book's database does not store such a table, but a query can produce it, by joining the four tables of stage D3. We write the join once, as a common table expression named `order_report` (Section 3.10), and use it throughout the chapter as if it were a stored table. Its rows for the first six orders are:

Table 7.2.: The Flat Order Report, First Six Orders

```
WITH order_report AS (
  SELECT o.order_number, c.customer_number,
         c.name AS customer_name, l.line_number,
         p.product_number, p.description, l.quantity
  FROM distribution.customer_order AS o
  JOIN distribution.customer AS c
    ON c.customer_pk = o.customer_fk
  JOIN distribution.order_line AS l
    ON l.customer_order_fk = o.customer_order_pk
  JOIN distribution.product AS p
    ON p.product_pk = l.product_fk
)
SELECT *
FROM order_report
WHERE order_number <= 'SO-5006'
ORDER BY order_number, line_number;
```

order_number	customer_number	customer_name	line_number	product_number	description	quantity
SO-5001	C-1001	Riverside Hardware	1	HT-100	Two-wheel hand truck	10
SO-5001	C-1001	Riverside Hardware	2	GC-300	Steel garden cart	4
SO-5002	C-1003	Cedar Creek Home Center	1	HT-200	Convertible hand truck	6
SO-5003	C-1002	Summit Farm Supply	1	HT-100	Two-wheel hand truck	12
SO-5003	C-1002	Summit Farm Supply	2	HT-200	Convertible hand truck	4
SO-5003	C-1002	Summit Farm Supply	3	GC-300	Steel garden cart	6
SO-5004	C-1001	Riverside Hardware	1	HT-100	Two-wheel hand truck	20
SO-5004	C-1001	Riverside Hardware	2	GC-300	Steel garden cart	5
SO-5004	C-1001	Riverside Hardware	3	HT-100	Two-wheel hand truck	10
SO-5005	C-1004	Midtown Tool Supply	1	GC-300	Steel garden cart	8
SO-5006	C-1006	Lakeside Distributors	1	HT-100	Two-wheel hand truck	25
SO-5006	C-1006	Lakeside Distributors	2	HT-200	Convertible hand truck	10

Read Table 7.2 for the facts that repeat. Riverside Hardware's name appears on five of these twelve rows, once for each line of each of its orders; the HT-100's description appears on five; and customer C-1001 is paired with its name every time. None of the repetitions records anything new. Each is a copy of a fact already recorded, and every copy is a place where the table can contradict itself. The contradictions come in three kinds, called **anomalies**.

An **update anomaly** occurs when a fact stored many times is changed in some copies but not all. If Riverside Hardware changed its name, the report would need the change on every row of every order the customer ever placed:

Table 7.3.: Rows That Hold Each Customer's Name

```
WITH order_report AS (
  SELECT o.order_number, c.customer_number,
         c.name AS customer_name, l.line_number
  FROM distribution.customer_order AS o
  JOIN distribution.customer AS c
    ON c.customer_pk = o.customer_fk
  JOIN distribution.order_line AS l
    ON l.customer_order_fk = o.customer_order_pk
)
SELECT customer_number, customer_name,
       COUNT(*) AS rows_holding_the_name
FROM order_report
GROUP BY customer_number, customer_name
ORDER BY customer_number;
```

customer_number	customer_name	rows_holding_the_name
C-1001	Riverside Hardware	6
C-1002	Summit Farm Supply	5
C-1003	Cedar Creek Home Center	3
C-1004	Midtown Tool Supply	1
C-1005	Prairie Garden Center	1
C-1006	Lakeside Distributors	4
C-1007	Tri-County Farm and Home	2

Table 7.3 says that renaming Riverside Hardware would take six changes, and missing one would leave two names for one customer number. A **deletion anomaly** occurs when removing a row removes a fact that should have stayed. Midtown Tool Supply (C-1004) has one row: its only order, SO-5005, has one line. Deleting that line, say because the line was entered in error, would remove every trace of Midtown Tool Supply

from the report, including that it is a customer at all. An **insertion anomaly** occurs when a fact cannot be recorded until an unrelated fact exists. Northgate Hardware (C-1008) is a customer with no orders, so it has no row in the report and cannot be given one without an order line to attach it to; the GC-400 poly cart, never ordered, has no row either.

The anomalies grow with the data. Over the year of history in stage G, the same report holds each fact about customers and products many times over:

Table 7.4.: How Many Times the Flat Report Stores Each Kind of Fact

```
WITH order_report AS (
  SELECT o.order_number, c.customer_number,
         p.product_number
  FROM distribution.customer_order AS o
  JOIN distribution.customer AS c
    ON c.customer_pk = o.customer_fk
  JOIN distribution.order_line AS l
    ON l.customer_order_fk = o.customer_order_pk
  JOIN distribution.product AS p
    ON p.product_pk = l.product_fk
)
SELECT COUNT(*) AS rows,
       COUNT(DISTINCT customer_number) AS customers,
       COUNT(DISTINCT product_number) AS products,
       COUNT(DISTINCT order_number) AS orders
FROM order_report;
```

rows	customers	products	orders
1065	29	4	614

Table 7.4 shows 1,065 rows that record the names of 29 customers and the

descriptions of 4 products. Each customer's name is stored, on average, about 37 times, and each product's description over 250 times. The tables of the book's database store each once. The next section finds the rules that decide which facts repeat.

7.3. Functional dependencies

This section illustrates steps 2 and 3: it states the rules that decide the values of columns, draws them, and finds the key.

A **functional dependency** holds between two sets of columns of a table when the values of the first set decide the values of the second: any two rows that agree on the first set also agree on the second. It is written with an arrow, `order_number → customer_number`, and read "order number determines customer number". The set on the left is called the **determinant**. In words, the dependency says: *whenever we see order SO-5001, we see customer C-1001*, on every row of the order, now and in every future state of the table.

Functional dependencies come from the rules of the business, as candidate keys do (Section 2.2). For the order report, the rules are:

- An order is placed by one customer: `order_number → customer_number`.
- A customer has one name: `customer_number → customer_name`.
- A line of an order is for one product, in one quantity: `order_number, line_number → product_number, quantity`.
- A product has one description: `product_number → description`.

Notice the third rule's determinant, two columns together. Neither the order nor the line number alone decides the product, since an order has several lines and every order has a line 1. The product does not identify a line either, since lines 1 and 3 of SO-5004 are both for the HT-100; the pair decides it.

A **functional dependency diagram** draws a table's dependencies. Each column is an oval, and an arrow runs from each determinant to each column it determines. A determinant of several columns is a larger outline drawn around their ovals, and its arrows leave the outline, since only the columns together decide the value. The columns of the key are shaded and underlined. Figure 7.1 draws the four dependencies of the order report.

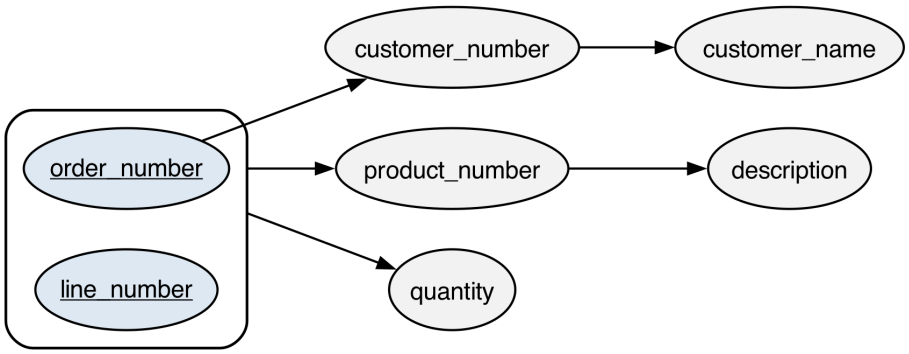


Figure 7.1.: The Functional Dependencies of the Flat Order Report

Read Figure 7.1 one arrow at a time. The outline around `order_number` and `line_number` sends arrows to `product_number` and `quantity`: the pair decides them. A separate arrow leaves the `order_number` oval alone, inside the outline, for `customer_number`, which sends its own arrow on to `customer_name`. The last arrow runs from `product_number` to `description`. Thus each rule of the list above is drawn as the arrows from its determinant, one to each column it determines: the key's outline sends two arrows for the third rule, and the figure draws four dependencies with five arrows.

A dependency is a rule, so the rows can refute it but never prove it, as with a candidate key. A query finds refutations: a dependency `X → Y` is broken whenever some value of `X` appears with two different values of `Y`. Grouping by `X` and counting the different values of `Y` (Section 4.2) checks

it over the whole year of history:

Table 7.5.: Customer Numbers with More Than One Name in the Report

```
WITH order_report AS (
  SELECT c.customer_number, c.name AS customer_name
  FROM distribution.customer_order AS o
  JOIN distribution.customer AS c
    ON c.customer_pk = o.customer_fk
  JOIN distribution.order_line AS l
    ON l.customer_order_fk = o.customer_order_pk
)
SELECT customer_number,
       COUNT(DISTINCT customer_name) AS names
FROM order_report
GROUP BY customer_number
HAVING COUNT(DISTINCT customer_name) > 1;
```

customer_number	names

Table 7.5 is empty: no customer number appears with two names, so the data does not refute `customer_number → customer_name`. In a real flat table kept by hand, the same query is the quickest way to find the update anomalies that have already happened.

The dependencies also give step 3, the key. A set of columns is a key of the table when it determines every other column, directly or through a chain of dependencies, and no smaller set does. Follow the chains of Figure 7.1 from `order_number` and `line_number`. Together they determine `product_number` and `quantity`; `order_number` determines `customer_number`, which determines `customer_name`; and `product_number` determines `description`. Every column is reached, so the pair (`order_number`, `line_number`) is a key, and neither column alone

is, since neither reaches `product_number`. Its two ovals are shaded and underlined in Figure 7.1.

Two kinds of dependency on a key matter for what follows. A dependency is **partial** when a column that is not part of a key depends on only part of a composite key: `customer_number` depends on `order_number` alone, not on the whole key. A dependency is **transitive** when a column that is not part of a key depends on the key only through another column that is not a key: `customer_name` depends on the key through `customer_number`. Both are visible in Figure 7.1. A partial dependency is an arrow that leaves one oval inside the key's outline rather than the outline itself; a transitive one is a chain of arrows, the second leaving an unshaded oval.

7.4. Normal forms

This section illustrates step 4: it defines the normal forms, from the first through Boyce-Codd, and tests the order report against each.

A **normal form** is a condition on a table's dependencies that rules out one kind of repeated fact. The forms are nested: a table in a higher normal form is in every lower one. The first three are often summarized by one sentence, due to William Kent (Kent 1983): every column that is not part of a key depends on *the key, the whole key, and nothing but the key.*

A table is in **first normal form** (1NF) when every value is atomic, a single value from its column's domain (Section 2.1), and there are no repeating groups. Recall from Section 2.1.1 that the relational model already requires all three: atomic values are the first property of a relation, values from the domain are the second, and the ban on repeating groups is the companion of the first. Thus, 1NF is those rules stated as a test of a table. The order report is in 1NF, since each row holds one line. A version of the report with one row per order and the order's products listed in a single cell is not:

Table 7.6.: Orders with Their Products in One Cell, Not in 1NF

```
SELECT o.order_number,
       STRING_AGG(p.product_number, ', '
                  ORDER BY l.line_number) AS products
FROM distribution.customer_order AS o
JOIN distribution.order_line AS l
  ON l.customer_order_fk = o.customer_order_pk
JOIN distribution.product AS p
  ON p.product_pk = l.product_fk
WHERE o.order_number <= 'SO-5004'
GROUP BY o.order_number
ORDER BY o.order_number;
```

order_number	products
SO-5001	HT-100, GC-300
SO-5002	HT-200
SO-5003	HT-100, HT-200, GC-300
SO-5004	HT-100, GC-300, HT-100

`STRING_AGG`, PostgreSQL's aggregate for the purpose (the SQL standard's is `LISTAGG`), joins a group's values into one string. The `products` column of Table 7.6 holds a list, not a value: no query can ask for the orders of the HT-200 without searching inside strings, and no foreign key can refer from one element of the list to the `product` table. Such a column belongs in a report, where it is read, and never in a stored table.

A table is in **second normal form** (2NF) when it is in 1NF and no column that is not part of a key depends on only part of a key; that is, it has no partial dependency. The order report fails. `customer_number` and `customer_name` depend on `order_number` alone, part of the key (`order_number`, `line_number`), and the failure is the cause of the rep-

etition: an order's customer is repeated on every line of the order because it depends on the order, not on the line. A table whose keys are all single columns cannot have a partial dependency, so it is in 2NF as soon as it is in 1NF.

A table is in **third normal form** (3NF) when it is in 2NF and no column that is not part of a key depends on another such column; that is, it has no transitive dependency on a key. The report fails here too, twice: `customer_name` depends on `customer_number`, and `description` on `product_number`, neither of which is a key. Each repeats a fact about a customer or a product on every row that mentions it. Thus, the order report is in 1NF, and it fails 2NF and 3NF; step 5 of the procedure must repair both.

The definitions of 2NF and 3NF speak of columns that are not part of a key, and that leaves a gap: a column that *is* part of a key may still depend on a column that is not. A table is in **Boyce-Codd normal form** (BCNF) when every determinant of a dependency is a candidate key. (Strictly, a determinant may also be a set of columns that contains a candidate key, called a **superkey**; the book states the rule with candidate keys because a determinant is taken to be as small as it can be.) BCNF is stricter than 3NF, and the difference shows up only in tables with overlapping composite keys. For example, suppose that HG Industries recorded which machine each operator runs in each work center, under two rules: an operator runs one machine in each work center, and each machine belongs to one work center. A few hypothetical rows illustrate:

Table 7.7.: Operators' Machines in Each Work Center (Hypothetical)

`operator`	`work_center`	`machine`
Maria Lopez	SAW	M-01
Maria Lopez	WLD	M-04
Linda Nguyen	WLD	M-04
Linda Nguyen	PNT	M-06

operator	work_center	machine
Aisha Patel	WLD	M-05

Figure 7.2 draws the two rules as dependencies.

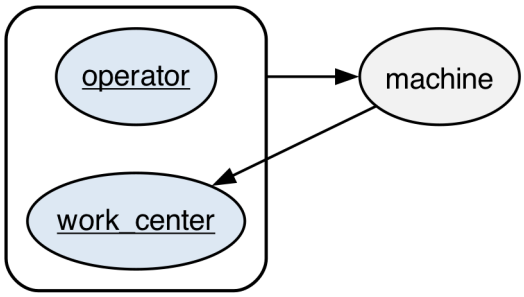


Figure 7.2.: The Functional Dependencies of the Operator Assignments

In Figure 7.2, the arrow leaving the outline around `operator` and `work_center` is the first rule, `operator, work_center → machine`, and the arrow from `machine` back into the outline, onto `work_center`, is the second, `machine → work_center`. An arrow that points into the key from a column outside it, a column that is not itself a candidate key, is the mark of a table that is not in BCNF. The table has two candidate keys, (`operator`, `work_center`) and (`operator`, `machine`), since a machine decides its work center. Every column is part of a key, so no column "not part of a key" exists, and the table is in 3NF. But the determinant `machine` is not a candidate key, so the table is not in BCNF. The consequence is a repeated fact: that M-04 is in Welding is recorded on two rows of Table 7.7, and moving M-04 to another work center would need both changed. Exercise 7.6 asks for the repair.

Table 7.8 collects the tests.

Table 7.8.: The Normal Forms and the Order Report

Normal form	The table must have	The order report
1NF	atomic values, no repeating groups	passes
2NF	1NF, and no column outside a key depending on part of a key	fails: the customer depends on the order alone
3NF	2NF, and no column outside a key depending on another such column	fails: names and descriptions depend on numbers that are not keys
BCNF	every determinant a candidate key	fails, as it fails 3NF

In practice, a table in 3NF is nearly always in BCNF, and designers aim for BCNF and settle for 3NF only when BCNF would cost a dependency, as Section 7.5 shows. Higher normal forms exist, and Section 7.7 presents the fourth. The next section repairs the order report.

7.5. Decomposition

This section illustrates step 5: it splits a table that fails a normal form into tables that pass, and checks that nothing is lost in the split.

A table that fails a normal form is repaired by **decomposition**: replacing it with two or more smaller tables whose columns, together, are the original columns. Each split follows one rule. Take a dependency `X → Y` that breaks the normal form; move `X` and `Y` into a new table, whose key is `X`; and remove `Y` from the original table, keeping `X` there to refer to the new table. Then test both tables again, and split again until every table passes.

Let's decompose the order report. It fails 2NF through the partial dependency `order_number → customer_number`, and `customer_name` depends on `customer_number`, so it travels with it. The first split moves the three columns about the order into a table of their own:

- `customer_order (order_number, customer_number, customer_name)`, key `order_number`;
- `order_line (order_number, line_number, product_number, description, quantity)`, key (`order_number`, `line_number`).

Both are in 2NF, since `customer_order` has a single-column key and every column left in `order_line` depends on the whole of its key. Neither is in 3NF. In `customer_order`, `customer_name` depends on `customer_number`, which is not a key; in `order_line`, `description` depends on `product_number`. The second and third splits remove the two transitive dependencies in the same way, moving `customer_number` and `customer_name` into a `customer` table and `product_number` and `description` into a `product` table. Listing 7.1 writes the four resulting tables in the relational schema notation of Section 2.4.

Listing 7.1 The Order Report Decomposed, with Its Natural Keys (Hypothetical)

```
customer (customer_number, customer_name)
    primary key: customer_number

customer_order (order_number, customer_number)
    primary key: order_number
    customer_number references customer_number in customer,
        must not be NULL

order_line (order_number, line_number, product_number,
            quantity)
    primary key: (order_number, line_number)
    order_number references order_number in customer_order,
        must not be NULL
    product_number references product_number in product,
        must not be NULL

product (product_number, description)
    primary key: product_number
```

Notice that each split left a foreign key behind: the determinant that moved into the new table stays in the old one, and refers to the new table's key. Figure 7.3 draws the dependencies of the four tables, each table in a dashed outline.

Compare Figure 7.3 with Figure 7.1. Every arrow of the original diagram now lies inside one outline, and in every outline the arrows leave the key and nothing else. Thus each table is in BCNF: its only determinant is its key. The two transitive chains of Figure 7.1 have been cut at the middle column, which is now a foreign key in one table and the key of another.

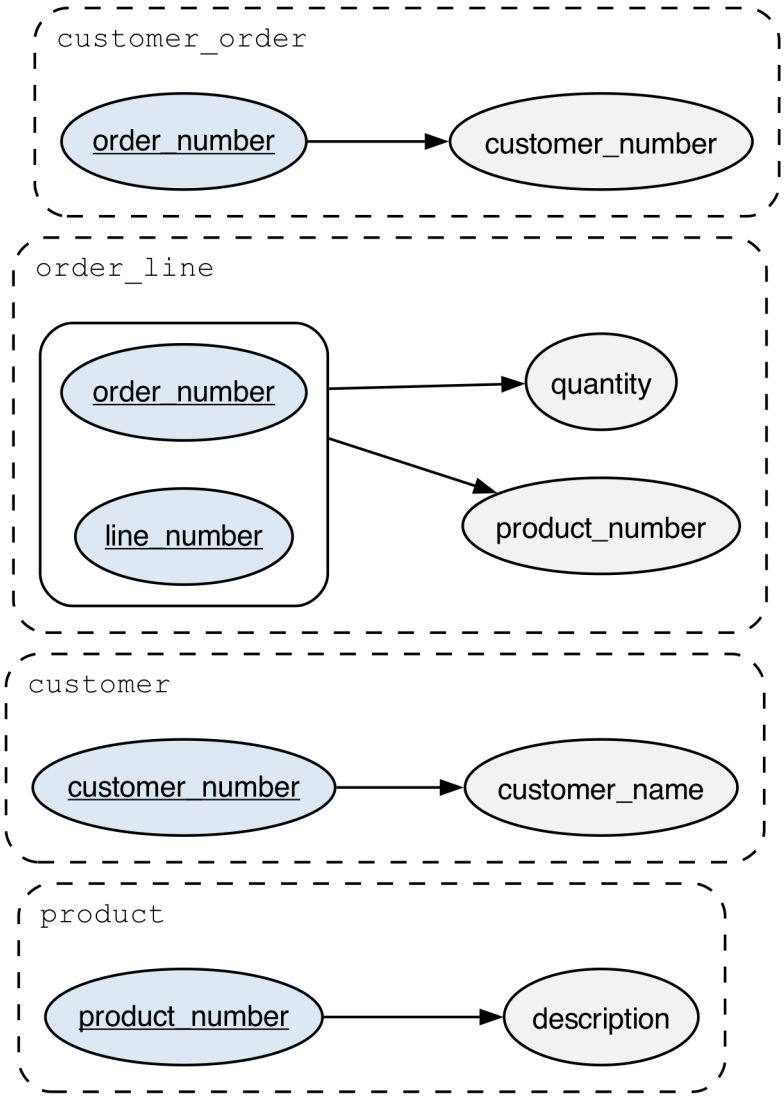


Figure 7.3.: The Order Report Decomposed into Four Tables

The four tables are the four tables of stage D3 that Chapter 6 mapped from the class model (Listing 6.12). The database adds what normalization does not decide: a surrogate primary key for each table, with the natural key kept as an alternate key (Section 6.6), and the columns the report did not carry, such as `order_date` and `unit_price`. The dependencies are the same. Normalizing the report and mapping the class model are two routes to one design.

7.5.1. Lossless decomposition

A decomposition must not lose information. A decomposition is **lossless** when joining its tables gives back the original table's rows, no fewer and no more. For a split on a functional dependency, the rule that guarantees it is simple: each split must join on columns that are a key of one of the two tables. Every split of the order report does, since the columns left behind (`order_number`, `customer_number`, `product_number`) are the keys of the new tables.

The order report itself is the evidence. Table 7.2 was produced by joining the four tables of stage D3 on their keys, so the join gives back every row the report holds. Over the year of history, the join gives one row for each order line, no more:

Table 7.9.: Order Lines and Rows of the Joined Report, Stage G

```
WITH order_report AS (
  SELECT o.order_number, l.line_number
  FROM distribution.customer_order AS o
  JOIN distribution.customer AS c
    ON c.customer_pk = o.customer_fk
  JOIN distribution.order_line AS l
    ON l.customer_order_fk = o.customer_order_pk
  JOIN distribution.product AS p
    ON p.product_pk = l.product_fk
)
SELECT (SELECT COUNT(*) FROM distribution.order_line)
         AS order_lines,
       (SELECT COUNT(*) FROM order_report) AS report_rows;
```

order_lines	report_rows
1065	1065

Table 7.9 shows 1,065 of each. A split that joins on a column that is *not* a key of either table is different. Suppose that we split the lines of the first three orders into a table of each line's product, (`order_number`, `line_number`, `product_number`), and a table of the quantities ordered of each product, (`product_number`, `quantity`). The common column, `product_number`, is a key of neither. Let's join the two tables by hand for one product, the HT-100. It appears on two lines, and with two quantities:

Table 7.10.: Joining the Two Parts on the HT-100 by Hand

Line's product	Quantity row	Pair kept by the join	A real line?
SO-5001, line 1, HT-100	HT-100, 10	SO-5001, line 1, 10	yes

Line's product	Quantity row	Pair kept by the join	A real line?
SO-5001, line 1, HT-100	HT-100, 12	SO-5001, line 1, 12	no
SO-5003, line 1, HT-100	HT-100, 10	SO-5003, line 1, 10	no
SO-5003, line 1, HT-100	HT-100, 12	SO-5003, line 1, 12	yes

Every pairing of Table 7.10 agrees on `product_number`, so the join keeps all four, and two of them are lines that were never ordered: the join says that SO-5001 ordered 12 hand trucks on line 1, and it ordered 10. The same happens for each product. The query below builds the two parts with common table expressions and joins them:

Table 7.11.: The Two Parts Joined on product_number, First Three Orders

```
WITH line AS (
  SELECT o.order_number, l.line_number,
         p.product_number, l.quantity
  FROM distribution.customer_order AS o
  JOIN distribution.order_line AS l
    ON l.customer_order_fk = o.customer_order_pk
  JOIN distribution.product AS p
    ON p.product_pk = l.product_fk
  WHERE o.order_number <= 'SO-5003'
),
line_product AS (
  SELECT order_number, line_number, product_number
  FROM line
),
product_quantity AS (
  SELECT DISTINCT product_number, quantity
  FROM line
)
SELECT lp.order_number, lp.line_number,
       lp.product_number, pq.quantity
FROM line_product AS lp
JOIN product_quantity AS pq
  ON pq.product_number = lp.product_number
ORDER BY lp.order_number, lp.line_number, pq.quantity;
```

order_number	line_number	product_number	quantity
SO-5001	1	HT-100	10
SO-5001	1	HT-100	12
SO-5001	2	GC-300	4
SO-5001	2	GC-300	6
SO-5002	1	HT-200	4
SO-5002	1	HT-200	6
SO-5003	1	HT-100	10
SO-5003	1	HT-100	12
SO-5003	2	HT-200	4
SO-5003	2	HT-200	6
SO-5003	3	GC-300	4
SO-5003	3	GC-300	6

The check. The first three orders have six lines, and each of the three products appears on two of them with two quantities, so the join gives $2 \times 2 = 4$ rows per product, 12 in all. Table 7.11 has 12 rows, of which six are real lines and six are not. Its first two rows are the two SO-5001 rows of Table 7.10. A lossy decomposition does not lose rows; it gains wrong ones, and the database cannot tell them apart from the right ones.

7.5.2. Preserving dependencies

A decomposition should also keep every dependency inside one table, where the table's key enforces it. A decomposition is **dependency preserving** when each dependency of the original table lies wholly within one of the new tables. The decomposition of the order report is: each of its four dependencies is the key of one table of Listing 7.1 and the columns it determines, so the `PRIMARY KEY` and `UNIQUE` constraints of Chapter 8 will enforce all four.

BCNF cannot always be reached without losing a dependency. The operator assignments of Table 7.7 are the standard case: any split that puts `machine` and `work_center` in a table keyed by `machine` leaves no table that holds `operator`, `work_center`, and `machine` together, so the rule "an operator runs one machine in each work center" has no key to enforce it (Exercise 7.6). You then choose between BCNF, with a rule that must be checked some other way, and 3NF, with a repeated fact. Most designers take BCNF when it preserves the dependencies and settle for 3NF when it does not. The example that follows normalizes a second report, this time from the quality department.

Example 7.1 (Normalizing an inspection sheet)**.** HG Industries' quality engineers inspect each lot of a part as it leaves production, and measure the part's characteristics: the weld strength and coating thickness of a hand truck frame, the diameter and length of an axle. One engineer keeps the results in a spreadsheet, one row per measurement, with the lot, the

characteristic, the part and its description, the inspector's employee number and name, and the value measured. The example assumes stage Q.

The request. Normalize the inspection sheet: state its dependencies, find its key and normal form, decompose it into tables in BCNF, and compare the result with the quality schema of the book's database.

The design. A query produces the sheet from the tables of stage Q. Its rows for the first four lots of the frame are:

Table 7.12.: The Inspection Sheet, First Four Lots of FR-110

```
SELECT i.lot_number, c.name AS characteristic,
       p.part_number, p.description,
       o.employee_number, o.name AS operator_name,
       m.measured_value
FROM quality.measurement AS m
JOIN quality.inspection AS i
  ON i.inspection_pk = m.inspection_fk
JOIN quality.characteristic AS c
  ON c.characteristic_pk = m.characteristic_fk
JOIN production.part AS p ON p.part_pk = i.part_fk
JOIN production.operator AS o
  ON o.operator_pk = i.operator_fk
WHERE i.lot_number <= 'L7004-04'
ORDER BY i.lot_number, c.name;
```

lot_number	characteristic	part_number	description	employee_number	operator_name	measured_value
L7004-01	Coating thickness	FR-110	Hand truck frame	E-204	Robert Hayes	3.1000
L7004-01	Weld strength	FR-110	Hand truck frame	E-204	Robert Hayes	1840.0000
L7004-02	Coating thickness	FR-110	Hand truck frame	E-203	Linda Nguyen	2.4000
L7004-02	Weld strength	FR-110	Hand truck frame	E-203	Linda Nguyen	1795.0000
L7004-03	Coating thickness	FR-110	Hand truck frame	E-204	Robert Hayes	3.3000
L7004-03	Weld strength	FR-110	Hand truck frame	E-204	Robert Hayes	1460.0000
L7004-04	Coating thickness	FR-110	Hand truck frame	E-203	Linda Nguyen	2.3000
L7004-04	Weld strength	FR-110	Hand truck frame	E-203	Linda Nguyen	1822.0000

The part and its description repeat on every row of Table 7.12, and each inspector's number and name on both rows of each lot the inspector checked. The rules of the quality department give four dependencies:

- A lot is of one part and, in this plant, is inspected once, by one inspector: `lot_number → part_number, employee_number`.
- A part has one description: `part_number → description`.
- An employee has one name: `employee_number → operator_name`.
- A lot's inspection measures each characteristic once: `lot_number, characteristic → measured_value`.

Figure 7.4 draws them.

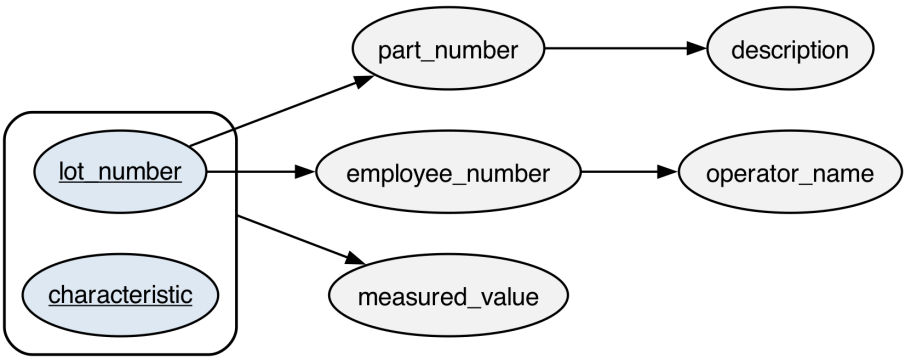


Figure 7.4.: The Functional Dependencies of the Inspection Sheet

In Figure 7.4, only the outline around `lot_number` and `characteristic` reaches `measured_value`, and the chains from `lot_number` reach every other column, so (`lot_number`, `characteristic`) is the key. The sheet is in 1NF. It fails 2NF, since the arrows to `part_number` and `employee_number` leave the `lot_number` oval alone; and it would fail 3NF through the two chains.

The first split moves the columns that depend on the lot into an `inspection` table keyed by `lot_number`, leaving `measurement (lot_number,`

`characteristic, measured_value)`. The new `inspection` table holds two transitive dependencies, and two more splits move them into `part` and `operator` tables. Figure 7.5 shows the four tables.

Each outline of Figure 7.5 has arrows only from its key, so each table is in BCNF, and each dependency of Figure 7.4 lies inside one outline, so the decomposition preserves them. Listing 7.2 shows the corresponding tables of the book's database.

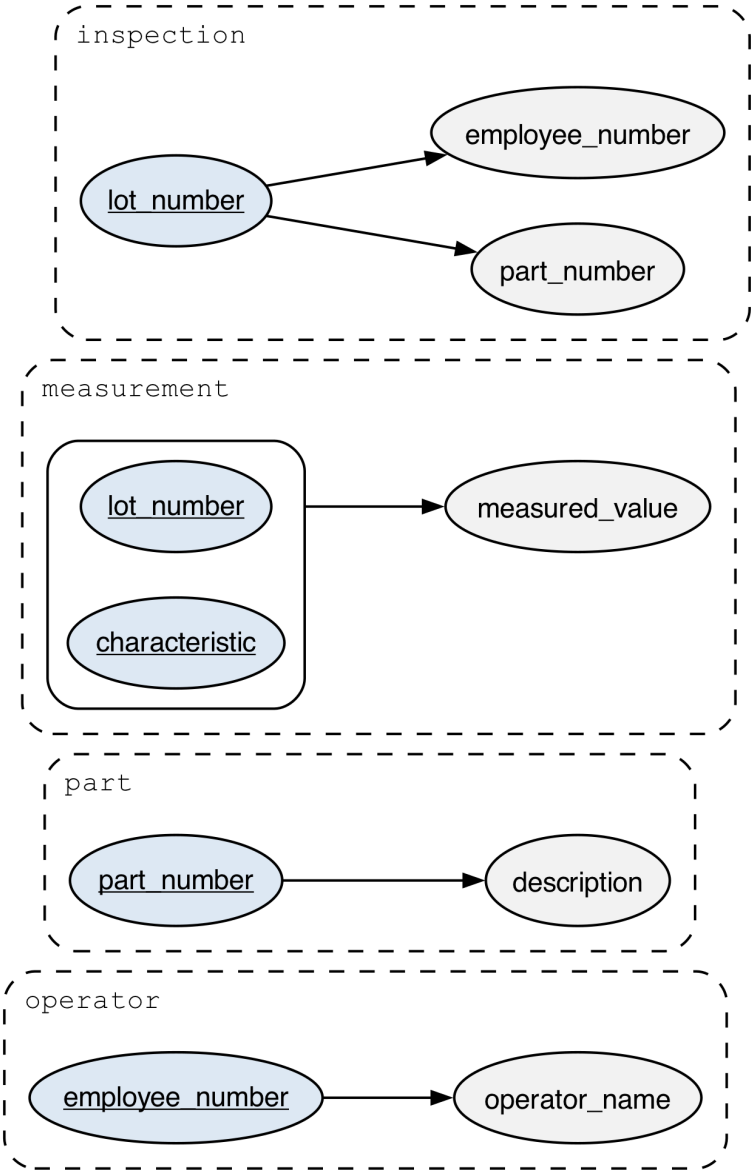


Figure 7.5.: The Inspection Sheet Decomposed into Four Tables

Listing 7.2 The Tables of Stage Q That Hold the Inspection Sheet

```
part (part_pk, part_number, description, unit_of_measure,
      standard_cost)
    primary key: part_pk
    alternate key: part_number

operator (operator_pk, employee_number, name, hire_date)
    primary key: operator_pk
    alternate key: employee_number

inspection (inspection_pk, part_fk, operator_fk,
            inspected_at, lot_number)
    primary key: inspection_pk
    part_fk references part_pk in production.part,
        must not be NULL
    operator_fk references operator_pk in
        production.operator, must not be NULL

measurement (measurement_pk, inspection_fk,
             characteristic_fk, measured_value)
    primary key: measurement_pk
    alternate key: (inspection_fk, characteristic_fk)
    inspection_fk references inspection_pk in inspection,
        must not be NULL
    characteristic_fk references characteristic_pk in
        characteristic, must not be NULL
```

The result. The four tables of Listing 7.2 match the four outlines of Figure 7.5, with surrogate keys added, with columns the sheet did not carry (such as `inspected_at`), and with `measurement` referring to a `characteristic` table, which records the specifications of each part's

characteristics (Exercise 7.8).

The check. The decomposition is lossless if the sheet has one row per measurement, since every join is on a key. Each of the first four lots has two measurements, and Table 7.12 has eight rows. Over the whole of stage Q:

Table 7.13.: Measurements and Rows of the Inspection Sheet, Stage Q

```
WITH sheet AS (
  SELECT i.lot_number, c.name AS characteristic
  FROM quality.measurement AS m
  JOIN quality.inspection AS i
    ON i.inspection_pk = m.inspection_fk
  JOIN quality.characteristic AS c
    ON c.characteristic_pk = m.characteristic_fk
  JOIN production.part AS p ON p.part_pk = i.part_fk
  JOIN production.operator AS o
    ON o.operator_pk = i.operator_fk
)
SELECT (SELECT COUNT(*) FROM quality.measurement)
         AS measurements,
       (SELECT COUNT(*) FROM sheet) AS sheet_rows;
```

measurements	sheet_rows
34	34

Table 7.13 shows 34 of each.

Interpretation. The spreadsheet is a fine way to *read* the results, and a query produces it on demand. Stored as the spreadsheet, it would repeat each inspector's name on every measurement the inspector took, could not record a new part until its first lot was inspected, and would lose an

inspector who left along with the last row naming that inspector. The tables record each of those facts once.

Where it breaks. The design rests on the rule that a lot is inspected once. If a failed lot were reworked and inspected again, `lot_number` would no longer determine the inspector, and the key of `inspection` would need the inspection's time as well. So would the key of `measurement`, since the lot's characteristics would be measured twice, and (`lot_number`, `characteristic`) would no longer decide the value measured. The book's database anticipates this: it identifies an inspection by its own surrogate key and does not declare `lot_number` unique, so a second inspection of a lot can be recorded. In stage Q, every lot happens to have been inspected once.

7.6. Normalization and class modeling

This section connects normalization to the class modeling of Chapter 5 and the mapping of Chapter 6.

Recall that step 7 of the mapping (Section 6.1) checks the tables a class model produces, and reviews their normal form. The review seldom finds a problem, and the reason is worth stating. A class describes one kind of object, and its attributes describe that object: an order's date, a product's description. Mapped to a table, each attribute becomes a column that depends on the object's identity, which is the table's key, and on nothing else. That is the condition of BCNF. Thus, a class model in which every attribute describes the object of its own class maps to tables in BCNF.

The converse makes normalization a test of the model. A normal form violation in a mapped table points to a modeling mistake, and Table 7.14 pairs the most common ones.

Table 7.14.: Normal Form Violations and the Modeling Mistakes Behind Them

The mapped table fails	Because the model	For example
1NF	stores several values in one attribute	the products of an order as one attribute of `CustomerOrder`
2NF	puts an attribute on a part that describes only its whole, or on an association class an attribute that describes only one end	`orderDate` as an attribute of `OrderLine`
3NF	puts an attribute on a class that describes another object	`customerName` as an attribute of `CustomerOrder`
BCNF	records as one association facts that belong to two	the operator assignments of Table 7.7

Each row of Table 7.14 has a repair in the model, not only in the tables. For example, `customerName` on `CustomerOrder` is a transitive dependency, `order_number → customer_number → customer_name`, and its repair is the class `Customer` of Figure 5.11, with the association that links an order to it. The operator assignments are two associations: the machine an operator is assigned to run, and the machine's place in a work center. The assignment is not the certification of Figure 5.20. Stage P3 certifies Linda Nguyen on two machines of Welding, while the rule of Table 7.7 lets her run only one of them. The repaired model draws the assignment as an association of its own between operator and machine, beside the machine's place in a work center. When the check of step 7 fails, the dashed arrow of Figure 6.1 sends you back to the class model, and the repair belongs there.

Normalization also judges what is *not* a repetition. The order line of stage D3 has a `unit_price`, and the product has a `list_price`, so the price of a product seems to be stored twice. It is not. Recall from Example 5.1 that order SO-5004 lists the HT-100 twice, at $89.00 and at $79.00. The unit price is the price of one line, determined by the line and not by the product, while the list price is the product's current price. The two columns record different facts, and a design that dropped `unit_price` in favor of `list_price` would lose the price at which each line was actually sold.

Normalization governs the tables that store facts, not every table a database shows. A report such as the order report is produced by a query when it is needed, and a view (Section 9.1) can give such a query a name, so that the report reads like a table while its facts stay stored once. In addition, a designer sometimes stores a repeated fact on purpose, to make reading faster or simpler, and accepts the anomalies it brings. This is **denormalization**, and it is the usual design of a data warehouse, where the data is loaded in bulk and rarely updated (Section 11.2). A denormalized design is a decision made about a normalized one, and you should be able to say which dependencies it gives up.

7.7. Multivalued dependencies and fourth normal form

This section presents one kind of repetition that BCNF does not catch, and the fourth normal form that removes it.

A table can be in BCNF and still repeat facts. Suppose that HG Industries kept, for each machine, the operators certified to run it and the routine maintenance tasks it needs, in one table with a row for each combination. The operators are real; stage P3 records the certifications of machine M-04:

Table 7.15.: The Operators Certified on Machine M-04

```
SELECT m.machine_number, o.name AS operator
FROM production.certification AS c
JOIN production.machine AS m
  ON m.machine_pk = c.machine_fk
JOIN production.operator AS o
  ON o.operator_pk = c.operator_fk
WHERE m.machine_number = 'M-04'
ORDER BY o.name;
```

machine_number	operator
M-04	Linda Nguyen
M-04	Maria Lopez

The tasks are hypothetical, since the book's database does not record maintenance: say that the welder needs its contact tip replaced, its wire feeder cleaned, and its gas flow checked. The rule of the business is that every certified operator performs every task on the machines they are certified on. The combined table must then hold each of the two operators of Table 7.15 with each of the three tasks:

Table 7.16.: Operators and Maintenance Tasks of M-04 in One Table (Hypothetical)

Machine	Operator	Task
M-04	Linda Nguyen	Replace contact tip
M-04	Linda Nguyen	Clean wire feeder
M-04	Linda Nguyen	Check gas flow
M-04	Maria Lopez	Replace contact tip
M-04	Maria Lopez	Clean wire feeder

Machine	Operator	Task
M-04	Maria Lopez	Check gas flow

No column of Table 7.16 determines another: a machine has several operators and several tasks, and an operator and a task each go with several machines. The key is all three columns, the only determinant is the key, and the table is in BCNF. Yet it repeats facts, and counting rows shows it. That Linda Nguyen is certified on M-04 is written three times, once per task, and that M-04 needs its gas flow checked is written twice, once per operator. Adding a fourth task to M-04 takes two new rows, one for each operator; certifying a third operator takes three, one for each task. Leaving one of those rows out does not record less; it records something false, that the operator skips that task.

The cause is a **multivalued dependency**. A multivalued dependency `machine ↠ operator` holds when a machine determines a *set* of operators, and that set is the same whatever the other columns hold. The second half of the definition is the important one. A machine having many operators is only a one-to-many relationship; the multivalued dependency says, in addition, that the set of operators does not depend on the task, and so every operator must be paired with every task. Here `machine ↠ task` holds as well, and the two sets vary independently of each other.

A table is in **fourth normal form** (4NF) when it is in BCNF and every multivalued dependency in it, other than one that covers all of the table's columns, has a candidate key, or a superkey, as its determinant. Table 7.16 fails, since `machine` is not a key. The decomposition follows the same rule as before: move the dependency into a table of its own. The result is two tables, `machine_operator (machine, operator)` and `machine_task (machine, task)`, with two rows and three rows for M-04 instead of six, and a new task or a new certification is one row. The first of the two is the `certification` table of stage P3.

The two tables join on `machine`, a key of neither, and yet the split loses nothing. The rule of Section 7.5.1 is the test for a split on a functional dependency; a split on a multivalued dependency is lossless for a different reason. Since every operator of a machine performs every task of the machine, joining the two tables pairs each operator with each task, and each of those pairings is a row the original table holds. For M-04, the join gives $2 \times 3 = 6$ rows, the six rows of Table 7.16 and no others.

Notice what the business rule decided. If the rule were instead that each task is assigned to particular operators, the three-way combination would be a fact of its own, and Table 7.16 would be a correct table recording a ternary association (Section 6.3.7). The test is the one of the counting argument: if deleting a single row would record something true, the combination is a fact, and the table stands; if it would record something false, the table hides two independent facts and belongs in 4NF.

Normal forms continue beyond the fourth. The **fifth normal form** concerns tables that can be rebuilt only by joining three or more smaller tables made from their columns, a situation rare enough in practice that this book names it and goes no further. A class model with two separate many-to-many associations, which is how Chapter 5 would model operators and tasks, maps to 4NF tables from the start.

7.8. Summary

Normalization tests a table's design against its dependencies and repairs it by decomposition. Its five steps look at the data, state the dependencies, find the key, name the normal form, and decompose.

A table that repeats facts invites anomalies. An update anomaly changes some copies of a fact and not others; a deletion anomaly loses a fact with an unrelated row; an insertion anomaly cannot record a fact until another exists. Queries find their symptoms and count the copies.

A functional dependency is a rule of the business that one set of columns decides another. Its determinant's values decide its dependents' values on every row, now and later; the data can refute a dependency but not prove it. A key is a smallest set of columns that determines every column, and a dependency of a column outside the keys on part of a key is partial, one through another such column transitive.

Each normal form rules out one kind of repeated fact. 1NF requires atomic values; 2NF forbids partial dependencies, 3NF transitive ones; BCNF requires every determinant to be a candidate key. Every column outside a key must depend on the key, the whole key, and nothing but the key.

Decomposition repairs a table by splitting it on a violating dependency. The determinant and the columns it decides move to a new table keyed by the determinant, and the determinant stays behind as a foreign key. Decomposing the order report produces the four tables of stage D3 that Chapter 6 mapped from the class model.

A decomposition must be lossless, and should preserve dependencies. Joining the tables must give back the original rows, which holds when each split joins on a key of one of its tables; a split on a column that is a key of neither can add rows that were never true. Each dependency should lie within one table, where a key enforces it, and when BCNF would lose one, 3NF is the usual compromise.

A sound class model maps to normalized tables. An attribute that describes the object of its own class depends on the key and nothing else, so a normal form violation in a mapped table points to a mistake in the model, and the repair belongs in the model. A report is a query, not a stored table, and denormalization is a deliberate decision about a normalized design.

A multivalued dependency repeats facts that BCNF allows. When a determinant fixes two sets of values that vary independently, one table must pair every member of one set with every member of the other, and

counting the rows an insertion needs exposes it. Fourth normal form moves each set into a table of its own.

7.9. Exercises

Exercise 7.1. Match each term on the left with the phrase on the right that defines it.

Term	Definition
a. determinant	1. a column outside the keys depends on part of a composite key
b. partial dependency	2. the left side of a functional dependency
c. transitive dependency	3. a fact changed in some of its copies but not all
d. update anomaly	4. every determinant is a candidate key
e. BCNF	5. a column depends on the key through a column outside any key

Exercise 7.2. Mark each statement true or false, and correct each false one.

a. If a query finds no customer number with two names, then `customer_number → customer_name` holds.
b. A table whose only key is a single column is in 2NF if it is in 1NF.
c. A table in 3NF is always in BCNF.
d. Normalization removes all repetition of values from a database.
e. The flat order report is in 1NF.

Exercise 7.3. The shipping department keeps a flat shipment report at stage D5 with the columns `shipment_number`, `shipped_on`, `warehouse_code`, `warehouse_city`, `order_number`, `order_line_number`, and `quantity_shipped`, one row per shipment line.

a. Produce the report with a query, and look at its first ten rows (step 1). Which facts repeat?
b. State its functional dependencies from the rules of stage D5, and draw them in the manner of Figure 7.1 (step 2).
c. Find its key (step 3).
d. Name the highest normal form it satisfies, and say why it fails the next (step 4).

Exercise 7.4. Using the flat order report at stage G, write a query that checks the dependency `product_number → description`, in the manner of Table 7.5. Then explain why an empty result does not prove the dependency, and what would.

Exercise 7.5. A warehouse manager keeps a table with one row per stock level and the columns `warehouse_code`, `warehouse_city`, `product_number`, `description`, `quantity_on_hand`, and `reorder_point`.

a. Give its dependencies and its key.
b. Test it against 1NF, 2NF, and 3NF, naming each dependency that causes a failure.
c. Describe an update anomaly, a deletion anomaly, and an insertion anomaly in this table, each with HG Industries' data.

Exercise 7.6. Using Table 7.7 and Figure 7.2:

a. Explain, with the table's rows, why the table is in 3NF.
b. Decompose it into two tables that are each in BCNF.
c. The rule "an operator runs one machine in each work center" was a dependency of the original table. Can either of your two tables enforce it with a key? What does that say about BCNF?

Exercise 7.7. Using the flat order report at stage D3, restricted to orders SO-5001 through SO-5006, consider splitting it into (`order_number`, `line_number`, `quantity`) and (`order_number`, `product_number`, `description`).

a. Is the split lossless? Name the rule you use.
b. Before running anything, predict the number of rows the join of the two parts on `order_number` returns. The lines of each order are in Table 7.2, and each part, being a table, holds no two identical rows.
c. Write a query, in the manner of Table 7.11, that builds the two parts and counts the rows of their join, and compare the count with your prediction.

Exercise 7.8. Suppose that the inspection sheet of Example 7.1 also carried each characteristic's `nominal` value, the target the specification sets for it.

a. State the dependency that decides `nominal`, using the rules of the quality department: a part's characteristic has one nominal value, and two parts may share a characteristic's name with different nominal values.
b. Which normal form does the extended sheet fail because of it, and why?
c. Decompose the extended sheet into tables in BCNF, and compare the result with the `quality.characteristic` table of stage Q.

Exercise 7.9. A student models an order with a single class, `Order`, whose attributes are `orderNumber`, `orderDate`, `customerNumber`, `customerName`, and `customerCity`.

a. Map the class to a table, state its dependencies, and name its highest normal form.
b. Which row of Table 7.14 describes the mistake?
c. Repair the class model, and show that the repaired model maps to tables in BCNF.

Exercise 7.10. HG Industries offers each garden cart in several colors and with several wheel types. Assume that every color is offered with every wheel type, and consider the table `cart_option (product_number,`

`color, wheel_type)`, which holds a row for each combination. The GC-300 comes in green and red, with pneumatic or solid wheels.

a. List the rows the table holds for the GC-300. What is its key, and is it in BCNF?
b. How many rows must be added to offer the GC-300 in blue? How many to offer a third wheel type, first to the rows of part (a), and then once blue is offered?
c. State the multivalued dependencies, and decompose the table into 4NF.
d. Suppose instead that the solid wheels are not offered in red. Should the table still be decomposed as in part (c)? Explain with the counting test of Section 7.7.

Exercise 7.11. Each invoice of the Chinook database of Section C.11, loaded with the book's loader (Section A.6) stores a billing address (`billing_address`, `billing_city`, and so on), and each customer an address.

a. Write a query that counts the invoices, and those whose billing address differs from their customer's address.
b. The two agree on every invoice. Is the billing address then redundant, a copy the table should not keep (Section 7.2)? State the functional dependency that would make it redundant, and say whether it holds for the business.

8. Defining and Populating Tables

Learning objectives

After reading this chapter you should be able to:

- implement a relational schema as tables in a PostgreSQL database, in a script that rebuilds them from nothing
- choose a PostgreSQL data type for each column, and avoid the types that lose or distort values
- write CREATE TABLE statements with primary key, foreign key, unique, not-null, and check constraints
- insert, update, and delete rows, and predict what each constraint does to a change
- load data from a CSV file through a staging table, and check it before it enters the database
- change a table's definition with ALTER TABLE

The chapters so far have queried tables that already existed, and then shown where those tables come from: a class model (Chapter 5), mapped to a relational schema (Chapter 6), checked by normalization (Chapter 7). This chapter takes the last step. It turns a relational schema into tables that a database system holds, fills them with rows, and changes them as the business changes.

The statements that do this are the other half of SQL. The part of SQL that defines tables and their rules is called the **data definition language** (DDL): `CREATE`, `ALTER`, and `DROP`. The part that changes rows is called

the **data manipulation language** (DML): `INSERT`, `UPDATE`, and `DELETE`, together with the `SELECT` of the earlier chapters. A database is built with the first and kept current with the second.

The chapter builds the distribution schema of HG Industries, stages D1 to D3, from the relational schemas of Chapter 6. It does so by reading the scripts that build the book's own database, one clause at a time, so that every statement in the chapter is one you can run and every table is one you have already queried.

> **Following along**
>
> The scripts are in the companion code: `code/sql/distribution/d1-customer.sql`, `d2-customer-order.sql`, and `d3-product-order-line.sql`. They create tables your `dbconcepts` database already holds, so run there they would fail. To watch them build the tables from nothing, run `code/sql/load/through-d3.sql`, which runs all three after removing the book's schemas, and then `through-q.sql` to return the database to the state the other chapters use. A query shown with its result after a change runs against a copy of the database that is discarded afterward, so the change never reaches yours; the text says which change each result assumes.

8.1. From a design to tables

This section presents the whole process of building a database from its design, before the sections that illustrate each step.

A relational schema says what tables a database holds and what rules they obey; it does not yet hold anything. Building the database from it takes five steps:

1. **Create a place for the tables.** A PostgreSQL server holds databases, and a database holds schemas, each a named group of tables. HG Industries' tables live in one database, `dbconcepts` in the book's setup (Appendix A), with a schema per department.
2. **Choose a data type for each column.** The type implements the column's domain, as far as a type can; a `CHECK` constraint narrows it further.
3. **Create each table with its constraints.** One `CREATE TABLE` statement per table of the relational schema declares its columns, its primary key, its alternate keys, its foreign keys with their delete rules, and its checks. A table is created after every table it refers to.
4. **Put rows into the tables.** Rows are inserted by statements or loaded from files, in the same order as the tables were created, so that a foreign key always refers to a row that exists.
5. **Check the result.** Queries count the rows and look for what the constraints cannot catch, as the hand check of each worked example has done.

The steps are written down as a **script**, a file of SQL statements run in order, rather than typed once and forgotten. A script that builds a database from nothing is its most useful description: it can be read, kept under version control beside the code that uses the database, and run again to rebuild the database on another machine or after an experiment has spoiled it. The book's database is kept this way. `code/sql/reset.sql` drops its four schemas, and each file of `code/sql/load/` rebuilds them through one stage. Table 8.1 says where each step is illustrated.

Table 8.1.: The Steps of Building a Database and Where They Are Illustrated

Step	Illustrated in
1: a place for the tables	this section
2: data types	Section 8.2
3: tables and constraints	Section 8.3

Step	Illustrated in
4: rows	Section 8.4 and Section 8.5
5: checking	each section's queries, and Section 8.5

The first step, for the distribution department, is one statement:

```
CREATE SCHEMA distribution;
```

A table in the schema is then named with the schema in front, `distribution.customer`, which is how the book has written table names in every query. PostgreSQL also keeps a *search path*, a list of schemas it looks in for a table named without one, so that `SET search_path TO distribution;` lets a session write `customer` alone. The book's scripts always qualify the name, so that they work whatever the search path holds.

DataGrip and DBeaver can create schemas and tables through dialogs, without a statement being typed. The dialog is a convenience, not a different mechanism: it writes a `CREATE` statement and sends it to the server. Both tools will show the statement, and will also *generate* the DDL of any existing table (DataGrip's *SQL Scripts, Generate DDL to Query Console*; DBeaver's *Generate SQL, DDL*), which is how you capture a database that someone built by hand as a script. The rest of this chapter writes the statements directly, since a statement can be read, kept, and run again, and a sequence of clicks cannot.

8.2. Data types

This section illustrates step 2: choosing the data type of each column.

Recall from Section 6.2 that a column's type is chosen in two steps: first the domain is stated in words, and then an SQL type and constraints implement it (Table 6.2). A **data type** is the set of values a column can hold, as the database system knows it, together with the operations it allows on them. The type decides more than storage: it decides how values compare, sort, and add, and whether a value that does not belong is refused or silently changed. The book's database uses few types, and the query below counts the columns of each across all four schemas:

Table 8.2.: The Data Types of the Book's Database

```
SELECT data_type, COUNT(*) AS columns
FROM information_schema.columns
WHERE table_schema IN ('distribution', 'production',
                       'quality', 'sales')
GROUP BY data_type
ORDER BY columns DESC, data_type;
```

data_type	columns
integer	73
character varying	36
numeric	16
date	10
character	3
boolean	1
timestamp without time zone	1

Recall from Section 2.5 that the `information_schema` of Table 8.2 is the standard's set of views of the catalog, the database's description of its own tables and columns. Its type names are the standard's long spellings: `character varying` is `VARCHAR`, and `character` is `CHAR`. The types fall into five kinds.

Whole numbers. `INTEGER` holds whole numbers up to about two billion, which covers every key and count in the book. `SMALLINT` and `BIGINT` hold smaller and larger ranges. The surrogate keys are `INTEGER` columns whose values the database generates, declared `GENERATED ALWAYS AS IDENTITY`, the standard form since PostgreSQL 10. Older scripts write `SERIAL` for the same purpose, a PostgreSQL shorthand that you will still meet.

Numbers with fractions. `NUMERIC(p, s)` holds exact decimal numbers with `p` digits in all, `s` of them after the decimal point: a price is `NUMERIC(10,2)`, a measured diameter `NUMERIC(10,4)`. The alternative, `REAL` or `DOUBLE PRECISION`, holds binary floating-point numbers, which are fast and compact but cannot represent most decimal fractions exactly. The difference is easy to show:

Table 8.3.: Adding 0.1 and 0.2 as Floating-Point and as Exact Numbers

```
SELECT 0.1::DOUBLE PRECISION + 0.2 AS double_sum,
       0.1::NUMERIC + 0.2 AS numeric_sum,
       0.1::DOUBLE PRECISION + 0.2 = 0.3 AS double_equal,
       0.1::NUMERIC + 0.2 = 0.3 AS numeric_equal;
```

double_sum	numeric_sum	double_equal	numeric_equal
0.30000000000000004	0.3	false	true

The `::` of Table 8.3 is PostgreSQL's shorthand for converting a value to a type, the standard's `CAST(0.1 AS DOUBLE PRECISION)`. The floating-point sum is not 0.3, and a comparison with 0.3 is false.

The error does not always show. PostgreSQL prints a `REAL` or `DOUBLE PRECISION` value as the shortest decimal that reads back to the same stored value, so a stored value that is not exact can print as if it were:

Table 8.4.: A REAL Value as Printed and as Stored

```
SELECT 32.38::REAL AS real_value,
       32.38::REAL::DOUBLE PRECISION AS stored_value;
```

real_value	stored_value
32.38	32.380001068115234

The `REAL` of Table 8.4 prints as 32.38, but the value stored is 32.380001068115234, which converting it to `DOUBLE PRECISION` reveals. Thus, a column of floating-point prices can look right and still not add up to the cent. For money, and for any measurement recorded to a known number of decimal places, use `NUMERIC`; a sum of prices that is off by a fraction of a cent fails every reconciliation. Keep floating point for quantities that are computed rather than recorded, such as the output of a simulation. PostgreSQL also has a `MONEY` type; avoid it, since it is PostgreSQL's own, and how it reads and prints values depends on the server's locale settings.

Text. `VARCHAR(n)` holds text of up to `n` characters, and refuses a longer value. `CHAR(n)` holds exactly `n`, padding a shorter value with spaces, which suits a code that always has the same length, such as a two-letter state, and little else. `TEXT` holds text of any length; it is PostgreSQL's (the standard has no `TEXT`), and suits notes and descriptions with no natural limit. A length limit in `VARCHAR(n)` is a small domain rule: an order number of 40 characters is a mistake, and the limit refuses it.

A code that is written with digits is still text, and belongs in a text column. Part numbers, employee numbers, and postal codes are identifiers, not quantities: nobody adds them, and a leading zero is part of the code. Stored as a number, the zero is lost:

Table 8.5.: A Postal Code Stored as a Number and as Text

```
SELECT '07201'::INTEGER AS as_integer,
       '07201'::VARCHAR(5) AS as_text;
```

as_integer	as_text
7201	07201

Dates and times. `DATE` holds a calendar date, `TIMESTAMP` a date and a time of day. The book writes date values in the ISO form `'2026-01-05'`, which PostgreSQL reads the same way whatever its settings, and the standard's typed literal `DATE '2026-01-05'`, used since Section 3.2. A `TIMESTAMP` records a time without saying where; `inspected_at` is one, since HG Industries inspects in one plant. A business spread across time zones uses `TIMESTAMP WITH TIME ZONE`, which records an instant and displays it in the reader's zone.

True or false. `BOOLEAN` holds `true`, `false`, or NULL, as the results of Table 8.3 show. The sales schema's date dimension marks weekends with one.

A domain narrower than its type is finished with a `CHECK` constraint, as Section 6.2 described: `list_price > 0`, `level BETWEEN 1 AND 3`. A domain that is a short list of values, an **enumeration**, can be implemented in two ways (Blaha and Premerlani 1998). The list can be written into the table as a check, `CHECK (unit_of_measure IN ('EA', 'FT'))`; or the values can be rows of a table of their own, a **lookup table**, which the column refers to with a foreign key. The quality schema's `defect_type` table is a lookup table: each defect code is a row, with its description. The check is simpler; the lookup table lets the list grow by inserting a row instead of changing a table's definition, and gives each value a place for its

description. You should choose the lookup table when the list will change or its values need more than a name. Table 8.6 collects the choices.

Table 8.6.: Choosing a Data Type

The attribute is	Use	Not
a key or a count	`INTEGER`	`NUMERIC`, text
money, or a measurement to fixed decimals	`NUMERIC(p, s)`	`REAL`, `DOUBLE PRECISION`, `MONEY`
a computed quantity, such as a simulation's output	`DOUBLE PRECISION`	text
a code or identifier, even one made of digits	`VARCHAR(n)`, or `CHAR(n)` if always `n` long	`INTEGER`
free text	`VARCHAR(n)` or `TEXT`	
a date, or a date and time	`DATE`, `TIMESTAMP`	text
yes or no	`BOOLEAN`	`CHAR(1)`, `INTEGER`
one of a short list	a check, or a lookup table	free text

Every "not" of Table 8.6 is a type that accepts the value and then misbehaves: a date stored as text sorts `'10/2/2026'` before `'9/2/2026'`, and a price stored as text cannot be summed. The next section writes the types into tables.

8.3. Creating tables and constraints

This section illustrates step 3: writing a `CREATE TABLE` statement for each table of a relational schema, with its constraints.

Recall the complete relational schema of the `customer` table from Listing 6.1: five columns with their types and NULL rules, a primary key, and an alternate key. The stage D1 script turns it into one statement:

```
CREATE TABLE distribution.customer (
    customer_pk     INTEGER GENERATED ALWAYS AS IDENTITY
                    PRIMARY KEY,
    customer_number VARCHAR(10) NOT NULL UNIQUE,
    name            VARCHAR(60) NOT NULL,
    city            VARCHAR(40) NOT NULL,
    state           CHAR(2)     NOT NULL
);
```

Each line inside the parentheses declares one column: its name, its type, and its **column constraints**, the rules that concern that column alone. Read them against Listing 6.1. `customer_pk` is `INTEGER`, filled by the database (`GENERATED ALWAYS AS IDENTITY`), and the primary key (`PRIMARY KEY`), which also makes it `NOT NULL` and unique. `customer_number` must be present (`NOT NULL`) and may not repeat (`UNIQUE`), the alternate key. The other three columns must be present. Nothing in the listing is left out, and nothing is added.

A rule that concerns several columns is written after the columns, as a **table constraint**. The `customer_order` table of stage D2 has one, a check that compares two dates, beside a foreign key written on its column:

```
CREATE TABLE distribution.customer_order (
    customer_order_pk   INTEGER GENERATED ALWAYS AS IDENTITY
                        PRIMARY KEY,
    order_number        VARCHAR(10) NOT NULL UNIQUE,
    order_date          DATE        NOT NULL,
    requested_ship_date DATE,
    customer_fk         INTEGER     NOT NULL
```

```
        REFERENCES distribution.customer (customer_pk),
    CHECK (requested_ship_date >= order_date)
);
```

`requested_ship_date` has no `NOT NULL`, so it may be missing, as the attribute `requestedShipDate [0..1]` allows. `customer_fk` is a foreign key: `REFERENCES distribution.customer (customer_pk)` says that every value must be the key of an existing customer. It is written here as a column constraint, which is allowed because it concerns one column; `FOREIGN KEY (customer_fk) REFERENCES distribution.customer (customer_pk)` after the columns says the same thing as a table constraint, and is the only form for a foreign key of several columns. The last line is a check that involves two columns, so it is written as a table constraint. The standard requires that form for a check of several columns; PostgreSQL also accepts such a check written on one of its columns, but the table constraint is the portable form, and it says what the rule concerns.

Notice that a check passes when its condition is true *or unknown*. An order with no requested ship date makes `requested_ship_date >= order_date` unknown (Section 3.3), and the order is accepted, as it should be. A check such as this one therefore passes a NULL and does not make a column required; `NOT NULL` is how a column is made required. (A check written to test for NULL, such as `CHECK (state IS NOT NULL)`, does refuse one, and `NOT NULL` says the same thing more plainly.)

The database gives every constraint a name, and names it in every message that reports a violation. A constraint left unnamed, as in the book's scripts, is named by PostgreSQL from its table and columns: the check of `customer_order` becomes `customer_order_check`, the alternate key `customer_order_order_number_key`. A constraint can be named explicitly by writing `CONSTRAINT order_ship_after_order CHECK (...)`, which you should do when the generated name would not tell a reader which rule was broken.

Example 8.1 (Creating the order line table)**.** The sales department of HG Industries wants to record what each order is for: the products, the quantity of each, and the price charged, which may differ from the list price. Chapter 6 mapped the order line, composed in its order, to the complete relational schema of Listing 6.10. The example assumes stage D2, with `customer_order` in place, and a `product` table already created by the first statement of the stage D3 script.

The request. Write the `CREATE TABLE` statement for `order_line` from Listing 6.10, and confirm that the database holds every constraint the listing names.

The design. The columns come first, each line of the listing becoming one column declaration:

```
CREATE TABLE distribution.order_line (
    order_line_pk     INTEGER GENERATED ALWAYS AS IDENTITY
                      PRIMARY KEY,
    customer_order_fk INTEGER NOT NULL
        REFERENCES distribution.customer_order
            (customer_order_pk)
        ON DELETE CASCADE,
    line_number       INTEGER NOT NULL
                      CHECK (line_number > 0),
    product_fk        INTEGER NOT NULL
        REFERENCES distribution.product (product_pk),
    quantity          INTEGER NOT NULL
                      CHECK (quantity > 0),
    unit_price        NUMERIC(10,2) NOT NULL
                      CHECK (unit_price > 0),
    UNIQUE (customer_order_fk, line_number)
);
```

The two foreign keys differ in one clause. `customer_order_fk` adds `ON DELETE CASCADE`: deleting an order deletes its lines, as the composition of

Section 6.3.6 requires. `product_fk` has no `ON DELETE` clause, so it takes the default, `NO ACTION`, and a product that has been ordered cannot be deleted. Each positive quantity, price, and line number of the listing is a `CHECK` on its column. The alternate key (`customer_order_fk`, `line_number`) spans two columns, so it is a table constraint, the last line.

The result. The standard view `information_schema.table_constraints` lists the constraints the database holds for the table:

Table 8.7.: The Constraints of the Order Line Table

```
SELECT constraint_name, constraint_type
FROM information_schema.table_constraints
WHERE table_schema = 'distribution'
  AND table_name = 'order_line'
  AND constraint_name NOT LIKE '%not_null'
ORDER BY constraint_type, constraint_name;
```

constraint_name	constraint_type
order_line_line_number_check	CHECK
order_line_quantity_check	CHECK
order_line_unit_price_check	CHECK
order_line_customer_order_fk_fkey	FOREIGN KEY
order_line_product_fk_fkey	FOREIGN KEY
order_line_pkey	PRIMARY KEY
order_line_customer_order_fk_line_number_key	UNIQUE

The last condition of the query leaves out the six `NOT NULL` rules, which PostgreSQL's version of the view lists as checks whose names end in `not_null`.

The check. Listing 6.10 names one primary key, one alternate key, two foreign keys, and three checks: seven constraints, and Table 8.7 has seven

rows, one of each kind named.

Interpretation. The table now enforces the business's rules about order lines without any program's help. A line cannot be recorded for an order or a product that does not exist, with a zero quantity or a zero price, or with a line number its order already uses; and an order cannot be removed while leaving its lines behind.

Where it breaks. The constraints check each line alone. Nothing requires an order to have at least one line, which the multiplicity `1..*` of the class model says it must; an order with no lines is accepted, and a query such as those of Section 3.7 must find it. Nor does anything tie `unit_price` to the product's `list_price`, deliberately, since the business sells below list.

The constraints are the database's defense, and they are worth seeing at work. A statement that breaks one is refused, with a message that names the constraint. For example, a second customer with an existing customer number breaks the alternate key:

```
INSERT INTO distribution.customer
    (customer_number, name, city, state)
VALUES ('C-1001', 'Riverside Supply', 'Rogers', 'AR');
```

```
ERROR:  duplicate key value violates unique constraint
 ↪  "customer_customer_number_key"
DETAIL:  Key (customer_number)=(C-1001) already exists.
```

An order line for a product that does not exist breaks a foreign key, and one with a quantity of zero breaks a check:

```
INSERT INTO distribution.order_line
    (customer_order_fk, line_number, product_fk,
     quantity, unit_price)
VALUES (12, 3, 9, 5, 89.00);
```

```
ERROR:  insert or update on table "order_line" violates
 ↪  foreign key constraint "order_line_product_fk_fkey"
DETAIL:  Key (product_fk)=(9) is not present in table
 ↪  "product".
```

```
INSERT INTO distribution.order_line
    (customer_order_fk, line_number, product_fk,
     quantity, unit_price)
VALUES (12, 3, 1, 0, 89.00);
```

```
ERROR:  new row for relation "order_line" violates check
 ↪  constraint "order_line_quantity_check"
DETAIL:  Failing row contains (23, 12, 3, 1, 0, 89.00).
```

Each message names the table, the constraint, and for a key the offending value, which is usually enough to find the bad row in a long script. An identity column declared `GENERATED ALWAYS` refuses a value supplied for it, so that the database alone numbers its rows; the message's hint names the override that a script copying rows from another database, keys and all, would use:

```
INSERT INTO distribution.customer
    (customer_pk, customer_number, name, city, state)
VALUES (9, 'C-1009', 'Rogers Farm Supply', 'Rogers', 'AR');
```

```
ERROR:  cannot insert a non-DEFAULT value into column
 ↪  "customer_pk"
DETAIL:  Column "customer_pk" is an identity column defined
 ↪  as GENERATED ALWAYS.
HINT:  Use OVERRIDING SYSTEM VALUE to override.
```

The order of the statements in a script follows from the foreign keys. A table can be created only after every table it refers to, so the stage scripts create `customer`, then `customer_order`, then `product` and `order_line`. Removing tables runs the other way: `DROP TABLE` removes a table and its rows, and it refuses to remove a table that another table refers to:

```
DROP TABLE distribution.customer;
```

```
ERROR:  cannot drop table distribution.customer because other
 ↪  objects depend on it
DETAIL:  constraint customer_order_customer_fk_fkey on table
 ↪  distribution.customer_order depends on table
 ↪  distribution.customer
HINT:  Use DROP ... CASCADE to drop the dependent objects
 ↪  too.
```

As the message's hint says, `DROP TABLE distribution.customer CASCADE` would remove the foreign key constraints that depend on `customer` and then the table; the tables that referred to it would remain, without their foreign keys. `DROP SCHEMA distribution CASCADE`, which `code/sql/reset.sql` runs, removes the schema with every table in it. Once kept, neither can be undone, and you should keep them in scripts that rebuild what they remove. (A change can be tried without being kept, inside a transaction, as Section 8.4 shows.) The tables now exist, and the next section changes their rows.

8.4. Inserting, updating, and deleting rows

This section illustrates step 4: putting rows into tables, and then changing and removing them as the business changes.

`INSERT` adds rows to a table. Its usual form names the table, lists the columns it supplies, and gives a row of values for each row to add, in the same order as the columns:

```
INSERT INTO distribution.customer
    (customer_number, name, city, state)
VALUES ('C-1009', 'Rogers Farm Supply', 'Rogers', 'AR'),
       ('C-1029', 'Crowley''s Ridge Hardware',
        'Jonesboro', 'AR');
```

The statement adds two customers. A column left out of the list, such as `customer_pk`, takes its default: the next identity value for a key, and NULL for a column with no default. You should always write the column list. Without it, the values are matched to the table's columns in the order the table declared them, and a script written that way breaks, or worse loads values into the wrong columns, when a column is added. Notice the second name: a quote inside a text value is written twice, `'Crowley''s Ridge Hardware'`, which is how SQL tells the quote in the value from the quote that ends it.

Trying a change without keeping it

DataGrip and DBeaver keep each statement's change the moment it runs (*auto-commit*). A change run that way alters your copy of the stage for good, and the examples and exercises that assume the stage as loaded no longer give the book's results. To try a change without keeping it, run `BEGIN` before it and `ROLLBACK` after it:

```
BEGIN;
INSERT INTO distribution.customer
    (customer_number, name, city, state)
VALUES ('C-1009', 'Rogers Farm Supply', 'Rogers',
 ↪  'AR');
SELECT * FROM distribution.customer;
ROLLBACK;
```

The statements between the two form a *transaction*, and `ROLLBACK` discards everything it changed, as if it had never run. After a statement is refused, PostgreSQL refuses every later statement of the transaction until the `ROLLBACK`, so try each change that may be refused in a transaction of its own. Chapter 9 presents transactions. Try the insert above, and every change of this chapter's examples and exercises, this way: Example 8.2 and Example 8.3 assume stage D3 as loaded, and both add C-1009 themselves. A change kept by mistake is undone by reloading the stage with its script in `code/sql/load/`.

A foreign key column needs the key of the row it refers to, and a generated key is not something you know in advance. The stage scripts can write the keys as numbers, since they insert into empty tables in a fixed order (`code/sql/README.md` explains). A statement that records a new order for an existing customer instead looks the key up by the customer's number,

with a scalar subquery (Section 3.8) in place of the value. The example below records a whole order this way.

Example 8.2 (Recording a new customer's first order)**.** On April 2, 2026, Rogers Farm Supply of Rogers, Arkansas, placed its first order with HG Industries: eight GC-300 steel garden carts at $119.00, twelve HT-100 hand trucks at $89.00, and ten HT-200 convertible hand trucks at $139.00, to ship by April 9. The sales department gave it customer number C-1009 and the order the number SO-5018. The example assumes stage D3.

The request. Record the customer, the order, and its three lines, referring to the customer and the products by their business numbers.

The design. The customer comes first, since the order refers to it:

```
INSERT INTO distribution.customer
    (customer_number, name, city, state)
VALUES ('C-1009', 'Rogers Farm Supply', 'Rogers', 'AR');
```

The order's `customer_fk` is found by a subquery on `customer_number`:

```
INSERT INTO distribution.customer_order
    (order_number, order_date, requested_ship_date,
     customer_fk)
VALUES ('SO-5018', DATE '2026-04-02', DATE '2026-04-09',
        (SELECT customer_pk
         FROM distribution.customer
         WHERE customer_number = 'C-1009'));
```

Each line looks up its order by `order_number` and its product by `product_number` in the same way:

```
INSERT INTO distribution.order_line
    (customer_order_fk, line_number, product_fk,
     quantity, unit_price)
VALUES
  ((SELECT customer_order_pk
    FROM distribution.customer_order
    WHERE order_number = 'SO-5018'), 1,
   (SELECT product_pk FROM distribution.product
    WHERE product_number = 'GC-300'), 8, 119.00),
  ((SELECT customer_order_pk
    FROM distribution.customer_order
    WHERE order_number = 'SO-5018'), 2,
   (SELECT product_pk FROM distribution.product
    WHERE product_number = 'HT-100'), 12, 89.00),
  ((SELECT customer_order_pk
    FROM distribution.customer_order
    WHERE order_number = 'SO-5018'), 3,
   (SELECT product_pk FROM distribution.product
    WHERE product_number = 'HT-200'), 10, 139.00);
```

The result. After the three statements, a query joins the new rows back together:

Table 8.8.: Order SO-5018 After the Three Inserts

```
SELECT c.customer_pk, o.customer_order_pk,
       l.line_number, p.product_number, l.quantity,
       l.unit_price, l.quantity * l.unit_price AS value
FROM distribution.customer AS c
JOIN distribution.customer_order AS o
  ON o.customer_fk = c.customer_pk
JOIN distribution.order_line AS l
  ON l.customer_order_fk = o.customer_order_pk
JOIN distribution.product AS p
  ON p.product_pk = l.product_fk
WHERE c.customer_number = 'C-1009'
ORDER BY l.line_number;
```

customer_pk	customer_order_pk	line_number	product_number	quantity	unit_price	value
9	13	1	GC-300	8	119.00	952.00
9	13	2	HT-100	12	89.00	1068.00
9	13	3	HT-200	10	139.00	1390.00

The check. A generated key comes from the column's identity counter, not from a count of the rows. On a freshly loaded stage D3 the counters of `customer` and `customer_order` stand at 8 and 12, the last keys the load used, so the new customer's key is 9 and the new order's is 13, as Table 8.8 shows. An insert takes the next value even when it is refused or rolled back, and the value is never given back; in a copy of the stage where inserts have been tried, the keys are higher and the rows are otherwise the same. The three line values are $8 \times 119.00 = 952.00$, $12 \times 89.00 = 1{,}068.00$, and $10 \times 139.00 = 1{,}390.00$, and the order is worth \$3,410.00.

Interpretation. Rogers Farm Supply is now a customer with one open order, and every query of the earlier chapters counts it: it appears in the order report, in the sales of each product, and among the orders due to ship on April 9.

Where it breaks. A subquery that finds no row returns NULL. A mistyped product number, `'HT-210'`, makes `product_fk` NULL, and the `NOT NULL` of the column refuses the line:

```
INSERT INTO distribution.order_line
    (customer_order_fk, line_number, product_fk,
     quantity, unit_price)
VALUES
  ((SELECT customer_order_pk
    FROM distribution.customer_order
    WHERE order_number = 'SO-5018'), 3,
   (SELECT product_pk FROM distribution.product
    WHERE product_number = 'HT-210'), 10, 139.00);
```

```
ERROR:  null value in column "product_fk" of relation
 ↪  "order_line" violates not-null constraint
DETAIL:  Failing row contains (23, 13, 3, null, 10, 139.00).
```

The refusal protects the order. Had `product_fk` allowed NULL, the line would have been recorded for no product at all.

`INSERT` can also take its rows from a query, `INSERT INTO t (columns) SELECT ...`, which adds every row the query returns. It is the form used to copy or transform rows already in the database, and Section 8.5 uses it to move loaded rows into their table. It has one danger the `VALUES` form lacks: a query that finds no rows inserts nothing, and reports success.

`UPDATE` changes values in existing rows. It names the table, sets one or more columns to new values, and chooses the rows with a `WHERE` clause, as a query does. Suppose that HG Industries raises the list prices of its garden carts by 5 percent, rounded to the cent. Before changing anything, preview the change with a query that selects the same rows and computes the new values:

Table 8.9.: The Garden Carts' Prices, Before and After a 5 Percent Increase

```
SELECT product_number, list_price,
       ROUND(list_price * 1.05, 2) AS new_price
FROM distribution.product
WHERE product_number LIKE 'GC-%'
ORDER BY product_number;
```

product_number	list_price	new_price
GC-300	119.00	124.95
GC-400	149.00	156.45

Table 8.9 shows the two rows the update will touch and what they will become. The update itself reuses the preview's `WHERE` clause and its expression:

```
UPDATE distribution.product
SET list_price = ROUND(list_price * 1.05, 2)
WHERE product_number LIKE 'GC-%';
```

Table 8.10.: The Product List After the Price Increase

```
SELECT product_number, description, list_price
FROM distribution.product
ORDER BY product_number;
```

product_number	description	list_price
GC-300	Steel garden cart	124.95
GC-400	Poly garden cart	156.45
HT-100	Two-wheel hand truck	89.00
HT-200	Convertible hand truck	139.00

The two carts in Table 8.10 have the prices the preview promised, $124.95 and $156.45, and the hand trucks are unchanged. The preview is a habit worth keeping. An `UPDATE` without a `WHERE` clause changes every row of the table, and the database will not ask whether you meant it; the preview shows the rows before they change. Notice also that the new prices apply to future orders only. The lines already recorded keep their `unit_price`, which is why Section 7.6 kept the price of a line apart from the product's.

An update must leave every row within its constraints, or it is refused as an insert would be. Moving an order's requested ship date before its order date breaks the check of `customer_order`:

```
UPDATE distribution.customer_order
SET requested_ship_date = DATE '2026-01-01'
WHERE order_number = 'SO-5001';
```

```
ERROR:  new row for relation "customer_order" violates check
 ↪  constraint "customer_order_check"
DETAIL:  Failing row contains (1, SO-5001, 2026-01-05,
 ↪  2026-01-01, 1).
```

`DELETE` removes rows, chosen by a `WHERE` clause. The foreign keys decide what happens to the rows that refer to them. Recall from Section 6.3.6 that an order line is composed in its order, so that deleting an order deletes its lines; Chapter 6 promised to show it, at stage D5, with order SO-5012, which has two lines and has not shipped. Before the deletion, the last two orders have these lines:

Table 8.11.: The Last Two Orders and Their Lines, Before the Deletion

```
SELECT o.order_number,
       COUNT(l.order_line_pk) AS lines
FROM distribution.customer_order AS o
LEFT JOIN distribution.order_line AS l
  ON l.customer_order_fk = o.customer_order_pk
WHERE o.order_number >= 'SO-5011'
GROUP BY o.order_number
ORDER BY o.order_number;
```

order_number	lines
SO-5011	2
SO-5012	2

The deletion names only the order:

```
DELETE FROM distribution.customer_order
WHERE order_number = 'SO-5012';
```

Table 8.12.: Order Lines Remaining After SO-5012 Is Deleted

```
SELECT COUNT(*) AS order_lines,
       COUNT(*) FILTER (WHERE customer_order_fk = 12)
         AS lines_of_so5012
FROM distribution.order_line;
```

order_lines	lines_of_so5012
20	0

Stage D5 holds 22 order lines, and Table 8.12 shows 20, none of them SO-5012's: the database deleted the two lines itself, because of the `ON DELETE CASCADE` of `customer_order_fk`. (`FILTER` limits one aggregate to some rows; it is standard SQL, and PostgreSQL supports it.) A foreign key without the cascade refuses instead. Order SO-5001 has shipped, and its shipment refers to it:

```
DELETE FROM distribution.customer_order
WHERE order_number = 'SO-5001';
```

```
ERROR:  update or delete on table "customer_order" violates
 ↪  foreign key constraint "shipment_customer_order_fk_fkey"
 ↪  on table "shipment"
DETAIL:  Key (customer_order_pk)=(1) is still referenced from
 ↪  table "shipment".
```

The refusal is the design working. The shipment is a record of goods that left the warehouse, and the database will not let the order it fulfilled disappear from under it. In the same way, a product that has been ordered cannot be deleted, since order lines refer to it:

```
DELETE FROM distribution.product
WHERE product_number = 'HT-100';
```

```
ERROR:  update or delete on table "product" violates foreign
 ↪  key constraint "order_line_product_fk_fkey" on table
 ↪  "order_line"
DETAIL:  Key (product_pk)=(1) is still referenced from table
 ↪  "order_line".
```

A `DELETE` without a `WHERE` clause removes every row and keeps the table. `TRUNCATE`, standard since SQL:2008, also empties a table, faster, without examining the rows one at a time, but it is not the same. A `DELETE` follows each foreign key's delete rule, so at stage D3 deleting every order deletes every order line with it; `TRUNCATE` refuses a table that another table refers to, unless the tables that refer to it are truncated with it:

```
TRUNCATE distribution.customer_order;
```

```
ERROR:  cannot truncate a table referenced in a foreign key
 ↪  constraint
DETAIL:  Table "order_line" references "customer_order".
HINT:  Truncate table "order_line" at the same time, or use
 ↪  TRUNCATE ... CASCADE.
```

Neither statement removes the table; `DROP TABLE` does (Section 8.3). The three are easy to confuse, and they differ in what remains: rows and table, table only, or nothing.

Under auto-commit, each statement of this section changes the database as soon as it runs, and each is complete in itself; inside a `BEGIN` and `ROLLBACK`, as in the callout of Section 8.4, its change is not kept. Chapter 9 shows

how several statements, such as the three inserts of Example 8.2, can be made to succeed or fail together. The next section adds rows in bulk, from files.

8.5. Loading data from files

This section illustrates step 4 again, for rows that arrive as files, and step 5, checking them before they enter the database.

Data often reaches a database as a file. A colleague exports a list from another system or a spreadsheet, and the rows must be loaded into tables that someone else designed. The usual form is a **CSV file** (comma-separated values): plain text, one row per line, the values separated by commas, and usually a first line of column names. A value that contains a comma or a quote is enclosed in double quotes. The file carries no types, no keys, and no rules; everything in it is text, and nothing in it has been checked.

PostgreSQL offers two ways to load such a file, and the tools add a third:

- `COPY table FROM 'file'` is an SQL statement that the *server* runs. It reads a file on the server's own machine, so it needs a server-side path, and PostgreSQL allows it only to superusers and to roles granted `pg_read_server_files`.
- `\copy table FROM 'file'` is a psql command (Appendix A) that reads a file on *your* machine and sends its rows to the server. It takes the same options as `COPY`, needs no special privilege, and is the form the book uses.
- **The import wizards of DataGrip and DBeaver** read a file on your machine, show its first rows, and map its columns to a table's columns. They are convenient for a single load, and they do what `\copy` does.

Whichever you use, do not load a file straight into its final table. If one row breaks a constraint, the whole load is refused, and you learn only about the first bad row; and a value that is wrong but legal, such as a city misspelled, goes in unnoticed. Load the file instead into a **staging table**, a table with the file's columns, all of type `TEXT` and without constraints, which accepts anything. Then examine the staged rows with queries, and move them into the real table with `INSERT ... SELECT`, cleaning them on the way.

Example 8.3 (Loading new customers from a file)**.** HG Industries is expanding into Oklahoma, Kansas, and Texas, and the sales department has signed 22 new customers. A sales assistant exported them from the contact list to `code/data/new-customers-raw.csv`, one customer per line with a header line: the customer number, name, city, and state. The first lines of the file are:

```
customer_number,name,city,state
C-1009,Rogers Farm Supply,Rogers,AR
C-1010,Boston Mountain Hardware,Harrison,AR
C-1011,Riverbend Home and Garden,Conway,AR
```

The example assumes stage D3, which holds the eight customers of stage D1.

The request. Load the new customers into `distribution.customer`, after finding and repairing whatever is wrong with the file.

The design. The staging table has the file's four columns, as text:

```
CREATE TABLE distribution.customer_staging (
    customer_number TEXT,
    name            TEXT,
    city            TEXT,
    state           TEXT
);
```

The file is loaded with psql, started in the companion code's folder. The options say that the file is CSV and that its first line is a header:

```
\copy distribution.customer_staging FROM
 ↪  'data/new-customers-raw.csv' WITH (FORMAT csv, HEADER
 ↪  true)
```

A `\copy` command is one line, however long, and ends without a semicolon. psql answers `COPY 23`: the file holds 23 rows, although the sales department signed 22 customers. The staged rows are now examined with queries. The first looks for customer numbers that appear more than once:

Table 8.13.: Customer Numbers That Appear More Than Once in the File

```
SELECT customer_number, COUNT(*) AS copies,
       COUNT(DISTINCT name || city || state)
         AS different_versions
FROM distribution.customer_staging
GROUP BY customer_number
HAVING COUNT(*) > 1;
```

customer_number	copies	different_versions
C-1015	2	1

C-1015 appears twice, and both copies are the same, so one can be dropped. The second query looks for states that are not two capital letters, with the state shown between brackets so that a space is visible:

Table 8.14.: Staged Customers Whose State Is Not Two Capital Letters

```
SELECT customer_number, city,
       '[' || state || ']' AS state_as_loaded
FROM distribution.customer_staging
WHERE state <> UPPER(state) OR LENGTH(state) <> 2;
```

customer_number	city	state_as_loaded
C-1017	Norman	[ok]

Norman's state was typed as `ok` with a trailing space. The `CHAR(2)` column would not refuse it: PostgreSQL drops extra characters that are spaces, and would store `ok` in lower case, where a query grouping by state would count it apart from every `OK`. The third query looks for customers that the database already holds, which the alternate key would refuse:

Table 8.15.: Staged Customers Already in the Database

```
SELECT s.customer_number, s.name
FROM distribution.customer_staging AS s
JOIN distribution.customer AS c
  ON c.customer_number = s.customer_number;
```

customer_number	name

Table 8.15 is empty, as it should be. The move into `customer` repairs the two problems found: `DISTINCT` keeps one copy of each identical row, and `UPPER(TRIM(state))` removes the space and capitalizes the state. The staging table is then dropped:

```
INSERT INTO distribution.customer
    (customer_number, name, city, state)
SELECT DISTINCT customer_number, name, city,
       UPPER(TRIM(state))
FROM distribution.customer_staging;

DROP TABLE distribution.customer_staging;
```

The result. The customers by state, after the load:

Table 8.16.: Customers by State After the Load

```
SELECT state, COUNT(*) AS customers
FROM distribution.customer
GROUP BY state
ORDER BY state;
```

state	customers
AR	12
KS	2
MO	6
OK	7
TX	3

The check. The file has 24 lines: a header and 23 rows. One row is a duplicate, so 22 customers are new, and with the 8 of stage D1 the table should hold 30. The counts of Table 8.16 add up to 30, and Norman is counted under `OK`, not in a group of its own.

Interpretation. HG Industries now has customers in five states, and eleven of its twelve customers outside Arkansas and Missouri are new; before the

load, Cedar Creek Home Center of Tulsa was the only one. Every query of the earlier chapters sees the new customers as if they had been entered one at a time.

Where it breaks. `DISTINCT` removes only rows that are identical in every column. Had the two copies of C-1015 differed, say in the spelling of the city, `DISTINCT` would have kept both, and the alternate key would have refused the whole `INSERT`, naming C-1015. The `different_versions` column of Table 8.13 checks for this case before the move, and a duplicate with two versions needs a person to decide which is right.

The staging pattern of Example 8.3 is the heart of every load: land the data where nothing can refuse it, look at it with queries, and move it with a query that repairs what the looking found. Chapter 11 builds on it, as extract, transform, and load. Two problems that files commonly carry did not occur in this one. A code written with digits loses its leading zeros if the file passed through a spreadsheet that took the code for a number (Table 8.5), and nothing in the database can restore them. Dates arrive in more than one form: `2026-04-13` and `4/13/2026` are the same day, and a staging query must find which form each row uses before converting it, as Exercise 8.8 asks. The next section changes the tables themselves.

8.6. Changing a table's definition

This section changes the definition of tables that already hold rows.

A design changes after the database is built: the business records something new, a rule is discovered, or two departments' data must be connected. Dropping and recreating a table would lose its rows, so SQL changes a table in place with `ALTER TABLE`. Table 8.17 lists the changes the book uses.

Table 8.17.: The Changes ALTER TABLE Makes

To	Write
add a column	`ALTER TABLE t ADD COLUMN c type`
remove a column	`ALTER TABLE t DROP COLUMN c`
require a value	`ALTER TABLE t ALTER COLUMN c SET NOT NULL`
allow NULL again	`ALTER TABLE t ALTER COLUMN c DROP NOT NULL`
change a column's type	`ALTER TABLE t ALTER COLUMN c TYPE type`
add a constraint	`ALTER TABLE t ADD CONSTRAINT name ...`
remove a constraint	`ALTER TABLE t DROP CONSTRAINT name`
rename a column	`ALTER TABLE t RENAME COLUMN c TO d`

A change must hold for the rows already there, and the database checks them before it accepts the change. Requiring a requested ship date on every order is refused at stage D3, since two orders have none:

```
ALTER TABLE distribution.customer_order
    ALTER COLUMN requested_ship_date SET NOT NULL;
```

```
ERROR:  column "requested_ship_date" of relation
 ↪  "customer_order" contains null values
```

A constraint added to a table applies to every later change. The customer load of Example 8.3 repaired a lower-case state in its `INSERT ... SELECT`; a check makes the database refuse one. Its name is chosen so that a message names the rule:

```
ALTER TABLE distribution.customer
    ADD CONSTRAINT customer_state_in_capitals
    CHECK (state = UPPER(state));
```

```
INSERT INTO distribution.customer
    (customer_number, name, city, state)
VALUES ('C-1017', 'Canadian River Home Center',
        'Norman', 'ok');
```

```
ERROR:  new row for relation "customer" violates check
 ↪  constraint "customer_state_in_capitals"
DETAIL:  Failing row contains (9, C-1017, Canadian River Home
 ↪  Center, Norman, ok).
```

The most common change adds a column to connect data that were designed apart. Stage P1 adds the production department's parts, and each product that distribution sells is a manufactured part: the HT-100 hand truck is part HT-100. The P1 script, `code/sql/production/p1-part.sql`, connects them in three steps, which is the pattern for adding a required column to a table that already has rows. First, the column is added, allowing NULL, since the existing products have no value for it yet:

```
ALTER TABLE distribution.product
    ADD COLUMN manufactured_part_fk INTEGER
        REFERENCES production.manufactured_part (part_fk);
```

Second, an `UPDATE` fills it for every product, finding each product's part by its number. The standard form sets the column to a correlated subquery (Section 3.8):

```
UPDATE distribution.product
SET manufactured_part_fk =
    (SELECT part_pk
     FROM production.part
     WHERE part.part_number = product.product_number);
```

The P1 script writes the same update in PostgreSQL's own form, `UPDATE ... FROM production.part WHERE ...`, which joins the table being updated to another; both give the same result here. Third, once every product has its part, the column is required:

```
ALTER TABLE distribution.product
    ALTER COLUMN manufactured_part_fk SET NOT NULL;
```

The stage P1 database holds the result, and a join through the new column finds each product's part:

Table 8.18.: Each Product and Its Manufactured Part, Stage P1

```
SELECT p.product_number, p.manufactured_part_fk,
       pt.part_number, pt.standard_cost
FROM distribution.product AS p
JOIN production.part AS pt
  ON pt.part_pk = p.manufactured_part_fk
ORDER BY p.product_number;
```

product_number	manufactured_part_fk	part_number	standard_cost
GC-300	3	GC-300	66.40
GC-400	4	GC-400	79.30
HT-100	1	HT-100	52.40
HT-200	2	HT-200	58.90

Each product of Table 8.18 has found the part with its own number, so the update matched all four. Had one product had no part of the same number, its column would have stayed NULL, and the third step would have been refused, as the ship-date change was: the order of the three steps turns a silent gap into a refusal. The same pattern, create bare, fill,

then constrain, is how a large load is often done: the tables are created without their keys, loaded, and the keys added with `ALTER TABLE ... ADD CONSTRAINT`, so that each constraint checks all the rows once.

A change made with `ALTER TABLE` must also be made to the scripts that build the database, or the next rebuild will lose it. The book's database does this by adding each change as a stage script that runs after the scripts before it, as `p1-part.sql` runs after the distribution stages. You should keep your own databases the same way: the scripts, not the running database, are the definition.

8.7. Summary

A database is built from its relational schema by a script. The script creates a schema, then each table after the tables it refers to, then the rows in the same order; kept and run again, it is the database's most useful description.

A data type implements a domain, and the wrong type distorts values. Use `INTEGER` for keys and counts, `NUMERIC` for money and recorded measurements, text for every code even one written in digits, and `DATE` or `TIMESTAMP` for dates. Floating point cannot hold most decimal fractions exactly, and a number drops a code's leading zero.

Constraints make the database enforce the business's rules. `PRIMARY KEY`, `UNIQUE`, `NOT NULL`, `REFERENCES`, and `CHECK` implement a relational schema's keys, NULL rules, foreign keys, and domains, and a statement that breaks one is refused with a message naming it. A rule about one column can be a column constraint; a rule about several is a table constraint, as the standard requires (PostgreSQL also accepts a check of several columns on one column).

INSERT, UPDATE, and DELETE change rows, and the constraints judge every change. Write the column list of every `INSERT`,

look up a generated key by its business number with a subquery, and preview every `UPDATE` and `DELETE` with a query that selects the same rows. A foreign key decides what a deletion does to the rows that refer to it: `ON DELETE CASCADE` deletes them with it, and the default refuses the deletion.

A file is loaded into a staging table, examined, and then moved. Load it with psql's `\copy` or a tool's import wizard into a table of text columns with no constraints; find duplicates, malformed values, and existing keys with queries; and move the rows with an `INSERT ... SELECT` that repairs what the queries found. Count the rows at each step.

ALTER TABLE changes a table that holds rows, and the rows must satisfy the change. A new required column is added in three steps: add it allowing NULL, fill it with an `UPDATE`, and then require it. Every change to a running database belongs in the scripts that rebuild it.

8.8. Exercises

Exercise 8.1. Match each term on the left with the phrase on the right that defines it.

Term	Definition
a. DDL	1. a rule written after the columns, which may involve several of them
b. DML	2. statements that define tables and their rules
c. table constraint	3. a column whose values the database generates
d. identity column	4. a table whose rows are the allowed values of a domain
e. lookup table	5. statements that change the rows of tables

Exercise 8.2. Mark each statement true or false, and correct each false one.

a. A `CHECK (quantity > 0)` constraint refuses a row whose quantity is NULL.
b. `DOUBLE PRECISION` is a good type for the unit price of an order line, since it holds more digits than `NUMERIC(10,2)`.
c. An employee number such as `0412` should be stored in an `INTEGER` column, since it is made of digits.
d. A foreign key of two columns must be written as a table constraint.
e. `DROP TABLE` removes a table's rows but keeps its definition.

Exercise 8.3. HG Industries wants to record the maintenance done on its machines, one row per job: the machine's number (such as `M-04`), the date of the job, the time the machine was stopped and restarted, the technician's employee number (such as `E-0204`), the labor hours to the nearest quarter hour, the cost of parts in dollars and cents, whether the job was planned or a breakdown, and a technician's note of any length.

a. State the domain of each attribute in words, in the manner of Table 6.2.
b. Choose a PostgreSQL type and constraints for each, and justify each choice that Table 8.6 does not settle.
c. Would you record "planned or breakdown" as a `BOOLEAN`, a check, or a lookup table? Give a reason for your choice.

Exercise 8.4. Stage D4 adds warehouses and the stock level of each product in each warehouse. Its relational schema is Listing 6.7; in addition, a warehouse code is at most 10 characters, a city at most 40, a state two letters, and a quantity on hand and a reorder point are whole numbers that are never negative.

a. Write the `CREATE TABLE` statements for `warehouse` and `stock_level`, in the order a script must run them.

b. Compare your statements with `code/sql/distribution/d4-warehouse-stock-level.sql` and explain any difference.
c. How many constraints, other than `NOT NULL`, does `stock_level` have? Check your answer with a query in the manner of Table 8.7, at stage D4.

Exercise 8.5. For each statement below, run at stage D3 in a transaction of its own that you roll back (Section 8.4), say whether the database accepts it, and if not, which constraint refuses it and why.

a. An order requested to ship before it was placed:

```
INSERT INTO distribution.customer_order
    (order_number, order_date, requested_ship_date,
     customer_fk)
VALUES ('SO-5013', DATE '2026-03-20',
        DATE '2026-03-18', 2);
```

b. An order with no requested ship date:

```
INSERT INTO distribution.customer_order
    (order_number, order_date, customer_fk)
VALUES ('SO-5013', DATE '2026-03-20', 2);
```

c. A line for order 12 (SO-5012):

```
INSERT INTO distribution.order_line
    (customer_order_fk, line_number, product_fk,
     quantity, unit_price)
VALUES (12, 1, 2, 3, 139.00);
```

d. A product given away:

```
INSERT INTO distribution.product
    (product_number, description, list_price)
VALUES ('GC-500', 'Folding garden cart', 0);
```

e. A customer with a three-letter state:

```
INSERT INTO distribution.customer
    (customer_number, name, city, state)
VALUES ('C-1009', 'Rogers Farm Supply', 'Rogers',
 ↪  'ARK');
```

Exercise 8.6. HG Industries raises the list prices of its two hand trucks, the HT-100 and the HT-200, by 3 percent, rounded to the cent. Use stage D3.

a. Compute the two new prices by hand.
b. Write a query that previews the change, in the manner of Table 8.9, and compare its result with your answer to a.
c. Write the `UPDATE` statement. If you run it, run it inside a transaction you roll back (Section 8.4).
d. What would your `UPDATE` do if its `WHERE` clause were left out, and how would the preview have warned you?

Exercise 8.7. For each statement below, run at stage D5 in a transaction of its own that you roll back (Section 8.4), say whether the database accepts it and what it removes, or which constraint refuses it.

a. `DELETE FROM distribution.customer_order WHERE order_number = 'SO-5011';`
b. `DELETE FROM distribution.customer_order WHERE order_number = 'SO-5004';`
c. `DELETE FROM distribution.product WHERE product_number = 'GC-400';`
d. `DELETE FROM distribution.customer WHERE customer_number = 'C-1008';`

Exercise 8.8. The new customers of Example 8.3 placed 40 orders in April 2026, which the sales assistant exported to `code/data/april-orders-raw.csv`, with the columns `order_number`, `customer_number`,

`order_date`, and `requested_ship_date`. Assume stage D3 with the customers of Example 8.3 loaded. If you run your statements, run them in one transaction that you roll back at the end (Section 8.4), loading the customers as Example 8.3 does and then the orders.

a. Write the statements that create a staging table for the file and load it with `\copy`.
b. Write a query that finds the rows whose dates are not in the ISO form `YYYY-MM-DD`. How many are there, and what form do they use?
c. Write the `INSERT ... SELECT` that moves the orders into `distribution.customer_order`, converting both dates and finding each order's `customer_fk` from its customer number. An empty requested ship date in the file must become NULL.
d. How many orders should the insert add, and how many of them should have no requested ship date? Check your answer with a query of stage G, which holds the same orders among those of all its customers: count the April 2026 orders of the new customers, C-1009 to C-1030.

Exercise 8.9. The sales department divides its customers into two territories: East for customers in Arkansas and Missouri, West for Oklahoma, Kansas, and Texas. Every customer must have a territory, and no other territory exists.

a. Write the statements that add a required column `territory` to `distribution.customer`, fill it, and restrict it to the two values, in the order the database requires. If you run them at stage G, run them inside a transaction you roll back (Section 8.4).
b. Write a query of stage G that previews the change by counting the customers each territory will hold.
c. Explain why adding the column as `NOT NULL` in one step would be refused at stage G, but accepted in a database with no customers yet.

Exercise 8.10. Northwind's `orders.freight` has the type `REAL`, a floating-point number. Using the Northwind database of Section C.11, loaded with the book's loader (Section A.6):

a. Compare the sum of `freight` as it is with the sum of each value rounded to cents (`freight::NUMERIC(10, 2)`), and count the values whose stored value is not a whole number of cents, by comparing each `REAL` value with its rounding (`freight <> freight::NUMERIC(10, 2)`).
b. Explain the difference, with Section 8.2, and explain why the values nevertheless look like whole numbers of cents when PostgreSQL prints them. Give the type you would use instead.

Exercise 8.11. The `meat_poultry_egg_establishments` table of the Practical SQL database of Section C.11 was loaded from the U.S. Department of Agriculture's directory. Some ZIP codes are shorter than five characters.

a. Count them by state, with an example of each, and explain how they lost their digits (Section 8.5).
b. Write the `UPDATE` statements that repair them, and say what you would check before and after running them inside a transaction you roll back (Section 8.4).

9. Views, Transactions, and Concurrency

Learning objectives

After reading this chapter you should be able to:

- define a view, use it in queries, and say when a view can be updated
- group statements into a transaction with BEGIN, COMMIT, and ROLLBACK, and explain the ACID properties
- recognize the lost update and the read anomalies of concurrent transactions, and choose an isolation level that prevents them
- grant and revoke privileges on tables and views

The chapters so far have treated a database as if one person used it at a time, and as if every statement stood alone. Neither is true in a working plant. Sales, shipping, and quality use the same tables at once, and a single piece of work, such as shipping an order, takes several statements that make sense only together. This chapter presents the database's answers: views, which give each user the queries they need under a name; transactions, which make several statements succeed or fail as one; isolation, which keeps concurrent users from spoiling each other's work; and privileges, which decide who may do what.

The chapter is brief, and it shows each idea once, mostly on HG Industries'

distribution schema at stages D3 to D5, and for privileges on the production schema of stage P1. The ideas matter to an engineer less as theory than as explanations: of why a report run twice gives two answers, why a statement waits, and why a program must sometimes try a change again.

Following along

Several results in this chapter come from a **schedule**: a sequence of statements run by two sessions, A and B, in the order the table shows, as two clerks at two terminals would run them. Each schedule runs against a copy of the book's database that is discarded afterward. To keep the statements short, a schedule names tables without their schema, as a session whose search path lists `distribution` may (Section 8.1). To try one yourself, open two query consoles in DataGrip or DBeaver, leave both in auto-commit, as in the earlier chapters, and run the statements in order, typing `BEGIN`, `COMMIT`, and `ROLLBACK` where the schedule shows them (Section 9.2).

9.1. Views

This section presents views: queries that the database keeps under a name.

A **view** is a named query stored in the database, which can be used in other queries as if it were a table. It stores no rows. Each time a query uses the view, the database runs the view's query and uses its result, so a view always reflects the tables as they are at that moment. Recall the flat order report of Section 7.2: normalization split it into four tables, and Section 7.6 observed that the report itself belongs in a query. A view gives that query a name:

```
CREATE VIEW distribution.order_report AS
SELECT o.order_number, c.customer_number,
       c.name AS customer_name, l.line_number,
       p.product_number, p.description, l.quantity
FROM distribution.customer_order AS o
JOIN distribution.customer AS c
  ON c.customer_pk = o.customer_fk
JOIN distribution.order_line AS l
  ON l.customer_order_fk = o.customer_order_pk
JOIN distribution.product AS p
  ON p.product_pk = l.product_fk;
```

The view is queried as a table is. A user who wants the lines of SO-5004 no longer writes the joins:

Table 9.1.: The Lines of SO-5004, from the Order Report View

```
SELECT order_number, customer_name, line_number,
       product_number, description, quantity
FROM distribution.order_report
WHERE order_number = 'SO-5004'
ORDER BY line_number;
```

order_number	customer_name	line_number	product_number	description	quantity
SO-5004	Riverside Hardware	1	HT-100	Two-wheel hand truck	20
SO-5004	Riverside Hardware	2	GC-300	Steel garden cart	5
SO-5004	Riverside Hardware	3	HT-100	Two-wheel hand truck	10

Because the view stores no rows, the anomalies of Section 7.2 cannot arise in it. A change to a product's description is made once, in `product`, and every row of the report shows it at once:

```
UPDATE distribution.product
SET description = 'Two-wheel steel hand truck'
WHERE product_number = 'HT-100';
```

Table 9.2.: The Lines of SO-5004 After the Description Changes

```
SELECT order_number, line_number, product_number,
       description
FROM distribution.order_report
WHERE order_number = 'SO-5004'
ORDER BY line_number;
```

order_number	line_number	product_number	description
SO-5004	1	HT-100	Two-wheel steel hand truck
SO-5004	2	GC-300	Steel garden cart
SO-5004	3	HT-100	Two-wheel steel hand truck

Both HT-100 lines of Table 9.2 show the new description, from the one row that changed. A view serves three purposes. It *simplifies*: a join or a calculation written once is used by name. It *insulates*: if the tables behind it are later split or renamed, the view can be redefined so that the queries built on it keep working. And it *restricts*: a user can be allowed to query a view without being allowed to query the tables behind it, which Section 9.4 takes up. A view that answers a standing question of the business is worth its name. For example, the warehouse manager's question "what needs reordering?" becomes, at stage D4:

```
CREATE VIEW distribution.stock_to_reorder AS
SELECT w.warehouse_code, p.product_number,
       s.quantity_on_hand, s.reorder_point
```

```
FROM distribution.stock_level AS s
JOIN distribution.warehouse AS w
  ON w.warehouse_pk = s.warehouse_fk
JOIN distribution.product AS p
  ON p.product_pk = s.product_fk
WHERE s.quantity_on_hand < s.reorder_point;
```

Table 9.3.: Stock Levels Below Their Reorder Points, Stage D4

```
SELECT * FROM distribution.stock_to_reorder
ORDER BY warehouse_code, product_number;
```

warehouse_code	product_number	quantity_on_hand	reorder_point
W-FAY	GC-300	18	25
W-TUL	GC-400	0	5

Can a view be changed, as a table can? A view is **updatable** when each of its rows comes from one row of one table, so that the database knows which row an `INSERT`, `UPDATE`, or `DELETE` on the view must change. In PostgreSQL, a view is updatable automatically when it selects from a single table and has none of `DISTINCT`, `GROUP BY`, `HAVING`, aggregates, window functions, `LIMIT` or `OFFSET`, a `WITH` clause, or set operations such as `UNION`. Even then, only the view's columns that are plain columns of the table can be changed through it. The order report joins four tables, and is not updatable:

```
DELETE FROM distribution.order_report
WHERE order_number = 'SO-5004';
```

```
ERROR:  cannot delete from view "order_report"
DETAIL:  Views that do not select from a single table or view
 ↪  are not automatically updatable.
HINT:  To enable deleting from the view, provide an INSTEAD
 ↪  OF DELETE trigger or an unconditional ON DELETE DO
 ↪  INSTEAD rule.
```

A single-table view is updatable, and it raises a question of its own. The view below shows the customers in Oklahoma, and a clerk who works only with that territory could add customers through it. Nothing yet stops the clerk from adding, through the view, a customer in Arkansas, a row the view itself would never show. The clause `WITH CHECK OPTION` makes the view refuse such a row:

```
CREATE VIEW distribution.oklahoma_customer AS
SELECT customer_number, name, city, state
FROM distribution.customer
WHERE state = 'OK'
WITH CHECK OPTION;
```

```
INSERT INTO distribution.oklahoma_customer
    (customer_number, name, city, state)
VALUES ('C-1009', 'Rogers Farm Supply', 'Rogers', 'AR');
```

```
ERROR:  new row violates check option for view
 ↪  "oklahoma_customer"
DETAIL:  Failing row contains (9, C-1009, Rogers Farm Supply,
 ↪  Rogers, AR).
```

A view stores no rows: its query runs, as part of the query that uses it, every time it is used, which is what keeps it current. A query of a view

therefore costs about what the same query would cost written out with the view's joins: PostgreSQL merges the view's query into the query that uses it, so that a condition such as `order_number = 'SO-5004'` is applied inside the joins. PostgreSQL also offers a **materialized view**, which stores its result and is brought up to date only when told to (`REFRESH MATERIALIZED VIEW`); it trades currency for speed, as the summary tables of Chapter 11 do. The next section turns from naming queries to grouping changes.

9.2. Transactions and the ACID properties

This section presents transactions: groups of statements that succeed or fail as one.

Shipping an order at HG Industries is several changes. The clerk records a shipment, a shipment line for each order line shipped, and a smaller quantity on hand for each product that left the warehouse. For order SO-5011 at stage D5, 30 HT-100 hand trucks and 12 GC-300 carts leaving Fayetteville, the changes are four statements; the stock levels of the two products in Fayetteville have keys 1 and 3 (`stock_level_pk`), and the order's lines have keys 19 and 20 (`order_line_pk`):

```
INSERT INTO distribution.shipment
    (shipment_number, shipped_on, carrier,
     customer_order_fk, warehouse_fk)
VALUES ('SH-9012', DATE '2026-03-10',
        'Boston Mountain Freight', 11, 1);

INSERT INTO distribution.shipment_line
    (shipment_fk, order_line_fk, quantity)
SELECT shipment_pk, 19, 30 FROM distribution.shipment
WHERE shipment_number = 'SH-9012'
UNION ALL
```

```
SELECT shipment_pk, 20, 12 FROM distribution.shipment
WHERE shipment_number = 'SH-9012';

UPDATE distribution.stock_level
SET quantity_on_hand = quantity_on_hand - 30
WHERE stock_level_pk = 1;

UPDATE distribution.stock_level
SET quantity_on_hand = quantity_on_hand - 12
WHERE stock_level_pk = 3;
```

The four statements belong together. A shipment recorded without its stock decrease overstates the stock; a stock decrease without its shipment loses carts from the books. Suppose that the clerk, not noticing that SO-5011 has already shipped, runs the four statements again, as shipment SH-9013. The warehouse had 18 GC-300 carts and now has 6, so the last statement drives the quantity below zero, and the check of `stock_level` refuses it. Run one at a time, as the tools run statements by default, the first three have already taken effect:

Table 9.4.: Shipping SO-5011 a Second Time, Statement by Statement

Step	Session	Statement	Result
1	A	`INSERT INTO shipment (shipment_number, shipped_on, carrier, customer_order_fk, warehouse_fk) VALUES ('SH-9013', DATE '2026-03-11', 'Boston Mountain Freight', 11, 1);`	1 row inserted
2	A	`INSERT INTO shipment_line (shipment_fk, order_line_fk, quantity) SELECT shipment_pk, 19, 30 FROM shipment WHERE shipment_number = 'SH-9013' UNION ALL SELECT shipment_pk, 20, 12 FROM shipment WHERE shipment_number = 'SH-9013';`	2 rows inserted
3	A	`UPDATE stock_level SET quantity_on_hand = quantity_on_hand - 30 WHERE stock_level_pk = 1;`	1 row updated
4	A	`UPDATE stock_level SET quantity_on_hand = quantity_on_hand - 12 WHERE stock_level_pk = 3;`	ERROR: new row for relation "stock_level" violates check constraint "stock_level_quantity_on_hand_check"
5	A	`SELECT COUNT(*) FROM shipment WHERE customer_order_fk = 11;`	2
6	A	`SELECT quantity_on_hand FROM stock_level WHERE stock_level_pk IN (1, 3) ORDER BY stock_level_pk;`	60; 6

The database of Table 9.4 now contradicts itself. It records two shipments of SO-5011, and a second 30 hand trucks gone from Fayetteville (90 became 60), but the 12 carts of the second shipment never left the stock (6 remain). Someone must find and repair the damage by hand.

A **transaction** is a sequence of statements that the database treats as one unit: either every change it makes takes effect, or none does. `BEGIN` (or the standard `START TRANSACTION`) opens a transaction; `COMMIT` ends it and makes its changes permanent; `ROLLBACK` ends it and discards them. In PostgreSQL, a statement that fails inside a transaction makes the whole transaction fail: the database refuses every later statement until the transaction ends, and a `COMMIT` then rolls it back. (The SQL standard, and several other systems, undo only the failed statement and let the

transaction go on.) The same mistake, made inside a transaction, leaves no trace:

Table 9.5.: Shipping SO-5011 a Second Time, Inside a Transaction

Step	Session	Statement	Result
1	A	`BEGIN;`	transaction begun
2	A	`INSERT INTO shipment (shipment_number, shipped_on, carrier, customer_order_fk, warehouse_fk) VALUES ('SH-9013', DATE '2026-03-11', 'Boston Mountain Freight', 11, 1);`	1 row inserted
3	A	`INSERT INTO shipment_line (shipment_fk, order_line_fk, quantity) SELECT shipment_pk, 19, 30 FROM shipment WHERE shipment_number = 'SH-9013' UNION ALL SELECT shipment_pk, 20, 12 FROM shipment WHERE shipment_number = 'SH-9013';`	2 rows inserted
4	A	`UPDATE stock_level SET quantity_on_hand = quantity_on_hand - 30 WHERE stock_level_pk = 1;`	1 row updated
5	A	`UPDATE stock_level SET quantity_on_hand = quantity_on_hand - 12 WHERE stock_level_pk = 3;`	ERROR: new row for relation "stock_level" violates check constraint "stock_level_quantity_on_hand_check"
6	A	`SELECT COUNT(*) FROM shipment WHERE customer_order_fk = 11;`	ERROR: current transaction is aborted, commands ignored until end of transaction block
7	A	`COMMIT;`	rolled back: the transaction had failed
8	A	`SELECT COUNT(*) FROM shipment WHERE customer_order_fk = 11;`	1
9	A	`SELECT quantity_on_hand FROM stock_level WHERE stock_level_pk IN (1, 3) ORDER BY stock_level_pk;`	90; 6

Read Table 9.5 step by step. The first three changes succeed, as before, but inside the transaction. The fourth is refused, and from then on the transaction has failed: even the harmless count of step 6 is refused, and the `COMMIT` of step 7 rolls everything back. Afterward SO-5011 has one shipment, and the stock stands at 90 and 6, as the first shipment left it.

The database is as it was before the clerk's mistake, and the clerk has an error message to act on.

An error typed in a console has the same effect on an open transaction. Until you type `ROLLBACK` (or `COMMIT`, which then rolls back), PostgreSQL refuses every statement with the message that the current transaction is aborted. The `ROLLBACK` undoes all of the transaction's work, not only the failed statement, and with it any setting made by `SET` inside the transaction, such as a search path or a role (Section 9.4). When a statement may fail and the work before it should survive, mark the point before it with a **savepoint**. Suppose that, after the first shipment of SO-5011, a clerk ships 30 more hand trucks from Fayetteville for another order, and 12 carts with them if the stock allows:

```
BEGIN;
UPDATE distribution.stock_level
SET quantity_on_hand = quantity_on_hand - 30
WHERE stock_level_pk = 1;
SAVEPOINT before_carts;
UPDATE distribution.stock_level
SET quantity_on_hand = quantity_on_hand - 12
WHERE stock_level_pk = 3;
ROLLBACK TO SAVEPOINT before_carts;
```

The second `UPDATE` is refused, since only 6 carts remain, and `ROLLBACK TO SAVEPOINT before_carts` undoes it alone. The transaction is usable again, with the hand trucks' decrease still in it. The clerk can ship the 6 carts that remain and commit, or end with `ROLLBACK` and undo everything.

The guarantees a transaction gives are summarized by four properties, known by their initials as **ACID**:

- **Atomicity**: the transaction's changes take effect all together or not at all, as Table 9.5 showed.

- **Consistency**: a transaction takes the database from a state that satisfies every rule to another that does: the constraints the database declares, and the business's rules that the transaction's statements carry out together. Each statement is checked against the declared constraints as it runs, and a transaction that cannot finish consistent is rolled back.
- **Isolation**: transactions running at the same time do not see each other's unfinished changes, and do not spoil each other's work, to the degree the isolation level promises (Section 9.3).
- **Durability**: once `COMMIT` returns, the changes survive a crash of the server or of the machine.

The first three properties can be seen in a schedule. Durability is the one you trust: PostgreSQL writes every change first to a **write-ahead log**, a file of changes in the order they were made, and, under its default settings, does not report a commit until the log records it, so that after a crash it can replay the log and recover every committed transaction. Durability does not protect against a mistaken `DELETE` that was committed, a failed disk, or a lost machine. For those you need a **backup**, a copy of the database made on purpose and kept elsewhere. PostgreSQL's `pg_dump` program writes a database to a single file, and `pg_restore` (or psql, for a plain SQL dump) rebuilds it, into a new database if need be; a backup that has never been restored should not be trusted.

A transaction also changes when some constraints are checked. Recall from Section 2.3 that PostgreSQL's `NO ACTION` and `RESTRICT` both refuse a deletion that would leave a foreign key pointing nowhere. The SQL standard has `RESTRICT` check at once and `NO ACTION` at the end of the statement. In PostgreSQL the difference that matters is another. A `NO ACTION` foreign key can be declared `DEFERRABLE INITIALLY DEFERRED`, and its check then waits until `COMMIT`, which lets a transaction pass through an inconsistent state on its way to a consistent one. A `RESTRICT` check is never deferred.

Two practical points follow. First, DataGrip and DBeaver run each statement in its own transaction, committed at once (*auto-commit*). Both

tools can switch a console to manual transactions, but this book keeps the consoles in auto-commit and types `BEGIN` where it wants a transaction. A `DELETE` typed in a console is therefore permanent the moment it runs; to experiment safely, type `BEGIN` first, and end with `ROLLBACK`. Second, the three inserts that recorded a new customer's order in Example 8.2 belong in one transaction, between `BEGIN` and `COMMIT`, for the reason this section has shown. The next section considers transactions that run at the same time.

9.3. Concurrency and isolation levels

This section considers transactions that run at the same time, and the isolation that keeps them from spoiling each other's work.

A database serves many sessions at once, and their transactions interleave: one clerk's statements run between another's. Most of the time they touch different rows and nothing goes wrong. Trouble comes when two transactions read and change the same rows, and the classic case is the **lost update**: two transactions read a value, each computes a new value from what it read, and the second to write overwrites the first, whose change is lost.

At stage D5 the Fayetteville warehouse holds 18 GC-300 carts (stock level 3). Clerk A ships the 12 carts of SO-5011, and at the same moment clerk B ships 10 carts for another order. Each reads the quantity on hand, subtracts, and writes the result back:

Table 9.6.: Two Clerks Ship GC-300 Carts at Once, Reading Before Writing

Step	Session	Statement	Result
1	A	`BEGIN;`	transaction begun
2	B	`BEGIN;`	transaction begun
3	A	`SELECT quantity_on_hand FROM stock_level WHERE stock_level_pk = 3;`	18
4	B	`SELECT quantity_on_hand FROM stock_level WHERE stock_level_pk = 3;`	18
5	A	`UPDATE stock_level SET quantity_on_hand = 6 WHERE stock_level_pk = 3;`	1 row updated
6	B	`UPDATE stock_level SET quantity_on_hand = 8 WHERE stock_level_pk = 3;`	waits: session A holds a lock on the row
7	A	`COMMIT;`	committed
8	B	`(step 6 completes)`	1 row updated
9	B	`COMMIT;`	committed
10	A	`SELECT quantity_on_hand FROM stock_level WHERE stock_level_pk = 3;`	8

Both clerks read 18 (steps 3 and 4). Clerk A writes 6, which is $18 - 12$. Clerk B's write must wait (step 6), because A's uncommitted change locks the row; when A commits, B's write goes ahead and sets 8, which is $18 - 10$. The books now say 8 carts remain, after 22 carts were shipped from 18. A's update is lost, and nothing refused anything: each statement was legal on its own.

Notice step 6. A transaction that changes a row holds a **lock** on it until the transaction ends, and another transaction that wants to change the same row waits. The lock did not prevent the lost update, because B's mistake was made earlier, when it read 18 and decided on 8. The repair is to let the database do the arithmetic, in a single statement that reads and writes the row while holding its lock, and to state the condition under which the change is allowed:

Table 9.7.: The Same Two Shipments, Letting the Database Subtract

Step	Session	Statement	Result
1	A	`BEGIN;`	transaction begun
2	B	`BEGIN;`	transaction begun
3	A	`UPDATE stock_level SET quantity_on_hand = quantity_on_hand - 12 WHERE stock_level_pk = 3 AND quantity_on_hand >= 12;`	1 row updated
4	B	`UPDATE stock_level SET quantity_on_hand = quantity_on_hand - 10 WHERE stock_level_pk = 3 AND quantity_on_hand >= 10;`	waits: session A holds a lock on the row
5	A	`COMMIT;`	committed
6	B	`(step 4 completes)`	0 rows updated
7	B	`ROLLBACK;`	rolled back
8	A	`SELECT quantity_on_hand FROM stock_level WHERE stock_level_pk = 3;`	6

B's update waits for A's lock, as before (step 4), and when A commits it reads the row again, as A left it: 6 carts, fewer than 10, so its condition fails and it updates no row (step 6). Clerk B's program learns that the stock is short, rolls back, and can offer a partial shipment or a backorder. The stock ends at 6, which is right. When a program must read a value and decide before it writes, as when it shows the quantity to a clerk, it can take the lock at the read with `SELECT ... FOR UPDATE`; a second transaction's `SELECT ... FOR UPDATE` of the same row then waits until the first ends.

The lost update is one of several ways concurrent transactions can interfere. The SQL standard names three more, which concern what a transaction *reads*:

- A **dirty read** reads a change that another transaction has made but not committed, and may yet roll back.
- A **nonrepeatable read** reads a row twice and gets two values, because another transaction changed and committed the row in between.

- A **phantom read** runs the same query twice and finds rows the second time that were not there the first, because another transaction inserted them.

One more interference involves no row that both transactions change. In a **write skew**, two transactions read the same rows, and each then changes a different row on the strength of what it read, so that the two changes together break a rule that each alone keeps. Table 9.11 shows one.

The **isolation level** of a transaction says which of these it is protected from. The standard defines four levels, and Table 9.8 gives what each allows, with PostgreSQL's behavior where it is stricter than the standard requires.

Table 9.8.: The Standard's Isolation Levels and the Anomalies Each Allows

Isolation level	Dirty read	Nonrepeatable read	Phantom read	Write skew
`READ UNCOMMITTED`	allowed (never in PostgreSQL)	allowed	allowed	allowed
`READ COMMITTED`	prevented	allowed	allowed	allowed
`REPEATABLE READ`	prevented	prevented	allowed (never in PostgreSQL)	allowed
`SERIALIZABLE`	prevented	prevented	prevented	prevented

PostgreSQL's default, and that of most systems, is `READ COMMITTED`: each statement sees the data as committed when the statement began. It is a sensible default for short transactions, and it allows the nonrepeatable read. Suppose that the production planner runs a report inside one transaction, totaling the GC-300 carts in both warehouses, while a shipment commits:

Table 9.9.: A Report's Total Changes Within One Transaction, Read Committed

Step	Session	Statement	Result
1	A	`BEGIN;`	transaction begun
2	A	`SELECT SUM(quantity_on_hand) FROM stock_level WHERE product_fk = 3;`	58
3	B	`UPDATE stock_level SET quantity_on_hand = quantity_on_hand - 12 WHERE stock_level_pk = 3;`	1 row updated
4	A	`SELECT SUM(quantity_on_hand) FROM stock_level WHERE product_fk = 3;`	46
5	A	`COMMIT;`	committed

The planner's two totals disagree, 58 and then 46, within one transaction. A report whose parts are computed by several queries, a total on one page and a breakdown on the next, can then fail to add up. Under `REPEATABLE READ`, the transaction sees the database as it was when its first statement ran, and every query in it sees the same data. The price is that such a transaction cannot change a row that another transaction changed after its first statement ran; it fails instead, and must be tried again:

Table 9.10.: A Repeatable-Read Transaction Fails, and Is Tried Again

Step	Session	Statement	Result
1	A	`BEGIN ISOLATION LEVEL REPEATABLE READ;`	transaction begun
2	A	`SELECT SUM(quantity_on_hand) FROM stock_level WHERE product_fk = 3;`	58
3	B	`UPDATE stock_level SET quantity_on_hand = quantity_on_hand - 12 WHERE stock_level_pk = 3;`	1 row updated
4	A	`SELECT SUM(quantity_on_hand) FROM stock_level WHERE product_fk = 3;`	58
5	A	`UPDATE stock_level SET quantity_on_hand = quantity_on_hand - 4 WHERE stock_level_pk = 3;`	ERROR: could not serialize access due to concurrent update
6	A	`ROLLBACK;`	rolled back
7	A	`BEGIN ISOLATION LEVEL REPEATABLE READ;`	transaction begun
8	A	`UPDATE stock_level SET quantity_on_hand = quantity_on_hand - 4 WHERE stock_level_pk = 3;`	1 row updated
9	A	`COMMIT;`	committed
10	A	`SELECT quantity_on_hand FROM stock_level WHERE stock_level_pk = 3;`	2

In Table 9.10, A's total is 58 both times (steps 2 and 4), although B's shipment has committed in between. When A then ships SO-5012's 4 carts from the same row, PostgreSQL refuses (step 5): A would be changing a row it has not seen in its current form. A tries again in a new transaction, which sees B's change, and the stock ends at $18 - 12 - 4 = 2$. A program that uses `REPEATABLE READ` or `SERIALIZABLE` must be written to retry a transaction that fails this way.

`REPEATABLE READ` refuses a change to a row that another transaction changed first. It does not stop two transactions that change *different* rows on the strength of what both read. Suppose that HG Industries keeps at least 30 GC-300 carts across its two warehouses: 18 in Fayetteville (stock level 3) and 40 in Tulsa (stock level 7). Two planners each check the total, 58, and each takes carts for a rush order. Planner A takes the 18 in Fayetteville, which leaves 40; planner B takes 25 in Tulsa, which leaves 33.

Each change keeps the rule, and the two together do not:

Table 9.11.: Two Planners Take GC-300 Carts at Once, Repeatable Read

Step	Session	Statement	Result
1	A	`BEGIN ISOLATION LEVEL REPEATABLE READ;`	transaction begun
2	B	`BEGIN ISOLATION LEVEL REPEATABLE READ;`	transaction begun
3	A	`SELECT SUM(quantity_on_hand) FROM stock_level WHERE product_fk = 3;`	58
4	B	`SELECT SUM(quantity_on_hand) FROM stock_level WHERE product_fk = 3;`	58
5	A	`UPDATE stock_level SET quantity_on_hand = quantity_on_hand - 18 WHERE stock_level_pk = 3;`	1 row updated
6	B	`UPDATE stock_level SET quantity_on_hand = quantity_on_hand - 25 WHERE stock_level_pk = 7;`	1 row updated
7	A	`COMMIT;`	committed
8	B	`COMMIT;`	committed
9	A	`SELECT SUM(quantity_on_hand) FROM stock_level WHERE product_fk = 3;`	15

Both planners read 58 (steps 3 and 4), both changes succeed, and both transactions commit. The total is now 15, half of the 30 the rule requires. Neither transaction changed a row that the other changed, so `REPEATABLE READ` found nothing to refuse. This is a write skew, and preventing it is what `SERIALIZABLE` adds. At `SERIALIZABLE`, PostgreSQL also keeps track of what each transaction has read, and refuses a transaction whose commit would give a result that no order of running the transactions one at a time could give. The same schedule at `SERIALIZABLE`:

Table 9.12.: The Same Two Planners, Serializable

Step	Session	Statement	Result
1	A	`BEGIN ISOLATION LEVEL SERIALIZABLE;`	transaction begun
2	B	`BEGIN ISOLATION LEVEL SERIALIZABLE;`	transaction begun
3	A	`SELECT SUM(quantity_on_hand) FROM stock_level WHERE product_fk = 3;`	58
4	B	`SELECT SUM(quantity_on_hand) FROM stock_level WHERE product_fk = 3;`	58
5	A	`UPDATE stock_level SET quantity_on_hand = quantity_on_hand - 18 WHERE stock_level_pk = 3;`	1 row updated
6	B	`UPDATE stock_level SET quantity_on_hand = quantity_on_hand - 25 WHERE stock_level_pk = 7;`	1 row updated
7	A	`COMMIT;`	committed
8	B	`COMMIT;`	ERROR: could not serialize access due to read/write dependencies among transactions
9	A	`SELECT SUM(quantity_on_hand) FROM stock_level WHERE product_fk = 3;`	40

In Table 9.12, planner A commits (step 7), and planner B's `COMMIT` fails (step 8): run after A's, B's transaction would have read 40, not 58. The total stays at 40. Planner B's program tries again, reads 40, and finds that taking 25 would break the rule. The price is the one `REPEATABLE READ` charges, a transaction that fails and must be retried, and PostgreSQL may refuse, to be safe, a transaction that would have done no harm. Thus, `SERIALIZABLE` is the level for a transaction that checks a rule spanning several rows before it changes one of them.

Two more facts complete the picture. First, PostgreSQL keeps several versions of a changed row, so that a reader sees the version its snapshot calls for; as a result, an ordinary read never waits for a write, and a write never waits for an ordinary read. A read that locks, such as `SELECT ... FOR UPDATE`, can wait. Two transactions that change the same row wait for each other, as B did in Table 9.6, and so do two that insert the same value of a `UNIQUE` column: the second waits until the first ends, then fails if the

first committed and goes ahead if it rolled back. Second, two transactions can wait for each other at once: A has changed row 1 and wants row 3, while B has changed row 3 and wants row 1. Neither can proceed, a situation called a **deadlock**. PostgreSQL detects it after a second or so and cancels one transaction with an error, which the program must retry. Programs that change several rows avoid most deadlocks by changing them in the same order, for example by key.

For an engineer who writes reports and programs against a shared database, three habits follow. Keep transactions short, since a transaction holds its locks until it ends. Let the database compute changes (`SET q = q - 12`) rather than writing back values computed elsewhere. And run a report that must add up within one transaction at `REPEATABLE READ`, and a transaction that checks a rule across several rows before it changes one at `SERIALIZABLE`. The next section turns from sessions that interfere by accident to users who are kept apart on purpose.

9.4. Security and privileges

This section presents how a database decides who may read and change what.

Every session connects to PostgreSQL as a **role**, a name that the database knows and grants privileges to. A role may be a person, a program, or a group: a role that can log in is a user, and a role granted to other roles is a group, whose privileges its members share. A **privilege** is permission to perform one kind of operation on one object: `SELECT`, `INSERT`, `UPDATE`, or `DELETE` on a table or view, and `USAGE` on a schema, which is needed to reach the tables in it at all. `GRANT` gives privileges, and `REVOKE` takes them back. A new role has none, apart from what is granted to everyone.

Privileges follow the jobs. A sales clerk at HG Industries reads customers and products, and records orders and their lines; the clerk has no reason to delete an order or to change a product. The statements below create a

group role for the sales department and grant it those privileges and no others:

```
CREATE ROLE hg_sales NOLOGIN;
GRANT USAGE ON SCHEMA distribution TO hg_sales;
GRANT SELECT ON distribution.customer,
                distribution.product TO hg_sales;
GRANT SELECT, INSERT ON distribution.customer_order,
                        distribution.order_line TO hg_sales;
```

`NOLOGIN` makes `hg_sales` a group: nobody logs in as it. Each clerk is a login role of their own, `CREATE ROLE ... LOGIN PASSWORD ...`, made a member of the group with `GRANT hg_sales TO` the clerk, so that the privileges are managed in one place and each session still identifies the clerk who opened it. A session can take on a role it belongs to with `SET ROLE`. Acting as `hg_sales`, a query of the orders succeeds:

```
SET ROLE hg_sales;
```

Table 9.13.: Orders Counted by a Session Acting as hg_sales

```
SELECT COUNT(*) AS orders
FROM distribution.customer_order;
```

orders
12

A deletion, which the role was not granted, is refused:

```
DELETE FROM distribution.customer_order
WHERE order_number = 'SO-5012';
```

```
ERROR:  permission denied for table customer_order
```

A privilege can be narrower than a table. The production department's `part` table holds each part's standard cost, which sales should not see. Privileges can be granted on some columns only, as in `GRANT SELECT (part_number, description) ON production.part TO hg_sales`; or a view can show the columns sales needs, and sales can be granted the view but not the table. A view is checked with the privileges of its owner, so a user who may read the view reads the table's rows through it without any privilege on the table. The query below, cut to the first four parts, runs as `hg_sales`; the one after it, which asks the table for costs, is refused:

```
CREATE VIEW production.part_catalog AS
SELECT part_number, description, unit_of_measure
FROM production.part;

GRANT USAGE ON SCHEMA production TO hg_sales;
GRANT SELECT ON production.part_catalog TO hg_sales;
```

Table 9.14.: The First Four Parts of the Catalog, Read as hg_sales

```
SELECT part_number, description, unit_of_measure
FROM production.part_catalog
ORDER BY part_number
LIMIT 4;
```

part_number	description	unit_of_measure
AX-10	Axle, 5/8 in	EA
BR-10	Ball bearing	EA
BS-075	Bar stock, 5/8 in	FT
CA-06	Swivel caster, 6 in	EA

```
SELECT part_number, standard_cost
FROM production.part;
```

```
ERROR:  permission denied for table part
```

PostgreSQL can go further, and restrict the *rows* a role may see or change with row-level security, so that, say, each warehouse's clerks see only their warehouse's stock. This book does not use it.

The same care applies to programs. A program that reads the database for a report should connect as a role that may only read; a program that records inspections, as Chapter 10 will write, as a role that may insert inspections and measurements and nothing more. No program should connect as the database's owner or as a superuser, and no password belongs in a script or in code kept under version control: the book's companion code reads its password from `db.properties`, a file that is never committed.

9.5. Summary

A view is a named query, and it is always current. It stores no rows; each use runs its query against the tables as they are. Views simplify queries, insulate them from changes to the tables, and restrict what a user sees. A simple view over one table, without grouping, `DISTINCT`, `LIMIT`, window functions, or set operations, can be updated, and `WITH CHECK OPTION` refuses a change the view could not show.

A transaction makes several statements succeed or fail as one. `BEGIN` opens it, `COMMIT` makes its changes permanent, and `ROLLBACK` discards them. In PostgreSQL a failed statement fails the whole transaction, and only `ROLLBACK`, or `ROLLBACK TO SAVEPOINT`, lets work go on. Its guarantees are atomicity, consistency, isolation, and durability, and durability is not a backup.

Concurrent transactions can spoil each other's work, and isolation limits the damage. Two transactions that read a value, compute, and write it back lose an update; letting the database compute the change, under a condition, prevents it. The isolation level decides which read anomalies a transaction can meet: `READ COMMITTED`, the default, allows nonrepeatable reads, and `REPEATABLE READ` prevents them at the price of transactions that fail and must be retried. `SERIALIZABLE` also prevents write skew, two transactions that each change a different row on the strength of what both read.

Privileges decide who may do what. A role is a user or a group; `GRANT` and `REVOKE` give and take privileges on schemas, tables, views, and columns. Grant each role only what its job needs, and use a view to show some columns or rows of a table without granting the table.

9.6. Exercises

Exercise 9.1. Match each term on the left with the phrase on the right that defines it.

Term	Definition
a. view	1. a group of statements whose changes take effect together or not at all
b. materialized view	2. a named query that stores no rows
c. transaction	3. a stored query result, brought up to date on request
d. atomicity	4. a file of changes that lets committed work survive a crash
e. write-ahead log	5. all of a transaction's changes, or none of them

Exercise 9.2. The shipping department asks, every morning, which orders have not yet shipped. Use stage D5.

a. Write a view `distribution.open_order` that lists the order number, the customer's name, the order date, and the requested ship date of every order that has no shipment.
b. Write the query the department runs against the view, oldest order first, and state the rows you expect before running it.
c. Is the view updatable in PostgreSQL? Explain from its definition.

Exercise 9.3. A clerk moves five GC-400 carts from Tulsa to Fayetteville on the books, running the statements below in DataGrip at stage D5, in a console in auto-commit mode. Before the clerk commits, the laptop loses its network connection, and the session ends.

```
BEGIN;
UPDATE distribution.stock_level
SET quantity_on_hand = quantity_on_hand - 5
WHERE stock_level_pk = 8;
UPDATE distribution.stock_level
SET quantity_on_hand = quantity_on_hand + 5
WHERE stock_level_pk = 4;
```

Stock level 8 is the GC-400 in Tulsa, which holds none, and stock level 4 is the GC-400 in Fayetteville, which holds 30.

a. What happens to the first `UPDATE`, and what does that do to the transaction?
b. What is the quantity on hand of the GC-400 in each warehouse afterward, and why?
c. What would it be had the clerk run the two updates without the `BEGIN`? Which property of a transaction does the difference illustrate?

Exercise 9.4. Name the problem each situation shows (a lost update, a dirty read, a nonrepeatable read, or a phantom read). For a, c, and d, give the lowest isolation level of Table 9.8 that prevents it in PostgreSQL. For b, give the change to the clerks' programs that prevents it at any isolation level.

a. A planner totals the open orders twice in one transaction, and the second total includes an order that a clerk entered and committed in between.
b. Two clerks each add 5 to the same stock level by reading it, adding 5 in their program, and writing the result; the level rises by 5.
c. A report reads the price of the HT-200 twice in one transaction and gets $139.00 and then $143.17, because a price change committed in between.
d. A report shows a price that a clerk had typed but then rolled back.

Exercise 9.5. At stage D5, Tulsa holds 60 HT-100 hand trucks (stock level 5) and Fayetteville 120 (stock level 1). Clerk A moves 20 from Tulsa to Fayetteville while clerk B ships 50 from Tulsa, both at `READ COMMITTED`. Clerk A's program reads Tulsa's quantity, subtracts 20, and writes the result; clerk B's program does the same with 50.

a. Write out an interleaving of the two clerks' statements, in the manner of Table 9.6, under which Tulsa ends at 10, and one under which it ends at 40. What should it be?
b. Rewrite each clerk's change so that no interleaving gives a wrong result, and so that a clerk learns when Tulsa holds too few.
c. Clerk A's move is two updates, one per warehouse. Why must they be in one transaction, and in what order would you make them to reduce the chance of a deadlock with another clerk moving hand trucks the other way?

Exercise 9.6. HG Industries' quality inspectors record inspections and their measurements. They must be able to read parts and their characteristics, and to add inspections and measurements, but not to change a characteristic's specification limits or delete an inspection.

a. Write the statements that create a group role `hg_quality` with these privileges, and a login role for the inspector Linda Nguyen that belongs to it. Use the tables of stage Q.
b. What happens when Linda Nguyen runs `UPDATE quality.characteristic SET upper_spec_limit = 2.0;`? Which part of your answer to a causes it?
c. The quality manager wants inspectors to see each part's number and description but not its standard cost. Give two ways to arrange it.

Part III.

Databases in Practice

10. Programming with Databases

Learning objectives

After reading this chapter you should be able to:

- connect to a PostgreSQL database from a Kotlin program, run a query, and read its result
- write parameterized statements, and explain why they prevent SQL injection
- organize a program's database work as create, read, update, and delete operations in a data access object, with a transaction where several changes belong together
- use the KSL database utilities to query a database, move rows into data classes and data frames, and summarize them
- move data between a database and CSV and Excel files from a program
- say what libraries such as jOOQ, Exposed, and Ktorm add, and when to reach for one

An engineer's programs meet a database for three reasons. A program records what a process produced: an inspection, a machine's cycle, a simulation run. A program reads data to analyze it, as a quality engineer reads measurements to judge a process. And a program moves data between the database and the files that other tools read and write. The chapters so far did all three by hand, in DataGrip or DBeaver; this chapter does them from Kotlin.

The chapter uses the quality schema of stage Q, and one running example: the quality lab. A gauge measures the parts of a new lot and writes its readings to a file; a program records the inspection and its measurements; another program reads a characteristic's measurements and summarizes them. The programs use the KSL, the Kotlin simulation library, whose database utilities rest on the Java standard for database access, JDBC, which the chapter presents first.

Following along

The programs are in the companion code, in the package `dbconcepts.programming` of `code/`, and each runs on its own: `./gradlew run -PmainClass=dbconcepts.programming.LotMeasurementsKt`, for example. They connect with the settings in `code/db.properties` (Section B.3) to your `dbconcepts` database. Each output shown in this chapter comes from running the program against a copy of that database, freshly loaded with `code/sql/load/through-q.sql`.

10.1. A program's view of a database

This section presents the pieces every database program uses, in the order a program uses them.

A program that works with a database does five things: it connects to the database, writes the SQL for what it needs, executes the SQL, uses the results, and closes what it opened. Java, and Kotlin with it, does all five through **JDBC** (Java Database Connectivity), a standard set of interfaces that every database system implements with a **driver**, a library that speaks that system's protocol. The KSL includes PostgreSQL's driver,

so a program that uses the KSL needs nothing more to reach the book's database. Three JDBC objects do the work:

- a **connection** (`Connection`) is an open session with the database, as a query console in DataGrip is; the program's statements run in it, and its transactions belong to it;
- a **prepared statement** (`PreparedStatement`) is an SQL statement with `?` in place of each value, which the program supplies separately before running it;
- a **result set** (`ResultSet`) is the rows a query returns, read by the program one row at a time.

Each of the three holds resources on the database server until it is closed, and a program that forgets to close them eventually exhausts the server's connections. Kotlin's `use` closes an object when the block that uses it ends, however it ends, so the book writes every one of them inside a `use` block.

Let's list the measurements of one lot. The program `LotMeasurements` first writes its query, with a `?` where the lot number goes:

```
val sql = """
    SELECT c.name, c.unit, m.measured_value
    FROM quality.measurement AS m
    JOIN quality.inspection AS i
      ON i.inspection_pk = m.inspection_fk
    JOIN quality.characteristic AS c
      ON c.characteristic_pk = m.characteristic_fk
    WHERE i.lot_number = ?
    ORDER BY c.name
""".trimIndent()
```

The query is ordinary SQL, of the kind Section 3.6 wrote: it joins `measurement` to `inspection` for the lot number and to `characteristic` for the name and unit. The program then does the other four things, each inside the `use` of the one before:

```
BookDatabase.connect().use { db ->
    db.getConnection().use { connection ->
        connection.prepareStatement(sql).use { statement ->
            statement.setString(1, lotNumber)
            statement.executeQuery().use { rows ->
                println("Measurements of lot $lotNumber")
                while (rows.next()) {
                    val name = rows.getString("name")
                    val unit = rows.getString("unit")
                    val value =
                     ↪  rows.getBigDecimal("measured_value")
                    println("  $name: $value $unit")
                }
            }
        }
    }
}
```

`BookDatabase.connect()` opens the book's database with the settings in `db.properties`, so that no password appears in the code, and `getConnection()` takes a JDBC connection from it. `prepareStatement(sql)` prepares the query, which PostgreSQL's driver sends to the database when the statement runs; `setString(1, lotNumber)` supplies the value of the first `?`; and `executeQuery()` runs it and returns the result set. The loop reads the rows. `rows.next()` moves to the next row and returns `false` when there is none, and `getString`, `getBigDecimal`, and their relatives read a column of the current row by its name, converted to the Kotlin type that matches its SQL type. Run against stage Q, the program prints:

```
Measurements of lot L7005-03
  Diameter: 0.6248 in
  Length: 12.0100 in
```

Notice the types. `measured_value` is `NUMERIC(10,4)`, and `getBigDecimal` reads it as a `BigDecimal`, Java's exact decimal number, so `12.0100` keeps its four decimal places; `getDouble` would have read it as floating point, with the consequences Section 8.2 described. Table 10.1 pairs the SQL types of the book's database with the method that reads each and the Kotlin type it returns.

Table 10.1.: Reading the Book's SQL Types from a Result Set

SQL type	Read with	Kotlin type
`INTEGER`	`getInt`	`Int`
`NUMERIC(p, s)`	`getBigDecimal`	`BigDecimal`
`VARCHAR(n)`, `CHAR(n)`, `TEXT`	`getString`	`String`
`DATE`	`getObject(column, LocalDate::class.java)`	`LocalDate`
`TIMESTAMP`	`getObject(column, LocalDateTime::class.java)`	`LocalDateTime`
`BOOLEAN`	`getBoolean`	`Boolean`

A NULL needs care. `getInt` returns 0 for a NULL, and `getBoolean` returns `false`, so a program cannot tell a missing value from a real zero unless it asks `rows.wasNull()` after reading, or reads with `getObject`, which returns `null`. The methods that return objects, `getString` and `getBigDecimal` among them, return `null` directly. The next section turns to the `?` of the query, which is more important than it looks.

10.2. Parameterized queries and SQL injection

This section shows why a program must never build SQL out of the values it is given, and what it does instead.

A program that searches for a lot receives the lot number from somewhere: a text box, a file, a scanned label. The obvious way to use it is to paste it into the SQL text, and Kotlin's string templates make that easy:

```
/** Wrong: the lot number becomes part of the SQL text. */
fun countMeasurementsUnsafely(
    connection: Connection, lotNumber: String
): Int {
    val sql = "SELECT COUNT(*) FROM quality.measurement AS m
     ↪  " +
        "JOIN quality.inspection AS i " +
        "ON i.inspection_pk = m.inspection_fk " +
        "WHERE i.lot_number = '$lotNumber'"
    connection.createStatement().use { statement ->
        statement.executeQuery(sql).use { rows ->
            rows.next()
            return rows.getInt(1)
        }
    }
}
```

The function builds the query `... WHERE i.lot_number = 'L7005-03'` and runs it, and for an ordinary lot number it works. It works, that is, until the lot number contains a quote. The value then ends the SQL string early, and what follows it is read as SQL. The program `LotSearch` runs the function on three inputs:

```
val inputs = listOf("L7005-03", "L7005-03' OR '1'='1",
 ↪  "L7005'03")
```

It then runs a second function on the same inputs, which differs from the first only in its last line of SQL, `WHERE i.lot_number = ?`, and in supplying the lot number with `setString`:

```
/** Right: the lot number is a parameter, sent apart from the
 ↪  SQL. */
fun countMeasurements(
    connection: Connection, lotNumber: String
): Int {
    val sql = "SELECT COUNT(*) FROM quality.measurement AS m
     ↪  " +
        "JOIN quality.inspection AS i " +
        "ON i.inspection_pk = m.inspection_fk " +
        "WHERE i.lot_number = ?"
    connection.prepareStatement(sql).use { statement ->
        statement.setString(1, lotNumber)
        statement.executeQuery().use { rows ->
            rows.next()
            return rows.getInt(1)
        }
    }
}
```

The two functions print:

```
Pasted into the SQL
  L7005-03 -> 2 measurements
  L7005-03' OR '1'='1 -> 34 measurements
  L7005'03 -> Unterminated string literal started at position
 ↪  142 in SQL SELECT COUNT(*) FROM quality.measurement AS m
 ↪  JOIN quality.inspection AS i ON i.inspection_pk =
 ↪  m.inspection_fk WHERE i.lot_number = 'L7005'03'. Expected
 ↪  char
As a parameter
  L7005-03 -> 2 measurements
  L7005-03' OR '1'='1 -> 0 measurements
  L7005'03 -> 0 measurements
```

The first input is a real lot, and both functions count its two measurements. The second is an attack. Pasted into the SQL, it makes the condition `i.lot_number = 'L7005-03' OR '1'='1'`, which is true of every row, and the query counts all 34 measurements of stage Q: the input has changed what the query does. The third input is merely a lot number with a mistyped apostrophe, and pasted into the SQL it leaves a string unterminated; the message, from the PostgreSQL driver, shows the broken SQL the function built. The second function treats each input as what it is, a lot number, finds no such lot, and counts zero.

Changing a query's meaning through the values supplied to it is called **SQL injection**, and it is among the most exploited weaknesses of real software. It is not limited to reading: an input that ends one statement and begins another can delete or change rows, with the program's privileges. The prepared statement prevents it by construction. The SQL, with its `?`, reaches the database first and is parsed; the values follow separately, and the database uses each as a value, never as SQL, whatever characters it contains. The same mechanism handles the ordinary case correctly: a customer named `Crowley's Ridge Hardware` needs no doubled quote (Section 8.4) when it is a parameter.

The rule is simple and has no exceptions: **a value that comes from outside the program goes into SQL only as a parameter.** Build the SQL text from constants alone, and supply every value with `setString`, `setInt`, `setBigDecimal`, or `setObject`. A table or column name cannot be a parameter; if a program must choose one at run time, it chooses from a fixed list written in the program, never from its input. The next section organizes a program's statements.

10.3. CRUD and the data access object

This section organizes a program's database work around the objects the program handles, and makes a group of changes one transaction.

Most of what a program does to a table falls into four operations, known by their initials as **CRUD**: *create* a row (`INSERT`), *read* rows (`SELECT`), *update* them (`UPDATE`), and *delete* them (`DELETE`). The programs of Section 10.1 and Section 10.2 wrote their SQL where they needed it, which is fine for a program of one query. It does not last. A program that records inspections, reports on them, and corrects them soon has the same joins and lookups in a dozen places, mixed with the code that decides what to do; a change to a table then means finding every one, and a reader of the program cannot see its purpose for the SQL.

10.3.1. The data access object pattern

Programmers meet this problem in every language, and they solve it the same way. A solution that recurs so often that it has a name is called a **design pattern**, and the one for this problem is the **data access object** (DAO): a class that holds all of the SQL for one kind of object, and offers the rest of the program methods named for what they do with it. A plant's tool crib works the same way. A machinist asks the crib for a tool by its number, and gets the tool; only the crib attendant knows which bin it is in, and when the crib is reorganized, nobody on the floor has to learn the new layout.

Figure 10.1 shows the parts of the pattern as a UML class diagram. It uses two pieces of UML notation that the class models of Chapter 5 did not need, because they describe software rather than the business.

First, a class box may have a third compartment, below its attributes, listing its **operations**: what an object of the class can be asked to do, with its parameters in parentheses. The data access object's operations are the four of CRUD, named for what they do: `create`, `find`, `update`, `delete`. Second, a dashed arrow is a **dependency**: the class at its tail uses the class at its head, calling its operations or handling its objects, without holding a lasting link to it. A solid arrow is an association, a link the object keeps. The line is the association line of 1; the arrowhead, which

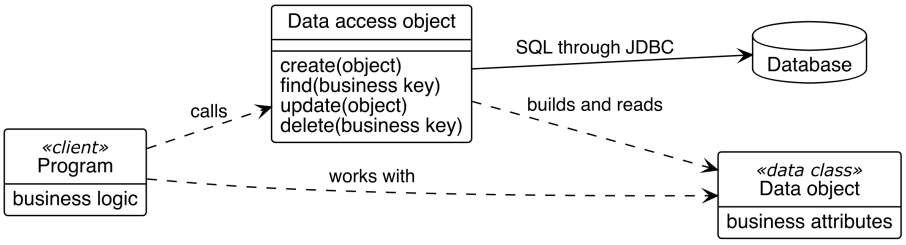


Figure 10.1.: The Data Access Object Pattern

the class models never needed, says which end knows the other: the object at the tail holds the link, and the one at the head does not know it. A word in guillemets, such as «client», is a **stereotype**: a label for the role a class plays. The cylinder is not a class at all but the database, drawn as UML tools conventionally draw one.

Read Figure 10.1 from left to right. The **program**, the client of the pattern, holds the business logic: what to record, when, and what to do if it fails. It works with **data objects**, plain data classes that hold the values of one thing, an inspection or a measurement, and know nothing about databases. When it needs something stored or retrieved, it calls the **data access object**, which builds data objects from rows and rows from data objects, and is the only class that sends SQL to the database. The program depends on the data access object, never on the database. Thus, the tables can change, or the database system can, and only the data access object changes with them.

The data access object's real work is translation. A data object describes a thing as the business does, by business keys, and a row describes it as the database does, by surrogate keys. Table 10.2 shows how the quality lab's inspection object becomes a row of `quality.inspection`. Recall from Chapter 6 that a class model was turned into tables once, at design time; a DAO performs the same correspondence at run time, every time an object is stored or read.

Table 10.2.: How an Inspection Object Becomes a Row of quality.inspection

Property of the object	Column of the row	How the DAO fills the column
`lotNumber`	`lot_number`	copies the value
`partNumber`	`part_fk`	looks up `part_pk` by `part_number`
`employeeNumber`	`operator_fk`	looks up `operator_pk` by `employee_number`
`inspectedAt`	`inspected_at`	copies the value
(none)	`inspection_pk`	the database generates it

10.3.2. The data access object for inspections

The data objects are Kotlin data classes, one per kind of thing, with the properties the program needs. For the quality lab they are an inspection and its measurements:

```
/** One inspection of a lot, as the quality lab knows it. */
data class Inspection(
    val lotNumber: String,
    val partNumber: String,
    val employeeNumber: String,
    val inspectedAt: LocalDateTime
)

/** One measurement taken in an inspection. */
data class Measurement(
    val characteristic: String,
    val value: BigDecimal
)
```

Notice what the classes leave out. They name the part, the inspector, and the characteristic by their business keys, a part number, an employee number, a characteristic's name, and have no surrogate keys at all; as in a class model (Section 5.3), an object's identity is not one of its attributes. Turning business keys into foreign keys is the DAO's job, as Table 10.2 showed. Figure 10.2 draws the quality lab's classes in the pattern of Figure 10.1.

Each box of Figure 10.2 plays one role of Figure 10.1. The program `RecordInspection` is the client, `Inspection` and `Measurement` are the data objects, and `InspectionDao` is the data access object, with an operation for each CRUD operation and a second read, `measurementsOf`. A sign before a member gives its **visibility**: `+` is public, part of what the rest of the program may call, and `-` is private, used only inside the class. `InspectionDao` keeps its `connection` private, so no other class can send SQL through it, and its solid arrow to `Connection`, JDBC's interface, is the one lasting link between classes in the diagram: a DAO holds one connection for its lifetime. The connection's own solid arrow to the database is the session it keeps open there. The operations' results follow a colon, as attributes' types do; `Inspection?` says that `findByLot` may find nothing and return `null`.

The `create` operation records an inspection and all of its measurements:

```
/** Records an inspection and its measurements, all or none.
 ↪  */
fun create(
    inspection: Inspection, measurements: List<Measurement>
): Int {
    connection.autoCommit = false
    try {
        val key = insertInspection(inspection)
        for (m in measurements) {
            insertMeasurement(key, inspection.partNumber, m)
```

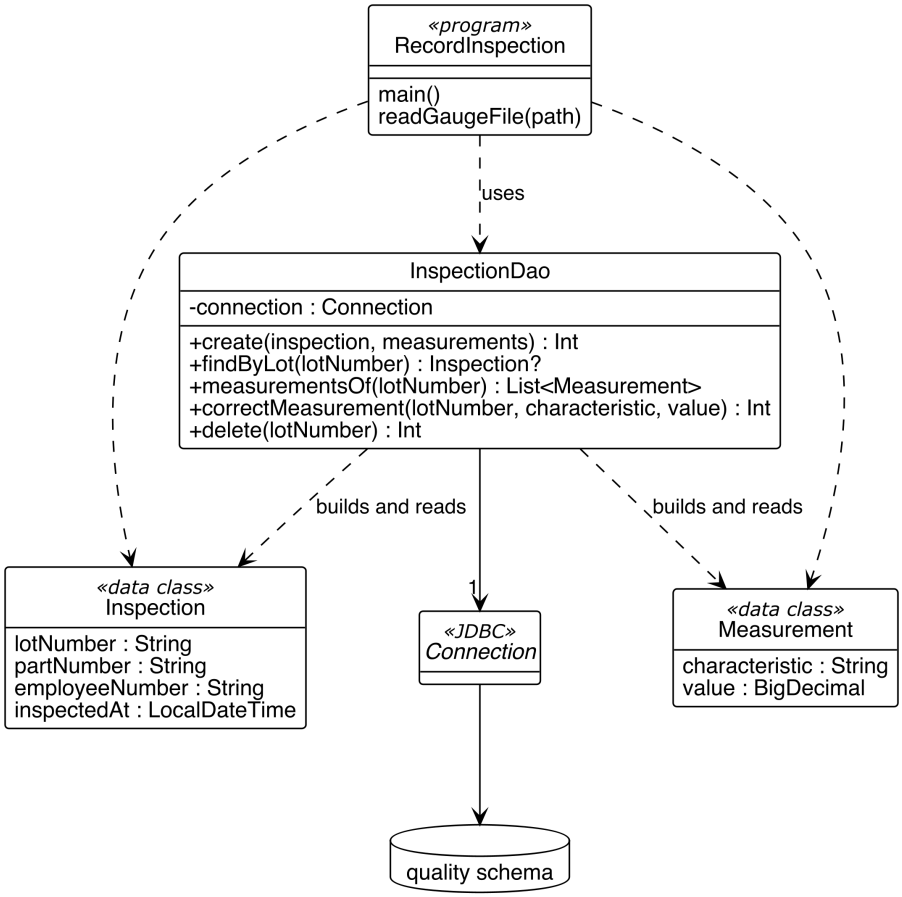


Figure 10.2.: The Quality Lab's Program, Data Access Object, and Data Classes

```
        }
        connection.commit()
        return key
    } catch (e: SQLException) {
        connection.rollback()
        throw e
    } finally {
        connection.autoCommit = true
    }
}
```

The method is a transaction (Section 9.2), written in JDBC's terms. A JDBC connection starts in auto-commit mode, as DataGrip's console does; setting `autoCommit` to `false` opens a transaction, `commit()` ends it, and `rollback()` discards it. If any statement fails, the `catch` rolls back everything the method did and passes the error on, and `finally` returns the connection to auto-commit however the method ends.

A class diagram shows what the classes are, but not the order in which they call each other. A UML **sequence diagram** shows that order, and Figure 10.3 draws one call of `create`.

Each object of Figure 10.3 is a box at the top with a dashed **lifeline** below it, and time runs down the page. A solid arrow is a message, a call of an operation or a statement sent; a dashed arrow is a reply. The narrow bar on the DAO's lifeline shows how long `create` is running. A frame labeled `loop` repeats its messages, once per measurement, and a frame labeled `alt` holds alternatives, each under a condition in brackets, of which one, and only one, happens. Read down the DAO's lifeline. It turns off auto-commit, inserts the inspection and receives its key, inserts each measurement, and then either commits, if every insert succeeded, or rolls back, if one failed. In both cases it restores auto-commit before it replies to the program with the key or the exception, since Kotlin runs `finally` before the method returns.

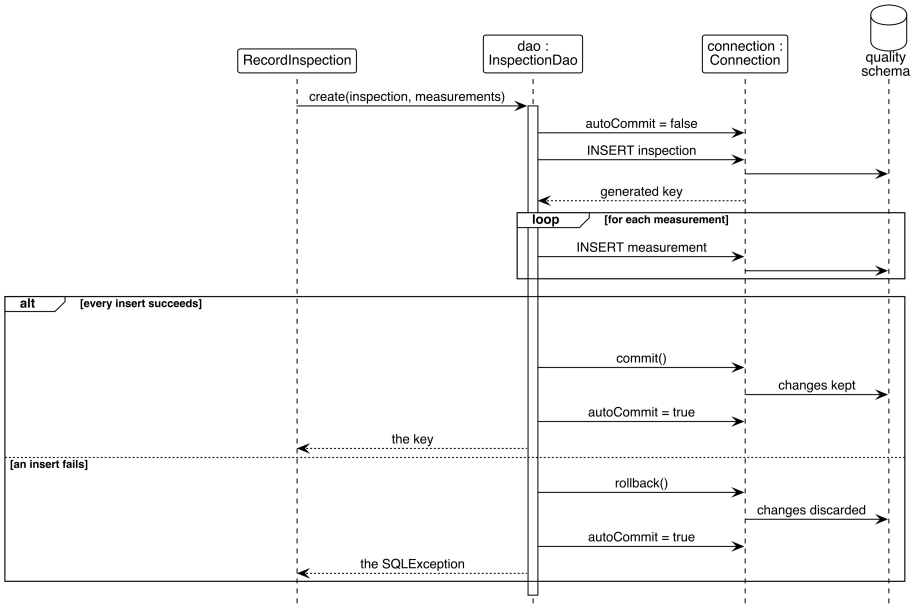


Figure 10.3.: The Messages of One Call of create

The inspection is inserted first, since the measurements refer to it:

```
private fun insertInspection(inspection: Inspection): Int {
    val sql = """
        INSERT INTO quality.inspection
            (part_fk, operator_fk, inspected_at, lot_number)
        VALUES ((SELECT part_pk FROM production.part
                 WHERE part_number = ?),
                (SELECT operator_pk FROM production.operator
                 WHERE employee_number = ?),
                ?, ?)
    """.trimIndent()
    val key = arrayOf("inspection_pk")
    connection.prepareStatement(sql, key).use { statement ->
        statement.setString(1, inspection.partNumber)
        statement.setString(2, inspection.employeeNumber)
        statement.setObject(3, inspection.inspectedAt)
        statement.setString(4, inspection.lotNumber)
        statement.executeUpdate()
        statement.generatedKeys.use { keys ->
            keys.next()
            return keys.getInt(1)
        }
    }
}
```

The two subqueries turn the part number and the employee number into `part_fk` and `operator_fk`, as Example 8.2 did by hand. The second argument of `prepareStatement`, `arrayOf("inspection_pk")`, asks the driver to return the key the database generates, and `generatedKeys` reads it; this is JDBC's standard way to learn a new row's key. `setObject` supplies the inspection time as a `LocalDateTime`, which the driver sends as a `TIMESTAMP`. Each measurement is then inserted with its characteristic

looked up by the part number and the characteristic's name, in the same way.

Reading returns objects, not rows. The method `findByLot` runs a query with the lot number as its parameter and builds an `Inspection` from the row it finds:

```
/** The inspection of a lot, or null if the lot has none. */
fun findByLot(lotNumber: String): Inspection? {
    val sql = """
        SELECT i.lot_number, p.part_number,
 ↪  o.employee_number,
               i.inspected_at
        FROM quality.inspection AS i
        JOIN production.part AS p ON p.part_pk = i.part_fk
        JOIN production.operator AS o ON o.operator_pk =
 ↪  i.operator_fk
        WHERE i.lot_number = ?
    """.trimIndent()
    connection.prepareStatement(sql).use { statement ->
        statement.setString(1, lotNumber)
        statement.executeQuery().use { rows ->
            if (!rows.next()) return null
            return Inspection(
                lotNumber = rows.getString("lot_number"),
                partNumber = rows.getString("part_number"),
                employeeNumber =
 ↪  rows.getString("employee_number"),
                inspectedAt = rows.getObject(
                    "inspected_at",
                     ↪  LocalDateTime::class.java)
            )
        }
    }
```

```
}
```

The other two operations follow the same pattern, each an SQL statement with parameters. `correctMeasurement` updates one measured value, and `delete` removes an inspection. Recall from Section 6.3.6 that only a composition's foreign key cascades. A measurement is not composed in its inspection (Figure C.3), so its foreign key does not cascade; `delete` therefore removes the inspection's measurements and defect counts first, and the inspection last, all in one transaction.

Example 10.1 (Recording an inspection from a gauge file)**.** After lot L7005-08 of the AX-10 axle measured above its upper diameter limit, the lathe was reset, and the next lot, L7005-09, was inspected by James Carter (employee E-202) on February 19, 2026. The gauge wrote its readings to `code/data/gauge-L7005-09.csv`:

```
lot_number,part_number,employee_number,inspected_at,characteristic,measured_valu
L7005-09,AX-10,E-202,2026-02-19 08:00,Diameter,0.6252
L7005-09,AX-10,E-202,2026-02-19 08:00,Length,12.0030
```

The next lot's file, `gauge-L7005-10.csv`, has the same form, but its second characteristic is spelled `Lenght`. The example assumes stage Q, which records lots through L7005-08.

The request. Record lot L7005-09's inspection from its file, read it back, correct its length, which the inspector misread as 12.0030 for 12.0003, and try to record lot L7005-10. Then delete lot L7005-09, so that the database ends as it began and each CRUD operation has run once.

The design. The program `RecordInspection` reads a gauge file into an `Inspection` and its `Measurement` objects with the KSL's `CSVUtil.readRows`, which Section 10.5 presents:

```
/** Reads a gauge's file: one inspection, and a measurement
 ↪  per line. */
fun readGaugeFile(path: Path): Pair<Inspection,
 ↪  List<Measurement>> {
    val stamp = DateTimeFormatter.ofPattern("yyyy-MM-dd
     ↪  HH:mm")
    val rows = CSVUtil.readRows(path, skipLines = 1)
    val first = rows.first()
    val inspection = Inspection(
        lotNumber = first[0],
        partNumber = first[1],
        employeeNumber = first[2],
        inspectedAt = LocalDateTime.parse(first[3], stamp)
    )
    val measurements = rows.map { Measurement(it[4],
     ↪  BigDecimal(it[5])) }
    return inspection to measurements
}
```

It then uses the DAO for each operation in turn. Creating and reading are two calls:

```
val dao = InspectionDao(connection)
val (inspection, measurements) =
    readGaugeFile(Path.of("data/gauge-L7005-09.csv"))
val key = dao.create(inspection, measurements)
println("Created inspection $key for lot
 ↪  ${inspection.lotNumber}")
```

```
println(dao.findByLot("L7005-09"))
for (m in dao.measurementsOf("L7005-09")) println("  $m")
```

The update corrects the length, and the attempt on the misspelled file is wrapped in a `try`, so that its failure is reported rather than ending the program:

```
val changed = dao.correctMeasurement(
    "L7005-09", "Length", BigDecimal("12.0003"))
val length = dao.measurementsOf("L7005-09")[1]
println("Updated $changed measurement: $length")
```

```
val (next, nextMeasurements) =
    readGaugeFile(Path.of("data/gauge-L7005-10.csv"))
try {
    dao.create(next, nextMeasurements)
} catch (e: SQLException) {
    println("Refused: ${e.message?.lineSequence()?.first()}")
}
val recorded = dao.findByLot("L7005-10") != null
println("Lot L7005-10 recorded: $recorded")
```

Last, the program deletes lot L7005-09:

```
val deleted = dao.delete("L7005-09")
println("Deleted $deleted inspection; " +
    "lot L7005-09 recorded: ${dao.findByLot("L7005-09") !=
     ↪ null}")
```

The result. Run against stage Q, the program prints:

```
Created inspection 21 for lot L7005-09
Inspection(lotNumber=L7005-09, partNumber=AX-10,
 ↪ employeeNumber=E-202, inspectedAt=2026-02-19T08:00)
```

```
  Measurement(characteristic=Diameter, value=0.6252)
  Measurement(characteristic=Length, value=12.0030)
Updated 1 measurement: Measurement(characteristic=Length,
 ↪  value=12.0003)
Refused: ERROR: null value in column "characteristic_fk" of
 ↪  relation "measurement" violates not-null constraint
Lot L7005-10 recorded: false
Deleted 1 inspection; lot L7005-09 recorded: false
```

The check. Stage Q holds 20 inspections, so the new one's generated key is 21, as the first line shows. The two measurements read back are the file's values, with their four decimal places. The failed file has two lines: the inspection and the diameter were inserted, the length's characteristic was not found, and its `NOT NULL` foreign key refused it; the rollback then removed the inspection and the diameter too, and lot L7005-10 is not recorded. After the delete, lot L7005-09 is gone, and stage Q is as it began.

Interpretation. The quality lab can now record a lot by dropping the gauge's file where the program reads it, with no one retyping numbers. A file with a mistake records nothing, rather than half a lot, and the message points to the problem: a line's characteristic was not found, so its key was NULL.

Where it breaks. The program trusts the file's lot number. A gauge file for a lot already recorded would be recorded again, as a second inspection, since the database does not declare `lot_number` unique (Section 7.5). A careful `create` first calls `findByLot` and refuses a lot it finds, or the business decides that a lot is inspected once and the database says so with a `UNIQUE` constraint.

A DAO takes a little more code than SQL scattered through a program, and repays it at once. The SQL for inspections is in one class, where a change to the tables is made once. The rest of the program reads as the business does, `dao.create(inspection, measurements)`, and never

handles a surrogate key. And the DAO is the natural place for a transaction, since it knows which changes belong together.

In a larger program, the DAO is often split in two: a Kotlin **interface** that lists its operations, which the rest of the program depends on, and a class that implements them with JDBC. The interface lets a second implementation take the first one's place, one that keeps inspections in memory for testing the program's logic without a database, for example, and the program cannot tell the difference. The book's DAO is a single class, which is enough for a program of this size. The next section shows what the KSL provides so that simple tables need no hand-written DAO at all.

10.4. The KSL database utilities

This section presents what the KSL adds on top of JDBC: a database object with methods for common tasks, CRUD for simple tables without hand-written SQL, and the step from a query to a data frame and to statistics.

The object that `BookDatabase.connect()` returns is a KSL `Database`, which implements the KSL's `DatabaseIfc` interface. Besides `getConnection()`, which the earlier sections used, it offers methods for the tasks an engineer does often, each a line of code: list a schema's tables (`tableNames`), count a table's rows (`numRows`), describe its columns (`tableMetaData`), print a table (`printTableAsText`), run a fixed SQL command or a whole script (`executeCommand`, `executeScript`), and run a query (`fetchCachedRowSet`, `fetchOpenResultSet`). The program `DefectTypes` looks around the quality schema:

```
db.defaultSchemaName = "quality"
println("Tables: ${db.tableNames("quality").sorted()}")
println("Rows of measurement: ${db.numRows("measurement")}")
```

```
db.printTableAsText("defect_type")
```

Setting `defaultSchemaName` tells the database which schema to use when a method is given a table name alone, so that the rest of the program can write `"measurement"` rather than `"quality.measurement"`. A method given a table name alone looks only in the default schema, and finding no table there is not an error: with the default schema set to `production`, `numRows("measurement")` returns 0, and the KSL only logs a warning. Thus, a program sets `defaultSchemaName` to the schema it works in, or names the schema whenever a method takes one. A command given to `executeCommand` is a finished string, like the unsafe function of Section 10.2, so it is for fixed SQL written in the program, never for SQL built from input.

For a simple table, the KSL can also do the CRUD work of a DAO without any SQL in the program. A data class that extends the KSL's `DbTableData` describes a table: its properties are the table's columns, named as the columns are and in the same order, and its constructor names the table, its key, and whether the database generates the key:

```
/** A row of quality.defect_type; the properties are named
 ↪  for its columns. */
data class DefectType(
    var defect_type_pk: Int = 0,
    var defect_code: String = "",
    var description: String = ""
) : DbTableData("defect_type", listOf("defect_type_pk"),
 ↪  autoIncField = true)
```

The property names break Kotlin's usual camel case on purpose: the KSL writes each property to the column of the same name, and the KSL's own tables are written the same way. It reads a row into the properties by

position, however, which is why they must be in the columns' order. Given the class, one call reads the table into a list of objects, one inserts an object, and one updates the row with the object's key:

```
val types = db.selectTableDataIntoDbData(::DefectType)
println("Read ${types.size} defect types; the first is
 ↪  ${types.first()}")
```

```
val warp = DefectType(defect_code = "WRP", description =
 ↪  "Warp")
db.insertDbDataIntoTable(warp)
println("Inserted $warp")
```

```
warp.description = "Warped frame"
db.updateDbDataInTable(warp)
println("Updated
 ↪  ${db.selectTableDataIntoDbData(::DefectType).last()}")
```

Run against stage Q, the program prints:

```
Tables: [characteristic, defect_count, defect_type,
 ↪  inspection, measurement]
Rows of measurement: 34
defect_type
+------------------+--------------+---------------+
|    defect_type_pk|    defect_code|    description|
+------------------+--------------+---------------+
|                 1|            SCR|        Scratch|
+------------------+--------------+---------------+
|                 2|            POR|  Weld porosity|
+------------------+--------------+---------------+
|                 3|            RUN|      Paint run|
```

```
+------------------+--------------+---------------+
|                 4|           DIM| Out of dime...|
+------------------+--------------+---------------+
|                 5|           BUR|           Burr|
+------------------+--------------+---------------+
|                 6|           CRK|          Crack|
+------------------+--------------+---------------+
Read 6 defect types; the first is
 ↪  DefectType(defect_type_pk=1, defect_code=SCR,
 ↪  description=Scratch)
Inserted DefectType(defect_type_pk=7, defect_code=WRP,
 ↪  description=Warp)
Updated DefectType(defect_type_pk=7, defect_code=WRP,
 ↪  description=Warped frame)
```

Notice the inserted object's key. `insertDbDataIntoTable` left `defect_type_pk` out of the `INSERT`, since the database generates it, and then set the property to the key the database chose, 7, so that the later update could find the row. Inside, the KSL builds prepared statements with a `?` for each property, so its CRUD is as safe from injection as the hand-written DAO. It is also limited to what one table can do: it knows nothing of business keys in other tables, joins, or transactions across tables. `DbTableData` suits a lookup table like `defect_type`; recording an inspection, which spans three tables and needs one transaction, is still the work of a DAO.

A query's result is often wanted not as objects but as a table of values to analyze, and the Kotlin library for such tables is Kotlin DataFrame, which the KSL uses. A **data frame** is an in-memory table of named, typed columns, the structure of R's and pandas' data frames, with operations to filter, group, and summarize it. The KSL's `DatabaseIfc.toDataFrame(resultSet)` turns a result set into one, and the example below takes the quality engineer's question from there to a

statistic.

Example 10.2 (The capability of the axle diameter)**.** Recall from Example 4.2 that the diameter a cutting tool leaves grows as the tool wears. At stage Q the diameter of the AX-10 axle drifted upward across its eight inspected lots, until lot L7005-08 measured above the upper specification limit, as Example 10.1 noted. The quality engineer must report to the plant manager how capable the turning process is of holding the diameter within its limits, 0.6200 to 0.6300 in.

The request. Summarize the eight diameters of stage Q: their number, mean, and standard deviation; how many lie outside the limits; and the process capability index C_{pk}, the distance from the mean to the nearer limit in units of three standard deviations, $C_{pk} = \min(\mathrm{USL} - \bar{x},\ \bar{x} - \mathrm{LSL})/(3s)$.

The design. The program `CharacteristicStatistics` selects each lot's diameter with the specification limits beside it:

```
val sql = """
    SELECT i.lot_number, m.measured_value,
           c.lower_spec_limit, c.upper_spec_limit
    FROM quality.measurement AS m
    JOIN quality.inspection AS i
      ON i.inspection_pk = m.inspection_fk
    JOIN quality.characteristic AS c
      ON c.characteristic_pk = m.characteristic_fk
    JOIN production.part AS p ON p.part_pk = c.part_fk
    WHERE p.part_number = 'AX-10' AND c.name = 'Diameter'
    ORDER BY i.lot_number
""".trimIndent()
```

It runs the query, turns the result set into a data frame, and takes the diameters out of the frame as an array of numbers:

```
val frame = db.fetchOpenResultSet(sql)?.use {
 ↪  DatabaseIfc.toDataFrame(it) }
    ?: error("the query failed")
val diameters = frame["measured_value"].values()
    .map { (it as Number).toDouble() }.toDoubleArray()
```

`toDataFrame` gives a `NUMERIC` or `DECIMAL` column the type `BigDecimal`, the exact decimal number that `getBigDecimal` returns, so the frame holds the measurements as stored, and a cast `as Double` fails. A `Statistic` needs `Double` values, so the program converts each value with `(it as Number).toDouble()`, which works whichever numeric type the column holds. The conversion makes the values floating point; for statistics that is harmless, and for anything that must add up to the cent, Section 8.2 applies. Printed, the frame shows each value as stored, with its four decimals:

```
println(frame)
```

A KSL `Statistic` collects the array and supplies the count, mean, and standard deviation; the program counts the values outside the limits and computes C_{pk}:

```
val stat = Statistic("AX-10 diameter", diameters)
val lsl = (frame["lower_spec_limit"][0] as Number).toDouble()
val usl = (frame["upper_spec_limit"][0] as Number).toDouble()
val outside = diameters.count { it < lsl || it > usl }
val cpk = minOf(usl - stat.average, stat.average - lsl) /
    (3.0 * stat.standardDeviation)
println("n = %.0f, mean = %.5f, standard deviation = %.5f"
    .format(stat.count, stat.average,
     ↪  stat.standardDeviation))
println("outside [%.4f, %.4f]: %d of %d; Cpk = %.2f"
    .format(lsl, usl, outside, diameters.size, cpk))
```

The result. Run against stage Q, the program prints:

```
  lot_number measured_value lower_spec_limit
↪  upper_spec_limit
0   L7005-01         0.6241           0.6200
↪  0.6300
1   L7005-02         0.6244           0.6200
↪  0.6300
2   L7005-03         0.6248           0.6200
↪  0.6300
3   L7005-04         0.6251           0.6200
↪  0.6300
4   L7005-05         0.6257           0.6200
↪  0.6300
5   L7005-06         0.6266           0.6200
↪  0.6300
6   L7005-07         0.6279           0.6200
↪  0.6300
7   L7005-08         0.6304           0.6200
↪  0.6300

n = 8, mean = 0.62612, standard deviation = 0.00213
outside [0.6200, 0.6300]: 1 of 8; Cpk = 0.61
```

The check. The eight diameters sum to 5.0090, so their mean is 5.0090 / 8 = 0.626125. Rounded by hand to five decimals that is 0.62613, but the program prints 0.62612: the `Statistic` updates its mean one value at a time in floating point and arrives at 0.6261249999999999, just below the halfway point. Only lot L7005-08, at 0.6304, lies outside the limits. The nearer limit is the upper one: $(0.6300 - 0.626125) \,/\, (3 \times 0.00213) = 0.61$.

Interpretation. A capable process has C_{pk} of at least 1, and most customers ask for 1.33. At 0.61 the turning process cannot hold the diameter within its limits, and the printed frame shows why: the mean is not merely off center

but moving, lot after lot. The engineer's report is that the lathe needs adjustment, which is what the reset before lot L7005-09 of Example 10.1 did.

Where it breaks. C_{pk} assumes a stable process whose values scatter around a fixed mean. A drifting process is not stable, so the index here describes the eight lots taken together, not the process as it will run; the engineer should say so, or compute it over a stable stretch. The query also assumes one diameter per lot. The database refuses a second diameter in one inspection, but not a second inspection of a lot, since `lot_number` is not unique; a lot inspected twice would be counted twice, and the query would return nine rows.

The steps of Example 10.2, a query, a result set, a data frame, and a statistic, are the path by which a database feeds any analysis in Kotlin, and Chapter 12 follows it further. The next section moves data in bulk between the database and files.

10.5. Importing and exporting data

This section moves whole tables between the database and files, from a program, which Section 8.5 did with psql and the tools.

The KSL writes a table to a CSV file with `exportTableAsCSV`, to the console with `printTableAsCSV`, and a whole schema to an Excel workbook, one sheet per table, with `exportToExcel`; `importWorkbookToSchema` reads a workbook back into tables. For reading a CSV file, `CSVUtil.readRows` returns its lines as records whose fields are read by position, `row[0]`, `row[1]`, and so on. It follows the CSV rules of Section 8.5, so a quoted value that contains a comma stays one field, which splitting each line at its commas would not do.

The program `RoundTrip` takes the measurements out of the database and back in. It first exports the table:

```
val file = Path.of("build/measurement.csv")
PrintWriter(file.toFile()).use { out ->
    db.exportTableAsCSV("measurement", "quality", out, header
 ↪  = true)
}
val lines = Files.readAllLines(file)
println("Exported ${lines.size - 1} rows; the file begins:")
for (line in lines.take(3)) println("  $line")
```

It then loads the file into a staging table, as Section 8.5 recommended, created by a fixed command:

```
db.executeCommand(
    "CREATE TABLE quality.measurement_staging (" +
        "inspection_fk INTEGER, characteristic_fk INTEGER, "
         ↪  +
        "measured_value NUMERIC(10,4))"
)
```

Loading many rows one `executeUpdate` at a time sends a statement to the server and waits for its answer, row after row. A **batch** sends them together: the program sets a prepared statement's parameters for a row, adds the row to the batch with `addBatch()`, and after the last row sends them all with `executeBatch()`. Made one transaction, the load is also all or nothing:

```
val rows = CSVUtil.readRows(file, skipLines = 1)
db.getConnection().use { connection ->
    connection.autoCommit = false
    val sql = "INSERT INTO quality.measurement_staging VALUES
     ↪  (?, ?, ?)"
    connection.prepareStatement(sql).use { statement ->
```

```
        for (row in rows) {
            statement.setInt(1, row[1].toInt())
            statement.setInt(2, row[2].toInt())
            statement.setBigDecimal(3, BigDecimal(row[3]))
            statement.addBatch()
        }
        val counts = statement.executeBatch()
        println("Sent ${counts.size} inserts in one batch")
    }
    connection.commit()
}
println("Rows staged: ${db.numRows("measurement_staging")}")
```

Last, it writes the schema to a workbook:

```
db.exportToExcel("quality", "quality", Path.of("build"))
println("Wrote a workbook to build/, a sheet per table")
```

Run against stage Q, the program prints:

```
Exported 34 rows; the file begins:

 ↪  measurement_pk,inspection_fk,characteristic_fk,measured_value
  1,1,1,0.6241
  2,1,2,12.0040
Sent 34 inserts in one batch
Rows staged: 34
Wrote a workbook to build/, a sheet per table
```

The export wrote the file's header from the table's column names and each value as the database has it, `12.0040` with its four decimals. The 34 rows came back through one batch, and the staging table holds 34 rows, the

count of Section 10.4. The workbook, `quality.xlsx`, has a sheet for each table of the schema, six with the staging table, ready for a colleague who works in a spreadsheet. The next section looks at libraries that write a program's SQL for it.

10.6. Libraries that write the SQL

This section compares three libraries that build SQL from Kotlin code, and says when each is worth its cost.

The programs of this chapter write their SQL as strings and read their results by column name, and the compiler checks neither: a misspelled column is found when the program runs. Libraries exist to move those mistakes to compile time, and to spare the program some of the repetitive code of the DAO. Three are common in Kotlin:

- **jOOQ** builds SQL from Kotlin (or Java) method calls that mirror SQL's own clauses, `select`, `from`, `join`, `where`. Its main strength is a code generator that reads the database's schema and writes a class for every table and a typed field for every column, so that the compiler knows the database.
- **Exposed**, from JetBrains, the makers of Kotlin, describes each table once as a Kotlin object and builds SQL from those objects with a Kotlin DSL (a small language made of Kotlin functions); it also offers a DAO layer that maps rows to entity classes.
- **Ktorm** is a lighter library of the same kind: tables described as Kotlin objects, a DSL for queries, and entity classes for CRUD.

To compare them, the companion code writes the same two operations in each: the measurements of lot L7005-03, as `LotMeasurements` of Section 10.1 found them, and a new defect type, as `DefectTypes` of Section 10.4 added it. The programs are in their own source set, `code/src/libraries/kotlin`, so that the chapter's other programs

depend on the KSL alone, and each runs with `./gradlew runLibrary -PmainClass=...`.

With jOOQ, used here without generated classes, tables and columns are named in strings, and the query is assembled clause by clause:

```
val sql = DSL.using(connection, SQLDialect.POSTGRES)
val m = table(name("quality", "measurement")).`as`("m")
val i = table(name("quality", "inspection")).`as`("i")
val c = table(name("quality", "characteristic")).`as`("c")
val characteristic = field(name("c", "name"),
 ↪  String::class.java)
val value = field(name("m", "measured_value"),
 ↪  BigDecimal::class.java)
val query = sql.select(characteristic, value)
    .from(m)
    .join(i).on(field(name("i", "inspection_pk"))
        .eq(field(name("m", "inspection_fk"))))
    .join(c).on(field(name("c", "characteristic_pk"))
        .eq(field(name("m", "characteristic_fk"))))
    .where(field(name("i", "lot_number"), String::class.java)
        .eq("L7005-03"))
    .orderBy(characteristic)
println(query.sql)
for (row in query.fetch()) println("  ${row.value1()}:
 ↪  ${row.value2()}")
```

```
val defectType = table(name("quality", "defect_type"))
sql.insertInto(defectType,
    field(name("defect_code")), field(name("description")))
    .values("WRP", "Warp")
    .execute()
println("Defect types: ${sql.fetchCount(defectType)}")
```

The program prints the SQL jOOQ built before running it:

```
select "c"."name", "m"."measured_value" from
 ↪  "quality"."measurement" as "m" join
 ↪  "quality"."inspection" as "i" on "i"."inspection_pk" =
 ↪  "m"."inspection_fk" join "quality"."characteristic" as
 ↪  "c" on "c"."characteristic_pk" = "m"."characteristic_fk"
 ↪  where "i"."lot_number" = ? order by "c"."name"
  Diameter: 0.6248
  Length: 12.0100
Defect types: 7
```

Notice the `?` in the SQL: jOOQ sends the lot number as a parameter, as a prepared statement would, and so do the other two libraries. With the code generator, the strings `"quality"`, `"measurement"`, and `"measured_value"` would become generated names such as `MEASUREMENT.MEASURED_VALUE`, checked by the compiler.

With Exposed, the tables are described first, each column with its SQL type:

```
object Inspection : Table("quality.inspection") {
    val inspectionPk = integer("inspection_pk")
    val lotNumber = varchar("lot_number", 12)
}

object Characteristic : Table("quality.characteristic") {
    val characteristicPk = integer("characteristic_pk")
    val name = varchar("name", 40)
}

object Measurement : Table("quality.measurement") {
    val inspectionFk = integer("inspection_fk")
```

```
    val characteristicFk = integer("characteristic_fk")
    val measuredValue = decimal("measured_value", 10, 4)
}

object DefectType : Table("quality.defect_type") {
    val defectTypePk =
     ↪  integer("defect_type_pk").autoIncrement()
    val defectCode = varchar("defect_code", 4)
    val description = varchar("description", 40)
}
```

The query and the insert then use those objects, inside Exposed's `transaction`, which also makes them one transaction:

```
Measurement
    .join(Inspection, JoinType.INNER,
        Measurement.inspectionFk, Inspection.inspectionPk)
    .join(Characteristic, JoinType.INNER,
        Measurement.characteristicFk,
 ↪  Characteristic.characteristicPk)
    .select(Characteristic.name, Measurement.measuredValue)
    .where { Inspection.lotNumber eq "L7005-03" }
    .orderBy(Characteristic.name)
    .forEach { row ->
        println("  ${row[Characteristic.name]}: " +
            "${row[Measurement.measuredValue]}")
    }
```

```
DefectType.insert {
    it[defectCode] = "WRP"
    it[description] = "Warp"
}
```

```
println("Defect types: ${DefectType.selectAll().count()}")
```

```
  Diameter: 0.6248
  Length: 12.0100
Defect types: 7
```

Ktorm reads much the same. Its tables are objects like Exposed's, and its query names the join condition in each join:

```
database.from(Measurements)
    .innerJoin(Inspections,
        on = Measurements.inspectionFk eq
 ↪  Inspections.inspectionPk)
    .innerJoin(Characteristics,
        on = Measurements.characteristicFk eq
 ↪  Characteristics.characteristicPk)
    .select(Characteristics.name, Measurements.measuredValue)
    .where { Inspections.lotNumber eq "L7005-03" }
    .orderBy(Characteristics.name.asc())
    .forEach { row ->
        println("  ${row[Characteristics.name]}: " +
            "${row[Measurements.measuredValue]}")
    }
```

```
database.insert(DefectTypes) {
    set(it.defectCode, "WRP")
    set(it.description, "Warp")
}
val types = database.from(DefectTypes).select(count())
    .map { it.getInt(1) }.first()
println("Defect types: $types")
```

```
  Diameter: 0.6248
  Length: 12.0100
Defect types: 7
```

All three print the same diameter and length as Section 10.1, and all three count seven defect types after the insert. Table 10.3 summarizes what each approach asks of you and what it gives back.

Table 10.3.: Four Ways for a Kotlin Program to Work with a Database

Approach	You write	The compiler checks	Best for
JDBC	SQL as strings	nothing in the SQL	learning; short programs; full control
KSL `Database`	SQL for queries; data classes for simple tables	property types	analysis and simulation programs; the book's examples
jOOQ	Kotlin calls that mirror SQL	with generated code, every table and column	large programs over a stable schema
Exposed, Ktorm	table objects, then a Kotlin DSL	columns and their types	Kotlin applications with many tables and CRUD

Each library is one more thing to learn, and each hides the SQL behind its own vocabulary; when a query misbehaves, you still debug it as SQL. For the programs an industrial engineer writes, to record data, analyze it, and feed a simulation, SQL through JDBC and the KSL is enough, and it

keeps the SQL, which the rest of this book taught, in view. Reach for a library when a program grows into an application with many tables that several people maintain, and choose by what the team knows.

10.7. Summary

A program reaches a database through a connection, a statement, and a result set. JDBC's `Connection`, `PreparedStatement`, and `ResultSet` do the five tasks of every database program: connect, write the SQL, execute it, use the results, and close what was opened, which Kotlin's `use` makes automatic. Read each column with the method that matches its SQL type, `getBigDecimal` for `NUMERIC`, and watch for NULL.

A value from outside the program enters SQL only as a parameter. Pasting a value into SQL text lets the value change the statement, which is SQL injection; a prepared statement sends the SQL and the values separately, so a value is only ever a value.

A data access object gathers a table's CRUD operations in one class. It offers create, read, update, and delete as methods on objects named by their business keys, holds all of the SQL for one kind of object, and makes the changes that belong together one transaction.

The KSL's `Database` does the common tasks in a line each. It lists and counts tables, prints and exports them, runs fixed commands and scripts, and runs queries. A `DbTableData` data class, its properties named for the table's columns, gives simple CRUD without SQL; a DAO is still needed where the work spans tables or needs a transaction.

A query becomes a data frame, and a data frame becomes statistics. `DatabaseIfc.toDataFrame` turns a result set into a data frame of typed columns, and a KSL `Statistic` summarizes a column. A `NUMERIC` column has the type `BigDecimal`, the values as stored,

and a `Statistic` needs `Double` values, so convert each with `(it as Number).toDouble()`.

Files move in and out through the KSL and a batch. Export a table to CSV or a schema to Excel with one call; read CSV with `CSVUtil`, which respects quoting; and load many rows with a batch of prepared inserts in one transaction, into a staging table first.

Libraries such as jOOQ, Exposed, and Ktorm write the SQL from Kotlin. They let the compiler check tables and columns and remove repetitive code, at the price of one more vocabulary over the SQL. For an engineer's programs, JDBC and the KSL are enough; a library earns its place in a large application.

10.8. Exercises

Exercise 10.1. Match each term on the left with the phrase on the right that defines it.

Term	Definition
a. driver	1. the rows a query returns, read one at a time
b. prepared statement	2. a class that holds the SQL for one kind of object
c. result set	3. a library that speaks one database system's protocol
d. SQL injection	4. SQL sent with `?` in place of its values
e. data access object	5. changing a statement's meaning through the values it is given

Exercise 10.2. Mark each statement true or false, and correct each false one.

a. A prepared statement is only needed when the program's input comes from a user who might be malicious.
b. `rows.getInt("quantity")` returns `null` when the quantity is NULL.
c. A JDBC connection starts in auto-commit mode, so each statement is its own transaction until the program sets `autoCommit` to `false`.
d. A table name can be supplied to a prepared statement as a parameter.
e. Reading a `NUMERIC(10,4)` column with `getDouble` can change the value the program sees.

Exercise 10.3. A colleague writes this function to find a customer's city at stage D1:

```
fun cityOf(connection: Connection, name: String): String? {
    val sql = "SELECT city FROM distribution.customer " +
        "WHERE name = '$name'"
    connection.createStatement().use { statement ->
        statement.executeQuery(sql).use { rows ->
            return if (rows.next()) rows.getString("city")
             ↪  else null
        }
    }
}
```

a. What does the function do when `name` is `Riverside Hardware`? When it is `Crowley's Ridge Hardware`, a customer of stage G?
b. Give a value of `name` that makes the function return the city of a customer whose name the caller did not know.
c. Rewrite the function with a prepared statement.

Exercise 10.4. Add to `InspectionDao` a method `lotsInspectedBy(employeeNumber: String): List<String>` that returns the lot numbers an operator inspected, in the order the inspections took place. Use a prepared statement,

and test the method at stage Q: which lots did James Carter (E-202) inspect?

Exercise 10.5. Design a data access object for the customers of stage D1, the table `distribution.customer`, whose business key is the customer number.

a. Write the data class `Customer` with the properties the program needs, and say which column of the table it leaves out, and why.
b. List the operations of `CustomerDao`, with their parameters and results, in the manner of Figure 10.2: create a customer, find one by customer number, list the customers of a state, change a customer's city, and delete a customer.
c. Draw the class diagram of the program, the DAO, the data class, and the connection, as Figure 10.2 does.
d. Which of the operations of b can the database refuse at stage D2, where orders refer to customers, and which constraint refuses it?

Exercise 10.6. Write a `DbTableData` data class for the table `quality.defect_count` of stage Q, whose columns are `defect_count_pk`, `inspection_fk`, `defect_type_fk`, and `quantity`, all integers, with a generated key. Use it to read the table, and compute in Kotlin the total number of paint runs (defect type 3) found in all inspections.

Exercise 10.7. Change `CharacteristicStatistics` of Example 10.2 to summarize the length of the AX-10 axle instead of its diameter; the specification limits are 11.9500 and 12.0500 in. Report the mean, the standard deviation, the number of lots outside the limits, and C_{pk}, and say whether the process is capable of holding the length.

Exercise 10.8. The program `RoundTrip` of Section 10.5 loads its file into a staging table rather than into `quality.measurement`.

a. What would happen if it inserted the exported rows into `quality.measurement` itself, with the same `INSERT`, which names no columns? Name the column property or constraint that refuses it. Which one would refuse it if the `INSERT` named the three columns it loads, `(inspection_fk, characteristic_fk, measured_value)`?
b. The exported file includes `measurement_pk`. Why does the batch not load that column?
c. Suppose the batch's twentieth insert failed. How many rows would the staging table hold afterward, and why?

Exercise 10.9. For each program below, say which approach of Table 10.3 you would choose, and give a reason.

a. A one-time analysis of last year's inspection data, run by one engineer.
b. A KSL simulation model that writes each run's results to a database.
c. A web application for the quality lab, with forms for inspections, defect counts, and specifications, maintained by a team for several years.

11. Data Warehousing and Decision Support

Learning objectives

After reading this chapter you should be able to:

- contrast an operational database with a data warehouse, and say what each is designed for
- design a star schema by choosing a business process, a grain, dimensions, and facts
- explain why a warehouse's dimension tables are denormalized, and when a measure can be summed
- describe an extract, transform, and load process, and load a warehouse incrementally
- keep or overwrite a dimension's history when it changes
- write roll-up, drill-down, slice, and dice queries with ROLLUP, CUBE, and GROUPING SETS

Every database in this book so far has been designed to *run* the business: to take an order, ship it, inspect a lot. Such a database answers the questions of the moment well, which order is late, what is in stock, and it answers the questions of a manager poorly. How did sales of hand trucks compare with garden carts, quarter by quarter, in each sales region? The answer is in the operational tables, spread across a year of orders, several schemas, and attributes that no table stores.

This chapter presents the other kind of database, the **data warehouse**, which is designed to *analyze* the business. It is built from the operational databases, holds their history, and is shaped for the questions managers and engineers ask of history. HG Industries' warehouse is the `sales` schema of stage S, which the chapter designs, loads, and queries.

> **Following along**
>
> The warehouse is the `sales` schema of your `dbconcepts_history` database (stage S, loaded with `code/sql/load/through-s.sql`), built from the operational data and year of history of stage G. Its script, `code/sql/sales/s-star-schema.sql`, is the design this chapter arrives at and the load process it reads.

11.1. Operational and analytical processing

This section contrasts the two uses of a database, and says what a data warehouse is.

The work of an operational database is called **online transaction processing** (OLTP): many short transactions, each reading or changing a few rows, run by the clerks and programs that operate the business, against the current state of the data. The work of analysis is called **online analytical processing** (OLAP): fewer, longer queries, each reading many rows, usually of history, and summarizing them for someone who must decide something. The two make opposite demands on a design.

Consider the sales manager's question above, for the last two quarters of the history. The operational tables of stage G can answer it, as chapter 4's queries did, but the query must join three tables and derive a product line that no table stores, from the pattern of the product number:

Table 11.1.: Sales by Quarter and Product Line, from the Operational Tables

```
WITH line AS (
  SELECT EXTRACT(YEAR FROM o.order_date) AS order_year,
         EXTRACT(QUARTER FROM o.order_date)
           AS order_quarter,
         CASE WHEN p.product_number LIKE 'HT-%'
              THEN 'Hand trucks'
              ELSE 'Garden carts' END AS product_line,
         l.quantity * l.unit_price AS amount
  FROM distribution.order_line AS l
  JOIN distribution.customer_order AS o
    ON o.customer_order_pk = l.customer_order_fk
  JOIN distribution.product AS p
    ON p.product_pk = l.product_fk
  WHERE o.order_date >= DATE '2026-10-01'
)
SELECT order_year, order_quarter, product_line,
       SUM(amount) AS sales
FROM line
GROUP BY order_year, order_quarter, product_line
ORDER BY order_year, order_quarter, product_line;
```

order_year	order_quarter	product_line	sales
2026	4	Garden carts	213852.52
2026	4	Hand trucks	225600.08
2027	1	Garden carts	187203.83
2027	1	Hand trucks	140291.17

The same question, asked of the warehouse, names what it wants directly: the calendar year and quarter from a date dimension, the product line

from a product dimension, and the sales amount from a table of facts, with no derivation:

Table 11.2.: The Same Sales, from the Warehouse

```
SELECT d.calendar_year, d.calendar_quarter,
       p.product_line, SUM(f.sales_amount) AS sales
FROM sales.order_line_fact AS f
JOIN sales.date_dim AS d
  ON d.date_dim_pk = f.order_date_fk
JOIN sales.product_dim AS p
  ON p.product_dim_pk = f.product_dim_fk
WHERE d.full_date >= DATE '2026-10-01'
GROUP BY d.calendar_year, d.calendar_quarter,
         p.product_line
ORDER BY d.calendar_year, d.calendar_quarter,
         p.product_line;
```

calendar_year	calendar_quarter	product_line	sales
2026	4	Garden carts	213852.52
2026	4	Hand trucks	225600.08
2027	1	Garden carts	187203.83
2027	1	Hand trucks	140291.17

Table 11.1 and Table 11.2 agree to the cent, as they must, since the warehouse was loaded from the same order lines. The difference is in the asking. The warehouse's query needs no knowledge of how the business numbers its products or dates its orders; what an analyst groups by is a column of a dimension, named for what it means. And a product line decided in one place, when the warehouse is loaded, is the same in every report, rather than re-derived, perhaps differently, in each query. Table 11.3 sets the two kinds of processing side by side.

Table 11.3.: Operational and Analytical Processing Compared

	Operational processing (OLTP)	Analytical processing (OLAP)
Serves	the people and programs that run the business	the people who analyze and plan it
Typical work	short transactions touching a few rows	long queries reading many rows
Data	current, changed constantly	history, added to in periodic loads
Design	normalized, so each fact is stored once (Chapter 7)	dimensional, so each question is easy to ask
At HG Industries	`distribution`, `production`, `quality`	`sales`

A **data warehouse** is a database built for analysis, loaded from the operational databases, that keeps their history. The usual definition, due to W. H. Inmon (Inmon 2005), names four properties, and HG's `sales` schema shows each. It is *subject-oriented*: organized around sales, a subject of the business, rather than around the transactions of taking orders. It is *integrated*: it combines distribution's orders with production's standard costs, which no operational schema holds together. It is *time-variant*: every fact is tied to a date, so trends can be followed. And it is *nonvolatile*: rows are added by periodic loads and are not updated by the business's daily work. A warehouse limited to one department's subject is often called a **data mart**; the `sales` schema, with its one subject, is one.

The two kinds of database are kept apart for practical reasons as well. An analyst's query that reads a year of orders would compete with the clerks' transactions for the same tables, and, as Chapter 9 showed, a report that runs while orders are changing may not add up. A warehouse loaded

once a night, after the day's business, gives the analyst a stable copy and leaves the operational database to its work. The next section designs the warehouse.

11.2. Dimensional modeling

This section presents how a warehouse is designed, as a process and then step by step on HG's sales.

A warehouse is designed as a **dimensional model**, the approach of Ralph Kimball (Kimball and Ross 2013). Its central idea is to divide what is recorded about a business process into two kinds of data. **Facts** are the numbers the process produces, which an analyst adds up: a quantity, an amount of money, a time. **Dimensions** are the contexts in which the facts occurred, which an analyst groups and filters by: when, who, what, where. A dimensional model is designed in four steps:

1. **Choose the business process.** The warehouse describes one activity of the business at a time, such as taking orders or inspecting lots.
2. **Declare the grain.** State what one row of the fact table records: one order line, one order, one product per day. The grain is the most important decision of the design, and every later step must agree with it.
3. **Identify the dimensions.** Ask what describes one fact at that grain: its date, its customer, its product.
4. **Identify the facts.** Ask what numbers are measured at that grain.

The facts go into a **fact table**, with one row per occurrence at the grain, and each dimension into a **dimension table**, with one row per member (one per day, per customer, per product). Each fact row refers to one row of each dimension through a foreign key, and a diagram of the tables, the

fact table in the middle and the dimensions around it, looks like a star. Such a design is called a **star schema**.

Example 11.1 (Designing the sales warehouse)**.** HG Industries' sales manager reviews the business every quarter. The questions come in many forms: sales by product line and region, by month, by customer; how much was given away in discounts; what margin each product earned over its standard cost. Each question is different, and each groups and sums the same underlying activity: customers ordering products.

The request. Design a warehouse in which each of these questions is a single grouping query.

The design. We follow the four steps.

1. **The business process** is order taking: customers placing orders for products, as stage D3 records it.
2. **The grain** is one order line: a quantity of one product, on the order one customer placed on one date. It is the finest grain the operational data holds, and the finest grain is the safest choice, since any coarser summary, by order, by month, can be computed from it, and nothing finer can.
3. **The dimensions** of an order line are its order date, its customer, and its product. Each gets a table of the attributes an analyst groups by: the date's year, quarter, month, and day of the week; the customer's city, state, and sales region; the product's description and product line.
4. **The facts** of an order line are its quantity; its sales amount, the quantity times the unit price; its discount amount, the quantity times the difference between the list price and the unit price; and its cost amount, the quantity times the standard cost from production.

Figure 11.1 draws the result, the star schema of stage S.

Read Figure 11.1 from the center. `OrderLineFact`, the fact table, holds a row per order line: its three foreign keys, one to each dimension, the order

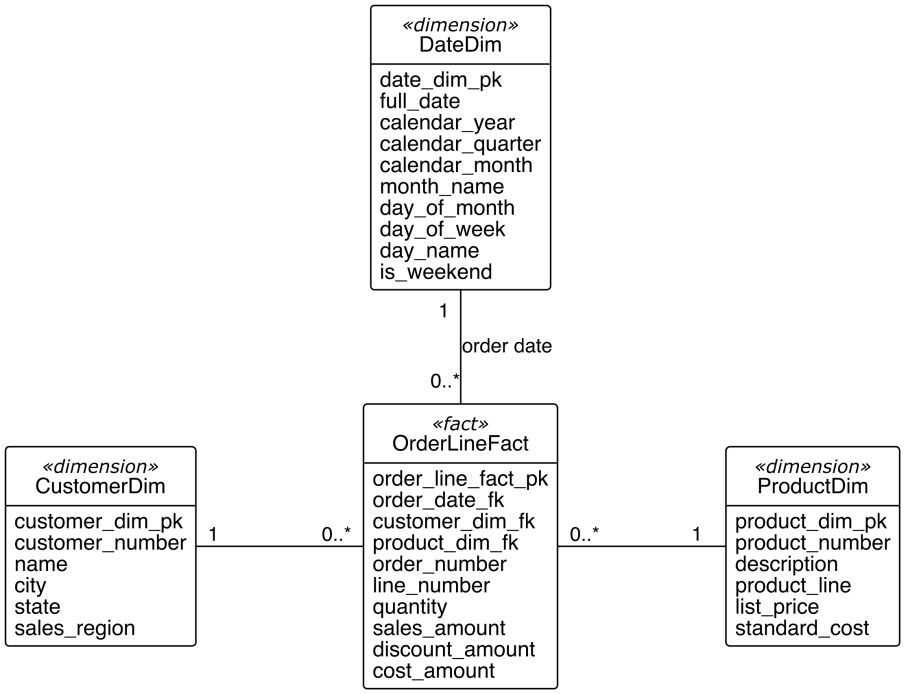


Figure 11.1.: The Sales Star Schema of Stage S

and line numbers, and the four facts. Each of the three dimension tables around it holds a row per date, customer, or product, with a surrogate key and the attributes to group by. The multiplicities are those of 1: each fact refers to one member of each dimension, and each member may have any number of facts. Listing 11.1 gives the tables as a relational schema.

Listing 11.1 The Relational Schema of the Sales Warehouse

```
date_dim (date_dim_pk, full_date, calendar_year,
          calendar_quarter, calendar_month, month_name,
          day_of_month, day_of_week, day_name, is_weekend)
    primary key: date_dim_pk
    alternate key: full_date

customer_dim (customer_dim_pk, customer_number, name, city,
              state, sales_region)
    primary key: customer_dim_pk
    alternate key: customer_number

product_dim (product_dim_pk, product_number, description,
             product_line, list_price, standard_cost)
    primary key: product_dim_pk
    alternate key: product_number

order_line_fact (order_line_fact_pk, order_date_fk,
                 customer_dim_fk, product_dim_fk,
                 order_number, line_number, quantity,
                 sales_amount, discount_amount, cost_amount)
    primary key: order_line_fact_pk
    alternate key: (order_number, line_number)
    order_date_fk references date_dim_pk in date_dim,
        must not be NULL
    customer_dim_fk references customer_dim_pk in
        customer_dim, must not be NULL
    product_dim_fk references product_dim_pk in product_dim,
        must not be NULL
```

The result. The stage S script creates these tables and loads them, as

Section 11.3 reads it. Their sizes are:

Table 11.4.: Order Lines, Facts, and Dimension Members, Stage S

```
SELECT
  (SELECT COUNT(*) FROM distribution.order_line)
    AS order_lines,
  (SELECT COUNT(*) FROM sales.order_line_fact) AS facts,
  (SELECT COUNT(*) FROM sales.date_dim) AS dates,
  (SELECT COUNT(*) FROM sales.customer_dim)
    AS customers,
  (SELECT COUNT(*) FROM sales.product_dim) AS products;
```

order_lines	facts	dates	customers	products
1065	1065	730	30	4

The check. At the grain of an order line, the fact table must have one row for every order line of the operational data, and Table 11.4 shows 1,065 of each. The dimensions have one row per member: 30 customers and 4 products, as the operational tables hold, and 730 dates, every day of 2026 and 2027, whether or not anyone ordered that day.

Interpretation. Each of the manager's questions is now a grouping query over the fact table, joined to the dimensions it groups by. Sales by product line and region join `product_dim` and `customer_dim`; the margin is `SUM(sales_amount) - SUM(cost_amount)`, grouped by product; discounts given are `SUM(discount_amount)`. None needs a derivation, and all give the same answer for the same question.

Where it breaks. The grain limits the facts. A fact that belongs to a whole order, such as a freight charge, has no place in a table of order lines: stored on every line of the order, it would be added once per line. It belongs in a second fact table, at the grain of an order. In the same way, the design

records each order at its order date only; a question about shipping, such as how often orders ship on time, needs the shipment process and a fact table of its own.

A dimension table uses a surrogate key, as almost every table of the book's database does, and in a warehouse the reason is stronger than elsewhere. A dimension may need several rows for one business object, when the object changes and the warehouse must keep both versions, as Section 11.4 shows; the business identifier, `customer_number`, then repeats, and only a surrogate key tells the rows apart. The order and line numbers, by contrast, sit in the fact table itself, with no dimension table: an identifier kept on the fact, with nothing to describe it, is called a **degenerate dimension**. It lets an analyst go back from a fact to the order it came from. The date's foreign key, finally, is named for its role, `order_date_fk`, since an order line has other dates, such as its requested ship date, which a later design could add as a second foreign key to the same date dimension.

Notice that the dimension tables are not normalized. In `customer_dim`, the sales region depends on the state, a transitive dependency of the kind Section 7.4 removes:

Table 11.5.: States and Sales Regions in the Customer Dimension

```
SELECT state, sales_region,
       COUNT(*) AS customers
FROM sales.customer_dim
GROUP BY state, sales_region
ORDER BY sales_region, state;
```

state	sales_region	customers
AR	Fayetteville region	12
MO	Fayetteville region	6
KS	Tulsa region	2
OK	Tulsa region	7
TX	Tulsa region	3

Each state of Table 11.5 has one region, repeated on every customer row of the state; normalized, the regions would be a table of their own, keyed by state. The repetition is deliberate. Recall from Section 7.6 that denormalization trades the anomalies of repeated facts for simpler reading, and a warehouse is where the trade pays: its rows are written by a load process, not by clerks, so update anomalies cannot arise from daily work, and every query that groups by region saves a join. A dimensional design whose dimensions are normalized into further tables is called a **snowflake schema**, from the shape of its diagram; it saves little space, since dimensions are small, and costs a join in every query, so the star is the usual choice.

The facts differ in how they may be summed. A fact is **additive** when summing it over any dimension gives a meaningful total: the sales amount of all customers, of all products, of all days. All four facts of `order_line_fact` are additive, which is why the design stores amounts, the quantity times a price, rather than prices. A unit price is **non-additive**: the sum of the

unit prices of two order lines means nothing, and an average price must be computed from additive facts, `SUM(sales_amount) / SUM(quantity)`. A fact is **semi-additive** when it can be summed over some dimensions but not over time: a stock level can be summed over warehouses, giving the stock on hand in all of them, but not over days, since the quantity on hand on Monday and on Tuesday are largely the same carts.

The fact table is by far the largest table of a warehouse, and its size can be estimated before it is built. The combinations of its dimensions' members give the estimate: about one row for each combination, and in practice a small fraction of them, since not every customer orders every product every day. The combinations are an estimate rather than a bound when the grain is the order line, since one order may list a product on two lines; it is the order number and line number that make each fact unique. For the sales warehouse the combinations of date, customer, and product come to $730 \times 30 \times 4 = 87{,}600$, and the 1,065 facts of Table 11.4 are about 1.2 percent of that number. A warehouse at a finer grain, or with more dimensions, grows by the product of their sizes, which is the first thing to estimate when you design one. The next section loads the warehouse.

11.3. Extract, transform, and load

This section loads the warehouse from the operational databases, first entirely and then with only what is new.

The process that fills a warehouse is called **extract, transform, and load** (ETL), after its three steps. **Extract** reads the data from the operational sources, often several databases or files. **Transform** turns it into the warehouse's form: it derives the attributes the dimensions carry, computes the facts at the declared grain, and repairs or rejects what is wrong, as Section 8.5's staging queries did. **Load** writes the result into the dimension and fact tables, looking up each dimension's surrogate key from the business identifier the source carries. Organizations run ETL with dedicated tools

and schedulers, but the three steps are the same in every one, and HG's are a single SQL script, `code/sql/sales/s-star-schema.sql`.

The dimensions are loaded first, since the facts refer to them. The date dimension is not extracted from anywhere: it is generated, one row for every day of the years the warehouse covers, with PostgreSQL's `generate_series`, and its attributes are computed from each date:

```
INSERT INTO sales.date_dim
    (full_date, calendar_year, calendar_quarter,
     calendar_month, month_name, day_of_month,
     day_of_week, day_name, is_weekend)
SELECT d::DATE,
       EXTRACT(YEAR FROM d)::INTEGER,
       EXTRACT(QUARTER FROM d)::INTEGER,
       EXTRACT(MONTH FROM d)::INTEGER,
       TO_CHAR(d, 'FMMonth'),
       EXTRACT(DAY FROM d)::INTEGER,
       EXTRACT(ISODOW FROM d)::INTEGER,
       TO_CHAR(d, 'FMDay'),
       EXTRACT(ISODOW FROM d) IN (6, 7)
FROM generate_series(DATE '2026-01-01',
                     DATE '2027-12-31',
                     INTERVAL '1 day') AS d;
```

`generate_series` (PostgreSQL's) returns one row per day between the two dates, and each column is a transformation of the date. Two details prevent errors that are easy to make. The year is `EXTRACT(YEAR ...)`, the calendar year; a widely copied template uses the ISO week-numbering year instead, which puts January 1, 2027, a Friday, in 2026. And `TO_CHAR` pads month and day names with spaces to a fixed width unless its format begins with `FM`, which would make `'May'` unequal to `'May       '` in every later comparison.

The customer and product dimensions are extracted from the operational tables, and transformed on the way: the customer dimension derives each customer's sales region from the state, and the product dimension derives the product line from the product number and brings in the standard cost from production. Both transformations are `CASE` expressions, of the kind Section 4.4 wrote.

The facts come last. Each order line is extracted with its order, customer, and product; its measures are computed; and each business identifier, the order date, the customer number, the product number, is joined to its dimension to find the surrogate key the fact table stores. A warehouse is refreshed on a schedule, nightly for example, and each refresh should add only what is new. The fact load therefore carries a condition that skips order lines already loaded, found by the degenerate dimension, the order and line numbers:

```
INSERT INTO sales.order_line_fact
    (order_date_fk, customer_dim_fk, product_dim_fk,
     order_number, line_number, quantity,
     sales_amount, discount_amount, cost_amount)
SELECT dd.date_dim_pk, cd.customer_dim_pk,
       pd.product_dim_pk, o.order_number, l.line_number,
       l.quantity,
       l.quantity * l.unit_price,
       l.quantity * (pd.list_price - l.unit_price),
       l.quantity * pd.standard_cost
FROM distribution.order_line AS l
JOIN distribution.customer_order AS o
  ON o.customer_order_pk = l.customer_order_fk
JOIN distribution.customer AS c
  ON c.customer_pk = o.customer_fk
JOIN distribution.product AS p
  ON p.product_pk = l.product_fk
JOIN sales.date_dim AS dd ON dd.full_date = o.order_date
```

```
JOIN sales.customer_dim AS cd
  ON cd.customer_number = c.customer_number
JOIN sales.product_dim AS pd
  ON pd.product_number = p.product_number
WHERE NOT EXISTS (
  SELECT 1 FROM sales.order_line_fact AS f
  WHERE f.order_number = o.order_number
    AND f.line_number = l.line_number);
```

The stage S script's fact load is the same statement without the last condition, since it loads an empty table. Read the statement in its three steps. The first three joins extract each order line with the rows it belongs to; the three expressions of the select list transform it into the facts of the grain; and the three joins to `sales` tables load it, replacing each business identifier with a surrogate key. Notice that the joins to the dimensions are inner joins: an order line whose customer is missing from the customer dimension would be dropped without a word, so the dimensions must be loaded, with the same kind of condition, before the facts.

Example 11.2 (Loading a new customer's first order)**.** On April 1, 2027, the day after the history of stage S ends, Arkansas Valley Supply of Ozark, Arkansas, became customer C-1031 and placed order SO-5615: twelve HT-100 hand trucks at the list price of $89.00 and six GC-400 poly carts at $149.00. The warehouse was last loaded on March 31. The example assumes stage S.

The request. Refresh the warehouse with the new order, loading only what is new, and confirm that loading twice adds nothing.

The design. The sales department's systems record the customer and the order in the operational tables, as Example 8.2 did:

```
INSERT INTO distribution.customer
    (customer_number, name, city, state)
VALUES ('C-1031', 'Arkansas Valley Supply', 'Ozark', 'AR');

INSERT INTO distribution.customer_order
    (order_number, order_date, requested_ship_date,
     customer_fk)
VALUES ('SO-5615', DATE '2027-04-01', DATE '2027-04-08',
        (SELECT customer_pk FROM distribution.customer
         WHERE customer_number = 'C-1031'));

INSERT INTO distribution.order_line
    (customer_order_fk, line_number, product_fk,
     quantity, unit_price)
SELECT o.customer_order_pk, v.line_number,
       p.product_pk, v.quantity, v.unit_price
FROM distribution.customer_order AS o,
     (VALUES (1, 'HT-100', 12, 89.00),
             (2, 'GC-400', 6, 149.00))
       AS v (line_number, product_number, quantity,
             unit_price)
JOIN distribution.product AS p
  ON p.product_number = v.product_number
WHERE o.order_number = 'SO-5615';
```

The order's two lines are inserted with one statement: a `VALUES` list with a name and column names, `AS v (line_number, ...)`, can stand in a `FROM` clause as a small table, which is joined to `product` to find each product's key.

The refresh loads the new customer into the customer dimension, with the same transformation and a condition that skips customers already there, and then runs the fact load above:

```
INSERT INTO sales.customer_dim
    (customer_number, name, city, state, sales_region)
SELECT c.customer_number, c.name, c.city, c.state,
       CASE WHEN c.state IN ('OK', 'KS', 'TX')
            THEN 'Tulsa region'
            ELSE 'Fayetteville region' END
FROM distribution.customer AS c
WHERE NOT EXISTS (
  SELECT 1 FROM sales.customer_dim AS cd
  WHERE cd.customer_number = c.customer_number);
```

The result. After the refresh, the new facts are:

Table 11.6.: The Facts of Order SO-5615 After the Refresh

```
SELECT f.order_number, f.line_number,
       c.customer_number, c.sales_region,
       p.product_number, f.sales_amount, f.cost_amount
FROM sales.order_line_fact AS f
JOIN sales.customer_dim AS c
  ON c.customer_dim_pk = f.customer_dim_fk
JOIN sales.product_dim AS p
  ON p.product_dim_pk = f.product_dim_fk
WHERE f.order_number = 'SO-5615'
ORDER BY f.line_number;
```

order_number	line_number	customer_number	sales_region	product_number	sales_amount	cost_amount
SO-5615	1	C-1031	Fayetteville region	HT-100	1068.00	628.80
SO-5615	2	C-1031	Fayetteville region	GC-400	894.00	475.80

A second refresh, run at once, finds nothing new:

Table 11.7.: Facts and Customers After Loading Twice

```
SELECT COUNT(*) AS facts,
       (SELECT COUNT(*) FROM sales.customer_dim)
         AS customers
FROM sales.order_line_fact;
```

facts	customers
1067	31

The check. The sales amounts are $12 \times 89.00 = 1{,}068.00$ and $6 \times 149.00 = 894.00$, and the cost amounts $12 \times 52.40 = 628.80$ and $6 \times 79.30 = 475.80$, from the standard costs the product dimension holds for the HT-100 and the GC-400. The warehouse held 1,065 facts and 30 customers; one refresh adds two facts and one customer, and Table 11.7 shows 1,067 and 31 after two refreshes, so the second added nothing.

Interpretation. The refresh can run every night without anyone deciding what to load: whatever the operational tables hold that the warehouse lacks is loaded, and nothing is loaded twice. A refresh that fails halfway can simply be run again.

Where it breaks. The refresh adds new rows but never changes old ones. If Arkansas Valley Supply later corrected its city in the operational table, the customer dimension would keep the old one, since the condition skips customer numbers already loaded; and an order line changed after it was loaded, a quantity corrected, would not reach the warehouse at all. Changed dimension rows are the subject of the next section; changed facts are usually handled by loading a correcting fact, so that the history of the change is kept.

Two habits make an ETL process trustworthy. Reconcile after every load: count the order lines of the source and the facts of the warehouse, and sum

the sales amounts of both, as Table 11.4 and Table 11.1 did; a difference means a lost or doubled row. And make every step safe to run twice, as the conditions above do, so that a failed night can be rerun without cleaning up first.

11.4. Slowly changing dimensions

This section considers what a warehouse should do when a dimension's member changes: keep its history, or rewrite it.

Dimensions change more slowly than facts, but they change. A customer moves, a product is renamed, a territory is redrawn. A dimension whose members change from time to time is called a **slowly changing dimension**, and the question each change raises is what the warehouse should say about the past. Every fact refers to a dimension row, and the dimension row says where the customer is. If that row changes, every fact that refers to it moves with it, including facts from before the change.

Suppose that Prairie Garden Center (C-1005), of Fort Smith, Arkansas, moved across the state line to Sallisaw, Oklahoma, on January 1, 2027, and became a customer of the Tulsa region. Before the change, the warehouse reports sales by year and region as:

Table 11.8.: Sales by Year and Region, Before the Move

```
SELECT d.calendar_year, c.sales_region,
       SUM(f.sales_amount) AS sales
FROM sales.order_line_fact AS f
JOIN sales.date_dim AS d
  ON d.date_dim_pk = f.order_date_fk
JOIN sales.customer_dim AS c
  ON c.customer_dim_pk = f.customer_dim_fk
GROUP BY d.calendar_year, c.sales_region
ORDER BY d.calendar_year, c.sales_region;
```

calendar_year	sales_region	sales
2026	Fayetteville region	580451.72
2026	Tulsa region	621080.27
2027	Fayetteville region	165070.05
2027	Tulsa region	162424.95

The simplest response, called **type 1**, overwrites the dimension row with the new values:

```
UPDATE sales.customer_dim
SET city = 'Sallisaw', state = 'OK',
    sales_region = 'Tulsa region'
WHERE customer_number = 'C-1005';
```

Table 11.9.: Sales by Year and Region After a Type 1 Change

```
SELECT d.calendar_year, c.sales_region,
       SUM(f.sales_amount) AS sales
FROM sales.order_line_fact AS f
JOIN sales.date_dim AS d
  ON d.date_dim_pk = f.order_date_fk
JOIN sales.customer_dim AS c
  ON c.customer_dim_pk = f.customer_dim_fk
GROUP BY d.calendar_year, c.sales_region
ORDER BY d.calendar_year, c.sales_region;
```

calendar_year	sales_region	sales
2026	Fayetteville region	554401.72
2026	Tulsa region	647130.27
2027	Fayetteville region	157138.05
2027	Tulsa region	170356.95

Compare Table 11.9 with Table 11.8. The 2027 figures are right: the customer's $7,932.00 of 2027 sales now count in the Tulsa region. But the 2026 figures have changed too. Prairie Garden Center's $26,050.00 of 2026 sales, made while it was a Fayetteville customer, have moved to Tulsa, and a report of 2026 run today disagrees with the same report run last month. The warehouse has rewritten its own history.

The response that keeps history, called **type 2**, adds a new row for the customer, with the new values, and keeps the old row for the facts that belong to it. Each row then needs the dates between which it describes the customer, and a flag for the current row, and the customer number is no longer unique in the dimension; which is the reason, noted in Section 11.2, that dimensions use surrogate keys. Stage S's customer dimension has no such columns, so the demonstration adds them, giving every existing row

the whole period and marking it current:

```
ALTER TABLE sales.customer_dim
  ADD COLUMN valid_from DATE NOT NULL
      DEFAULT DATE '2026-01-01',
  ADD COLUMN valid_to DATE NOT NULL
      DEFAULT DATE '9999-12-31',
  ADD COLUMN is_current BOOLEAN NOT NULL DEFAULT true,
  DROP CONSTRAINT customer_dim_customer_number_key;
```

The move then closes the old row on December 31, 2026, and inserts a new row from January 1, 2027:

```
UPDATE sales.customer_dim
SET valid_to = DATE '2026-12-31', is_current = false
WHERE customer_number = 'C-1005';

INSERT INTO sales.customer_dim
    (customer_number, name, city, state, sales_region,
     valid_from)
VALUES ('C-1005', 'Prairie Garden Center', 'Sallisaw',
        'OK', 'Tulsa region', DATE '2027-01-01');
```

Table 11.10.: The Two Rows of Customer C-1005 After the Move

```
SELECT customer_dim_pk, customer_number, city,
       sales_region, valid_from, valid_to, is_current
FROM sales.customer_dim
WHERE customer_number = 'C-1005'
ORDER BY valid_from;
```

customer_dim_pk	customer_number	city	sales_region	valid_from	valid_to	is_current
5	C-1005	Fort Smith	Fayetteville region	2026-01-01	2026-12-31	false
31	C-1005	Sallisaw	Tulsa region	2027-01-01	9999-12-31	true

A fact must now refer to the row that was valid on its date. The fact load of Section 11.3 finds it by adding the dates to the customer join, `ON cd.customer_number = c.customer_number AND o.order_date BETWEEN cd.valid_from AND cd.valid_to`, so that orders from 2027 on refer to the new row. Stage S's facts for 2027 were loaded before the move, so the demonstration repoints them, as a reload would:

```
UPDATE sales.order_line_fact
SET customer_dim_fk =
    (SELECT cd.customer_dim_pk
     FROM sales.customer_dim AS cd
     JOIN sales.date_dim AS d
       ON d.full_date BETWEEN cd.valid_from
                          AND cd.valid_to
     WHERE cd.customer_number = 'C-1005'
       AND d.date_dim_pk = order_line_fact.order_date_fk)
WHERE customer_dim_fk IN
    (SELECT customer_dim_pk FROM sales.customer_dim
     WHERE customer_number = 'C-1005');
```

Table 11.11.: Sales by Year and Region After a Type 2 Change

```
SELECT d.calendar_year, c.sales_region,
       SUM(f.sales_amount) AS sales
FROM sales.order_line_fact AS f
JOIN sales.date_dim AS d
  ON d.date_dim_pk = f.order_date_fk
JOIN sales.customer_dim AS c
  ON c.customer_dim_pk = f.customer_dim_fk
GROUP BY d.calendar_year, c.sales_region
ORDER BY d.calendar_year, c.sales_region;
```

calendar_year	sales_region	sales
2026	Fayetteville region	580451.72
2026	Tulsa region	621080.27
2027	Fayetteville region	157138.05
2027	Tulsa region	170356.95

Table 11.11 has the 2026 figures of Table 11.8, unchanged, and the 2027 figures of Table 11.9: each fact is reported in the region its customer was in when the order was placed. A query that wants the customer as it is now, rather than as it was, joins through the customer number to the current row instead: the customer number and `is_current` together. The customer number alone matches both rows of C-1005 and counts its facts twice.

A third response, **type 3**, adds a column for the previous value beside the current one, `previous_sales_region`, so that a report can use either; it keeps one step of history and no more. The choice is made attribute by attribute, and it is a business decision, not a technical one. A misspelled name should be corrected everywhere, so type 1. A sales region decides whose results a sale counts toward, so a region change that should not

rewrite last year's commissions calls for type 2. You should decide, for each attribute of each dimension, which history the business needs before the first load, since a type 1 overwrite cannot be undone later.

11.5. Multidimensional queries

This section writes the queries a warehouse exists for: totals at several levels at once, and the moves an analyst makes between them.

An analyst looks at a warehouse's facts through its dimensions, and moves among them with a few standard operations, named for a picture of the facts as a **cube**, one axis per dimension: date, customer, product, with each cell holding the facts where the three meet.

- To **roll up** is to summarize to a coarser level of a dimension: from quarters to years, from products to product lines.
- To **drill down** is the reverse: to break a total into the finer levels beneath it, from years to quarters to months.
- To **slice** is to fix one dimension at one value, as a report of 2026 alone does, which leaves a thinner cube of the other dimensions.
- To **dice** is to select a sub-cube with conditions on several dimensions at once: hand trucks, in the Tulsa region, in the first quarter of 2027.

A slice and a dice are `WHERE` conditions on dimension attributes. A roll-up is a `GROUP BY` on fewer or coarser columns, and a report that shows several levels together, each quarter and each year and the grand total, needs several groupings at once, which SQL writes with `ROLLUP`, `CUBE`, and `GROUPING SETS`.

Let's evaluate a roll-up by hand first. For the first quarter of 2027, the sales by product line and region are:

Table 11.12.: Sales by Product Line and Region, First Quarter of 2027

```
SELECT p.product_line, c.sales_region,
       SUM(f.sales_amount) AS sales
FROM sales.order_line_fact AS f
JOIN sales.date_dim AS d
  ON d.date_dim_pk = f.order_date_fk
JOIN sales.product_dim AS p
  ON p.product_dim_pk = f.product_dim_fk
JOIN sales.customer_dim AS c
  ON c.customer_dim_pk = f.customer_dim_fk
WHERE d.calendar_year = 2027 AND d.calendar_quarter = 1
GROUP BY p.product_line, c.sales_region
ORDER BY p.product_line, c.sales_region;
```

product_line	sales_region	sales
Garden carts	Fayetteville region	92824.47
Garden carts	Tulsa region	94379.36
Hand trucks	Fayetteville region	72245.58
Hand trucks	Tulsa region	68045.59

A report of these cells with subtotals by product line and a grand total needs three groupings: by product line and region (the four cells of Table 11.12), by product line alone, and by nothing, the whole slice. The subtotals are sums of the cells:

Table 11.13.: Rolling Up the First Quarter of 2027 by Hand

Product line	Region	Grouping	Sales
Garden carts	Fayetteville region	line and region	92,824.47
Garden carts	Tulsa region	line and region	94,379.36

Product line	Region	Grouping	Sales
Garden carts	(all)	line	92,824.47 + 94,379.36 = 187,203.83
Hand trucks	Fayetteville region	line and region	72,245.58
Hand trucks	Tulsa region	line and region	68,045.59
Hand trucks	(all)	line	72,245.58 + 68,045.59 = 140,291.17
(all)	(all)	none	187,203.83 + 140,291.17 = 327,495.00

`GROUP BY ROLLUP (a, b)` asks for these three groupings in one query: by `a` and `b`, by `a`, and by nothing. It produces every row of Table 11.13:

Table 11.14.: The Same Roll-Up in SQL

```
SELECT p.product_line, c.sales_region,
       SUM(f.sales_amount) AS sales
FROM sales.order_line_fact AS f
JOIN sales.date_dim AS d
  ON d.date_dim_pk = f.order_date_fk
JOIN sales.product_dim AS p
  ON p.product_dim_pk = f.product_dim_fk
JOIN sales.customer_dim AS c
  ON c.customer_dim_pk = f.customer_dim_fk
WHERE d.calendar_year = 2027 AND d.calendar_quarter = 1
GROUP BY ROLLUP (p.product_line, c.sales_region)
ORDER BY p.product_line, c.sales_region;
```

product_line	sales_region	sales
Garden carts	Fayetteville region	92824.47
Garden carts	Tulsa region	94379.36
Garden carts	NULL	187203.83
Hand trucks	Fayetteville region	72245.58
Hand trucks	Tulsa region	68045.59
Hand trucks	NULL	140291.17
NULL	NULL	327495.00

In a subtotal row, a column that was rolled up holds NULL: the region of a product line's subtotal, and both columns of the grand total. The NULL there means "all", not a value that is unknown or does not apply, which Section 3.3's NULL meant. The function `GROUPING(column)` tells the two apart: it returns 1 when the column was rolled up in that row and 0 otherwise, so a report can label its subtotals.

A roll-up follows a hierarchy, and the order of `ROLLUP`'s columns must be

the hierarchy's, from coarse to fine. The date dimension's hierarchy is year, quarter, month, and a quarter means nothing without its year. The history of stage S runs from January 2026 to March 2027, so grouping by quarter alone silently merges two first quarters:

Table 11.15.: Sales by Quarter Alone: Two Years in One Row

```
SELECT d.calendar_quarter, SUM(f.sales_amount) AS sales
FROM sales.order_line_fact AS f
JOIN sales.date_dim AS d
  ON d.date_dim_pk = f.order_date_fk
GROUP BY d.calendar_quarter
ORDER BY d.calendar_quarter;
```

calendar_quarter	sales
1	349063.00
2	437131.67
3	303379.72
4	439452.60

The first row of Table 11.15 adds the first quarters of 2026 and 2027, a number no one asked for. Rolled up along the hierarchy, with `GROUPING` to label the subtotals, the quarters stay in their years:

Table 11.16.: Sales by Year and Quarter, with Subtotals

```
SELECT CASE WHEN GROUPING(d.calendar_year) = 1
            THEN 'All years'
            ELSE d.calendar_year::TEXT END AS year_label,
       CASE WHEN GROUPING(d.calendar_quarter) = 1
            THEN 'All quarters'
            ELSE 'Q' || d.calendar_quarter END
         AS quarter_label,
       SUM(f.sales_amount) AS sales
FROM sales.order_line_fact AS f
JOIN sales.date_dim AS d
  ON d.date_dim_pk = f.order_date_fk
GROUP BY ROLLUP (d.calendar_year, d.calendar_quarter)
ORDER BY d.calendar_year, d.calendar_quarter;
```

year_label	quarter_label	sales
2026	Q1	21568.00
2026	Q2	437131.67
2026	Q3	303379.72
2026	Q4	439452.60
2026	All quarters	1201531.99
2027	Q1	327495.00
2027	All quarters	327495.00
All years	All quarters	1529026.99

Each year of Table 11.16 is followed by its subtotal, and the last row is the grand total, $1,529,026.99. The first row of Table 11.15, 349,063.00, is the first quarter of 2026, 21,568.00, plus the first quarter of 2027, 327,495.00; the history of stage G begins in April 2026, so the first quarter of 2026 holds only the twelve starting orders of stage D5. To drill down from Table 11.16 to months, add `d.calendar_month` as the third column of the

`ROLLUP`.

`ROLLUP` produces the groupings of one hierarchy. When the columns are independent dimensions, a report may want every combination of them: the cells, the subtotals by each dimension, and the total. `GROUP BY CUBE (a, b)` produces all of them, the four groupings (`a`, `b`), (`a`), (`b`), and (). For 2026, a slice of the cube, by product line and region:

Table 11.17.: Sales of 2026 by Every Combination of Product Line and Region

```
SELECT CASE WHEN GROUPING(p.product_line) = 1
            THEN '(all)' ELSE p.product_line END
         AS product_line,
       CASE WHEN GROUPING(c.sales_region) = 1
            THEN '(all)' ELSE c.sales_region END
         AS sales_region,
       SUM(f.sales_amount) AS sales
FROM sales.order_line_fact AS f
JOIN sales.date_dim AS d
  ON d.date_dim_pk = f.order_date_fk
JOIN sales.product_dim AS p
  ON p.product_dim_pk = f.product_dim_fk
JOIN sales.customer_dim AS c
  ON c.customer_dim_pk = f.customer_dim_fk
WHERE d.calendar_year = 2026
GROUP BY CUBE (p.product_line, c.sales_region)
ORDER BY GROUPING(p.product_line),
         GROUPING(c.sales_region),
         p.product_line, c.sales_region;
```

product_line	sales_region	sales
Garden carts	Fayetteville region	253811.04
Garden carts	Tulsa region	270799.22
Hand trucks	Fayetteville region	326640.68
Hand trucks	Tulsa region	350281.05
Garden carts	(all)	524610.26
Hand trucks	(all)	676921.73
(all)	Fayetteville region	580451.72
(all)	Tulsa region	621080.27
(all)	(all)	1201531.99

The nine rows of Table 11.17 are the four cells, two subtotals by product line, two by region, and the total. `GROUPING SETS` names the groupings wanted explicitly, when neither `ROLLUP` nor `CUBE` gives the groupings wanted: `GROUP BY GROUPING SETS ((p.product_line), (c.sales_region))` gives the two sets of subtotals without the cells or the total. In fact `ROLLUP` and `CUBE` are shorthand for grouping sets: `ROLLUP (a, b)` is `GROUPING SETS ((a, b), (a), ())`. All three are standard SQL, and PostgreSQL supports them. The pivot of Section 4.4, with `FILTER`, lays the same numbers out as a cross tab, one dimension across the columns.

A report that many people run, a monthly summary by product, reads the whole fact table each time. A warehouse often stores such a summary instead, as a table of its own, refreshed with each load. In PostgreSQL, a materialized view (Section 9.1) does this: it stores its query's result, and `REFRESH MATERIALIZED VIEW` recomputes it after the nightly load:

```
CREATE MATERIALIZED VIEW sales.monthly_product_sales AS
SELECT d.calendar_year, d.calendar_month,
       p.product_number, SUM(f.quantity) AS quantity,
       SUM(f.sales_amount) AS sales
FROM sales.order_line_fact AS f
JOIN sales.date_dim AS d
  ON d.date_dim_pk = f.order_date_fk
JOIN sales.product_dim AS p
  ON p.product_dim_pk = f.product_dim_fk
GROUP BY d.calendar_year, d.calendar_month,
         p.product_number;
```

Table 11.18.: The Monthly Summary for March 2027

```
SELECT calendar_year, calendar_month, product_number,
       quantity, sales
FROM sales.monthly_product_sales
WHERE calendar_year = 2027 AND calendar_month = 3
ORDER BY product_number;
```

calendar_year	calendar_month	product_number	quantity	sales
2027	3	GC-300	421	48775.72
2027	3	GC-400	505	73026.39
2027	3	HT-100	237	20695.17
2027	3	HT-200	228	30994.22

The summary has one row per month and product, 15 months of four products at most, where the fact table has 1,065 rows; a report of any month reads a few rows. Because the summary holds sums, which are additive, it can be rolled up further, to quarters or years, without returning to the facts; a summary of averages could not.

11.6. Summary

Operational databases run the business; a data warehouse analyzes it. Online transaction processing is many short transactions on current data; online analytical processing is long queries over history. A warehouse is loaded from the operational databases, keeps their history, and is subject-oriented, integrated, time-variant, and nonvolatile.

A dimensional model divides a business process into facts and dimensions. It is designed in four steps: choose the business process, declare the grain, identify the dimensions, and identify the facts. The facts

go into a fact table at the declared grain, the dimensions into tables around it, and the result is a star schema.

A warehouse's dimensions are denormalized, and its facts should be additive. Repeated attributes in a dimension save joins and cannot drift, since a load process writes them. Additive facts can be summed over any dimension; non-additive ones must be computed from additive ones, and semi-additive ones cannot be summed over time.

Extract, transform, and load fills the warehouse, and a refresh loads only what is new. Extract reads the sources, transform derives attributes and computes facts at the grain, and load replaces business identifiers with surrogate keys, dimensions before facts. A condition that skips rows already loaded makes each step safe to run twice; reconcile counts and sums after every load.

A slowly changing dimension either rewrites history or keeps it. Type 1 overwrites the dimension row, so old facts move with the change; type 2 adds a row with the dates it is valid, so each fact keeps the member as it was; type 3 keeps one previous value in a column. The choice is the business's, made attribute by attribute before the first load.

ROLLUP, CUBE, and GROUPING SETS compute several levels of totals in one query. Roll-up and drill-down move along a dimension's hierarchy, and slice and dice select part of the cube. `ROLLUP` follows one hierarchy from coarse to fine, `CUBE` takes every combination, `GROUPING SETS` names the groupings, and `GROUPING()` tells a subtotal's NULL from a missing value. Keep a hierarchy's levels together, a quarter with its year.

11.7. Exercises

Exercise 11.1. Match each term on the left with the phrase on the right that defines it.

Term	Definition
a. grain	1. an identifier kept on a fact, with no dimension table
b. fact table	2. what one row of the fact table records
c. dimension table	3. a dimensional design whose dimensions are normalized
d. degenerate dimension	4. the rows of measurements at the declared grain
e. snowflake schema	5. the members of one context of the facts, with their attributes

Exercise 11.2. Mark each statement true or false, and correct each false one.

a. A data warehouse should be normalized to third normal form, like the operational databases it is loaded from.
b. The grain of `order_line_fact` could be changed to one row per order without losing any question the warehouse can answer.
c. `SUM(quantity_on_hand)` over all days of a month gives the stock on hand during the month.
d. The fact table of a star schema is usually much larger than any of its dimension tables.
e. A dimension table's business identifier, such as `customer_number`, is always unique within the table.

Exercise 11.3. Classify each measure as additive, semi-additive, or non-additive, and say how you would report it for a month of the history.

a. The quantity ordered on an order line.
b. The unit price of an order line.
c. The quantity on hand of a product in a warehouse, recorded daily.
d. The discount amount of an order line.
e. A product's margin as a percentage of its sales amount.

Exercise 11.4. HG Industries' quality department wants a warehouse of its inspections (stage Q), to analyze how its parts measure over time, by part, characteristic, inspector, and lot.

a. Apply the four steps of dimensional design: name the business process, declare the grain, and list the dimensions and the facts.
b. Draw the star schema in the manner of Figure 11.1, with the columns of each table.
c. Is the measured value an additive fact? What fact would you add so that the fraction of measurements out of specification can be computed with `SUM`?
d. Estimate the largest number of rows the fact table could have for one year, if the plant inspected one lot of each part that has characteristics on each of 250 working days, and compare it with the rows stage Q actually holds.

Exercise 11.5. For each change below to HG Industries' customer dimension, say whether you would handle it as type 1 or type 2, and give the business reason.

a. Riverside Hardware's name was loaded misspelled, and is corrected.
b. Lakeside Distributors moves from Springfield, Missouri, to Tulsa, Oklahoma.
c. The sales department splits the Tulsa region into a Tulsa region and a Dallas region, and assigns the Texas customers to Dallas from July 1.
d. A customer's telephone number, which no report groups by, changes.

Exercise 11.6. In Example 11.2, suppose that the nightly refresh ran the fact load before the customer load.

a. What would the fact load have done with the two lines of SO-5615, and would anything have reported it?
b. What would the next night's refresh do, if it ran in the right order?

c. What reconciliation of Section 11.3 would have caught the problem on the first night?

Exercise 11.7. Using stage S, write one query that reports, for 2026, the sales amount and the margin (sales amount less cost amount) by sales region and product line, with a subtotal for each region and a grand total, and with each subtotal row labeled `(all)` in place of its NULL. Check the grand total of the sales amount against Table 11.8.

Exercise 11.8. At stage S, the query below groups the facts by `CUBE` over three dimension attributes.

```
SELECT p.product_line, c.sales_region, d.calendar_year,
       SUM(f.sales_amount)
FROM sales.order_line_fact AS f
JOIN sales.date_dim AS d
  ON d.date_dim_pk = f.order_date_fk
JOIN sales.product_dim AS p
  ON p.product_dim_pk = f.product_dim_fk
JOIN sales.customer_dim AS c
  ON c.customer_dim_pk = f.customer_dim_fk
GROUP BY CUBE (p.product_line, c.sales_region,
               d.calendar_year);
```

a. How many groupings does `CUBE` of three columns produce? Name them.
b. Predict the number of rows the query returns, given two product lines, two regions, and two years with sales, and assuming every combination has sales.
c. Run the query and compare its row count with your prediction.

Exercise 11.9. Using the Chinook database of Section C.11, loaded with the book's loader (Section A.6):

a. Name the grain, the dimensions, and the facts of a star schema for the store's sales, following the four steps of Section 11.2.
b. Without building it, compute from the operational tables the sales of 2024 and 2025 by year and media type, with a subtotal for each year and a grand total, using `ROLLUP` (Section 11.5).

12. Databases for Simulation and Analytics

Learning objectives

After reading this chapter you should be able to:

- read a simulation model's input parameters from the operational database that records them
- navigate the database the KSL writes for a simulation experiment, including its hierarchy of model elements
- summarize replications and compare scenarios with SQL, and judge a difference both statistically and practically
- reshape results between long and wide tables, and choose the shape a design or an analysis needs
- design a database to hold the results of experimental work, and judge when quick tables from data classes will do

Industrial engineers build simulation models to answer questions about systems that are expensive to experiment with directly: will another machine shorten lead times, how much work in process does a line carry, where is the bottleneck. A model needs data going in, the routings, processing times, and capacities of the system, and it produces data coming out, statistics of every response in every replication of every scenario. Both kinds of data belong in a database, and the rest of this book applies to both.

This chapter joins the book to simulation. It builds a small simulation of HG Industries' hand truck frame line with the KSL, reads the model's inputs from the production schema, lets the KSL record its results in its own database, and analyzes them with SQL. The simulation itself, how the model runs and why its statistics are computed as they are, is the subject of the KSL textbook (Rossetti 2025), which this chapter cites rather than repeats.

Following along

The simulation's results are stage K of the book's database: stage Q with the KSL's output schema, `ksl_db`, added by simulating the frame line. To add it to your `dbconcepts` database, run the companion program `dbconcepts.simulation.RunFrameLineKt` against it, as Section B.4 describes. The KSL's random number streams are fixed, so the statistics are the same on every build. If you cannot run the program, run `code/sql/load/add-k.sql` instead; it creates the same `ksl_db`.

12.1. Input data for models

This section reads a simulation model's parameters from the database that already records them.

Much of what a model of a plant needs is already in the plant's operational database. The routing of a part says which work centers it visits and in what order; each operation's setup and run times say how long it takes; the machines of each work center, and the operators certified to run them, say how many lots can be worked at once. A model that types these numbers into its code is correct on the day it is written and wrong after the next engineering change. A model that reads them from the database stays

current with the plant, and a scenario can often be expressed as a change to the data rather than to the model.

HG Industries' hand truck frame, part FR-110, is cut, welded, and powder coated. Its routing, with the machines of each work center and the number of them that have a certified operator (Section 5.5), is:

Table 12.1.: The Routing of the Hand Truck Frame, with Machines and Staffed Machines

```
SELECT o.operation_number, w.work_center_code,
       o.setup_minutes, o.run_minutes_per_unit,
       COUNT(DISTINCT m.machine_pk) AS machines,
       COUNT(DISTINCT c.machine_fk) AS staffed
FROM production.operation AS o
JOIN production.part AS p
  ON p.part_pk = o.manufactured_part_fk
JOIN production.work_center AS w
  ON w.work_center_pk = o.work_center_fk
LEFT JOIN production.machine AS m
  ON m.work_center_fk = w.work_center_pk
LEFT JOIN production.certification AS c
  ON c.machine_fk = m.machine_pk
WHERE p.part_number = 'FR-110'
GROUP BY o.operation_number, w.work_center_code,
         o.setup_minutes, o.run_minutes_per_unit
ORDER BY o.operation_number;
```

operation_number	work_center_code	setup_minutes	run_minutes_per_unit	machines	staffed
10	SAW	15.0	2.50	1	1
20	WLD	20.0	6.00	3	2
30	PNT	30.0	3.00	1	1

Notice the welding work center of Table 12.1: it has three machines, but

only two have a certified operator. The third, the robotic welder M-07, was installed in January 2026, and no one is certified on it yet. Whether to train an operator for it is a question the model can answer, and the two counts give the two scenarios: the line as it is staffed, with two welders, and the line with M-07 in use, with three.

Figure 12.1 draws the model. Lots of 100 frames, the part's lot size, arrive at random, on average one every 400 minutes, and visit the three work centers in the order of the routing. Each work center is a single queue in front of its machines, and a lot's processing time there is the setup plus the run time of all 100 frames, varied at random from 10 percent below that time to 20 percent above it, since a real lot is never quite average.

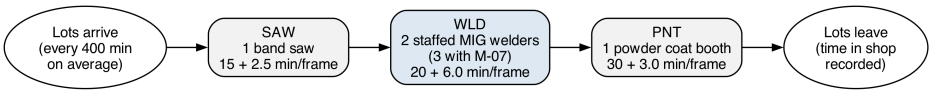


Figure 12.1.: The Frame Line as the Model Represents It

Before running a model, compute what you can by hand, as the book has done before every query. A lot's nominal time at a work center is the setup plus 100 times the run minutes per frame; the random variation, from 0.9 to 1.2 times the nominal time with the nominal time most likely, has a mean of (0.9 + 1.0 + 1.2) / 3 = 1.033 times it. A work center with c machines, receiving a lot every 400 minutes on average, is busy a fraction of the time equal to its mean time per lot divided by $400c$:

Table 12.2.: Expected Utilization of Each Work Center, Computed by Hand

Work center	Nominal minutes per lot	Mean minutes per lot	Machines	Fraction of time busy
SAW	15 + 2.5 × 100 = 265	273.8	1	273.8 / 400 = 0.685

Work center	Nominal minutes per lot	Mean minutes per lot	Machines	Fraction of time busy
WLD	20 + 6.0 × 100 = 620	640.7	2	640.7 / 800 = 0.801
WLD, with M-07	620	640.7	3	640.7 / 1,200 = 0.534
PNT	30 + 3.0 × 100 = 330	341.0	1	341.0 / 400 = 0.853

Table 12.2 already says something important: the powder coat booth, not welding, is the busiest work center, and a third welder does nothing for it. The simulation will show what the hand calculation cannot, how long lots wait, but its utilizations must agree with these, and Section 12.2 checks that they do.

The companion program reads the routing with the query of Table 12.1, as `RunFrameLine.kt`'s constant `ROUTING_SQL`:

```
/** The routing of a part, with each work center's machines:
 ↪  all, and staffed. */
const val ROUTING_SQL = """
    SELECT w.work_center_code, o.setup_minutes,
 ↪  o.run_minutes_per_unit,
           COUNT(DISTINCT m.machine_pk) AS machines,
           COUNT(DISTINCT c.machine_fk) AS staffed_machines
    FROM production.operation AS o
    JOIN production.part AS p ON p.part_pk =
 ↪  o.manufactured_part_fk
    JOIN production.work_center AS w ON w.work_center_pk =
 ↪  o.work_center_fk
    LEFT JOIN production.machine AS m ON m.work_center_fk =
 ↪  w.work_center_pk
```

```
    LEFT JOIN production.certification AS c ON c.machine_fk =
 ↪  m.machine_pk
    WHERE p.part_number = ?
    GROUP BY o.operation_number, w.work_center_code,
             o.setup_minutes, o.run_minutes_per_unit
    ORDER BY o.operation_number
"""
```

It turns each row into a `RoutingStep`, taking the count of machines or of staffed machines as the scenario requires, and the model builds one station per step, a KSL `SingleQStation` with as many servers as the step has machines, each passing its lots to the next:

```
/** The stations, one per routing step, in routing order. */
val stations: List<SingleQStation> = routing.mapIndexed { i,
 ↪  step ->
    val minutes = step.setupMinutes + step.runMinutesPerUnit
     ↪  * lotSize
    SingleQStation(
        parent = this,
        activityTime = TriangularRV(0.9 * minutes, minutes,
 ↪  1.2 * minutes, i + 2),
        initialCapacity = step.machines,
        name = step.workCenterCode
    )
}

init {
    for (i in 0 until stations.size - 1)
     ↪  stations[i].nextReceiver(stations[i + 1])
    stations.last().nextReceiver(Exit())
}
```

The stations of the model are named for the work center codes, `SAW`, `WLD`, and `PNT`, so that the results the KSL records can be traced back to the production schema. A lot that leaves the last station records its time in the shop, in hours:

```
private inner class Exit : QObjectReceiverIfc {
    override fun receive(arrivingQObject: QObject) {
        timeInShop.value = (time -
 ↪  arrivingQObject.createTime) / 60.0
        lotsCompleted.increment()
        workInProcess.decrement()
    }
}
```

The program then creates the KSL's output database in the same PostgreSQL database. Recall from Section 10.4 that `BookDatabase.connect()` returns a KSL `Database` opened with the settings in `db.properties`. The program drops any `ksl_db` schema an earlier run left, has the KSL run its creation script for the schema, makes `ksl_db` the default schema, and gives the database to a `KSLDatabase`:

```
val db = BookDatabase.connect()
db.executeCommand("DROP SCHEMA IF EXISTS ksl_db CASCADE")
KSLDatabase.executeKSLDbCreationScriptOnDatabase(db)
db.defaultSchemaName = "ksl_db"
val kslDb = KSLDatabase(db)
```

It then runs one experiment per scenario, 30 replications of 30 days each, the first 5 days of each discarded as warm-up:

```
val scenarios = listOf(
    "TwoWelders" to readRouting("FR-110", allMachines =
     ↪  false),
```

```
    "ThreeWelders" to readRouting("FR-110", allMachines =
     ↪ true)
)
for ((experiment, routing) in scenarios) {
    val model = Model("FrameLine", autoCSVReports = false)
    model.experimentName = experiment
    model.numberOfReplications = 30
    model.lengthOfReplication = 30.0 * 24.0 * 60.0
    model.lengthOfReplicationWarmUp = 5.0 * 24.0 * 60.0
    model.resetStartStreamOption = true
    FrameLine(model, routing, lotSize = 100,
        meanMinutesBetweenLots = 400.0, name = "Line")
    KSLDatabaseObserver(model, kslDb)
    model.simulate()
    println("$experiment: ${routing.joinToString {
 ↪  "${it.workCenterCode} x${it.machines}" }}")
}
```

Setting `resetStartStreamOption` makes each experiment start from the same random numbers, so that replication 7 of one scenario sees the same arrivals and the same processing-time variations as replication 7 of the other. This is called **common random numbers**, and it makes the comparison of Section 12.3 sharper, since the scenarios then differ only in what differs between them. The `KSLDatabaseObserver` attached to each model writes its results to the database as the replications finish. The next section reads them.

12.2. Simulation output databases

This section navigates the database the KSL writes for its experiments.

A simulation produces a great deal of output: for every response of the model, for every replication, of every experiment, a set of statistics. The KSL records all of it in a database schema of its own, `ksl_db`, which it creates in any database it is given, here PostgreSQL. It is a schema someone else designed, and reading it uses everything this book has taught. Figure 12.2 draws the core of it as a class model.

Read Figure 12.2 from the top. An `Experiment` is one scenario of a model, run for a number of replications, and it owns its `SimulationRun`s and the `ModelElement`s of its model, as the filled diamonds say: deleting an experiment deletes everything it produced, and the KSL's foreign keys cascade (Section 6.3.6). Model elements form a tree, each with at most one parent: the model, the frame line inside it, the stations inside the line, and the queue and resource inside each station. A `WithinRepStat` holds the statistics of one response in one replication of one run, and an `AcrossRepStat` holds the summary of one response over all the replications of a run. Listing 12.1 gives four of the tables.

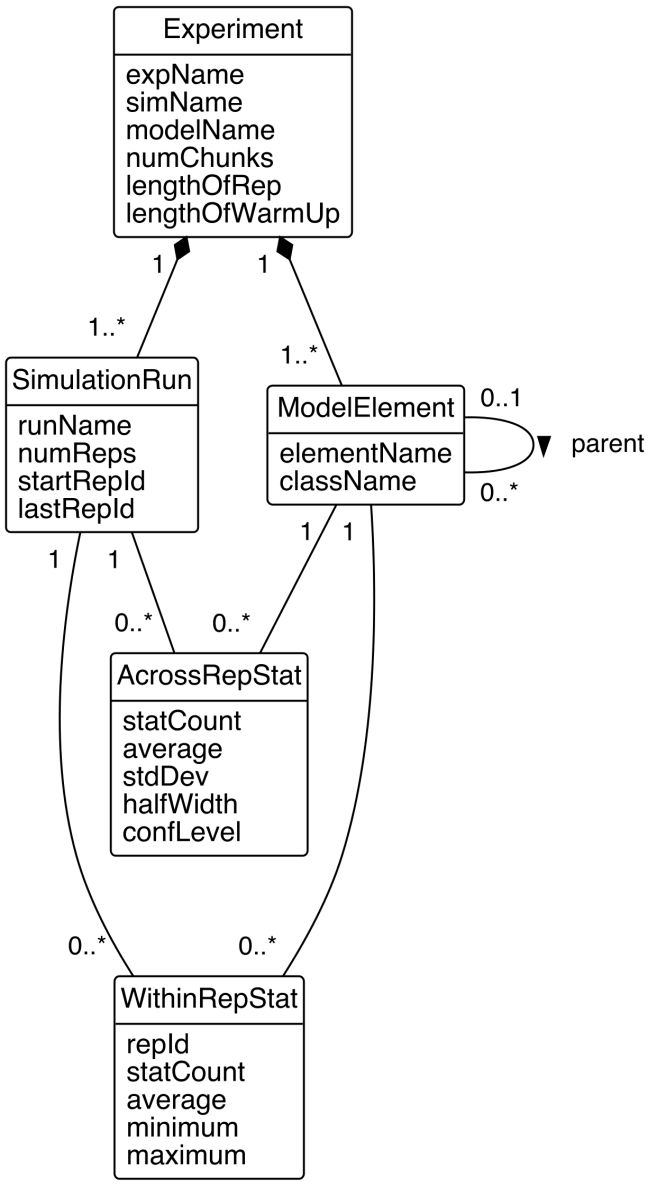


Figure 12.2.: The Core of the KSL's Output Database

Listing 12.1 Four Tables of the KSL's Output Schema

```
experiment (exp_id, sim_name, model_name, exp_name,
            num_chunks, length_of_rep, length_of_warm_up,
            rep_allowed_exec_time, rep_init_option,
            reset_start_stream_option, antithetic_option,
            adv_next_sub_stream_option, num_stream_advances,
            gc_after_rep_option)
    primary key: exp_id
    alternate key: exp_name

simulation_run (run_id, exp_id_fk, run_name, num_reps,
                start_rep_id, last_rep_id,
                run_start_time_stamp, run_end_time_stamp,
                run_error_msg)
    primary key: run_id
    alternate key: (exp_id_fk, run_name)
    exp_id_fk references exp_id in experiment,
        must not be NULL

model_element (exp_id_fk, element_id, element_name,
               class_name, parent_id_fk, parent_name,
               left_count, right_count)
    primary key: (exp_id_fk, element_id)
    alternate key: (exp_id_fk, element_name)
    exp_id_fk references exp_id in experiment,
        must not be NULL

within_rep_stat (id, element_id_fk, sim_run_id_fk, rep_id,
                 stat_name, stat_count, average, minimum,
                 maximum, weighted_sum, sum_of_weights,
                 weighted_ssq, last_value, last_weight)
    primary key: id
    alternate key: (element_id_fk, sim_run_id_fk, rep_id)
    sim_run_id_fk references run_id in simulation_run,
        must not be NULL
```

The KSL's conventions differ from the book's. Its keys are named `exp_id` and `run_id` rather than `experiment_pk`, and a foreign key `exp_id_fk`; its tables were written in upper case, which PostgreSQL folds to the lower case shown. Real databases differ in their conventions, and reading one written by others is a skill of its own. Notice also that `within_rep_stat`'s `element_id_fk` refers to a model element but is not declared a foreign key. That is, the key of `model_element` is the pair `(exp_id_fk, element_id)`, and a statistic stores only half of it; the other half is reached through the statistic's run. Thus, the database cannot check that a statistic names a real element, and a query that joins the two must match the experiment as well as the element. The experiments of stage K are:

Table 12.3.: The Experiments of Stage K

```
SELECT exp_id, exp_name, num_chunks,
       length_of_rep / 1440 AS rep_days,
       length_of_warm_up / 1440 AS warm_up_days
FROM ksl_db.experiment
ORDER BY exp_id;
```

exp_id	exp_name	num_chunks	rep_days	warm_up_days
1	TwoWelders	1	30	5
2	ThreeWelders	1	30	5

The lengths are stored in the model's time unit, minutes, and the query converts them to days. Each experiment of Table 12.3 ran in one chunk, as one simulation run of 30 replications; the KSL can split a long experiment into chunks run separately, which is why runs are a table of their own.

The model elements of each experiment are stored as a tree, and the KSL stores it in a way this book has not shown. Chapter 4's product structure tree kept each component's parent and walked the tree with a recursive

query (Section 4.6). The KSL keeps the parent too, and in addition numbers each element twice, with a `left_count` when a depth-first walk of the tree arrives at it and a `right_count` when the walk leaves it. This is called the **nested set** model of a tree (Celko 2012). Its advantage is that the elements below any element are the ones whose numbers lie between its own, so a subtree is found by a simple self-join, with no recursion. The elements below the welding work center's resource, for example, are:

Table 12.4.: The Model Elements Below the Welding Resource, Experiment 1

```
SELECT child.element_id, child.element_name,
       child.class_name, child.left_count,
       child.right_count
FROM ksl_db.model_element AS parent
JOIN ksl_db.model_element AS child
  ON child.exp_id_fk = parent.exp_id_fk
 AND child.left_count > parent.left_count
 AND child.right_count < parent.right_count
WHERE parent.exp_id_fk = 1
  AND parent.element_name = 'WLD:R'
ORDER BY child.left_count;
```

element_id	element_name	class_name	left_count	right_count
33	WLD:R:SeizeCount	Counter	63	64
34	WLD:R:NumBusy	TWResponse	65	68
41	WLD:R:Util	TWResponseFunction	66	67
35	WLD:R:Capacity	TWResponse	69	70
36	WLD:R:NumFailures	Counter	71	72
37	WLD:R:FailedState	TWResponse	73	74
38	WLD:R:IdleState	TWResponse	75	76
39	WLD:R:BusyState	TWResponse	77	78
40	WLD:R:InactiveState	TWResponse	79	80

Each row of Table 12.4 is something the welding resource records. Seven are responses: its utilization, the number of units busy, its capacity, and the time it spends in each state. Two are counters (`class_name Counter`): the number of times it was seized and the number of its failures. The KSL named them by joining its elements' names with colons, `WLD:R:Util` for the utilization of the resource `R` of the station `WLD`, and the response names are what an analysis queries by.

The nested set numbers also give a check before the query runs. The welding resource `WLD:R` has left number 62 and right number 81, and every element below it uses two of the numbers between them. Thus, there are $(81 - 62 - 1) \ / \ 2 = 9$ elements below it, the 9 rows of Table 12.4. Notice that `WLD:R:Util` lies between the numbers of `WLD:R:NumBusy`, 65 and 68: the utilization is computed from the number of units busy, and the KSL makes it a child of that response. A join on the parent column alone would have found only the resource's direct children and missed it; following the parent column to any depth takes a recursive query, as Section 4.6 showed for the product structure.

The statistics of the responses are in `within_rep_stat`, one row per response per replication. A counter has only its final value in each replication, and the KSL keeps it in a table of its own, `within_rep_counter_stat`. Thus, `LotsCompleted`, `WLD:R:SeizeCount`, and `WLD:R:NumFailures` have no rows in `within_rep_stat`. For the time in shop of the first experiment, the first five replications are:

Table 12.5.: Time in Shop, in Hours, in the First Five Replications of TwoWelders

```
SELECT w.rep_id, w.stat_count,
       ROUND(w.average::NUMERIC, 3) AS average,
       ROUND(w.minimum::NUMERIC, 3) AS minimum,
       ROUND(w.maximum::NUMERIC, 3) AS maximum
FROM ksl_db.within_rep_stat AS w
JOIN ksl_db.simulation_run AS r
  ON r.run_id = w.sim_run_id_fk
JOIN ksl_db.experiment AS e ON e.exp_id = r.exp_id_fk
WHERE e.exp_name = 'TwoWelders'
  AND w.stat_name = 'TimeInShop'
  AND w.rep_id <= 5
ORDER BY w.rep_id;
```

rep_id	stat_count	average	minimum	maximum
1	85	28.126	19.726	47.705
2	85	28.981	19.795	48.766
3	86	36.980	20.127	76.134
4	92	31.599	19.497	57.773
5	82	27.002	19.303	51.369

Each row of Table 12.5 summarizes one replication: the number of lots that left the shop after the warm-up (`stat_count`), their average time in the shop, and the shortest and longest. The replications differ because each used different random numbers, which is the point of replicating: the averages of the replications are independent observations of the same quantity, and the analysis of Section 12.3 works with them.

The KSL also provides views that do the joins once, as Section 9.1 described: `within_rep_view` gives each response's and each counter's value in each

replication by experiment name and response name, combining the two tables, and `across_rep_view` gives each response's summary over the replications. Their definitions are worth reading. Each statistics table stores the element's name, as `stat_name`, beside the key `element_id_fk`, a copy of the kind Section 7.2 warned of, kept for convenience, and several views match on both, joining tables with commas and `WHERE` conditions, the older style Section 3.6 mentioned. The utilizations the model recorded, read through `across_rep_view`, are:

Table 12.6.: Simulated Utilization of Each Work Center, Averaged over 30 Replications

```
SELECT exp_name, stat_name,
       ROUND(average::NUMERIC, 3) AS average,
       ROUND(std_dev::NUMERIC, 3) AS std_dev
FROM ksl_db.across_rep_view
WHERE stat_name IN ('SAW:R:Util', 'WLD:R:Util',
                    'PNT:R:Util')
ORDER BY stat_name, exp_name;
```

exp_name	stat_name	average	std_dev
ThreeWelders	PNT:R:Util	0.848	0.078
TwoWelders	PNT:R:Util	0.848	0.078
ThreeWelders	SAW:R:Util	0.693	0.077
TwoWelders	SAW:R:Util	0.693	0.077
ThreeWelders	WLD:R:Util	0.539	0.058
TwoWelders	WLD:R:Util	0.802	0.082

The simulated utilizations of Table 12.6 agree with the hand calculation of Table 12.2: 0.693 against 0.685 at the saw, 0.802 and 0.539 against 0.801 and 0.534 at welding, 0.848 against 0.853 at paint. The small differences are the random variation of 30 replications. The model is doing what its

inputs say, and its waiting times, which the hand calculation could not give, can now be trusted. The next section analyzes them.

12.3. Analyzing experiments with SQL

This section summarizes replications and compares scenarios with SQL.

The KSL summarizes each response over the replications of a run as it finishes, in `across_rep_stat`: the average of the replication averages, their standard deviation, and the half-width of a confidence interval for the response's mean. For the responses that matter to the frame line, the summaries are:

Table 12.7.: Summaries over 30 Replications of Four Responses, Both Experiments

```
SELECT a.stat_name, e.exp_name,
       ROUND(a.average::NUMERIC, 3) AS average,
       ROUND(a.std_dev::NUMERIC, 3) AS std_dev,
       ROUND(a.half_width::NUMERIC, 3) AS half_width
FROM ksl_db.across_rep_stat AS a
JOIN ksl_db.simulation_run AS r
  ON r.run_id = a.sim_run_id_fk
JOIN ksl_db.experiment AS e ON e.exp_id = r.exp_id_fk
WHERE a.stat_name IN ('TimeInShop', 'WorkInProcess',
                      'WLD:Q:TimeInQ', 'PNT:Q:TimeInQ')
ORDER BY a.stat_name, e.exp_name;
```

stat_name	exp_name	average	std_dev	half_width
PNT:Q:TimeInQ	ThreeWelders	482.438	338.937	126.561
PNT:Q:TimeInQ	TwoWelders	235.022	166.786	62.279
TimeInShop	ThreeWelders	33.537	7.282	2.719
TimeInShop	TwoWelders	33.654	7.258	2.710
WLD:Q:TimeInQ	ThreeWelders	0.000	0.000	0.000
WLD:Q:TimeInQ	TwoWelders	264.834	190.690	71.205
WorkInProcess	ThreeWelders	5.200	1.750	0.653
WorkInProcess	TwoWelders	5.217	1.748	0.653

Read Table 12.7 by pairs of rows. The third welder removes the wait at welding: `WLD:Q:TimeInQ`, the minutes a lot waits for a welder, falls from about 265 to 0. The wait at paint, `PNT:Q:TimeInQ`, doubles, from about 235 to 482 minutes. The time in the shop, in hours, barely moves. That is, the lots that no longer wait for a welder reach the powder coat booth sooner and wait there instead, as Table 12.2's utilizations predicted.

12.3.1. A confidence interval computed in SQL

The half-widths of Table 12.7 are computed by the KSL, and an engineer should know how to compute them too, since an analysis often needs a summary the KSL did not store. The replication averages are independent observations of the response's mean, so the standard confidence interval applies (Rossetti 2025): the average of the n replication averages, plus or minus a half-width of t times their standard deviation divided by the square root of n. Here t is the 0.975 quantile of Student's t distribution with $n - 1$ degrees of freedom, for a 95 percent interval.

PostgreSQL has aggregate functions for the average and the sample standard deviation, `AVG` and `STDDEV_SAMP`, but no function for the t distribution's quantiles. The quantile is supplied as a number in the query: for 29 degrees of freedom it is 2.045, from a table of the t distribution, a spreadsheet's `T.INV(0.975, 29)`, or the KSL. The time in shop, summarized from the replication values of `within_rep_view`, is:

Table 12.8.: Time in Shop Summarized in SQL, with t = 2.045

```
SELECT exp_name, COUNT(*) AS reps,
       ROUND(AVG(rep_value)::NUMERIC, 3) AS average,
       ROUND(STDDEV_SAMP(rep_value)::NUMERIC, 3)
           AS std_dev,
       ROUND((2.045 * STDDEV_SAMP(rep_value)
              / SQRT(COUNT(*)))::NUMERIC, 3) AS half_width
FROM ksl_db.within_rep_view
WHERE stat_name = 'TimeInShop'
GROUP BY exp_name
ORDER BY exp_name;
```

exp_name	reps	average	std_dev	half_width
ThreeWelders	30	33.537	7.282	2.719
TwoWelders	30	33.654	7.258	2.710

The check is Table 12.7: the averages, 33.537 and 33.654 hours, and the half-widths, 2.719 and 2.710, agree to the last digit shown. Thus, the query computes what the KSL computes. Notice that `COUNT(*)` is part of the check as well: a count other than 30 would mean a replication was missing, or counted twice.

The two intervals of Table 12.8 overlap almost completely: 33.537 ± 2.719 and 33.654 ± 2.710. Read separately, they say that the difference between the scenarios, about 0.12 hours, is far smaller than the uncertainty in either one. The next example shows that this reading throws information away.

12.3.2. Comparing scenarios by pairing replications

Recall from Section 12.1 that the two experiments used common random numbers: replication 7 of one saw the same arrivals and processing-time variations as replication 7 of the other. The replications are then naturally paired, and the right comparison is of the pairs. The KSL's view `pw_diff_within_rep_view` pairs them: it joins `within_rep_view` to itself on the response name and the replication number, and subtracts. Its `a_value` and `b_value` are the two experiments' values, and `a_minus_b` their difference, where experiment A is the one recorded later, here `ThreeWelders`.

Example 12.1 (Is a third welder worth an operator?)**.** HG Industries' plant manager has been asked to certify an operator on the robotic welder M-07. The welding supervisor argues that lots wait for a welder every day; the manager wants to know what a third welder does for the time a lot of frames spends in the shop, which is what a customer's lead time depends on. The example assumes stage K.

The request. Estimate the change in the mean time in shop when the third welder is used, with a 95 percent confidence interval.

The hand evaluation. Before SQL averages 30 pairs, we pair five by hand. The first five replications of the time in shop are:

Table 12.9.: Time in Shop in the First Five Replications, Both Experiments

```
SELECT rep_id,
       ROUND(b_value::NUMERIC, 3) AS two_welders,
       ROUND(a_value::NUMERIC, 3) AS three_welders,
       ROUND(a_minus_b::NUMERIC, 3) AS difference
FROM ksl_db.pw_diff_within_rep_view
WHERE stat_name = 'TimeInShop'
  AND rep_id <= 5
ORDER BY rep_id;
```

rep_id	two_welders	three_welders	difference
1	28.126	27.745	-0.382
2	28.981	28.962	-0.019
3	36.980	36.971	-0.009
4	31.599	31.131	-0.468
5	27.002	26.859	-0.142

Look first at the two middle columns of Table 12.9. The replications differ from each other by as much as 10 hours, 26.9 to 37.0, and yet the two values in each row are within half an hour. Replication 3 was a bad month for both scenarios: the same arrivals made the same congestion. The differences remove what the scenarios share and keep what distinguishes them. We average them, one row at a time:

Table 12.10.: Averaging the First Five Paired Differences by Hand

Replication	Difference (hours)	Running sum
1	−0.382	−0.382
2	−0.019	−0.401
3	−0.009	−0.410

Replication	Difference (hours)	Running sum
4	−0.468	−0.878
5	−0.142	−1.020

The average of the five differences in Table 12.10 is −1.020 / 5 = −0.204 hours. Their deviations from it are −0.178, 0.185, 0.195, −0.264, and 0.062; the squares sum to 0.1775, and the sample standard deviation is the square root of 0.1775 / 4, or 0.211.

The query. The same summary, first on the five replications and then on all 30, is a `GROUP BY` over the view:

Table 12.11.: The Paired Differences of the First Five Replications Summarized

```
SELECT COUNT(*) AS reps,
       ROUND(AVG(a_minus_b)::NUMERIC, 3) AS average,
       ROUND(STDDEV_SAMP(a_minus_b)::NUMERIC, 3)
           AS std_dev
FROM ksl_db.pw_diff_within_rep_view
WHERE stat_name = 'TimeInShop'
  AND rep_id <= 5;
```

reps	average	std_dev
5	-0.204	0.211

Table 12.11 agrees with the hand evaluation: −0.204 and 0.211. For all 30 replications, with the half-width computed as before:

Table 12.12.: The Paired Difference in Time in Shop, ThreeWelders minus TwoWelders

```
SELECT COUNT(*) AS reps,
       ROUND(AVG(a_minus_b)::NUMERIC, 4) AS average,
       ROUND(STDDEV_SAMP(a_minus_b)::NUMERIC, 4)
           AS std_dev,
       ROUND((2.045 * STDDEV_SAMP(a_minus_b)
              / SQRT(COUNT(*)))::NUMERIC, 4) AS half_width,
       ROUND(CORR(a_value, b_value)::NUMERIC, 4)
           AS correlation
FROM ksl_db.pw_diff_within_rep_view
WHERE stat_name = 'TimeInShop';
```

reps	average	std_dev	half_width	correlation
30	-0.1164	0.1615	0.0603	0.9998

The check. One row, over 30 replications, was expected; Table 12.12 has it. The average, -0.1164 hours, is the difference of the two averages of Table 12.8, as an average of differences must be. Those averages are rounded to three decimals, and $33.537 - 33.654$ gives -0.117; before rounding they are 33.5374 and 33.6538, whose difference is -0.1164.

Interpretation. The 95 percent confidence interval for the change in the mean time in shop is -0.1164 ± 0.0603 hours, from -0.177 to -0.056. It excludes zero, so the third welder does shorten the time in shop; the separate intervals could not show it, and the pairs can because the two scenarios' values move together, with a correlation of nearly 1 (`CORR`, the standard SQL aggregate). The standard deviation of the differences, 0.16 hours, is a small fraction of the replications' own, 7.3 hours, because what the scenarios share cancels.

The difference is also about 7 minutes on a time in shop of 33.6 hours, and that is the answer to the plant manager. A difference can be **statistically significant**, distinguishable from zero with the replications at hand, without being **practically significant**, large enough to matter to the decision. Common random numbers and 30 replications made a 7-minute difference visible; they do not make it worth an operator. The paint booth, whose queue the third welder doubled, is where the time goes.

Where it breaks. The view pairs rows by `rep_id` and requires the two experiments to have the same number of replications (`num_reps`). An experiment run again with 40 replications silently drops out of the view, and the query summarizes no pairs, with a count of 0 and a NULL average, rather than raising an error. In addition, the pairing is meaningful because the experiments used common random numbers; the view pairs any two experiments with equal replication counts, whether or not they did, and no single column of the database records whether they did. The experiments' stream options are evidence. Two experiments run one after the other in one program, as Section 12.1 ran them, share random numbers when both have `reset_start_stream_option` true and the same `num_stream_advances`, `antithetic_option`, and `adv_next_sub_stream_option`, the last of which moves each replication to the next substream so that replication 7 of one experiment draws what replication 7 of the other draws; at stage K both have true, 0, false, and true. Two experiments run as separate programs can share them with `reset_start_stream_option` false, since each program starts the streams from their beginning. Before trusting a paired comparison, you should compare these columns of `ksl_db.experiment` for both experiments and know how the experiments were run.

The comparison of Example 12.1 extends to every response at once by grouping on the response name, which shows where the time moved:

Table 12.13.: Paired Differences, ThreeWelders minus TwoWelders, for Five Responses

```
SELECT stat_name,
       ROUND(AVG(a_minus_b)::NUMERIC, 3) AS average,
       ROUND((2.045 * STDDEV_SAMP(a_minus_b)
              / SQRT(COUNT(*)))::NUMERIC, 3) AS half_width
FROM ksl_db.pw_diff_within_rep_view
WHERE stat_name IN ('SAW:Q:TimeInQ', 'WLD:Q:TimeInQ',
                    'PNT:Q:TimeInQ', 'TimeInShop',
                    'WorkInProcess')
GROUP BY stat_name
ORDER BY stat_name;
```

stat_name	average	half_width
PNT:Q:TimeInQ	247.417	68.080
SAW:Q:TimeInQ	0.000	0.000
TimeInShop	-0.116	0.060
WLD:Q:TimeInQ	-264.834	71.197
WorkInProcess	-0.017	0.009

The saw's row of Table 12.13 is a check of the experiment itself. The saw comes before welding, so a third welder cannot change a lot's wait at the saw, and with common random numbers the waits are the same in every replication: the difference is 0.000 with a half-width of 0.000. A nonzero difference there would mean the scenarios did not share their random numbers. The welding queue loses about 265 minutes and the paint queue gains about 247, and the time in shop and the work in process are nearly unchanged.

The KSL can also make these comparisons itself, with its `MultipleComparisonAnalyzer` class, which computes the pairwise differences of any number of scenarios

(Rossetti 2025). The SQL is worth knowing because the results are in the database long after the program that produced them, and because a question the KSL's reports do not answer is one query away. The next section turns to databases that analysts design for their own experiments.

12.4. Designing a database for experimental work

This section describes the databases that analysts design to hold their own experimental work.

The KSL's `ksl_db` is a database someone designed to hold the results of simulation experiments. Analysts design such databases for their own work as well. An engineer who compares optimization algorithms on a set of test problems, or who tests how well a distribution-fitting procedure recovers a known distribution, produces thousands of results, and the questions asked of them afterwards are grouping queries: which algorithm came within 1 percent of the optimum most often, and how often the procedure chose the right family at each sample size. The KSL and its examples contain two databases of this kind, one for benchmarking the KSL's simulation optimization solvers and one for testing its distribution fitting. This section describes their common shape, the choice between long and wide tables, the other design decisions the shape raises, and the KSL's quick way of building one; the exercises of Section 12.6 carry it into mini-projects.

12.4.1. The common shape

Figure 12.3 draws the shape as a class model. A `Study` is one execution of the experimental plan, recorded with when it ran and with which version of the software. A `Method` is one thing being tested, such as a solver with its settings, and its `MethodSettings` record those settings by name. A `TestCase` is one thing it is tested on, such as a benchmark problem or a

true distribution, often with a known answer to measure against. A `Run` is one method on one test case, one replication, with the random number stream it used, and its `RunResult`s record what it produced by name.

Recall Figure 12.2: the KSL's own output database has the same shape, with an `Experiment` for a study and a scenario, `SimulationRun`s for runs, and statistics for results. The shape recurs because experimental work has a grid: methods crossed with test cases crossed with replications. Each cell of the grid is a run, and every question asked of the study groups the runs by one side of the grid or another.

12.4.2. Long and wide tables

The results of experimental work can be stored in two shapes, and the choice between them comes up in nearly every study. An **observation** is one thing measured, such as one replication, and a **variable** is one thing measured about it, such as its time in shop. A **wide table** has a row per observation and a column per variable. A **long table** has a row per observation and variable, with one column naming the variable and another holding its value. The two hold the same facts in different shapes.

The KSL stores its statistics long. Recall from Section 12.2 that `within_rep_stat` has a row per response per replication and `within_rep_counter_stat` a row per counter per replication; `within_rep_view` combines the two and reduces each row to the replication, the response's name, and its value. For the first three replications of `TwoWelders` and three of its responses, the long table is:

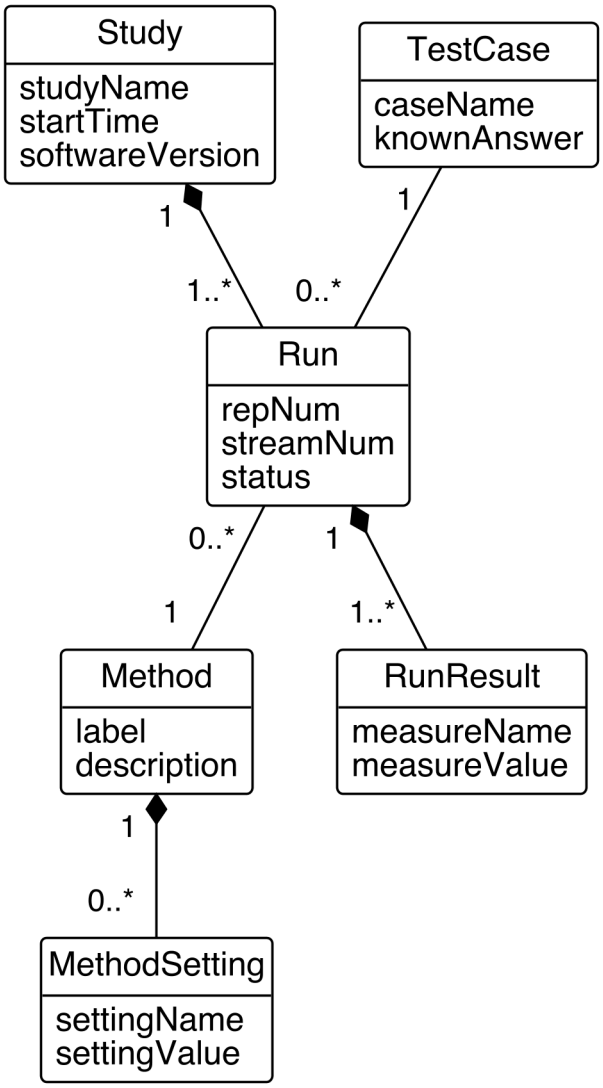


Figure 12.3.: The Common Shape of a Database for Experimental Work

Table 12.14.: Three Responses of Three Replications of TwoWelders, Long

```
SELECT rep_id, stat_name,
       ROUND(rep_value::NUMERIC, 3) AS rep_value
FROM ksl_db.within_rep_view
WHERE exp_name = 'TwoWelders'
  AND stat_name IN ('LotsCompleted', 'TimeInShop',
                    'WorkInProcess')
  AND rep_id <= 3
ORDER BY rep_id, stat_name;
```

rep_id	stat_name	rep_value
1	LotsCompleted	85.000
1	TimeInShop	28.126
1	WorkInProcess	3.908
2	LotsCompleted	85.000
2	TimeInShop	28.981
2	WorkInProcess	4.094
3	LotsCompleted	86.000
3	TimeInShop	36.980
3	WorkInProcess	5.904

Table 12.14 has $3 \times 3 = 9$ rows, one per replication and response. Let's turn it wide by hand. The first step groups the rows by the observation, the replication, giving three groups of three rows. The second step makes one row of each group, placing each value in the column its `stat_name` names:

Table 12.15.: The Rows of Table 12.14 Made Wide by Hand

Replication	Lots completed	Time in shop (hours)	Work in process
1	85.000	28.126	3.908
2	85.000	28.981	4.094
3	86.000	36.980	5.904

Each cell of Table 12.15 is one row of Table 12.14: the row's `rep_id` chooses the cell's row, its `stat_name` the cell's column, and its `rep_value` fills the cell. Nine rows of three columns became three rows of four, and no value was added or lost.

The grouping and placing are a `GROUP BY` on the observation and one `FILTER` per variable (Section 4.4). Each group holds one row for each response, so `MAX` merely returns the one value its `FILTER` keeps:

Table 12.16.: The Same Responses, Wide, by the Query

```
SELECT rep_id,
       ROUND((MAX(rep_value) FILTER (
           WHERE stat_name = 'LotsCompleted'))::NUMERIC, 3)
           AS lots_completed,
       ROUND((MAX(rep_value) FILTER (
           WHERE stat_name = 'TimeInShop'))::NUMERIC, 3)
           AS time_in_shop,
       ROUND((MAX(rep_value) FILTER (
           WHERE stat_name = 'WorkInProcess'))::NUMERIC, 3)
           AS work_in_process
FROM ksl_db.within_rep_view
WHERE exp_name = 'TwoWelders'
  AND rep_id <= 3
GROUP BY rep_id
ORDER BY rep_id;
```

rep_id	lots_completed	time_in_shop	work_in_process
1	85.000	28.126	3.908
2	85.000	28.981	4.094
3	86.000	36.980	5.904

Table 12.16 matches the hand evaluation of Table 12.15, row for row. Notice that the `WHERE` clause no longer lists the responses: a response without a `FILTER` column is grouped and dropped. Thus, the long table's list of variables becomes the wide query's list of columns, and a new variable means a new column in the query.

The wide shape makes a calculation across variables a calculation within a row. Little's law says that, over a long run, the average number of lots in the shop equals the rate at which lots leave it times the average time each

spends there (Rossetti 2025). Each replication observes 25 days, or 600 hours, after its warm-up, so its throughput is its lots completed divided by 600, and the law predicts its work in process:

Table 12.17.: Work in Process Predicted by Little's Law and Observed, TwoWelders

```
WITH wide AS (
    SELECT rep_id,
           MAX(rep_value) FILTER (
               WHERE stat_name = 'LotsCompleted') AS lots,
           MAX(rep_value) FILTER (
               WHERE stat_name = 'TimeInShop') AS hours,
           MAX(rep_value) FILTER (
               WHERE stat_name = 'WorkInProcess') AS wip
    FROM ksl_db.within_rep_view
    WHERE exp_name = 'TwoWelders'
    GROUP BY rep_id
)
SELECT rep_id,
       ROUND((lots / 600 * hours)::NUMERIC, 3)
           AS predicted_wip,
       ROUND(wip::NUMERIC, 3) AS observed_wip
FROM wide
WHERE rep_id <= 5
ORDER BY rep_id;
```

rep_id	predicted_wip	observed_wip
1	3.985	3.908
2	4.106	4.094
3	5.300	5.904
4	4.845	4.716
5	3.690	3.635

The common table expression (Section 3.10) builds the wide table, and the final query does arithmetic across its columns. The prediction and the

observation of Table 12.17 agree closely in most replications. Replication 3, the congested month of Table 12.9, differs most, since the lots still in the shop when the replication ends count in its work in process but have no time in shop to report. In the long table, the same check needs a self-join per variable, one copy of the table for each of the three responses.

Going the other way, from wide to long, takes one `SELECT` per variable, combined with `UNION ALL` (Section 3.9). Each `SELECT` names its variable as a constant and returns its column as the value:

Table 12.18.: The Wide Table Made Long Again

```
WITH wide AS (
    SELECT rep_id,
           MAX(rep_value) FILTER (
               WHERE stat_name = 'LotsCompleted') AS lots,
           MAX(rep_value) FILTER (
               WHERE stat_name = 'TimeInShop') AS hours,
           MAX(rep_value) FILTER (
               WHERE stat_name = 'WorkInProcess') AS wip
    FROM ksl_db.within_rep_view
    WHERE exp_name = 'TwoWelders'
      AND rep_id <= 3
    GROUP BY rep_id
)
SELECT rep_id, 'LotsCompleted' AS stat_name,
       ROUND(lots::NUMERIC, 3) AS rep_value
FROM wide
UNION ALL
SELECT rep_id, 'TimeInShop', ROUND(hours::NUMERIC, 3)
FROM wide
UNION ALL
SELECT rep_id, 'WorkInProcess', ROUND(wip::NUMERIC, 3)
FROM wide
ORDER BY rep_id, stat_name;
```

rep_id	stat_name	rep_value
1	LotsCompleted	85.000
1	TimeInShop	28.126
1	WorkInProcess	3.908
2	LotsCompleted	85.000
2	TimeInShop	28.981
2	WorkInProcess	4.094
3	LotsCompleted	86.000
3	TimeInShop	36.980
3	WorkInProcess	5.904

The check is Table 12.14: Table 12.18 has the same nine rows. `UNION ALL` is the right operator, since the three `SELECT`s cannot produce the same row twice and `UNION` would only spend work looking for duplicates.

Neither shape is right for every purpose. Table 12.19 compares them on the matters that decide a design.

Table 12.19.: Long and Wide Tables Compared

Matter	Long	Wide
Adding a variable	new rows with a new name; no change to the table	a new column: `ALTER TABLE`, and every query that lists the columns changes
Types and constraints	one value column, one type for every variable	each column its own type, `NOT NULL`, and `CHECK`
A missing value	the row is absent	a NULL in the row
One summary for every variable	a single `GROUP BY` on the name	one aggregate per column
A calculation across variables	a self-join per variable, or a pivot first	arithmetic within a row
What statistical tools expect	grouped summaries and plots by variable	regression, correlation, and scatter plots: a row per observation

What counts as an observation and what as a variable depends on the question, not on the data. For the summaries of Section 12.3, a replication of one experiment is an observation and each response a variable. For the paired comparison of Example 12.1, the observation is a replication and each *experiment* is a variable: `pw_diff_within_rep_view` is that wide table, with `a_value` and `b_value` as its two columns, so that the difference is arithmetic within a row.

A table can also be too wide. A spreadsheet with a column for each replication, `time_rep1`, `time_rep2`, and so on to `time_rep30`, has put values of a variable, the replication number, into its column names. It is the repeating group that first normal form forbids (Section 7.4): a 31st replication changes the table, and an average over the replications is 30 columns added by hand rather than one `AVG`. The rule that keeps a table from either excess is the one statisticians call **tidy data** (Wickham 2014): each variable a column, each observation a row. Thus, a study stores its results in the shape that keeps its rules, typed columns for a fixed set of variables and a long table for an open one, and reshapes them for each analysis with a query or a view.

12.4.3. The design decisions

The shape of Figure 12.3 leaves open other decisions as well, which the rest of this book has prepared you to make.

- **Record what produced each result.** A result without its method's settings, its random number stream, and the software version that computed it cannot be reproduced, or trusted when it surprises you. These are columns and rows like any other, and they are cheap to store and impossible to recover later.
- **Choose between long and wide results** (Section 12.4.2). Wide gives each variable its own type and constraints; long accepts a new variable without a change to the schema, which matters when different methods report different things. The KSL's benchmark database stores its solvers' settings, and their state at each iteration, long, since different solvers have different ones.
- **Store the raw results and derive the summaries.** A proportion correct, an average gap, or a count of true positives can be computed by a query from the runs. Stored as well, it is a copy that can disagree with the runs it summarizes, the redundancy of Section 7.2.

- **Key the grid.** A run is identified by its study, method, test case, and replication, and that combination is an alternate key even when a surrogate such as `run_pk` is the primary key (Section 6.6). Declaring it prevents a run from being recorded twice when a study is resumed.
- **Link to data you do not own by its alternate keys.** A study that uses the plant's work centers or the KSL's experiments refers to them by their codes and names, and keeps its own tables in a schema of its own.

12.4.4. Quick tables from data classes

A designed schema takes time, and a study often needs a database the day it starts. Recall from Section 10.4 that a Kotlin data class that extends the KSL's `DbTableData` describes a table, naming it and its key fields, and that the KSL reads and writes the table's rows through the class. The KSL can also create the table from the class, with a column for each `var` property. The class for a solver run might be:

```
data class SolverRun(
    var runId: Int = -1,
    var problemName: String = "",
    var solverLabel: String = "",
    var repNum: Int = 0,
    var bestObjective: Double = 0.0,
    var numReplications: Int = 0,
    var gap: Double? = null
) : DbTableData("solver_run", listOf("runId"))
```

The program `QuickTable.kt` asks the KSL for the statement it would use to create the table:

```
println(SolverRun().createTableSQLStatement())
```

```
CREATE TABLE solver_run (
runId INTEGER NOT NULL,
problemName VARCHAR(512) NOT NULL,
solverLabel VARCHAR(512) NOT NULL,
repNum INTEGER NOT NULL,
bestObjective DOUBLE NOT NULL,
numReplications INTEGER NOT NULL,
gap DOUBLE,
PRIMARY KEY(runId)
)
```

Read the statement against the class. Each property became a column, with its Kotlin type mapped to an SQL type and `NOT NULL` wherever the type does not allow `null`; `gap`, a `Double?`, may be NULL. The key fields became the primary key. Nothing else did: no foreign keys, no `UNIQUE` or `CHECK` constraints, and the column names are the properties' camel case, which PostgreSQL folds to lower case (`runId` becomes `runid`). The type `DOUBLE` is not PostgreSQL's spelling, which is `DOUBLE PRECISION`; the KSL builds these quick databases as SQLite or Derby files (`Database.createSimpleDb`), which accept it, and PostgreSQL refuses the statement as it stands.

Quick tables suit a study that one program writes and one analyst reads, where the tables are few and the rules live in the program that fills them. A designed schema is worth its time when the database will outlive the program, be shared, or be joined to other data, since then the rules must live in the database, where every program that writes it must obey them. The mini-projects of Section 12.6 ask you to design both kinds and to compare them.

12.5. Summary

A simulation model's inputs belong in the database that records them. A routing, its times, and the machines and operators of each work center are already in the production schema; a model that reads them stays current with the plant, and a scenario can be a change to the data.

The KSL records every experiment in a database schema of its own. Experiments own their runs and model elements; statistics are recorded per response per replication, and summarized across replications. The model elements form a tree stored as a nested set, whose subtrees a self-join finds.

Replications are summarized, and scenarios compared, with grouping queries. The average, standard deviation, and half-width of a response over its replications are `AVG`, `STDDEV_SAMP`, and a t quantile supplied as a number, and they agree with what the KSL stores. With common random numbers, pairing the replications of two scenarios cancels what they share, and the paired difference's interval is far narrower than the separate intervals.

A difference can be statistically significant without being practically significant. The third welder shortens the mean time in shop by an amount the pairs can distinguish from zero, about 7 minutes in 33.6 hours, because the queue moves from welding to paint. The decision turns on the size of the difference, not on its significance.

The same results can be stored long or wide. A wide table has a row per observation and a column per variable; a long table has a row per observation and variable. `GROUP BY` with `FILTER` turns long into wide, and `UNION ALL` turns wide into long. Long accepts new variables and summarizes them all at once; wide gives each variable its type and puts a calculation across variables in one row. Which is which depends on the question, and a column per replication is the repeating group that first normal form forbids.

A database for experimental work has a common shape. Methods are run on test cases, replicated, within a study, and each run records its results. Its design records what produced each result, chooses between long and wide results, derives summaries rather than storing them, keys the grid, and links to data it does not own by alternate keys. The KSL's quick tables from data classes suit a study one program writes; a designed schema suits a database that is shared, joined, or kept.

12.6. Exercises

Exercise 12.1. Match each term on the left with the phrase on the right that defines it.

Term	Definition
a. replication	1. starting each scenario from the same random numbers
b. warm-up period	2. a tree stored with a left and a right number for each node
c. common random numbers	3. one run of the model, with its own random numbers
d. nested set	4. the start of each replication, whose statistics are discarded
e. experiment	5. one scenario of a model, run for a number of replications

Exercise 12.2. Besides the routing, list three facts in HG Industries' database, at stage Q or stage G, that a model of the frame line could read instead of assuming, and name the table and columns each comes from. For one of them, say what the model of Figure 12.1 assumes in its place.

Exercise 12.3. Suppose that HG Industries buys a second powder coat booth and certifies an operator on it, while welding keeps its two staffed welders.

a. Compute the utilization of each work center by hand, as Table 12.2 does.
b. Which work center is then the busiest?
c. Which change to the database of stage Q would the model's query of Table 12.1 see, so that the scenario needs no change to the model?

Exercise 12.4. Using stage K:

a. Write a query that lists every model element below the paint station, `PNT`, in experiment 2, with each element's name, class name, and nested set numbers, in the manner of Table 12.4.
b. Before running it, predict the number of rows from the station's own left and right numbers, and say how you obtained them.
c. Explain why the right number of an element is always one more than its left number when the element has no children.

Exercise 12.5. Using stage K, write a query that counts the model elements below each of the three stations, `SAW`, `WLD`, and `PNT`, in experiment 1, using the nested set numbers alone and a single `GROUP BY`. Why should the three counts be equal?

Exercise 12.6. True or false? Explain each answer in a sentence or two.

a. If a 95 percent confidence interval for the difference between two scenarios excludes zero, the difference is large enough to justify the change.
b. The average of `a_minus_b` over the replications in `pw_diff_within_rep_view` equals the difference between the two scenarios' averages over the replications.

c. Since PostgreSQL has no function for the t distribution, a confidence interval cannot be computed in SQL.
d. Had the two experiments of stage K not used common random numbers, the half-width in Table 12.12 would be expected to be larger.
e. A table the KSL creates from a `DbTableData` data class refuses a row whose key names a test case that no other table records.

Exercise 12.7. Using stage K, write a query on `ksl_db.within_rep_view` that gives, for the response `WorkInProcess` in each experiment, the number of replications, the average, the standard deviation, and the half-width of a 95 percent confidence interval, each rounded to 3 decimal places, in the manner of Table 12.8. Compare your half-widths with those the KSL stored in Table 12.7.

Exercise 12.8. Using stage K, write a query on `ksl_db.across_rep_view` that returns one row per queue, the responses whose names end in `:Q:TimeInQ`, with three columns: the response name, its average under `TwoWelders`, and its average under `ThreeWelders`, the averages rounded to 1 decimal place. Use `FILTER` (Section 4.4), and order the rows by the response name. Which work center's queue changes least, and why?

Exercise 12.9. Using stage K:

a. Write a query that gives the paired difference, `ThreeWelders` minus `TwoWelders`, in the response `LotsCompleted`: the number of pairs, the average difference, and the half-width of a 95 percent confidence interval, each rounded to 3 decimal places.
b. Does the interval include zero? Explain in terms of the frame line why a third welder should change the number of lots completed so little.

Exercise 12.10. Using stage K:

a. Before writing any query, predict how many rows `ksl_db.within_rep_view` holds for the responses `SAW:Q:TimeInQ`, `WLD:Q:TimeInQ`, and `PNT:Q:TimeInQ` in the experiment `ThreeWelders`, and how many rows and columns the same values occupy in a wide table with a row per replication.
b. Write a query that returns that wide table for the first five replications of `ThreeWelders`: the replication number and the three queue times, each rounded to 1 decimal place, in replication order.
c. Add a column giving the total of the three queue times, in minutes. Why is the total easier to compute in the wide shape than in the long one?

Exercise 12.11. Using stage K, write a query on `ksl_db.within_rep_view`, without using `pw_diff_within_rep_view`, that returns for the response `TimeInShop` one row per replication with the value under `TwoWelders`, the value under `ThreeWelders`, and `ThreeWelders` minus `TwoWelders`, each rounded to 3 decimal places, for the first five replications. Compare your result with Table 12.9, and say what counts as the observation and what as the variables in your table.

Exercise 12.12. An engineer keeps the results of a study in a spreadsheet with one row per scenario and the columns `scenario`, `tis_rep1`, `tis_rep2`, …, `tis_rep30`, `wip_rep1`, …, `wip_rep30`, the time in shop and the work in process of each replication, 61 columns in all.

a. Name the observations and the variables of the study, and say what is wrong with the spreadsheet's shape, in the terms of Section 12.4.2.
b. Give two tidy designs for a table holding the same values, one long and one wide, as `CREATE TABLE` statements, and say how many rows each holds for two scenarios.
c. For each of your designs, say what changes when the engineer runs 10 more replications, and when the engineer adds a third response, the number of lots completed.

The remaining exercises are mini-projects. Each asks for a design, the SQL that builds and queries it, and a short written justification, in the manner of Section 12.4.

Exercise 12.13 (Mini-project: a staffing study for the frame line)**.** After Example 12.1, HG Industries' plant manager asks for a comparison of three options for the frame line: `TwoWelders`, the line as it is staffed (one staffed machine at SAW, two at WLD, one at PNT); `ThreeWelders`, with an operator certified on M-07 (three at WLD); and `SecondBooth`, with a second powder coat booth, installed and staffed (two at PNT). Training the M-07 operator costs $2,400 once, and installing the booth $45,000 once. The cost per hour of running the line is the sum, over the work centers, of the staffed machines times the work center's `hourly_rate` in `production.work_center`. The results of `TwoWelders` and `ThreeWelders` are the experiments of those names in stage K; `SecondBooth` has not been simulated.

a. Draw a class model of what the study adds, with the classes of the production schema and of `ksl_db` that it links to.
b. Map it to tables in a schema `study`, and write the `CREATE` statements, following the book's conventions for keys and names. A scenario must be recordable before it has been simulated.
c. Write the statements that load the three scenarios and their staffing.
d. Write a query that returns, for each scenario, its name, its one-time cost, its cost per hour, and the average and half-width of its time in shop from `ksl_db.across_rep_stat`, keeping the scenarios not yet simulated. Run parts (b) to (d) at stage K in one transaction, ending with `ROLLBACK`, and state the result.
e. Explain how your design links a scenario to its experiment, and why it is not a foreign key to `ksl_db.experiment`.
f. The manager then asks for scenarios with lots of 50 frames instead of 100. Assuming the study covers only the frame, FR-110, say where the lot size belongs in your class model and give the change to your tables as SQL. How would the answer change if the study

also covered the cart frame, CF-310, each part with its own lot size in each scenario?

Exercise 12.14 (Mini-project: a database for benchmarking optimization solvers)**.** A benchmark study runs solver configurations on test problems. A solver configuration has a label, such as `SA-slow`, the name of the solver algorithm it uses, and its parameter settings by name, such as an initial temperature of 100; different algorithms have different parameters. A test problem has a name, whether it is minimized or maximized, its input variables, each with a name and lower and upper bounds, and, for some problems, the known optimal objective value. Every configuration is run on every problem for a number of macro-replications, each run under the same budget of simulation replications. A run records its random number stream, its starting point and the best point it found (a value for each input variable), the best objective value, the number of replications it used, its status (`completed` or `failed`), and, when the optimum is known, its gap, the absolute difference between its best objective value and the optimum divided by the absolute value of the optimum. A run may also record a trace: for each iteration, the replications used so far and the best objective value so far.

The study must answer: (Q1) for each problem and configuration, the proportion of completed runs with a gap of at most 0.01; (Q2) the average number of replications used, by configuration; and (Q3) a given run's trace, in iteration order.

a. Draw a class model of the study, and say which classes of Figure 12.3 each of yours corresponds to.
b. Map it to PostgreSQL tables in a schema `benchmark`, and write the `CREATE` statements with their keys, foreign keys, and `CHECK` constraints.
c. The points (starting and best) have one value per input variable, and the number of variables differs from problem to problem. Compare storing a point as rows (one per variable) with storing it as a single text value (for example JSON), and choose.

d. Write Q1, Q2, and Q3 as queries against your tables.
e. The KSL's own benchmark database is defined by the data classes in `KSLCore/src/main/kotlin/ksl/simopt/benchmark/io/BenchmarkDbData.kt` of the KSL source (https://github.com/rossetti/KSL). Read it and name two decisions its designers made differently from yours. For each, say what it gains and what it costs.

Exercise 12.15 (Mini-project: a database for testing distribution fitting)**.** A study tests a distribution-fitting procedure by fitting samples from known distributions. Each test case is a true distribution: a family, such as gamma, and its parameter values by name, such as a shape of 2 and a scale of 5. For each case and each sample size in a set, such as 25, 50, 100, and 200, the study generates a number of samples, the same number for every size, and records each sample's average, standard deviation, skewness, minimum, and maximum. It fits each candidate family to each sample and records, for each fitted family, its parameter estimates by name, its score under each of several scoring models (such as the chi-squared, Kolmogorov-Smirnov, and Anderson-Darling statistics), its rank under each scoring model used alone, and its overall rank, where rank 1 is the family the procedure recommends.

The study must answer: (Q1) for each case and sample size, the proportion of samples for which the true family has overall rank 1; (Q2) for each scoring model, the proportion of samples for which it alone would rank the true family first; and (Q3) for each case and sample size, the average relative error of each parameter's estimate when the true family is fitted.

a. Draw a class model of the study.
b. Map it to PostgreSQL tables in a schema `fitting`, and write the `CREATE` statements.
c. Scores, ranks, and parameter estimates can each be stored long (a row per named value) or wide (a column per value). Decide for each, and justify the decision against the study's questions.
d. Write Q1 and Q3 as queries against your tables.

e. The KSL's examples hold a database for such a study, defined in `KSLExamples/src/main/kotlin/ksl/examples/general/utilities/fitting/ResultsDb.kt` of the KSL source. Find in it (1) a table with columns whose values can be computed from its other columns, and (2) a table whose single value column holds several kinds of result. For each, say what the choice costs and why its authors might have made it.

Exercise 12.16 (Mini-project: quick tables and a designed schema)**.** Take the run table of your design for Exercise 12.14, or, if you have not done that exercise, the class `SolverRun` of Section 12.4 with a result table of your own.

a. Write `DbTableData` data classes for the run table and for one table that refers to it, and print the `CREATE TABLE` statement the KSL generates for each, as `QuickTable.kt` does.
b. List every rule of your designed tables that the generated statements do not state: keys, foreign keys, `UNIQUE` and `CHECK` constraints, types, and names.
c. Run one generated statement at any stage of the book's database, in a transaction ended by `ROLLBACK`, and explain what PostgreSQL reports.
d. Write a paragraph advising a colleague when the quick tables will do for a study and when the designed schema is worth its time.

Exercise 12.17 (Mini-project: a database for your own analysis)**.** Choose an analysis question about HG Industries' operations that the book's database can support, for example the capacity of the garden cart line, the effect of inspection on the time in shop, or the reorder points of the Fayetteville warehouse.

a. State the questions the study must answer, as requests in words.
b. List the data each question needs, and for each, the schema it comes from or that it belongs in.

c. Draw a class model of what the study adds, with the classes it links to, and compare it with Figure 12.3.
d. Map it to a schema of the study's own, as SQL, and load realistic data, enough that some rows do not contribute to each question's answer.
e. Write each question as a query, and state its result.
f. Write down your assumptions, grouped as (1) about the classes and their attributes, (2) about the associations, and (3) about everything else, and keep a log of your work: the date, what you did, and why.

A. Installing PostgreSQL and Its Tools

This appendix sets up what the book's examples need: a PostgreSQL server on your own computer, DataGrip or DBeaver to work with it, and the book's database loaded into it. It also introduces psql and pgAdmin, which come with PostgreSQL, so that you recognize them. The steps take under an hour, most of it waiting for downloads.

The book's results were produced with PostgreSQL 18. Later releases give the same query results, but the order of the rows in a result without `ORDER BY`, what a result shows of the database's own catalog, and the wording of an error message can differ from one release to the next. PostgreSQL publishes a new major release about once a year, and supports each for five years. Install the current one unless your course or employer asks for another.

A.1. Installing the PostgreSQL server

PostgreSQL runs as a **server**, a program that starts with your computer and waits for connections (Section 1.3). Installing it creates the server, a first database named `postgres`, and a first user, a superuser who may do anything. The installers differ by operating system.

- **macOS.** Postgres.app (`postgresapp.com`) is the simplest: move it to the Applications folder, open it, and choose *Initialize*. The server

then runs whenever the app does, and its first user has your macOS login name and no password. To use psql and the other command-line tools from a terminal, add them to your path once, as the app's site describes:

```
sudo mkdir -p /etc/paths.d &&
echo
 ↪  /Applications/Postgres.app/Contents/Versions/latest/bin
 ↪  |
sudo tee /etc/paths.d/postgresapp
```

Homebrew (`brew install postgresql@18`) and the EDB installer below also work on macOS.

- **Windows.** Download the interactive installer by EDB from the Windows page of `postgresql.org/download`. Keep the default components (the server, pgAdmin 4, and the command-line tools); Stack Builder is not needed. The installer asks for a password for the superuser, named `postgres`: choose one you will remember, since every tool asks for it. Keep the default port,

  5432. The server then starts with Windows. The installer does not put the command-line tools on your path: search the Start menu for *Edit environment variables for your account*, select `Path`, choose *Edit*, then *New*, enter the folder `C:\Program Files\PostgreSQL\18\bin`, choose *OK* in both windows, and open a new terminal. A terminal inside an application that was already open, such as IntelliJ IDEA's, keeps the old path until the application restarts. The Start menu's *SQL Shell (psql)* also opens psql, and asks for each connection setting in turn, showing its default in brackets; press Enter to accept it.

- **Linux.** Install the server from your distribution's packages, or from the PostgreSQL project's own repository for a newer release, as the

Linux page of `postgresql.org/download` explains. The superuser is `postgres`, reached with `sudo -u postgres psql`.

To check the installation, open a terminal and ask psql for its version:

```
psql --version
```

If the command is not found, the tools are not on your path; add them as the step for your system describes. The rest of this appendix assumes the server runs on your own computer (`localhost`) at port 5432.

The superuser is all you need for the book: you create and load your own databases on your own computer. On a shared server, you would instead be given a role with only the privileges your work needs, as Section 9.4 describes.

A.2. Connecting with DataGrip

DataGrip, by JetBrains, is a database tool for many DBMSs, and the one this book's steps assume. It is free for non-commercial use, including learning, from version 2025.2.4, and free to students with a JetBrains student license; choose *Non-commercial use* when it first starts.

To connect to your server:

1. Choose *File*, *New*, *Data Source*, *PostgreSQL*.
2. Enter the host `localhost`, the port `5432`, your user (your login name with Postgres.app, `postgres` otherwise) and its password, and the database `postgres` for now.
3. If DataGrip offers to download the PostgreSQL driver, accept.
4. Choose *Test Connection*, then *OK*.

DataGrip lists the new data source in its *Database Explorer*. By default it shows only some of a database's schemas; open the data source's properties and, on its *Schemas* tab, select *All schemas*, or the book's schemas will seem to be missing after you load them.

Two ways of running SQL cover everything the book asks. A **query console** (right-click the data source, *New*, *Query Console*) holds statements you type; place the cursor in one and press Ctrl+Enter (Cmd+Enter on macOS) to run it. A **script file** (*File*, *Open*, then choose the data source to run it against) runs whole: right-click in it and choose *Run*. The book's load files are script files.

A.3. Connecting with DBeaver

DBeaver is a database tool for many DBMSs whose Community Edition is free and open source (`dbeaver.io`). The book's steps work in it as well as in DataGrip.

1. Choose *Database*, *New Database Connection*, *PostgreSQL*.
2. Enter the host `localhost`, the port `5432`, the database `postgres`, and your user and password, as for DataGrip.
3. On the *PostgreSQL* tab, select *Show all databases*, so that the databases you create later appear.
4. Accept the offer to download the driver, choose *Test Connection*, then *Finish*.

Open an SQL editor on the connection (*SQL Editor*, *New SQL script*). Press Ctrl+Enter to run the statement under the cursor, and Alt+X to run the whole script. The book's load files are run with Alt+X.

A.4. psql and pgAdmin

Two tools come with PostgreSQL itself. The book does not build on either, but you will meet both.

psql is PostgreSQL's command-line client. It connects to a database, runs SQL typed at its prompt or read from a file, and adds its own commands, which begin with a backslash. Run from a terminal, `psql -d dbconcepts` connects to the book's database; `\dn` lists its schemas, `\dt distribution.*` the tables of one schema, `\d distribution.customer` one table's columns, keys, and constraints, `\i file.sql` runs a script, and `\q` quits. On Windows, psql connects as your Windows login name, which is not a role on the server, so name the superuser, `psql -U postgres -d dbconcepts`, and give the password you chose at installation. In the Command Prompt, psql may also warn that the console code page differs from the Windows code page. The warning concerns only how accented letters show in that window, such as those in some names of the public sample databases, and DataGrip and DBeaver are not affected. On Linux, run psql as the superuser, `sudo -u postgres psql -d dbconcepts`. Listing the schemas of the book's database, `dbconcepts`, gives:

```
         List of schemas
     Name     |       Owner
--------------+-------------------
 distribution | rossetti
 production   | rossetti
 public       | pg_database_owner
 quality      | rossetti
(4 rows)
```

The owner is the user who loaded the database, here the author's login name. psql is the quickest way to run a script from a terminal, and the book's own tooling uses it for that.

pgAdmin is a graphical administration tool, installed with the Windows installer and available separately for macOS and Linux. It shows a server's databases, schemas, tables, roles, and settings, and has a query tool. It suits administration (Section 1.3); for the work of this book, DataGrip or DBeaver is the better fit.

A.5. Creating the book's databases

The book's database is loaded into two databases of its own, which you create once and load once (Table C.2): `dbconcepts`, the small world that most of the book uses, and `dbconcepts_history`, the same database with a year of history and the sales warehouse. Create both, connected to the `postgres` database, in a DataGrip or DBeaver console:

```
CREATE DATABASE dbconcepts;
CREATE DATABASE dbconcepts_history;
```

or from a terminal with `createdb dbconcepts` and `createdb dbconcepts_history` (on Windows, `createdb -U postgres dbconcepts`, and the same for the second). Then change your data source to the `dbconcepts` database (in DataGrip, edit the data source's *Database* field; in DBeaver, select it under the connection) and check the connection by running the companion script `code/sql/setup/check-connection.sql`. It reports the server's version and which database and user you are connected as; its second result should read:

```
 current_database | current_user
------------------+--------------
 dbconcepts       | rossetti
(1 row)
```

with your own user name in place of the author's. Now load the database: open `code/sql/load/through-q.sql`, from the book's companion code (https://github.com/rossetti/DbConceptsBookCode), and run it as a script. It builds the distribution, production, and quality schemas in a few seconds.

Make a second data source (DataGrip) or connection (DBeaver) in the same way, for `dbconcepts_history`, and in it run `code/sql/load/through-s.sql`, which adds the year of history and the `sales` schema. Finally, in each database, run `code/sql/setup/which-stage.sql`. It should report `through-q.sql` in `dbconcepts` and `through-s.sql` in `dbconcepts_history`, with no reset needed. Section C.2 explains the stages, and how to use the load files to reset a database.

If a load file does not finish, but runs on with no message, another console is holding a transaction open on the same database, usually one in which a `BEGIN` was never followed by `COMMIT` or `ROLLBACK`. The load file waits for that transaction to end. Find the console, roll it back (in DataGrip, the console's *Rollback* button; in DBeaver, its *Rollback* toolbar button), or close it, and the load file finishes.

Millbrook Transit, the exercise database of Section C.10, goes in a database of its own: create `dbconcepts_transit` as above and run `code/sql/transit/t-schema.sql` and then `code/sql/transit/t-data.sql`. The public sample databases, next, have a loader of their own.

A.6. Loading public sample databases

The public sample databases of Section C.11 are not distributed with the book. The companion code's `code/sql/public/` holds what is needed to fetch and load them: `manifest.tsv`, which gives each file's address at

a fixed release or commit of its source, its size, its license, and its SHA-256 checksum; `load-public.sh`, which does the work; and `practical-schema.sql`, which defines the tables for the Practical SQL files.

On macOS or Linux, or in Git Bash on Windows, with PostgreSQL running and `psql` on the path, run the loader with the database's name. In Git Bash, first name the superuser and its password, so that each of the loader's psql commands can connect:

```
export PGUSER=postgres
export PGPASSWORD='your-password'
```

The two settings last until the window closes. Do not save the password in a script or in any file you share. Then run:

```
code/sql/public/load-public.sh chinook
code/sql/public/load-public.sh northwind
code/sql/public/load-public.sh practical
code/sql/public/load-public.sh employees
```

Each command downloads the files once, into `~/.cache/dbconcepts-public`, refuses any file whose checksum differs from the manifest's, creates the database `dbconcepts_<name>`, and loads it. The employees database downloads about 172 MB and takes a minute or so to load; the others take seconds. A second argument names a different database, and `--replace` drops and rebuilds a database that exists; without it, the loader stops rather than overwrite anything.

Without a shell, the same can be done by hand. Create the database in DataGrip or DBeaver, download each file listed for it in `manifest.tsv`, and run it: Chinook's script after deleting its first lines, which create and connect to a database of its own; Northwind's as it is; for Practical SQL, `practical-schema.sql` followed by an import of each CSV file into its

table (Section 8.5); and for employees, the schema script `employees.sql`, less its database lines, followed by the eight `load_*.dump` files after removing the MySQL backquotes (`) they contain. The loader's comments give each step exactly.

B. Kotlin and KSL Setup

Most of this book needs only PostgreSQL and a database tool (Appendix A). Two chapters also run programs: Chapter 10, which connects Kotlin programs to the database, and Chapter 12, which reads a simulation model's inputs from the database and analyzes its results. Their programs are written in Kotlin and use the Kotlin Simulation Library, the KSL, whose database utilities wrap JDBC (Rossetti 2025). This appendix sets up what those programs need: a Java Development Kit, an editor, the companion code project, and its connection to your database.

B.1. Installing a JDK and IntelliJ IDEA

Kotlin programs run on the Java Virtual Machine, so they need a **Java Development Kit** (JDK). The companion project asks for version 21, the version the KSL is built for, and you need not install it yourself: the project's build is configured to download a JDK 21 the first time it runs on a machine that has none. If you prefer to install one, any JDK 21 distribution works, such as Eclipse Temurin from `adoptium.net`.

The book uses **IntelliJ IDEA**, JetBrains' editor for Kotlin and Java, from `jetbrains.com/idea`. Since release 2025.3 it is one download whose free features include everything the companion code needs; a subscription adds features the book does not use. IntelliJ IDEA can also download a JDK for you, from *File*, *Project Structure*, *SDKs*.

If you use the KSL in a simulation course, you may already have both. The KSL's own textbook has an appendix on the same setup (Rossetti 2025), and the companion project here is an ordinary Gradle project like its examples.

B.2. Opening the companion code project

The companion code is the public repository https://github.com/rossetti/DbConceptsBookCode. Download it from there (*Code*, then *Download ZIP*) and unzip it, or clone it with Git. The book calls the folder that holds it, the one that contains `build.gradle.kts`, the `code` folder. It holds the SQL scripts the chapters refer to and the Kotlin programs, and it is a **Gradle** project: `build.gradle.kts` declares what the programs depend on, the KSL among them, and the Gradle wrapper (`gradlew`, or `gradlew.bat` on Windows) downloads Gradle and those dependencies the first time the project is built.

To open it, start IntelliJ IDEA, choose *File*, *Open*, and select the `code` folder, the one that contains `build.gradle.kts`. IntelliJ IDEA recognizes the Gradle project and imports it, downloading the dependencies from Maven Central; the first import takes a few minutes, and later ones seconds. The project's `README.md` describes its layout. The programs are under `src/main/kotlin/dbconcepts/`, one package for each subject: `connecting` for the connection itself, `programming` for the programs of Chapter 10, and `simulation` for those of Chapter 12.

B.3. Configuring the database connection

The programs read their connection settings from a file, `db.properties`, in the `code` folder, so that no password is written into a program (Sec-

tion 10.2 explains why that matters). The project provides a template, `db.properties.example`; copy it to `db.properties` and fill it in:

```
host=localhost
port=5432
database=dbconcepts
user=postgres
password=
```

Give the user and password you connect with in DataGrip or DBeaver (Section A.1): with Postgres.app, your login name and an empty password; with the Windows installer, `postgres` and the password you chose. The database is `dbconcepts`, created and loaded with `through-q.sql` as Section A.5 describes, which is what the programs of Chapter 10 and Chapter 12 expect.

Keep `db.properties` to yourself. It holds a password, so it is excluded from the project's version control, and you should not e-mail it or copy it into a report. Every program opens the database the same way, through the object `BookDatabase` in the `connecting` package, which reads the file.

B.4. Running your first program

The first program to run is `CheckConnection.kt`, in the `connecting` package. It connects with your settings, asks the server for its version, and lists the tables of each of the book's schemas:

```
val rs = db.fetchCachedRowSet("SELECT
 ↪  current_setting('server_version')")
if (rs != null && rs.next()) {
    println("Connected to PostgreSQL ${rs.getString(1)}")
```

```
} else {
    println("Could not query the server. Check the settings
 ↪  in db.properties.")
    return
}
for (schema in listOf("distribution", "production",
 ↪  "quality")) {
    val tables = db.tableNames(schema).sorted()
    if (tables.isEmpty()) {
        println("$schema: (not loaded)")
    } else {
        println("$schema:")
        tables.chunked(4).forEach { println("    " +
 ↪  it.joinToString(", ")) }
    }
}
```

In IntelliJ IDEA, open the file and click the green arrow beside `fun main()`. From a terminal in the `code` folder, the Gradle wrapper runs it:

```
./gradlew run
```

In the Windows Command Prompt, the command is `gradlew run`. With `dbconcepts` loaded, it prints:

```
Connected to PostgreSQL 18.6 (Postgres.app)
distribution:
    customer, customer_order, order_line, product
    shipment, shipment_line, stock_level, warehouse
production:
    bom_line, certification, machine, manufactured_part
    operation, operator, part, purchased_part
```

```
    work_center, work_order
quality:
    characteristic, defect_count, defect_type, inspection
    measurement
```

The first line gives your server's version, which may differ from the book's; the words in parentheses appear only with Postgres.app. If the program stops instead, its message says why. *No connection settings* means `db.properties` is missing from the `code` folder. A failed password means the user or password in the file is not the one the server expects. A refused connection means the server is not running, or is not at the host and port in the file. A database that does not exist means `dbconcepts` has not been created yet. A schema reported as *(not loaded)* means the database exists but the load file has not been run.

Every other program runs the same way: from IntelliJ IDEA, with the green arrow beside its `main`, or from a terminal by naming its class, the file's name followed by `Kt`:

```
./gradlew run
↪  -PmainClass=dbconcepts.programming.LotMeasurementsKt
```

The simulation of Section 12.1, `RunFrameLine.kt` in the `simulation` package, writes the KSL's output schema, `ksl_db`, into the database it connects to. Run on your `dbconcepts`, it adds stage K of Section 12.2, with the same statistics, since the model's random number streams are fixed. If it cannot be run, `code/sql/load/add-k.sql`, run in DataGrip or DBeaver, creates the same `ksl_db`. The KSL also writes its reports and any files a program creates under `kslOutput` in the `code` folder.

C. The Book's Databases

The examples and exercises in this book use one database, built for the book. It records the business of HG Industries, a small manufacturer of hand trucks and garden carts, as its distribution, production, and quality departments see it. The chapters introduce the database a few tables at a time, as each new idea needs them. This appendix shows it whole: what it contains, how to load it, how its tables and columns are named, and what each table holds.

C.1. HG Industries

HG Industries makes four products: two hand trucks and two garden carts. It sells them to hardware stores, farm stores, and distributors from two warehouses, one at the plant in Fayetteville, Arkansas, and one in Tulsa, Oklahoma. The products are simple enough to picture and still have the structure of real manufactured goods. For example, both hand trucks and both carts roll on the same wheel assembly, which is itself built from purchased tires, rims, and bearings and from an axle the plant machines from bar stock.

Each department keeps its own data, and each department's data is a separate **schema**, a named group of tables within one database. Table C.1 lists them. The `sales` schema is not a department's working data. It is built from the other schemas for analysis, and Chapter 11 explains why a company keeps its data twice in this way.

Table C.1.: Schemas of the Book's Database

Schema	What it records	Tables
`distribution`	Customers, their orders, the products ordered, stock in the warehouses, and shipments	8
`production`	Parts, bills of materials, work centers and their machines, operators and their certifications, routings, and work orders	10
`quality`	Characteristics of parts with their specification limits, inspections, measurements, and defects found	5
`sales`	A star schema for analyzing sales, built from `distribution` and `production`	4

The schemas refer to one another where the business connects. A product that distribution sells is one of the parts production makes, and an inspection in quality examines one of production's parts and is performed by one of production's operators. Thus, a query can join tables across departments, as the company's own reports must.

C.2. Loading the database

The database is built by SQL scripts that come with the book, in the `sql` folder of the companion code project (https://github.com/rossetti/DbConceptsBookCode) and as downloads from the book's site. Appendix A explains how to install PostgreSQL and connect to it with DataGrip or DBeaver, and Section A.5 how to create the two databases you work in and load each one once. Table C.2 lists them.

Table C.2.: The Two Databases You Work In

Database	Loaded with	Holds	Used by
`dbconcepts`	`through-q.sql`	The distribution, production, and quality schemas, with the base data of January to March 2026	Most examples, activities, and exercises
`dbconcepts_history`	`history-through-s.sql`	The same schemas with a year of history added, April 2026 to March 2027, and the `sales` schema	The examples that need volume, in Chapter 4 and Chapter 7, and Chapter 11

Chapter 12 adds one more schema to `dbconcepts`: `ksl_db`, the output of a simulation, written by the companion program `RunFrameLineKt` or, if you cannot run it, by the script `sql/load/add-k.sql`.

The scripts build the database in **stages**, each adding one idea's tables and rows to the stages before it. The stages are the order in which the book designs the database, and the chapters name them as they introduce each idea. Table C.3 lists them in the order they load, with the database that holds each. For example, the single-table queries of Chapter 3 need only stage D1, and the analytical queries of Chapter 4 need the year of history in stage G. You need not load a stage to use it: `dbconcepts` holds every stage from D1 to Q, and `dbconcepts_history` every stage from D1 to S. Thus, a chapter's example at stage P3 runs in `dbconcepts`, and one at stage G in `dbconcepts_history`.

Table C.3.: Stages of the Book's Database

Stage	Adds	The new idea	In
D1	`customer`	One table: rows, columns, and a key	both
D2	`customer_order`	A one-to-many association and its foreign key	both
D3	`product`, `order_line`	A many-to-many association resolved by a table with its own data	both
D4	`warehouse`, `stock_level`	A second association class	both
D5	`shipment`, `shipment_line`	Optional participation	both
P1	`part`, `manufactured_part`, `purchased_part`	Generalization; connecting distribution to production	both
P2	`bom_line`	A recursive association, the bill of materials	both
P3	`work_center`, `machine`, `operator`, `certification`, `operation`, `work_order`	Qualified associations and aggregation	both
Q	the `quality` schema	Measurements over time	both
G	a year of history	Enough data to analyze	`dbconcepts_history`
S	the `sales` schema	A star schema	`dbconcepts_history`
K	the `ksl_db` schema	A simulation's output	`dbconcepts`

Each load file, in the `sql/load` folder, is named for its stage (`through-d1.sql`, `through-d2.sql`, and so on to `through-s.sql`) and rebuilds the database as it stands at that stage. It first deletes the four schemas of Table C.1, then runs every stage script up to its own. A load file touches no schema other than these four, so a table you create in the `public` schema, or in a schema of your own, survives it, and so does `ksl_db`. Thus, a load file is a **reset**: running `through-q.sql` in `dbconcepts`, or `through-s.sql` in `dbconcepts_history`, returns the database to the state the book assumes,

whatever you have done to it. The other load files show the database as it stood at an earlier stage; if you run one, run `through-q.sql` afterward.

In DataGrip or DBeaver, open the load file, choose the connection to the database, and run the whole script. From a terminal, psql does the same:

```
psql -d dbconcepts -f sql/load/through-q.sql
```

The results in this book, and the answers to its activities and homework, assume a database as its load file leaves it. A change you made in an earlier activity, or a transaction you did not roll back, can make a correct query give a different answer. Before an activity or a homework, run `sql/setup/which-stage.sql` in the database you will use. It reads every table of the four schemas, changes nothing, and returns one row: `loaded_with` names the load file the database matches, `reset` says whether to run it again, `changed` lists each table that differs from the load (changed, missing, or not in the load), and `simulation` says whether `ksl_db` is there and complete.

The stage scripts themselves are in `sql/distribution`, `sql/production`, `sql/quality`, `sql/history`, and `sql/sales`, one file per stage. They are worth reading, since each is a short, complete example of defining and populating tables. Run them only through a load file, though. Each assumes the stages before it were loaded into a freshly reset database, because its rows refer to earlier rows by their key numbers.

C.3. Naming and keys

Every table follows the same rules, and knowing them makes any table readable at a glance. Chapter 2 and Chapter 6 explain the reasoning behind them.

- **Table names** are singular nouns in lower case with underscores (`customer`, `order_line`), like the class each table implements. No table or column is named with a word reserved in the SQL standard. That is why an order is a `customer_order` (`ORDER` is reserved) and a date column is `full_date` rather than `date`.
- **Primary keys** are named for their table with the suffix `_pk` (`customer.customer_pk`). They are surrogate keys: integers that PostgreSQL generates, which carry no business meaning.
- **Business identifiers**, the numbers people use (`customer_number`, `part_number`, `order_number`), are separate columns declared `UNIQUE`. Where a row is identified by two columns together, such as an order line by its order and its line number, the pair is declared `UNIQUE`.
- **Foreign keys** are named for the table they refer to with the suffix `_fk` (`customer_order.customer_fk`). Where the table's name would not say what a reference means, the column is named for its role: a bill of materials line has an `assembly_fk`, referring to `manufactured_part`, and a `component_fk`, referring to `part`, and a sales fact has an `order_date_fk`.

C.4. The distribution schema

Figure C.1 shows the class model of the distribution schema. A customer places orders, and each order is made of order lines, numbered within the order, that name a product and a quantity. Notice that the order line is drawn with a qualifier, `lineNumber`: an order and a line number together identify at most one line. The stock of each product in each warehouse is an association class, `StockLevel`, because a quantity on hand belongs to neither the warehouse nor the product alone. An order may have any number of shipments, including none, and each shipment line ships some quantity of one order line.

Table C.4 lists the tables that implement the model. The foreign key from `product` to `production.manufactured_part` is added in stage P1, when

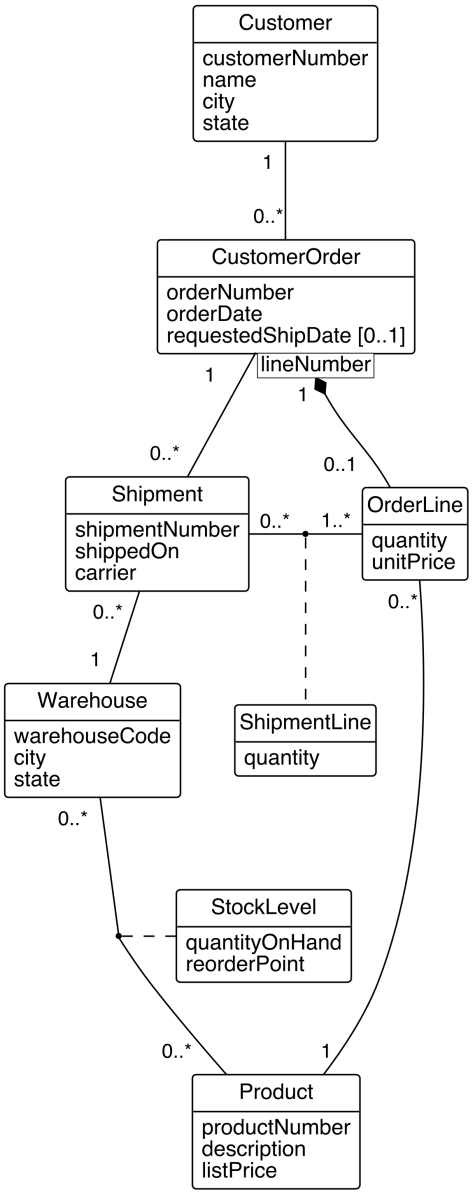


Figure C.1.: Distribution Schema Class Model

the production schema exists to refer to.

Table C.4.: Tables of the Distribution Schema

Table	Columns, in order, and their keys
`customer`	`customer_pk` (primary key), `customer_number` (unique), `name`, `city`, `state`
`customer_order`	`customer_order_pk` (primary key), `order_number` (unique), `order_date`, `requested_ship_date` (optional), `customer_fk` (refers to `customer`)
`product`	`product_pk` (primary key), `product_number` (unique), `description`, `list_price`, `manufactured_part_fk` (refers to `production.manufactured_part`)
`order_line`	`order_line_pk` (primary key), `customer_order_fk` (refers to `customer_order`), `line_number`, `product_fk` (refers to `product`), `quantity`, `unit_price`; (`customer_order_fk`, `line_number`) is unique
`warehouse`	`warehouse_pk` (primary key), `warehouse_code` (unique), `city`, `state`
`stock_level`	`stock_level_pk` (primary key), `warehouse_fk` (refers to `warehouse`), `product_fk` (refers to `product`), `quantity_on_hand`, `reorder_point`; (`warehouse_fk`, `product_fk`) is unique
`shipment`	`shipment_pk` (primary key), `shipment_number` (unique), `shipped_on`, `carrier`, `customer_order_fk` (refers to `customer_order`), `warehouse_fk` (refers to `warehouse`)

Table	Columns, in order, and their keys
`shipment_line`	`shipment_line_pk` (primary key), `shipment_fk` (refers to `shipment`), `order_line_fk` (refers to `order_line`), `quantity`; (`shipment_fk`, `order_line_fk`) is unique

The base data (stages D1 to D5) covers January through March 2026 and is small enough to print whole. It was written to contain the situations the chapters' queries must handle:

- One customer, C-1008 Northgate Hardware, has never ordered, and one product, the GC-400 poly garden cart, has never been ordered.
- Two orders have no requested ship date, so the column holds NULL.
- Order SO-5004 lists the HT-100 on two lines at two prices, which is why an order line is a class of its own rather than a simple pairing of an order and a product (Chapter 5).
- Order SO-5004 shipped in two shipments. Three order lines shipped only in part, leaving backorders, and orders SO-5011 and SO-5012 have not shipped at all.
- Two stock levels are below their reorder points, and the GC-400 in Tulsa has none on hand.

C.5. The production schema

Figure C.2 shows the class model of the production schema. Every part is either purchased or manufactured, never both, and there is no third kind; the generalization is `{disjoint, complete}`. A manufactured part is an assembly of components through the bill of materials, where the quantity of each component per assembly is the association class `BomLine`. Notice that a component may itself be an assembly, which makes the association recursive. Each manufactured part has a routing: operations

numbered within the part, each performed at a work center. Work centers aggregate machines, operators hold certifications on machines, and work orders release a quantity of a manufactured part to the plant floor.

Table C.5 lists the tables. Notice that `manufactured_part` and `purchased_part` have no surrogate key of their own: each row's primary key, `part_fk`, is also its reference to the `part` it is.

Table C.5.: Tables of the Production Schema

Table	Columns, in order, and their keys
`part`	`part_pk` (primary key), `part_number` (unique), `description`, `unit_of_measure`, `standard_cost`
`manufactured_part`	`part_fk` (primary key, refers to `part`), `lot_size`
`purchased_part`	`part_fk` (primary key, refers to `part`), `lead_time_days`
`bom_line`	`bom_line_pk` (primary key), `assembly_fk` (refers to `manufactured_part`), `component_fk` (refers to `part`), `quantity_per`; (`assembly_fk`, `component_fk`) is unique
`work_center`	`work_center_pk` (primary key), `work_center_code` (unique), `name`, `hourly_rate`
`machine`	`machine_pk` (primary key), `machine_number` (unique), `model`, `installed_on`, `work_center_fk` (refers to `work_center`)
`operator`	`operator_pk` (primary key), `employee_number` (unique), `name`, `hire_date`
`certification`	`certification_pk` (primary key), `operator_fk` (refers to `operator`), `machine_fk` (refers to `machine`), `certified_on`, `level`; (`operator_fk`, `machine_fk`) is unique

Table	Columns, in order, and their keys
`operation`	`operation_pk` (primary key), `manufactured_part_fk` (refers to `manufactured_part`), `operation_number`, `description`, `setup_minutes`, `run_minutes_per_unit`, `work_center_fk` (refers to `work_center`); (`manufactured_part_fk`, `operation_number`) is unique
`work_order`	`work_order_pk` (primary key), `work_order_number` (unique), `manufactured_part_fk` (refers to `manufactured_part`), `quantity`, `released_on`, `due_on`, `completed_on` (optional)

The eighteen parts form a bill of materials up to four levels deep. The HT-100 hand truck contains the WA-10 wheel assembly, which contains the AX-10 axle, which is machined from BS-075 bar stock. The wheel assembly is used in all four products. Tubing and bar stock are bought by the foot, so a quantity per assembly can be a fraction: 1.25 feet of bar stock for each axle. In addition, the data holds:

- a work center, Assembly, with no machines;
- a machine, the M-07 robotic welder, on which no one is certified yet;
- an operator, Tom Becker, hired in January 2026, with no certification;
- ten work orders, of which three are still open (no completion date) and two finished after their due dates.

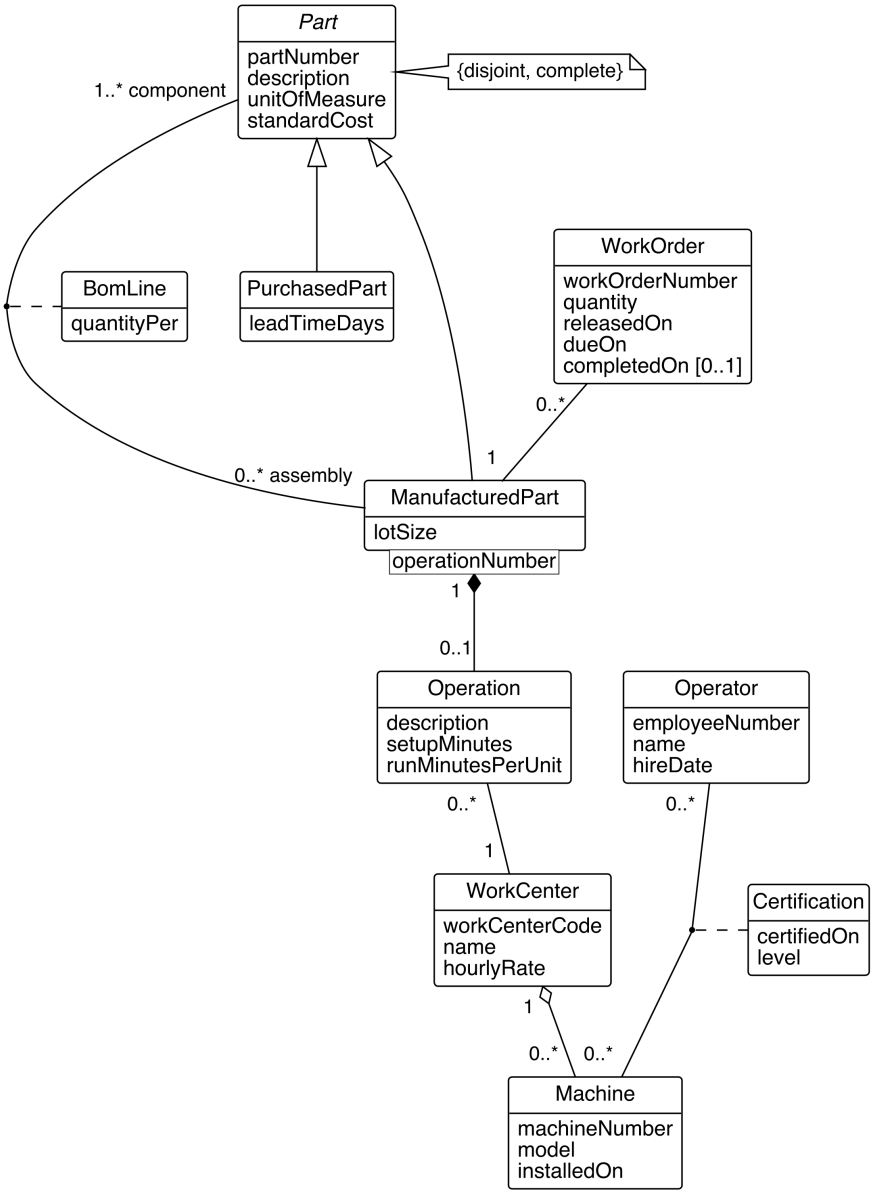


Figure C.2.: Production Schema Class Model

C.6. The quality schema

Figure C.3 shows the class model of the quality schema. Each characteristic, such as an axle's diameter, belongs to a part and carries its nominal value and specification limits. An inspection examines a lot of one part, is performed by one of production's operators, and records a measurement of each of the part's characteristics and a count of each kind of defect it finds. Measurements and defect counts are both association classes.

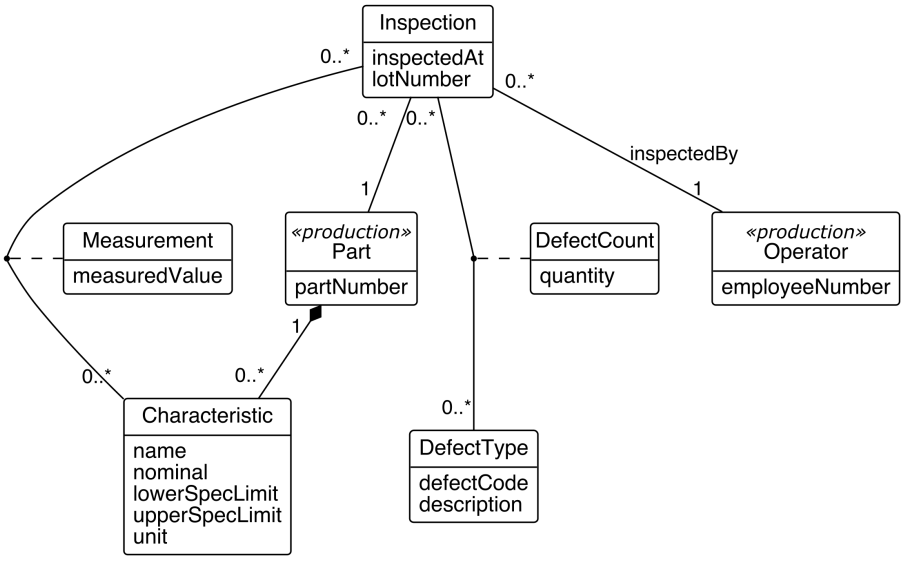


Figure C.3.: Quality Schema Class Model

Table C.6.: Tables of the Quality Schema

Table	Columns, in order, and their keys
`characteristic`	`characteristic_pk` (primary key), `part_fk` (refers to `production.part`), `name`, `nominal`, `lower_spec_limit` (optional), `upper_spec_limit` (optional), `unit`; (`part_fk`, `name`) is unique
`defect_type`	`defect_type_pk` (primary key), `defect_code` (unique), `description`
`inspection`	`inspection_pk` (primary key), `part_fk` (refers to `production.part`), `operator_fk` (refers to `production.operator`), `inspected_at`, `lot_number`
`measurement`	`measurement_pk` (primary key), `inspection_fk` (refers to `inspection`), `characteristic_fk` (refers to `characteristic`), `measured_value`; (`inspection_fk`, `characteristic_fk`) is unique
`defect_count`	`defect_count_pk` (primary key), `inspection_fk` (refers to `inspection`), `defect_type_fk` (refers to `defect_type`), `quantity`; (`inspection_fk`, `defect_type_fk`) is unique

The base data holds twenty inspections of axles, hand truck frames, and cart frames in January and February 2026. Weld strength has a lower specification limit and no upper one, since a stronger weld is never a defect, so its upper limit is NULL. The data was built to show:

- an axle diameter that rises at every inspection and ends above its upper specification limit;
- five measurements outside their specification limits in all;

- one inspector, Linda Nguyen, whose coating thickness readings run consistently lower than Robert Hayes's readings of the same parts;
- paint runs outnumbering every other defect combined, and one defect type, cracks, that has never been found.

C.7. The year of history

The base data is small by design, and some questions need more. Stage G adds a year of business to the distribution and quality schemas:

- 22 more customers, for 30 in all, in Arkansas, Oklahoma, Missouri, Kansas, and Texas;
- 602 orders from April 2026 through March 2027, with 1,043 order lines, 641 shipments, and 1,078 shipment lines;
- 236 inspections from April through September 2026, with 419 measurements and 101 defect counts.

The history follows patterns that the analytical chapters look for. Garden carts sell mainly in the spring: the GC-300 sells 1.75 times as many units from April to June as from October to December. Hand trucks rise toward the end of the year. The GC-400 is sold from its launch on 1 September 2026, and large customers and large order lines receive discounts. On the plant floor, the axle diameter creeps upward as the lathe's cutting tool wears and drops back at each of five tool changes.

The history was generated by a program, `GenerateHistory`, in the companion code project, which draws its random choices from seeded random number streams of the KSL. Thus, the program writes the same history on every run, and every reader's queries return the same results. Chapter 10 discusses programs that work with databases in this way.

C.8. The sales star schema

Stage S builds the `sales` schema of Figure C.4, a star schema for analyzing sales. The fact table, `order_line_fact`, has one row for every order line. Its measures (quantity, sales, discount from list price, and standard cost) can be added up along any of the three dimensions around it: the date of the order, the customer, and the product. The dimensions carry attributes the operational tables do not, such as each customer's sales region and each product's product line.

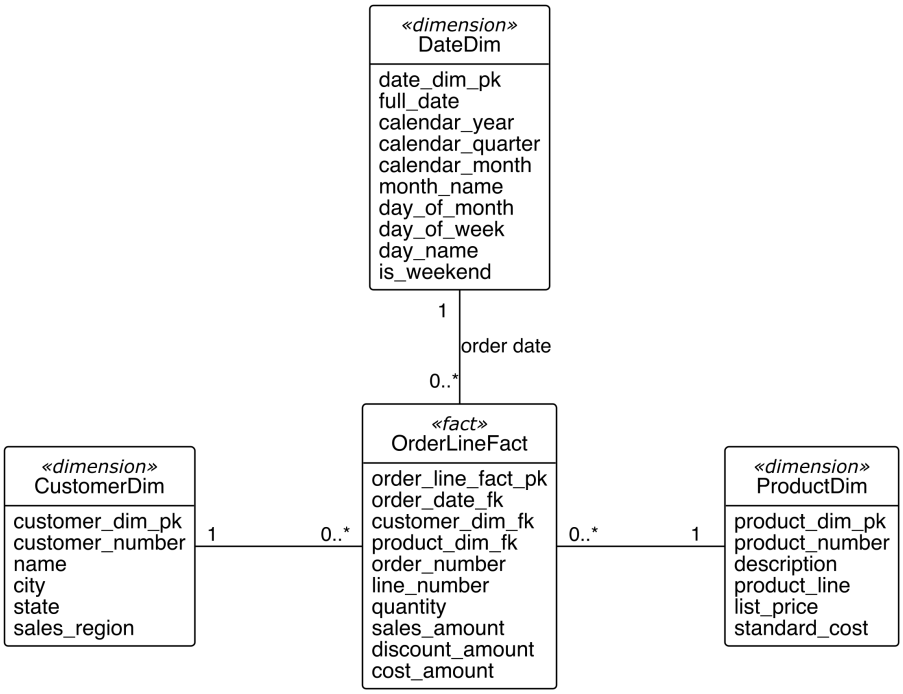


Figure C.4.: Sales Star Schema

Notice that the diagram shows the tables' key columns, unlike the class

models above. A star schema is a design for tables, not a model of the business, and Chapter 11 develops it at that level.

Table C.7.: Tables of the Sales Schema

Table	Columns, in order, and their keys
`date_dim`	`date_dim_pk` (primary key), `full_date` (unique), `calendar_year`, `calendar_quarter`, `calendar_month`, `month_name`, `day_of_month`, `day_of_week`, `day_name`, `is_weekend`
`customer_dim`	`customer_dim_pk` (primary key), `customer_number` (unique), `name`, `city`, `state`, `sales_region`
`product_dim`	`product_dim_pk` (primary key), `product_number` (unique), `description`, `product_line`, `list_price`, `standard_cost`
`order_line_fact`	`order_line_fact_pk` (primary key), `order_date_fk` (refers to `date_dim`), `customer_dim_fk` (refers to `customer_dim`), `product_dim_fk` (refers to `product_dim`), `order_number`, `line_number`, `quantity`, `sales_amount`, `discount_amount`, `cost_amount`; (`order_number`, `line_number`) is unique

The stage S script is itself an extract, transform, and load process written in SQL. It reads the distribution and production schemas, derives the new attributes, and loads the fact table by looking up each dimension's key from the business identifiers. Chapter 11 walks through it.

C.9. Rows in each table

Table C.8 gives the number of rows in each table at the two sizes the book uses. Notice that the base data, through stage Q, keeps every table small enough to print, while the history adds its volume to five of the distribution tables and three of the quality tables. The production tables are the same at both sizes.

Table C.8.: Rows in Each Table

Table	Base data (through stage Q)	With the history (through stage S)
`distribution.customer`	8	30
`distribution.customer_order`	12	614
`distribution.product`	4	4
`distribution.order_line`	22	1065
`distribution.warehouse`	2	2
`distribution.stock_level`	8	8
`distribution.shipment`	11	652
`distribution.shipment_line`	18	1096
`production.part`	18	18
`production.manufactured_part`	8	8
`production.purchased_part`	10	10
`production.bom_line`	23	23
`production.work_center`	5	5
`production.machine`	7	7
`production.operator`	6	6
`production.certification`	10	10
`production.operation`	13	13
`production.work_order`	10	10
`quality.characteristic`	5	5
`quality.defect_type`	6	6
`quality.inspection`	20	256

Table	Base data (through stage Q)	With the history (through stage S)
`quality.measurement`	34	453
`quality.defect_count`	15	116
`sales.date_dim`		730
`sales.customer_dim`		30
`sales.product_dim`		4
`sales.order_line_fact`		1065

C.10. Millbrook Transit

A second, smaller database gives the query exercises of chapters 3 and 4 a setting outside HG Industries. It records one week of service, Monday 2 March to Sunday 8 March 2026, of Millbrook Transit, a fictional town's bus agency. Its design is adapted from the class model of the transit exercises of chapter 5, which come from a Transportation Research Board study of transit monitoring (Rossetti 1999), and mapped to tables as chapter 6 teaches. It differs from the exercises' model in a few places. Every person has a rider category. An account has an opening date and no balance. A smart card names the person who carries it (`person_fk`, NULL for an anonymous card), an association the model does not have. A trip refers to the point traversals where it boarded and alighted, not to an origin and a destination point, and records no times of its own. A driver has an employee number and a hire date, not a license number. An assignment has no comments. A route traversal refers to an assignment, not to a vehicle and a route; the assignment names both. A schedule adds the times of its first and last departures (`first_departure`, `last_departure`), and a schedule line holds the minutes after the bus's departure, not an arrival and a departure time. An access point's block location may also be off street. Finally, the database has no tables for authorizations, points of interest, or link traversals.

The agency runs three loop routes, each starting and ending at the Transit Center: the Plant Loop (route 1), the Campus Loop (route 2), and the Hospital Loop (route 3). Buses leave every 30 minutes on weekdays, from 6:00 to 19:30, and every hour on Saturday, from 8:00 to 17:00; there is no service on Sunday. The tables fall into four groups.

- **The network.** `route`, `point`, and `segment`: each route is its ordered segments, each segment joining a start point to an end point. Points are specialized into `access_point`, where passengers board and alight, and `scheduled_point`, which appears on the timetables, mapped to a table for each class (Section 6.5.1).
- **Schedules.** `schedule`, with its days (`schedule_day`, `day_of_week`) and its lines (`schedule_line`), each giving the minutes from departure to a scheduled point.
- **Operations.** `vehicle`, `driver`, and `assignment`, which puts a driver on a vehicle on a route for a shift; `route_traversal`, one run of a route under an assignment; and `point_traversal`, each point the bus reached on a run, with the passengers boarding and alighting there.
- **Riders.** `person`, `mailing_address`, `account`, `smart_card`, `trip`, and `payment`. Smart cards are debit or credit cards, a generalization mapped to one table with `card_kind` (Section 6.5.3). Each trip made with a card refers to the point traversals where it boarded and alighted. Riders who pay cash are counted in the boardings and alightings but recorded as no trip.

Listing C.1 gives the tables in the notation of Section 2.4.

Listing C.1 The Millbrook Transit Database

```
route (route_pk, route_number, name, color, distance_miles)
    primary key: route_pk
    alternate key: route_number

point (point_pk, point_code, name, latitude, longitude)
    primary key: point_pk
    alternate key: point_code

access_point (point_fk, block_location, waiting_capacity,
              bench_count, shelter)
    primary key: point_fk
    point_fk references point_pk in point, must not be NULL

scheduled_point (access_point_fk, timetable_posted)
    primary key: access_point_fk
    access_point_fk references point_fk in access_point,
        must not be NULL

segment (segment_pk, route_fk, sequence_number,
         start_point_fk, end_point_fk, length_miles)
    primary key: segment_pk
    alternate key: (route_fk, sequence_number)
    route_fk references route_pk in route, must not be NULL
    start_point_fk references point_pk in point,
        must not be NULL
    end_point_fk references point_pk in point,
        must not be NULL

day_of_week (day_of_week_pk, day_name, iso_day)
    primary key: day_of_week_pk
    alternate key: day_name
    alternate key: iso_day

schedule (schedule_pk, route_fk, name, first_departure,
          last_departure, headway_minutes, last_updated)
    primary key: schedule_pk
    alternate key: (route_fk, name)
    route_fk references route_pk in route, must not be NULL

schedule_day (schedule_day_pk, schedule_fk, day_of_week_fk)
    primary key: schedule_day_pk
    alternate key: (schedule_fk, day_of_week_fk)
    schedule_fk references schedule_pk in schedule,
        must not be NULL
    day_of_week_fk references day_of_week_pk in day_of_week,
        must not be NULL

schedule_line (schedule_line_pk, schedule_fk, line_number,
               scheduled_point_fk, minutes_after_departure)
```

The week holds a few conditions that exercises turn on: bus 105 spends the week in the shop and is never assigned; two people have no smart card, one card is never used, and four cards are anonymous, sold for cash with no person; some people share a mailing address, and some have none; and Sunday has no schedule. The rows in each table, in the order of the four groups, are:

Table C.9.: Rows in Each Table of Millbrook Transit

```
SELECT 'route' AS table_name, COUNT(*) AS row_count
FROM transit.route
UNION ALL
SELECT 'point', COUNT(*) FROM transit.point
UNION ALL
SELECT 'access_point', COUNT(*) FROM transit.access_point
UNION ALL
SELECT 'scheduled_point', COUNT(*)
FROM transit.scheduled_point
UNION ALL
SELECT 'segment', COUNT(*) FROM transit.segment
UNION ALL
SELECT 'day_of_week', COUNT(*) FROM transit.day_of_week
UNION ALL
SELECT 'schedule', COUNT(*) FROM transit.schedule
UNION ALL
SELECT 'schedule_day', COUNT(*) FROM transit.schedule_day
UNION ALL
SELECT 'schedule_line', COUNT(*) FROM transit.schedule_line
UNION ALL
SELECT 'vehicle', COUNT(*) FROM transit.vehicle
UNION ALL
SELECT 'driver', COUNT(*) FROM transit.driver
UNION ALL
SELECT 'assignment', COUNT(*) FROM transit.assignment
UNION ALL
SELECT 'route_traversal', COUNT(*)
FROM transit.route_traversal
UNION ALL
SELECT 'point_traversal', COUNT(*)
FROM transit.point_traversal
UNION ALL
SELECT 'mailing_address', COUNT(*)
FROM transit.mailing_address
UNION ALL
SELECT 'person', COUNT(*) FROM transit.person
UNION ALL
SELECT 'account', COUNT(*) FROM transit.account
UNION ALL
SELECT 'smart_card', COUNT(*) FROM transit.smart_card
UNION ALL
SELECT 'trip', COUNT(*) FROM transit.trip
UNION ALL
SELECT 'payment', COUNT(*) FROM transit.payment;
```

table_name	row_count
route	3
point	12

To load the database, create a database named `dbconcepts_transit` (or use any database of your own) and run two scripts from `code/sql/transit/` in order: `t-schema.sql`, which creates the `transit` schema and its tables, and `t-data.sql`, which loads the week. The second is generated by `scripts/build-transit-data.py` from a seeded random number generator, so it is the same on every machine. Running the first again deletes and recreates the schema.

C.11. Public sample databases for further practice

HG Industries' data is small so that every result can be checked. For practice at a larger scale, several public sample databases can be loaded into the same PostgreSQL server, each into a database of its own. They are not distributed with the book: Section A.6 explains how to fetch each one from its source, where its license is stated.

Table C.10 lists the four the book's exercises use. Each is openly licensed; the license travels with the data, not with this book, and you should read it before you reuse the data in work of your own.

Table C.10.: Public Sample Databases Used in the Exercises

Database	What it records	Source and license	Size
Chinook	A digital music store: artists, albums, tracks, playlists, customers, employees, invoices, 2021 to 2025	Luis Rocha, `lerocha/chinook-database`, release 1.4.5; MIT	11 tables, 0.6 MB

Database	What it records	Source and license	Size
Northwind	A food importer: customers, suppliers, products, orders and their lines, employees, shippers	Microsoft's sample, PostgreSQL port `pthom/northwind_psql`; Microsoft Public License	14 tables, 0.3 MB
Practical SQL	Four data sets from Anthony DeBarros's *Practical SQL*: county population estimates, the USDA's directory of meat, poultry, and egg establishments, monthly exports, daily temperatures	`anthonydb/practical-sql-2`; MIT, which covers the data	4 tables, 1.5 MB
Employees	A fabricated company's 300,024 employees over 15 years: departments, titles, and 2.8 million salary records	Wang, Zaniolo, Maxia, and Crews, `datacharmer/test_db`, PostgreSQL port; Creative Commons Attribution-Share Alike 3.0	6 tables, 172 MB

Each is useful for something the book's own data is too small or too clean to show. Chinook and Northwind are ordinary business databases designed by other people, with their own naming conventions (`customer_id` rather than `customer_fk`, plural table names in Northwind), which is practice in reading a schema you did not write. The Practical SQL data are real public records, with the inconsistencies real records have. Employees is large enough that a careless query is slow, and it records history with dates rather than replacing rows.

Load each with the book's loader, `code/sql/public/load-public.sh`, as Section A.6 describes. It fetches the files from their original sources at the versions in the table, checks each against a recorded checksum, and loads them into a database of their own, `dbconcepts_chinook`, `dbconcepts_northwind`, `dbconcepts_practical`, and `dbconcepts_employees`. Exercises that use them name the database, and their answers assume these versions.

D. Translating Between Crow's Foot and UML

This book draws its data models as UML class diagrams. Other books, many courses, and most database tools draw them as entity-relationship diagrams in the crow's foot notation. Section 5.10 showed how to read the crow's foot marks; this appendix translates every construct in both directions, says what is lost on the way, and ends with a procedure for translating a whole model and a worked example.

The examples come from HG Industries' production and distribution schemas, drawn both ways, so that each crow's foot diagram can be set beside a class diagram the book has already explained.

D.1. Why two notations

The entity-relationship model was introduced by Peter Chen as a way to describe data independently of how it is stored (Chen 1976). Chen's diagrams drew relationships as diamonds; the crow's foot notation, which draws a relationship as a line with marks at its ends, dates from the same decade and spread with the Information Engineering method and the design tools built on it. UML came later, in the 1990s, from object-oriented software design, and its class diagrams are a standard of the Object Management Group.

The two notations describe the same things: kinds of objects, their facts, and the associations among them, with how many of each may be linked.

They differ in emphasis. A crow's foot diagram is usually drawn close to the tables it will become, with identifiers marked and foreign keys often shown, while a UML class diagram describes the business and leaves keys to the mapping of Chapter 6. That is why this book models in UML and maps afterwards. You will still meet crow's foot diagrams in other textbooks, in DataGrip's and DBeaver's generated diagrams, and in design documents at work, and you should be able to read one as a class model and draw one when asked.

D.2. Entities and classes

An **entity type** is the ERD's name for a class: a kind of thing the business records, drawn as a box with its name above its attributes. An **entity**, or entity instance, is one object of it. Crow's foot diagrams conventionally write entity names in upper case with underscores (`WORK_CENTER`), the style of the tables they become, where UML writes class names in upper camel case (`WorkCenter`); Figure 5.21 kept the class names only so that it could be set beside its class diagram. The difference is one of spelling only: an entity type translates to a class with the same meaning, and a class to an entity type.

Recall from Section 5.3 that the book distinguishes a class from the table it maps to; in a crow's foot diagram that distinction is often blurred, and the box is read as both. Translating to UML restores it: the class describes the business, and Chapter 6 decides the table.

D.3. Attributes and keys

Figure D.1 draws work centers and their machines as a design tool draws them. Each entity lists its identifier above the line, marked with an asterisk, and its other attributes below. `MACHINE` also lists `work_center_code`,

marked (FK): the foreign key by which each machine refers to its work center.

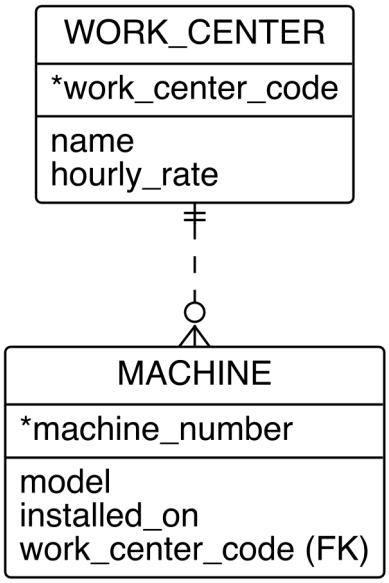


Figure D.1.: Work Centers and Machines, with Identifiers and a Foreign Key

Three rules translate these boxes to classes.

- **An identifier that the business uses becomes an attribute.** Work center codes and machine numbers are business identifiers, so `workCenterCode` and `machineNumber` are attributes of `WorkCenter` and `Machine`, as in Figure 5.16. The class description records that each is unique (Section 5.3).
- **An artificial identifier is dropped.** A column such as `checkout_id` that exists only to number rows has no place in a class model; the mapping of Section 6.6 adds a surrogate key to every

table in any case.

- **A foreign key is dropped, and the line carries its meaning.** `work_center_code (FK)` in `MACHINE` repeats what the relationship line already says, that each machine belongs to a work center. A class model shows the association and no foreign key attribute (1).

Some ERD styles mark other kinds of attribute: a *multivalued* attribute, such as an employee's several phone numbers, and a *derived* attribute, such as an age computed from a birth date. A multivalued attribute translates to a class of its own with a one-to-many association, since a table's column holds one value (Section 2.1); a derived attribute is left out of the class model, or noted in the class description, since it is computed rather than recorded.

D.4. Relationships and multiplicity

A **relationship** is the ERD's association, drawn as a line between two entity types. Recall Table 5.8: the mark nearer the box gives the maximum (a bar for one, the crow's foot for many) and the mark farther from the box the minimum (a bar for one, a circle for zero). The marks sit at the same ends as UML's numbers, so each end translates on its own:

Table D.1.: Translating the Four Crow's Foot Ends

Crow's foot end	UML multiplicity
bar, bar	`1`
circle, bar	`0..1`
circle, crow's foot	`0..*`
bar, crow's foot	`1..*`

In Figure D.1, the line has two bars at `WORK_CENTER` and a circle with a crow's foot at `MACHINE`. The UML association of Figure 5.16 has `1` at the

work center and `0..*` at the machine: each machine belongs to exactly one work center, and a work center has zero or more machines.

The translation is exact for the four ends of Table D.1 and no further. Crow's foot has no mark for a count other than zero, one, and many, so a UML multiplicity such as `2..*` or `1..3` becomes the nearest end (`1..*`) and a note. A relationship's name, often a verb phrase written on the line ("performs", "is assigned to"), translates to a UML association name or, when it names what one end is to the other, to a role name. A relationship between an entity type and itself, a **unary** or recursive relationship, translates to a recursive association with role names (1).

D.5. Associative entities and association classes

Crow's foot draws only lines between two boxes. A many-to-many relationship with facts of its own, or a relationship among three entity types, is therefore drawn as an **associative entity**: an entity type in the middle, with a one-to-many line to each of the entity types it connects. Figure D.2 draws the stock levels of stage D4 this way.

Read each line of Figure D.2 on its own: a warehouse has zero or more stock levels, each stock level is of exactly one warehouse, and the same for products. Its identifier is the pair of foreign keys. In UML the same facts are the association class `StockLevel` of Figure 5.7: an association between `Warehouse` and `Product`, `0..*` at both ends, with `quantityOnHand` and `reorderPoint` on the association class.

An associative entity does not always become an association class. Recall the decision of Section 5.5 about order lines: an association class allows at most one link per pair of objects, so it is right only when the pair identifies the associative entity. The test in translation is the identifier:

- **Identified by the pair of foreign keys**, as `STOCK_LEVEL` is: an association class.

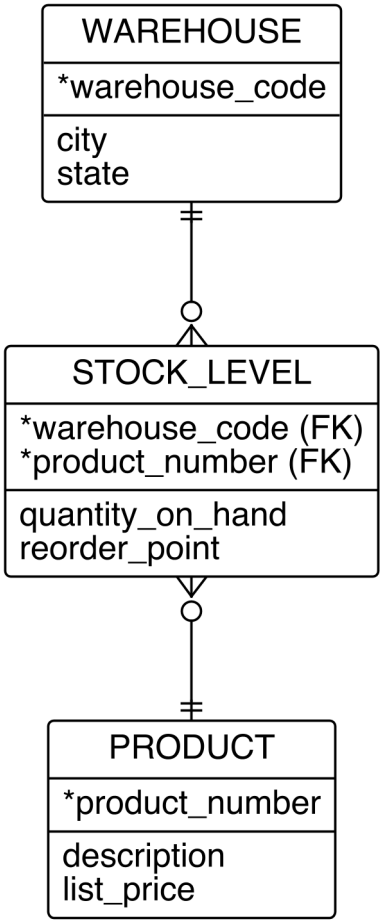


Figure D.2.: Stage D4's Stock Levels in Crow's Foot Notation

- **Identified by something else**, a number of its own or the pair plus another attribute, because the same pair may occur more than once: a class with two associations. The order line of Figure 5.11, drawn as `OrderLine` in Figure 5.21, is one; so is a checkout of a tool (Exercise 5.18).

The same test applies to an associative entity with three lines, the crow's foot form of a ternary association (Section 5.5.1): identified by the three foreign keys, it is a ternary association with an association class; otherwise, a class with three binary associations.

D.6. Identifying relationships and composition

Crow's foot distinguishes two kinds of relationship line. An **identifying relationship**, drawn as a solid line, is one whose parent's identifier is part of the child's identifier: the child cannot be identified, and so cannot exist, without its parent. A **non-identifying relationship**, drawn as a dashed line, only refers to the parent. A child entity type with an identifying relationship is often called a **weak entity**, and the part of its identifier that it adds is its partial identifier. Figure D.3 shows both kinds.

An operation of Figure D.3 is identified by its part's number and its own operation number, so its line to `MANUFACTURED_PART` is identifying, solid: operation 20 means nothing without the part it belongs to. Its line to `WORK_CENTER` is dashed: the work center is a fact about the operation, not part of its name.

Now compare Figure 5.16. The identifying relationship with a partial identifier is the qualified composition `ManufacturedPart [operationNumber] "1" *-- "0..1" Operation`: the partial identifier `operation_number` becomes the qualifier, and the dependence of the operation on its part becomes the filled diamond. The non-identifying relationship is an ordinary association. The translation rules are:

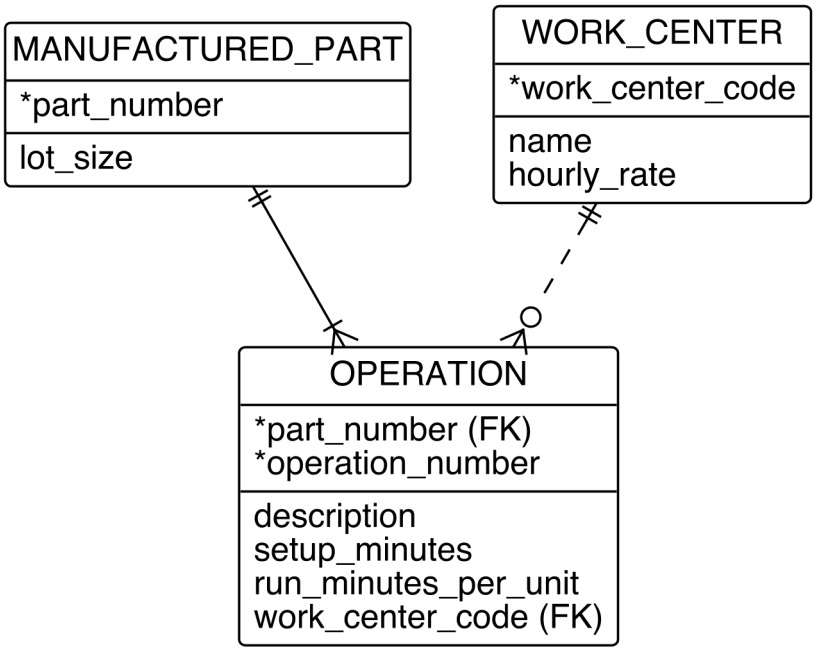


Figure D.3.: Stage P3's Routing in Crow's Foot Notation

- **Identifying, with a partial identifier**: a qualified association, usually a composition (Section 5.6).
- **Identifying, with no partial identifier** (the child's identifier is the parent's alone): a one-to-one association, often a composition, or, if the child is a kind of the parent, a generalization (the next section).
- **Non-identifying**: an ordinary association or an aggregation.

The translation back is looser, because UML distinguishes what crow's foot does not. A UML composition says that the part cannot exist without its whole and is deleted with it, which Section 6.3.6 maps to `on delete cascade`; an identifying relationship says only how the child is named. An aggregation, such as a work center's machines in Figure 5.16, has no crow's foot mark at all: it becomes a non-identifying line, and the diamond is lost.

D.7. Subtypes and generalization

ERDs record kinds of an entity type as **supertypes and subtypes**, drawn in many styles: a subtype box inside its supertype, a circle joining the lines, or, as design tools and Figure D.4 draw it, a one-to-zero-or-one line from the supertype to each subtype, with the subtype sharing the supertype's identifier.

Each line of Figure D.4 says that a part may have one row of manufactured-part facts and one of purchased-part facts. The note adds what the lines cannot: a **subtype discriminator**, `part_kind`, whose value says which subtype a part belongs to, and two constraints. *Exclusive* (also called disjoint) means a part belongs to at most one subtype; *total* (also called complete) means every part belongs to one. Their opposites are *overlapping* and *partial*.

The translation is the generalization of Figure 5.17: `Part` is the superclass, `ManufacturedPart` and `PurchasedPart` its subclasses, and the constraints

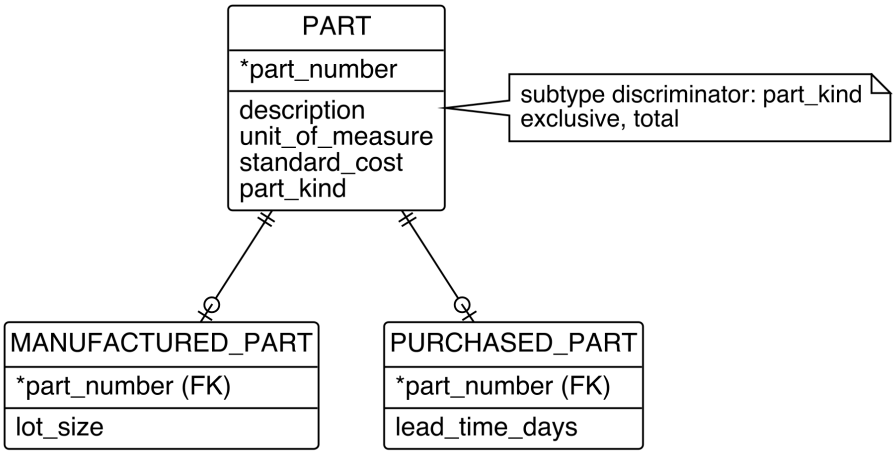


Figure D.4.: Stage P1's Parts as a Supertype and Two Subtypes

translate word for word: exclusive to `{disjoint}` and total to `{complete}`, and their opposites, overlapping and partial, to `{overlapping}` and `{incomplete}`. A total supertype has no objects of its own, so it becomes an abstract class, its name in italics, as `Part` is in Figure 5.17. The discriminator is dropped from the class model, since in UML an object's class says its kind; it returns in the mapping, as the `part_kind` column of the single-table mapping of Section 6.5.3. The subtypes' copies of `part_number`, marked `(FK)`, are dropped as foreign keys are.

Reading a diagram drawn with plain one-to-zero-or-one lines and no note, you cannot tell a subtype from an ordinary optional one-to-one relationship. The test is the one of Section 5.7: is every manufactured part a part, with all of a part's facts? If so, it is a subtype.

D.8. What does not translate

Most of a model survives translation in either direction. What does not is worth knowing before you translate, so that the losses are recorded in a note rather than silently dropped.

Table D.2.: What Is Lost Translating UML to Crow's Foot

UML construct	In crow's foot	What is lost
Multiplicity other than 0, 1, many (`2..*`)	nearest end (`1..*`)	the exact count; keep it in a note
Qualifier	partial identifier of a weak entity	nothing, if the relationship is drawn as identifying
Composition	identifying relationship	that the part is deleted with its whole
Aggregation	ordinary relationship	the whole-part meaning
Association class	associative entity	nothing, but the reverse translation needs the identifier test
`{ordered}`	a sequence-number attribute	the constraint becomes data
`{disjoint, complete}` and their opposites	a note or a style-specific symbol	the constraint, where the style has no symbol
Operations of a class	nothing	not used in data models

In the other direction, a crow's foot diagram carries facts that a class model leaves to the mapping: identifiers, foreign keys, and sometimes types and

lengths. None of these is lost in translation, since Chapter 6 recreates them, but you should record in the class descriptions any business identifier and any rule the ERD states through its keys, such as the uniqueness of an operation number within its part.

D.9. Translating a model, step by step

The sections above translate one construct at a time. A whole model is translated in a fixed order, so that each decision rests on the ones before it. From crow's foot to UML:

1. **Entity types to classes.** Rename each to upper camel case. Set aside the associative entities and the subtypes for steps 4 and 5.
2. **Attributes.** Keep each business attribute and business identifier; drop artificial identifiers and every attribute marked as a foreign key.
3. **Relationships to associations.** Translate each end with Table D.1. Turn relationship names into association names or role names; give role names to every recursive relationship and to every second relationship between the same pair.
4. **Associative entities.** Apply the identifier test of Section D.5: an association class if the foreign keys alone identify it, a class of its own otherwise.
5. **Identifying relationships and subtypes.** A partial identifier becomes a qualifier; decide whether the dependence is a composition. Subtypes become a generalization with the constraints of the note.
6. **Read the result both ways.** Read every association as two sentences

   (1) and review the model with Table 5.6.

From UML to crow's foot, the steps run in the same order: classes to entity types; attributes, adding the identifiers the class descriptions name; associations to relationships by Table D.1; association classes to associative

entities; qualifiers to partial identifiers on identifying relationships; generalizations to subtypes with a discriminator; and every loss of Table D.2 to a note.

Example D.1 (Translating a production model from crow's foot to UML)**.** A consultant's report on HG Industries' production floor includes Figure D.5, drawn in crow's foot notation with a design tool. The plant's engineers maintain their models in UML.

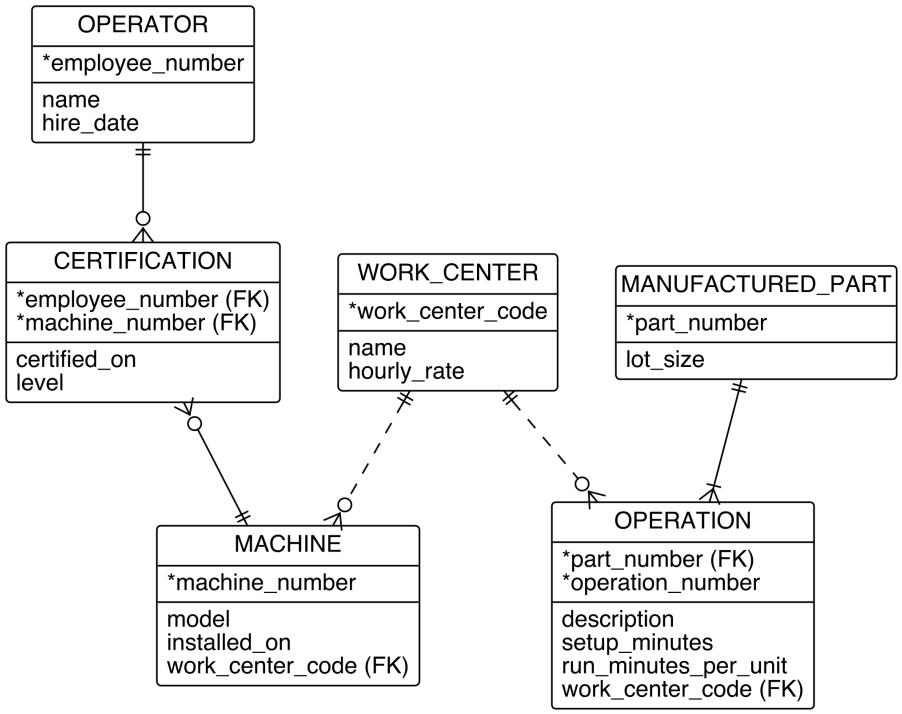


Figure D.5.: Part of HG Industries' Production Model in Crow's Foot Notation

The request. Translate Figure D.5 into a UML class diagram, and check it

against the class model of Figure 5.20.

The translation. We follow the procedure.

1. **Classes.** `WORK_CENTER`, `MACHINE`, `OPERATOR`, `OPERATION`, and `MANUFACTURED_PART` become `WorkCenter`, `Machine`, `Operator`, `Operation`, and `ManufacturedPart`. `CERTIFICATION` is associative and waits for step 4.
2. **Attributes.** The business identifiers stay as attributes: `workCenterCode`, `machineNumber`, `employeeNumber`, `partNumber`. Every `(FK)` attribute is dropped: `work_center_code` from `MACHINE` and `OPERATION`, and the foreign keys of `CERTIFICATION` and `OPERATION`. `operation_number` is not a foreign key; it waits for step 5.
3. **Associations.** `WORK_CENTER ||..o{ MACHINE` becomes `1` at the work center and `0..*` at the machine; `WORK_CENTER ||..o{ OPERATION` becomes `1` and `0..*`.
4. **The associative entity.** `CERTIFICATION` is identified by the pair `(employee_number, machine_number)`, its two foreign keys alone, so it is the association class `Certification` (`certifiedOn`, `level`) of an association between `Operator` and `Machine`, `0..*` at both ends. Each end's multiplicity comes from the crow's foot at the associative entity: an operator has zero or more certifications, so zero or more machines.
5. **The identifying relationship.** `OPERATION` is identified by its part's number and `operation_number`, through a solid line: `operationNumber` becomes a qualifier, and since an operation cannot exist without its part's routing, the association is a composition, `ManufacturedPart [operationNumber] "1" *-- "0..1" Operation`. The crow's foot `1..*` at `OPERATION` (every manufactured part has at least one operation) is not visible through the qualifier and goes into the class description.

The check. The result is the part of Figure 5.20 that Figure D.5 covers, with one difference: the diagram draws the work center's machines with

an ordinary line, and the class model with an aggregation. The crow's foot diagram cannot state the whole-part meaning, so the translation cannot recover it; whether a work center is an assembly of its machines is a question for the engineers, not for the notation. Reading each association both ways confirms the rest: *each operation is performed at exactly one work center; a work center performs zero or more operations.*

Where it breaks. Had `CERTIFICATION` carried an identifier of its own, a certification number, the identifier test would have made it a class with two associations, allowing an operator to be certified twice on one machine. The diagram alone cannot say which the business intends; Exercise 5.5 asks the question the business must answer.

D.10. Exercises

Exercise D.1. Match each crow's foot term with the UML term it translates to, and say where the translation can lose information.

Crow's foot	UML
a. entity type	1. qualifier
b. relationship	2. generalization
c. associative entity	3. class
d. partial identifier	4. association
e. supertype and subtypes	5. association class, or a class of its own

Exercise D.2. Translate Figure D.6 to a UML class diagram with the procedure of Section D.9, and compare it with your model of Exercise 5.18.

a. Which attributes did you drop, and why?
b. `CHECKOUT` connects `TOOL` and `MECHANIC`. Why is it not an association class?

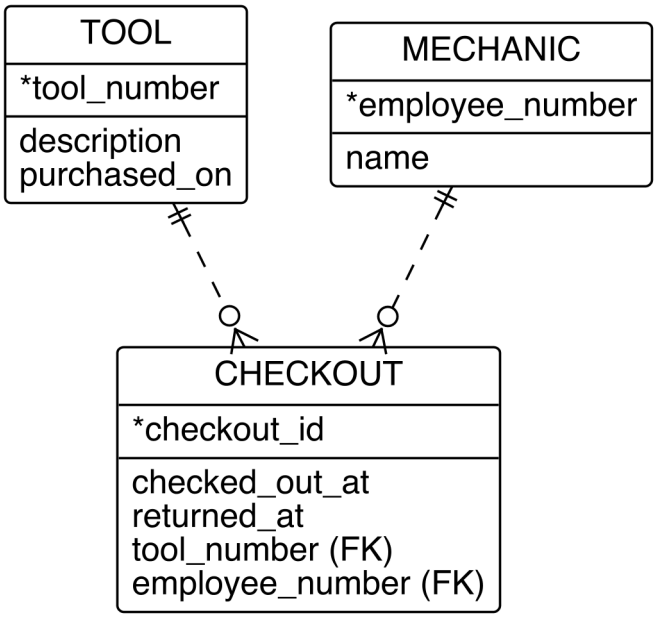


Figure D.6.: The Tool Crib in Crow's Foot Notation

Exercise D.3. Translate Figure D.7 to UML.

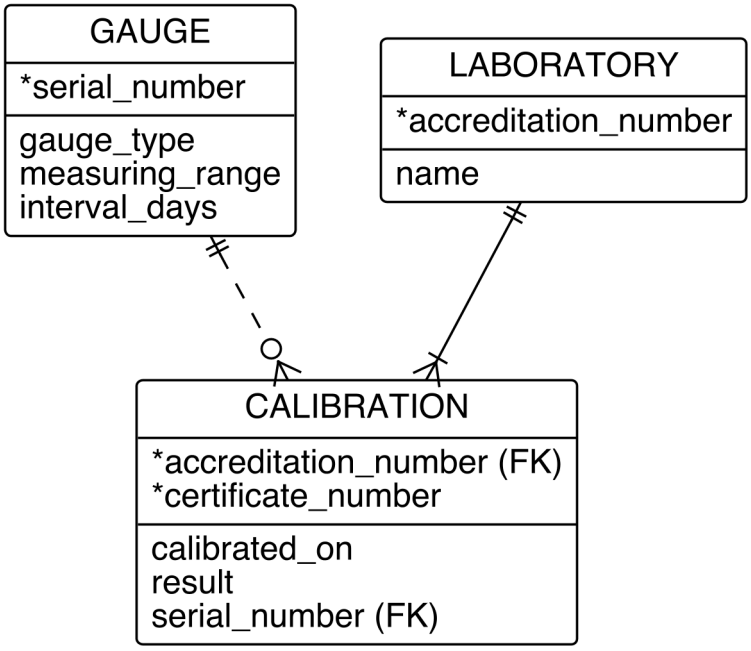


Figure D.7.: Gauge Calibration in Crow's Foot Notation

a. Which relationship is identifying, and what does it become?
b. The line from `LABORATORY` has a bar with a crow's foot at `CALIBRATION`. Where does that minimum of one go in your UML model?

Exercise D.4. Translate Figure D.8 to UML.

a. `PRECEDENCE` is related to `ACTIVITY` twice. What does that become?
b. `PRECEDENCE` carries `project_name` once, as part of both foreign keys. What rule does that state, and can your UML model state it?

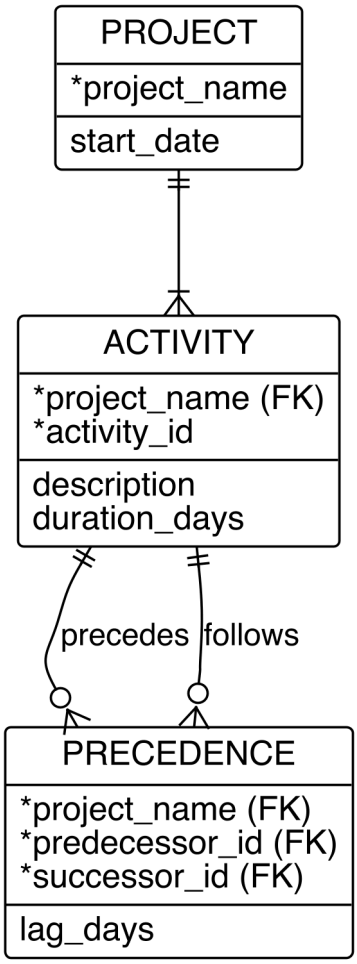


Figure D.8.: A Project Network in Crow's Foot Notation

Exercise D.5. Translate Figure D.9 to UML, and state the generalizations' constraints.

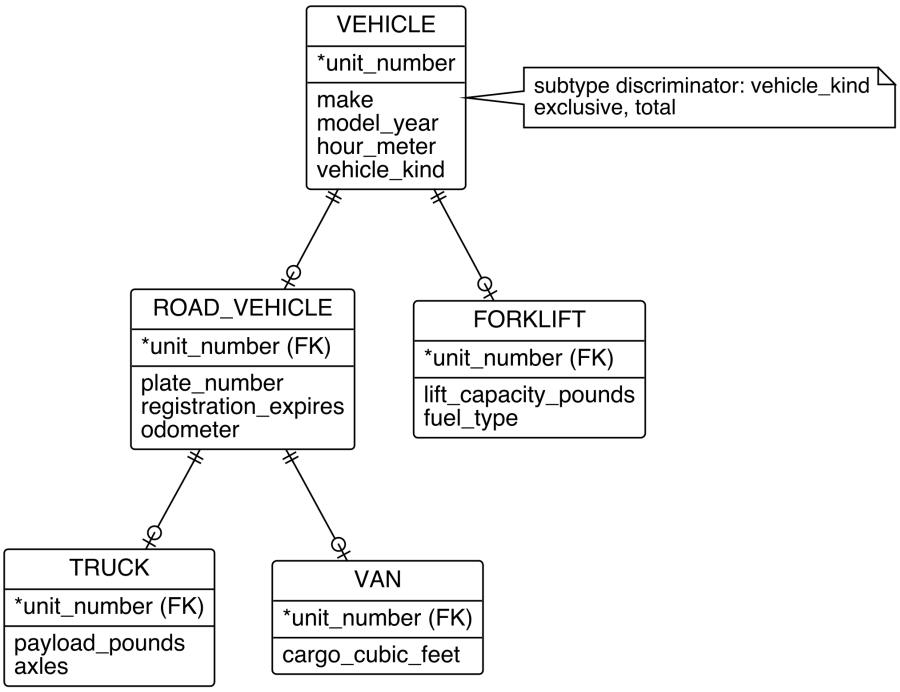


Figure D.9.: The Fleet as Supertypes and Subtypes

a. What happens to `vehicle_kind`?
b. The note applies to `VEHICLE`. Does it say anything about the subtypes of `ROAD_VEHICLE`? What would you ask the business?

Exercise D.6. Translate the kanban model of Figure 6.15 from UML to crow's foot notation, with identifiers.

a. Which multiplicity cannot be drawn, and how do you record it?

b. Choose an identifier for each entity type, and say which you had to invent.

Exercise D.7. Translate the equipment rental model of Figure 6.21 from UML to crow's foot notation, with identifiers and foreign keys.

a. What does the association class `Rate` become, and what is its identifier?
b. Which of your relationships are identifying?

Exercise D.8. Translate the bike share model of Figure 6.19 from UML to crow's foot notation.

a. What does the qualifier on `dockNumber` become?
b. Which UML construct of the model has no crow's foot mark, and what is lost?

Exercise D.9. Translate the route network of Figure 6.23 to crow's foot notation, and list everything Table D.2 says is lost, for this model in particular.

Exercise D.10. Describe Figure D.10 in English, writing each relationship as two sentences with its multiplicities in words, as 1 does for UML.

Which relationship is one-to-one, and which end of it is optional?

Exercise D.11. Describe Figure D.11 in English.

a. Which entity type is associative, and is it an association class or a class in UML?
b. Which relationships are identifying, and what do they become in UML?

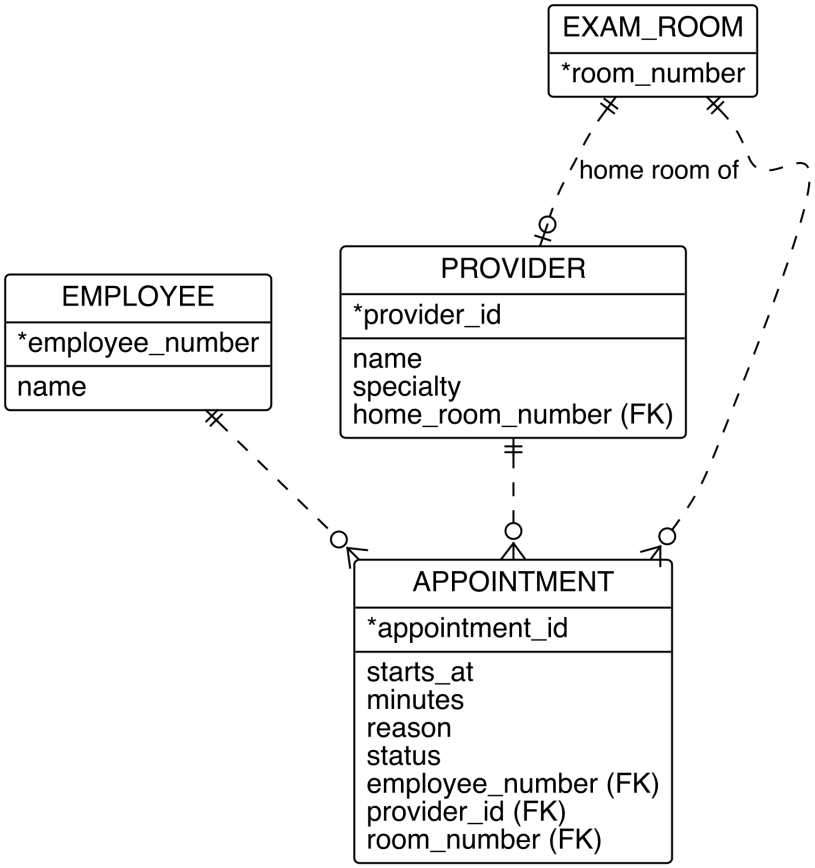


Figure D.10.: A Clinic's Appointments in Crow's Foot Notation

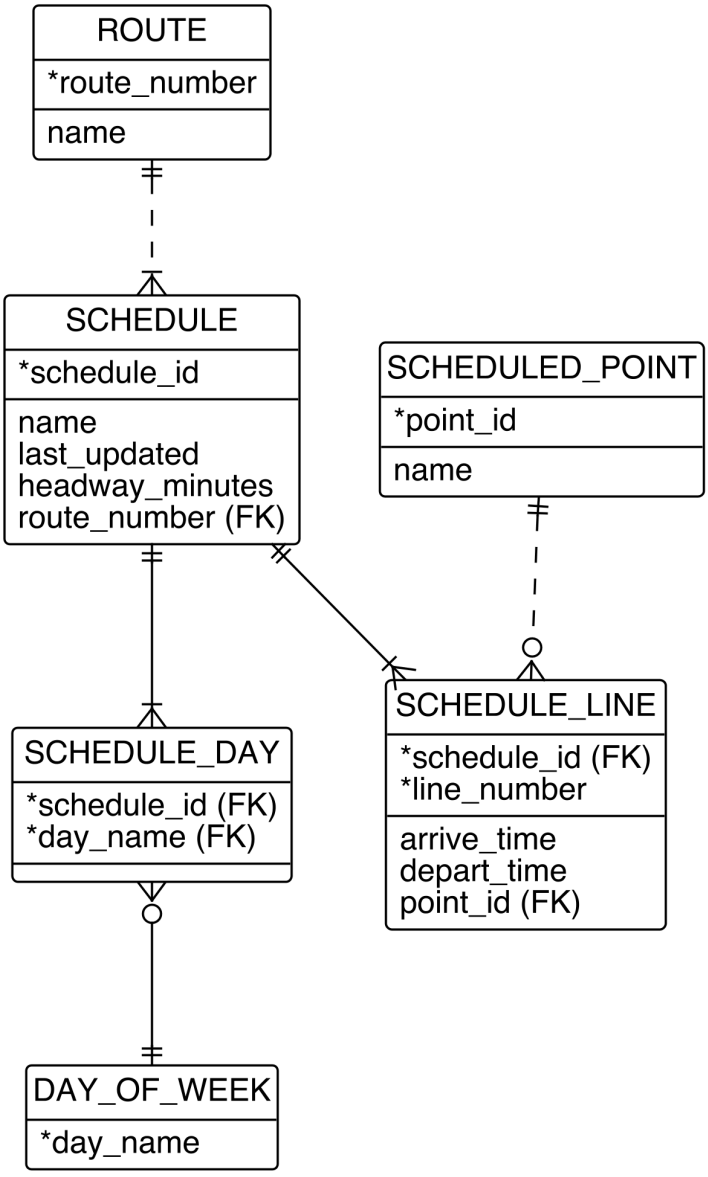


Figure D.11.: Transit Schedules in Crow's Foot Notation

E. Tools for Drawing UML and ER Diagrams

Students of this course ask, every term, what software to use for their diagrams. The answer depends on the job. Drawing a class model of a business before any table exists is a design job, and needs a tool that draws what you tell it to. Drawing a picture of a database that already exists is a documentation job, and a database tool can do it from the database itself. This appendix describes both kinds of tool, starting with the approach every diagram in this book was drawn with: diagrams written as text.

The tools change quickly, and their prices and licenses change with them. The descriptions here were checked against each tool's own documentation in September 2026; check again before you depend on one.

E.1. Two jobs, two kinds of tool

A **design diagram** states what a business records, in UML (Chapter 5) or in crow's foot notation (Appendix D), before any table exists. It shows classes, associations with their multiplicities, association classes, qualifiers, and generalization. A UML design diagram shows no foreign keys, since keys are decided later, in the mapping of Chapter 6; a crow's foot diagram often shows them, marked as attributes, as Appendix D explains. Any tool that lets you place and connect these elements will do.

A **generated diagram** is drawn by a database tool from a database's catalog (Section 2.5): each table as a box with its columns, and a line for

each foreign key. It is exact, since it is read from the database, and it stays current with one click. It is also a picture of the implementation, not of the business. Recall stage D4's stock levels: the class model of Figure 5.7 draws them as an association class of warehouses and products, while a generated diagram draws a table, `stock_level`, with two foreign keys, and nothing on it says that a warehouse and a product may have at most one stock level between them, except a `UNIQUE` constraint the diagram may not show.

Thus, you need both. A design tool for the models you hand in and discuss, and your database tool for checking what you built. The next section describes the design approach this book uses.

E.2. Diagrams as text: PlantUML and Graphviz

Every diagram in this book is written as text. A diagram is a small file that names the classes, their attributes, and the associations between them, and a program lays it out and draws it. Writing a diagram as text has four advantages that matter for engineering work. The file sits beside the SQL and code it describes, in the same project, under the same version control, so a change to the model is recorded with the change to the tables. A diagram can be generated by a program, as this book's product structure trees and schema diagram are generated from its database. One shared style file makes every diagram look alike. And nothing needs dragging into place: the layout program places the boxes.

PlantUML (`plantuml.com`) is the tool for UML and crow's foot diagrams. It is free, written in Java, and draws class, object, sequence, and many other UML diagrams, and entity diagrams in crow's foot notation. The class model of stage D4, Figure E.1, is drawn from this file:

```
@startuml
!include _style.iuml
```

```
' Stage D4: a second association class.
class Customer {
  customerNumber
  name
  city
  state
}
class CustomerOrder {
  orderNumber
  orderDate
  requestedShipDate [0..1]
}
class OrderLine {
  lineNumber
  quantity
  unitPrice
}
class Product {
  productNumber
  description
  listPrice
}
class Warehouse {
  warehouseCode
  city
  state
}
class StockLevel {
  quantityOnHand
  reorderPoint
}
Customer "1  " -- "0..*  " CustomerOrder
CustomerOrder "1  " *-- "1..*  " OrderLine
```

```
OrderLine "0..*  " -- "1  " Product
Warehouse "0..*  " -- "0..*  " Product
(Warehouse, Product) .. StockLevel
@enduml
```

Read the file against Figure E.1. Each `class` block is a class with its attributes. `"1  " -- "0..*  "` is an association with its multiplicities, written at the ends they belong to (1); the spaces inside the quotes are optional padding that moves a label clear of its line. `*--` is a composition, its filled diamond at the first class. The last two lines draw the association class: an association between `Warehouse` and `Product`, and `StockLevel` attached to it with a dashed line. A qualifier is written in square brackets, `CustomerOrder [lineNumber] "\n1  " *-- "0..1" OrderLine`, with the composition's `1` at the whole end; the `\n` and the trailing spaces push that `1` below and beside the qualifier's box, where it does not cover the diamond. A generalization is written `Part <|-- ManufacturedPart`. These are the constructs that simpler tools cannot draw, and the reason this book chose PlantUML.

The same tool draws crow's foot diagrams. The stock levels of Figure D.2 come from this file, whose `||--o{` is a line with two bars at one end and a circle with a crow's foot at the other (Table D.1):

```
@startuml
!include _style.iuml
hide stereotype
' Stage D4's stock levels in crow's foot notation: the
 ↪  association class
' StockLevel as an associative entity (translation appendix).
entity WAREHOUSE {
  * warehouse_code
  --
  city
```

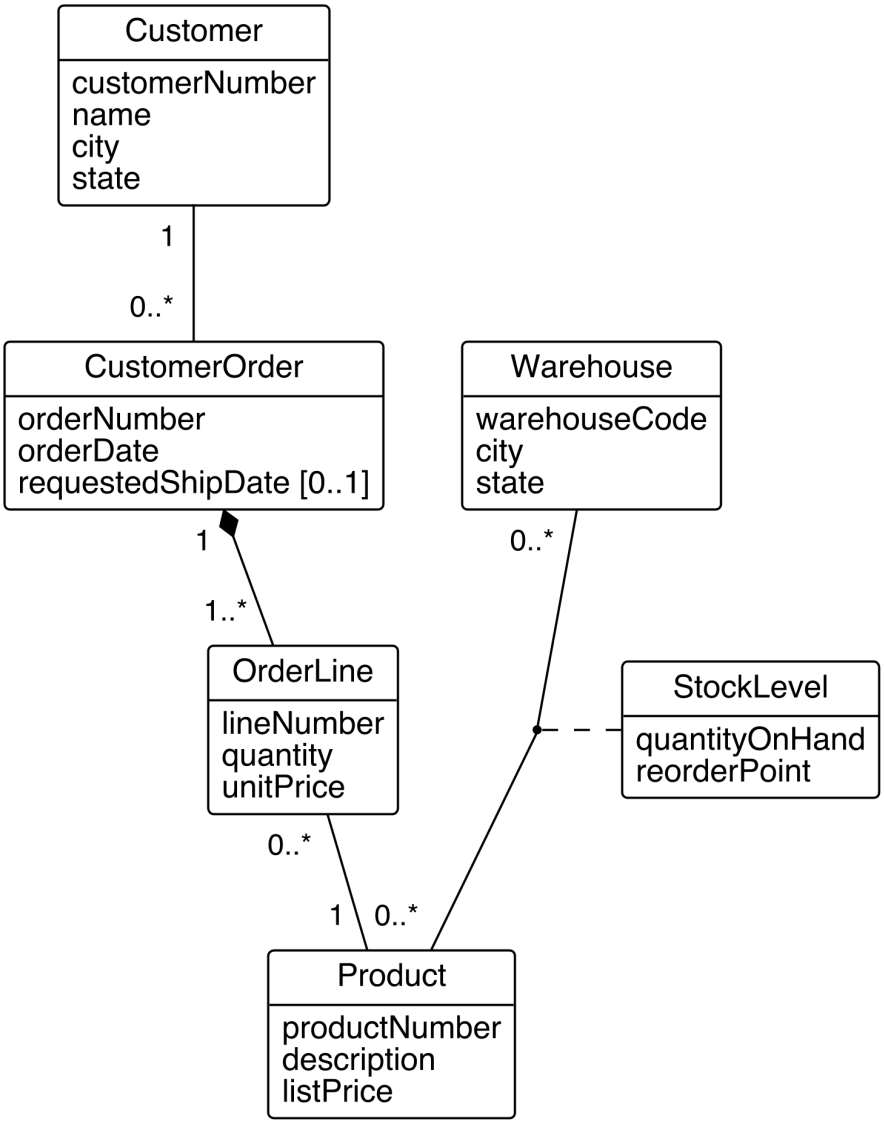


Figure E.1.: Stage D4's Class Model, Drawn from the File Above

```
  state
}
entity STOCK_LEVEL {
  * warehouse_code (FK)
  * product_number (FK)
  --
  quantity_on_hand
  reorder_point
}
entity PRODUCT {
  * product_number
  --
  description
  list_price
}
WAREHOUSE ||--o{ STOCK_LEVEL
STOCK_LEVEL }o--|| PRODUCT
@enduml
```

The first line of each file after `@startuml` includes the book's style file, which is why every figure in the book looks alike:

```
' Shared style for every diagram in the book. Included by
 ↪  each .puml with
'   !include _style.iuml
' Changing this file marks every diagram stale; rerun
 ↪  scripts/build-diagrams.sh.
!theme plain
hide circle
hide empty methods
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam defaultFontName Helvetica
```

```
' Diagram text at about the size of the book's body text.
scale 1.2
```

PlantUML lays out a class diagram with **Graphviz** (`graphviz.org`), a separate, free program for drawing graphs, which you install beside it. Without Graphviz, PlantUML falls back to a layout engine of its own, whose layouts differ and, in this book's experience, mishandle qualifiers; the book's build therefore requires Graphviz. Graphviz is also useful on its own, for diagrams that are not UML: the product structure trees of Section 4.6, the loop of a recursive query in Figure 4.8, and the dependency diagrams of Chapter 7 are Graphviz graphs. Its language, **DOT**, names nodes and the edges between them:

```
@startdot
digraph loop {
  // How a recursive query runs: the anchor once, then the
 ↪  recursive part
  // again and again on the rows of the previous step, until
 ↪  a step finds no
  // rows. Drawn by hand. Graphviz diagrams set their own
 ↪  fonts, matching the
  // Helvetica of _style.iuml, which applies to UML diagrams
 ↪  only.
  graph [rankdir=TB, nodesep=0.4, ranksep=0.35,
 ↪  fontname="Helvetica"];
  node  [shape=box, style="rounded,filled",
 ↪  fillcolor="#f2f2f2",
         fontname="Helvetica", fontsize=11, penwidth=1];
  edge  [fontname="Helvetica", fontsize=10, arrowsize=0.7];

  anchor  [label="Anchor\nrun once: find the starting
 ↪  rows\n(the HT-100)"];
  add     [label="Add the new rows to the result,\nand make
 ↪  them the working table", fillcolor="#dde8f3"];
```

```
  step    [label="Recursive part\njoin the working table to
↪  bom_line\nto find the next rows", fillcolor="#e3f0dc"];
  bom     [label="bom_line", shape=cylinder, style="filled",
↪  fillcolor="white"];
  any     [label="Any new\nrows?", shape=diamond,
↪  style="filled", fillcolor="white"];
  stop    [label="Stop: the result holds\nthe rows of every
↪  step", fillcolor="#f6e6d4"];

  anchor -> add;
  add -> step;
  bom -> step [style=dashed, arrowhead=none];
  step -> any;
  any -> add [label=" yes"];
  any -> stop [label=" no"];

  { rank=same; step; bom }
}
@enddot
```

Three ways of running PlantUML suit a student. The PlantUML plugins for IntelliJ IDEA and for VS Code draw a preview beside the file as you type, which is the easiest way to learn the notation. The command line renders files to images, as this book's build does (`java -jar plantuml.jar -tsvg file.puml`). And Quarto documents, like this book, can include the rendered images in a report. PlantUML's web site also offers an online server that renders a diagram pasted into a browser. It is convenient for a homework problem, but the text you paste leaves your computer, so you should not paste an employer's schema or data model into it.

Mermaid (`mermaid.js.org`) is a lighter text notation with class diagrams and entity-relationship diagrams, and it needs no Java: GitHub renders it in Markdown files, and Quarto renders it in documents. The stock levels

as a Mermaid entity-relationship diagram are:

```
erDiagram
    WAREHOUSE ||--o{ STOCK_LEVEL : holds
    PRODUCT ||--o{ STOCK_LEVEL : "is stocked as"
    STOCK_LEVEL {
        int quantity_on_hand
        int reorder_point
    }
```

Mermaid suits a quick diagram in a README or a report. Its class diagrams, however, have no association classes and no qualifiers, so the models of Chapter 5 cannot all be drawn in it; that is why this book uses PlantUML.

E.3. Diagrams from a database: DataGrip, DBeaver, and pgAdmin

The database tools of Appendix A each draw a diagram of a database that exists, from its catalog.

- **DataGrip.** Select a schema or some tables in the *Database Explorer*, right-click, and choose *Diagrams*, *Show Diagram*. Each table is a box with its columns, keys marked, and an arrow for each foreign key. The diagram can be saved as an image, and exported as PlantUML, a text file you can then edit and keep beside your model.
- **DBeaver.** Open a schema or a table and choose its *ER Diagram* tab. Right- clicking the diagram's background offers a choice of *Notation*, crow's foot among them (IDEF1X is the default). The diagram can be saved as an image, PNG among the free edition's formats, or as GraphML for other diagram tools.

- **pgAdmin.** Right-click a database and choose *ERD For Database*. pgAdmin's ERD tool draws the tables, and it also works the other way: you can draw new tables in it and choose *Generate SQL*, which writes the `CREATE TABLE` statements for the design and opens them in the query tool.

Each of these shows what Figure 2.2, the book's generated diagram of the distribution schema, shows: tables, columns, types, keys, and references. None shows an association class as such, a composition's diamond, or a generalization, since the catalog holds only tables and constraints. Use a generated diagram to check that the database you built matches your design, and to learn an unfamiliar database quickly, as the public databases of Section C.11 invite.

E.4. Drawing tools with a canvas

Many students prefer to draw on a canvas, placing shapes with the mouse. Table E.1 lists the tools they most often use.

Table E.1.: Canvas and Web Tools for Diagrams (September 2026)

Tool	Cost to a student	UML class diagrams	Crow's foot	Notes
diagrams.net (draw.io)	free	yes, as shapes	yes	runs in a browser or on the desktop; files are XML you can keep in a project

Tool	Cost to a student	UML class diagrams	Crow's foot	Notes
Lucidchart	free education accounts with a school e-mail; free plan limited to a few documents	yes	yes	runs in a browser; shared editing
Visual Paradigm Community Edition	free for non-commercial use, including education	full UML, with association classes and qualifiers	yes	desktop; exported images carry a small watermark
StarUML	a one-time license; an evaluation version	full UML	yes	desktop
dbdiagram.io	free plan limited to a number of diagrams	no	yes	draws an ER diagram from a text description of tables

A canvas tool draws what you place, so it can draw anything, including a diagram that UML does not allow. Check what you draw against Chapter 5: an association class is attached to its association by a dashed line, a qualifier is a small box on the class at the qualified end, and a multiplicity is written at the far end from the objects it counts.

E.5. Choosing a tool

For a homework design or a project model, any of the tools above will do; pick one that draws association classes and qualifiers, which rules out Mermaid and some web tools. For a model you will revise over weeks, or keep with the code and SQL of a project, write it as text in PlantUML, since a text file records its own history and never needs redrawing. For seeing what a database holds, use your database tool's diagram, and never draw by hand what the database can draw for you. Whatever you use, a diagram you hand in is a class model in the notation of Chapter 5, or a crow's foot diagram translated as Appendix D describes.

E.6. Exercises

Exercise E.1. Write the class model of the kanban loop of Exercise 5.20 in PlantUML, include the style file shown in Section E.2, and render it with the IntelliJ IDEA or VS Code plugin or the command line. Compare your figure with Figure 6.15.

Exercise E.2. In your `dbconcepts` database, draw the diagram of the six tables of the `distribution` schema that stage D4 holds (`customer`, `customer_order`, `order_line`, `product`, `warehouse`, and `stock_level`) with DataGrip, DBeaver, or pgAdmin.

a. List three things the class model of Figure 5.7 states that the generated diagram does not.
b. List three things the generated diagram shows that the class model does not.

Exercise E.3. A classmate drew the tool crib of Exercise 5.18 in Mermaid:

```
erDiagram
    TOOL ||--o{ CHECKOUT : "is checked out in"
    MECHANIC ||--o{ CHECKOUT : makes
    TOOL {
        string tool_number
        string description
        date purchased_on
    }
    MECHANIC {
        string employee_number
        string name
    }
    CHECKOUT {
        timestamp checked_out_at
        timestamp returned_at
    }
```

a. Rewrite it in PlantUML as a crow's foot diagram, in the style of Figure D.6.
b. Read each relationship both ways, in words.
c. Could the same classmate draw the calibration model of Figure 6.14 in Mermaid? Explain.

Exercise E.4. Recommend a tool, and say why, for each of these:

a. A one-time diagram for a homework problem on qualified associations.
b. The data model of a semester project, revised every week by a team of three, and handed in with the project's SQL.
c. Learning the tables of an unfamiliar database a company has given you read access to.
d. A diagram for the README of a project on GitHub.

F. SQL Quick Reference

This appendix collects, in one place, the SQL this book teaches: the statements that define tables, the statements that change data, the clauses of a query, and the data types and functions the chapters use. It is a reminder, not a tutorial. Each entry names the section that teaches it, where the examples and the reasons are.

The skeletons follow three conventions. Words in upper case are SQL; words in lower case stand for names or values you supply. Square brackets enclose what may be left out, and `...` means the previous item may repeat. PostgreSQL is the reference system (Section 1.3); a feature marked **(PostgreSQL)** is PostgreSQL's own, not standard SQL, and may be written differently in another DBMS.

F.1. Defining tables

A table is created with its columns and its rules together. The skeleton gives the parts the book's tables use:

```
CREATE TABLE schema_name.table_name (
    table_pk INTEGER GENERATED ALWAYS AS IDENTITY
        PRIMARY KEY,
    column_name type [NOT NULL] [DEFAULT value]
        [UNIQUE] [CHECK (condition)],
    other_fk INTEGER [NOT NULL]
        REFERENCES schema_name.other_table
```

```
        [ON DELETE action] [ON UPDATE action],
    ...
    [UNIQUE (column_name, ...)],
    [CHECK (condition)]
);
```

Table F.1.: The Parts of CREATE TABLE

Part	What it does	Taught in
`CREATE SCHEMA name`	creates a named group of tables	Section 8.1
`GENERATED ALWAYS AS IDENTITY`	numbers the rows itself: a surrogate key	Section 8.3, Section 6.6
`PRIMARY KEY`	the column or columns that identify a row	Section 2.2
`UNIQUE`	an alternate key: no two rows share the value	Section 2.2
`NOT NULL`	the column must have a value	Section 2.3
`DEFAULT value`	the value used when an insert gives none	Section 2.3
`REFERENCES table`	a foreign key to the table's primary key	Section 2.3
`ON DELETE` / `ON UPDATE`	what happens to referring rows: `NO ACTION` or `RESTRICT` (refuse), `CASCADE` (follow), `SET NULL`	Section 6.3.6, Section 8.3
`CHECK (condition)`	a condition every row must satisfy	Section 8.3

A table's definition changes with `ALTER TABLE`, and the other statements that define things follow the same pattern:

```
ALTER TABLE table_name ADD COLUMN column_name type;
ALTER TABLE table_name DROP COLUMN column_name;
ALTER TABLE table_name
    ALTER COLUMN column_name SET NOT NULL;
ALTER TABLE table_name
    ADD CONSTRAINT name CHECK (condition);
ALTER TABLE table_name RENAME TO new_name;
DROP TABLE table_name [CASCADE];
DROP SCHEMA schema_name [CASCADE];
```

Table F.2.: Other Statements That Define a Database

Statement	What it does	Taught in
`ALTER TABLE`	adds, drops, or changes columns and constraints	Section 8.6
`DROP TABLE`, `DROP SCHEMA`	removes a table or schema; `CASCADE` removes what depends on it	Section 8.3
`TRUNCATE table`	removes every row at once	Section 8.4
`CREATE VIEW name AS query`	a stored query that is used like a table; `WITH CHECK OPTION` keeps changes through it within its condition	Section 9.1

Statement	What it does	Taught in
`CREATE MATERIALIZED VIEW`, `REFRESH MATERIALIZED VIEW`	a view whose result is stored, and recomputed on request **(PostgreSQL)**	Section 9.1, Section 11.5
`CREATE ROLE name`	a user or group that privileges are granted to	Section 9.4
`GRANT`, `REVOKE`	give or take away a privilege (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) on a table or view	Section 9.4

F.2. Changing data

Three statements change rows, and a fourth loads them from a file:

```
INSERT INTO table_name (column_name, ...)
VALUES (value, ...), ...;

INSERT INTO table_name (column_name, ...)
SELECT ...;

UPDATE table_name
SET column_name = expression, ...
[WHERE condition];

DELETE FROM table_name
[WHERE condition];
```

Table F.3.: Statements That Change Data

Statement	What it does	Taught in
`INSERT ... VALUES`	adds the rows listed	Section 8.4
`INSERT ... SELECT`	adds the rows a query returns	Section 8.4, Section 11.3
`UPDATE ... SET ... WHERE`	changes the rows the condition selects; without `WHERE`, every row	Section 8.4
`DELETE FROM ... WHERE`	removes the rows the condition selects; without `WHERE`, every row	Section 8.4
`\copy table FROM 'file.csv' WITH (FORMAT csv, HEADER true)`	loads a CSV file from your computer, in psql **(PostgreSQL)**	Section 8.5

A **transaction** groups changes so that they happen together or not at all (Section 9.2):

```
BEGIN;          -- or START TRANSACTION
...             -- statements
COMMIT;         -- keep the changes, or
ROLLBACK;       -- undo them
```

Table F.4.: Statements That Control Transactions

Statement	What it does	Taught in
`BEGIN`, `START TRANSACTION`	starts a transaction	Section 9.2
`COMMIT`, `ROLLBACK`	ends it, keeping or undoing its changes	Section 9.2
`BEGIN ISOLATION LEVEL level`	starts a transaction at `READ COMMITTED` (PostgreSQL's default), `REPEATABLE READ`, or `SERIALIZABLE`; the standard sets the level with `SET TRANSACTION`	Section 9.3
`SELECT ... FOR UPDATE`	reads rows and locks them against other writers until the transaction ends	Section 9.3

F.3. Queries

A query's clauses are written in one order and evaluated in another. The skeleton gives the written order; the database evaluates `FROM` and its joins first, then `WHERE`, `GROUP BY`, `HAVING`, the window functions, the `SELECT` list, `DISTINCT`, `ORDER BY`, and finally `LIMIT`. That is why a `WHERE` clause cannot name a column alias from the `SELECT` list, and why it runs before the grouping that `HAVING` filters.

```
[WITH [RECURSIVE] name AS (query), ...]
SELECT [DISTINCT] expression [AS alias], ...
```

```
FROM table [AS alias]
    [[LEFT | RIGHT | FULL] JOIN table [AS alias]
        ON condition] ...
[WHERE condition]
[GROUP BY expression, ...]
[HAVING condition]
[ORDER BY expression [ASC | DESC], ...]
[LIMIT n];
```

Table F.5.: The Clauses of a Query

Clause	What it does	Taught in
`SELECT`, `FROM`, `AS`	the columns and expressions wanted, from which table, under what names	Section 3.1
`DISTINCT`	removes duplicate rows from the result	Section 3.1
`WHERE`	keeps the rows for which the condition is true	Section 3.2
`ORDER BY`	sorts the result; without it, the order is not defined	Section 3.4
`LIMIT n`	keeps the first `n` rows **(PostgreSQL)**; the standard's form is `FETCH FIRST n ROWS ONLY`	Section 3.4
`JOIN ... ON`	pairs the rows of two tables that satisfy the condition	Section 3.6

Clause	What it does	Taught in
`LEFT`, `RIGHT`, `FULL JOIN`	also keeps the unmatched rows of one or both tables, with NULL for the other's columns	Section 3.7
`GROUP BY`	forms one group for each combination of values	Section 4.2
`HAVING`	keeps the groups for which the condition is true	Section 4.3
`WITH name AS (query)`	a common table expression: a named step used by the query after it	Section 3.10
`WITH RECURSIVE`	a common table expression that refers to itself, for chains of rows	Section 4.6

Conditions combine comparisons with `AND`, `OR`, and `NOT`, under the three-valued logic of Section 3.3:

Table F.6.: Conditions

Condition	True when	Taught in
`a = b`, `<>`, `<`, `<=`, `>`, `>=`	the comparison holds; unknown if either side is NULL	Section 3.2
`a BETWEEN low AND high`	`a >= low AND a <= high`	Section 3.2

Condition	True when	Taught in
`a IN (value, ...)`	`a` equals one of the values	Section 3.2
`a LIKE 'pattern'`	`a` matches the pattern, `%` for any characters and `_` for one; `ILIKE` ignores case **(PostgreSQL)**	Section 3.2
`a IS NULL`, `a IS NOT NULL`	`a` has no value, or has one	Section 3.3
`EXISTS (subquery)`	the subquery returns at least one row	Section 3.8
`a IN (subquery)`, `a NOT IN (subquery)`	`a` is (or is not) among the subquery's values; `NOT IN` is never true if the subquery returns a NULL	Section 3.8

Two queries' results combine with the set operations, which require the same number of columns with compatible types in each:

Table F.7.: Set Operations

Operation	Result	Taught in
`query UNION query`	the rows in either, each once; `UNION ALL` keeps duplicates	Section 3.9
`query INTERSECT query`	the rows in both	Section 3.9
`query EXCEPT query`	the rows in the first and not the second	Section 3.9

A `CASE` expression chooses a value by condition, and an aggregate's `FILTER` restricts the rows it counts:

```
CASE WHEN condition THEN value
     [WHEN condition THEN value] ...
     [ELSE value]
END

COUNT(*) FILTER (WHERE condition)
```

A **window function** computes a value for each row from a window of related rows, without collapsing them into one:

```
function(...) OVER (
    [PARTITION BY expression, ...]
    [ORDER BY expression, ...]
    [ROWS BETWEEN n PRECEDING AND CURRENT ROW])
```

The `PARTITION BY` divides the rows into windows, the `ORDER BY` orders each window, and the frame (`ROWS BETWEEN ...`) limits the rows an aggregate sees, as a moving average needs (Section 4.5). Grouping for summaries at several levels at once extends `GROUP BY`:

Table F.8.: Grouping at Several Levels

Form	Groups formed	Taught in
`GROUP BY ROLLUP (a, b)`	`(a, b)`, `(a)`, and the grand total	Section 11.5
`GROUP BY CUBE (a, b)`	every combination: `(a, b)`, `(a)`, `(b)`, and the grand total	Section 11.5

Form	Groups formed	Taught in
`GROUP BY GROUPING SETS ((a), (b))`	only the groupings listed	Section 11.5
`GROUPING(a)`	1 in a row where `a` was summarized away, 0 otherwise	Section 11.5

F.4. PostgreSQL data types and functions

Table F.9 lists the data types the book's tables use, with the engineer's choice for each kind of value (Section 8.2).

Table F.9.: Data Types

Type	Holds	Use it for
`INTEGER`, `SMALLINT`, `BIGINT`	whole numbers	counts, quantities, keys
`NUMERIC(p, s)`	exact decimals, `p` digits with `s` after the point	money, and any measure whose decimals matter
`REAL`, `DOUBLE PRECISION`	approximate binary floating-point numbers	scientific measures and statistics; never money
`VARCHAR(n)`	text of at most `n` characters	names, descriptions, codes
`CHAR(n)`	text of exactly `n` characters	fixed codes, such as a state
`TEXT`	text of any length **(PostgreSQL)**	long descriptions
`DATE`	a calendar date	order and ship dates
`TIME`, `TIMESTAMP`	a time of day, or a date and time	events during a day

Type	Holds	Use it for
`INTERVAL`	a length of time	durations, date arithmetic
`BOOLEAN`	true or false	flags

Table F.10 lists the functions the chapters use. Text is joined with the operator `||`: `product_number || ' ' || description` gives a label such as `HT-100 Two-wheel hand truck` (Section 3.5).

Table F.10.: Functions Used in This Book

Function	Result	Taught in
`UPPER(t)`, `LOWER(t)`	`t` in upper or lower case	Section 3.5
`LENGTH(t)`, `TRIM(t)`, `SUBSTRING(t FROM i FOR n)`	the length of `t`; `t` without surrounding spaces; `n` characters from position `i`	Section 3.5, Section 8.5
`ROUND(x, d)`	`x` rounded to `d` places	Section 3.5
`ABS(x)`, `SQRT(x)`	the absolute value of `x`; its square root	Section 4.7, Section 12.3
`CAST(x AS type)`, `x::type`	`x` converted to the type; `::` is **(PostgreSQL)**	Section 3.5
`COALESCE(a, b, ...)`	the first argument that is not NULL	Section 3.5
`CURRENT_DATE`	today's date	Section 3.3

Function	Result	Taught in
`EXTRACT(part FROM d)`	a part of a date or time, such as `YEAR`, `MONTH`, `DAY`, or `ISODOW`, the day of the week from 1 for Monday **(PostgreSQL)**	Section 3.5, Section 11.3
`d2 - d1`	the number of days between two dates, as a whole number **(PostgreSQL)**	Section 3.5
`d + INTERVAL '7 days'`	a date or time moved by an interval	Section 11.3
`TO_CHAR(d, 'format')`	a date formatted as text **(PostgreSQL)**	Section 11.3
`generate_series(a, b, step)`	a set of rows from `a` to `b` **(PostgreSQL)**	Section 11.3
`COUNT(*)`, `COUNT(x)`, `COUNT(DISTINCT x)`	rows; values of `x` that are not NULL; distinct values of `x`	Section 4.2
`SUM`, `AVG`, `MIN`, `MAX`	the total, average, least, and greatest of a group's values, ignoring NULL	Section 4.2
`STDDEV_SAMP(x)`, `CORR(x, y)`	the sample standard deviation; the correlation of two columns	Section 4.7, Section 12.3
`PERCENTILE_CONT(p) WITHIN GROUP (ORDER BY x)`	the `p` quantile of `x`, interpolated	Section 4.7

Function	Result	Taught in
`STRING_AGG(t, ', ')`	a group's text values joined into one **(PostgreSQL)**	Section 7.4
`ROW_NUMBER()`, `RANK()`, `DENSE_RANK()`	a row's position within its partition, with ties handled in different ways	Section 4.5.5
`NTILE(n)`	which of `n` nearly equal groups a row falls in	Section 4.7
`LAG(x)`, `LEAD(x)`, `FIRST_VALUE(x)`	`x` from the previous row, the next row, or the frame's first row	Section 4.5.4

A function not listed here is likely in PostgreSQL's documentation (The PostgreSQL Global Development Group, n.d.), whose chapter on functions and operators is organized by the type of value each works on.

G. Embedded Databases: SQLite and DuckDB

This book's databases run on a PostgreSQL server, which programs and tools connect to over the network. An embedded database runs instead inside the program that uses it, with its data in a file. This appendix explains what an embedded database is and when you should use one, introduces two of them, SQLite, which holds the KSL's simulation output, and DuckDB, which is built for analysis, and shows how a Kotlin program creates and queries one.

G.1. What an embedded database is

PostgreSQL is a **client-server** database. The server is a program of its own, always running; DataGrip, psql, and the book's Kotlin programs are clients that connect to it, sign in as a user, and send it SQL (Appendix A). The server holds the data, enforces the constraints, and lets many clients work at once.

An **embedded database** has no server. Its database engine is a library that runs inside the program that opens it, and its data is an ordinary file, or a directory of files, on the program's own disk. There is nothing to install and start, no user to create, and no password; opening the database means opening the file. When the program ends, so does the engine. The same SQL creates tables and queries them, with differences this appendix points out.

Thus, an embedded database trades the server's services for simplicity. It cannot serve many users at once as a server does, and whoever can read the file can read the data. It needs no administration at all.

G.2. When to use one

An embedded database suits data that belongs to one program or one person.

- **A study's own results.** The quick tables of Section 12.4, written by one program and read by one analyst, need no server. The KSL writes its simulation output to an embedded SQLite database unless it is given another.
- **Data that travels with a program or a report.** A file can be copied, e-mailed, or archived with the study it belongs to, and opened years later with any tool that reads its format.
- **A tool on one desktop,** a test that needs a fresh database each time it runs, or a device with no network.

A client-server database suits data that is shared: many people or programs writing at once, users with different privileges (Section 9.4), data that must be backed up and kept available by someone whose job it is. HG Industries' order and production databases are shared, which is why the book runs them on PostgreSQL. A study that starts in an embedded file and becomes shared should move to a server; the SQL mostly moves with it.

G.3. SQLite

SQLite is the most widely used embedded database. It is in the public domain, keeps a whole database in one file, and is built into phones, web browsers, and many desktop programs. Its command-line shell, `sqlite3`,

comes with macOS and Linux and is a small download for Windows; DataGrip and DBeaver open a SQLite file directly, as a data source of type SQLite.

The companion scripts in `code/sql/embedded/` build the warehouses and stock levels of stage D4 in a SQLite file. The first creates two tables, loads a few rows, and queries them. Run with `sqlite3 -echo stock.db < sqlite-stock-create.sql`, the shell creates `stock.db` if it does not exist, echoes each statement, and prints each result:

```
.headers on
.mode column
CREATE TABLE warehouse (
    warehouse_pk INTEGER PRIMARY KEY,
    warehouse_code TEXT NOT NULL UNIQUE,
    city TEXT NOT NULL,
    state TEXT NOT NULL
);
CREATE TABLE stock_level (
    stock_level_pk INTEGER PRIMARY KEY,
    warehouse_fk INTEGER NOT NULL REFERENCES warehouse,
    product_number TEXT NOT NULL,
    quantity_on_hand INTEGER NOT NULL
        CHECK (quantity_on_hand >= 0),
    UNIQUE (warehouse_fk, product_number)
);
INSERT INTO warehouse (warehouse_code, city, state)
VALUES ('W-FAY', 'Fayetteville', 'AR'), ('W-TUL', 'Tulsa',
 ↪  'OK');
INSERT INTO stock_level
    (warehouse_fk, product_number, quantity_on_hand)
VALUES (1, 'HT-100', 120), (1, 'GC-300', 18), (2, 'HT-100',
 ↪  60);
SELECT w.warehouse_code, s.product_number, s.quantity_on_hand
```

```
FROM stock_level AS s
JOIN warehouse AS w ON w.warehouse_pk = s.warehouse_fk
ORDER BY w.warehouse_code, s.product_number;
warehouse_code  product_number  quantity_on_hand
--------------  --------------  ----------------
W-FAY           GC-300                        18
W-FAY           HT-100                       120
W-TUL           HT-100                        60
```

The statements are the book's own, simplified: `stock_level` holds the product's number rather than a foreign key to `product`, and no reorder point. In addition, they make three changes that SQLite's rules call for. The types are SQLite's (`TEXT` for strings); a key column declared `INTEGER PRIMARY KEY` is numbered by SQLite itself, the role an identity column plays in PostgreSQL; and `.headers` and `.mode` are commands of the shell, not SQL, which format the output.

SQLite departs from PostgreSQL, and from the standard, in two ways that matter to a designer. The first is its types. A column's declared type is a preference, called its *type affinity*, not a rule, so SQLite stores a value of another type rather than refuse it. The second script inserts the text `forty` into `quantity_on_hand`:

```
.headers on
.mode column
INSERT INTO stock_level
    (warehouse_fk, product_number, quantity_on_hand)
VALUES (2, 'GC-300', 'forty');
SELECT product_number, quantity_on_hand,
       typeof(quantity_on_hand) AS stored_as
FROM stock_level
WHERE warehouse_fk = 2;
product_number  quantity_on_hand  stored_as
```

```
--------------  ---------------  ---------
GC-300          forty            text
HT-100                       60  integer
CREATE TABLE stock_count (
    product_number TEXT NOT NULL,
    counted INTEGER NOT NULL
) STRICT;
INSERT INTO stock_count VALUES ('GC-300', 'forty');
Error near line 15: cannot store TEXT value in INTEGER column
 ↪  stock_count.counted
```

The insert succeeded, and `typeof` shows that the column now holds a text value in one row and an integer in another. Even the `CHECK` constraint passed, since SQLite compares any text as greater than any number. A **strict table**, declared with the word `STRICT` after its definition (SQLite 3.37 and later), enforces the declared types, and the last statement shows it refusing the same value. You should declare a SQLite table `STRICT` whenever its data will be analyzed, since a wrong type is otherwise stored silently.

The second departure is foreign keys. SQLite parses a `REFERENCES` clause but, for compatibility with old databases, enforces it only on a connection that asks, with `PRAGMA foreign_keys = ON`:

```
.headers on
.mode column
INSERT INTO stock_level
    (warehouse_fk, product_number, quantity_on_hand)
VALUES (9, 'HT-200', 5);
PRAGMA foreign_keys = ON;
INSERT INTO stock_level
    (warehouse_fk, product_number, quantity_on_hand)
VALUES (9, 'GC-400', 7);
```

```
Error near line 9: FOREIGN KEY constraint failed
SELECT stock_level_pk, warehouse_fk, product_number
FROM stock_level
WHERE warehouse_fk NOT IN (SELECT warehouse_pk FROM
 ↪  warehouse);
stock_level_pk  warehouse_fk  product_number
--------------  ------------  --------------
             5             9  HT-200
```

The first insert referred to warehouse 9, which does not exist, and SQLite accepted it: the shell's connection had not asked. After the `PRAGMA`, the same kind of insert was refused. The orphan row remains, and the last query finds it with an anti-join (Section 3.7), written with `NOT IN`. The setting lasts only as long as the connection, so every program and tool that writes the file must turn it on; the KSL's SQLite connections do (Section G.5).

A few other differences are worth knowing before you design for SQLite:

- **No date type.** Dates and times are stored as text in the ISO form (`2026-09-29 14:30:00`), as numbers, or as both, and SQLite's date functions work on any of them.
- **One writer at a time.** Many connections may read at once, but a write locks the whole file briefly. That suits one program, not a busy system.
- **Limited `ALTER TABLE`.** A table can be renamed and columns added, renamed, or dropped. Recent versions (3.53 and later; this book used 3.54) can also set or drop a column's `NOT NULL` and add or drop a `CHECK` constraint, but a foreign key or a `UNIQUE` constraint still cannot be added to an existing table's columns (a new column may carry a `REFERENCES` clause); the table is rebuilt instead.
- **No schemas.** A file is one database with one set of tables. `ATTACH` opens a second file under a name, which serves where PostgreSQL would use a schema (Section 8.1).

G.4. DuckDB

DuckDB is an embedded database built for analysis. Like SQLite, it runs inside the program that opens it and keeps a database in one file, and it is open source, under the MIT license. Where SQLite is designed for a program's many small reads and writes, DuckDB is designed for queries that scan, group, and summarize many rows. Its shell, `duckdb`, is a single program, installed on macOS with `brew install duckdb` and downloaded from `duckdb.org` for Windows and Linux; DataGrip and DBeaver open a DuckDB file as a data source of type DuckDB.

The difference in design lies in how the rows are stored. PostgreSQL and SQLite are **row stores**: each row's values are kept together, so finding, inserting, or changing one row touches one place. DuckDB is a **column store**: each column's values are kept together, so a query that averages 3 columns of a 20-column table reads only those 3, and a column of like values compresses well. Thus, a row store suits a system that records transactions one at a time, and a column store suits a study that summarizes what was recorded.

G.4.1. The stock tables in DuckDB

The companion script `duckdb-stock.sql` builds the tables of Section G.3 with the book's own DDL, `VARCHAR` columns and all, simplified as there, with one further change: DuckDB has no identity columns, so each key draws its values from a **sequence**, a named counter that `nextval` advances. (PostgreSQL has sequences too; an identity column is built on one.) Run in a terminal with `duckdb -echo stock.duckdb < duckdb-stock.sql`, it creates the tables, loads them, and queries them:

```
.mode column
CREATE SEQUENCE warehouse_seq;
CREATE TABLE warehouse (
```

```
    warehouse_pk INTEGER
        DEFAULT nextval('warehouse_seq') PRIMARY KEY,
    warehouse_code VARCHAR(10) NOT NULL UNIQUE,
    city VARCHAR(40) NOT NULL,
    state CHAR(2) NOT NULL
);
CREATE SEQUENCE stock_level_seq;
CREATE TABLE stock_level (
    stock_level_pk INTEGER
        DEFAULT nextval('stock_level_seq') PRIMARY KEY,
    warehouse_fk INTEGER NOT NULL REFERENCES warehouse,
    product_number VARCHAR(10) NOT NULL,
    quantity_on_hand INTEGER NOT NULL
        CHECK (quantity_on_hand >= 0),
    UNIQUE (warehouse_fk, product_number)
);
INSERT INTO warehouse (warehouse_code, city, state)
VALUES ('W-FAY', 'Fayetteville', 'AR'), ('W-TUL', 'Tulsa',
 ↪  'OK');
INSERT INTO stock_level
    (warehouse_fk, product_number, quantity_on_hand)
VALUES (1, 'HT-100', 120), (1, 'GC-300', 18), (2, 'HT-100',
 ↪  60);
SELECT w.warehouse_code, s.product_number, s.quantity_on_hand
FROM stock_level AS s
JOIN warehouse AS w ON w.warehouse_pk = s.warehouse_fk
ORDER BY w.warehouse_code, s.product_number;
warehouse_code  product_number  quantity_on_hand
--------------  --------------  ----------------
W-FAY           GC-300          18
W-FAY           HT-100          120
W-TUL           HT-100          60
```

The result is the one SQLite and PostgreSQL gave. DuckDB's SQL is close to PostgreSQL's, and its types are enforced. One exception is worth knowing: a length in a type, as in `VARCHAR(10)` or `CHAR(2)`, is accepted and ignored, so a longer value is stored without complaint. A length rule that matters belongs in a `CHECK` constraint.

The rest of the script repeats SQLite's two wrong inserts and then changes a warehouse twice:

```
INSERT INTO stock_level
    (warehouse_fk, product_number, quantity_on_hand)
VALUES (2, 'GC-300', 'forty');
Conversion Error: Could not convert string 'forty' to INT32

LINE 3: VALUES (2, 'GC-300', 'forty');
                           ^
INSERT INTO stock_level
    (warehouse_fk, product_number, quantity_on_hand)
VALUES (9, 'HT-200', 5);
Constraint Error: Violates foreign key constraint because key
 ↪  "warehouse_pk: 9" does not exist in the referenced table
UPDATE warehouse SET city = 'Springdale'
WHERE warehouse_code = 'W-FAY';
UPDATE warehouse SET warehouse_code = 'W-SPR'
WHERE warehouse_code = 'W-FAY';
Constraint Error: Violates foreign key constraint because key
 ↪  "warehouse_fk: 1" is still referenced by a foreign key in
 ↪  a different table. If this is an unexpected constraint
 ↪  violation, please refer to our foreign key limitations in
 ↪  the documentation
SELECT * FROM warehouse;
warehouse_pk  warehouse_code  city        state
------------  --------------  ----------  -----
1             W-FAY           Springdale  AR
```

```
2             W-TUL           Tulsa       OK
```

DuckDB refused both inserts that SQLite accepted: the text in an `INTEGER` column, and the stock level of a warehouse that does not exist. It accepted the change of the warehouse's city. But it refused the change of its code, although no stock level refers to the code; the stock levels refer to `warehouse_pk`, which the update leaves alone. PostgreSQL would accept the update. DuckDB carries out an update of a column in a key or a `UNIQUE` constraint as a delete of the old row and an insert of the new one, and its foreign key sees the delete of a warehouse that stock levels still refer to. Its foreign keys have two more limits: no `ON DELETE CASCADE`, and one transaction cannot delete the rows that refer to a row and then the row itself. The last query shows that the refused update changed nothing.

Thus, DuckDB's constraints suit data that is loaded and then analyzed, not data that is edited row by row. A study's results are loaded once and queried many times, which is the work DuckDB is built for. A system of record, such as HG Industries' orders, belongs on a server, or in SQLite when it belongs to one program.

G.4.2. Analysis straight from files

DuckDB can query a file directly: a CSV file's name, in quotes, may stand where a table's name would, and DuckDB reads the column names from the header and infers each column's type from the values. The companion file `code/data/frame-line-replications.csv` holds three responses of every replication of the frame line's two scenarios, long, as `within_rep_view` holds them (Section 12.4.2): 2 scenarios × 30 replications × 3 responses, 180 rows. The companion script `duckdb-results.sql`, run from the `code` folder with `duckdb -echo < sql/embedded/duckdb-results.sql`, first asks what DuckDB made of the file:

```
.mode column
SELECT column_name, column_type
FROM (DESCRIBE
      SELECT * FROM 'data/frame-line-replications.csv');
column_name  column_type
-----------  -----------
exp_name     VARCHAR
rep_id       BIGINT
stat_name    VARCHAR
rep_value    DOUBLE
```

`DESCRIBE` reports each column of a query's result with its type. DuckDB read `rep_id` as an integer (`BIGINT`) and `rep_value` as a `DOUBLE`, with no `CREATE TABLE` and no load step. The script then summarizes the replications by scenario and response:

```
SELECT exp_name, stat_name, COUNT(*) AS reps,
       ROUND(AVG(rep_value), 3) AS average,
       ROUND(STDDEV_SAMP(rep_value), 3) AS std_dev
FROM 'data/frame-line-replications.csv'
GROUP BY exp_name, stat_name
ORDER BY stat_name, exp_name;
exp_name      stat_name      reps  average  std_dev
------------  -------------  ----  -------  -------
ThreeWelders  LotsCompleted  30    89.567   8.123
TwoWelders    LotsCompleted  30    89.6     8.101
ThreeWelders  TimeInShop     30    33.537   7.282
TwoWelders    TimeInShop     30    33.654   7.258
ThreeWelders  WorkInProcess  30    5.2      1.75
TwoWelders    WorkInProcess  30    5.217    1.748
```

Each group has 30 rows, one per replication, which accounts for all 180 rows of the file. As a check, the averages and standard deviations agree to three

decimals with the across-replication statistics that the KSL computed and stored in `across_rep_view` (Section 12.2). The two scenarios' averages differ by a small fraction of a standard deviation, which is why Example 12.1 compares them replication by replication.

The last statement turns the long rows of `TwoWelders`' first three replications wide:

```
PIVOT (
    SELECT rep_id, stat_name,
           ROUND(rep_value, 3) AS rep_value
    FROM 'data/frame-line-replications.csv'
    WHERE exp_name = 'TwoWelders' AND rep_id <= 3
)
ON stat_name
USING MAX(rep_value)
ORDER BY rep_id;
rep_id  LotsCompleted  TimeInShop  WorkInProcess
------  -------------  ----------  -------------
1       85.0           28.126      3.908
2       85.0           28.981      4.094
3       86.0           36.98       5.904
```

`PIVOT` names the column whose values become columns (`ON stat_name`) and the aggregate that fills each cell (`USING MAX(rep_value)`), and the remaining column, `rep_id`, gives the rows. The result is Table 12.15, which Section 12.4.2 built by hand and then with one `FILTER` per response. Notice that `PIVOT` found the three columns in the data; the `FILTER` query listed them by hand. `PIVOT` is DuckDB's own statement: neither PostgreSQL nor the SQL standard has it. DuckDB's `UNPIVOT` goes the other way, and its `COPY` statement writes a result to a CSV file, or to a Parquet file (a column-store file format that many analysis tools read), for the next tool in a study.

G.5. Embedded databases from Kotlin and the KSL

A program opens an embedded database through JDBC, as it opens PostgreSQL (Section 10.1), with a URL that names a file instead of a server: `jdbc:sqlite:stock.db` or `jdbc:duckdb:stock.duckdb`. The driver is the engine, so the program needs only the driver's library. The companion project's build includes SQLite's through the KSL.

The KSL wraps SQLite in classes of its `ksl.utilities.io.dbutil` package that Section 10.4 introduced: `SQLiteDb` creates or opens a database in a directory, and the function `Database.createSimpleDb` creates one with tables defined by `DbTableData` data classes, the quick tables of Section 12.4. The KSL also supports Apache Derby, an embedded database written in Java, through `DerbyDb`; its developers retired Derby in October 2025, so use it only to open a database that already uses it. A `KSLDatabase` created with only a name, `KSLDatabase("study.db")`, is an embedded SQLite database that holds the KSL's simulation output. The KSL's SQLite connections turn on foreign key enforcement, so the problem of the previous section does not arise in them.

The companion program `SolverRunsSQLite.kt` writes the solver runs of Section 12.4 to a SQLite file and summarizes them. It creates the database and its table from the data class `SolverRun`:

```
val dir = Files.createDirectories(Path.of("build",
 ↪  "embedded"))
val db = Database.createSimpleDb(
    tableDefinitions = setOf(SolverRun()),
    dbName = "solver-runs.db",
    dbDirectory = dir
)
```

It then inserts four runs, as objects of the data class:

```
val runs = listOf(
    SolverRun(1, "Rosenbrock", "SA-slow", 1, 10.05, 900,
 ↪  0.005),
    SolverRun(2, "Rosenbrock", "SA-slow", 2, 10.30, 1000,
 ↪  0.030),
    SolverRun(3, "Rosenbrock", "RS", 1, 10.20, 1000, 0.020),
    SolverRun(4, "Rosenbrock", "RS", 2, 10.90, 1000, 0.090)
)
val inserted = db.insertAllDbDataIntoTable(runs,
 ↪  "solver_run")
println("Inserted $inserted rows into solver_run")
```

and summarizes them by solver with a grouping query:

```
val sql = """
    SELECT solverLabel, COUNT(*) AS runs,
           ROUND(AVG(gap), 3) AS average_gap
    FROM solver_run
    GROUP BY solverLabel
    ORDER BY solverLabel
    """.trimIndent()
db.fetchCachedRowSet(sql)?.let { rs ->
    while (rs.next()) {
        println("%-8s %d  %.3f".format(rs.getString(1),
 ↪  rs.getInt(2), rs.getDouble(3)))
    }
}
```

It prints:

```
Inserted 4 rows into solver_run
RS       2  0.055
SA-slow  2  0.017
```

The table is the one whose `CREATE TABLE` statement Section 12.4 printed, with its `DOUBLE` columns; SQLite accepts that type, where PostgreSQL refused it. The file `build/embedded/solver-runs.db` can be opened in DataGrip or with `sqlite3` and queried like any other database, and it can be copied, with the study it belongs to, to anyone.

The KSL does not include DuckDB. A program that writes to DuckDB adds its driver, `org.duckdb:duckdb_jdbc`, to the build; the driver carries DuckDB's engine compiled for each operating system, and is some 85 MB. A study more often needs DuckDB only for analysis, and then needs no program of its own: every KSL database can write its views as CSV files, one per view, with the function `exportAllViewsAsCSV`, and DuckDB queries those files as Section G.4 queried `frame-line-replications.csv`.

G.6. Choosing a database

Table G.1 sets the three databases side by side on the matters this appendix has raised.

Table G.1.: PostgreSQL, SQLite, and DuckDB Compared

Matter	PostgreSQL	SQLite	DuckDB
Runs as	a server	a library in the program	a library in the program
Data held in	the server's own storage	one file	one file
Built for	many users' transactions	one program's reads and writes	analysis of many rows
Stores rows	together	together	by column

Matter	PostgreSQL	SQLite	DuckDB
Users and privileges	yes	no	no
Declared types	enforced	a preference, unless `STRICT`	enforced, lengths ignored
Foreign keys	enforced	enforced only when a connection asks	enforced, with limits on updates and deletes
Key numbering	identity column	`INTEGER PRIMARY KEY`	sequence
Schemas	yes	no (`ATTACH` instead)	yes
Queries files directly	no	no	CSV and Parquet
Writers at once	many	one	one program

Choose PostgreSQL, or another server, for data that is shared. Choose SQLite for data that belongs to one program, one study, or one person, and declare its tables `STRICT` and its connections' foreign keys on. Choose DuckDB to analyze data that has already been recorded, a study's results or a set of exported files, where its column storage and its SQL make the summaries quick to write and to run.

G.7. Exercises

Exercise G.1. For each situation, say whether an embedded database or a database server suits it, and why.

a. A simulation study run by one engineer, whose results are archived with the final report.

b. HG Industries' order entry, used by six people in two warehouses.
c. A tablet application that inspectors carry onto the plant floor, which records measurements while out of network range and sends them later.
d. A benchmark of optimization solvers run by three students on their own laptops, whose results must be combined for one analysis.

Exercise G.2. Using the SQLite scripts of Section G.3:

a. Rewrite the `stock_level` table of `sqlite-stock-create.sql` as a strict table, and say which of the inserts of `sqlite-stock-affinity.sql` it would refuse.
b. The `CHECK (quantity_on_hand >= 0)` of the original table accepted the text `forty`. Write a check that refuses any value that is not an integer, for a table that is not strict, using SQLite's `typeof` function.

Exercise G.3. A colleague's SQLite file of stock levels was written for months by a program that never turned on foreign keys.

a. Write a query that finds every stock level whose warehouse does not exist.
b. Why does turning the setting on now not find or remove those rows?
c. What should every program that writes the file do from now on?

Exercise G.4. The same `CREATE TABLE warehouse (...)` statement is run in PostgreSQL, SQLite, and DuckDB.

a. Under what name does each database store the table?
b. A query copied from a database that folds unquoted names to upper case, as the SQL standard specifies, names the table `"WAREHOUSE"`, in double quotes. Does it work in PostgreSQL? In SQLite and DuckDB? Explain.

Exercise G.5. The script `duckdb-stock.sql` of Section G.4 changed warehouse W-FAY's city, and DuckDB refused to change its code.

a. Which column do the stock levels refer to, and did the refused update change it?
b. Why did DuckDB refuse the update, when PostgreSQL would accept it?
c. A colleague proposes keeping HG Industries' order entry in a DuckDB file, since DuckDB's SQL is so close to PostgreSQL's. Name two operations of order entry that the limits of Section G.4 would make fail or need a workaround, and recommend where the orders should be kept instead.

Exercise G.6. Using DuckDB and the file `code/data/frame-line-replications.csv` of Section G.4, run from the `code` folder:

a. Write a query that returns, for each scenario, the replication with the longest time in shop: the scenario, the replication, and the time in shop rounded to three decimals. Explain what you notice about the two rows.
b. Repeat the Little's law check of Table 12.17 for the first five replications of `TwoWelders`, building the wide table with `PIVOT` instead of `FILTER`. Does your result match Table 12.17?

Exercise G.7. Run `SolverRunsSQLite.kt` of Section G.5, and open the file it writes in DataGrip, DBeaver, or `sqlite3`.

a. Write a query that returns, for each solver, the run with the smallest gap: the solver, the replication, and the gap.
b. Add a fifth run of your own with a gap of `NULL`. What does the program's summary query now report for its solver, and why?

H. NoSQL and Other Data Stores

This book teaches the relational model because it holds most of the data an industrial engineer works with: orders, parts, routings, inspections, and the results of studies. It is not the only way to store data, however, and you will meet others, in a plant's historian, in a supplier's web service, or in a product team's application. This appendix describes the databases grouped under the name NoSQL, the trade-offs they make against the relational model, and how PostgreSQL itself stores and queries JSON documents, so that you can try the ideas on the server you already have.

Recall from Section 2.7 that two questions separate one way of organizing data from another: where the description of the data lives, and how one record is connected to another. Those two questions run through the whole appendix.

> **Following along**
>
> The queries of Section H.4 run against stage D5 of the book's database. The script `sql/documents/order-documents.sql` of the companion code holds every statement of that section, so you can run them in DataGrip or DBeaver. Reload stage D5 afterward (Section C.2) to remove the table it creates.

H.1. Why NoSQL databases exist

A relational database was designed to run on one server and to keep one copy of each fact, checked against the rules in its catalog on every change. That design suits most business and plant data, and it still does. In the 2000s, however, companies running large web services met three needs it served poorly. Their data no longer fit on one machine, so it had to be spread over many machines and copied for safety. The structure of their records changed every few weeks, as the applications that wrote them changed. And much of their data arrived already shaped as documents, from browsers, phones, and other services.

The databases built for those needs came to be called **NoSQL**, usually read "not only SQL" (Sadalage and Fowler 2012). The name describes a family, not a rule: several NoSQL databases now accept a query language that looks much like SQL, and relational databases, PostgreSQL among them, have taken on features of NoSQL ones. What the family shares is a different answer to the two questions of Section 2.7. A NoSQL database usually keeps the description of its data in the records or in the programs, not in a catalog, and it connects records by nesting one inside another, or not at all, more often than by keys.

H.2. The four families

NoSQL databases fall into four families, distinguished by what they store and how a program finds it. Table H.1 summarizes them, with a use for each in a company like HG Industries.

Table H.1.: The Four Families of NoSQL Databases

Family	Stores	A program finds a record by	Examples	A use at HG Industries
Key-value	A value of any form under a key	Its key, and nothing else	Redis, Amazon DynamoDB	The current state of each machine, for a dashboard
Document	JSON documents, grouped in collections	Its key, or the values of its fields	MongoDB, Couchbase	Test records whose fields differ from product to product
Wide-column	Rows of many columns, grouped and spread over machines by a key	Its key, often a range of keys	Apache Cassandra, Apache HBase	Sensor readings, by machine and time
Graph	Nodes and the relationships between them	Following relationships from a node	Neo4j	The network of suppliers behind each product

A **key-value store** is the simplest: it stores a value under a key and returns the value when given the key. The value may be a number, a string, or a document, and the store does not look inside it. That simplicity makes it very fast, which is why it often sits in front of another database, holding the answers to frequent questions. For example, a dashboard on the plant floor that shows each machine's current state reads one value per machine

number, many times a second.

A **document database** stores each record as a JSON document, as Listing 2.5 showed, and groups documents of one kind into a **collection**, its counterpart of a table. Unlike a key-value store, it looks inside the documents: a query can ask for every order whose lines include a GC-300. Documents in one collection need not have the same fields. Thus, a document database suits records whose structure varies, such as test records in which a hand truck's inspection measures different characteristics than a garden cart's.

A **wide-column store** holds rows that may have very many columns, grouped by a key that decides which machine stores them. It is built for writing enormous numbers of rows quickly and reading them back by key and range. Sensor readings fit that pattern: each reading is written once, never changed, and read back by machine and time.

A **graph database** stores nodes, such as suppliers, parts, and products, and the relationships between them as data in their own right. Its queries follow relationships: which products depend, through any number of tiers, on one supplier's plant. A relational database answers the same question with a recursive query (Section 4.6); a graph database makes following a path its basic operation, which pays off when the paths are long and the questions are about the paths themselves.

H.2.1. Time-series databases and plant historians

The non-relational data you are most likely to meet in a plant is neither of these four. A **time-series database** records the value of a measurement at each moment: a spindle's speed, an oven's temperature, a line's count of finished units. In manufacturing, the usual one is a **plant historian**, such as the AVEVA PI System, which collects thousands of such measurements, called tags, from the plant's controllers, many times a second, and

keeps years of them. Other time-series databases include InfluxDB and TimescaleDB, the latter an extension of PostgreSQL.

A historian answers questions about one tag over a span of time well, such as the average oven temperature during each shift. It does not hold orders, parts, or work orders, so the questions an industrial engineer asks usually combine the two: the temperatures during the runs of one lot that failed inspection. The usual design summarizes the readings (an average, a minimum, and a maximum for each tag over each hour or each run) and loads the summaries into a relational database, where they join the rest of the plant's data. That is the extract, transform, and load of Section 11.3, and the summaries are often the input data of the simulation models of Section 12.1.

H.3. The trade-offs

Each family gives up something the relational model provides in return for something it does not. This section names the four trade-offs that matter most when you choose.

H.3.1. Schema on write and schema on read

A relational database applies its schema when data is written. Every row is checked against the catalog of Section 2.5 before it is stored, and a row that does not fit is refused. This is called **schema on write**. A document database usually applies no schema when data is written: it stores whatever document it is given, and the program that reads a document decides what its fields mean. This is called **schema on read**.

Schema on read lets the structure change without redesigning the database, and it lets one collection hold records of different shapes. Its cost is that every program that reads the data must cope with every shape that has ever been written, including the mistakes. Section H.4 shows a document

with a misspelled field being accepted, and a query answering wrongly as a result. Many document databases now let a designer declare rules that documents must obey, which moves them part of the way back to schema on write.

H.3.2. Nesting is denormalization

A document that nests an order's lines and copies its customer's and products' facts is a table that has not been normalized. Recall from Section 7.2 that such a table suffers anomalies, and the order document of Listing 2.5 suffers them all: a product's description is stored in every order that includes it, a product that has never been ordered has nowhere to be recorded, and removing a customer's only order removes the only record of the customer.

A designer of documents therefore decides, for each connection, whether to nest or to refer. The class model of Chapter 5 gives a good rule. Nest a **composition**, whose parts belong to one whole and are created and removed with it (Section 6.3.6): an order's lines belong in the order's document. Refer, by key, to anything shared: a customer and a product are shared by many orders, so each belongs in a document of its own, and the order holds its customer number and each line's product number. A document database that follows this rule copies only what never changes after it is written, such as the price on an order line.

H.3.3. Consistency across copies

A NoSQL database spread over many machines keeps several copies of each record, so that losing a machine loses nothing. Keeping every copy identical at every moment would mean that each change waits for every machine, and that a change is refused whenever the machines cannot reach one another. Many NoSQL databases choose instead to accept a change at one copy and pass it to the others shortly afterward. This is called

eventual consistency: for a short time, a read from another copy may return the value from before the change.

Compare the transactions of Section 9.2, in which a change is complete and visible to all, or not made at all. For a dashboard, a value a second old does no harm; for the quantity on hand of a part that two planners are allocating, it does. Most NoSQL databases let a program ask for stronger consistency on the operations that need it, at the cost of speed.

H.3.4. Questions nobody planned for

A relational database answers any question its data can answer: a new question is a new query, joining whatever tables it needs. NoSQL databases are designed the other way round, around the questions known in advance. A key-value store finds a record only by its key, a wide-column store by its key and range, and a document database most easily by the fields of one collection; combining collections is often left to the program. Thus, the designer of a NoSQL database starts from the questions and shapes the records to answer them, while the designer of a relational database starts from the business, as Chapter 5 does, and lets the questions come later. For the data of an industrial engineer, whose next question is rarely the last one, that difference usually decides the matter.

H.4. Documents in PostgreSQL

PostgreSQL stores JSON documents in a column of type `JSONB`, a binary form of JSON that it can search and index, and it provides operators to reach inside them. This lets one database hold tables and documents side by side. The type and its operators are PostgreSQL's own; the SQL standard defines functions for JSON since 2016, and PostgreSQL implements some of them, but other systems store and query JSON differently.

Let's store the twelve orders of stage D5 as documents. The table has a key, the order number, and one column for the document; the database checks the key and that the document is well-formed JSON, and nothing more.

```
CREATE TABLE distribution.order_document (
    order_number VARCHAR(10) PRIMARY KEY,
    document     JSONB NOT NULL
);
```

The next statement builds a document for each order from the relational tables, in the form of Listing 2.5. The function `jsonb_build_object` builds a document from pairs of a field name and a value, and `jsonb_agg` gathers the documents of an order's lines into a list.

```
INSERT INTO distribution.order_document
SELECT o.order_number,
       jsonb_build_object(
         'order_number', o.order_number,
         'order_date', o.order_date,
         'customer', jsonb_build_object(
             'customer_number', c.customer_number,
             'name', c.name,
             'city', c.city,
             'state', c.state),
         'lines', (
             SELECT jsonb_agg(jsonb_build_object(
                       'line_number', l.line_number,
                       'product_number', p.product_number,
                       'description', p.description,
                       'quantity', l.quantity,
                       'unit_price', l.unit_price)
                     ORDER BY l.line_number)
```

```
            FROM distribution.order_line AS l
            JOIN distribution.product AS p
              ON p.product_pk = l.product_fk
            WHERE l.customer_order_fk
                  = o.customer_order_pk))
FROM distribution.customer_order AS o
JOIN distribution.customer AS c
  ON c.customer_pk = o.customer_fk;
```

Two operators reach inside a document. The operator `->` returns the value of a field as JSON, so that another operator can be applied to it, and `->>` returns it as text. The following query lists the orders placed in January, with each one's customer and its number of lines.

Table H.2.: January's Orders, Read from Their Documents

```
SELECT order_number,
       document -> 'customer' ->> 'name' AS customer,
       jsonb_array_length(document -> 'lines') AS lines
FROM distribution.order_document
WHERE document ->> 'order_date' < '2026-02-01'
ORDER BY order_number;
```

order_number	customer	lines
SO-5001	Riverside Hardware	2
SO-5002	Cedar Creek Home Center	1
SO-5003	Summit Farm Supply	3
SO-5004	Riverside Hardware	3
SO-5005	Midtown Tool Supply	1
SO-5006	Lakeside Distributors	2

Table H.2 lists six orders. Notice that the condition compares the order date as text, since `->>` returns text; it works here only because dates written in the form `2026-01-05` sort as text in the same order as dates. The customer's name comes from the copy in each document, without a join. PostgreSQL stores a `JSONB` document's fields in an order of its own, which does no harm: a query finds a field by its name, as it finds a column.

The operator `@>` asks whether a document contains another. The following query finds the orders whose lines include a GC-300, by asking whether the document contains a list of lines with one line for that product.

Table H.3.: The Orders That Include a GC-300

```
SELECT order_number,
       document ->> 'order_date' AS order_date
FROM distribution.order_document
WHERE document @> '{"lines":
          [{"product_number": "GC-300"}]}'
ORDER BY order_number;
```

order_number	order_date
SO-5001	2026-01-05
SO-5003	2026-01-12
SO-5004	2026-01-15
SO-5005	2026-01-21
SO-5007	2026-02-03
SO-5009	2026-02-18
SO-5011	2026-03-04
SO-5012	2026-03-10

Table H.3 lists the eight orders of stage D5 that include a GC-300. A

document database answers the same question with a query of its own language, which looks different and does the same.

Example H.1 (Units ordered, from the documents)**.** *The system.* The twelve order documents of stage D5, loaded into `order_document` as above.

The request. The production planner asks how many units of each product have been ordered, and the order documents are all the program has.

The units are inside each document's list of lines, so the query must first turn each list into rows, one per line. The function `jsonb_array_elements` does that; written in the `FROM` clause after the table, it produces the lines of each document, which the query names `line`.

Table H.4.: Units Ordered by Product, from the Order Documents

```
SELECT line ->> 'product_number' AS product,
       SUM((line ->> 'quantity')::INTEGER) AS units
FROM distribution.order_document,
     jsonb_array_elements(document -> 'lines') AS line
GROUP BY line ->> 'product_number'
ORDER BY product;
```

product	units
GC-300	55
HT-100	130
HT-200	30

Notice the cast. A quantity read with `->>` is text, so the query must say that it is a whole number before it can be added. The catalog cannot say so, because the catalog knows only that `document` is a `JSONB` column. This is schema on read: the query supplies the type that a relational table would have declared once.

The check. The same totals come from the relational tables, grouped as in Section 4.2:

Table H.5.: Units Ordered by Product, from the Relational Tables

```
SELECT p.product_number AS product,
       SUM(l.quantity) AS units
FROM distribution.order_line AS l
JOIN distribution.product AS p
  ON p.product_pk = l.product_fk
GROUP BY p.product_number
ORDER BY product;
```

product	units
GC-300	55
HT-100	130
HT-200	30

Table H.4 and Table H.5 agree, product by product, as they must: the documents were built from the tables.

Interpretation. Customers have ordered 130 two-wheel hand trucks, 30 convertible hand trucks, and 55 steel garden carts, the demand the frame line must meet.

Where it breaks. Nothing checks the fields of a document when it is written. Suppose that a program records order SO-5013, for five GC-300 carts, and misspells `quantity`:

```
INSERT INTO distribution.order_document
VALUES ('SO-5013', '{"order_number": "SO-5013",
  "order_date": "2026-03-12",
```

```
  "customer": {"customer_number": "C-1009"},
  "lines": [{"line_number": 1,
             "product_number": "GC-300",
             "qty": 5, "unit_price": 119.00}]}');
```

The database accepts the document, since it is well-formed JSON. The same query, run after it, counts the new line and adds nothing for it:

Table H.6.: Units Ordered by Product, After the Misspelled Document

```
SELECT line ->> 'product_number' AS product,
       SUM((line ->> 'quantity')::INTEGER) AS units,
       COUNT(*) AS lines
FROM distribution.order_document,
     jsonb_array_elements(document -> 'lines') AS line
GROUP BY line ->> 'product_number'
ORDER BY product;
```

product	units	lines
GC-300	55	9
HT-100	130	8
HT-200	30	6

Table H.6 shows nine lines for the GC-300 and still 55 units. The line's `quantity` is missing, so `->>` returns NULL, and `SUM` ignores a NULL (Section 4.2). The five carts are silently lost. A relational table would have refused the row, since `quantity` is declared `NOT NULL` in `order_line` (Chapter 8). The document also names customer C-1009, who does not exist, and nothing checks it; a foreign key would have refused that too.

The second weakness of documents is the copying of Section 7.2. Suppose that HG Industries renames the GC-300 "Steel utility cart", and changes the `product` table:

```
UPDATE distribution.product
SET description = 'Steel utility cart'
WHERE product_number = 'GC-300';
```

The following query sets the description in `product` beside the copies in the documents. The keywords `CROSS JOIN` before the function produce each document's lines, as the comma did above; written out, the cross join reads more clearly than a comma when a `JOIN` to `product` follows it.

Table H.7.: The GC-300's Description in the Table and in the Documents

```
SELECT p.description AS in_product,
       line ->> 'description' AS in_documents,
       COUNT(*) AS lines
FROM distribution.order_document AS d
CROSS JOIN jsonb_array_elements(d.document -> 'lines')
     AS line
JOIN distribution.product AS p
  ON p.product_number = line ->> 'product_number'
WHERE p.product_number = 'GC-300'
GROUP BY p.description, line ->> 'description';
```

in_product	in_documents	lines
Steel utility cart	Steel garden cart	8

Table H.7 shows the product renamed and all eight copies still carrying the old name. Each copy must be found and changed by a program, and

the database cannot tell the program where they are. Thus, a document should copy only what must not change once it is written. A line's unit price is such a fact: it records the price the customer was charged, which a later change of list price must not alter. A product's description is not.

PostgreSQL's documents are most useful beside tables, not instead of them. A table of test results, for example, can keep the columns every test has (the lot, the date, the inspector, the result) as ordinary columns with keys and constraints, and hold the measurements that differ from product to product in one `JSONB` column. The rules that matter are kept by the database, and the part of the record that varies is free to vary.

H.5. Choosing

Most of the data an industrial engineer designs a database for is shared by many people, obeys rules that matter, and will be asked questions no one has thought of yet. That is the data the relational model was made for, and it should be your starting point. The other kinds earn their place for particular data.

- **Records that are written and read whole, and vary in structure**, such as test records, equipment configurations, or the replies of a supplier's web service: a `JSONB` column in PostgreSQL, beside the columns that do not vary, or a document database if the volume calls for one.
- **Measurements at a high rate**, such as sensor readings and machine states: a historian or another time-series database, with summaries loaded into a relational database for analysis.
- **A value read by key many times a second**, such as a dashboard's current state: a key-value store in front of the database that holds the facts.

- **Networks whose questions are about paths**, such as a multi-tier supply network: a graph database, or a recursive query (Section 4.6) while the network is small.

A system that uses several kinds of database, each for the data it suits, is said to practice **polyglot persistence** (Sadalage and Fowler 2012). A plant's information systems usually do: a historian on the floor, a relational database for orders and production, and a warehouse for analysis (Chapter 11). The skills of this book carry over to all of them, because each still has to answer the two questions of Section 2.7, and a designer who can answer them for a relational database can see how another database answers them.

H.6. Exercises

Exercise H.1. For each kind of data, name the family of Table H.1, or a time-series database, that suits it best, or say that a relational database does, and give the reason in one sentence.

a. The torque reading of every fastening on the HT-100 assembly line, about twenty a minute, kept for two years.
b. HG Industries' customers, orders, and order lines.
c. The suppliers of each purchased part, the suppliers of those suppliers, and so on, where the question asked most is which products depend on one supplier's plant.
d. The last known position of each of forty forklifts, shown on a map of the warehouse and refreshed every second.
e. A record of each customer complaint, whose fields differ by product and by the kind of complaint.

Exercise H.2. A document database holds HG Industries' orders, one document per order in the form of Listing 2.5. For each of the following, say whether it should be nested in the order's document, or kept in a

document of its own and referred to by key. Justify each answer with the rule of Section H.3, and name the association or composition of the distribution class model behind it.

a. The order's lines.
b. The customer's name and city.
c. The unit price on each line.
d. The product's description.
e. The shipments that ship the order.

Exercise H.3. Load stage D5, and run `sql/documents/order-documents.sql` from the companion code, which leaves the `order_document` table holding the twelve orders of stage D5 as documents.

a. Using the documents alone, list each customer's number and name with the total value of its orders (the sum of quantity times unit price over its lines), largest first.
b. Write the same query against the relational tables of stage D5, and confirm that the two agree.
c. Which part of the query in (a) supplies what the catalog would have supplied for the query in (b)?

Exercise H.4. The misspelled document of Example H.1 was accepted without complaint.

a. List every way in which that document breaks a rule that the relational tables of stage D5 would have enforced.
b. A developer argues that the program writing the documents should simply be tested more carefully. Give one reason a database's own rules protect data better than a careful program does. (Recall Section 1.3.)
c. Describe a query that would find every order line, across all documents, that has no `quantity` field. You need not write it.

Exercise H.5. A historian records the spindle speed of each of the seven machines in the `production.machine` table of stage P3 once a second, every hour of the year (365 days).

a. How many readings does it hold after one year?
b. The industrial engineers want, for each machine and each hour, the average, minimum, and maximum speed, to join with the work orders run in that hour. How many rows does a relational table of those summaries hold after one year?
c. Write the relational schema of the summary table in the notation of Section 2.4, with a foreign key to a table of machines.
d. Name one question about the spindles that the summary table cannot answer and the historian can.

References

Blaha, Michael, and William Premerlani. 1998. *Object-Oriented Modeling and Design for Database Applications*. Prentice Hall.

Celko, Joe. 2012. *Joe Celko's Trees and Hierarchies in SQL for Smarties*. 2nd ed. Morgan Kaufmann.

Chen, Peter Pin-Shan. 1976. "The Entity-Relationship Model: Toward a Unified View of Data." *ACM Transactions on Database Systems* 1 (1): 9–36. https://doi.org/10.1145/320434.320440.

Codd, E. F. 1970. "A Relational Model of Data for Large Shared Data Banks." *Communications of the ACM* 13 (6): 377–87. https://doi.org/10.1145/362384.362685.

Inmon, W. H. 2005. *Building the Data Warehouse*. 4th ed. Wiley.

Jacobson, Ivar, Magnus Christerson, Patrik Jonsson, and Gunnar Övergaard. 1992. *Object-Oriented Software Engineering: A Use Case Driven Approach*. Addison-Wesley.

Kent, William. 1983. "A Simple Guide to Five Normal Forms in Relational Database Theory." *Communications of the ACM* 26 (2): 120–25. https://doi.org/10.1145/358024.358054.

Kimball, Ralph, and Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3rd ed. Wiley.

Kleppmann, Martin. 2017. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.

Larman, Craig. 2004. *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. 3rd ed. Prentice Hall.

Rossetti, Manuel D. 1999. *Transit Integrated Monitoring System*. IDEA Program Final Report TRANSIT-19. Transportation Research Board, National Research Council.

Rossetti, Manuel D. 2025. *Simulation Modeling Using the Kotlin Simulation Library (KSL)*. Https://rossetti.github.io/KSLBook/.

Rumbaugh, James, Michael Blaha, William Premerlani, Frederick Eddy, and William Lorensen. 1991. *Object-Oriented Modeling and Design*. Prentice Hall.

Sadalage, Pramod J., and Martin Fowler. 2012. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley.

Starr, Leon. 1996. *How to Build Shlaer-Mellor Object Models*. Yourdon Press.

The PostgreSQL Global Development Group. n.d. *PostgreSQL Documentation*. Https://www.postgresql.org/docs/.

Wickham, Hadley. 2014. "Tidy Data." *Journal of Statistical Software* 59 (10): 1–23. https://doi.org/10.18637/jss.v059.i10.